TEMCO LEARN

T3000 API & Protocol

1.0 · 84 pages · built Nov 13, 2026

1. T3000 API & Protocol
2. API Reference Overview
3. REST API
4. WebSocket API
5. Modbus Protocol
6. Data Points API
7. Device Management API
8. Control & Automation API
9. Trend Logging API
10. Events
11. Generic Tables API
12. Database Management API
13. System & Utilities API
14. Developer Tools API
15. BACnet Commands Reference
16. Data Structures Reference
17. Device Settings (READ_SETTING_COMMAND)
18. Building Platform Overview
19. Control Messages Index
20. Graphics Messages: GET_INITIAL_DATA & LOAD_GRAPHIC_ENTRY
21. Message: GET_PANELS_LIST
22. Message: GET_PANEL_DATA
23. Message: GET_ENTRIES
24. Message 5: GET_PANEL_RANGE_INFO
25. Message 8: OPEN_ENTRY_EDIT_WINDOW
26. Message 9: SAVE_IMAGE
27. Message 10: SAVE_LIBRARY_DATA
28. Message 11: DELETE_IMAGE
29. Message 12: GET_SELECTED_DEVICE_INFO
30. Message 13: BIND_DEVICE
31. Message 14: SAVE_NEW_LIBRARY_DATA
32. Message 15: LOGGING_DATA
33. Message 17: GET_WEBVIEW_LIST
34. Message: SAVE_GRAPHIC_DATA
35. Message: UPDATE_ENTRY
36. Message: UPDATE_WEBVIEW_LIST
37. Installation Guide
38. Configuration Guide
39. Quick Start Overview
40. Inputs
41. Outputs
42. PID Loops
43. Programs
44. Variables
45. Connecting Devices
46. Device Configuration
47. Device Monitoring
48. Device Troubleshooting
49. LAN Scanner — UDP Device Discovery
50. Alarms
51. Graphics
52. Holidays
53. Schedules
54. Settings Field Mapping: C++ ↔ Frontend
55. Trend Center Unified Design
56. TrendLog Chart X-Axis Design
57. Trend Logs
58. Best Practices
59. Frequently Asked Questions
60. Performance Tuning
61. Troubleshooting Guide
62. Centralized Database Design
63. Centralized Database: Multi-PC Architecture (v4 Final)
64. Centralized Logging Design
65. Device Panel Loading
66. Flow Logging Architecture — Design Document
67. T3000 Mobile Architecture Guide
68. Multi-PC Data Flow Details
69. T3000 WebView System Architecture
70. T3 Firmware — LVGL Display Architecture
71. Device Interface Deployment via BACnet — Design
72. Import from Device — Design
73. .EEZ-Project JSON Format — Design Document
74. Static Resource Copy — Build-Time Asset Pipeline
75. LVGL WASM Browser Build Migration
76. Status Bar Component
77. Shared Center DB — Overview
78. SQL Server Express 2022 Setup for T3000 (Detailed Step-by-Step)
79. T3000 Center DB Configuration
80. Tstat LCD System — Documentation Index
81. LCD Screen Designer
82. JSON Schema — screenDefinition.json

T3000 API & Protocol

The developer reference for T3000: the REST and WebSocket API, the Modbus and BACnet protocol, the webview message set, the data structures behind every point, and the semantic tagging layer.

Contents

- API Reference Overview
- REST API
- WebSocket API
- Modbus Protocol
- Data Points API
- Device Management API
- Control & Automation API
- Trend Logging API
- Events
- Generic Tables API
- Database Management API
- System & Utilities API
- Developer Tools API
- BACnet Commands Reference
- Data Structures Reference
- Device Settings (READ_SETTING_COMMAND)
- Building Platform Overview
  - Control Messages Index
  - Graphics Messages: GET_INITIAL_DATA & LOAD_GRAPHIC_ENTRY
  - Message: GET_PANELS_LIST
  - Message: GET_PANEL_DATA
  - Message: GET_ENTRIES
  - Message 5: GET_PANEL_RANGE_INFO
  - Message 8: OPEN_ENTRY_EDIT_WINDOW
  - Message 9: SAVE_IMAGE
  - Message 10: SAVE_LIBRARY_DATA
  - Message 11: DELETE_IMAGE
  - Message 12: GET_SELECTED_DEVICE_INFO
  - Message 13: BIND_DEVICE
  - Message 14: SAVE_NEW_LIBRARY_DATA
  - Message 15: LOGGING_DATA
  - Message 17: GET_WEBVIEW_LIST
  - Message: SAVE_GRAPHIC_DATA
  - Message: UPDATE_ENTRY
  - Message: UPDATE_WEBVIEW_LIST
- Installation Guide
- Configuration Guide
- Quick Start Overview
- Inputs
- Outputs
- PID Loops
- Programs
- Variables

- Connecting Devices
- Device Configuration
- Device Monitoring
- Device Troubleshooting
- LAN Scanner — UDP Device Discovery
- Alarms
- Graphics
- Holidays
- Schedules
- Settings Field Mapping: C++ ↔ Frontend
- Trend Center Unified Design
- TrendLog Chart X-Axis Design
- Trend Logs
- Best Practices
- Frequently Asked Questions
- Performance Tuning
- Troubleshooting Guide
- Centralized Database Design
- Centralized Database: Multi-PC Architecture (v4 Final)
- Centralized Logging Design
- Device Panel Loading
- Flow Logging Architecture — Design Document
- T3000 Mobile Architecture Guide
- Multi-PC Data Flow Details
- T3000 WebView System Architecture
- T3 Firmware — LVGL Display Architecture
- Device Interface Deployment via BACnet — Design
- Import from Device — Design
- .EEZ-Project JSON Format — Design Document
- Static Resource Copy — Build-Time Asset Pipeline
- LVGL WASM Browser Build Migration
- Status Bar Component
- Shared Center DB — Overview
- SQL Server Express 2022 Setup for T3000 (Detailed Step-by-Step)
- T3000 Center DB Configuration
- Tstat LCD System — Documentation Index
- LCD Screen Designer
- JSON Schema — screenDefinition.json
- System Overview
- Widget Types

API Reference Overview

Introduction

The T3000 WebView provides a comprehensive REST API for building automation and device management. This overview lists all **215+ available endpoints** organized by category.

Base URL: `http://localhost:9103`

Quick Navigation

TEXT

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ 🔳 Device Mgmt       │ 📊 Data Points       │ ⚙️ Control & Auto    │
│ 15 endpoints         │ 60 endpoints         │ 40 endpoints         │
│ ──────────────       │ ──────────────       │ ──────────────       │
│ • List devices       │ • Read inputs        │ • Manage programs    │
│ • CRUD operations    │ • Control outputs    │ • Configure PIDs     │
│ • Status checks      │ • Batch updates      │ • Set schedules      │
└──────────────────────┴──────────────────────┴──────────────────────┘

┌──────────────────────┬──────────────────────┬──────────────────────┐
│ 📈 Trend Logging     │ 📁 Generic Tables    │ 💾 Database Mgmt     │
│ 25 endpoints         │ 30 endpoints         │ 20 endpoints         │
│ ──────────────       │ ──────────────       │ ──────────────       │
│ • Historical data    │ • Arrays/Users       │ • Settings store     │
│ • Real-time logs     │ • Graphics/Alarms    │ • Maintenance        │
│ • Export CSV         │ • Custom units       │ • Partitioning       │
└──────────────────────┴──────────────────────┴──────────────────────┘

┌──────────────────────┬──────────────────────┬──────────────────────┐
│ 🛠️ Developer Tools   │ 🔧 System Utils      │ 🌐 WebSocket         │
│ 10 endpoints         │ 15 endpoints         │ Real-time comms      │
│ ──────────────       │ ──────────────       │ ──────────────       │
│ • File browser       │ • Authentication     │ • Live updates       │
│ • DB viewer          │ • Health checks      │ • Device events      │
│ • Logs               │ • File uploads       │ • Notifications      │
└──────────────────────┴──────────────────────┴──────────────────────┘
```

Complete API Index

🔳 Device Management (15 endpoints)

TEXT

```
GET    /api/t3_device/devices ............................... List all devices
GET    /api/t3_device/devices/:id ........................... Get device by ID
POST   /api/t3_device/devices ............................... Create new device
PUT    /api/t3_device/devices/:id ........................... Update device
DELETE /api/t3_device/devices/:id ........................... Delete device
GET    /api/t3_device/devices/count ......................... Get device count
GET    /api/t3_device/devices/:id/capabilities ............... Get capabilities
POST   /api/t3_device/devices/:id/refresh .................... Refresh from hardware
GET    /api/t3_device/devices/:id/connection-status ........... Check connection
```

Quick Example:

HTTP

```
GET /api/t3_device/devices
→ Returns: [{ id: 1, serial_number: 123456, product_name: "T3-BB", online: true }]
```

📊 Data Points (60 endpoints)

Inputs (20 endpoints)

```
GET    /api/t3_device/devices/:id/input-points ..... Get all inputs
GET    /api/t3_device/inputs/:objectinstance ..... Get single input
PUT    /api/t3_device/inputs/:objectinstance ..... Update input
POST   /api/t3_device/devices/:id/inputs/refresh ..... Refresh inputs
GET    /api/t3_device/inputs/:objectinstance/history ..... Input history
```

Outputs (20 endpoints)

```
GET    /api/t3_device/devices/:id/output-points ..... Get all outputs
GET    /api/t3_device/outputs/:objectinstance ..... Get single output
PUT    /api/t3_device/outputs/:objectinstance ..... Update output
PUT    /api/t3_device/outputs/batch ..... Batch update outputs
POST   /api/t3_device/devices/:id/outputs/refresh ..... Refresh outputs
GET    /api/t3_device/outputs/:objectinstance/history ..... Output history
```

Variables (20 endpoints)

```
GET    /api/t3_device/devices/:id/variable-points ..... Get all variables
GET    /api/t3_device/variables/:objectinstance ..... Get single variable
PUT    /api/t3_device/variables/:objectinstance ..... Update variable
PUT    /api/t3_device/variables/batch ..... Batch update variables
POST   /api/t3_device/devices/:id/variables/refresh ..... Refresh variables
GET    /api/t3_device/variables/:objectinstance/history ..... Variable history
```

Quick Example:

```
PUT /api/t3_device/outputs/5000000
{ "value": 75.0, "auto_manual": 1 }
→ Sets output to 75% in manual mode
```

⚙ Control & Automation (40 endpoints)

Programs (10 endpoints)

```
GET    /api/t3_device/devices/:id/programs ..... Get all programs
GET    /api/t3_device/programs/:id ..... Get single program
POST   /api/t3_device/programs ..... Create program
PUT    /api/t3_device/programs/:id ..... Update program
DELETE /api/t3_device/programs/:id ..... Delete program
POST   /api/t3_device/programs/:id/compile ..... Compile program
POST   /api/t3_device/programs/:id/start ..... Start program
POST   /api/t3_device/programs/:id/stop ..... Stop program
```

Schedules (10 endpoints)

```
GET    /api/t3_device/devices/:id/schedules ................. Get all schedules
GET    /api/t3_device/schedules/:id ......................... Get single schedule
POST   /api/t3_device/schedules ............................. Create schedule
PUT    /api/t3_device/schedules/:id ......................... Update schedule
DELETE /api/t3_device/schedules/:id ......................... Delete schedule
POST   /api/t3_device/schedules/:id/enable .................. Enable schedule
POST   /api/t3_device/schedules/:id/disable ................. Disable schedule
```

PIDs (10 endpoints)

```
GET    /api/t3_device/devices/:id/pids ...................... Get all PIDs
GET    /api/t3_device/pids/:id .............................. Get single PID
POST   /api/t3_device/pids .................................. Create PID
PUT    /api/t3_device/pids/:id .............................. Update PID tuning
DELETE /api/t3_device/pids/:id .............................. Delete PID
POST   /api/t3_device/pids/:id/autotune ..................... Auto-tune PID
```

Holidays (5 endpoints)

```
GET    /api/t3_device/devices/:id/holidays ................... Get all holidays
GET    /api/t3_device/holidays/:id ........................... Get single holiday
POST   /api/t3_device/holidays ............................... Create holiday
PUT    /api/t3_device/holidays/:id ........................... Update holiday
DELETE /api/t3_device/holidays/:id ........................... Delete holiday
```

Annual Routines (5 endpoints)

```
GET    /api/t3_device/devices/:id/annual-routines ............ Get all routines
GET    /api/t3_device/annual-routines/:id .................... Get single routine
POST   /api/t3_device/annual-routines ........................ Create routine
PUT    /api/t3_device/annual-routines/:id .................... Update routine
DELETE /api/t3_device/annual-routines/:id .................... Delete routine
```

📈 Trend Logging (25 endpoints)

```
GET    /api/t3_device/devices/:id/trendlogs .................. Get all trendlogs
GET    /api/t3_device/trendlogs/:id .......................... Get trendlog config
POST   /api/t3_device/trendlogs .............................. Create trendlog
PUT    /api/t3_device/trendlogs/:id .......................... Update trendlog
DELETE /api/t3_device/trendlogs/:id .......................... Delete trendlog
GET    /api/t3_device/trendlogs/:id/data ..................... Get historical data
POST   /api/t3_device/trendlogs/:id/clear .................... Clear trendlog data
GET    /api/t3_device/trendlogs/:id/export ................... Export to CSV
POST   /api/t3_device/trendlogs/:id/start .................... Start logging
POST   /api/t3_device/trendlogs/:id/stop ..................... Stop logging
GET    /api/t3_device/trendlogs/:id/stats .................... Get statistics
```

Quick Example:

```http
GET /api/t3_device/trendlogs/1/data?start=2026-01-01&end=2026-01-09
→ Returns: { data: [{ timestamp: "2026-01-09T08:00:00Z", value: 72.5 }] }
```

📁 Generic Tables (30 endpoints)

```
GET    /api/t3_device/:table ................................ Get table records
GET    /api/t3_device/:table/count ........................... Get record count
GET    /api/t3_device/:table/:id ............................. Get single record
PUT    /api/t3_device/:table/:id ............................. Update record
POST   /api/t3_device/:table ................................. Create record
DELETE /api/t3_device/:table/:id ............................. Delete record
```

Supported Tables:

- `arrays` - Array configuration
- `users` - User management
- `graphics` - Graphics screens
- `alarms` - Alarm definitions
- `units` - Unit conversions
- `conversions` - Custom conversions
- `custom_units` - Custom unit definitions

Quick Example:

```http
GET /api/t3_device/users?device_id=1
→ Returns: [{ id: 1, username: "admin", access_level: 255 }]
```

💾 Database Management (20 endpoints)

```
GET    /api/db_management/settings ........................... Get all settings
POST   /api/db_management/settings ........................... Save setting
DELETE /api/db_management/settings/:key ...................... Delete setting
GET    /api/database/stats ................................... Database statistics
POST   /api/db_management/tools/vacuum ....................... Vacuum database
POST   /api/database/cleanup/old ............................. Clean old files
GET    /api/db_management/partitions ......................... Get partitions
POST   /api/db_management/partitions ......................... Create partition
DELETE /api/db_management/partitions/:name ................... Drop partition
GET    /api/database/config .................................. Get DB config
PUT    /api/database/config .................................. Update DB config
GET    /api/database/backup .................................. Create backup
POST   /api/database/restore ................................. Restore from backup
```

🛠️ Developer Tools (10 endpoints)

```
GET    /api/develop/files/list ............................. List files
GET    /api/develop/files/read ............................. Read file
POST   /api/develop/files/write ............................ Write file
DELETE /api/develop/files/delete ........................... Delete file
GET    /api/develop/database/list .......................... List databases
POST   /api/develop/database/query ......................... Execute SQL query
GET    /api/develop/database/schema ........................ Get schema
GET    /api/develop/logs/get ............................... Get system logs
POST   /api/develop/logs/clear ............................. Clear logs
GET    /api/develop/logs/download .......................... Download log file
```

Quick Example:

```
POST /api/develop/database/query
{ "sql": "SELECT * FROM DEVICES LIMIT 10" }
→ Returns: { columns: ["id", "serial_number"], rows: [[1, 123456]] }
```

🔧 System & Utilities (15 endpoints)

```
GET    /api/health ......................................... Health check
GET    /api/version ........................................ API version
POST   /api/login .......................................... User login
POST   /api/logout ......................................... User logout
GET    /api/session ........................................ Check session
POST   /api/upload ......................................... Upload file
GET    /api/uploads/:filename .............................. Download file
DELETE /api/uploads/:filename .............................. Delete file
GET    /api/modbus/registers ............................... Read Modbus registers
POST   /api/modbus/write ................................... Write Modbus register
POST   /api/ffi/call ....................................... Call BACnet FFI
GET    /api/sync/metadata .................................. Get sync metadata
POST   /api/sync/start ..................................... Start sync
POST   /api/sync/stop ...................................... Stop sync
GET    /api/sync/status .................................... Sync status
```

Quick Example:

```
POST /api/login
{ "username": "admin", "password": "password" }
→ Returns: { success: true, session_token: "abc123..." }
```

🌐 WebSocket (Real-time Communication)

```
WS     /ws ................................................. WebSocket connection
```

Message Types:

- `1` - SELECT_PANEL: Select active device

- `2` - UPDATE_POINT: Point value updated
- `3` - DEVICE_STATUS: Device online/offline
- `4` - REFRESH_COMPLETE: Refresh operation complete

Quick Example:

```javascript
const ws = new WebSocket('ws://localhost:9103/ws');
ws.send(JSON.stringify({ action: 1, panelId: 5, serialNumber: 123456 }));
```

Common Workflows

Workflow 1: Device Setup & Monitoring

```
1. GET  /api/t3_device/devices              → Get all devices
2. GET  /api/t3_device/devices/1            → Get device details
3. GET  /api/t3_device/devices/1/input-points   → Read inputs
4. GET  /api/t3_device/devices/1/output-points  → Read outputs
5. WS   /ws (subscribe)                     → Live updates
```

Workflow 2: Control Operations

```
1. GET  /api/t3_device/outputs/5000000      → Read current value
2. PUT  /api/t3_device/outputs/5000000      → Write new value
   { "value": 75.0, "auto_manual": 1 }
3. GET  /api/t3_device/outputs/5000000      → Verify change
```

Workflow 3: Historical Data Analysis

```
1. GET  /api/t3_device/devices/1/trendlogs  → List trendlogs
2. GET  /api/t3_device/trendlogs/1/data     → Query data
   ?start=2026-01-01&end=2026-01-09
3. GET  /api/t3_device/trendlogs/1/export   → Export CSV
```

Workflow 4: Batch Control

```
1. PUT  /api/t3_device/outputs/batch        → Update multiple outputs
   {
     "updates": [
       { "objectinstance": 5000000, "value": 50.0 },
       { "objectinstance": 5000001, "value": 75.0 }
     ]
   }
```

Authentication

Currently, the API uses basic session-based authentication.

TYPESCRIPT

```typescript
// Login
const response = await fetch('http://localhost:9103/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'admin',
    password: 'password'
  })
});
```

Response Format

Success Response

JSON

```json
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully"
}
```

Error Response

JSON

```json
{
  "error": "Device not found",
  "code": "DEVICE_NOT_FOUND",
  "status": 404
}
```

Common HTTP Status Codes

- `200 OK` - Request successful
- `201 Created` - Resource created
- `400 Bad Request` - Invalid parameters
- `401 Unauthorized` - Authentication required
- `404 Not Found` - Resource not found
- `500 Internal Server Error` - Server error
- `503 Service Unavailable` - Database unavailable

CORS

CORS is enabled for all origins in development. Configure for production:

RUST

```rust
CorsLayer::new()
    .allow_origin("https://your-domain.com")
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
```

Complete Endpoint Reference

Quick Reference Table

All **215+ endpoints** organized by HTTP method and category:

TEXT

Method	Endpoint Pattern	Auth	Category
GET	/api/t3_device/devices	✓	Device
GET	/api/t3_device/devices/:id	✓	Device
POST	/api/t3_device/devices	✓	Device
PUT	/api/t3_device/devices/:id	✓	Device
DELETE	/api/t3_device/devices/:id	✓	Device
GET	/api/t3_device/devices/count	✓	Device
GET	/api/t3_device/devices/:id/capabilities	✓	Device
POST	/api/t3_device/devices/:id/refresh	✓	Device
GET	/api/t3_device/devices/:id/input-points	✓	Data Points
GET	/api/t3_device/inputs/:objectinstance	✓	Data Points
PUT	/api/t3_device/inputs/:objectinstance	✓	Data Points
POST	/api/t3_device/devices/:id/inputs/refresh	✓	Data Points
GET	/api/t3_device/outputs/:objectinstance	✓	Data Points
PUT	/api/t3_device/outputs/:objectinstance	✓	Data Points
PUT	/api/t3_device/outputs/batch	✓	Data Points
POST	/api/t3_device/devices/:id/outputs/refresh	✓	Data Points
GET	/api/t3_device/variables/:objectinstance	✓	Data Points
PUT	/api/t3_device/variables/:objectinstance	✓	Data Points
PUT	/api/t3_device/variables/batch	✓	Data Points
GET	/api/t3_device/programs	✓	Control
POST	/api/t3_device/programs	✓	Control
PUT	/api/t3_device/programs/:id	✓	Control
DELETE	/api/t3_device/programs/:id	✓	Control
GET	/api/t3_device/schedules	✓	Control
POST	/api/t3_device/schedules	✓	Control
PUT	/api/t3_device/schedules/:id	✓	Control
DELETE	/api/t3_device/schedules/:id	✓	Control
GET	/api/t3_device/pids	✓	Control
PUT	/api/t3_device/pids/:id	✓	Control
GET	/api/t3_device/holidays	✓	Control
PUT	/api/t3_device/holidays/:id	✓	Control
GET	/api/t3_device/trendlogs	✓	Trends
GET	/api/t3_device/trendlogs/:id/data	✓	Trends
GET	/api/t3_device/trendlogs/:id/export	✓	Trends
POST	/api/t3_device/trendlogs/:id/clear	✓	Trends
PUT	/api/t3_device/trendlogs/:id	✓	Trends
GET	/api/t3_device/:table	✓	Tables
GET	/api/t3_device/:table/count	✓	Tables
GET	/api/t3_device/users	✓	Tables
POST	/api/t3_device/users	✓	Tables
PUT	/api/t3_device/users/:id	✓	Tables
DELETE	/api/t3_device/users/:id	✓	Tables
GET	/api/db_management/settings	✓	Database
POST	/api/db_management/settings	✓	Database
DELETE	/api/db_management/settings/:key	✓	Database
GET	/api/database/stats	✓	Database
POST	/api/db_management/tools/vacuum	✓	Database
GET	/api/db_management/partitions	✓	Database
GET	/api/develop/files/list	✓	Developer

```
| GET    | /api/develop/files/read                | ✓    | Developer    |
| POST   | /api/develop/database/query            | ✓    | Developer    |
| GET    | /api/develop/database/schema           | ✓    | Developer    |
| GET    | /api/develop/logs/get                  | ✓    | Developer    |
├────────┼────────────────────────────────────────┼──────┼──────────────┤
| GET    | /api/health                            | -    | System       |
| GET    | /api/version                           | -    | System       |
| POST   | /api/login                             | -    | Auth         |
| POST   | /api/logout                            | ✓    | Auth         |
| GET    | /api/session                           | ✓    | Auth         |
| POST   | /api/upload                            | ✓    | System       |
| GET    | /api/uploads/:filename                 | ✓    | System       |
| DELETE | /api/uploads/:filename                 | ✓    | System       |
| GET    | /api/modbus/registers                  | ✓    | System       |
| POST   | /api/modbus/write                      | ✓    | System       |
| POST   | /api/ffi/call                          | ✓    | System       |
├────────┼────────────────────────────────────────┼──────┼──────────────┤
| WS     | /ws                                    | ✓    | WebSocket    |
└────────┴────────────────────────────────────────┴──────┴──────────────┘


Legend: ✓ = Authentication required  |  - = Public endpoint
```

API Coverage Matrix

Feature availability by resource type:

TEXT

```
──────────────────────────────────────────────────────────────────
Resource          | List | Get | Create | Update | Delete | Special
──────────────────┼──────┼─────┼────────┼────────┼────────┼─────────
Devices           |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | Refresh
Inputs            |  ✓   |  ✓  |   -    |   ✓    |   -    | Refresh
Outputs           |  ✓   |  ✓  |   -    |   ✓    |   -    | Batch
Variables         |  ✓   |  ✓  |   -    |   ✓    |   -    | Batch
Programs          |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | Compile
Schedules         |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | Enable
PIDs              |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | Autotune
Holidays          |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | -
Trendlogs         |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | Export
Users             |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | -
Settings          |  ✓   |  ✓  |   ✓    |   ✓    |   ✓    | -
──────────────────────────────────────────────────────────────────


Special Operations:
• Batch Updates      → Outputs, Variables
• Refresh from HW    → Inputs, Outputs, Variables, Devices
• Export to CSV      → Trendlogs
• Real-time Stream   → WebSocket (/ws)
• SQL Queries        → Developer Tools
• File Management    → Developer Tools
──────────────────────────────────────────────────────────────────
```

API Architecture

The T3000 WebView API is built with:

- **Framework**: Axum (Rust) - High-performance async web framework
- **Database**: SQLite via SeaORM - Type-safe ORM
- **WebSocket**: Real-time bidirectional communication
- **FFI**: BACnet protocol via C++ bridge

Request/Response Lifecycle

```
TEXT
```

```
Client Request
     |
     ▼
┌──────────────────┐
│  Axum Router     │ ← Route matching (/api/t3_device/*)
└──────────┬───────┘
           |
           ▼
┌──────────────────┐
│  Middleware      │ ← CORS, Auth, Logging
└──────────┬───────┘
           |
           ▼
┌──────────────────┐
│  Handler Fn      │ ← Business logic (routes.rs)
└──────────┬───────┘
           |
           ▼
┌──────────────────┐
│  Database/FFI    │ ← Data: SQLite | Hardware: BACnet FFI
└──────────┬───────┘
           |
           ▼
┌──────────────────┐
│  JSON Response   │ ← Serialized data (serde_json)
└──────────────────┘
```

Content Types

Request Headers:

```
HTTP
```

```
Content-Type: application/json
Accept: application/json
```

Response Headers:

```
HTTP
```

```
Content-Type: application/json; charset=utf-8
Access-Control-Allow-Origin: *
```

Rate Limiting

Currently no rate limiting is enforced. For production, implement:

```
RUST
```

```
use tower::limit::RateLimitLayer;

app.layer(RateLimitLayer::new(100, Duration::from_secs(60)))
```

WebSocket Connection

For real-time updates, connect via WebSocket:

TYPESCRIPT

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  console.log('Connected to T3000 WebView');
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  console.log('Received:', message);
};

// Send control message
ws.send(JSON.stringify({
  action: 1,  // SELECT_PANEL
  panelId: 5,
  serialNumber: 123456
}));
```

Error Handling

Client-Side Pattern:

TYPESCRIPT

```typescript
async function apiCall(endpoint: string, options?: RequestInit) {
  try {
    const response = await fetch(`http://localhost:9103${endpoint}`, options);

    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.message || `HTTP ${response.status}`);
    }

    return await response.json();
  } catch (error) {
    console.error('API Error:', error);
    throw error;
  }
}
```

Server-Side Error Types:

RUST

```rust
pub enum ApiError {
    DatabaseError(String),
    DeviceOffline,
    InvalidRequest(String),
    NotFound(String),
    Unauthorized,
    InternalError(String),
}

impl IntoResponse for ApiError {
    fn into_response(self) -> Response {
        let (status, message) = match self {
            ApiError::NotFound(msg) => (StatusCode::NOT_FOUND, msg),
            ApiError::Unauthorized => (StatusCode::UNAUTHORIZED, "Unauthorized".into()),
            // ...
        };

        (status, Json(json!({ "error": message }))).into_response()
    }
}
```

Performance Considerations

Pagination:

HTTP

```http
GET /api/t3_device/devices?page=1&per_page=50
```

Field Selection (future):

HTTP

```http
GET /api/t3_device/devices/1?fields=id,serial_number,ip_address
```

Batch Operations:

HTTP

```http
POST /api/t3_device/inputs/batch
{
  "updates": [
    { "objectinstance": 3000000, "value": 72.5 },
    { "objectinstance": 3000001, "value": 65.0 }
  ]
}
```

API Versioning

Current API is v1 (implicit). Future versions:

TEXT

```
/api/v2/t3_device/devices
```

Testing Endpoints

Using curl:

```shell
## Get devices
curl http://localhost:9103/api/t3_device/devices

## Create device
curl -X POST http://localhost:9103/api/t3_device/devices \
  -H "Content-Type: application/json" \
  -d '{"serial_number": 123456, "product_name": "T3-BB"}'
```

Using JavaScript:

```javascript
// GET request
fetch('http://localhost:9103/api/t3_device/devices')
  .then(res => res.json())
  .then(data => console.log(data));

// POST request
fetch('http://localhost:9103/api/t3_device/devices', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    serial_number: 123456,
    product_name: 'T3-BB'
  })
})
  .then(res => res.json())
  .then(data => console.log(data));
```

Database Schema Reference

Key tables accessed by API:

- `DEVICES` - Device registry
- `INPUTS` - Analog/Binary inputs
- `OUTPUTS` - Analog/Binary outputs
- `VARIABLES` - Analog values
- `PROGRAMS` - Control programs
- `SCHEDULES` - Time schedules
- `TRENDLOGS` - Trendlog configuration
- `TRENDLOG_DATA` - Historical data

File Locations

- **Routes**: `api/src/t3_device/routes.rs`
- **Services**: `api/src/t3_device/*_service.rs`
- **State**: `api/src/app_state.rs`
- **Server**: `api/src/server.rs`

Development Setup

```shell
## Start backend
cd api
cargo run

## Backend runs on http://localhost:9103

## Test endpoint
curl http://localhost:9103/api/health
```

Production Deployment

1. Build optimized binary:

```shell
cargo build --release
```

2. Configure environment:

```env
DATABASE_URL=./production.db
PORT=9103
RUST_LOG=info
```

3. Run as service:

```shell
./target/release/t3000-api
```

REST API

What is the REST API?

The REST API allows you to access T3000 device data through web requests. Think of it like asking questions to your building automation system and getting answers back.

When Would I Use This?

Common Uses:

- View live temperature readings from anywhere
- Check if HVAC equipment is running
- Update setpoints remotely
- Build custom dashboards
- Integrate with other software

How Does It Work?

1. **You send a request** (like "show me all devices")
2. **T3000 processes it** and gathers the information
3. **You receive the data** in an easy-to-read format

Example: Get All Devices

What you want: See a list of all connected devices

How to do it:

```
TEXT
Open your web browser and visit:
http://localhost:9103/api/t3_device/devices
```

What you get back: A list showing each device's name, location, and status.

Getting Started

Requirements:

- T3000 API server must be running
- Know the server address (usually `localhost` if on same computer)
- Use port `9103` (default)

Testing the API:

1. Start the T3000 API server
2. Open a web browser
3. Type in the address bar: `http://localhost:9103/api/t3_device/devices`
4. Press Enter
5. You should see a list of your devices

Common Operations

See all devices:

```
TEXT
http://localhost:9103/api/t3_device/devices
```

View device inputs (sensors):

```
TEXT
http://localhost:9103/api/t3_device/devices/YOUR_DEVICE_ID/inputs
```

View device outputs (controls):

```
TEXT
http://localhost:9103/api/t3_device/devices/YOUR_DEVICE_ID/outputs
```

Replace `YOUR_DEVICE_ID` with your actual device number.

Need Help?

- **Can't connect?** Make sure the API server is running
- **Getting errors?** Check that the device ID is correct
- **Want to do more?** Switch to the "Technical" tab for code examples

REST API Technical Reference

HTTP REST API for T3000 device data access and control.

Base URL

```
TEXT
http://localhost:9103/api
```

API Endpoints

Device Management

List all devices:

```
HTTP
GET /t3_device/devices
```

Response:

```
JSON
[
  {
    "serial": 237219,
    "name": "Controller_01",
    "online": true,
    "product": "T3-BB",
    "address": "192.168.1.100"
  }
]
```

Get device details:

```
HTTP
GET /t3_device/devices/{serial}
```

Get table data:

```
HTTP
GET /t3_device/devices/{serial}/table/{table}
```

Available tables: `INPUTS`, `OUTPUTS`, `VARIABLES`, `PROGRAMS`, `SCHEDULES`, `HOLIDAYS`, `CONTROLLERS`, `SCREENS`, `MONITORS`

Data Points

Get all inputs:

```HTTP
GET /t3_device/devices/{serial}/inputs
```

Response:

```JSON
{
  "points": [
    {
      "index": 1,
      "label": "Zone_Temp",
      "value": 72.5,
      "units": "DEG_F",
      "range": "0-100",
      "calibration": 0.0,
      "filter": 3,
      "status": "OK"
    }
  ]
}
```

Get all outputs:

```HTTP
GET /t3_device/devices/{serial}/outputs
```

Batch save inputs:

```HTTP
POST /t3_device/devices/{serial}/batch_save_inputs
Content-Type: application/json

{
  "points": [
    {
      "index": 1,
      "label": "Zone_Temp",
      "value": 72.5,
      "units": "DEG_F"
    }
  ]
}
```

FFI (Foreign Function Interface) Calls

Execute FFI function:

```HTTP
POST /t3_device/ffi/call
Content-Type: application/json

{
  "action": 17,
  "serial": 237219,
  "panel": 0,
  "parameter": 0
}
```

Common action codes:

- `17` : Read inputs
- `18` : Read outputs
- `19` : Read variables
- `20` : Write output
- `1000` : Scan network

Authentication

Currently no authentication required (development mode only).

Production recommendations:

- Implement API key authentication
- Use OAuth2 for user-based access
- Enable HTTPS/TLS encryption

Response Format

All responses are JSON format with standard HTTP status codes:

Success codes:

- `200 OK` : Request successful
- `201 Created` : Resource created
- `204 No Content` : Success with no response body

Error codes:

- `400 Bad Request` : Invalid parameters
- `404 Not Found` : Resource not found
- `500 Internal Server Error` : Server error
- `503 Service Unavailable` : Database locked or service down

Error response format:

```JSON
{
  "error": "Device not found",
  "code": 404,
  "details": "Serial number 999999 does not exist"
}
```

Rate Limiting

Current limits (development):

- 100 requests per minute per IP
- 10 concurrent connections
- No burst allowance

Exceeding limits:

```JSON
{
  "error": "Rate limit exceeded",
  "code": 429,
  "retry_after": 60
}
```

Code Examples

JavaScript/TypeScript

TYPESCRIPT

```typescript
// Fetch all devices
async function getDevices() {
  const response = await fetch('http://localhost:9103/api/t3_device/devices');
  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }
  const devices = await response.json();
  return devices;
}

// Get device inputs
async function getInputs(serial: number) {
  const url = `http://localhost:9103/api/t3_device/devices/${serial}/inputs`;
  const response = await fetch(url);
  const data = await response.json();
  return data.points;
}

// Update input point
async function updateInput(serial: number, index: number, label: string) {
  const url = `http://localhost:9103/api/t3_device/devices/${serial}/batch_save_inputs`;
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      points: [{ index, label }]
    })
  });
  return response.json();
}
```

Python

```python
import requests

## Get all devices
def get_devices():
    response = requests.get('http://localhost:9103/api/t3_device/devices')
    response.raise_for_status()
    return response.json()

## Get device inputs
def get_inputs(serial):
    url = f'http://localhost:9103/api/t3_device/devices/{serial}/inputs'
    response = requests.get(url)
    response.raise_for_status()
    return response.json()['points']

## Update input
def update_input(serial, index, label):
    url = f'http://localhost:9103/api/t3_device/devices/{serial}/batch_save_inputs'
    payload = {
        'points': [{'index': index, 'label': label}]
    }
    response = requests.post(url, json=payload)
    response.raise_for_status()
    return response.json()
```

cURL

```shell
## List devices
curl http://localhost:9103/api/t3_device/devices

## Get inputs
curl http://localhost:9103/api/t3_device/devices/237219/inputs

## Update input (POST)
curl -X POST http://localhost:9103/api/t3_device/devices/237219/batch_save_inputs \
  -H "Content-Type: application/json" \
  -d '{"points":[{"index":1,"label":"Updated_Label"}]}'
```

Best Practices

1. **Error Handling**
   - Always check response status codes
   - Implement retry logic with exponential backoff
   - Log errors for debugging
2. **Performance**
   - Use batch operations when updating multiple points
   - Cache responses appropriately (consider stale-while-revalidate)
   - Implement request debouncing for user input
3. **Security**
   - Use HTTPS in production
   - Validate and sanitize all inputs
   - Implement proper authentication
   - Don't expose API directly to internet

4. **Monitoring**
    - Track API response times
    - Monitor error rates
    - Set up alerts for service degradation

Next Steps

- WebSocket API - Real-time data updates
- Events - Event system documentation
- Modbus Protocol - Modbus integration

WebSocket API

Real-time bi-directional communication for live data updates.

Connection

```javascript
const ws = new WebSocket('ws://localhost:9103/ws');
```

Messages

Subscribe to Updates

```json
{
  "type": "subscribe",
  "serial": 237219,
  "points": ["IN1", "IN2"]
}
```

Receive Data

```json
{
  "type": "data",
  "point": "IN1",
  "value": 72.5,
  "timestamp": 1704582000
}
```

Best Practices

- Reconnect on disconnect
- Limit subscriptions
- Handle backpressure

WebSocket API

Protocol Specification

Connection Endpoint

```text
ws://localhost:9103/ws
wss://localhost:9103/ws  (TLS)
```

Message Format

All messages are JSON-encoded:

```typescript
interface WebSocketMessage {
  type: string;
  [key: string]: any;
}
```

Client-to-Server Messages

Subscribe

TYPESCRIPT

```typescript
interface SubscribeMessage {
  type: 'subscribe';
  deviceId?: number;
  serial?: number;
  points: string[];
  interval?: number;       // Update interval (ms)
  changeOfValue?: number;  // COV threshold
}

// Example
ws.send(JSON.stringify({
  type: 'subscribe',
  deviceId: 389001,
  points: ['IN1', 'IN2', 'OUT1'],
  changeOfValue: 0.5  // Only send if value changes by 0.5+
}));
```

Unsubscribe

TYPESCRIPT

```typescript
interface UnsubscribeMessage {
  type: 'unsubscribe';
  deviceId?: number;
  points?: string[];  // If omitted, unsubscribe from all
}
```

Write Point

TYPESCRIPT

```typescript
interface WritePointMessage {
  type: 'write';
  deviceId: number;
  point: string;
  value: number;
  priority?: number;  // BACnet priority 1-16
}
```

Server-to-Client Messages

Point Update

TYPESCRIPT

```typescript
interface PointUpdateMessage {
  type: 'point-update';
  deviceId: number;
  point: string;
  value: number;
  timestamp: number;
  quality: 'good' | 'uncertain' | 'bad';
  cov?: boolean;  // True if triggered by COV
}
```

Alarm Notification

TYPESCRIPT

```typescript
interface AlarmMessage {
  type: 'alarm';
  alarmId: string;
  deviceId: number;
  point: string;
  alarmType: 'high-limit' | 'low-limit' | 'fault';
  priority: 'high' | 'medium' | 'low';
  message: string;
  value: number;
  timestamp: number;
}
```

Device Status

TYPESCRIPT

```typescript
interface DeviceStatusMessage {
  type: 'device-status';
  deviceId: number;
  status: 'online' | 'offline' | 'error';
  lastSeen: number;
}
```

Implementation

Robust WebSocket Client

TYPESCRIPT

```typescript
class T3000WebSocketClient {
  private ws: WebSocket | null = null;
  private url: string;
  private reconnectDelay = 1000;
  private maxReconnectDelay = 30000;
  private subscriptions = new Set<string>();

  constructor(url: string) {
    this.url = url;
    this.connect();
  }

  private connect() {
    this.ws = new WebSocket(this.url);

    this.ws.onopen = () => {
      console.log('WebSocket connected');
      this.reconnectDelay = 1000;
      this.resubscribe();
    };

    this.ws.onmessage = (event) => {
      const message = JSON.parse(event.data);
      this.handleMessage(message);
    };

    this.ws.onerror = (error) => {
      console.error('WebSocket error:', error);
    };

    this.ws.onclose = () => {
      console.log('WebSocket closed, reconnecting...');
      setTimeout(() => this.connect(), this.reconnectDelay);
      this.reconnectDelay = Math.min(
        this.reconnectDelay * 2,
        this.maxReconnectDelay
      );
    };
  }

  subscribe(points: string[], deviceId: number) {
    const message = {
      type: 'subscribe',
      deviceId,
      points
    };

    // Store subscription for reconnect
    this.subscriptions.add(JSON.stringify(message));

    this.send(message);
  }

  private resubscribe() {
    for (const sub of this.subscriptions) {
```

```
      this.send(JSON.parse(sub));
    }
  }

  private send(message: any) {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify(message));
    }
  }

  on(event: string, callback: (data: any) => void) {
    // Event emitter implementation
  }
}
```

Server-Side Implementation

TYPESCRIPT

```typescript
import WebSocket from 'ws';

class WebSocketServer {
  private wss: WebSocket.Server;
  private clients = new Map<WebSocket, ClientState>();

  constructor(port: number) {
    this.wss = new WebSocket.Server({ port });

    this.wss.on('connection', (ws) => {
      const clientState: ClientState = {
        subscriptions: new Map(),
        lastHeartbeat: Date.now()
      };

      this.clients.set(ws, clientState);

      ws.on('message', (data) => {
        const message = JSON.parse(data.toString());
        this.handleMessage(ws, message);
      });

      ws.on('close', () => {
        this.clients.delete(ws);
      });
    });

    // Heartbeat to detect dead connections
    setInterval(() => this.checkHeartbeats(), 30000);
  }

  private handleMessage(ws: WebSocket, message: any) {
    switch (message.type) {
      case 'subscribe':
        this.handleSubscribe(ws, message);
        break;
      case 'unsubscribe':
        this.handleUnsubscribe(ws, message);
        break;
      case 'write':
        this.handleWrite(ws, message);
        break;
    }
  }

  broadcastPointUpdate(deviceId: number, point: string, value: number) {
    for (const [ws, state] of this.clients.entries()) {
      const subs = state.subscriptions.get(deviceId);

      if (subs?.has(point)) {
        ws.send(JSON.stringify({
          type: 'point-update',
          deviceId,
          point,
          value,
          timestamp: Date.now()
        }));
```

```
      }
    }
  }
}
```

Next Steps

- REST API
- Events
- Modbus Protocol

Modbus Protocol

Modbus TCP/IP and RTU protocol support for legacy devices.

Modbus TCP

Connection:

- IP Address: Device IP
- Port: 502 (standard)
- Protocol: Modbus TCP

Function Codes:

- 0x03: Read Holding Registers
- 0x04: Read Input Registers
- 0x06: Write Single Register
- 0x10: Write Multiple Registers

Modbus RTU

Serial Settings:

- Baud Rate: 9600, 19200, 38400
- Data Bits: 8
- Parity: None, Even, Odd
- Stop Bits: 1, 2

Slave ID Range: 1-247

Register Mapping

Register	Type	Description
1-100	Input	Analog inputs
101-200	Output	Analog outputs
201-300	Variable	Variables

Error Codes

- 0x01: Illegal Function
- 0x02: Illegal Data Address
- 0x03: Illegal Data Value
- 0x04: Slave Device Failure

Modbus Protocol Implementation

Protocol Specifications

Modbus TCP Frame Format

```
MBAP Header (7 bytes) + PDU (up to 253 bytes)

[Transaction ID (2)] [Protocol ID (2)] [Length (2)] [Unit ID (1)] [Function Code (1)] [Data (n)]
```

Modbus RTU Frame Format

```text
[Slave ID (1)] [Function Code (1)] [Data (n)] [CRC (2)]
```

Complete Register Map

T3000 Modbus Address Mapping

```typescript
const REGISTER_MAP = {
  // Input Registers (30001-30064)
  inputs: {
    base: 30001,
    count: 64,
    access: 'read-only',
    functionCode: 0x04
  },

  // Output Registers (40001-40032)
  outputs: {
    base: 40001,
    count: 32,
    access: 'read-write',
    functionCode: 0x03  // Read
    // 0x06 for single write, 0x10 for multiple write
  },

  // Variable Registers (40033-40096)
  variables: {
    base: 40033,
    count: 64,
    access: 'read-write',
    functionCode: 0x03
  },

  // Control Parameters (40097-40200)
  control: {
    base: 40097,
    count: 104,
    access: 'read-write',
    functionCode: 0x03
  }
};
```

Data Scaling

TYPESCRIPT

```typescript
// Temperature values are scaled by 10
function scaleTemperature(rawValue: number): number {
  return rawValue / 10.0;
}

function unscaleTemperature(tempValue: number): number {
  return Math.round(tempValue * 10);
}

// Example: Read IN1 temperature
const rawValue = await readInputRegister(30001);
const temperature = scaleTemperature(rawValue);  // 725 -> 72.5°F
```

Modbus TCP Client Implementation

Full-Featured Client

TYPESCRIPT

```typescript
import ModbusRTU from 'modbus-serial';

class T3000ModbusTCPClient {
  private client: ModbusRTU;
  private host: string;
  private port: number;
  private unitId: number;
  private connected = false;

  constructor(host: string, port = 502, unitId = 1) {
    this.client = new ModbusRTU();
    this.host = host;
    this.port = port;
    this.unitId = unitId;
  }

  async connect(): Promise<void> {
    try {
      await this.client.connectTCP(this.host, { port: this.port });
      this.client.setID(this.unitId);
      this.client.setTimeout(5000);
      this.connected = true;
    } catch (error) {
      throw new Error(`Failed to connect: ${error.message}`);
    }
  }

  async readInputs(startRegister: number, count: number): Promise<number[]> {
    this.ensureConnected();

    // Convert from Modbus address to zero-based offset
    const address = startRegister - 30001;

    const result = await this.client.readInputRegisters(address, count);
    return result.data.map(v => scaleTemperature(v));
  }

  async readHoldingRegisters(startRegister: number, count: number): Promise<number[]> {
    this.ensureConnected();

    const address = startRegister - 40001;
    const result = await this.client.readHoldingRegisters(address, count);
    return result.data.map(v => scaleTemperature(v));
  }

  async writeSingleRegister(register: number, value: number): Promise<void> {
    this.ensureConnected();

    const address = register - 40001;
    const scaledValue = unscaleTemperature(value);

    await this.client.writeRegister(address, scaledValue);
  }

  async writeMultipleRegisters(startRegister: number, values: number[]): Promise<void> {
```

```
    this.ensureConnected();

    const address = startRegister - 40001;
    const scaledValues = values.map(v => unscaleTemperature(v));

    await this.client.writeRegisters(address, scaledValues);
  }

  private ensureConnected() {
    if (!this.connected) {
      throw new Error('Not connected. Call connect() first.');
    }
  }

  async disconnect(): Promise<void> {
    await this.client.close();
    this.connected = false;
  }
}
```

Usage Examples

TYPESCRIPT

```
// Initialize client
const modbus = new T3000ModbusTCPClient('192.168.1.100');
await modbus.connect();

// Read all inputs
const inputs = await modbus.readInputs(30001, 64);
console.log('IN1:', inputs[0]);

// Read specific output
const outputs = await modbus.readHoldingRegisters(40001, 1);
console.log('OUT1:', outputs[0]);

// Write to output
await modbus.writeSingleRegister(40001, 75.5);

// Write multiple values
await modbus.writeMultipleRegisters(40001, [75.5, 50.0, 60.0]);

await modbus.disconnect();
```

Modbus RTU Implementation

Serial Port Client

PYTHON

```python
from pymodbus.client import ModbusSerialClient
import struct

class T3000ModbusRTU:
    def __init__(self, port='/dev/ttyUSB0', baudrate=19200, slave_id=1):
        self.client = ModbusSerialClient(
            port=port,
            baudrate=baudrate,
            bytesize=8,
            parity='N',
            stopbits=1,
            timeout=1
        )
        self.slave_id = slave_id

    def connect(self):
        return self.client.connect()

    def read_inputs(self, start_register, count):
        """Read input registers (function code 0x04)"""
        address = start_register - 30001  # Convert to zero-based
        result = self.client.read_input_registers(
            address,
            count,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

        return [value / 10.0 for value in result.registers]

    def read_holding_registers(self, start_register, count):
        """Read holding registers (function code 0x03)"""
        address = start_register - 40001
        result = self.client.read_holding_registers(
            address,
            count,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

        return [value / 10.0 for value in result.registers]

    def write_register(self, register, value):
        """Write single register (function code 0x06)"""
        address = register - 40001
        scaled_value = int(value * 10)

        result = self.client.write_register(
            address,
            scaled_value,
            unit=self.slave_id
```

```
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

    def write_registers(self, start_register, values):
        """Write multiple registers (function code 0x10)"""
        address = start_register - 40001
        scaled_values = [int(v * 10) for v in values]

        result = self.client.write_registers(
            address,
            scaled_values,
            unit=self.slave_id
        )

        if result.isError():
            raise Exception(f"Modbus error: {result}")

    def close(self):
        self.client.close()
```

Usage Example

PYTHON

```
## Initialize
modbus = T3000ModbusRTU(port='/dev/ttyUSB0', baudrate=19200, slave_id=1)
modbus.connect()

## Read inputs
inputs = modbus.read_inputs(30001, 10)
print(f"IN1: {inputs[0]}°F")

## Write output
modbus.write_register(40001, 72.5)

modbus.close()
```

Error Handling

Exception Handling

TYPESCRIPT

```typescript
async function robustModbusOperation<T>(
  operation: () => Promise<T>,
  maxRetries = 3,
  delayMs = 1000
): Promise<T> {
  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error;

      // Check for non-recoverable errors
      if (error.modbusCode === 0x02) {  // Illegal data address
        throw error;  // Don't retry
      }

      if (attempt < maxRetries - 1) {
        await new Promise(resolve => setTimeout(resolve, delayMs));
        delayMs *= 2;  // Exponential backoff
      }
    }
  }

  throw lastError!;
}

// Usage
const value = await robustModbusOperation(
  () => modbus.readInputs(30001, 1)
);
```

Performance Optimization

Batch Reading

TYPESCRIPT

```typescript
// ❌ Inefficient: 64 separate reads
for (let i = 0; i < 64; i++) {
  const value = await modbus.readInputs(30001 + i, 1);
  console.log(`IN${i+1}: ${value[0]}`);
}

// ✅ Efficient: 1 batch read
const allInputs = await modbus.readInputs(30001, 64);
allInputs.forEach((value, i) => {
  console.log(`IN${i+1}: ${value}`);
});
```

Connection Pooling

```typescript
class ModbusConnectionPool {
  private connections: T3000ModbusTCPClient[] = [];
  private available: T3000ModbusTCPClient[] = [];
  private maxConnections: number;

  constructor(host: string, maxConnections = 5) {
    this.maxConnections = maxConnections;
  }

  async acquire(): Promise<T3000ModbusTCPClient> {
    if (this.available.length > 0) {
      return this.available.pop()!;
    }

    if (this.connections.length < this.maxConnections) {
      const client = new T3000ModbusTCPClient(this.host);
      await client.connect();
      this.connections.push(client);
      return client;
    }

    // Wait for available connection
    return new Promise((resolve) => {
      const checkInterval = setInterval(() => {
        if (this.available.length > 0) {
          clearInterval(checkInterval);
          resolve(this.available.pop()!);
        }
      }, 10);
    });
  }

  release(client: T3000ModbusTCPClient): void {
    this.available.push(client);
  }
}
```

Next Steps

- REST API
- WebSocket API
- Events

Data Points API

Overview

The Data Points API provides access to inputs, outputs, and variables on BACnet devices. Use these endpoints to read sensor values, control outputs, and manage internal variables.

Point Types

- **Inputs**: Sensor readings (analog: temperature, pressure; binary: switch states)
- **Outputs**: Controllable points (analog: damper position, valve; binary: fan on/off)
- **Variables**: Internal memory locations for calculations and logic

Common Operations

Get All Inputs

```typescript
const inputs = await fetch('http://localhost:9103/api/t3_device/devices/1/input-points')
  .then(res => res.json());

console.log(inputs);
// [
//   {
//     "id": 1,
//     "device_id": 1,
//     "panel": 5,
//     "point_number": 1,
//     "label": "Room Temperature",
//     "value": 72.5,
//     "auto_manual": 0,
//     "units": "Deg F",
//     "range": 3,
//     "calibration": 0,
//     "filter": 3,
//     "status": "OK"
//   }
// ]
```

Get All Outputs

```typescript
const outputs = await fetch('http://localhost:9103/api/t3_device/devices/1/output-points')
  .then(res => res.json());
```

Get All Variables

```typescript
const variables = await fetch('http://localhost:9103/api/t3_device/devices/1/variable-points')
  .then(res => res.json());
```

Update Single Output

```typescript
await fetch('http://localhost:9103/api/t3_device/outputs/5000000', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    value: 50.0,
    auto_manual: 1  // Manual mode
  })
});
```

Batch Update

Update multiple points in one request:

```typescript
await fetch('http://localhost:9103/api/t3_device/outputs/batch', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    updates: [
      { objectinstance: 5000000, value: 50.0 },
      { objectinstance: 5000001, value: 75.0 },
      { objectinstance: 5000002, value: 100.0 }
    ]
  })
});
```

Refresh from Device

Force read from hardware:

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1/inputs/refresh', {
  method: 'POST'
});
```

React Component Usage

Point Display Component

TYPESCRIPT

```typescript
import { usePointStore } from '@/stores/pointStore';

export function InputList({ deviceId }: { deviceId: number }) {
  const inputs = usePointStore(state => state.inputs);
  const loadInputs = usePointStore(state => state.loadInputs);

  useEffect(() => {
    loadInputs(deviceId);
  }, [deviceId, loadInputs]);

  return (
    <table>
      <thead>
        <tr>
          <th>Point</th>
          <th>Label</th>
          <th>Value</th>
          <th>Units</th>
        </tr>
      </thead>
      <tbody>
        {inputs.map(input => (
          <tr key={input.id}>
            <td>{input.point_number}</td>
            <td>{input.label}</td>
            <td>{input.value}</td>
            <td>{input.units}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Output Control Component

```typescript
export function OutputControl({ output }: { output: Output }) {
  const [value, setValue] = useState(output.value);

  const handleUpdate = async () => {
    await fetch(`http://localhost:9103/api/t3_device/outputs/${output.objectinstance}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ value })
    });
  };

  return (
    <div>
      <label>{output.label}</label>
      <input
        type="number"
        value={value}
        onChange={(e) => setValue(Number(e.target.value))}
      />
      <button onClick={handleUpdate}>Update</button>
    </div>
  );
}
```

Input Point Endpoints

GET /api/t3_device/devices/:id/input-points

Description: Get all inputs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/input-points HTTP/1.1
```

Response:

JSON


```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Room Temperature",
    "value": 72.5,
    "auto_manual": 0,
    "units": "Deg F",
    "range": 3,
    "calibration": 0.0,
    "filter": 3,
    "status": "OK",
    "objectinstance": 3000000,
    "decom": 0,
    "type": "analog"
  }
]
```


Used By:

- InputTable.tsx
- PointGrid.tsx

Handler: `api/src/t3_device/routes.rs::get_input_points`

GET /api/t3_device/inputs/:objectinstance

Description: Get single input by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (3000000+)

Request:

HTTP

```
GET /api/t3_device/inputs/3000001 HTTP/1.1
```

Response:

JSON


```
{
  "id": 2,
  "device_id": 1,
  "panel": 5,
  "point_number": 2,
  "label": "Supply Air Temp",
  "value": 55.0,
  "auto_manual": 0,
  "units": "Deg F",
  "range": 3,
  "objectinstance": 3000001
}
```


Used By:

- PointDetail.tsx

Errors:

- `404` - Input not found

PUT /api/t3_device/inputs/:objectinstance

Description: Update input properties (label, calibration, filter, etc.)

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

```http
PUT /api/t3_device/inputs/3000001 HTTP/1.1
Content-Type: application/json

{
  "label": "Supply Air Temperature",
  "calibration": 0.5,
  "filter": 5
}
```

Updatable Fields:

- `label`, `calibration`, `filter`, `status`, `decom`

Response:

```json
{
  "id": 2,
  "label": "Supply Air Temperature",
  "calibration": 0.5,
  "filter": 5
  // ... all fields
}
```

Used By:

- EditInputDialog.tsx

POST /api/t3_device/devices/:id/inputs/refresh

Description: Refresh all inputs from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
POST /api/t3_device/devices/1/inputs/refresh HTTP/1.1
```

Response:

JSON

```
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 16,
  "duration_ms": 342
}
```

Used By:

- RefreshPointsButton.tsx

Errors:

- `503` - Device offline

Output Point Endpoints

GET /api/t3_device/devices/:id/output-points

Description: Get all outputs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

HTTP

```
GET /api/t3_device/devices/1/output-points HTTP/1.1
```

Response:

JSON

```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Cooling Valve",
    "value": 50.0,
    "auto_manual": 0,
    "units": "Percent",
    "range": 7,
    "objectinstance": 5000000,
    "pwm_period": 15,
    "decom": 0,
    "type": "analog"
  }
]
```

Used By:

- OutputTable.tsx

GET /api/t3_device/outputs/:objectinstance

Description: Get single output by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (5000000+)

Request:

```http
GET /api/t3_device/outputs/5000000 HTTP/1.1
```

Response:

```json
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "point_number": 1,
  "label": "Cooling Valve",
  "value": 50.0,
  "auto_manual": 0,
  "objectinstance": 5000000
}
```

Used By:

- OutputControl.tsx

PUT /api/t3_device/outputs/:objectinstance

Description: Update output value and properties

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

```http
PUT /api/t3_device/outputs/5000000 HTTP/1.1
Content-Type: application/json

{
  "value": 75.0,
  "auto_manual": 1
}
```

Updatable Fields:

- `value` - Output value (0-100 for analog)
- `auto_manual` - 0=Auto, 1=Manual
- `label`, `pwm_period`, `decom`

Response:

JSON

```
{
  "success": true,
  "objectinstance": 5000000,
  "value": 75.0,
  "updated_at": "2026-01-09T10:00:00Z"
}
```

Used By:

- OutputControl.tsx
- ManualOverride.tsx

Errors:

- `400` - Invalid value or mode
- `404` - Output not found
- `503` - Device offline

PUT /api/t3_device/outputs/batch

Description: Batch update multiple outputs

Request:

HTTP

```
PUT /api/t3_device/outputs/batch HTTP/1.1
Content-Type: application/json

{
  "updates": [
    { "objectinstance": 5000000, "value": 50.0, "auto_manual": 1 },
    { "objectinstance": 5000001, "value": 75.0, "auto_manual": 1 },
    { "objectinstance": 5000002, "value": 100.0, "auto_manual": 0 }
  ]
}
```

Response:

JSON

```
{
  "success": true,
  "updated": 3,
  "failed": 0,
  "results": [
    { "objectinstance": 5000000, "success": true },
    { "objectinstance": 5000001, "success": true },
    { "objectinstance": 5000002, "success": true }
  ]
}
```

Used By:

- BatchControl.tsx

POST /api/t3_device/devices/:id/outputs/refresh

Description: Refresh all outputs from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
POST /api/t3_device/devices/1/outputs/refresh HTTP/1.1
```

Response:

```json
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 8,
  "duration_ms": 215
}
```

Used By:

- RefreshPointsButton.tsx

Variable Point Endpoints

GET /api/t3_device/devices/:id/variable-points

Description: Get all variables for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/variable-points HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "point_number": 1,
    "label": "Setpoint",
    "value": 72.0,
    "auto_manual": 0,
    "units": "Deg F",
    "range": 3,
    "objectinstance": 2000000
  }
]
```

Used By:

- VariableTable.tsx

GET /api/t3_device/variables/:objectinstance

Description: Get single variable by object instance

Parameters:

- `objectinstance` (path, integer) - BACnet object instance (2000000+)

Request:

```http
GET /api/t3_device/variables/2000000 HTTP/1.1
```

Response:

```json
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "point_number": 1,
  "label": "Setpoint",
  "value": 72.0,
  "objectinstance": 2000000
}
```

PUT /api/t3_device/variables/:objectinstance

Description: Update variable value and properties

Parameters:

- `objectinstance` (path, integer) - BACnet object instance

Request:

```http
PUT /api/t3_device/variables/2000000 HTTP/1.1
Content-Type: application/json

{
  "value": 70.0,
  "auto_manual": 1
}
```

Response:

```json
{
  "success": true,
  "objectinstance": 2000000,
  "value": 70.0
}
```

Used By:

- VariableControl.tsx

PUT /api/t3_device/variables/batch

Description: Batch update multiple variables

Request:

```http
PUT /api/t3_device/variables/batch HTTP/1.1
Content-Type: application/json

{
  "updates": [
    { "objectinstance": 2000000, "value": 70.0 },
    { "objectinstance": 2000001, "value": 65.0 }
  ]
}
```

Response:

```json
{
  "success": true,
  "updated": 2,
  "failed": 0
}
```

Used By:

- BatchControl.tsx

POST /api/t3_device/devices/:id/variables/refresh

Description: Refresh all variables from device hardware

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
POST /api/t3_device/devices/1/variables/refresh HTTP/1.1
```

Response:

```json
{
  "success": true,
  "device_id": 1,
  "points_refreshed": 64,
  "duration_ms": 892
}
```

Implementation Details

Database Schema

SQL

```sql
CREATE TABLE INPUTS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    calibration REAL DEFAULT 0,
    filter INTEGER DEFAULT 3,
    status TEXT,
    objectinstance INTEGER UNIQUE,
    decom INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE OUTPUTS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    pwm_period INTEGER DEFAULT 15,
    objectinstance INTEGER UNIQUE,
    decom INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE VARIABLES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    point_number INTEGER NOT NULL,
    label TEXT,
    value REAL DEFAULT 0,
    auto_manual INTEGER DEFAULT 0,
    units TEXT,
    range INTEGER,
    objectinstance INTEGER UNIQUE,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

Object Instance Mapping

TEXT

```
Inputs:    3000000 + point_number
Outputs:   5000000 + point_number
Variables: 2000000 + point_number
```

Rust Handler Example

RUST

```rust
// Update output with BACnet write
async fn update_output(
    Path(objectinstance): Path<i32>,
    State(state): State<T3AppState>,
    Json(payload): Json<UpdateOutputRequest>,
) -> Result<Json<UpdateResponse>, ApiError> {
    let conn = state.get_conn().await?;

    // Find output
    let output = Output::find()
        .filter(output::Column::Objectinstance.eq(objectinstance))
        .one(&conn)
        .await?
        .ok_or(ApiError::NotFound("Output not found".into()))?;

    // Write to BACnet device via FFI
    let result = unsafe {
        write_bacnet_property(
            output.device_id,
            objectinstance,
            payload.value,
        )
    };

    if result != 0 {
        return Err(ApiError::DeviceOffline);
    }

    // Update database
    let mut active_model: output::ActiveModel = output.into();
    active_model.value = Set(payload.value);
    if let Some(am) = payload.auto_manual {
        active_model.auto_manual = Set(am);
    }

    active_model.update(&conn).await?;

    Ok(Json(UpdateResponse {
        success: true,
        objectinstance,
        value: payload.value,
        updated_at: Utc::now(),
    }))
}
```

TypeScript Types

TYPESCRIPT

```typescript
export interface Input {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
  units: string;
  range: number;
  calibration: number;
  filter: number;
  status: string;
  objectinstance: number;
  decom: number;
}

export interface Output {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
  units: string;
  range: number;
  pwm_period: number;
  objectinstance: number;
  decom: number;
}

export interface Variable {
  id: number;
  device_id: number;
  panel: number;
  point_number: number;
  label: string;
  value: number;
  auto_manual: number;
  units: string;
  range: number;
  objectinstance: number;
}
```

Auto/Manual Modes

- `0` = Auto (device controlled)
- `1` = Manual (user override)

Range Codes

Common range values:

- `3` = Deg F
- `4` = Deg C
- `7` = Percent
- `21` = PPM

- `62` = PSI

See unit conversion tables for complete mapping.

Error Codes

- `POINT_NOT_FOUND` - Object instance doesn't exist
- `DEVICE_OFFLINE` - Cannot communicate with device
- `INVALID_VALUE` - Value out of range
- `READ_ONLY` - Point not writable
- `WRITE_FAILED` - BACnet write error

Device Management API

Overview

The Device Management API allows you to discover, connect to, and manage BACnet devices on your network. Use these endpoints to scan for devices, check their online status, and retrieve device information.

Common Operations

List All Devices

Get all registered devices with their connection status:

TYPESCRIPT

```typescript
const devices = await fetch('http://localhost:9103/api/t3_device/devices')
  .then(res => res.json());

console.log(devices);
// [
//   {
//     "id": 1,
//     "serial_number": 123456,
//     "product_name": "T3-BB",
//     "ip_address": "192.168.1.100",
//     "online": true
//   }
// ]
```

Get Single Device

Retrieve detailed information about a specific device:

TYPESCRIPT

```typescript
const device = await fetch('http://localhost:9103/api/t3_device/devices/1')
  .then(res => res.json());
```

Add New Device

Register a device manually:

TYPESCRIPT

```typescript
const newDevice = await fetch('http://localhost:9103/api/t3_device/devices', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    serial_number: 654321,
    product_name: 'T3-LB',
    ip_address: '192.168.1.101'
  })
}).then(res => res.json());
```

Update Device

Modify device information:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    ip_address: '192.168.1.200',
    notes: 'Relocated to second floor'
  })
});
```

Delete Device

Remove a device from the system:

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/devices/1', {
  method: 'DELETE'
});
```

Check Device Capabilities

Get device features and supported objects:

TYPESCRIPT

```typescript
const capabilities = await fetch('http://localhost:9103/api/t3_device/devices/1/capabilities')
  .then(res => res.json());

console.log(capabilities);
// {
//   "inputs": 16,
//   "outputs": 8,
//   "variables": 64,
//   "programs": 8,
//   "supports_trendlogs": true
// }
```

React Component Usage

Device List Component

TYPESCRIPT

```typescript
import { useDeviceStore } from '@/stores/deviceStore';

export function DeviceList() {
  const devices = useDeviceStore(state => state.devices);
  const loadDevices = useDeviceStore(state => state.loadDevices);

  useEffect(() => {
    loadDevices();
  }, [loadDevices]);

  return (
    <div>
      {devices.map(device => (
        <DeviceCard key={device.id} device={device} />
      ))}
    </div>
  );
}
```

Zustand Store

TYPESCRIPT

```typescript
// stores/deviceStore.ts
import create from 'zustand';

interface DeviceStore {
  devices: Device[];
  loadDevices: () => Promise<void>;
  addDevice: (device: Partial<Device>) => Promise<void>;
}

export const useDeviceStore = create<DeviceStore>((set) => ({
  devices: [],

  loadDevices: async () => {
    const response = await fetch('http://localhost:9103/api/t3_device/devices');
    const devices = await response.json();
    set({ devices });
  },

  addDevice: async (device) => {
    const response = await fetch('http://localhost:9103/api/t3_device/devices', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(device)
    });
    const newDevice = await response.json();
    set(state => ({ devices: [...state.devices, newDevice] }));
  }
}));
```

Device Management Endpoints

GET /api/t3_device/devices

Description: Retrieve all devices with statistics

Request:

```http
GET /api/t3_device/devices HTTP/1.1
Host: localhost:9103
Accept: application/json
```

Response:


```json
[
  {
    "id": 1,
    "serial_number": 123456,
    "product_name": "T3-BB",
    "product_id": 211,
    "ip_address": "192.168.1.100",
    "port": 47808,
    "online": true,
    "modbus_id": 1,
    "baudrate": 19200,
    "protocol": "bacnet_ip",
    "panel_number": 5,
    "firmware_version": "95.2",
    "hardware_version": "2.0",
    "location": "Building 1 - 2nd Floor",
    "notes": "",
    "created_at": "2026-01-01T10:00:00Z",
    "updated_at": "2026-01-09T08:30:00Z",
    "last_seen": "2026-01-09T08:29:45Z",
    "stats": {
      "inputs": 16,
      "outputs": 8,
      "variables": 64,
      "programs": 4,
      "schedules": 12
    }
  }
]
```


Used By:

- DeviceTree.tsx
- DeviceList.tsx

Handler: `api/src/t3_device/routes.rs::get_devices_with_stats`

Errors:

- `500` - Database error

GET /api/t3_device/devices/:id

Description: Get single device by ID

Parameters:

- `id` (path, integer) - Device ID

Request:

```
HTTP
GET /api/t3_device/devices/1 HTTP/1.1
```

Response:

```
JSON
{
  "id": 1,
  "serial_number": 123456,
  "product_name": "T3-BB",
  "ip_address": "192.168.1.100",
  "online": true
  // ... all device fields
}
```

Used By:

- DeviceDetail.tsx

Errors:

- `404` - Device not found

POST /api/t3_device/devices

Description: Create new device

Request:

```
HTTP
POST /api/t3_device/devices HTTP/1.1
Content-Type: application/json

{
  "serial_number": 654321,
  "product_name": "T3-LB",
  "product_id": 213,
  "ip_address": "192.168.1.101",
  "port": 47808,
  "modbus_id": 2,
  "baudrate": 19200,
  "location": "Building 2"
}
```

Required Fields:

- `serial_number` (integer)
- `product_name` (string)

Optional Fields:

- `product_id`, `ip_address`, `port`, `modbus_id`, `baudrate`, `location`, `notes`

Response:

JSON

```
{
  "id": 2,
  "serial_number": 654321,
  "product_name": "T3-LB",
  "created_at": "2026-01-09T09:00:00Z"
  // ... all fields
}
```

Used By:

- AddDeviceDialog.tsx

Errors:

- `400` - Invalid request body
- `409` - Device with serial number already exists

PUT /api/t3_device/devices/:id

Description: Update existing device

Parameters:

- `id` (path, integer) - Device ID

Request:

HTTP

```
PUT /api/t3_device/devices/1 HTTP/1.1
Content-Type: application/json

{
  "ip_address": "192.168.1.200",
  "location": "Building 1 - 3rd Floor",
  "notes": "Relocated during renovation"
}
```

Response:

JSON

```
{
  "id": 1,
  "ip_address": "192.168.1.200",
  "location": "Building 1 - 3rd Floor",
  "updated_at": "2026-01-09T09:15:00Z"
  // ... all fields
}
```

Used By:

- EditDeviceDialog.tsx

Errors:

- `404` - Device not found
- `400` - Invalid request body

DELETE /api/t3_device/devices/:id

Description: Delete device and all associated data

Parameters:

- `id` (path, integer) - Device ID

Request:

```
HTTP
DELETE /api/t3_device/devices/1 HTTP/1.1
```

Response:

```
HTTP
HTTP/1.1 204 No Content
```

Warning: This cascades and deletes all inputs, outputs, variables, programs, schedules, and trendlogs associated with the device.

Used By:

- DeleteDeviceDialog.tsx

Errors:

- `404` - Device not found

GET /api/t3_device/devices/:id/capabilities

Description: Get device capabilities and supported features

Parameters:

- `id` (path, integer) - Device ID

Request:

```
HTTP
GET /api/t3_device/devices/1/capabilities HTTP/1.1
```

Response:

JSON

```
{
  "device_id": 1,
  "inputs": {
    "count": 16,
    "max": 32,
    "types": ["analog", "binary"]
  },
  "outputs": {
    "count": 8,
    "max": 16,
    "types": ["analog", "binary"]
  },
  "variables": {
    "count": 64,
    "max": 200
  },
  "programs": {
    "count": 4,
    "max": 8
  },
  "schedules": {
    "count": 12,
    "max": 50
  },
  "trendlogs": {
    "supported": true,
    "count": 6,
    "max": 8
  },
  "features": {
    "bacnet_mstp": false,
    "bacnet_ip": true,
    "modbus": true,
    "web_interface": true
  }
}
```

Used By:

- DeviceCapabilities.tsx

Errors:

- `404` - Device not found

GET /api/t3_device/devices/count

Description: Get total device count

Request:

HTTP

```
GET /api/t3_device/devices/count HTTP/1.1
```

Response:

```JSON
{
  "count": 12
}
```

Used By:

- Dashboard.tsx

POST /api/t3_device/devices/:id/refresh

Description: Force refresh device from hardware (via BACnet)

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
POST /api/t3_device/devices/1/refresh HTTP/1.1
```

Response:

```JSON
{
  "success": true,
  "device_id": 1,
  "refreshed_at": "2026-01-09T09:30:00Z",
  "objects_updated": 142
}
```

Used By:

- RefreshDeviceButton.tsx

Errors:

- `404` - Device not found
- `503` - Device offline or unreachable

GET /api/t3_device/devices/:id/connection-status

Description: Check real-time device connection status

Parameters:

- `id` (path, integer) - Device ID

Request:

```HTTP
GET /api/t3_device/devices/1/connection-status HTTP/1.1
```

Response:

JSON

```
{
  "online": true,
  "last_seen": "2026-01-09T09:29:45Z",
  "response_time_ms": 45,
  "connection_type": "bacnet_ip",
  "errors": []
}
```

Used By:

- DeviceStatusIndicator.tsx

Errors:

- `404` - Device not found

Implementation Details

Database Schema

SQL

```
CREATE TABLE DEVICES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    serial_number INTEGER NOT NULL UNIQUE,
    product_name TEXT NOT NULL,
    product_id INTEGER,
    ip_address TEXT,
    port INTEGER DEFAULT 47808,
    online INTEGER DEFAULT 0,
    modbus_id INTEGER,
    baudrate INTEGER DEFAULT 19200,
    protocol TEXT DEFAULT 'bacnet_ip',
    panel_number INTEGER,
    firmware_version TEXT,
    hardware_version TEXT,
    location TEXT,
    notes TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    last_seen DATETIME
);

CREATE INDEX idx_devices_serial ON DEVICES(serial_number);
CREATE INDEX idx_devices_online ON DEVICES(online);
```

Rust Handler Example

RUST

```rust
// api/src/t3_device/routes.rs
async fn get_devices_with_stats(
    State(state): State<T3AppState>,
) -> Result<Json<Vec<DeviceWithStats>>, ApiError> {
    let conn = state.get_conn().await?;

    let devices = Device::find()
        .all(&conn)
        .await
        .map_err(|e| ApiError::DatabaseError(e.to_string()))?;

    let mut result = Vec::new();

    for device in devices {
        let stats = get_device_stats(&conn, device.id).await?;
        result.push(DeviceWithStats {
            device,
            stats,
        });
    }

    Ok(Json(result))
}

async fn get_device_stats(conn: &DatabaseConnection, device_id: i32) -> Result<DeviceStats, ApiError> {
    let inputs = Input::find()
        .filter(input::Column::DeviceId.eq(device_id))
        .count(conn)
        .await?;

    let outputs = Output::find()
        .filter(output::Column::DeviceId.eq(device_id))
        .count(conn)
        .await?;

    // ... count other objects

    Ok(DeviceStats {
        inputs: inputs as i32,
        outputs: outputs as i32,
        // ...
    })
}
```

TypeScript Types

TYPESCRIPT

```typescript
// types/device.ts
export interface Device {
  id: number;
  serial_number: number;
  product_name: string;
  product_id?: number;
  ip_address?: string;
  port?: number;
  online: boolean;
  modbus_id?: number;
  baudrate?: number;
  protocol?: string;
  panel_number?: number;
  firmware_version?: string;
  hardware_version?: string;
  location?: string;
  notes?: string;
  created_at: string;
  updated_at: string;
  last_seen?: string;
}

export interface DeviceStats {
  inputs: number;
  outputs: number;
  variables: number;
  programs: number;
  schedules: number;
}

export interface DeviceWithStats {
  device: Device;
  stats: DeviceStats;
}
```

Error Codes

- `DEVICE_NOT_FOUND` - Device ID doesn't exist
- `DEVICE_OFFLINE` - Device not reachable
- `DUPLICATE_SERIAL` - Serial number already registered
- `DATABASE_ERROR` - Internal database error
- `INVALID_REQUEST` - Malformed request body

Control & Automation API

Overview

The Control & Automation API manages programs, schedules, PIDs, holidays, and annual routines for automated building control. Use these endpoints to configure time-based control, logic programs, and temperature regulation.

Features

- **Programs**: Custom control logic and sequences
- **Schedules**: Time-based control patterns
- **PIDs**: Temperature and pressure control loops
- **Holidays**: Special day schedules
- **Annual Routines**: Recurring yearly events

Common Operations

Get All Schedules

TYPESCRIPT

```typescript
const schedules = await fetch('http://localhost:9103/api/t3_device/devices/1/schedules')
  .then(res => res.json());

console.log(schedules);
// [
//   {
//     "id": 1,
//     "device_id": 1,
//     "panel": 5,
//     "index": 1,
//     "label": "Office Hours",
//     "auto_manual": 0,
//     "output": 1,
//     "holiday_id": 0,
//     "status": "ON",
//     "override": 0
//   }
// ]
```

Get Schedule Details

TYPESCRIPT

```typescript
const schedule = await fetch('http://localhost:9103/api/t3_device/schedules/1')
  .then(res => res.json());
```

Update Schedule

TYPESCRIPT

```typescript
await fetch('http://localhost:9103/api/t3_device/schedules/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    label: "Office Hours - Updated",
    auto_manual: 1,
    output: 100
  })
});
```

Get All Programs

```typescript
const programs = await fetch('http://localhost:9103/api/t3_device/devices/1/programs')
  .then(res => res.json());
```

Update Program Code

```typescript
await fetch('http://localhost:9103/api/t3_device/programs/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    label: "VAV Control",
    description: "Variable Air Volume Logic",
    code: "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END"
  })
});
```

Get PIDs

```typescript
const pids = await fetch('http://localhost:9103/api/t3_device/devices/1/pids')
  .then(res => res.json());
```

Schedule Endpoints

GET /api/t3_device/devices/:id/schedules

Description: Get all schedules for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/schedules HTTP/1.1
```

Response:

JSON

```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Office Hours",
    "auto_manual": 0,
    "output": 1,
    "holiday_id": 0,
    "status": "ON",
    "override": 0,
    "times": {
      "monday": [
        { "start": "08:00", "end": "18:00", "value": 1 }
      ],
      "tuesday": [
        { "start": "08:00", "end": "18:00", "value": 1 }
      ]
      // ... other days
    }
  }
]
```

Used By:

- ScheduleTable.tsx
- ScheduleCalendar.tsx

Handler: `api/src/t3_device/routes.rs::get_schedules_by_device`

GET /api/t3_device/schedules/:id

Description: Get single schedule by ID

Parameters:

- `id` (path, integer) - Schedule ID

Request:

HTTP

```
GET /api/t3_device/schedules/1 HTTP/1.1
```

Response:

JSON

```
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "index": 1,
  "label": "Office Hours",
  "auto_manual": 0,
  "output": 1,
  "holiday_id": 0,
  "status": "ON",
  "override": 0,
  "times": {
    // ... weekly schedule
  }
}
```

Used By:

- ScheduleDetail.tsx

Errors:

- `404` - Schedule not found

PUT /api/t3_device/schedules/:id

Description: Update schedule configuration

Parameters:

- `id` (path, integer) - Schedule ID

Request:

HTTP

```
PUT /api/t3_device/schedules/1 HTTP/1.1
Content-Type: application/json

{
  "label": "Office Hours - Winter",
  "auto_manual": 0,
  "output": 1,
  "times": {
    "monday": [
      { "start": "07:00", "end": "19:00", "value": 1 }
    ]
  }
}
```

Updatable Fields:

- `label`, `auto_manual`, `output`, `holiday_id`, `status`, `override`, `times`

Response:

```JSON
{
  "success": true,
  "id": 1,
  "updated_at": "2026-01-09T10:30:00Z"
}
```

Used By:

- EditScheduleDialog.tsx

POST /api/t3_device/schedules

Description: Create new schedule

Request:

```HTTP
POST /api/t3_device/schedules HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "panel": 5,
  "label": "Night Setback",
  "auto_manual": 0,
  "output": 0
}
```

Required Fields:

- `device_id`, `panel`, `label`

Response:

```JSON
{
  "id": 13,
  "device_id": 1,
  "label": "Night Setback",
  "created_at": "2026-01-09T10:35:00Z"
}
```

Used By:

- AddScheduleDialog.tsx

DELETE /api/t3_device/schedules/:id

Description: Delete schedule

Parameters:

- `id` (path, integer) - Schedule ID

Request:

```HTTP
DELETE /api/t3_device/schedules/1 HTTP/1.1
```

Response:

```http
HTTP/1.1 204 No Content
```

Used By:

- DeleteScheduleDialog.tsx

Program Endpoints

GET /api/t3_device/devices/:id/programs

Description: Get all programs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/programs HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "VAV Control",
    "description": "Variable Air Volume Logic",
    "auto_manual": 0,
    "status": "RUNNING",
    "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END",
    "size": 52,
    "execution_time": 15
  }
]
```

Used By:

- ProgramTable.tsx

GET /api/t3_device/programs/:id

Description: Get single program by ID

Parameters:

- `id` (path, integer) - Program ID

Request:

```http
GET /api/t3_device/programs/1 HTTP/1.1
```

Response:

```JSON
{
  "id": 1,
  "device_id": 1,
  "panel": 5,
  "index": 1,
  "label": "VAV Control",
  "description": "Variable Air Volume Logic",
  "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n20 ELSE OUT1 = 0\n30 END",
  "status": "RUNNING"
}
```

Used By:

- ProgramEditor.tsx

PUT /api/t3_device/programs/:id

Description: Update program code and settings

Parameters:

- `id` (path, integer) - Program ID

Request:

```HTTP
PUT /api/t3_device/programs/1 HTTP/1.1
Content-Type: application/json

{
  "label": "VAV Control v2",
  "description": "Updated logic for winter mode",
  "code": "10 IF IN1 > VAR1 THEN OUT1 = 100\n15 IF IN2 < 40 THEN OUT1 = 50\n20 ELSE OUT1 = 0\n30 END",
  "auto_manual": 0
}
```

Updatable Fields:

- `label`, `description`, `code`, `auto_manual`, `status`

Response:

```JSON
{
  "success": true,
  "id": 1,
  "compiled": true,
  "errors": []
}
```

Used By:

- ProgramEditor.tsx

Errors:

- `400` - Syntax error in program code
- `404` - Program not found

POST /api/t3_device/programs

Description: Create new program

Request:

```http
POST /api/t3_device/programs HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "panel": 5,
  "label": "New Logic",
  "description": "Custom control sequence",
  "code": ""
}
```

Response:

```json
{
  "id": 5,
  "device_id": 1,
  "label": "New Logic",
  "created_at": "2026-01-09T11:00:00Z"
}
```

Used By:

- AddProgramDialog.tsx

DELETE /api/t3_device/programs/:id

Description: Delete program

Parameters:

- `id` (path, integer) - Program ID

Request:

```http
DELETE /api/t3_device/programs/1 HTTP/1.1
```

Response:

```http
HTTP/1.1 204 No Content
```

PID Endpoints

GET /api/t3_device/devices/:id/pids

Description: Get all PID loops for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/pids HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Zone Temperature",
    "input": 1,
    "setpoint": 72.0,
    "output": 1,
    "proportional": 10.0,
    "integral": 5.0,
    "derivative": 1.0,
    "bias": 0.0,
    "auto_manual": 0,
    "status": "RUNNING"
  }
]
```

Used By:

- PIDTable.tsx

GET /api/t3_device/pids/:id

Description: Get single PID by ID

Parameters:

- `id` (path, integer) - PID ID

Request:

```http
GET /api/t3_device/pids/1 HTTP/1.1
```

Response:

```json
{
  "id": 1,
  "label": "Zone Temperature",
  "input": 1,
  "setpoint": 72.0,
  "output": 1,
  "proportional": 10.0,
  "integral": 5.0,
  "derivative": 1.0
}
```

Used By:

- PIDTuning.tsx

PUT /api/t3_device/pids/:id

Description: Update PID parameters

Parameters:

- `id` (path, integer) - PID ID

Request:

```http
PUT /api/t3_device/pids/1 HTTP/1.1
Content-Type: application/json

{
  "setpoint": 70.0,
  "proportional": 12.0,
  "integral": 6.0,
  "derivative": 1.5,
  "auto_manual": 0
}
```

Updatable Fields:

- `label`, `input`, `setpoint`, `output`, `proportional`, `integral`, `derivative`, `bias`, `auto_manual`

Response:

```json
{
  "success": true,
  "id": 1,
  "updated_at": "2026-01-09T11:15:00Z"
}
```

Used By:

- PIDTuning.tsx

Holiday Endpoints

GET /api/t3_device/devices/:id/holidays

Description: Get all holidays for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/holidays HTTP/1.1
```

Response:

JSON

```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "New Year",
    "auto_manual": 0,
    "value": 0,
    "date": "2026-01-01"
  }
]
```

Used By:

- HolidayCalendar.tsx

PUT /api/t3_device/holidays/:id

Description: Update holiday

Parameters:

- `id` (path, integer) - Holiday ID

Request:

HTTP

```
PUT /api/t3_device/holidays/1 HTTP/1.1
Content-Type: application/json

{
  "label": "New Year's Day",
  "date": "2027-01-01",
  "auto_manual": 0,
  "value": 0
}
```

Response:

JSON

```
{
  "success": true,
  "id": 1
}
```

Used By:

- EditHolidayDialog.tsx

Annual Routine Endpoints

GET /api/t3_device/devices/:id/annual-routines

Description: Get all annual routines for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/annual-routines HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Daylight Savings Start",
    "auto_manual": 0,
    "value": 1,
    "month": 3,
    "day": 14
  }
]
```

Used By:

- AnnualRoutineTable.tsx

PUT /api/t3_device/annual-routines/:id

Description: Update annual routine

Parameters:

- `id` (path, integer) - Annual routine ID

Request:

```http
PUT /api/t3_device/annual-routines/1 HTTP/1.1
Content-Type: application/json

{
  "label": "DST Start",
  "month": 3,
  "day": 14,
  "value": 1
}
```

Response:

```json
{
  "success": true,
  "id": 1
}
```

Used By:

- EditAnnualRoutineDialog.tsx

Implementation Details

Database Schema

SQL

```sql
CREATE TABLE SCHEDULES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    output INTEGER DEFAULT 0,
    holiday_id INTEGER DEFAULT 0,
    status TEXT,
    override INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE PROGRAMS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    description TEXT,
    auto_manual INTEGER DEFAULT 0,
    status TEXT DEFAULT 'STOPPED',
    code TEXT,
    size INTEGER DEFAULT 0,
    execution_time INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE PIDS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    input INTEGER,
    setpoint REAL DEFAULT 0,
    output INTEGER,
    proportional REAL DEFAULT 10.0,
    integral REAL DEFAULT 5.0,
    derivative REAL DEFAULT 1.0,
    bias REAL DEFAULT 0.0,
    auto_manual INTEGER DEFAULT 0,
    status TEXT,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE HOLIDAYS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    value INTEGER DEFAULT 0,
    date TEXT,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

```
CREATE TABLE ANNUAL_ROUTINES (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    auto_manual INTEGER DEFAULT 0,
    value INTEGER DEFAULT 0,
    month INTEGER,
    day INTEGER,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

Program Language Syntax

T3000 programs use a BASIC-like syntax:

```BASIC
10 IF IN1 > 72 THEN OUT1 = 100
20 ELSE OUT1 = 0
30 END
```

Commands: IF, THEN, ELSE, FOR, NEXT, GOTO, GOSUB, RETURN, END

Operators: =, <, >, <=, >=, <>, +, -, *, /

References: IN1-IN16, OUT1-OUT8, VAR1-VAR64

PID Tuning Guidelines

- **Proportional**: Initial response strength (typical: 5-20)
- **Integral**: Eliminates steady-state error (typical: 2-10)
- **Derivative**: Dampens oscillations (typical: 0.5-3)

TypeScript Types

TYPESCRIPT

```typescript
export interface Schedule {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  auto_manual: number;
  output: number;
  holiday_id: number;
  status: string;
  override: number;
  times?: ScheduleTimes;
}

export interface Program {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  description: string;
  code: string;
  status: string;
  auto_manual: number;
}

export interface PID {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  input: number;
  setpoint: number;
  output: number;
  proportional: number;
  integral: number;
  derivative: number;
  bias: number;
  auto_manual: number;
  status: string;
}
```

Trend Logging API

Overview

The Trend Logging API manages historical data collection and retrieval. Configure trendlogs to record sensor values, control outputs, and system performance over time.

Common Operations

Get All Trendlogs

```typescript
const trendlogs = await fetch('http://localhost:9103/api/t3_device/devices/1/trendlogs')
  .then(res => res.json());
```

Get Trendlog Data

```typescript
const data = await fetch('http://localhost:9103/api/t3_device/trendlogs/1/data?start=2026-01-01&end=2026-01-09')
  .then(res => res.json());

console.log(data);
// [
//   { timestamp: "2026-01-09T08:00:00Z", value: 72.5 },
//   { timestamp: "2026-01-09T08:05:00Z", value: 72.3 }
// ]
```

Update Trendlog Configuration

```typescript
await fetch('http://localhost:9103/api/t3_device/trendlogs/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    sample_rate: 300,  // 5 minutes
    enabled: true
  })
});
```

Trendlog Endpoints

GET /api/t3_device/devices/:id/trendlogs

Description: Get all trendlogs for a device

Parameters:

- `id` (path, integer) - Device ID

Request:

```http
GET /api/t3_device/devices/1/trendlogs HTTP/1.1
```

Response:

JSON

```
[
  {
    "id": 1,
    "device_id": 1,
    "panel": 5,
    "index": 1,
    "label": "Room Temp Log",
    "input_index": 1,
    "sample_rate": 300,
    "buffer_size": 1000,
    "enabled": true,
    "status": "RUNNING"
  }
]
```

Used By: TrendlogTable.tsx

GET /api/t3_device/trendlogs/:id/data

Description: Get historical data from trendlog

Parameters:

- `id` (path, integer) - Trendlog ID
- `start` (query, string) - Start date (ISO 8601)
- `end` (query, string) - End date (ISO 8601)
- `limit` (query, integer, optional) - Max records (default: 1000)

Request:

HTTP

```
GET /api/t3_device/trendlogs/1/data?start=2026-01-01T00:00:00Z&end=2026-01-09T23:59:59Z&limit=5000 HTTP/1.1
```

Response:

JSON

```
{
  "trendlog_id": 1,
  "label": "Room Temp Log",
  "start": "2026-01-01T00:00:00Z",
  "end": "2026-01-09T23:59:59Z",
  "total_records": 2592,
  "data": [
    {
      "timestamp": "2026-01-09T08:00:00Z",
      "value": 72.5,
      "quality": "GOOD"
    },
    {
      "timestamp": "2026-01-09T08:05:00Z",
      "value": 72.3,
      "quality": "GOOD"
    }
  ]
}
```

Used By: TrendChart.tsx

Errors:

- `404` - Trendlog not found
- `400` - Invalid date range

PUT /api/t3_device/trendlogs/:id

Description: Update trendlog configuration

Parameters:

- `id` (path, integer) - Trendlog ID

Request:

```HTTP
PUT /api/t3_device/trendlogs/1 HTTP/1.1
Content-Type: application/json

{
  "label": "Zone 1 Temperature",
  "sample_rate": 600,
  "enabled": true
}
```

Response:

```JSON
{
  "success": true,
  "id": 1
}
```

Used By: EditTrendlogDialog.tsx

POST /api/t3_device/trendlogs/:id/clear

Description: Clear all data from trendlog

Parameters:

- `id` (path, integer) - Trendlog ID

Request:

```HTTP
POST /api/t3_device/trendlogs/1/clear HTTP/1.1
```

Response:

```JSON
{
  "success": true,
  "records_deleted": 2592
}
```

Used By: ClearTrendlogButton.tsx

GET /api/t3_device/trendlogs/:id/export

Description: Export trendlog data as CSV

Parameters:

- `id` (path, integer) - Trendlog ID
- `start` (query, string) - Start date
- `end` (query, string) - End date

Request:

```http
GET /api/t3_device/trendlogs/1/export?start=2026-01-01&end=2026-01-09 HTTP/1.1
```

Response:

```csv
timestamp,value,quality
2026-01-09T08:00:00Z,72.5,GOOD
2026-01-09T08:05:00Z,72.3,GOOD
```

Used By: ExportTrendlogButton.tsx

Database Schema

```sql
CREATE TABLE TRENDLOGS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    panel INTEGER NOT NULL,
    index_ INTEGER NOT NULL,
    label TEXT,
    input_index INTEGER,
    sample_rate INTEGER DEFAULT 300,
    buffer_size INTEGER DEFAULT 1000,
    enabled INTEGER DEFAULT 1,
    status TEXT,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE TRENDLOG_DATA (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    trendlog_id INTEGER NOT NULL,
    timestamp DATETIME NOT NULL,
    value REAL,
    quality TEXT DEFAULT 'GOOD',
    FOREIGN KEY (trendlog_id) REFERENCES TRENDLOGS(id) ON DELETE CASCADE
);

CREATE INDEX idx_trendlog_data_timestamp ON TRENDLOG_DATA(trendlog_id, timestamp);
```

TypeScript Types

TYPESCRIPT

```typescript
export interface Trendlog {
  id: number;
  device_id: number;
  panel: number;
  index: number;
  label: string;
  input_index: number;
  sample_rate: number;
  buffer_size: number;
  enabled: boolean;
  status: string;
}

export interface TrendlogDataPoint {
  timestamp: string;
  value: number;
  quality: 'GOOD' | 'BAD' | 'UNCERTAIN';
}
```

Events

Event system for alarms, state changes, and notifications.

Event Types

Alarm Events

- Alarm activated
- Alarm acknowledged
- Alarm cleared

Device Events

- Device connected
- Device disconnected
- Communication error

Data Events

- Value changed
- Limit exceeded
- Override activated

Event Structure

JSON

```
{
  "id": "evt_001",
  "type": "alarm",
  "severity": "high",
  "message": "High temperature",
  "timestamp": 1704582000,
  "source": "Zone_Temp",
  "value": 85.2
}
```

Subscribing to Events

Use WebSocket API to receive real-time events.

Event System Architecture

Event Bus Implementation

Core Event Types

TYPESCRIPT

```typescript
interface BaseEvent {
  id: string;
  type: string;
  timestamp: number;
  source: string;
  metadata?: Record<string, any>;
}

interface AlarmEvent extends BaseEvent {
  type: 'alarm' | 'alarm-acknowledged' | 'alarm-cleared';
  alarmId: string;
  deviceId: number;
  point: string;
  severity: 'critical' | 'high' | 'medium' | 'low';
  message: string;
  value?: number;
  threshold?: number;
}

interface DeviceEvent extends BaseEvent {
  type: 'device-connected' | 'device-disconnected' | 'device-error';
  deviceId: number;
  deviceName: string;
  status: 'online' | 'offline' | 'error';
  errorMessage?: string;
}

interface DataEvent extends BaseEvent {
  type: 'value-changed' | 'limit-exceeded' | 'override-activated';
  deviceId: number;
  point: string;
  value: number;
  previousValue?: number;
  limit?: number;
}

type T3000Event = AlarmEvent | DeviceEvent | DataEvent;
```

Event Emitter

TYPESCRIPT

```typescript
class EventEmitter {
  private listeners = new Map<string, Set<EventListener>>();
  private eventHistory: T3000Event[] = [];
  private maxHistorySize = 1000;

  on(eventType: string, listener: EventListener): () => void {
    if (!this.listeners.has(eventType)) {
      this.listeners.set(eventType, new Set());
    }

    this.listeners.get(eventType)!.add(listener);

    // Return unsubscribe function
    return () => {
      this.listeners.get(eventType)?.delete(listener);
    };
  }

  emit(event: T3000Event): void {
    // Add to history
    this.eventHistory.push(event);
    if (this.eventHistory.length > this.maxHistorySize) {
      this.eventHistory.shift();
    }

    // Notify type-specific listeners
    const typeListeners = this.listeners.get(event.type) || new Set();
    for (const listener of typeListeners) {
      this.safeCall(listener, event);
    }

    // Notify wildcard listeners
    const wildcardListeners = this.listeners.get('*') || new Set();
    for (const listener of wildcardListeners) {
      this.safeCall(listener, event);
    }
  }

  private safeCall(listener: EventListener, event: T3000Event): void {
    try {
      listener(event);
    } catch (error) {
      console.error('Event listener error:', error);
    }
  }

  getHistory(filter?: (event: T3000Event) => boolean): T3000Event[] {
    return filter
      ? this.eventHistory.filter(filter)
      : [...this.eventHistory];
  }
}

const eventBus = new EventEmitter();
export default eventBus;
```

Event Persistence

Database Schema

SQL

```sql
CREATE TABLE events (
  id TEXT PRIMARY KEY,
  type TEXT NOT NULL,
  timestamp INTEGER NOT NULL,
  source TEXT NOT NULL,
  device_id INTEGER,
  point TEXT,
  severity TEXT,
  message TEXT,
  data JSON NOT NULL,
  metadata JSON,
  INDEX idx_type_timestamp (type, timestamp),
  INDEX idx_device_timestamp (device_id, timestamp),
  INDEX idx_severity (severity)
);
```

Event Logger

TYPESCRIPT

```typescript
import { Database } from 'sqlite3';

class EventLogger {
  private db: Database;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.setupTables();

    // Subscribe to all events
    eventBus.on('*', (event) => this.logEvent(event));
  }

  private async logEvent(event: T3000Event): Promise<void> {
    const sql = `
      INSERT INTO events (id, type, timestamp, source, device_id, point, severity, message, data, metadata)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;

    const deviceId = 'deviceId' in event ? event.deviceId : null;
    const point = 'point' in event ? event.point : null;
    const severity = 'severity' in event ? event.severity : null;
    const message = 'message' in event ? event.message : null;

    await this.db.run(sql, [
      event.id,
      event.type,
      event.timestamp,
      event.source,
      deviceId,
      point,
      severity,
      message,
      JSON.stringify(event),
      JSON.stringify(event.metadata || {})
    ]);
  }

  async queryEvents(options: EventQueryOptions): Promise<T3000Event[]> {
    let sql = 'SELECT data FROM events WHERE 1=1';
    const params: any[] = [];

    if (options.type) {
      sql += ' AND type = ?';
      params.push(options.type);
    }

    if (options.deviceId) {
      sql += ' AND device_id = ?';
      params.push(options.deviceId);
    }

    if (options.severity) {
      sql += ' AND severity = ?';
      params.push(options.severity);
    }
```

```
    if (options.startTime) {
      sql += ' AND timestamp >= ?';
      params.push(options.startTime);
    }

    if (options.endTime) {
      sql += ' AND timestamp <= ?';
      params.push(options.endTime);
    }

    sql += ' ORDER BY timestamp DESC LIMIT ?';
    params.push(options.limit || 100);

    const rows = await this.db.all(sql, params);
    return rows.map(row => JSON.parse(row.data));
  }
}
```

Event Filtering and Aggregation

Event Filters

TYPESCRIPT

```
class EventFilter {
  static byDevice(deviceId: number) {
    return (event: T3000Event) => {
      return 'deviceId' in event && event.deviceId === deviceId;
    };
  }

  static byPoint(deviceId: number, point: string) {
    return (event: T3000Event) => {
      return 'deviceId' in event &&
             'point' in event &&
             event.deviceId === deviceId &&
             event.point === point;
    };
  }

  static bySeverity(severity: string) {
    return (event: T3000Event) => {
      return 'severity' in event && event.severity === severity;
    };
  }

  static byTimeRange(start: number, end: number) {
    return (event: T3000Event) => {
      return event.timestamp >= start && event.timestamp <= end;
    };
  }
}

// Usage
eventBus.on('alarm', (event) => {
  if (EventFilter.bySeverity('critical')(event)) {
    sendEmailNotification(event);
  }
});
```

Change-of-Value Detection

TYPESCRIPT

```typescript
class COVDetector {
  private lastValues = new Map<string, number>();
  private thresholds = new Map<string, number>();

  setThreshold(point: string, threshold: number): void {
    this.thresholds.set(point, threshold);
  }

  check(deviceId: number, point: string, value: number): void {
    const key = `${deviceId}:${point}`;
    const lastValue = this.lastValues.get(key);
    const threshold = this.thresholds.get(point) || 0.1;

    if (lastValue !== undefined) {
      const change = Math.abs(value - lastValue);

      if (change >= threshold) {
        eventBus.emit({
          id: generateEventId(),
          type: 'value-changed',
          timestamp: Date.now(),
          source: 'cov-detector',
          deviceId,
          point,
          value,
          previousValue: lastValue
        });
      }
    }

    this.lastValues.set(key, value);
  }
}
```

WebSocket Event Streaming

TYPESCRIPT

```typescript
import WebSocket from 'ws';

class EventWebSocketHandler {
  private subscriptions = new Map<WebSocket, EventSubscription>();

  constructor() {
    eventBus.on('*', (event) => this.broadcastEvent(event));
  }

  handleConnection(ws: WebSocket): void {
    this.subscriptions.set(ws, {
      eventTypes: new Set(['*']),
      deviceIds: new Set(),
      severity: null
    });

    ws.on('message', (data) => {
      const message = JSON.parse(data.toString());
      this.handleMessage(ws, message);
    });

    ws.on('close', () => {
      this.subscriptions.delete(ws);
    });
  }

  private handleMessage(ws: WebSocket, message: any): void {
    if (message.type === 'subscribe') {
      const sub = this.subscriptions.get(ws)!;

      if (message.eventTypes) {
        sub.eventTypes = new Set(message.eventTypes);
      }

      if (message.deviceIds) {
        sub.deviceIds = new Set(message.deviceIds);
      }

      if (message.severity) {
        sub.severity = message.severity;
      }
    }
  }

  private broadcastEvent(event: T3000Event): void {
    for (const [ws, sub] of this.subscriptions.entries()) {
      if (this.matchesSubscription(event, sub)) {
        ws.send(JSON.stringify(event));
      }
    }
  }

  private matchesSubscription(event: T3000Event, sub: EventSubscription): boolean {
    // Check event type
    if (!sub.eventTypes.has('*') && !sub.eventTypes.has(event.type)) {
      return false;
    }
```

```
    // Check device ID
    if (sub.deviceIds.size > 0 && 'deviceId' in event) {
      if (!sub.deviceIds.has(event.deviceId)) {
        return false;
      }
    }

    // Check severity
    if (sub.severity && 'severity' in event) {
      if (event.severity !== sub.severity) {
        return false;
      }
    }

    return true;
  }
}
```

Usage Examples

Subscribe to Alarm Events

TYPESCRIPT

```typescript
import eventBus from './eventBus';

// Listen for all alarms
const unsubscribe = eventBus.on('alarm', (event) => {
  console.log(`Alarm: ${event.message}`);

  if (event.severity === 'critical') {
    sendEmailNotification(event);
  }
});

// Unsubscribe later
unsubscribe();
```

Emit Custom Event

TYPESCRIPT

```typescript
eventBus.emit({
  id: generateEventId(),
  type: 'alarm',
  timestamp: Date.now(),
  source: 'temperature-monitor',
  alarmId: 'ALM_001',
  deviceId: 389001,
  point: 'IN1',
  severity: 'high',
  message: 'Temperature exceeded high limit',
  value: 85.2,
  threshold: 80.0
});
```

Query Event History

```typescript
const logger = new EventLogger('./events.db');

const criticalAlarms = await logger.queryEvents({
  type: 'alarm',
  severity: 'critical',
  startTime: Date.now() - 86400000,  // Last 24 hours
  limit: 50
});
```

Next Steps

- REST API
- WebSocket API
- Modbus Protocol

Generic Tables API

Overview

The Generic Tables API provides access to device configuration tables including arrays, user management, graphics screens, alarms, and unit conversions.

Common Operations

Get Table Records

```typescript
const arrays = await fetch('http://localhost:9103/api/t3_device/arrays?device_id=1')
  .then(res => res.json());
```

Get Users

```typescript
const users = await fetch('http://localhost:9103/api/t3_device/users?device_id=1')
  .then(res => res.json());
```

Update User

```typescript
await fetch('http://localhost:9103/api/t3_device/users/1', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'admin',
    password: 'newpassword',
    access_level: 255
  })
});
```

Generic Table Endpoints

GET /api/t3_device/:table

Description: Get records from any table

Parameters:

- `table` (path, string) - Table name (arrays, users, graphics, alarms, etc.)
- `device_id` (query, integer) - Device ID

Supported Tables:

- `arrays`, `users`, `graphics`, `alarms`, `units`, `conversions`, `custom_units`

Request:

```http
GET /api/t3_device/arrays?device_id=1 HTTP/1.1
```

Response:

```json
[
  {
    "id": 1,
    "device_id": 1,
    "label": "Zone Setpoints",
    "values": [70, 72, 68, 75]
  }
]
```

Used By: GenericTableView.tsx

GET /api/t3_device/:table/count

Description: Get record count for table

Parameters:

- `table` (path, string) - Table name
- `device_id` (query, integer) - Device ID

Request:

```http
GET /api/t3_device/arrays/count?device_id=1 HTTP/1.1
```

Response:

```json
{
  "count": 24
}
```

User Management

GET /api/t3_device/users

Get all users for a device.

PUT /api/t3_device/users/:id

Update user credentials and permissions.

POST /api/t3_device/users

Create new user.

DELETE /api/t3_device/users/:id

Delete user.

Graphics Screens

GET /api/t3_device/graphics

Get all graphics screens.

PUT /api/t3_device/graphics/:id

Update graphics screen configuration.

Alarms

GET /api/t3_device/alarms

Get all alarm definitions.

PUT /api/t3_device/alarms/:id

Update alarm configuration.

Database Schema

```sql
CREATE TABLE ARRAYS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    label TEXT,
    values TEXT,  -- JSON array
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE USERS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    username TEXT NOT NULL,
    password TEXT,
    access_level INTEGER DEFAULT 0,
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);

CREATE TABLE GRAPHICS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id INTEGER NOT NULL,
    label TEXT,
    screen_data TEXT,  -- JSON
    FOREIGN KEY (device_id) REFERENCES DEVICES(id) ON DELETE CASCADE
);
```

Database Management API

Overview

The Database Management API provides application-level settings storage, database partitioning, maintenance, and monitoring.

Common Operations

Get Settings

```typescript
const settings = await fetch('http://localhost:9103/api/db_management/settings')
  .then(res => res.json());
```

Save Setting

```typescript
await fetch('http://localhost:9103/api/db_management/settings', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    key: 'theme',
    value: 'dark'
  })
});
```

Get Database Stats

```typescript
const stats = await fetch('http://localhost:9103/api/database/stats')
  .then(res => res.json());

console.log(stats);
// {
//   size: 52428800,
//   tables: 45,
//   total_records: 152340
// }
```

Settings Endpoints

GET /api/db_management/settings

Description: Get all application settings

Request:

```http
GET /api/db_management/settings HTTP/1.1
```

Response:

```JSON
[
  {
    "id": 1,
    "key": "theme",
    "value": "dark",
    "created_at": "2026-01-01T10:00:00Z",
    "updated_at": "2026-01-09T08:00:00Z"
  }
]
```

Used By: SettingsPage.tsx

POST /api/db_management/settings

Description: Create or update setting

Request:

```HTTP
POST /api/db_management/settings HTTP/1.1
Content-Type: application/json

{
  "key": "auto_refresh",
  "value": "true"
}
```

Response:

```JSON
{
  "id": 2,
  "key": "auto_refresh",
  "value": "true"
}
```

DELETE /api/db_management/settings/:key

Description: Delete setting by key

Parameters:

- `key` (path, string) - Setting key

Request:

```HTTP
DELETE /api/db_management/settings/theme HTTP/1.1
```

Response:

```HTTP
HTTP/1.1 204 No Content
```

Database Maintenance

GET /api/database/stats

Description: Get database statistics

Request:

```http
GET /api/database/stats HTTP/1.1
```

Response:

```json
{
  "size": 52428800,
  "tables": 45,
  "total_records": 152340,
  "indexes": 62,
  "fragmentation": 5.2
}
```

Used By: DatabaseStats.tsx

POST /api/db_management/tools/vacuum

Description: Vacuum database to reclaim space

Request:

```http
POST /api/db_management/tools/vacuum HTTP/1.1
```

Response:

```json
{
  "success": true,
  "size_before": 52428800,
  "size_after": 41943040,
  "space_reclaimed": 10485760
}
```

Used By: DatabaseMaintenance.tsx

POST /api/database/cleanup/old

Description: Delete old database files

Request:

```http
POST /api/database/cleanup/old HTTP/1.1
Content-Type: application/json

{
  "days": 30
}
```

Response:

```JSON
{
  "success": true,
  "files_deleted": 12
}
```

GET /api/db_management/partitions

Description: Get partition information

Request:

```HTTP
GET /api/db_management/partitions HTTP/1.1
```

Response:

```JSON
[
  {
    "name": "trendlog_2026_01",
    "table": "TRENDLOG_DATA",
    "start_date": "2026-01-01",
    "end_date": "2026-01-31",
    "records": 124800
  }
]
```

Database Schema

```SQL
CREATE TABLE APP_SETTINGS (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    key TEXT UNIQUE NOT NULL,
    value TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

TypeScript Types

TYPESCRIPT

```typescript
export interface AppSetting {
  id: number;
  key: string;
  value: string;
  created_at: string;
  updated_at: string;
}

export interface DatabaseStats {
  size: number;
  tables: number;
  total_records: number;
  indexes: number;
  fragmentation: number;
}
```

System & Utilities API

Overview

The System & Utilities API provides authentication, health monitoring, file uploads, and system-level operations.

Common Operations

Health Check

```typescript
const health = await fetch('http://localhost:9103/api/health')
  .then(res => res.json());

console.log(health);
// {
//   status: "healthy",
//   version: "1.0.0",
//   uptime: 86400
// }
```

Login

```typescript
const response = await fetch('http://localhost:9103/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'admin',
    password: 'password'
  })
});
```

Upload File

```typescript
const formData = new FormData();
formData.append('file', fileInput.files[0]);

await fetch('http://localhost:9103/api/upload', {
  method: 'POST',
  body: formData
});
```

Health & Status Endpoints

GET /api/health

Description: System health check

Request:

```http
GET /api/health HTTP/1.1
```

Response:

```JSON
{
  "status": "healthy",
  "version": "1.0.0",
  "uptime": 86400,
  "database": "connected",
  "devices_online": 12
}
```

Used By: SystemStatus.tsx

Handler: `api/src/server.rs::health_check`

GET /api/version

Description: Get API version

Request:

```HTTP
GET /api/version HTTP/1.1
```

Response:

```JSON
{
  "version": "1.0.0",
  "build": "20260109",
  "rust_version": "1.75.0"
}
```

Authentication Endpoints

POST /api/login

Description: User login

Request:

```HTTP
POST /api/login HTTP/1.1
Content-Type: application/json

{
  "username": "admin",
  "password": "password"
}
```

Response:

```JSON
{
  "success": true,
  "user": {
    "id": 1,
    "username": "admin",
    "access_level": 255
  },
  "session_token": "abc123..."
}
```

Used By: LoginPage.tsx

Handler: `api/src/auth.rs::login`

Errors:

- `401` - Invalid credentials

POST /api/logout

Description: User logout

Request:

```HTTP
POST /api/logout HTTP/1.1
```

Response:

```JSON
{
  "success": true
}
```

Used By: LogoutButton.tsx

GET /api/session

Description: Check current session

Request:

```HTTP
GET /api/session HTTP/1.1
```

Response:

```JSON
{
  "authenticated": true,
  "user": {
    "id": 1,
    "username": "admin"
  }
}
```

Used By: AuthProvider.tsx

File Upload Endpoints

POST /api/upload

Description: Upload file

Request:

```http
POST /api/upload HTTP/1.1
Content-Type: multipart/form-data

------boundary
Content-Disposition: form-data; name="file"; filename="config.json"

{...file content...}
------boundary--
```

Response:

```json
{
  "success": true,
  "filename": "config.json",
  "path": "/uploads/config.json",
  "size": 2048
}
```

Used By: FileUpload.tsx

Handler: `api/src/file/routes.rs::upload_file`

Errors:

- `400` - No file provided
- `413` - File too large (max 10MB)

GET /api/uploads/:filename

Description: Download uploaded file

Parameters:

- `filename` (path, string) - Filename

Request:

```http
GET /api/uploads/config.json HTTP/1.1
```

Response: File contents with appropriate Content-Type

Used By: FileList.tsx

DELETE /api/uploads/:filename

Description: Delete uploaded file

Parameters:

- `filename` (path, string) - Filename

Request:

```
HTTP
DELETE /api/uploads/config.json HTTP/1.1
```

Response:

```
HTTP
HTTP/1.1 204 No Content
```

Modbus Endpoints

GET /api/modbus/registers

Description: Read Modbus registers

Parameters:

- `device_id` (query, integer) - Device ID
- `start` (query, integer) - Start register
- `count` (query, integer) - Register count

Request:

```
HTTP
GET /api/modbus/registers?device_id=1&start=1000&count=10 HTTP/1.1
```

Response:

```
JSON
{
  "device_id": 1,
  "start": 1000,
  "values": [72, 65, 100, 50, 0, 1, 255, 128, 64, 32]
}
```

Used By: ModbusDebugger.tsx

Handler: `api/src/modbus_register/routes.rs::read_registers`

POST /api/modbus/write

Description: Write Modbus register

Request:

```
HTTP
POST /api/modbus/write HTTP/1.1
Content-Type: application/json

{
  "device_id": 1,
  "register": 1000,
  "value": 75
}
```

Response:

```JSON
{
  "success": true
}
```

Errors:

- `503` - Device offline or Modbus error

FFI Bridge Endpoints

POST /api/ffi/call

Description: Call BACnet FFI function

Request:

```HTTP
POST /api/ffi/call HTTP/1.1
Content-Type: application/json

{
  "function": "read_property",
  "device_id": 123456,
  "object_type": "analog_input",
  "object_instance": 0,
  "property": "present_value"
}
```

Response:

```JSON
{
  "success": true,
  "value": 72.5
}
```

Used By: BACnetDebugger.tsx

Handler: `api/src/t3_shell/routes.rs::call_ffi_function`

WebSocket Connection

WS /ws

Description: WebSocket for real-time updates

Connection:

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  console.log('WebSocket connected');
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
  console.log('Received:', message);
};

// Send message
ws.send(JSON.stringify({
  action: 1,  // SELECT_PANEL
  panelId: 5,
  serialNumber: 123456
}));
```

Message Types:

- `1` - SELECT_PANEL: Select active device
- `2` - UPDATE_POINT: Point value updated
- `3` - DEVICE_STATUS: Device online/offline
- `4` - REFRESH_COMPLETE: Refresh operation done

Used By:

- WebSocketProvider.tsx
- useWebSocket.ts

Handler: `api/src/t3_socket/handler.rs::handle_websocket`

Implementation Details

Session Management

Sessions are stored in memory with 24-hour expiration:

```rust
pub struct SessionManager {
    sessions: Arc<RwLock<HashMap<String, Session>>>,
}

pub struct Session {
    user_id: i32,
    username: String,
    created_at: DateTime<Utc>,
    expires_at: DateTime<Utc>,
}
```

File Upload Limits

- Max file size: 10MB
- Allowed types: All (no restrictions)
- Upload directory: `/uploads`

CORS Configuration

```rust
let cors = CorsLayer::new()
    .allow_origin(Any)
    .allow_methods([Method::GET, Method::POST, Method::PUT, Method::DELETE])
    .allow_headers([header::CONTENT_TYPE]);
```

WebSocket Protocol

Messages are JSON-encoded:

```typescript
interface WebSocketMessage {
  action: number;
  panelId?: number;
  serialNumber?: number;
  data?: any;
}
```

TypeScript Types

```typescript
export interface HealthStatus {
  status: 'healthy' | 'degraded' | 'unhealthy';
  version: string;
  uptime: number;
  database: string;
  devices_online: number;
}

export interface LoginResponse {
  success: boolean;
  user: {
    id: number;
    username: string;
    access_level: number;
  };
  session_token: string;
}

export interface UploadResponse {
  success: boolean;
  filename: string;
  path: string;
  size: number;
}
```

Error Codes

- `UNAUTHORIZED` - Authentication required
- `INVALID_CREDENTIALS` - Wrong username/password
- `SESSION_EXPIRED` - Session timeout
- `FILE_TOO_LARGE` - Upload exceeds 10MB
- `DEVICE_OFFLINE` - Cannot reach device via Modbus/BACnet

Developer Tools API

Overview

The Developer Tools API provides debugging and inspection capabilities including file browsing, database queries, and system logs.

Common Operations

Browse Files

```typescript
const files = await fetch('http://localhost:9103/api/develop/files/list?path=/')
  .then(res => res.json());

console.log(files);
// [
//   { name: "database.db", type: "file", size: 52428800 },
//   { name: "Logs", type: "directory" }
// ]
```

Query Database

```typescript
const result = await fetch('http://localhost:9103/api/develop/database/query', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    sql: 'SELECT * FROM DEVICES LIMIT 10'
  })
}).then(res => res.json());
```

Get Logs

```typescript
const logs = await fetch('http://localhost:9103/api/develop/logs/get?lines=100')
  .then(res => res.json());
```

File Browser Endpoints

GET /api/develop/files/list

Description: List files and directories

Parameters:

- `path` (query, string) - Directory path

Request:

```http
GET /api/develop/files/list?path=/Database HTTP/1.1
```

Response:

```JSON
{
  "path": "/Database",
  "items": [
    {
      "name": "t3000.db",
      "type": "file",
      "size": 52428800,
      "modified": "2026-01-09T08:00:00Z"
    },
    {
      "name": "backups",
      "type": "directory"
    }
  ]
}
```

Used By: FileBrowser.tsx

Handler: `api/src/t3_develop/file_browser/routes.rs::list_files`

GET /api/develop/files/read

Description: Read file contents

Parameters:

- `path` (query, string) - File path

Request:

```HTTP
GET /api/develop/files/read?path=/logs/app.log HTTP/1.1
```

Response:

```TEXT
2026-01-09 08:00:00 INFO Server started on port 9103
2026-01-09 08:00:05 INFO Device 1 connected
```

Used By: FileViewer.tsx

Database Viewer Endpoints

GET /api/develop/database/list

Description: List all database files

Request:

```HTTP
GET /api/develop/database/list HTTP/1.1
```

Response:

```JSON
[
  {
    "name": "t3000.db",
    "path": "/Database/t3000.db",
    "size": 52428800,
    "tables": 45
  }
]
```

Used By: DatabaseViewer.tsx

Handler: `api/src/t3_develop/database_viewer/routes.rs::list_databases`

POST /api/develop/database/query

Description: Execute SQL query

Request:

```HTTP
POST /api/develop/database/query HTTP/1.1
Content-Type: application/json

{
  "sql": "SELECT id, serial_number, product_name FROM DEVICES LIMIT 10",
  "database": "t3000.db"
}
```

Response:

```JSON
{
  "columns": ["id", "serial_number", "product_name"],
  "rows": [
    [1, 123456, "T3-BB"],
    [2, 654321, "T3-LB"]
  ],
  "row_count": 2,
  "execution_time_ms": 5
}
```

Used By: QueryEditor.tsx

Handler: `api/src/t3_develop/database_viewer/routes.rs::execute_query`

Errors:

- `400` - Invalid SQL syntax
- `403` - Forbidden operation (DROP, DELETE without WHERE, etc.)

GET /api/develop/database/schema

Description: Get database schema information

Parameters:

- `database` (query, string) - Database name

Request:

```http
GET /api/develop/database/schema?database=t3000.db HTTP/1.1
```

Response:

```json
{
  "tables": [
    {
      "name": "DEVICES",
      "columns": [
        { "name": "id", "type": "INTEGER", "primary_key": true },
        { "name": "serial_number", "type": "INTEGER", "nullable": false }
      ],
      "indexes": ["idx_devices_serial"]
    }
  ]
}
```

Used By: SchemaViewer.tsx

System Logs Endpoints

GET /api/develop/logs/get

Description: Get application logs

Parameters:

- `lines` (query, integer, optional) - Number of lines (default: 100)
- `level` (query, string, optional) - Filter by level (INFO, WARN, ERROR)

Request:

```http
GET /api/develop/logs/get?lines=50&level=ERROR HTTP/1.1
```

Response:

```json
{
  "lines": [
    {
      "timestamp": "2026-01-09T08:00:00Z",
      "level": "ERROR",
      "message": "Failed to connect to device 192.168.1.100"
    }
  ],
  "total": 50
}
```

Used By: LogViewer.tsx

Handler: `api/src/t3_develop/system_logs/routes.rs::get_logs`

Implementation Details

File Browser Security

File browser is restricted to specific directories:

- `/Database` - Database files
- `/logs` - Log files
- `/uploads` - User uploads

Paths outside these directories are blocked.

Query Safety

SQL queries are validated to prevent:

- DROP, DELETE, UPDATE without WHERE clause
- ATTACH DATABASE
- System table modifications

TypeScript Types

```typescript
export interface FileItem {
  name: string;
  type: 'file' | 'directory';
  size?: number;
  modified?: string;
}

export interface QueryResult {
  columns: string[];
  rows: any[][];
  row_count: number;
  execution_time_ms: number;
}

export interface LogEntry {
  timestamp: string;
  level: 'INFO' | 'WARN' | 'ERROR' | 'DEBUG';
  message: string;
}
```

Error Codes

- `FORBIDDEN_PATH` - Access to path denied
- `FILE_NOT_FOUND` - File doesn't exist
- `INVALID_SQL` - SQL syntax error
- `UNSAFE_QUERY` - Query blocked by safety checks

BACnet Commands Reference

BACnet commands are the low-level protocol operations used to communicate with field devices. T3000 implements the BACnet protocol to read and write data from controllers.

Key Concepts:

- Commands are identified by constants (e.g., READINPUT_T3000)
- Data is read/written in chunks (groups)
- Blocking operations ensure synchronous communication

Overview

T3000's C++ core implements BACnet MS/TP and BACnet/IP protocols for device communication. Each data type has specific read and write commands.

Read Commands

Input Commands

READINPUT_T3000

- **Purpose:** Read analog/digital inputs
- **Structure:** `Str_in_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_INPUT_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_INPUT_GROUP_NUMBER` (8)
- **Usage:**

```C++
GetPrivateDataSaveSPBlocking(
    objectinstance,
    READINPUT_T3000,
    start_index,
    end_index,
    sizeof(Str_in_point),
    4  // retry count
);
```

Output Commands

READOUTPUT_T3000

- **Purpose:** Read analog/digital outputs
- **Structure:** `Str_out_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_OUTPUT_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_OUTPUT_GROUP_NUMBER` (8)

Variable Commands

READVARIABLE_T3000

- **Purpose:** Read user variables
- **Structure:** `Str_variable_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_VARIABLE_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_VARIABLE_GROUP_NUMBER` (8)

Program Commands

READPROGRAM_T3000

- **Purpose:** Read control programs
- **Structure:** `Str_program_point` (sizeof = 37 bytes)
- **Max Count:** `BAC_PROGRAM_ITEM_COUNT` (16)

- **Group Size:** `BAC_READ_PROGRAM_GROUP_NUMBER` (10)

Controller Commands

READCONTROLLER_T3000

- **Purpose:** Read PID controllers
- **Structure:** `Str_controller_point` (sizeof = 24 bytes)
- **Max Count:** `BAC_PID_COUNT` (16)
- **Group Size:** `BAC_READ_PID_GROUP_NUMBER`

Monitor Commands

READMONITOR_T3000

- **Purpose:** Read trendlog monitors
- **Structure:** `Str_monitor_point` (sizeof = 133 bytes)
- **Max Count:** `BAC_MONITOR_COUNT` (12)
- **Group Size:** `BAC_READ_MONITOR_GROUP_NUMBER`

Schedule Commands

READWEEKLYSCHEDULE_T3000

- **Purpose:** Read weekly schedules
- **Structure:** `Str_weekly_routine_point`
- **Max Count:** 32

READANNUALSCHEDULE_T3000

- **Purpose:** Read annual holidays
- **Structure:** `Str_annual_routine_point`
- **Max Count:** 32

Write Commands

WRITEINPUT_T3000

Write input configuration (calibration, range, filter).

WRITEOUTPUT_T3000

Write output values and modes.

WRITEVARIABLE_T3000

Write variable values.

WRITEPROGRAM_T3000

Write program code and control.

WRITECONTROLLER_T3000

Write PID parameters.

WRITEMONITOR_T3000

Write trendlog configuration.

Reading Patterns

Chunked Reading

Data is read in groups to prevent network overload:

```cpp
// Example: Read all programs (16 total) in chunks of 10
int totalCount = 16;
int groupSize = 10;  // BAC_READ_PROGRAM_GROUP_NUMBER
int groups = (totalCount + groupSize - 1) / groupSize;  // = 2 groups

for (int i = 0; i < groups; i++)
{
    int start = i * groupSize;  // Group 0: 0, Group 1: 10
    int end = min(start + groupSize - 1, totalCount - 1);  // Group 0: 9, Group 1: 15

    GetPrivateDataSaveSPBlocking(
        objectinstance,
        READPROGRAM_T3000,
        start,
        end,
        sizeof(Str_program_point),
        4
    );
}
```

Blocking vs Non-Blocking

GetPrivateDataSaveSPBlocking() - Synchronous (used in WebView messages)

- Waits for device response
- Retries on timeout (up to 4 times)
- Returns > 0 on success, < 0 on failure

GetPrivateData_Blocking() - Synchronous (used in refresh threads)

- Similar to above but different retry logic

Post_Refresh_Message() - Asynchronous (used in UI refresh)

- Queues read request
- Returns immediately
- Result handled in callback

Register Mapping

Each BACnet point type maps to specific Modbus registers:

Type	Start Register	Size (registers)
Inputs	`BAC_IN_START_REG` (11472)	23 per input
Outputs	`BAC_OUT_START_REG` (12945)	23 per output
Variables	`BAC_VAR_START_REG` (14417)	23 per variable
Programs	`BAC_PRG_START_REG` (15503)	19 per program
Controllers	`BAC_PID_START_REG` (15807)	12 per controller

Timing Parameters

```cpp
#define SEND_COMMAND_DELAY_TIME 200  // Delay between commands (ms)
```

Recommended delays:

- Between sequential reads: 200ms

- After write commands: 200ms
- Retry timeout: 3000ms (3 seconds)

Error Codes

Return Value	Meaning
> 0	Success (bytes read)
-1	Timeout
-2	Invalid parameters
-3	Device not found
-4	Protocol error

Usage in Control Messages

Example: Action 17 (GET_WEBVIEW_LIST) for Programs

```C++
case BAC_PRG:
{
    // Calculate groups
    int totalCount = entry_index_end - entry_index_start + 1;
    int groupSize = BAC_READ_PROGRAM_GROUP_NUMBER;
    int read_program_group = (totalCount + groupSize - 1) / groupSize;

    for (int i = 0; i < read_program_group; ++i)
    {
        int temp_start = entry_index_start + i * groupSize;
        int temp_end = min(temp_start + groupSize - 1, entry_index_end);

        // Execute BACnet read command
        if (GetPrivateDataSaveSPBlocking(
                entry_objectinstance,
                READPROGRAM_T3000,  // ← BACnet command
                (uint8_t)temp_start,
                (uint8_t)temp_end,
                sizeof(Str_program_point),
                4) > 0)
        {
            // Success - data now in s_Program_data buffer
            // Copy to global array g_Program_data
        }
    }
}
```

Command Constants Definition

Location: Defined in header files

The actual numeric values are defined in the T3000 codebase. Commands are used by name (constant) rather than hardcoded numbers.

See Also

- Data Structures - Point structure definitions
- Message 17: GET_WEBVIEW_LIST - Using read commands
- Platform Overview - Architecture

Data Structures Reference

Data structures define the format and fields of each BACnet point type. Understanding these structures helps you work with device data effectively.

Key Points:

- All structures are C++ structs defined in `ud_str.h`
- Fields are packed for network transmission
- Values often use fixed-point representation (e.g., 720 = 72.0°F)

Overview

T3000 uses C++ structures to represent BACnet point data. These structures mirror the device's memory layout for efficient reading/writing.

Header File: `T3000-Source/T3000/CM5/ud_str.h`

Common Pattern

All point structures follow a similar pattern:

C++

```
typedef struct {
    char description[DESCRIPTION_LENGTH];  // Full description
    char label[LABEL_LENGTH];              // Short label
    // ... type-specific fields ...
    uint8_t auto_manual;                   // 0=Auto, 1=Manual
    uint16_t value;                        // Current value
    uint8_t range;                         // Unit/range
} Str_xxx_point;
```

Input Structure

Str_in_point

Size: 46 bytes **Location:** ud_str.h line ~421 **Max Count:** 64

C++

```
typedef struct {
    union {
        char description[STR_IN_DESCRIPTION_LENGTH+1];  // 21 bytes
    };
    union {
        char label[STR_IN_LABEL_LENGTH+1];  // 9 bytes
    };
    uint8_t auto_manual;        // 0=Auto, 1=Manual
    unsigned short value;       // Current value (scaled)
    unsigned char filter;       // Filter strength (0-5)
    unsigned char control;      // Control type
    unsigned char digital_analog;  // 0=Analog, 1=Digital
    unsigned char range;        // Unit/range index
    signed char calibration_sign;  // Calibration direction
    unsigned short calibration_h;  // High calibration
    unsigned short calibration_l;  // Low calibration
    unsigned char decom;        // 0=Normal, 1=Open, 2=Shorted
    unsigned char sub_id;
    unsigned char sub_product;
    unsigned char unused[4];
} Str_in_point;
```

Field Details

Field	Type	Range	Description
`description`	string	20 chars	Full name
`label`	string	8 chars	Short name
`value`	uint16	0-65535	Scaled value (÷10 for actual)
`auto_manual`	uint8	0-1	0=Auto, 1=Manual
`filter`	uint8	0-5	Filter level
`digital_analog`	uint8	0-1	Point type
`range`	uint8	0-255	Unit selection
`decom`	uint8	0-2	Sensor status

Value Scaling:

- Temperature: value = actual × 10 (720 = 72.0°F)
- Percentage: value = actual × 10 (500 = 50.0%)

Output Structure

Str_out_point

Size: 46 bytes **Max Count:** 64

C++

```
typedef struct {
    char description[21];
    char label[9];
    uint8_t auto_manual;        // 0=Auto, 1=Manual
    unsigned short value;       // Output value
    unsigned char digital_analog;
    unsigned char range;
    unsigned char control;
    unsigned char decom;
    unsigned char low_voltage;
    unsigned short pwm_period;
    unsigned char sub_id;
    unsigned char sub_product;
    unsigned char unused[8];
} Str_out_point;
```

Variable Structure

Str_variable_point

Size: 46 bytes **Max Count:** 64

```C++
typedef struct {
    char description[21];
    char label[9];
    uint8_t auto_manual;
    unsigned short value;
    unsigned char digital_analog;
    unsigned char range;
    unsigned char control;
    unsigned char unused[13];
} Str_variable_point;
```

Program Structure

Str_program_point

Size: 37 bytes **Location:** ud_str.h line 514 **Max Count:** 16

```C++
typedef struct {
    union {
        char description[STR_PRG_DESCRIPTION_LENGTH+1];  // 21 bytes
    };
    union {
        char label[STR_PRG_LABEL_LENGTH+1];  // 9 bytes
    };
    unsigned char bytes[400];   // Program bytecode
    unsigned char on_off;       // 0=Off, 1=On
    unsigned char auto_manual;  // 0=Auto, 1=Manual
    unsigned char com_prg;      // Communication flag
    unsigned char errcode;      // Error code
    unsigned char unused[3];
} Str_program_point;
```

Field Details

Field	Type	Description
`description`	string	Program name
`label`	string	Short name
`bytes`	array	Compiled program code (400 bytes)
`on_off`	uint8	0=Off, 1=On
`auto_manual`	uint8	0=Auto, 1=Manual
`com_prg`	uint8	Communication status
`errcode`	uint8	Last error code

Controller Structure

Str_controller_point

Size: 24 bytes **Location:** ud_str.h line 543 **Max Count:** 16

```C++
typedef struct {
    unsigned char input;        // Input point index
    short input_value;          // Input value
    short value;                // Output value
    unsigned char setpoint;     // Setpoint point index
    short setpoint_value;       // Setpoint value
    unsigned char units;        // Unit type
    unsigned char auto_manual;  // 0=Auto, 1=Manual
    unsigned char action;       // 0=Direct, 1=Reverse
    short proportional;         // P gain
    short reset;                // I gain (integral time)
    char bias;                  // Output bias
    unsigned char rate;         // D gain (derivative time)
} Str_controller_point;
```

PID Parameters

Field	Range	Description
`proportional`	0-100	Proportional gain
`reset`	0-100	Integral time
`rate`	0-100	Derivative time
`bias`	-100 to +100	Output offset
`action`	0-1	0=Direct acting, 1=Reverse acting

Monitor Structure

Str_monitor_point

Size: 133 bytes **Location:** ud_str.h line 639 **Max Count:** 12

```C++
#define MAX_POINTS_IN_MONITOR 14

typedef struct {
    union {
        char label[STR_MONITOR_LABEL_LENGTH+1];  // 9 bytes
    };
    unsigned char inputs[MAX_POINTS_IN_MONITOR];   // Input point indices
    unsigned char range[MAX_POINTS_IN_MONITOR];    // Range for each input
    unsigned char second_interval_time;
    unsigned char minute_interval_time;
    unsigned char hour_interval_time;
    unsigned short max_time_length;   // Maximum log duration
    unsigned char num_inputs;         // Number of inputs configured
    unsigned char an_inputs;          // Number of analog inputs
    unsigned char status;             // 0=Stopped, 1=Running
    unsigned long next_sample_time;   // Next sample timestamp
    unsigned char unused[85];
} Str_monitor_point;
```

Trendlog Timing

Field	Range	Description
`second_interval_time`	0-59	Sample every N seconds
`minute_interval_time`	0-59	Sample every N minutes
`hour_interval_time`	0-23	Sample every N hours
`max_time_length`	0-65535	Max log duration (minutes)

Sample Intervals:

- Fast logging: 1-10 seconds
- Normal: 1-5 minutes
- Slow: 1-24 hours

Global Data Arrays

Structures are stored in vectors indexed by panel and point:

```C++
// Defined in global_variable.h
vector<vector<Str_in_point>> g_Input_data;
vector<vector<Str_out_point>> g_Output_data;
vector<vector<Str_variable_point>> g_Variable_data;
vector<vector<Str_program_point>> g_Program_data;
vector<vector<Str_controller_point>> g_controller_data;
vector<vector<Str_monitor_point>> g_monitor_data;
```

Access pattern:

```C++
// Read input 5 from panel 0
Str_in_point& input = g_Input_data[0].at(5);
int temp = input.value;  // Get value

// Write output 3 on panel 1
g_Output_data[1].at(3).value = 750;  // Set to 75%
```

Temporary Buffers

During reads, data first goes to temporary arrays:

```C++
// Defined globally
Str_in_point s_Input_data[BAC_INPUT_ITEM_COUNT];
Str_out_point s_Output_data[BAC_OUTPUT_ITEM_COUNT];
Str_program_point s_Program_data[BAC_PROGRAM_ITEM_COUNT];
// etc.
```

Read flow:

```TEXT
Device → BACnet → s_Program_data[] → g_Program_data[panel][]
```

Union Usage

Structures use unions for backward compatibility:

```C++
union {
    char description[STR_IN_DESCRIPTION_LENGTH+1];  // 21
};
```

This allows old code to access fields differently while maintaining same memory layout.

Packing and Alignment

Structures are packed to match device memory:

```C++
#pragma pack(push, 1)  // Byte-aligned packing
typedef struct { ... } Str_in_point;
#pragma pack(pop)
```

Important: No padding between fields - exact byte layout matters for network transmission.

See Also

- BACnet Commands - Reading/writing structures
- Message 17: GET_WEBVIEW_LIST - Using structures in messages
- Platform Overview - Architecture

Device Settings (READ_SETTING_COMMAND)

What Are Device Settings?

Device settings control how your T3000 panel operates, including network configuration, communication ports, display options, and more. These settings are stored in a 400-byte structure on the device.

Where to Access Settings

Web Interface:

- Navigate to: `http://localhost:3003/#/t3000/settings`
- View and edit all device configuration
- Changes are sent to the device automatically

API Endpoint:

- Command: `READ_SETTING_COMMAND`
- Returns: 400-byte array with all settings
- Format: Binary data that needs parsing

Common Settings You Can Configure

Network Settings:

- IP Address, Subnet Mask, Gateway
- DHCP or Static IP
- MAC Address (read-only)

Communication Ports:

- COM0, COM1, COM2 configuration
- Baud rates (9600 to 115200)
- Parity and stop bits

Device Identity:

- Panel Name (up to 20 characters)
- Panel Number
- Serial Number
- Device Instance ID

Time Synchronization:

- Time Zone
- NTP Server address
- Auto sync with PC or NTP server

Display Options:

- LCD always on/off
- Panel type
- Customer units

Example: Changing IP Address

What you see in the interface:

```text
IP Address: 192.168.0.144
Subnet: 255.255.255.0
Gateway: 192.168.0.1
```

How it's stored (first 12 bytes):

TEXT

```
[192, 168, 0, 144,    // IP address
 255, 255, 255, 0,    // Subnet mask
 192, 168, 0, 1,      // Gateway
 ...]
```

Binary Data Structure

The device settings are stored as a C/C++ union structure with 400 bytes total. The data can be accessed either as raw bytes (`all[400]`) or as a structured format (`reg`).

C++ Structure Definition

C++

```
typedef union {
    uint8_t all[400];
    struct {
        // Network Configuration (Bytes 0-18)
        uint8_t ip_addr[4];           // Bytes 0-3
        uint8_t subnet[4];            // Bytes 4-7
        uint8_t gate_addr[4];         // Bytes 8-11
        uint8_t mac_addr[6];          // Bytes 12-17
        uint8_t tcp_type;             // Byte 18: 0=DHCP, 1=Static
        uint8_t mini_type;            // Byte 19
        uint8_t debug;                // Byte 20

        // Protocol Info (Bytes 21-37)
        Str_Pro_Info pro_info;        // 17 bytes (21-37)

        // COM Port Configuration (Bytes 38-43)
        uint8_t com0_config;          // Byte 38
        uint8_t com1_config;          // Byte 39
        uint8_t com2_config;          // Byte 40
        uint8_t refresh_flash_timer;  // Byte 41
        uint8_t en_plug_n_play;       // Byte 42
        uint8_t reset_default;        // Byte 43: 88=reset, 77=identify
        uint8_t com_baudrate0;        // Byte 44
        uint8_t com_baudrate1;        // Byte 45
        uint8_t com_baudrate2;        // Byte 46

        // User Settings (Bytes 47-51)
        uint8_t user_name;            // Byte 47: 0=no, 1=disable, 2=enable
        uint8_t custmer_unite;        // Byte 48: 0=no
        uint8_t usb_mode;             // Byte 49: 0=device, 1=host
        uint8_t network_number;       // Byte 50
        uint8_t panel_type;           // Byte 51

        // Panel Name (Bytes 52-71)
        char panel_name[20];          // Bytes 52-71

        // Panel Configuration (Bytes 72-73)
        uint8_t en_panel_name;        // Byte 72
        uint8_t panel_number;         // Byte 73

        // DynDNS Configuration (Bytes 74-169)
        char dyndns_user[DYNDNS_MAX_USERNAME_SIZE];
        char dyndns_pass[DYNDNS_MAX_PASSWORD_SIZE];
        char dyndns_domain[DYNDNS_MAX_DOMAIN_SIZE];
        uint8_t en_dyndns;            // 0=no, 1=disable, 2=enable
        uint8_t dyndns_provider;      // 0=3322.org, 1=dyndns.com, 2=no-ip.com
        uint16_t dyndns_update_time;  // Minutes

        // Time Configuration
        uint8_t en_sntp;              // 0=no, 1=disable
        signed short time_zone;       // Timezone offset
        unsigned int n_serial_number; // Device serial number
        UN_Time update_dyndns;

        // Advanced Settings
        uint16_t mstp_network_number;
        uint8_t BBMD_EN;
```

```
        uint8_t sd_exist;             // 1=no, 2=yes, 3=format error
        unsigned short modbus_port;
        unsigned char modbus_id;
        unsigned int object_instance;
        unsigned int time_update_since_1970;
        unsigned char time_zone_summer_daytime;
        char sntp_server[30];         // NTP server address

        // Device Capabilities
        unsigned char zegbee_exsit;
        unsigned char LCD_Display;    // 1=always on, 0=off
        unsigned char flag_time_sync_pc;
        unsigned char time_sync_auto_manual; // 0=NTP, 1=PC sync
        unsigned char sync_time_results;     // 0=failed, 1=success
        unsigned char mstp_id;
        unsigned short zigbee_panid;
        unsigned char max_master;            // Max 245
        unsigned char special_flag;          // bit0=PT1K, bit1=PT100
        unsigned char uart_parity[3];
        unsigned char uart_stopbit[3];

        lcdconfig display_lcd;
        unsigned char start_month;
        unsigned char start_day;
        unsigned char end_month;
        unsigned char end_day;
        unsigned char network_number_hi;
        unsigned char webview_json_flash;    // 0=old, 2=new JSON
        unsigned char max_var;               // ESP32 only, ST fixed 128
        unsigned char max_in;                // ESP32 only, ST fixed 64
        unsigned char max_out;               // ESP32 only, ST fixed 64
        unsigned char fix_com_config;        // 0=auto, non-0=fixed
        unsigned char write_flash;           // Minutes, 0=disabled
    } reg;
} Str_Setting_Info;
```

TypeScript/JavaScript Parser

Complete Parsing Implementation

TYPESCRIPT

```typescript
interface DeviceSettings {
  // Network
  ipAddress: string;
  subnet: string;
  gateway: string;
  macAddress: string;
  tcpType: 'DHCP' | 'Static';

  // COM Ports
  com0: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };
  com1: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };
  com2: {
    config: number;
    baudrate: number;
    parity: number;
    stopbit: number;
  };

  // Panel Info
  panelName: string;
  panelNumber: number;
  panelType: number;
  serialNumber: number;

  // BACnet
  networkNumber: number;
  objectInstance: number;
  mstpId: number;

  // Time Settings
  timeZone: number;
  sntpServer: string;
  enableSntp: boolean;
  timeSyncMode: 'NTP' | 'PC';

  // Modbus
  modbusPort: number;
  modbusId: number;

  // Display
  lcdDisplay: boolean;

  // Advanced
  usbMode: 'Device' | 'Host';
  debugMode: boolean;
```

```
  plugAndPlay: boolean;
  sdCardExists: boolean;

  // Limits (ESP32)
  maxVar: number;
  maxIn: number;
  maxOut: number;
}

/**
 * Parse 400-byte settings array into DeviceSettings object
 */
export function parseDeviceSettings(data: number[]): DeviceSettings {
  if (data.length !== 400) {
    throw new Error(`Invalid data length: ${data.length}, expected 400 bytes`);
  }

  return {
    // Network Configuration (Bytes 0-18)
    ipAddress: `${data[0]}.${data[1]}.${data[2]}.${data[3]}`,
    subnet: `${data[4]}.${data[5]}.${data[6]}.${data[7]}`,
    gateway: `${data[8]}.${data[9]}.${data[10]}.${data[11]}`,
    macAddress: [12, 13, 14, 15, 16, 17]
      .map(i => data[i].toString(16).padStart(2, '0').toUpperCase())
      .join(':'),
    tcpType: data[18] === 0 ? 'DHCP' : 'Static',

    // COM Port Configuration
    com0: {
      config: data[38],
      baudrate: parseBaudRate(data[44]),
      parity: data[260],  // uart_parity[0]
      stopbit: data[263]  // uart_stopbit[0]
    },
    com1: {
      config: data[39],
      baudrate: parseBaudRate(data[45]),
      parity: data[261],  // uart_parity[1]
      stopbit: data[264]  // uart_stopbit[1]
    },
    com2: {
      config: data[40],
      baudrate: parseBaudRate(data[46]),
      parity: data[262],  // uart_parity[2]
      stopbit: data[265]  // uart_stopbit[2]
    },

    // Panel Information (Bytes 52-73)
    panelName: parseString(data, 52, 20),
    panelNumber: data[73],
    panelType: data[51],

    // Serial Number (4 bytes, little-endian at byte 181)
    serialNumber:
      data[181] |
      (data[182] << 8) |
      (data[183] << 16) |
      (data[184] << 24),

    // Network
```

```
    networkNumber: data[50],
    objectInstance:
      data[198] |
      (data[199] << 8) |
      (data[200] << 16) |
      (data[201] << 24),
    mstpId: data[244],

    // Time Configuration
    timeZone: (data[180] | (data[181] << 8)) as number,  // signed short
    sntpServer: parseString(data, 208, 30),
    enableSntp: data[179] === 2,
    timeSyncMode: data[241] === 0 ? 'NTP' : 'PC',

    // Modbus
    modbusPort: data[194] | (data[195] << 8),
    modbusId: data[196],

    // Display
    lcdDisplay: data[239] === 1,

    // Advanced
    usbMode: data[49] === 0 ? 'Device' : 'Host',
    debugMode: data[20] === 1,
    plugAndPlay: data[42] === 1,
    sdCardExists: data[193] === 2,

    // ESP32 Limits
    maxVar: data[267],
    maxIn: data[268],
    maxOut: data[269]
  };
}

/**
 * Helper: Parse null-terminated string from byte array
 */
function parseString(data: number[], offset: number, maxLength: number): string {
  const bytes = [];
  for (let i = 0; i < maxLength; i++) {
    const byte = data[offset + i];
    if (byte === 0) break;  // Null terminator
    bytes.push(byte);
  }
  return String.fromCharCode(...bytes);
}

/**
 * Helper: Convert baudrate code to actual value
 */
function parseBaudRate(code: number): number {
  const rates = [9600, 19200, 38400, 57600, 115200];
  return rates[code] || 9600;
}

/**
 * Helper: Convert settings object back to 400-byte array
 */
export function encodeDeviceSettings(settings: Partial<DeviceSettings>, original: number[]): number[] {
  const data = [...original];  // Clone original array
```

```
  // Update IP address if changed
  if (settings.ipAddress) {
    const parts = settings.ipAddress.split('.').map(Number);
    data[0] = parts[0];
    data[1] = parts[1];
    data[2] = parts[2];
    data[3] = parts[3];
  }

  // Update subnet if changed
  if (settings.subnet) {
    const parts = settings.subnet.split('.').map(Number);
    data[4] = parts[0];
    data[5] = parts[1];
    data[6] = parts[2];
    data[7] = parts[3];
  }

  // Update gateway if changed
  if (settings.gateway) {
    const parts = settings.gateway.split('.').map(Number);
    data[8] = parts[0];
    data[9] = parts[1];
    data[10] = parts[2];
    data[11] = parts[3];
  }

  // Update TCP type
  if (settings.tcpType !== undefined) {
    data[18] = settings.tcpType === 'DHCP' ? 0 : 1;
  }

  // Update panel name
  if (settings.panelName !== undefined) {
    encodeString(data, 52, settings.panelName, 20);
  }

  // Update NTP server
  if (settings.sntpServer !== undefined) {
    encodeString(data, 208, settings.sntpServer, 30);
  }

  return data;
}

/**
 * Helper: Write string to byte array with null termination
 */
function encodeString(data: number[], offset: number, str: string, maxLength: number): void {
  for (let i = 0; i < maxLength; i++) {
    data[offset + i] = i < str.length ? str.charCodeAt(i) : 0;
  }
}
```

Usage Examples

Example 1: Parse Sample Data

TYPESCRIPT

```typescript
// Sample 400-byte array from device
const sampleData = [
  192, 168, 0, 144,      // IP: 192.168.0.144
  255, 255, 255, 0,      // Subnet: 255.255.255.0
  192, 168, 0, 0,        // Gateway: 192.168.0.0
  0, 14, 198, 242, 224, 195,  // MAC: 00:0E:C6:F2:E0:C3
  0,                     // TCP type: DHCP
  5,                     // Mini type
  0,                     // Debug off
  // ... rest of 400 bytes
];

// Parse the data
const settings = parseDeviceSettings(sampleData);

console.log('Network Configuration:');
console.log(`  IP Address: ${settings.ipAddress}`);
console.log(`  Subnet: ${settings.subnet}`);
console.log(`  Gateway: ${settings.gateway}`);
console.log(`  MAC Address: ${settings.macAddress}`);
console.log(`  Connection Type: ${settings.tcpType}`);

console.log('\nDevice Information:');
console.log(`  Panel Name: ${settings.panelName}`);
console.log(`  Serial Number: ${settings.serialNumber}`);
console.log(`  Panel Number: ${settings.panelNumber}`);

console.log('\nCommunication:');
console.log(`  COM0 Baudrate: ${settings.com0.baudrate}`);
console.log(`  Modbus Port: ${settings.modbusPort}`);
console.log(`  Modbus ID: ${settings.modbusId}`);

console.log('\nTime Settings:');
console.log(`  NTP Server: ${settings.sntpServer}`);
console.log(`  Time Sync: ${settings.timeSyncMode}`);
```

Expected Output:

TEXT

```
Network Configuration:
  IP Address: 192.168.0.144
  Subnet: 255.255.255.0
  Gateway: 192.168.0.0
  MAC Address: 00:0E:C6:F2:E0:C3
  Connection Type: DHCP

Device Information:
  Panel Name: Fandu144-BB
  Serial Number: 1581674
  Panel Number: 144

Communication:
  COM0 Baudrate: 19200
  Modbus Port: 502
  Modbus ID: 1

Time Settings:
  NTP Server: nz.pool.ntp.org
  Time Sync: NTP
```

Example 2: Update Settings

TYPESCRIPT

```typescript
// Read current settings
const currentData = await fetchDeviceSettings(deviceId);
const currentSettings = parseDeviceSettings(currentData);

// Modify specific fields
const updatedSettings = {
  ...currentSettings,
  ipAddress: '192.168.1.100',
  subnet: '255.255.255.0',
  gateway: '192.168.1.1',
  tcpType: 'Static' as const,
  panelName: 'New Panel Name'
};

// Encode back to binary
const updatedData = encodeDeviceSettings(updatedSettings, currentData);

// Send to device
await saveDeviceSettings(deviceId, updatedData);
```

Example 3: React Component

TYPESCRIPT

```typescript
import { useState, useEffect } from 'react';
import { parseDeviceSettings, type DeviceSettings } from './settingsParser';

export function SettingsPage({ deviceId }: { deviceId: number }) {
  const [settings, setSettings] = useState<DeviceSettings | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadSettings();
  }, [deviceId]);

  async function loadSettings() {
    try {
      const response = await fetch(`/api/device/${deviceId}/settings`);
      const data = await response.json();
      const parsed = parseDeviceSettings(data.settings);
      setSettings(parsed);
    } catch (error) {
      console.error('Failed to load settings:', error);
    } finally {
      setLoading(false);
    }
  }

  async function saveSettings() {
    if (!settings) return;

    try {
      const response = await fetch(`/api/device/${deviceId}/settings`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(settings)
      });

      if (response.ok) {
        alert('Settings saved successfully!');
      }
    } catch (error) {
      console.error('Failed to save settings:', error);
    }
  }

  if (loading) return <div>Loading...</div>;
  if (!settings) return <div>Failed to load settings</div>;

  return (
    <div>
      <h1>Device Settings</h1>

      <section>
        <h2>Network Configuration</h2>
        <label>
          IP Address:
          <input
            value={settings.ipAddress}
            onChange={(e) => setSettings({
              ...settings,
```

```
              ipAddress: e.target.value
            })}
          />
        </label>
        <label>
          Connection Type:
          <select
            value={settings.tcpType}
            onChange={(e) => setSettings({
              ...settings,
              tcpType: e.target.value as 'DHCP' | 'Static'
            })}
          >
            <option value="DHCP">DHCP</option>
            <option value="Static">Static</option>
          </select>
        </label>
      </section>

      <section>
        <h2>Device Information</h2>
        <label>
          Panel Name:
          <input
            value={settings.panelName}
            maxLength={20}
            onChange={(e) => setSettings({
              ...settings,
              panelName: e.target.value
            })}
          />
        </label>
        <p>Serial Number: {settings.serialNumber}</p>
        <p>MAC Address: {settings.macAddress}</p>
      </section>

      <button onClick={saveSettings}>Save Settings</button>
    </div>
  );
}
```

Byte Offset Reference Table

Quick lookup for field locations in the 400-byte array:

Offset	Size	Field	Format	Example
0-3	4	IP Address	IPv4	`[192,168,0,144]` = 192.168.0.144
4-7	4	Subnet Mask	IPv4	`[255,255,255,0]`
8-11	4	Gateway	IPv4	`[192,168,0,1]`
12-17	6	MAC Address	Hex	`[00,0E,C6,F2,E0,C3]`
18	1	TCP Type	0=DHCP, 1=Static	`0`
19	1	Mini Type	Module Number	`5` = T3-BB (see below)
20	1	Debug Mode	Boolean	`0`
38	1	COM0 Config	Uint8	`1`
39	1	COM1 Config	Uint8	`0`
40	1	COM2 Config	Uint8	`0`
44	1	COM0 Baudrate	Index	`1` = 19200
45	1	COM1 Baudrate	Index	`0` = 9600
46	1	COM2 Baudrate	Index	`9` = 115200
49	1	USB Mode	0=Device, 1=Host	`0`
50	1	Network Number	Uint8	`255`
51	1	Panel Type	Uint8	`74`
52-71	20	Panel Name	String	`"Fandu144-BB"`
73	1	Panel Number	Uint8	`144`
181-184	4	Serial Number	Uint32 LE	`1581674`
194-195	2	Modbus Port	Uint16 LE	`502`
196	1	Modbus ID	Uint8	`1`
197-200	4	Object Instance	Uint32 LE	`246`
208-237	30	SNTP Server	String	`"nz.pool.ntp.org"`
239	1	LCD Display	1=On, 0=Off	`0`
240	1	Time Sync Flag	Boolean	`1`
241	1	Sync Mode	0=NTP, 1=PC	`1`
244	1	MSTP ID	Uint8	`0`
260-262	3	UART Parity	Uint8[3]	`[0,0,0]`
263-265	3	UART Stop Bit	Uint8[3]	`[0,0,0]`
267	1	Max Variables	Uint8	`255` (or 128)
268	1	Max Inputs	Uint8	`255` (or 64)
269	1	Max Outputs	Uint8	`255` (or 64)

Mini Type (Byte 19) - Module Number Mapping

The **mini_type** field at byte 19 identifies the hardware module type. This maps to user-friendly product names:

Value	Constant	Product Name	Description
0	PRODUCT_CM5	CM5	Original CM5 controller
1	BIG_MINIPANEL	T3-BB (Asix)	BACnet Building Controller (Asix chip)
2	SMALL_MINIPANEL	T3-LB (Asix)	LON Building Controller (Asix chip)
3	TINY_MINIPANEL	T3-TB (Asix)	Tiny Building Controller (Asix chip)
4	TINY_EX_MINIPANEL	T3-TB (Asix)	Tiny Building Controller Extended (Asix chip)
5	MINIPANELARM	**T3-BB**	BACnet Building Controller (ARM chip)
6	MINIPANELARM_LB	**T3-LB**	LON Building Controller (ARM chip)
7	MINIPANELARM_TB	**T3-TB**	Tiny Building Controller (ARM chip)
8	MINIPANELARM_NB	**T3-Nano**	Nano Building Controller
9	T3_TSTAT10	**TSTAT10**	Thermostat
11	T3_OEM	**T3-OEM**	OEM Controller
12	T3_TB_11I	**T3-TB-11I**	Tiny Building Controller with 11 inputs
13	T3_FAN_MODULE	**T3-FAN-MODULE**	Fan control module
14	T3_OEM_12I	**T3-OEM-12I**	OEM Controller with 12 inputs
15	T3_AIRLAB	**T3-AIRLAB**	Air quality lab controller
16	T3_ESP_TRANSDUCER	**T3-ESP-TRANSDUCER**	ESP32-based transducer
17	T3_ESP_TSTAT9	**T3-ESP-TSTAT9**	ESP32-based thermostat
18	T3_ESP_SAUTER	**T3-ESP-SAUTER**	ESP32-based Sauter integration
19	T3_ESP_RMC	**T3-RMC**	Room Management Controller
21	T3_ESP_LW	**T3-ESP-LW**	ESP32-based lighting controller
22	T3_NG2_TYPE2	**T3-NG2**	Next-gen controller type 2
26	T3_3IIC	**T3-3IIC**	3-channel IIC controller
43	PID_T322AI	**T322AI**	22-channel analog input
44	T38AI8AO6DO	**T38AI8AO6DO**	8 AI, 8 AO, 6 DO controller

Code Reference: `T3000-Source/T3000/global_define.h` and `T3000-Source/T3000/BacnetSetting.cpp::Getminitypename()`

Usage Example:

TYPESCRIPT

```typescript
// Parse mini_type from byte 19
const miniType = data[19];

// Map to friendly name
const moduleNames: { [key: number]: string } = {
  0: 'CM5',
  1: 'T3-BB', 2: 'T3-LB', 3: 'T3-TB', 4: 'T3-TB',
  5: 'T3-BB', 6: 'T3-LB', 7: 'T3-TB', 8: 'T3-Nano',
  9: 'TSTAT10', 11: 'T3-OEM', 12: 'T3-TB-11I',
  13: 'T3-FAN-MODULE', 14: 'T3-OEM-12I', 15: 'T3-AIRLAB',
  16: 'T3-ESP-TRANSDUCER', 17: 'T3-ESP-TSTAT9', 18: 'T3-ESP-SAUTER',
  19: 'T3-RMC', 21: 'T3-ESP-LW', 22: 'T3-NG2',
  26: 'T3-3IIC', 43: 'T322AI', 44: 'T38AI8AO6DO',
};

const moduleName = moduleNames[miniType] || `Unknown (${miniType})`;
console.log(`Module: ${moduleName}`); // "Module: T3-BB"
```

Note: Values 1-4 are legacy Asix-based controllers. Modern ARM-based controllers use values 5-8 and above.

Rust API Endpoint

RUST

```rust
// api/src/device_settings/routes.rs

use axum::{extract::Path, Json};
use crate::error::Result;

#[derive(Serialize, Deserialize)]
pub struct SettingsResponse {
    pub device_id: u32,
    pub settings: Vec<u8>,  // 400 bytes
}

/// GET /api/devices/:id/settings
pub async fn get_device_settings(
    Path(device_id): Path<u32>
) -> Result<Json<SettingsResponse>> {
    // Call C++ FFI to read settings
    let settings_data = read_device_settings_ffi(device_id)?;

    Ok(Json(SettingsResponse {
        device_id,
        settings: settings_data
    }))
}

/// POST /api/devices/:id/settings
pub async fn update_device_settings(
    Path(device_id): Path<u32>,
    Json(data): Json<Vec<u8>>
) -> Result<Json<SettingsResponse>> {
    if data.len() != 400 {
        return Err(Error::InvalidDataLength);
    }

    // Call C++ FFI to write settings
    write_device_settings_ffi(device_id, &data)?;

    Ok(Json(SettingsResponse {
        device_id,
        settings: data
    }))
}
```

Frontend Service

TYPESCRIPT

```typescript
// src/services/deviceSettingsService.ts

export class DeviceSettingsService {
  private baseUrl = '/api/devices';

  async getSettings(deviceId: number): Promise<DeviceSettings> {
    const response = await fetch(`${this.baseUrl}/${deviceId}/settings`);
    if (!response.ok) throw new Error('Failed to fetch settings');

    const data = await response.json();
    return parseDeviceSettings(data.settings);
  }

  async updateSettings(
    deviceId: number,
    settings: Partial<DeviceSettings>,
    originalData: number[]
  ): Promise<void> {
    const encoded = encodeDeviceSettings(settings, originalData);

    const response = await fetch(`${this.baseUrl}/${deviceId}/settings`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(encoded)
    });

    if (!response.ok) throw new Error('Failed to update settings');
  }
}
```

Common Issues & Solutions

Issue: String Fields Show Garbage Characters

Problem: Panel name or NTP server shows random characters

Solution: Strings are null-terminated. Always check for null byte (0) when parsing:

TYPESCRIPT

```typescript
function parseString(data: number[], offset: number, maxLength: number): string {
  const bytes = [];
  for (let i = 0; i < maxLength; i++) {
    const byte = data[offset + i];
    if (byte === 0) break;  // Stop at null terminator
    if (byte >= 32 && byte <= 126) {  // Printable ASCII only
      bytes.push(byte);
    }
  }
  return String.fromCharCode(...bytes);
}
```

Issue: Wrong Byte Order for Multi-byte Values

Problem: Serial number or object instance shows wrong value

Solution: Most multi-byte integers use **little-endian** format:

TYPESCRIPT

```typescript
// Correct: Little-endian (LSB first)
const serialNumber =
  data[181] |
  (data[182] << 8) |
  (data[183] << 16) |
  (data[184] << 24);

// Wrong: Big-endian
const wrong =
  (data[181] << 24) |
  (data[182] << 16) |
  (data[183] << 8) |
  data[184];
```

Issue: Settings Not Persisting

Problem: Changes revert after device restart

Solution: Check `write_flash` field (byte 270). If 0, settings aren't saved to flash:

TYPESCRIPT

```typescript
// Enable flash writing every 5 minutes
data[270] = 5;  // Save to flash every 5 minutes
```

See Also

- Device Configuration Guide
- Control Messages
- BACnet Protocol
- REST API Reference

Building Platform Overview

The T3000 Building Platform is the core engine that powers the building automation system. It handles communication between the web interface and physical devices, manages data flow, and executes control commands.

What You'll Find Here:

- System architecture and communication flow
- Control message reference
- BACnet protocol integration
- Data structures and models

Architecture

T3000 uses a three-layer architecture:

TEXT

```
┌─────────────────────────────────────────────────┐
│  Frontend Layer (React/TypeScript)              │
│  - User interface                               │
│  - Real-time visualization                      │
│  - User interactions                            │
└─────────────────┬───────────────────────────────┘
                  │ HTTP/WebSocket
┌─────────────────▼───────────────────────────────┐
│  API Layer (Rust)                               │
│  - REST API endpoints                           │
│  - WebSocket connections                        │
│  - Session management                           │
└─────────────────┬───────────────────────────────┘
                  │ WebView Messages
┌─────────────────▼───────────────────────────────┐
│  Platform Layer (C++ T3000.exe)                 │
│  - BACnet protocol implementation               │
│  - Modbus protocol implementation               │
│  - Device communication                         │
│  - Data caching and persistence                 │
└─────────────────┬───────────────────────────────┘
                  │ BACnet/Modbus
┌─────────────────▼───────────────────────────────┐
│  Field Devices                                  │
│  - Controllers, sensors, actuators              │
└─────────────────────────────────────────────────┘
```

Communication Flow

Request Flow (Frontend → Devices)

1. **User Action** - User clicks button or updates value in web UI
2. **API Request** - React app sends HTTP/WebSocket request to Rust API
3. **WebView Message** - Rust API creates JSON message and sends to C++ via WebView
4. **Message Handler** - C++ `HandleWebViewMsg()` processes message
5. **BACnet Command** - C++ sends BACnet read/write command to device
6. **Device Response** - Device responds with data
7. **JSON Response** - C++ formats response as JSON
8. **API Response** - Rust forwards to frontend
9. **UI Update** - React updates interface

Background Sync Flow

- **Trendlog Collection** - Automatic data logging every minute
- **Device Polling** - Periodic status checks
- **Cache Updates** - Persistent storage of device data
- **Event Notifications** - Real-time alarms and events

Control Messages

Control messages are the primary communication mechanism between the API layer and Platform layer. Each message has:

- **Message Type** - Enum identifier (e.g., GET_WEBVIEW_LIST)
- **Request Parameters** - JSON payload with required data
- **Response Format** - Structured JSON response
- **Error Handling** - Standardized error messages

Message Categories

Category	Purpose	Examples
Data Retrieval	Reading device data	GET_PANEL_DATA, GET_WEBVIEW_LIST
Data Updates	Writing to devices	UPDATE_WEBVIEW_LIST, UPDATE_ENTRY
Graphics	Graphics screen management	LOAD_GRAPHIC_ENTRY, SAVE_GRAPHIC_DATA
Discovery	Device and panel listing	GET_PANELS_LIST, GET_ENTRIES

View All Control Messages →

BACnet Integration

T3000 implements BACnet protocol for device communication:

Read Commands

- `READINPUT_T3000` - Read analog/digital inputs
- `READOUTPUT_T3000` - Read outputs
- `READVARIABLE_T3000` - Read variables
- `READPROGRAM_T3000` - Read programs
- `READCONTROLLER_T3000` - Read PID controllers
- `READMONITOR_T3000` - Read trendlog monitors

Write Commands

- `WRITEINPUT_T3000` - Write input configurations
- `WRITEOUTPUT_T3000` - Write output values
- `WRITEVARIABLE_T3000` - Write variable values
- `WRITEPROGRAM_T3000` - Write program code

Reading Strategy

Data is read in **chunks (groups)** to optimize network traffic:

```C++
// Example: Reading 64 inputs in groups of 8
for (int i = 0; i < 8 groups; i++)
{
    // Read 8 items per iteration
    ReadInputs(start: i*8, end: i*8+7)
}
```

Data Flow

Global Data Arrays

The platform maintains in-memory caches of device data:

```cpp
vector<vector<Str_in_point>> g_Input_data;        // Inputs
vector<vector<Str_out_point>> g_Output_data;      // Outputs
vector<vector<Str_variable_point>> g_Variable_data; // Variables
vector<vector<Str_program_point>> g_Program_data; // Programs
vector<vector<Str_controller_point>> g_controller_data; // Controllers
vector<vector<Str_monitor_point>> g_monitor_data; // Monitors
```

First dimension: Panel ID (0-254) **Second dimension:** Point index (varies by type)

Temporary Buffers

During reads, data is first loaded into temporary buffers:

```cpp
Str_in_point s_Input_data[BAC_INPUT_ITEM_COUNT];
Str_program_point s_Program_data[BAC_PROGRAM_ITEM_COUNT];
```

Then copied to global arrays after successful read.

Data Structures

Common Point Structure Pattern

All BACnet point types follow similar structure:

```cpp
typedef struct {
    char description[DESCRIPTION_LENGTH];  // Full description
    char label[LABEL_LENGTH];              // Short label
    // ... type-specific fields ...
    uint8_t auto_manual;                   // Auto(0) or Manual(1)
    uint16_t value;                        // Current value
    uint8_t range;                         // Unit/range selection
} Str_xxx_point;
```

View Data Structure Reference →

File Organization

Platform Code Location:

```
T3000-Source/
└── T3000/
    ├── BacnetWebView.cpp        - WebView message handler (main entry)
    ├── BacnetWebView.h          - Message type definitions
    ├── global_define.h          - Constants and definitions
    ├── global_variable.h        - Global data arrays
    ├── CM5/ud_str.h             - Data structure definitions
    ├── BacnetInput.cpp          - Input handling
    ├── BacnetOutput.cpp         - Output handling
    ├── BacnetVariable.cpp       - Variable handling
    ├── BacnetProgram.cpp        - Program handling
    ├── BacnetController.cpp     - Controller handling
    └── BacnetMonitor.cpp        - Monitor/trendlog handling
```

Performance Considerations

Caching Strategy

- **In-Memory Cache** - All device data cached in RAM for fast access
- **File Persistence** - Periodic writes to `DatabaseBak/` folder
- **Smart Refresh** - Only read changed data when possible

Network Optimization

- **Chunked Reads** - Large data sets split into manageable groups
- **Retry Logic** - Automatic retry on timeout (up to 4 attempts)
- **Delay Timing** - 200ms delay between commands to prevent flooding

Thread Safety

- **Blocking Reads** - `GetPrivateDataSaveSPBlocking()` ensures synchronous operations
- **Message Queue** - WebView messages queued and processed sequentially

Error Handling

Standard error response format:

```json
{
    "action": "ERROR_RES",
    "error": {
        "code": -1,
        "message": "Read timeout",
        "details": "Failed to read inputs 0-7 from device 123456"
    }
}
```

Common error scenarios:

- **Device offline** - No response within timeout period
- **Invalid range** - Index out of bounds
- **Cache miss** - No cached data available
- **Permission denied** - Device access restricted

Next Steps

- Control Messages Reference →
- Message 17: GET_WEBVIEW_LIST →
- BACnet Commands →

- Data Structures →

Control Messages Index

Control messages are how the web interface communicates with the T3000 platform. Each message performs a specific operation like reading device data, updating values, or managing graphics.

Quick Reference: Jump to any message documentation below.

All Control Messages

Message	Type	Purpose	Common Use
GET_PANEL_DATA	Data Retrieval	Load all cached data for a panel	Initial panel view load
GET_INITIAL_DATA	Graphics	Load graphics screen data	Opening graphics editor
LOAD_GRAPHIC_ENTRY	Graphics	Load specific graphic entry	Graphics screen display
SAVE_GRAPHIC_DATA	Graphics	Save graphics screen changes	Saving graphics edits
GET_WEBVIEW_LIST	Data Retrieval	Read specific entry type with range	Loading inputs, outputs, programs, etc.
UPDATE_WEBVIEW_LIST	Data Update	Write values to device entries	Updating setpoints, outputs, variables
UPDATE_ENTRY	Data Update	Update single entry field	Quick field update
GET_PANELS_LIST	Discovery	Get all online panels	Device list refresh
GET_ENTRIES	Data Retrieval	Get multiple specific entries	Batch data retrieval

Messages by Category

Data Retrieval Messages

Purpose: Reading device data and configuration

- **GET_PANEL_DATA** - Load all cached panel data
- **GET_WEBVIEW_LIST** - Read specific entry types (inputs, outputs, programs, etc.)
- **GET_ENTRIES** - Read multiple specific entries by index

Typical Flow:

1. Frontend requests data via Rust API
2. C++ checks cache first
3. If cache miss, reads from device via BACnet
4. Returns JSON response

Data Update Messages

Purpose: Writing values and configurations to devices

- **UPDATE_WEBVIEW_LIST** - Update multiple entries in bulk
- **UPDATE_ENTRY** - Update single entry field

Typical Flow:

1. Frontend sends new values
2. C++ validates data
3. Writes to device via BACnet
4. Updates local cache
5. Returns success/failure

Graphics Messages

Purpose: Managing graphics screens

- **GET_INITIAL_DATA** - Initial graphics screen load
- **LOAD_GRAPHIC_ENTRY** - Load specific graphics entry

- **SAVE_GRAPHIC_DATA** - Save graphics changes

Graphics Storage:

- Stored as ZIP files in `DatabaseImages/` folder
- Contains PNG images and metadata
- Linked to device panel and entry index

Discovery Messages

Purpose: Finding devices and panels

- **GET_PANELS_LIST** - List all online panels
- **GET_ENTRIES** - Query available entries

Message Format

Request Structure

```JSON
{
  "action": 17,  // Message type number
  "source": 1,   // Source: 0=T3000, 1=WebUI
  "panelId": 0,
  "serialNumber": 12345,
  // ... message-specific parameters ...
}
```

Response Structure

```JSON
{
  "action": "MESSAGE_TYPE_RES",  // Response action name
  "data": {
    // ... response data ...
  }
}
```

Error Response

```JSON
{
  "action": "ERROR_RES",
  "error": {
    "code": -1,
    "message": "Error description",
    "details": "Additional context"
  }
}
```

Implementation Location

File: `T3000-Source/T3000/BacnetWebView.cpp`

Main Handler Function:

C++

```
void CBacnetWebView::HandleWebViewMsg(const CString& receivedMessage)
{
    // Parse JSON
    Json::Value json;
    // ...

    int action = json.get("action", Json::nullValue).asInt();

    switch (action)
    {
        case WEBVIEW_MESSAGE_TYPE::GET_PANEL_DATA:
            // Handle message...
            break;
        case WEBVIEW_MESSAGE_TYPE::GET_WEBVIEW_LIST:
            // Handle message...
            break;
        // ... more cases ...
    }
}
```

Entry Types Reference

Used with GET_WEBVIEW_LIST and UPDATE_WEBVIEW_LIST:

Type	Value	Name	Max Count	Description
BAC_IN	1	Inputs	64	Analog/digital inputs
BAC_OUT	2	Outputs	64	Analog/digital outputs
BAC_VAR	3	Variables	64	User variables
BAC_PRG	6	Programs	16	Control programs
BAC_TL	10	Trendlogs/Monitors	12	Data logging
BAC_PID	11	Controllers	16	PID control loops
BAC_SCH	12	Schedules	32	Weekly schedules
BAC_HOL	13	Holidays	32	Annual holidays
BAC_GRP	23	Graphics	50	Graphics screens
BAC_ALM	9	Alarms	64	Alarm points

Common Parameters

panelId

- Type: Integer
- Range: 0-254
- Description: Device panel number in network

serialNumber

- Type: Integer (unsigned)
- Range: 0-4294967295
- Description: Device unique serial number

objectinstance

- Type: Integer
- Range: 0-4194303

- Description: BACnet object instance ID

entryType

- Type: Integer
- Range: 1-23
- Description: Type of entry (see Entry Types table)

entryIndexStart / entryIndexEnd

- Type: Integer
- Range: 0 to (maxCount - 1)
- Description: Range of entries to read/write

Response Times

Typical response times:

- **Cached data:** < 50ms
- **Fresh read (1-10 items):** 200-500ms
- **Fresh read (full range):** 1-5 seconds
- **Bulk update:** 500ms-3 seconds

Error Codes

Code	Meaning	Common Cause
-1	Read timeout	Device offline or not responding
-2	Invalid parameter	Out of range index or invalid type
-3	Cache miss	No cached data available
-4	Write failed	Device rejected write command
-5	Object mismatch	Wrong device responded

See Also

- Platform Overview →
- BACnet Commands →
- Data Structures →

Graphics Messages: GET_INITIAL_DATA & LOAD_GRAPHIC_ENTRY

Graphics messages load graphic screen configurations and element data. These messages work together to display HMI screens with interactive elements like text boxes, buttons, and live data displays.

When to Use:

- Loading a graphic screen for display
- Initializing the HMI editor
- Fetching element configurations

Overview

Graphics functionality uses two related messages to load HMI screens:

1. **GET_INITIAL_DATA** - Loads complete graphic screen data
2. **LOAD_GRAPHIC_ENTRY** - Loads individual graphic element

Both messages share similar implementation in BacnetWebView.cpp.

Message: GET_INITIAL_DATA

Request Format

Action: `GET_INITIAL_DATA` (constant defined in message types)

JSON Structure:

```JSON
{
  "action": "GET_INITIAL_DATA",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Request Parameters

Parameter	Type	Required	Description
`panel_id`	number	Yes	Target panel ID (0-254)
`graphic_screen_num`	number	Yes	Screen index (1-20)

Response Format

JSON Structure:

JSON


```
{
  "action": "GET_INITIAL_DATA",
  "data": {
    "screen_name": "Main HVAC",
    "background_color": "#FFFFFF",
    "elements": [
      {
        "element_id": 0,
        "type": "TEXT",
        "x": 100,
        "y": 50,
        "width": 200,
        "height": 30,
        "text": "Zone Temperature",
        "font_size": 14,
        "color": "#000000"
      },
      {
        "element_id": 1,
        "type": "LIVE_VALUE",
        "x": 320,
        "y": 50,
        "width": 80,
        "height": 30,
        "point_type": "IN",
        "point_index": 5,
        "format": "%.1f",
        "color": "#0000FF"
      }
    ]
  }
}
```


Implementation

Location: BacnetWebView.cpp line 1227

C++

```cpp
case GET_INITIAL_DATA:
case LOAD_GRAPHIC_ENTRY:
{
    int panel_id;
    int graphic_screen_num;

    // Parse request
    nlohmann::json json_res = nlohmann::json::parse(message);
    panel_id = json_res["panel_id"];
    graphic_screen_num = json_res["graphic_screen_num"];

    // Validate screen number
    if (graphic_screen_num < 1 || graphic_screen_num > MAX_GRPHICS_20)
    {
        return;  // Invalid screen
    }

    // Build response
    nlohmann::json response_json;
    response_json["action"] = (action == GET_INITIAL_DATA) ?
                              "GET_INITIAL_DATA" :
                              "LOAD_GRAPHIC_ENTRY";

    // Get graphic data from global array
    Control_group_point& graphic =
        g_GraphicConfigData[panel_id].at(graphic_screen_num - 1);

    // Serialize graphic data
    nlohmann::json data_json;
    data_json["screen_name"] = graphic.screen_name;
    data_json["background_color"] = graphic.background_color;

    // Serialize elements
    nlohmann::json elements_array = nlohmann::json::array();
    for (int i = 0; i < graphic.element_count; i++)
    {
        Graphic_element& elem = graphic.elements[i];
        nlohmann::json elem_json;

        elem_json["element_id"] = i;
        elem_json["type"] = elem.element_type;
        elem_json["x"] = elem.x_position;
        elem_json["y"] = elem.y_position;
        elem_json["width"] = elem.width;
        elem_json["height"] = elem.height;

        // Type-specific fields
        if (elem.element_type == ELEM_TEXT)
        {
            elem_json["text"] = elem.text_content;
            elem_json["font_size"] = elem.font_size;
        }
        else if (elem.element_type == ELEM_LIVE_VALUE)
        {
            elem_json["point_type"] = elem.point_type;
            elem_json["point_index"] = elem.point_index;
            elem_json["format"] = elem.value_format;
        }
```

```
        elements_array.push_back(elem_json);
    }

    data_json["elements"] = elements_array;
    response_json["data"] = data_json;

    // Send response
    std::string str_json = response_json.dump();
    SendWebViewMessage(str_json.c_str());
}
```

Message: LOAD_GRAPHIC_ENTRY

Purpose: Identical to GET_INITIAL_DATA - loads complete screen

Note: These two messages share the same implementation. The distinction exists for semantic clarity in frontend code.

Use Case Difference:

- `GET_INITIAL_DATA` - Used when first opening HMI editor
- `LOAD_GRAPHIC_ENTRY` - Used when switching between screens

Graphic Screen Structure

Control_group_point

Location: ud_str.h **Max Screens:** 20 (MAX_GRPHICS_20)

C++

```
typedef struct {
    char screen_name[32];          // Screen title
    unsigned char element_count;   // Number of elements (0-64)
    unsigned long background_color; // RGB color
    Graphic_element elements[64];  // Element array
} Control_group_point;
```

Graphic_element

Element Types:

Type	Constant	Description
Text Label	ELEM_TEXT	Static text
Live Value	ELEM_LIVE_VALUE	Real-time point value
Button	ELEM_BUTTON	Clickable control
Image	ELEM_IMAGE	Static graphic
Line	ELEM_LINE	Decorative line
Rectangle	ELEM_RECTANGLE	Shape

C++

```
typedef struct {
    unsigned char element_type;    // Element type constant
    unsigned short x_position;     // X coordinate
    unsigned short y_position;     // Y coordinate
    unsigned short width;
    unsigned short height;

    // Type-specific data
    union {
        struct {  // For ELEM_TEXT
            char text_content[64];
            unsigned char font_size;
            unsigned long text_color;
        };
        struct {  // For ELEM_LIVE_VALUE
            unsigned char point_type;   // BAC_IN, BAC_OUT, etc.
            unsigned char point_index;  // Point number
            char value_format[8];       // Printf format
            unsigned long value_color;
        };
        struct {  // For ELEM_BUTTON
            unsigned char action_type;
            unsigned char target_point;
            char button_text[32];
        };
    };
} Graphic_element;
```

Global Storage

Graphics Configuration Array:

C++

```
// Defined in global_variable.h
vector<vector<Control_group_point>> g_GraphicConfigData;

// Access pattern
Control_group_point& screen = g_GraphicConfigData[panel_id][screen_index];
```

Dimensions:

- Outer vector: panel_id (0-254)
- Inner vector: screen_index (0-19, maps to screen 1-20)

Frontend Usage

Loading Screen for Display

TYPESCRIPT

```typescript
const loadGraphicScreen = async (panelId: number, screenNum: number) => {
  const message = {
    action: 'GET_INITIAL_DATA',
    panel_id: panelId,
    graphic_screen_num: screenNum
  };

  // Send via WebSocket
  sendMessage(JSON.stringify(message));

  // Response handled in message listener
};

// Response handler
webSocket.on('message', (data) => {
  const response = JSON.parse(data);

  if (response.action === 'GET_INITIAL_DATA') {
    const screenData = response.data;

    // Render screen
    renderGraphicScreen(screenData);
  }
});

const renderGraphicScreen = (screenData: any) => {
  // Clear canvas
  canvas.clear();

  // Set background
  canvas.setBackgroundColor(screenData.background_color);

  // Render each element
  screenData.elements.forEach((elem: any) => {
    switch (elem.type) {
      case 'TEXT':
        canvas.addText(elem.x, elem.y, elem.text, {
          fontSize: elem.font_size,
          color: elem.color
        });
        break;

      case 'LIVE_VALUE':
        // Create live value display with point binding
        canvas.addLiveValue(elem.x, elem.y, {
          pointType: elem.point_type,
          pointIndex: elem.point_index,
          format: elem.format,
          color: elem.color
        });
        break;

      case 'BUTTON':
        canvas.addButton(elem.x, elem.y, elem.width, elem.height, {
          text: elem.button_text,
```

```
          onClick: () => handleButtonClick(elem.action_type, elem.target_point)
        });
        break;
    }
  });
};
```

Switching Between Screens

TYPESCRIPT

```typescript
const switchScreen = async (newScreenNum: number) => {
  const message = {
    action: 'LOAD_GRAPHIC_ENTRY',
    panel_id: currentPanelId,
    graphic_screen_num: newScreenNum
  };

  sendMessage(JSON.stringify(message));
};
```

Performance Characteristics

Data Size:

- Small screen (10 elements): ~2 KB JSON
- Large screen (64 elements): ~15 KB JSON
- Average load time: 50-150ms

Caching:

- Graphics loaded on demand
- Frontend should cache loaded screens
- Backend loads from global array (already in memory)

Error Handling

Invalid Screen Number

Request:

JSON

```json
{
  "action": "GET_INITIAL_DATA",
  "panel_id": 0,
  "graphic_screen_num": 25  // Invalid: max is 20
}
```

Result: No response sent (request silently ignored)

Recommendation: Frontend should validate screen numbers before sending.

Empty Screen

Response:

JSON

```
{
  "action": "GET_INITIAL_DATA",
  "data": {
    "screen_name": "",
    "background_color": "#FFFFFF",
    "elements": []  // Empty array
  }
}
```

Related Messages

SAVE_GRAPHIC_DATA

After loading and modifying a screen, use SAVE_GRAPHIC_DATA to persist changes.

Workflow:

TEXT

```
1. GET_INITIAL_DATA → Load screen for editing
2. User modifies elements in HMI editor
3. SAVE_GRAPHIC_DATA → Save modified screen
```

Live Value Updates

Live value elements display real-time data. Use GET_WEBVIEW_LIST or GET_PANEL_DATA to fetch current point values for display.

Example:

TYPESCRIPT

```
// After loading graphic screen with live value elements
const liveElements = screenData.elements.filter(e => e.type === 'LIVE_VALUE');

// Fetch current values
const message = {
  action: 'GET_WEBVIEW_LIST',
  panel_id: panelId,
  entry_type: 'IN',  // For input points
  entry_index_start: 0,
  entry_index_end: 63
};

// Update live values when data arrives
const updateLiveValues = (pointData: any[]) => {
  liveElements.forEach(elem => {
    if (elem.point_type === 'IN') {
      const value = pointData[elem.point_index].value;
      updateElementValue(elem.element_id, value);
    }
  });
};
```

Element Coordinate System

Origin: Top-left corner (0, 0) **Units:** Pixels **Typical Canvas:** 800×600 or 1024×768

Position Calculation:

TYPESCRIPT

```typescript
const centerElement = (canvasWidth: number, elementWidth: number) => {
  return (canvasWidth - elementWidth) / 2;
};

// Example: Center 200px wide element on 800px canvas
const x = centerElement(800, 200);  // = 300
```

See Also

- SAVE_GRAPHIC_DATA - Saving modified screens
- GET_WEBVIEW_LIST - Fetching live point values
- Data Structures - Graphic structure definitions
- Platform Overview - Architecture

Message: GET_PANELS_LIST

GET_PANELS_LIST retrieves all online panels (devices) in the network. Use this to discover available devices and their basic information.

Common Uses:

- Device discovery
- Refreshing device list
- Checking device online status
- Building device selection menus

Overview

Message Type: GET_PANELS_LIST **File Location:** `BacnetWebView.cpp` line 2503 **Purpose:** List all online panels

Request Format

JSON

```
{
  "action": "GET_PANELS_LIST",
  "source": 1
}
```

Parameters

Parameter	Type	Required	Description
`action`	String	✅	Must be "GET_PANELS_LIST"
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI

Response Format

JSON

```
{
  "action": "GET_PANELS_LIST_RES",
  "data": [
    {
      "panel_number": 0,
      "object_instance": 123456,
      "serial_number": 12345,
      "online_time": 1704672000,
      "pid": 19,
      "panel_name": "Controller 1"
    },
    {
      "panel_number": 1,
      "object_instance": 123457,
      "serial_number": 12346,
      "online_time": 1704672100,
      "pid": 19,
      "panel_name": "Controller 2"
    }
  ]
}
```

Response Fields

Field	Type	Description
`panel_number`	Integer	Panel ID (0-254)
`object_instance`	Integer	BACnet object instance
`serial_number`	Integer	Device serial number
`online_time`	Integer	Last response time (Unix timestamp)
`pid`	Integer	Product ID / panel type
`panel_name`	String	Device name

Implementation

C++

```
case WEBVIEW_MESSAGE_TYPE::GET_PANELS_LIST:
{
    tempjson["action"] = "GET_PANELS_LIST_RES";

    int send_index = 0;
    for (int i = 0; i < g_bacnet_panel_info.size(); i++)
    {
        int nret = LoadOnlinePanelData(g_bacnet_panel_info.at(i).panel_number);

        if (nret > 0)
        {
            tempjson["data"][send_index]["panel_number"] =
                g_bacnet_panel_info.at(i).panel_number;
            tempjson["data"][send_index]["object_instance"] =
                g_bacnet_panel_info.at(i).object_instance;
            tempjson["data"][send_index]["serial_number"] =
                g_bacnet_panel_info.at(i).nseiral_number;
            tempjson["data"][send_index]["online_time"] =
                g_bacnet_panel_info.at(i).online_time;
            tempjson["data"][send_index]["pid"] =
                g_bacnet_panel_info.at(i).npid;
            tempjson["data"][send_index]["panel_name"] =
                (char*)g_Device_Basic_Setting[...].reg.panel_name;
            send_index++;
        }
    }

    break;
}
```

Panel Types (PID)

PID	Device Type
19	T3-BB
213	T3-8O
220	T3-LB
316	T3-TB

Online Status

online_time values:

- `0` - Never connected (offline)
- `> 0` - Unix timestamp of last response

Determining if online:

```javascript
const isOnline = (panel) => {
  const now = Math.floor(Date.now() / 1000);
  const diff = now - panel.online_time;
  return diff < 300; // Online if responded within 5 minutes
};
```

Performance

Scenario	Time
1-5 panels	< 100ms
10-20 panels	100-300ms
50+ panels	300-500ms

Usage Example

```typescript
// Request panels list
const response = await fetch('/api/webview/message', {
  method: 'POST',
  body: JSON.stringify({
    action: 'GET_PANELS_LIST',
    source: 1
  })
});

const data = await response.json();

// Filter online panels
const onlinePanels = data.data.filter(panel => {
  const now = Math.floor(Date.now() / 1000);
  return (now - panel.online_time) < 300;
});

// Build device list UI
onlinePanels.forEach(panel => {
  console.log(`${panel.panel_name} (SN: ${panel.serial_number})`);
});
```

See Also

- GET_PANEL_DATA - Load specific panel data
- GET_ENTRIES - Query available entries
- Control Messages Index

Message: GET_PANEL_DATA

GET_PANEL_DATA loads all cached data for a specific panel. This is the fastest way to get a complete snapshot of a device's current state without querying the device directly.

Common Uses:

- Initial panel view load
- Refreshing entire panel data
- Offline data access

Overview

Message Type: GET_PANEL_DATA **File Location:** `BacnetWebView.cpp` line 893 **Purpose:** Load all cached panel data from local storage

Request Format

JSON

```
{
  "action": "GET_PANEL_DATA",
  "source": 1,
  "panelId": 0
}
```

Parameters

Parameter	Type	Required	Description
`action`	String	✅	Must be "GET_PANEL_DATA"
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`panelId`	Integer	✅	Panel number (0-254)

Response Format

JSON

```
{
  "action": "GET_PANEL_DATA_RES",
  "data": {
    "panel_info": {
      "panel_number": 0,
      "serial_number": 12345,
      "object_instance": 123456,
      "panel_name": "Controller 1",
      "panel_type": 19,
      "online_time": 1704672000
    },
    "inputs": [...],
    "outputs": [...],
    "variables": [...],
    "programs": [...],
    "controllers": [...],
    "monitors": [...],
    "schedules": [...],
    "holidays": [...]
  }
}
```

Implementation

Code Location

C++

```
case WEBVIEW_MESSAGE_TYPE::GET_PANEL_DATA:
{
    tempjson["action"] = "GET_PANEL_DATA_RES";
    int npanel_id = json.get("panelId", Json::nullValue).asInt();

    if (npanel_id == 0)
        npanel_id = bac_gloab_panel;

    int nret = LoadOnlinePanelData(npanel_id);

    if (nret < 0)
    {
        CString temp_message;
        temp_message.Format(_T("No cached data found for panel %d"), npanel_id);
        WrapErrorMessage(builder, tempjson, outmsg, temp_message);
        break;
    }

    // Populate all data arrays...
    break;
}
```

Data Loading

1. **Check cache** - Reads from `DatabaseBak/` folder
2. **Load file** - Deserializes saved panel data
3. **Populate arrays** - Fills global arrays (g_Input_data, g_Output_data, etc.)
4. **Build JSON** - Creates response with all data

Performance

Scenario	Time
Cache hit	< 100ms
No cache	Returns error immediately
Large panel (all data)	100-500ms

Error Handling

Cache Miss:

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -3,
    "message": "No cached data found for panel 0"
  }
}
```

Use Cases

Initial Load

Load all data when opening panel view for the first time.

Offline Access

Access cached data when device is offline or unreachable.

Quick Refresh

Get current cached state without network delay.

See Also

- GET_WEBVIEW_LIST - Read fresh data from device
- Control Messages Index

Message: GET_ENTRIES

GET_ENTRIES retrieves multiple specific entries by their exact indices. Unlike GET_WEBVIEW_LIST which reads a continuous range, this message fetches scattered individual entries.

Common Uses:

- Reading specific points (e.g., inputs 0, 5, 12)
- Batch retrieval of non-sequential data
- Updating specific dashboard widgets
- Custom point monitoring

Overview

Message Type: GET_ENTRIES **File Location:** `BacnetWebView.cpp` line 2550 **Purpose:** Read specific entries by exact indices

Request Format

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    {
      "panelId": 0,
      "index": 0,
      "type": 1
    },
    {
      "panelId": 0,
      "index": 5,
      "type": 1
    },
    {
      "panelId": 0,
      "index": 12,
      "type": 2
    }
  ]
}
```

Parameters

Parameter	Type	Required	Description
`action`	String	✅	Must be "GET_ENTRIES"
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`data`	Array	✅	Array of entry specifications

Entry Specification

Field	Type	Required	Description
`panelId`	Integer	✅	Panel number
`index`	Integer	✅	Entry index
`type`	Integer	✅	Entry type (1=IN, 2=OUT, 3=VAR, etc.)

Response Format

JSON


```
{
  "action": "GET_ENTRIES_RES",
  "data": [
    {
      "panelId": 0,
      "type": "INPUT",
      "index": 0,
      "description": "Room Temp",
      "label": "RM_TEMP",
      "value": 720,
      "range": 1
    },
    {
      "panelId": 0,
      "type": "INPUT",
      "index": 5,
      "description": "Outside Temp",
      "label": "OUT_TEMP",
      "value": 450,
      "range": 1
    },
    {
      "panelId": 0,
      "type": "OUTPUT",
      "index": 12,
      "description": "Cooling Valve",
      "label": "COOL_VLV",
      "value": 500,
      "range": 0
    }
  ]
}
```

Implementation

C++

```
case WEBVIEW_MESSAGE_TYPE::GET_ENTRIES:
{
    tempjson["action"] = "GET_ENTRIES_RES";
    Json::Value data = json["data"];

    if (data.size())
    {
        for (int i = 0; i < data.size(); i++)
        {
            Json::Value entry = data[i];
            int npanel_id = entry.get("panelId", -1).asInt();
            int entry_index = entry.get("index", -1).asInt();
            int entry_type = entry.get("type", -1).asInt();

            switch (entry_type)
            {
                case BAC_IN:
                    tempjson["data"][i]["type"] = "INPUT";
                    tempjson["data"][i]["description"] =
                        (char*)g_Input_data[npanel_id].at(entry_index).description;
                    tempjson["data"][i]["value"] =
                        g_Input_data[npanel_id].at(entry_index).value;
                    // ... more fields ...
                    break;

                case BAC_OUT:
                    tempjson["data"][i]["type"] = "OUTPUT";
                    // ... populate fields ...
                    break;

                // ... other types ...
            }
        }
    }

    break;
}
```

Performance

Entries	Time
1-5	< 50ms
10-20	50-150ms
50+	150-500ms

Usage Examples

Example 1: Dashboard Widgets

Read specific points for dashboard display:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 0, "type": 1 },
    { "panelId": 0, "index": 1, "type": 1 },
    { "panelId": 0, "index": 0, "type": 2 },
    { "panelId": 0, "index": 1, "type": 2 }
  ]
}
```

Example 2: Mixed Types

Read different entry types in one request:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 5, "type": 1 },
    { "panelId": 0, "index": 3, "type": 2 },
    { "panelId": 0, "index": 10, "type": 3 },
    { "panelId": 0, "index": 0, "type": 6 }
  ]
}
```

Example 3: Multiple Panels

Read from different panels:

JSON

```
{
  "action": "GET_ENTRIES",
  "source": 1,
  "data": [
    { "panelId": 0, "index": 0, "type": 1 },
    { "panelId": 1, "index": 0, "type": 1 },
    { "panelId": 2, "index": 0, "type": 1 }
  ]
}
```

Comparison: GET_ENTRIES vs GET_WEBVIEW_LIST

Feature	GET_ENTRIES	GET_WEBVIEW_LIST
Selection	Specific indices	Continuous range
Types	Mixed types allowed	Single type only
Best for	Scattered points	Sequential ranges
Request size	Smaller (specific)	Larger (range)
Use case	Dashboards, widgets	Full list views

Use GET_ENTRIES when:

- You need specific scattered points
- Different entry types in one request

- Dashboard widgets with custom point selections

Use GET_WEBVIEW_LIST when:

- You need a continuous range
- Loading full lists (all inputs, all outputs)
- Single entry type operations

Error Handling

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid entry",
    "details": "Entry not found: panel=0, type=1, index=99"
  }
}
```

See Also

- GET_WEBVIEW_LIST - Read continuous ranges
- GET_PANEL_DATA - Load all panel data
- Control Messages Index

Message 5: GET_PANEL_RANGE_INFO

The GET_PANEL_RANGE_INFO message retrieves panel capability and range information. This message helps determine what features a panel supports.

Note: This message is currently not implemented in the C++ backend.

Overview

Action: `GET_PANEL_RANGE_INFO` (5) **Direction:** Frontend → Backend **Location:** Not implemented **Purpose:** Query panel capabilities and supported ranges **Status:** ⚠ Not implemented

Expected Request Format

JSON Structure:

```JSON
{
  "action": "GET_PANEL_RANGE_INFO",
  "panelId": 0
}
```

Expected Response Format

JSON Structure:

```JSON
{
  "action": "GET_PANEL_RANGE_INFO_RES",
  "panelId": 0,
  "data": {
    "inputCount": 64,
    "outputCount": 64,
    "variableCount": 64,
    "programCount": 16,
    "pidCount": 16,
    "scheduleCount": 32,
    "holidayCount": 32,
    "monitorCount": 12,
    "graphicCount": 20
  }
}
```

Implementation Status

This message type is defined in the WEBVIEW_MESSAGE_TYPE enum but does not have a corresponding case handler in BacnetWebView.cpp HandleWebViewMsg function.

To implement:

1. Add case handler for GET_PANEL_RANGE_INFO
2. Read panel configuration from g_Device_Basic_Setting
3. Return count limits for each entry type

Workaround

Until implemented, use hardcoded constants:

TYPESCRIPT

```typescript
const PANEL_RANGES = {
  inputs: 64,
  outputs: 64,
  variables: 64,
  programs: 16,
  controllers: 16,
  schedules: 32,
  holidays: 32,
  monitors: 12,
  graphics: 20
};
```

See Also

- Platform Overview - Architecture
- Data Structures - Entry type constants

Message 8: OPEN_ENTRY_EDIT_WINDOW

The OPEN_ENTRY_EDIT_WINDOW message opens edit dialogs for schedules, holidays, and programs in the T3000 desktop application. This bridges the web interface with native T3000 editing windows.

When to Use:

- User clicks "Edit" button on a schedule
- Double-clicking a holiday entry
- Opening program editor from web interface

Overview

Action: `OPEN_ENTRY_EDIT_WINDOW` (8) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 2807 **Purpose:** Trigger native T3000 edit dialogs from web interface

Request Format

JSON Structure:

```JSON
{
  "action": "OPEN_ENTRY_EDIT_WINDOW",
  "panelId": 0,
  "entryIndex": 5,
  "entryType": 12
}
```

Request Parameters

Parameter	Type	Required	Description
`panelId`	number	Yes	Panel ID (must match local panel)
`entryIndex`	number	Yes	Entry index to edit
`entryType`	number	Yes	Entry type constant

Entry Types

Type	Constant	Value	Editor Opened
Schedule	BAC_SCH	12	Weekly Schedule Editor
Holiday	BAC_HOL	13	Annual Holiday Editor
Program	BAC_PRG	6	Program Code Editor

Response Format

No response - This is a one-way command that triggers UI action

Implementation

Location: BacnetWebView.cpp line 2807

C++

```
case OPEN_ENTRY_EDIT_WINDOW:
{
    tempjson["action"] = "OPEN_ENTRY_EDIT_WINDOW";
    int npanel_id = json.get("panelId", Json::nullValue).asInt();
    int entry_index = json.get("entryIndex", Json::nullValue).asInt();
    int entry_type = json.get("entryType", Json::nullValue).asInt();

    if (npanel_id == bac_gloab_panel) // Only for local panel
    {
        switch (entry_type)
        {
        case BAC_SCH:  // Schedule
        {
            if (entry_index < BAC_SCHEDULE_COUNT)
            {
                weekly_list_line = entry_index;
                WeeklyRoutine_Window->OnBnClickedButtonWeeklyScheduleEdit();
            }
        }
        break;
        case BAC_HOL:  // Holiday
        {
            if (entry_index < BAC_HOLIDAY_COUNT)
            {
                annual_list_line = entry_index;
                AnnualRoutine_Window->OnBnClickedButtonAnnualEdit();
            }
        }
        break;
        case BAC_PRG:  // Program
        {
            if (entry_index < BAC_PROGRAM_ITEM_COUNT)
            {
                program_list_line = entry_index;
                Program_Window->OnBnClickedButtonProgramEdit();
            }
        }
        break;
        default:
            break;
        }
    }
}
```

Behavior

Local Panel Only

The message **only works for the local panel** (npanel_id == bac_gloab_panel). Remote panel editing is not supported.

Editor Windows

Weekly Schedule Editor:

- Maximum 32 schedules (BAC_SCHEDULE_COUNT)
- Sets `weekly_list_line` to entry_index
- Calls `WeeklyRoutine_Window->OnBnClickedButtonWeeklyScheduleEdit()`

Annual Holiday Editor:

- Maximum 32 holidays (BAC_HOLIDAY_COUNT)
- Sets `annual_list_line` to entry_index
- Calls `AnnualRoutine_Window->OnBnClickedButtonAnnualEdit()`

Program Editor:

- Maximum 16 programs (BAC_PROGRAM_ITEM_COUNT)
- Sets `program_list_line` to entry_index
- Calls `Program_Window->OnBnClickedButtonProgramEdit()`

Frontend Usage

Open Schedule Editor

TYPESCRIPT

```typescript
const openScheduleEditor = (panelId: number, scheduleIndex: number) => {
  const message = {
    action: 'OPEN_ENTRY_EDIT_WINDOW',
    panelId: panelId,
    entryIndex: scheduleIndex,
    entryType: 12  // BAC_SCH
  };

  sendMessage(JSON.stringify(message));
};

// Usage
openScheduleEditor(0, 5);  // Edit schedule #5
```

Open Holiday Editor

TYPESCRIPT

```typescript
const openHolidayEditor = (panelId: number, holidayIndex: number) => {
  const message = {
    action: 'OPEN_ENTRY_EDIT_WINDOW',
    panelId: panelId,
    entryIndex: holidayIndex,
    entryType: 13  // BAC_HOL
  };

  sendMessage(JSON.stringify(message));
};
```

Open Program Editor

TYPESCRIPT

```typescript
const openProgramEditor = (panelId: number, programIndex: number) => {
  const message = {
    action: 'OPEN_ENTRY_EDIT_WINDOW',
    panelId: panelId,
    entryIndex: programIndex,
    entryType: 6  // BAC_PRG
  };

  sendMessage(JSON.stringify(message));
};
```

Add Edit Buttons to Tables

TYPESCRIPT

```typescript
const renderEditButton = (entryType: number, entryIndex: number) => {
  return (
    <button
      onClick={() => {
        openEditor(currentPanelId, entryIndex, entryType);
      }}
    >
      Edit
    </button>
  );
};

const openEditor = (panelId: number, index: number, type: number) => {
  const message = {
    action: 'OPEN_ENTRY_EDIT_WINDOW',
    panelId: panelId,
    entryIndex: index,
    entryType: type
  };

  sendMessage(JSON.stringify(message));
};
```

Limitations

Local Panel Only

Not supported:

JSON

```json
{
  "action": "OPEN_ENTRY_EDIT_WINDOW",
  "panelId": 5,  // Remote panel - will be ignored
  "entryIndex": 0,
  "entryType": 12
}
```

The command is silently ignored for remote panels.

No Response

This message does not send a response. The web interface cannot detect:

- Whether the dialog opened successfully
- Whether the user saved or canceled
- What changes were made

Limited Entry Types

Only 3 entry types supported:

- Schedules (BAC_SCH = 12)
- Holidays (BAC_HOL = 13)
- Programs (BAC_PRG = 6)

Other types (inputs, outputs, variables, etc.) are ignored.

Validation

Index Range Checks

TYPESCRIPT

```typescript
const validateEditRequest = (entryType: number, entryIndex: number): boolean => {
  switch (entryType) {
    case 12:  // BAC_SCH
      return entryIndex >= 0 && entryIndex < 32;
    case 13:  // BAC_HOL
      return entryIndex >= 0 && entryIndex < 32;
    case 6:   // BAC_PRG
      return entryIndex >= 0 && entryIndex < 16;
    default:
      return false;
  }
};

// Usage
if (validateEditRequest(entryType, entryIndex)) {
  openEditor(panelId, entryIndex, entryType);
} else {
  showError('Invalid entry type or index');
}
```

Use Cases

Schedule Management

TYPESCRIPT

```typescript
// Schedule table with edit buttons
schedules.forEach((schedule, index) => {
  renderScheduleRow({
    name: schedule.label,
    status: schedule.status,
    onEdit: () => openScheduleEditor(panelId, index)
  });
});
```

Holiday Calendar

TYPESCRIPT

```typescript
// Holiday list with edit capability
holidays.forEach((holiday, index) => {
  renderHolidayItem({
    date: holiday.date,
    name: holiday.name,
    onEdit: () => openHolidayEditor(panelId, index)
  });
});
```

Program Editor Access

TYPESCRIPT

```typescript
// Program table with code editor link
programs.forEach((program, index) => {
  renderProgramRow({
    label: program.label,
    status: program.on_off ? 'Running' : 'Stopped',
    onEditCode: () => openProgramEditor(panelId, index)
  });
});
```

See Also

- Platform Overview - Architecture
- Data Structures - Entry type constants

Message 9: SAVE_IMAGE

The SAVE_IMAGE message uploads graphic image files from the web interface to the T3000 server for use in HMI graphics screens.

When to Use:

- Uploading background images for graphics
- Adding icon graphics
- Importing custom images for HMI screens

Overview

Action: `SAVE_IMAGE` (9) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 2855 **Purpose:** Upload and save image files to server uploads folder

Request Format

JSON Structure:

```JSON
{
  "action": "SAVE_IMAGE",
  "filename": "floor-plan.png",
  "fileLength": 45820,
  "fileData": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

Request Parameters

Parameter	Type	Required	Description
`filename`	string	Yes	Image filename with extension
`fileLength`	number	Yes	File size in bytes
`fileData`	string	Yes	Base64-encoded image data

Supported File Types

Extension	Format	Common Use
.png	PNG	Icons, graphics with transparency
.jpg/.jpeg	JPEG	Photos, backgrounds
.gif	GIF	Simple animations
.bmp	Bitmap	Legacy graphics

Response Format

Success Response:

```JSON
{
  "action": "SAVE_IMAGE_RES",
  "data": {
    "name": "floor-plan.png",
    "path": "/uploads/floor-plan.png"
  }
}
```

Error Response:

JSON

```
{
  "action": "SAVE_IMAGE_RES",
  "error": "Invalid file format"
}
```

Implementation

Location: BacnetWebView.cpp line 2855

C++

```cpp
case SAVE_IMAGE:
{
    const std::string filename = json.get("filename", Json::nullValue).asString();
    int file_length = json.get("fileLength", Json::nullValue).asInt();
    const std::string file_data = Json::writeString(builder, json["fileData"]);

    // Ensure uploads directory exists
    CString web_image_folder = g_strExePth + _T("ResourceFile\\webview\\www\\uploads");

    int ret = FALSE;
    WIN32_FIND_DATA fd;
    HANDLE hFind_folder = FindFirstFile(web_image_folder, &fd);
    if ((hFind_folder != INVALID_HANDLE_VALUE) &&
        (fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY))
    {
        ret = TRUE;
    }
    FindClose(hFind_folder);

    // Create directory if not exists
    if (ret == false)
    {
        SECURITY_ATTRIBUTES attrib;
        attrib.bInheritHandle = FALSE;
        attrib.lpSecurityDescriptor = NULL;
        attrib.nLength = sizeof(SECURITY_ATTRIBUTES);
        CreateDirectory(web_image_folder, &attrib);
    }

    // Handle duplicate filenames
    CString cs_filename = filename.c_str();
    CStringArray temp_array;
    CString temp_file_name_without_suffix;
    CString temp_suffix;
    SplitCStringA(temp_array, cs_filename, _T("."));

    if (temp_array.GetSize() >= 2)
    {
        CString temp_last = temp_array.GetAt(temp_array.GetSize() - 1);
        int temp_length = temp_last.GetLength();
        int total_length = cs_filename.GetLength();
        temp_file_name_without_suffix = cs_filename.Left(total_length - temp_length - 1);
        temp_suffix = temp_last;
    }
    else
    {
        break;  // Invalid filename (no extension)
    }

    // Generate unique filename if duplicate exists
    CString newfilename = cs_filename;
    int ncount = 1;
    while (PathFileExists(web_image_folder + _T("\\") + newfilename))
    {
        newfilename.Format(_T("%s(%d).%s"),
            temp_file_name_without_suffix, ncount++, temp_suffix);
    }
```

```
    // Decode base64 and write file
    CFile file;
    file.Open(web_image_folder + _T("\\") + newfilename,
        CFile::modeCreate | CFile::modeWrite, NULL);

    char* ret_result = (char*)malloc(file_length);
    base64_decode((char const*)temp_image_data.c_str(), ret_result,
        (int)strlen(temp_image_data.c_str()));
    file.write(ret_result, file_length);
    free(ret_result);

    // Send response
    tempjson["action"] = "SAVE_IMAGE_RES";
    tempjson["data"]["name"] = filename;
    tempjson["data"]["path"] = "/uploads/" + newfilename;

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

File Storage

Upload Directory:

```
TEXT
[T3000 Install Path]\ResourceFile\webview\www\uploads\
```

File Access:

- Web path: `/uploads/filename.png`
- Full URL: `http://localhost:3003/uploads/filename.png`

Frontend Usage

Upload Image File

TYPESCRIPT

```typescript
const uploadImage = async (file: File): Promise<string> => {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();

    reader.onload = () => {
      const base64Data = reader.result as string;
      // Remove data URL prefix (e.g., "")
      const base64Content = base64Data.split(',')[1];

      const message = {
        action: 'SAVE_IMAGE',
        filename: file.name,
        fileLength: file.size,
        fileData: base64Content
      };

      sendMessage(JSON.stringify(message));

      // Wait for response
      const handler = (data: string) => {
        const response = JSON.parse(data);

        if (response.action === 'SAVE_IMAGE_RES') {
          if (response.error) {
            reject(new Error(response.error));
          } else {
            resolve(response.data.path);
          }

          webSocket.off('message', handler);
        }
      };

      webSocket.on('message', handler);
    };

    reader.onerror = () => reject(new Error('Failed to read file'));
    reader.readAsDataURL(file);
  });
};

// Usage
const path = await uploadImage(imageFile);
console.log('Image uploaded to:', path);
```

Image Upload Component

TYPESCRIPT

```typescript
const ImageUploader: React.FC = () => {
  const [uploading, setUploading] = useState(false);
  const [uploadedPath, setUploadedPath] = useState<string | null>(null);

  const handleFileSelect = async (event: React.ChangeEvent<HTMLInputElement>) => {
    const file = event.target.files?.[0];
    if (!file) return;

    // Validate file type
    const validTypes = ['image/png', 'image/jpeg', 'image/gif', 'image/bmp'];
    if (!validTypes.includes(file.type)) {
      alert('Invalid file type. Please select an image file.');
      return;
    }

    // Validate file size (e.g., max 5MB)
    const maxSize = 5 * 1024 * 1024;
    if (file.size > maxSize) {
      alert('File too large. Maximum size is 5MB.');
      return;
    }

    try {
      setUploading(true);
      const path = await uploadImage(file);
      setUploadedPath(path);
      alert('Image uploaded successfully!');
    } catch (error) {
      alert(`Upload failed: ${error.message}`);
    } finally {
      setUploading(false);
    }
  };

  return (
    <div>
      <input
        type="file"
        accept="image/*"
        onChange={handleFileSelect}
        disabled={uploading}
      />
      {uploading && <span>Uploading...</span>}
      {uploadedPath && (
        <div>
          <p>Uploaded: {uploadedPath}</p>
          <img src={uploadedPath} alt="Uploaded" style={{ maxWidth: 300 }} />
        </div>
      )}
    </div>
  );
};
```

Duplicate Handling

If a file with the same name exists, the system automatically appends a number:

```text
Original: floor-plan.png
Exists, so saves as: floor-plan(1).png
Exists, so saves as: floor-plan(2).png
...
```

Validation

Client-Side Validation

```typescript
const validateImage = (file: File): { valid: boolean; error?: string } => {
  // Check file type
  const validTypes = ['image/png', 'image/jpeg', 'image/jpg', 'image/gif', 'image/bmp'];
  const fileType = file.type.toLowerCase();

  if (!validTypes.includes(fileType)) {
    return {
      valid: false,
      error: 'Invalid file type. Supported: PNG, JPEG, GIF, BMP'
    };
  }

  // Check file size (max 10MB)
  const maxSize = 10 * 1024 * 1024;
  if (file.size > maxSize) {
    return {
      valid: false,
      error: `File too large (${(file.size / 1024 / 1024).toFixed(2)}MB). Max: 10MB`
    };
  }

  // Check filename
  if (!/^[a-zA-Z0-9_\-\.]+$/.test(file.name)) {
    return {
      valid: false,
      error: 'Invalid filename. Use only letters, numbers, dashes, underscores.'
    };
  }

  return { valid: true };
};

// Usage
const validation = validateImage(file);
if (!validation.valid) {
  alert(validation.error);
  return;
}
```

Performance

Upload Times (approximate):

- Small icon (10KB): < 100ms
- Medium image (500KB): 200-500ms
- Large image (5MB): 1-3 seconds

Recommendations:

- Show upload progress indicator
- Optimize images before upload (compress, resize)
- Use web-optimized formats (PNG for graphics, JPEG for photos)

Error Handling

Common Errors

Error	Cause	Solution
No file extension	Invalid filename	Ensure filename has extension
Directory creation failed	Permission denied	Run T3000 as administrator
Base64 decode failed	Corrupted data	Re-encode file data
File write failed	Disk full	Free up disk space

Retry Logic

TYPESCRIPT

```typescript
const uploadWithRetry = async (
  file: File,
  maxRetries: number = 3
): Promise<string> => {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      return await uploadImage(file);
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;
      }

      // Wait before retry (exponential backoff)
      await new Promise(resolve => setTimeout(resolve, 1000 * attempt));
    }
  }

  throw new Error('Upload failed after retries');
};
```

See Also

- DELETE_IMAGE - Deleting uploaded images
- SAVE_GRAPHIC_DATA - Using images in graphics
- Platform Overview - Architecture

Message 10: SAVE_LIBRARY_DATA

The SAVE_LIBRARY_DATA message saves HVAC equipment library data to a JSON file on the server.

When to Use:

- Saving HVAC equipment library
- Persisting equipment configurations
- Backing up library data

Overview

Action: `SAVE_LIBRARY_DATA` (10) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp (around line 2969) **Purpose:** Save HVAC library JSON file

Request Format

JSON Structure:

```JSON
{
  "action": "SAVE_LIBRARY_DATA",
  "data": {
    "equipment": [
      {
        "name": "AHU-1",
        "type": "Air Handler",
        "inputs": [...]
      }
    ]
  }
}
```

Request Parameters

Parameter	Type	Required	Description
`data`	object	Yes	Complete library data structure

Response Format

JSON Structure:

```JSON
{
  "action": "SAVE_LIBRAY_DATA_RES",
  "status": true
}
```

Implementation

C++

```
case SAVE_LIBRAY_DATA:
{
    CString temp_lib_file = _T("hvac_library.json");
    des_lib_file = image_fordor + _T("\\") + temp_lib_file;
    const std::string file_output = Json::writeString(builder, json["data"]);

    CFile file;
    CString file_temp_cs(file_output.c_str());
    file.Open(des_lib_file, CFile::modeCreate | CFile::modeWrite, NULL);
    file.Write(file_temp_cs, file_temp_cs.GetLength() * 2);
    file.Close();

    Json::Value tempjson;
    tempjson["action"] = "SAVE_LIBRAY_DATA_RES";
    tempjson["status"] = true;

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

File Location: `hvac_library.json`

Frontend Usage

TYPESCRIPT

```
const saveLibrary = async (libraryData: any) => {
  const message = {
    action: 'SAVE_LIBRARY_DATA',
    data: libraryData
  };

  sendMessage(JSON.stringify(message));

  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);
      if (response.action === 'SAVE_LIBRAY_DATA_RES') {
        resolve(response.status);
        webSocket.off('message', handler);
      }
    };
    webSocket.on('message', handler);
  });
};
```

See Also

- SAVE_NEW_LIBRARY_DATA - Save T3 library

Message 11: DELETE_IMAGE

The DELETE_IMAGE message removes previously uploaded graphic image files from the server.

When to Use:

- Cleaning up unused images
- Removing old graphics assets
- Managing storage space

Overview

Action: `DELETE_IMAGE` (11) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3012 **Purpose:** Delete image files from uploads folder

Request Format

JSON Structure:

```JSON
{
  "action": "DELETE_IMAGE",
  "data": "/uploads/floor-plan.png"
}
```

Request Parameters

Parameter	Type	Required	Description
`data`	string	Yes	Relative path to image file

Path Format:

- Must start with `/uploads/`
- Example: `/uploads/image.png`
- Full server path: `ResourceFile\webview\www\uploads\image.png`

Response Format

No response - Silent deletion

Implementation

Location: BacnetWebView.cpp line 3012

C++

```cpp
case DELETE_IMAGE:
{
    const std::string file_path = Json::writeString(builder, json["data"]);
    CString web_image_folder = g_strExePth + _T("ResourceFile\\webview\\www");
    CString temp_delete_file_path;
    CString temp_file_name = file_path.c_str();

    // Trim quotes from JSON string
    temp_file_name = temp_file_name.TrimLeft(_T("\""));
    temp_file_name = temp_file_name.TrimRight(_T("\""));

    // Build full path
    temp_delete_file_path = web_image_folder + temp_file_name;

    // Check if file exists and delete
    CFileFind temp_find;
    if (temp_find.FindFile(temp_delete_file_path))
    {
        DeleteFile(temp_delete_file_path);
    }
}
```

Frontend Usage

Delete Single Image

TYPESCRIPT

```typescript
const deleteImage = (imagePath: string) => {
  const message = {
    action: 'DELETE_IMAGE',
    data: imagePath  // e.g., "/uploads/image.png"
  };

  sendMessage(JSON.stringify(message));
};

// Usage
deleteImage('/uploads/old-image.png');
```

Delete with Confirmation

TYPESCRIPT

```typescript
const deleteImageWithConfirm = async (imagePath: string, imageName: string) => {
  const confirmed = confirm(`Delete image "${imageName}"?`);

  if (confirmed) {
    deleteImage(imagePath);

    // Remove from UI immediately (optimistic update)
    removeImageFromList(imagePath);
  }
};
```

Batch Delete

```typescript
const deleteMultipleImages = (imagePaths: string[]) => {
  imagePaths.forEach(path => {
    deleteImage(path);
  });
};
```

Validation

```typescript
const validateImagePath = (path: string): boolean => {
  // Must start with /uploads/
  if (!path.startsWith('/uploads/')) {
    console.error('Invalid path: must start with /uploads/');
    return false;
  }

  // Check for path traversal attempts
  if (path.includes('..')) {
    console.error('Invalid path: path traversal not allowed');
    return false;
  }

  return true;
};
```

Safety Considerations

No Confirmation

The backend deletes files immediately without confirmation. Implement confirmation dialogs in the frontend.

No Undo

Deleted files cannot be recovered. Consider implementing:

- Confirmation dialogs
- Trash/recycle bin functionality
- Usage tracking (don't delete if used in graphics)

Silent Failure

If file doesn't exist, deletion fails silently. No error is returned.

Use Cases

Graphics Manager

TYPESCRIPT

```typescript
const GraphicsManager: React.FC = () => {
  const [images, setImages] = useState<string[]>([]);

  const handleDelete = (imagePath: string) => {
    if (confirm('Delete this image?')) {
      deleteImage(imagePath);
      setImages(images.filter(img => img !== imagePath));
    }
  };

  return (
    <div>
      {images.map(img => (
        <div key={img}>
          <img src={img} alt="" />
          <button onClick={() => handleDelete(img)}>Delete</button>
        </div>
      ))}
    </div>
  );
};
```

See Also

- SAVE_IMAGE - Uploading images
- SAVE_GRAPHIC_DATA - Using images in graphics

Message 12: GET_SELECTED_DEVICE_INFO

The GET_SELECTED_DEVICE_INFO message retrieves information about the currently selected device in T3000.

When to Use:

- Getting current device context
- Determining active panel details
- Synchronizing frontend with backend device selection

Overview

Action: `GET_SELECTED_DEVICE_INFO` (12) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3028 **Purpose:** Get currently selected device information

Request Format

JSON Structure:

```JSON
{
  "action": "GET_SELECTED_DEVICE_INFO"
}
```

No parameters required

Response Format

JSON Structure:

```JSON
{
  "action": "GET_SELECTED_DEVICE_INFO_RES",
  "data": {
    "product_id": 19,
    "panel_id": 0,
    "serial_number": 237219
  }
}
```

Response Fields

Field	Type	Description
`product_id`	number	Device product type ID
`panel_id`	number	Panel ID (0-254)
`serial_number`	number	Device serial number

Product IDs

Common product IDs:

- **19**: T3-BB (BACnet Building Controller)
- **PM_T322AI**: T3-22AI (22 Universal Inputs)
- **PM_T38AI8AO6DO**: T3-8O (8 Outputs)
- See `defines.h` for complete list

Implementation

Location: BacnetWebView.cpp line 3028

C++

```cpp
case GET_SELECTED_DEVICE_INFO:
{
    Json::Value tempjson;
    tempjson["action"] = "GET_SELECTED_DEVICE_INFO_RES";
    tempjson["data"]["product_id"] = g_selected_product_id;
    tempjson["data"]["panel_id"] = bac_gloab_panel;
    tempjson["data"]["serial_number"] = g_selected_serialnumber;

    const std::string output = Json::writeString(builder, tempjson);
    CString temp_cs(output.c_str());
    outmsg = temp_cs;
}
```

Frontend Usage

Get Current Device

TYPESCRIPT

```typescript
const getCurrentDevice = (): Promise<DeviceInfo> => {
  return new Promise((resolve, reject) => {
    const message = {
      action: 'GET_SELECTED_DEVICE_INFO'
    };

    sendMessage(JSON.stringify(message));

    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'GET_SELECTED_DEVICE_INFO_RES') {
        resolve({
          productId: response.data.product_id,
          panelId: response.data.panel_id,
          serialNumber: response.data.serial_number
        });

        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    // Timeout after 5 seconds
    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Timeout waiting for device info'));
    }, 5000);
  });
};

// Usage
const device = await getCurrentDevice();
console.log('Current device:', device);
```

Device Context Provider

TYPESCRIPT

```typescript
const DeviceContext = React.createContext<DeviceInfo | null>(null);

export const DeviceProvider: React.FC = ({ children }) => {
  const [device, setDevice] = useState<DeviceInfo | null>(null);

  useEffect(() => {
    // Get device info on mount
    getCurrentDevice().then(setDevice);
  }, []);

  return (
    <DeviceContext.Provider value={device}>
      {children}
    </DeviceContext.Provider>
  );
};

// Usage in components
const MyComponent = () => {
  const device = useContext(DeviceContext);

  if (!device) return <div>Loading device info...</div>;

  return (
    <div>
      <h3>Device: {device.serialNumber}</h3>
      <p>Panel: {device.panelId}</p>
    </div>
  );
};
```

Sync on Device Change

TYPESCRIPT

```typescript
const syncDeviceInfo = async () => {
  try {
    const device = await getCurrentDevice();

    // Update application state
    store.dispatch(setCurrentDevice(device));

    // Refresh data for new device
    await loadPanelData(device.panelId);
  } catch (error) {
    console.error('Failed to sync device:', error);
  }
};

// Call when user changes device in T3000
window.addEventListener('deviceChanged', syncDeviceInfo);
```

Use Cases

Device-Specific UI

TYPESCRIPT

```typescript
const DevicePanel: React.FC = () => {
  const [device, setDevice] = useState<DeviceInfo | null>(null);

  useEffect(() => {
    getCurrentDevice().then(setDevice);
  }, []);

  if (!device) return <Spinner />;

  return (
    <div>
      <h2>Panel {device.panelId}</h2>
      <p>Serial: {device.serialNumber}</p>
      <p>Type: {getProductName(device.productId)}</p>
    </div>
  );
};
```

Request Validation

TYPESCRIPT

```typescript
const loadData = async (requestedPanelId: number) => {
  const device = await getCurrentDevice();

  // Verify requested panel matches current device
  if (requestedPanelId !== device.panelId) {
    throw new Error(`Panel mismatch: ${requestedPanelId} vs ${device.panelId}`);
  }

  // Proceed with data load
  await fetchPanelData(requestedPanelId);
};
```

Global Variables

The response uses these C++ global variables:

- `g_selected_product_id` - Current product type
- `bac_gloab_panel` - Active panel ID
- `g_selected_serialnumber` - Device serial number

These are set when user selects a device in T3000 tree view.

See Also

- GET_PANELS_LIST - List all available panels
- Platform Overview - Architecture

Message 13: BIND_DEVICE

The BIND_DEVICE message associates a device with a panel. This message is currently not implemented in the C++ backend.

Note: This message is defined but not yet implemented.

Overview

Action: `BIND_DEVICE` (13) **Direction:** Frontend → Backend **Location:** Not implemented **Purpose:** Bind device to panel **Status:** ⚠ Not implemented

Expected Request Format

JSON Structure:

```JSON
{
  "action": "BIND_DEVICE",
  "panelId": 0,
  "deviceId": 5,
  "serialNumber": 237219
}
```

Expected Response Format

JSON Structure:

```JSON
{
  "action": "BIND_DEVICE_RES",
  "status": "success",
  "panelId": 0,
  "deviceId": 5
}
```

Implementation Status

This message type is defined in the WEBVIEW_MESSAGE_TYPE enum but does not have a corresponding case handler in BacnetWebView.cpp HandleWebViewMsg function.

Expected functionality:

- Associate device with panel
- Update device binding tables
- Notify frontend of binding status

Workaround

Device binding is currently handled through T3000's native UI. Use the tree view to associate devices with panels.

See Also

- GET_SELECTED_DEVICE_INFO - Get current device
- GET_PANELS_LIST - List available panels

Message 14: SAVE_NEW_LIBRARY_DATA

The SAVE_NEW_LIBRARY_DATA message saves the new T3 HVAC equipment library data to a JSON file on the server.

When to Use:

- Saving T3 HVAC library
- Persisting updated equipment configurations
- Backing up library changes

Overview

Action: `SAVE_NEW_LIBRARY_DATA` (14) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 2990 **Purpose:** Save T3 HVAC library JSON file

Request Format

JSON Structure:

```JSON
{
  "action": "SAVE_NEW_LIBRARY_DATA",
  "data": {
    "version": "2.0",
    "equipment": [
      {
        "id": "ahu-001",
        "name": "Air Handler Unit 1",
        "category": "HVAC",
        "points": [...]
      }
    ]
  }
}
```

Request Parameters

Parameter	Type	Required	Description
`data`	object	Yes	Complete T3 library data structure

Response Format

JSON Structure:

```JSON
{
  "action": "SAVE_NEW_LIBRARY_DATA_RES",
  "status": true
}
```

Implementation

Location: BacnetWebView.cpp line 2990

C++

```
case SAVE_NEW_LIBRARY_DATA:
{
    CString temp_lib_file = _T("t3_hvac_library.json");
    des_lib_file = image_fordor + _T("\\") + temp_lib_file;
    const std::string file_output = Json::writeString(builder, json["data"]);

    CFile file;
    CString file_temp_cs(file_output.c_str());
    file.Open(des_lib_file, CFile::modeCreate | CFile::modeWrite, NULL);
    file.Write(file_temp_cs, file_temp_cs.GetLength() * 2);
    file.Close();

    Json::Value tempjson;
    tempjson["action"] = "SAVE_NEW_LIBRARY_DATA_RES";
    tempjson["status"] = true;

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

File Location: `t3_hvac_library.json`

Difference from SAVE_LIBRARY_DATA

Message	File	Purpose
SAVE_LIBRARY_DATA (10)	`hvac_library.json`	Legacy library format
SAVE_NEW_LIBRARY_DATA (14)	`t3_hvac_library.json`	New T3 library format

Frontend Usage

TYPESCRIPT

```typescript
const saveT3Library = async (libraryData: any): Promise<boolean> => {
  const message = {
    action: 'SAVE_NEW_LIBRARY_DATA',
    data: libraryData
  };

  sendMessage(JSON.stringify(message));

  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'SAVE_NEW_LIBRARY_DATA_RES') {
        resolve(response.status);
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Save timeout'));
    }, 5000);
  });
};

// Usage
try {
  const success = await saveT3Library(libraryData);
  if (success) {
    showNotification('Library saved successfully');
  }
} catch (error) {
  showError('Failed to save library');
}
```

Auto-save

```typescript
const LibraryEditor: React.FC = () => {
  const [library, setLibrary] = useState<any>({});
  const [saving, setSaving] = useState(false);

  // Auto-save after 2 seconds of inactivity
  useEffect(() => {
    const timer = setTimeout(async () => {
      setSaving(true);
      try {
        await saveT3Library(library);
      } finally {
        setSaving(false);
      }
    }, 2000);

    return () => clearTimeout(timer);
  }, [library]);

  return (
    <div>
      {saving && <span>Saving...</span>}
      {/* Library editor UI */}
    </div>
  );
};
```

See Also

- SAVE_LIBRARY_DATA - Save legacy library

Message 15: LOGGING_DATA

The LOGGING_DATA message (Action 15) triggers comprehensive data collection from all panel points. It reads inputs, outputs, and variables for logging purposes, similar to GET_WEBVIEW_LIST but for bulk data capture.

Important: This feature has a 1-minute cooldown and is disabled by default in release builds.

Overview

Action: `LOGGING_DATA` (15) **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 3040 **Purpose:** Bulk data collection for all panel points **Status:** ⚠ Disabled by default in release builds

Request Format

JSON Structure:

```JSON
{
  "action": "LOGGING_DATA",
  "panelId": 0,
  "serialNumber": 237219
}
```

Request Parameters

Parameter	Type	Required	Description
`panelId`	number	Yes	Target panel ID
`serialNumber`	number	Yes	Panel serial number

Response Format

JSON Structure:

```JSON
{
  "action": "LOGGING_DATA_RES",
  "data": [
    {
      "panel_id": 0,
      "panel_name": "Main Panel",
      "panel_serial_number": 237219,
      "panel_ipaddress": "192.168.1.100",
      "input_logging_time": "2026-01-08 10:30:00",
      "output_logging_time": "2026-01-08 10:30:05",
      "variable_logging_time": "2026-01-08 10:30:10",
      "inputs": [...],
      "outputs": [...],
      "variables": [...]
    }
  ]
}
```

Implementation

Location: BacnetWebView.cpp line 3040

C++

```cpp
case LOGGING_DATA:
{
    // Disabled in release builds
    if (enable_trendlog_background_read == false)
    {
        break;
    }

    int temp_panel_id = json.get("panelId", Json::nullValue).asInt();
    int temp_serial_number = json.get("serialNumber", Json::nullValue).asInt();

    // Cooldown: 1 minute between requests
    static DWORD last_logging_time = 0;
    DWORD current_time = GetTickCount();
    if (current_time - last_logging_time < 60 * 1000)
    {
        OutputDebugString(_T("LOGGING_DATA SKIPPED - within 1 minute cooldown\n"));
        break;
    }
    last_logging_time = current_time;

    Json::Value tempjson;
    tempjson["action"] = "LOGGING_DATA_RES";

    // Trigger trendlog reading
    int nret = Post_ReadTrendlog_Message(temp_panel_id, temp_serial_number);
    if (nret < 0)
    {
        WrapErrorMessage(builder, tempjson, outmsg, _T("Panel is offline"));
        break;
    }

    // Collect data from all validated panels
    int device_count = 0;
    for (int panel_idx = 0; panel_idx < g_bacnet_panel_info.size(); panel_idx++)
    {
        int npanel_id = g_bacnet_panel_info.at(panel_idx).panel_number;

        // Validation checks
        if (g_bacnet_panel_info.at(panel_idx).object_instance != g_logging_time[npanel_id].bac_instance)
            continue;
        if (g_bacnet_panel_info.at(panel_idx).nseiral_number != g_logging_time[npanel_id].sn)
            continue;
        if (g_bacnet_panel_info.at(panel_idx).panel_number != g_logging_time[npanel_id].n_panel_number)
            continue;
        if (g_logging_time[npanel_id].basic_setting_status != 1)
            continue;

        // Get IP address
        unsigned char* ipAddr = g_Device_Basic_Setting[npanel_id].reg.ip_addr;
        char ipStr[16];
        sprintf(ipStr, "%d.%d.%d.%d", ipAddr[0], ipAddr[1], ipAddr[2], ipAddr[3]);

        // Add device info
        tempjson["data"][device_count]["panel_id"] = npanel_id;
        tempjson["data"][device_count]["panel_name"] = (char*)g_Device_Basic_Setting[npanel_id].reg.panel_name;
        tempjson["data"][device_count]["panel_serial_number"] = g_Device_Basic_Setting[npanel_id].reg.n_serial_number;
        tempjson["data"][device_count]["panel_ipaddress"] = ipStr;
```

```
        tempjson["data"][device_count]["input_logging_time"] = g_logging_time[npanel_id].input_log_time;
        tempjson["data"][device_count]["output_logging_time"] = g_logging_time[npanel_id].output_log_time;
        tempjson["data"][device_count]["variable_logging_time"] = g_logging_time[npanel_id].variable_log_time;

        // Add inputs, outputs, variables data...
        device_count++;
    }

    const std::string output = Json::writeString(builder, tempjson);
    outmsg = CString(output.c_str());
}
```

Behavior

Cooldown Period

1-minute cooldown between requests to prevent server overload. Subsequent requests within 60 seconds are silently ignored.

Release Build Disable

Controlled by `enable_trendlog_background_read` flag:

- **Debug builds:** Typically enabled
- **Release builds:** Disabled by default

Data Collection

Collects from all validated panels:

- Panel basic info (name, serial, IP)
- Last logging timestamps
- All inputs (0-63)
- All outputs (0-63)
- All variables (0-63)

Frontend Usage

TYPESCRIPT

```typescript
const requestLoggingData = async (panelId: number, serialNumber: number) => {
  const message = {
    action: 'LOGGING_DATA',
    panelId: panelId,
    serialNumber: serialNumber
  };

  sendMessage(JSON.stringify(message));

  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'LOGGING_DATA_RES') {
        if (response.error) {
          reject(new Error(response.error));
        } else {
          resolve(response.data);
        }
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);

    setTimeout(() => {
      webSocket.off('message', handler);
      reject(new Error('Logging timeout'));
    }, 30000);  // 30 second timeout
  });
};
```

With Cooldown Management

TYPESCRIPT

```typescript
class LoggingManager {
  private lastRequest: number = 0;
  private readonly COOLDOWN_MS = 60 * 1000;  // 1 minute

  async requestLogging(panelId: number, serialNumber: number) {
    const now = Date.now();
    const timeSinceLastRequest = now - this.lastRequest;

    if (timeSinceLastRequest < this.COOLDOWN_MS) {
      const remaining = this.COOLDOWN_MS - timeSinceLastRequest;
      throw new Error(`Cooldown active. Wait ${Math.ceil(remaining / 1000)}s`);
    }

    this.lastRequest = now;
    return await requestLoggingData(panelId, serialNumber);
  }
}

const logger = new LoggingManager();

// Usage
try {
  const data = await logger.requestLogging(0, 237219);
  console.log('Logging data:', data);
} catch (error) {
  console.error('Logging failed:', error);
}
```

Comparison: Action 15 vs Action 17

Feature	LOGGING_DATA (15)	GET_WEBVIEW_LIST (17)
Purpose	Bulk logging	Selective retrieval
Cooldown	1 minute	None
Data Scope	All panels, all points	Specific range
Release Build	Disabled	Enabled
Use Case	Background logging	UI data display

Recommendation: Use GET_WEBVIEW_LIST (17) for regular data retrieval. Reserve LOGGING_DATA for periodic bulk snapshots.

Limitations

Disabled by Default

Not available in standard release builds. Enable by setting `enable_trendlog_background_read = true`.

Rate Limiting

1-minute cooldown prevents frequent polling. For real-time updates, use GET_WEBVIEW_LIST instead.

Panel Validation

Only includes panels that pass strict validation:

- Instance match
- Serial number match
- Panel number match

- Basic settings loaded

See Also

- GET_WEBVIEW_LIST - Selective data retrieval (Action 17)
- GET_PANEL_DATA - Cached panel data
- Platform Overview - Architecture

Message 17: GET_WEBVIEW_LIST

GET_WEBVIEW_LIST retrieves specific data from BACnet devices. Use this message to read inputs, outputs, variables, programs, PID controllers, or monitors from a device.

Common Uses:

- Loading input values for display
- Reading output states
- Fetching program status
- Getting PID controller setpoints
- Retrieving trendlog configurations

Overview

Message Type: 17 **Action Name:** `GET_WEBVIEW_LIST` **File Location:** `BacnetWebView.cpp` line 1614 **Purpose:** Selective data retrieval with index range support

Request Format

JSON

```
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "entryIndexStart": 0,
  "entryIndexEnd": 15,
  "objectinstance": 123456
}
```

Parameters

Parameter	Type	Required	Description
`action`	Integer	✅	Must be 17
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`panelId`	Integer	✅	Panel number (0-254)
`serialNumber`	Integer	✅	Device serial number
`entryType`	Integer	✅	Type of entry to read (see Entry Types)
`entryIndexStart`	Integer	✅	Starting index (0-based)
`entryIndexEnd`	Integer	✅	Ending index (inclusive)
`objectinstance`	Integer	✅	BACnet object instance

Entry Types

Type	Value	Name	Max Count	Group Size	Command
BAC_IN	1	Inputs	64	8	READINPUT_T3000
BAC_OUT	2	Outputs	64	8	READOUTPUT_T3000
BAC_VAR	3	Variables	64	8	READVARIABLE_T3000
BAC_PRG	6	Programs	16	10	READPROGRAM_T3000
BAC_PID	11	Controllers	16	varies	READCONTROLLER_T3000
BAC_TL	10	Monitors	12	varies	READMONITOR_T3000

Response Format

JSON

```
{
  "action": "GET_WEBVIEW_LIST_RES",
  "data": {
    "panelId": 0,
    "serialNumber": 12345,
    "entryType": 6,
    "entryIndexStart": 0,
    "entryIndexEnd": 15,
    "objectinstance": 123456,
    "device_data": [
      {
        "pid": 0,
        "type": "PROGRAM",
        "index": 0,
        "id": "PRG1",
        "command": "0PRG1",
        "description": "Program 1",
        "label": "PRG1",
        "on_off": 1,
        "auto_manual": 0,
        "com_prg": 0,
        "errcode": 0
      },
      {
        "pid": 0,
        "type": "PROGRAM",
        "index": 1,
        "id": "PRG2",
        "command": "0PRG2",
        "description": "Program 2",
        "label": "PRG2",
        "on_off": 0,
        "auto_manual": 1,
        "com_prg": 0,
        "errcode": 0
      }
      // ... more entries ...
    ]
  }
}
```

Response Fields by Entry Type

BAC_IN (Inputs)

JSON

```
{
  "pid": 0,
  "type": "INPUT",
  "index": 0,
  "id": "IN1",
  "command": "0IN1",
  "description": "Room Temperature",
  "label": "ROOM_TEMP",
  "value": 720,           // 72.0°F (value * 0.1)
  "auto_manual": 0,       // 0=Auto, 1=Manual
  "filter": 3,
  "control": 1,
  "digital_analog": 0,    // 0=Analog, 1=Digital
  "range": 1,             // Unit type
  "calibration_sign": 0,
  "calibration_h": 0,
  "calibration_l": 0,
  "decom": 0              // 0=Normal, 1=Open, 2=Shorted
}
```

BAC_OUT (Outputs)

JSON

```
{
  "pid": 0,
  "type": "OUTPUT",
  "index": 0,
  "id": "OUT1",
  "command": "0OUT1",
  "description": "Cooling Valve",
  "label": "COOL_VLV",
  "value": 500,           // 50.0%
  "auto_manual": 0,
  "digital_analog": 0,
  "range": 0,
  "control": 1,
  "decom": 0,
  "low_voltage": 0,
  "pwm_period": 0
}
```

BAC_VAR (Variables)

JSON

```
{
  "pid": 0,
  "type": "VARIABLE",
  "index": 0,
  "id": "VAR1",
  "command": "0VAR1",
  "description": "Setpoint",
  "label": "SP",
  "value": 720,
  "auto_manual": 0,
  "digital_analog": 0,
  "range": 1,
  "control": 0,
  "unused": 0
}
```

BAC_PRG (Programs)

JSON

```
{
  "pid": 0,
  "type": "PROGRAM",
  "index": 0,
  "id": "PRG1",
  "command": "0PRG1",
  "description": "Morning Startup",
  "label": "MORN_UP",
  "on_off": 1,            // 0=Off, 1=On
  "auto_manual": 0,       // 0=Auto, 1=Manual
  "com_prg": 0,
  "errcode": 0
}
```

BAC_PID (Controllers)

JSON

```
{
  "pid": 0,
  "type": "CONTROLLER",
  "index": 0,
  "id": "PID1",
  "command": "0PID1",
  "input": 0,             // Input point index
  "input_value": 720,
  "value": 500,           // Output value
  "setpoint": 0,          // Setpoint point index
  "setpoint_value": 720,
  "units": 1,
  "auto_manual": 0,
  "action": 0,            // 0=Direct, 1=Reverse
  "proportional": 50,     // P gain
  "reset": 0,             // I gain
  "bias": 0,
  "rate": 0               // D gain
}
```

BAC_TL (Monitors/Trendlogs)

JSON


```
{
  "pid": 0,
  "type": "MONITOR",
  "index": 0,
  "id": "TL1",
  "command": "0TL1",
  "label": "Temp Log",
  "num_inputs": 2,
  "an_inputs": 2,
  "status": 0,
  "second_interval_time": 0,
  "minute_interval_time": 5,
  "hour_interval_time": 0,
  "max_time_length": 1440,
  "next_sample_time": 0,
  "inputs": [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  "range": [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
}
```


Implementation Details

Code Location

File: `T3000-Source/T3000/BacnetWebView.cpp` **Line:** 1614-1955 **Function:** `HandleWebViewMsg()`

Processing Flow

TEXT

```
1. Parse JSON request
   ↓
2. Validate parameters (panelId, objectinstance, indices)
   ↓
3. Check cache for panel data
   ↓
4. Switch on entryType
   ↓
5. Calculate read groups (chunks)
   ↓
6. Loop through groups:
   a. Calculate chunk start/end
   b. Call GetPrivateDataSaveSPBlocking()
   c. Copy data from temp buffer to global array
   d. Build JSON response for this chunk
   ↓
7. Return complete JSON response
```

Reading Algorithm

C++

```
case BAC_PRG:  // Programs
{
    if (entry_index_end >= entry_index_start)
    {
        // Step 1: Calculate groups
        int totalCount = entry_index_end - entry_index_start + 1;
        int groupSize = BAC_READ_PROGRAM_GROUP_NUMBER;  // 10
        int read_program_group = (totalCount + groupSize - 1) / groupSize;

        // Step 2: Loop through groups
        for (int i = 0; i < read_program_group; ++i)
        {
            int temp_start = entry_index_start + i * groupSize;
            if (temp_start > entry_index_end)
                break;

            int temp_end = temp_start + groupSize - 1;
            if (temp_end > entry_index_end)
                temp_end = entry_index_end;

            // Boundary check
            if (temp_start >= BAC_PROGRAM_ITEM_COUNT)
                break;
            if (temp_end >= BAC_PROGRAM_ITEM_COUNT)
                temp_end = BAC_PROGRAM_ITEM_COUNT - 1;

            // Step 3: Blocking read from device
            if (GetPrivateDataSaveSPBlocking(entry_objectinstance,
                READPROGRAM_T3000,
                (uint8_t)temp_start,
                (uint8_t)temp_end,
                sizeof(Str_program_point),
                4) > 0)  // 4 retries
            {
                // Step 4: Copy to global array
                for (int idx = temp_start; idx <= temp_end; ++idx)
                {
                    if (idx < BAC_PROGRAM_ITEM_COUNT) {
                        g_Program_data[temp_panel_id].at(idx) = s_Program_data[idx];
                    }
                }

                Sleep(SEND_COMMAND_DELAY_TIME);  // 200ms delay
            }
            else
            {
                // Handle read failure
                continue;
            }

            // Step 5: Build JSON for this chunk
            int npanel_id = temp_panel_id;
            for (int idx = temp_start; idx < temp_end; idx++)
            {
                tempjson["data"]["device_data"][point_idx]["pid"] = npanel_id;
                tempjson["data"]["device_data"][point_idx]["type"] = "PROGRAM";
                // ... populate all fields ...
```

```
                point_idx++;
            }
        }
    }
    break;
}
```

Group Sizes (Chunking)

Reading data in chunks prevents network overload:

Entry Type	Group Size	Example: Reading 0-15
Inputs	8	2 groups: [0-7], [8-15]
Outputs	8	2 groups: [0-7], [8-15]
Variables	8	2 groups: [0-7], [8-15]
Programs	10	2 groups: [0-9], [10-15]
Controllers	varies	Depends on configuration
Monitors	varies	Depends on configuration

Data Flow

TEXT

```
Device → BACnet Protocol → GetPrivateDataSaveSPBlocking()
                                ↓
                        s_Program_data[] (temp buffer)
                                ↓
                        g_Program_data[panelId][] (global cache)
                                ↓
                        JSON Response → Rust API → Frontend
```

Error Handling

Common Errors

1. Device Offline

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -1,
    "message": "Read timeout",
    "details": "ERROR: Read program failed start=0, end=9"
  }
}
```

2. Invalid Index Range

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid parameter",
    "details": "entryIndexEnd (20) exceeds BAC_PROGRAM_ITEM_COUNT (16)"
  }
}
```

3. Object Instance Mismatch

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -5,
    "message": "Object mismatch",
    "details": "Expected objectinstance=123456, got 123457"
  }
}
```

Performance

Timing

Scenario	Typical Time
Cache hit (no device read)	< 50ms
Fresh read (1-10 items)	200-500ms
Fresh read (full range 0-15)	1-2 seconds
Multiple groups (chunked)	+200ms per group

Optimization Tips

1. **Request only needed range** - Don't read 0-63 if you only need 0-15
2. **Batch requests** - Read multiple items at once vs individual requests
3. **Use cache when possible** - Check if fresh data is needed
4. **Avoid overlapping requests** - Wait for response before new request

Usage Examples

Example 1: Read All Programs

JSON

```
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "entryIndexStart": 0,
  "entryIndexEnd": 15,
  "objectinstance": 123456
}
```

Example 2: Read Single Input

```JSON
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 1,
  "entryIndexStart": 0,
  "entryIndexEnd": 0,
  "objectinstance": 123456
}
```

Example 3: Read Controllers 0-7

```JSON
{
  "action": 17,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 11,
  "entryIndexStart": 0,
  "entryIndexEnd": 7,
  "objectinstance": 123456
}
```

Comparison with Other Messages

vs GET_PANEL_DATA

- **GET_PANEL_DATA:** Loads ALL cached data for entire panel
- **GET_WEBVIEW_LIST:** Reads specific entry type with range

Use GET_WEBVIEW_LIST when: You need specific data, especially fresh from device **Use GET_PANEL_DATA when:** Initial panel load, need all data types

vs GET_ENTRIES

- **GET_ENTRIES:** Read specific individual entries by exact indices
- **GET_WEBVIEW_LIST:** Read continuous range of one entry type

Use GET_WEBVIEW_LIST when: Reading sequential range **Use GET_ENTRIES when:** Reading scattered individual entries

See Also

- Message Index →
- UPDATE_WEBVIEW_LIST →
- GET_PANEL_DATA →
- BACnet Commands →
- Data Structures →

Message: SAVE_GRAPHIC_DATA

The SAVE_GRAPHIC_DATA message saves modified graphic screen configurations back to the device. Use this after editing HMI screens in the graphic editor.

When to Use:

- After creating or modifying a graphic screen
- When user clicks "Save" in HMI editor
- To persist graphic element changes

Overview

Action: `SAVE_GRAPHIC_DATA` **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 1479 **Purpose:** Write graphic screen configuration to device memory

Request Format

JSON Structure:

JSON

```
{
  "action": "SAVE_GRAPHIC_DATA",
  "panel_id": 0,
  "graphic_screen_num": 5,
  "data": {
    "screen_name": "Main HVAC",
    "background_color": "#FFFFFF",
    "elements": [
      {
        "type": "TEXT",
        "x": 100,
        "y": 50,
        "width": 200,
        "height": 30,
        "text": "Zone Temperature",
        "font_size": 14,
        "color": "#000000"
      },
      {
        "type": "LIVE_VALUE",
        "x": 320,
        "y": 50,
        "width": 80,
        "height": 30,
        "point_type": "IN",
        "point_index": 5,
        "format": "%.1f",
        "color": "#0000FF"
      }
    ]
  }
}
```

Request Parameters

Parameter	Type	Required	Description
`panel_id`	number	Yes	Target panel ID (0-254)
`graphic_screen_num`	number	Yes	Screen index (1-20)
`data`	object	Yes	Complete screen configuration

Data Object Structure

Field	Type	Description
`screen_name`	string	Screen title (max 31 chars)
`background_color`	string	Hex color (#RRGGBB)
`elements`	array	Array of graphic elements (max 64)

Element Object Structure

Common Fields (all element types):

Field	Type	Required	Description
`type`	string	Yes	Element type (TEXT, LIVE_VALUE, BUTTON, etc.)
`x`	number	Yes	X coordinate (pixels)
`y`	number	Yes	Y coordinate (pixels)
`width`	number	Yes	Width (pixels)
`height`	number	Yes	Height (pixels)

Type-Specific Fields:

TEXT:

- `text` (string): Text content (max 63 chars)
- `font_size` (number): Font size in points
- `color` (string): Text color (#RRGGBB)

LIVE_VALUE:

- `point_type` (string): "IN", "OUT", "VAR", "PRG", "PID", "TL"
- `point_index` (number): Point number (0-63)
- `format` (string): Printf format (e.g., "%.1f", "%d")
- `color` (string): Value color (#RRGGBB)

BUTTON:

- `button_text` (string): Button label (max 31 chars)
- `action_type` (string): "WRITE_VALUE", "TOGGLE", "NAVIGATE"
- `target_point` (number): Point to control (if applicable)

Response Format

Success Response:

JSON

```
{
  "action": "SAVE_GRAPHIC_DATA",
  "status": "success",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Error Response:

JSON

```
{
  "action": "SAVE_GRAPHIC_DATA",
  "status": "error",
  "error": "Too many elements (max 64)",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Implementation

Location: BacnetWebView.cpp line 1479

C++

```cpp
case SAVE_GRAPHIC_DATA:
{
    int panel_id;
    int graphic_screen_num;

    // Parse request
    nlohmann::json json_req = nlohmann::json::parse(message);
    panel_id = json_req["panel_id"];
    graphic_screen_num = json_req["graphic_screen_num"];

    // Validate screen number
    if (graphic_screen_num < 1 || graphic_screen_num > MAX_GRPHICS_20)
    {
        // Send error response
        nlohmann::json error_json;
        error_json["action"] = "SAVE_GRAPHIC_DATA";
        error_json["status"] = "error";
        error_json["error"] = "Invalid screen number";
        SendWebViewMessage(error_json.dump().c_str());
        return;
    }

    // Get data object
    nlohmann::json data_json = json_req["data"];

    // Get graphic structure from global array
    Control_group_point& graphic =
        g_GraphicConfigData[panel_id].at(graphic_screen_num - 1);

    // Parse screen properties
    std::string screen_name = data_json["screen_name"];
    strncpy(graphic.screen_name, screen_name.c_str(), 31);
    graphic.screen_name[31] = '\0';

    // Parse background color
    std::string bg_color = data_json["background_color"];
    graphic.background_color = parseHexColor(bg_color);

    // Parse elements
    nlohmann::json elements_array = data_json["elements"];
    int element_count = elements_array.size();

    if (element_count > 64)
    {
        // Send error response
        nlohmann::json error_json;
        error_json["action"] = "SAVE_GRAPHIC_DATA";
        error_json["status"] = "error";
        error_json["error"] = "Too many elements (max 64)";
        SendWebViewMessage(error_json.dump().c_str());
        return;
    }

    graphic.element_count = element_count;

    for (int i = 0; i < element_count; i++)
    {
        nlohmann::json elem_json = elements_array[i];
```

```
    Graphic_element& elem = graphic.elements[i];

    // Parse common fields
    std::string type = elem_json["type"];
    elem.element_type = stringToElementType(type);
    elem.x_position = elem_json["x"];
    elem.y_position = elem_json["y"];
    elem.width = elem_json["width"];
    elem.height = elem_json["height"];

    // Parse type-specific fields
    if (elem.element_type == ELEM_TEXT)
    {
        std::string text = elem_json["text"];
        strncpy(elem.text_content, text.c_str(), 63);
        elem.text_content[63] = '\0';

        elem.font_size = elem_json["font_size"];

        std::string color = elem_json["color"];
        elem.text_color = parseHexColor(color);
    }
    else if (elem.element_type == ELEM_LIVE_VALUE)
    {
        std::string pt_type = elem_json["point_type"];
        elem.point_type = stringToPointType(pt_type);
        elem.point_index = elem_json["point_index"];

        std::string format = elem_json["format"];
        strncpy(elem.value_format, format.c_str(), 7);
        elem.value_format[7] = '\0';

        std::string color = elem_json["color"];
        elem.value_color = parseHexColor(color);
    }
    else if (elem.element_type == ELEM_BUTTON)
    {
        std::string btn_text = elem_json["button_text"];
        strncpy(elem.button_text, btn_text.c_str(), 31);
        elem.button_text[31] = '\0';

        std::string action = elem_json["action_type"];
        elem.action_type = stringToActionType(action);
        elem.target_point = elem_json["target_point"];
    }
}

// Write to device (synchronous BACnet write)
int result = WriteGraphicScreen(
    panel_id,
    graphic_screen_num,
    &graphic,
    sizeof(Control_group_point)
);

// Send response
nlohmann::json response_json;
response_json["action"] = "SAVE_GRAPHIC_DATA";
response_json["panel_id"] = panel_id;
response_json["graphic_screen_num"] = graphic_screen_num;
```

```
    if (result > 0)
    {
        response_json["status"] = "success";
    }
    else
    {
        response_json["status"] = "error";
        response_json["error"] = "Device write failed";
    }

    SendWebViewMessage(response_json.dump().c_str());
    break;
}
```

Helper Functions

parseHexColor()

Converts hex color string to RGB integer:

C++

```
unsigned long parseHexColor(const std::string& hex_color)
{
    // Remove '#' if present
    std::string hex = hex_color;
    if (hex[0] == '#')
        hex = hex.substr(1);

    // Parse as hex
    unsigned long color = std::stoul(hex, nullptr, 16);
    return color;
}

// Example: "#FF0000" → 0x00FF0000 (red)
```

stringToElementType()

Converts element type string to constant:

C++

```
unsigned char stringToElementType(const std::string& type)
{
    if (type == "TEXT") return ELEM_TEXT;
    if (type == "LIVE_VALUE") return ELEM_LIVE_VALUE;
    if (type == "BUTTON") return ELEM_BUTTON;
    if (type == "IMAGE") return ELEM_IMAGE;
    if (type == "LINE") return ELEM_LINE;
    if (type == "RECTANGLE") return ELEM_RECTANGLE;
    return ELEM_TEXT;  // Default
}
```

Frontend Usage

Save Graphic Screen

TYPESCRIPT

```typescript
const saveGraphicScreen = async (
  panelId: number,
  screenNum: number,
  screenData: GraphicScreenData
) => {
  const message = {
    action: 'SAVE_GRAPHIC_DATA',
    panel_id: panelId,
    graphic_screen_num: screenNum,
    data: {
      screen_name: screenData.screenName,
      background_color: screenData.backgroundColor,
      elements: screenData.elements.map(elem => ({
        type: elem.type,
        x: elem.x,
        y: elem.y,
        width: elem.width,
        height: elem.height,
        ...getTypeSpecificFields(elem)
      }))
    }
  };

  sendMessage(JSON.stringify(message));

  // Wait for response
  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'SAVE_GRAPHIC_DATA') {
        if (response.status === 'success') {
          resolve(response);
        } else {
          reject(new Error(response.error));
        }

        // Remove listener
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);
  });
};

const getTypeSpecificFields = (elem: GraphicElement) => {
  switch (elem.type) {
    case 'TEXT':
      return {
        text: elem.text,
        font_size: elem.fontSize,
        color: elem.color
      };
```

```
    case 'LIVE_VALUE':
      return {
        point_type: elem.pointType,
        point_index: elem.pointIndex,
        format: elem.format,
        color: elem.color
      };

    case 'BUTTON':
      return {
        button_text: elem.buttonText,
        action_type: elem.actionType,
        target_point: elem.targetPoint
      };

    default:
      return {};
  }
};
```

Complete Editor Workflow

TYPESCRIPT

```typescript
// 1. Load existing screen (or create new)
const screenData = await loadGraphicScreen(panelId, screenNum);

// 2. User edits in graphic editor
const editor = new GraphicEditor(screenData);
editor.on('change', () => {
  // Mark as modified
  hasUnsavedChanges = true;
});

// 3. User clicks Save
const handleSave = async () => {
  try {
    const modifiedData = editor.getScreenData();

    await saveGraphicScreen(panelId, screenNum, modifiedData);

    hasUnsavedChanges = false;
    showNotification('Screen saved successfully');
  } catch (error) {
    showError(`Save failed: ${error.message}`);
  }
};
```

Validation

Frontend Validation

Before sending, validate:

TYPESCRIPT

```typescript
const validateGraphicScreen = (data: GraphicScreenData): string[] => {
  const errors: string[] = [];

  // Check screen name length
  if (data.screenName.length > 31) {
    errors.push('Screen name too long (max 31 chars)');
  }

  // Check element count
  if (data.elements.length > 64) {
    errors.push('Too many elements (max 64)');
  }

  // Check each element
  data.elements.forEach((elem, i) => {
    // Check coordinates
    if (elem.x < 0 || elem.y < 0) {
      errors.push(`Element ${i}: negative coordinates`);
    }

    // Check dimensions
    if (elem.width <= 0 || elem.height <= 0) {
      errors.push(`Element ${i}: invalid dimensions`);
    }

    // Type-specific validation
    if (elem.type === 'TEXT' && elem.text.length > 63) {
      errors.push(`Element ${i}: text too long (max 63 chars)`);
    }

    if (elem.type === 'LIVE_VALUE') {
      if (elem.pointIndex < 0 || elem.pointIndex > 63) {
        errors.push(`Element ${i}: invalid point index`);
      }
    }
  });

  return errors;
};

// Usage
const errors = validateGraphicScreen(screenData);
if (errors.length > 0) {
  showErrors(errors);
  return;
}

// Proceed with save
await saveGraphicScreen(panelId, screenNum, screenData);
```

Performance Characteristics

Data Size:

- Small screen (10 elements): ~2 KB request
- Large screen (64 elements): ~15 KB request

Write Time:

- Average: 200-500ms
- Depends on device response time
- Synchronous operation (blocks until complete)

Recommendations:

- Show loading indicator during save
- Disable editor during save operation
- Implement auto-save with debouncing (save after 2s of inactivity)

Error Handling

Common Errors

Error	Cause	Solution
"Invalid screen number"	Screen num < 1 or > 20	Validate before sending
"Too many elements"	> 64 elements	Limit element count
"Device write failed"	Device offline or timeout	Retry or show error
"Invalid color format"	Bad hex color	Validate color strings

Retry Logic

TYPESCRIPT

```typescript
const saveWithRetry = async (
  panelId: number,
  screenNum: number,
  data: GraphicScreenData,
  maxRetries: number = 3
): Promise<void> => {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      await saveGraphicScreen(panelId, screenNum, data);
      return;  // Success
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;  // Final attempt failed
      }

      // Wait before retry (exponential backoff)
      await sleep(1000 * attempt);
    }
  }
};
```

Related Messages

GET_INITIAL_DATA / LOAD_GRAPHIC_ENTRY

Load screen before saving:

TEXT

```
1. GET_INITIAL_DATA → Load existing screen
2. User modifies in editor
3. SAVE_GRAPHIC_DATA → Save changes
```

Workflow Integration

TYPESCRIPT

```typescript
const graphicEditorWorkflow = async (panelId: number, screenNum: number) => {
  // Load
  const data = await loadGraphicScreen(panelId, screenNum);

  // Edit
  const editor = new GraphicEditor(data);
  await editor.waitForUserEdits();

  // Save
  const modifiedData = editor.getScreenData();
  await saveGraphicScreen(panelId, screenNum, modifiedData);

  // Refresh display
  await loadGraphicScreen(panelId, screenNum);
};
```

See Also

- GET_INITIAL_DATA - Loading graphic screens
- Data Structures - Control_group_point structure
- Platform Overview - Architecture

Message: UPDATE_ENTRY

UPDATE_ENTRY updates a single field of a specific entry. This is the fastest way to change one value without sending a full entry update.

Common Uses:

- Quick value changes
- Toggle on/off states
- Single setpoint adjustments
- Manual/auto mode switches

Overview

Message Type: UPDATE_ENTRY **File Location:** `BacnetWebView.cpp` line 2167 **Purpose:** Update single entry field

Request Format

JSON

```
{
  "action": "UPDATE_ENTRY",
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 2,
  "index": 5,
  "field": "value",
  "value": 500,
  "objectinstance": 123456
}
```

Parameters

Parameter	Type	Required	Description
`action`	String	✅	Must be "UPDATE_ENTRY"
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`panelId`	Integer	✅	Panel number (0-254)
`serialNumber`	Integer	✅	Device serial number
`entryType`	Integer	✅	Type of entry
`index`	Integer	✅	Entry index
`field`	String	✅	Field name to update
`value`	Any	✅	New value
`objectinstance`	Integer	✅	BACnet object instance

Response Format

JSON

```
{
  "action": "UPDATE_ENTRY_RES",
  "data": {
    "status": true
  }
}
```

Updatable Fields by Type

Outputs (BAC_OUT)

- `value` - Output value (0-1000)
- `auto_manual` - 0=Auto, 1=Manual
- `range` - Unit selection

Variables (BAC_VAR)

- `value` - Variable value
- `auto_manual` - 0=Auto, 1=Manual

Programs (BAC_PRG)

- `on_off` - 0=Off, 1=On
- `auto_manual` - 0=Auto, 1=Manual

Controllers (BAC_PID)

- `setpoint` - Setpoint value
- `auto_manual` - 0=Auto, 1=Manual
- `action` - 0=Direct, 1=Reverse
- `proportional` - P gain
- `reset` - I gain
- `rate` - D gain

Implementation

C++

```
case WEBVIEW_MESSAGE_TYPE::UPDATE_ENTRY:
{
    int entry_type = json.get("entryType", -1).asInt();
    int index = json.get("index", -1).asInt();
    std::string field = json.get("field", "").asString();

    switch (entry_type)
    {
        case BAC_OUT:
            if (field == "value")
            {
                int value = json.get("value", 0).asInt();
                g_Output_data[panel_id].at(index).value = value;
                WriteOutputValue(objectinstance, index, value);
            }
            break;
        // ... other types ...
    }

    tempjson["action"] = "UPDATE_ENTRY_RES";
    tempjson["data"]["status"] = true;
    break;
}
```

Performance

Operation	Time
Value update	100-200ms
Mode change	100-200ms
PID parameter	150-300ms

Usage Examples

Example 1: Change Output Value

JSON

```
{
  "action": "UPDATE_ENTRY",
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 2,
  "index": 0,
  "field": "value",
  "value": 750,
  "objectinstance": 123456
}
```

Example 2: Toggle Program

JSON

```
{
  "action": "UPDATE_ENTRY",
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "index": 3,
  "field": "on_off",
  "value": 1,
  "objectinstance": 123456
}
```

Example 3: Change PID Setpoint

JSON

```
{
  "action": "UPDATE_ENTRY",
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 11,
  "index": 0,
  "field": "setpoint",
  "value": 720,
  "objectinstance": 123456
}
```

Error Handling

JSON

```
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid field",
    "details": "Field 'xyz' is not writable for entry type 2"
  }
}
```

See Also

- UPDATE_WEBVIEW_LIST - Bulk updates
- GET_WEBVIEW_LIST - Read values

Message: UPDATE_WEBVIEW_LIST

UPDATE_WEBVIEW_LIST writes values to device entries in bulk. Use this to update multiple inputs, outputs, variables, or other entry types in a single operation.

Common Uses:

- Updating output values
- Setting multiple setpoints
- Batch configuration changes
- Program control (start/stop)

Overview

Message Type: UPDATE_WEBVIEW_LIST **File Location:** `BacnetWebView.cpp` line 1957 **Purpose:** Write multiple entry values to device

Request Format

JSON

```
{
  "action": 16,
  "source": 1,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 2,
  "objectinstance": 123456,
  "data": [
    {
      "index": 0,
      "value": 500
    },
    {
      "index": 1,
      "value": 750
    }
  ]
}
```

Parameters

Parameter	Type	Required	Description
`action`	Integer	✅	Must be 16
`source`	Integer	✅	Message source: 0=T3000, 1=WebUI
`panelId`	Integer	✅	Panel number (0-254)
`serialNumber`	Integer	✅	Device serial number
`entryType`	Integer	✅	Type of entry to update
`objectinstance`	Integer	✅	BACnet object instance
`data`	Array	✅	Array of updates (index + values)

Update Data Fields

Varies by entry type. Common fields:

Field	Type	Description
`index`	Integer	Entry index to update
`value`	Number	New value
`auto_manual`	Integer	0=Auto, 1=Manual
`range`	Integer	Unit/range selection

Response Format

JSON

```
{
  "action": "UPDATE_WEBVIEW_LIST_RES",
  "data": {
    "status": true,
    "updated_count": 2,
    "failed_indices": []
  }
}
```

Supported Entry Types

Type	Value	Writable Fields
BAC_OUT	2	value, auto_manual, range
BAC_VAR	3	value, auto_manual
BAC_PRG	6	on_off, auto_manual
BAC_PID	11	setpoint, auto_manual, proportional, reset, rate

Implementation

Write Process

1. **Parse updates** - Extract index and field changes
2. **Validate range** - Check indices are valid
3. **Update global array** - Modify cached data
4. **Write to device** - Send BACnet write command
5. **Verify write** - Confirm success
6. **Update cache file** - Persist changes

Code Pattern

```C++
case WEBVIEW_MESSAGE_TYPE::UPDATE_WEBVIEW_LIST:
{
    int entry_type = json.get("entryType", Json::nullValue).asInt();
    Json::Value data = json["data"];

    for (int i = 0; i < data.size(); i++)
    {
        int index = data[i].get("index", -1).asInt();

        switch (entry_type)
        {
            case BAC_OUT:
                // Update output value
                if (data[i].isMember("value"))
                {
                    int value = data[i]["value"].asInt();
                    g_Output_data[panel_id].at(index).value = value;
                    WriteOutputValue(objectinstance, index, value);
                }
                break;
            // ... other types ...
        }
    }

    break;
}
```

Error Handling

Invalid Index:

```JSON
{
  "action": "ERROR_RES",
  "error": {
    "code": -2,
    "message": "Invalid index",
    "details": "Index 70 exceeds max count 64 for outputs"
  }
}
```

Write Timeout:

```JSON
{
  "action": "ERROR_RES",
  "error": {
    "code": -4,
    "message": "Write failed",
    "details": "Device did not acknowledge write for output 5"
  }
}
```

Performance

Operation	Time
Single value	100-300ms
2-5 values	200-500ms
10+ values	500ms-2s

Usage Examples

Example 1: Update Single Output

JSON

```
{
  "action": 18,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 2,
  "objectinstance": 123456,
  "data": [
    {
      "index": 0,
      "value": 500,
      "auto_manual": 0
    }
  ]
}
```

Example 2: Update Multiple Variables

JSON

```
{
  "action": 18,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 3,
  "objectinstance": 123456,
  "data": [
    { "index": 0, "value": 720 },
    { "index": 1, "value": 680 },
    { "index": 2, "value": 750 }
  ]
}
```

Example 3: Start/Stop Programs

JSON

```
{
  "action": 18,
  "panelId": 0,
  "serialNumber": 12345,
  "entryType": 6,
  "objectinstance": 123456,
  "data": [
    { "index": 0, "on_off": 1 },
    { "index": 1, "on_off": 0 }
  ]
}
```

See Also

- UPDATE_ENTRY - Update single field
- GET_WEBVIEW_LIST - Read values
- Control Messages Index

Installation Guide

This guide walks you through installing T3000 Building Automation System on your Windows computer.

System Requirements

Minimum Requirements

- **OS**: Windows 10 (64-bit)
- **Processor**: Intel Core i3 or equivalent
- **RAM**: 4 GB
- **Storage**: 500 MB available space
- **Network**: Ethernet or Wi-Fi adapter

Recommended Requirements

- **OS**: Windows 11 (64-bit)
- **Processor**: Intel Core i5 or better
- **RAM**: 8 GB or more
- **Storage**: 1 GB SSD space
- **Network**: Gigabit Ethernet adapter

Installation Steps

1. Download the Installer

Visit the official website to download the latest version:

```
https://temcocontrols.com/downloads/t3000
```

Select the appropriate version for your system architecture (x64).

2. Run the Installer

1. Locate the downloaded file: `T3000_Setup_v9.0.exe`
2. Right-click and select **Run as Administrator**
3. If prompted by User Account Control, click **Yes**

3. Follow the Installation Wizard

Welcome Screen

- Click **Next** to continue

License Agreement

- Read the terms and conditions
- Select **I accept the agreement**
- Click **Next**

Choose Install Location

- Default: `C:\Program Files\Temco Controls\T3000`
- Click **Browse** to change location (optional)
- Click **Next**

Select Components

- ✓ T3000 Application (Required)
- ✓ USB Drivers (Recommended)
- ✓ Documentation (Optional)
- ✓ Sample Projects (Optional)

- Click **Next**

Start Menu Folder

- Default: `Temco Controls\T3000`
- Click **Next**

Additional Tasks

- ✓ Create desktop shortcut
- ✓ Create Quick Launch icon
- Click **Next**

Ready to Install

- Review your selections
- Click **Install**

4. Complete Installation

Wait for the installation to complete (typically 2-3 minutes).

When finished:

- Click **Finish** to exit the installer
- Optionally select **Launch T3000** to start the application

Post-Installation Configuration

First Launch

When you launch T3000 for the first time:

1. **License Activation**
   - Enter your license key (provided with purchase)
   - Or select **Trial Mode** for evaluation (30 days)
2. **Initial Configuration Wizard**
   - Select your network adapter
   - Choose default communication protocol
   - Set up database location
3. **User Account**
   - Create administrator account
   - Set password (minimum 8 characters)

Verify Installation

To verify T3000 is installed correctly:

1. Click **Help** → **About**
2. Check version number matches downloaded version
3. Verify all components are listed

Installing USB Drivers

If you need to connect via USB-RS485 adapters:

1. Connect the USB adapter
2. Windows will attempt to install drivers automatically
3. If manual installation is needed:
   - Open **Device Manager**
   - Locate the device under **Ports (COM & LPT)**
   - Right-click and select **Update Driver**

- Browse to: `C:\Program Files\Temco Controls\T3000\Drivers\USB`

Firewall Configuration

Add firewall exceptions for T3000:

1. Open **Windows Defender Firewall**
2. Click **Allow an app through firewall**
3. Click **Change settings**
4. Find **T3000** in the list
5. Check both **Private** and **Public** networks
6. Click **OK**

Or add manually:

- Port **502** (Modbus TCP)
- Port **47808** (BACnet)

Updating T3000

To update to a newer version:

1. Download the latest installer
2. Run the installer (existing installation will be detected)
3. Select **Upgrade** when prompted
4. Your settings and data will be preserved

Uninstalling

To remove T3000:

1. Go to **Settings** → **Apps** → **Installed apps**
2. Find **T3000 Building Automation System**
3. Click **Uninstall**
4. Follow the uninstall wizard

Note: Your project files and database will be preserved unless you explicitly choose to delete them.

Troubleshooting Installation Issues

Installation Fails

- Ensure you have administrator privileges
- Temporarily disable antivirus software
- Check available disk space
- Download installer again (may be corrupted)

USB Drivers Not Working

- Try different USB port
- Reinstall USB drivers manually
- Check for Windows updates

Application Won't Start

- Right-click and **Run as Administrator**
- Check Windows Event Viewer for errors
- Verify .NET Framework is installed

Next Steps

- Configuration Guide - Set up your system
- Connecting Devices - Add your first device

- SQL Server Express Setup for T3000 - Detailed Step 1 to Step 23 setup with screenshot slots

Support

For installation support:

- Email: support@temcocontrols.com
- Phone: 1-800-TEMCO-00
- Online: https://support.temcocontrols.com

Installation Guide

Automated Installation

Silent Installation

```powershell
## Silent install with default options
.\T3000_Setup_v9.0.exe /S

## Silent install with custom path
.\T3000_Setup_v9.0.exe /S /D=C:\CustomPath\T3000

## Install with specific components
.\T3000_Setup_v9.0.exe /S /COMPONENTS="core,drivers,docs"
```

Command-Line Parameters

```shell
/S              # Silent mode
/D=path         # Installation directory
/COMPONENTS=x   # Component selection
/NOICONS        # Don't create shortcuts
/PORTABLE       # Portable installation
/LOG=file.txt   # Enable installation logging
```

System Dependencies

Required Runtimes

```powershell
## Check .NET installation
dotnet --list-runtimes

## Install .NET 7.0 Runtime
winget install Microsoft.DotNet.Runtime.7

## Verify Windows version
[System.Environment]::OSVersion.Version
```

Network Stack Requirements

```yaml
Required Services:
  - Windows Firewall Service (MpsSvc)
  - TCP/IP NetBIOS Helper (lmhosts)
  - Network Location Awareness (NlaSvc)

Required Protocols:
  - TCP/IPv4
  - UDP
  - IGMP (for BACnet broadcast)
```

Docker Installation

```dockerfile
## Dockerfile for T3000 Server
FROM mcr.microsoft.com/windows/servercore:ltsc2022

## Install dependencies
RUN powershell -Command \
    Install-WindowsFeature -Name NET-Framework-45-Core

## Copy installer
COPY T3000_Setup_v9.0.exe C:\Temp\

## Run silent installation
RUN C:\Temp\T3000_Setup_v9.0.exe /S /D=C:\T3000

## Expose ports
EXPOSE 502 9103 47808/udp

## Start T3000 service
CMD ["C:\\T3000\\T3000Service.exe"]
```

Build and run:

```shell
docker build -t t3000-server .
docker run -d -p 502:502 -p 9103:9103 -p 47808:47808/udp t3000-server
```

Registry Configuration

Installation Registry Keys

```powershell
## Installation path
$regPath = "HKLM:\SOFTWARE\Temco Controls\T3000"
Get-ItemProperty -Path $regPath -Name "InstallPath"

## Version information
Get-ItemProperty -Path $regPath -Name "Version"
Get-ItemProperty -Path $regPath -Name "BuildNumber"

## License information
Get-ItemProperty -Path $regPath -Name "LicenseKey"
Get-ItemProperty -Path $regPath -Name "LicenseType"
```

Firewall Rules via PowerShell

```powershell
## Create firewall rules programmatically
New-NetFirewallRule -DisplayName "T3000 Modbus TCP" `
  -Direction Inbound `
  -Protocol TCP `
  -LocalPort 502 `
  -Action Allow

New-NetFirewallRule -DisplayName "T3000 BACnet/IP" `
  -Direction Inbound `
  -Protocol UDP `
  -LocalPort 47808 `
  -Action Allow

New-NetFirewallRule -DisplayName "T3000 API Server" `
  -Direction Inbound `
  -Protocol TCP `
  -LocalPort 9103 `
  -Action Allow
```

Database Initialization

SQLite Database Setup

```sql
-- Default database location
-- C:\ProgramData\Temco\T3000\t3000.db

-- Initialize database schema
.read schema.sql

-- Create indexes for performance
CREATE INDEX idx_devices_serial ON devices(serial_number);
CREATE INDEX idx_points_device ON data_points(device_id);
CREATE INDEX idx_trends_time ON trend_data(timestamp);

-- Set database parameters
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = 10000;
```

Database Migration Script

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

const db = new Database('t3000.db');

// Run migrations
const migrations = [
  `CREATE TABLE IF NOT EXISTS devices (...)`,
  `CREATE TABLE IF NOT EXISTS data_points (...)`,
  `CREATE TABLE IF NOT EXISTS trend_data (...)`
];

for (const migration of migrations) {
  db.exec(migration);
}

console.log('Database initialized');
```

Environment Variables

POWERSHELL

```powershell
## Set T3000 environment variables
[Environment]::SetEnvironmentVariable(
  "T3000_HOME",
  "C:\Program Files\Temco Controls\T3000",
  "Machine"
)

[Environment]::SetEnvironmentVariable(
  "T3000_DATA",
  "C:\ProgramData\Temco\T3000",
  "Machine"
)

[Environment]::SetEnvironmentVariable(
  "T3000_PORT",
  "9103",
  "Machine"
)
```

Service Installation

Install as Windows Service

POWERSHELL

```powershell
## Create service
New-Service -Name "T3000Service" `
  -BinaryPathName "C:\Program Files\Temco Controls\T3000\T3000Service.exe" `
  -DisplayName "T3000 Building Automation Service" `
  -Description "T3000 BACnet and Modbus communication service" `
  -StartupType Automatic

## Start service
Start-Service -Name "T3000Service"

## Check status
Get-Service -Name "T3000Service"
```

Service Configuration

XML

```xml
<service>
  <id>T3000Service</id>
  <name>T3000 Building Automation</name>
  <description>BACnet/Modbus communication service</description>
  <executable>T3000Service.exe</executable>
  <startmode>Automatic</startmode>
  <delayedAutoStart>false</delayedAutoStart>
  <priority>Normal</priority>
  <stoptimeout>15000</stoptimeout>
</service>
```

Verification Script

POWERSHELL

```powershell
## Post-installation verification
function Test-T3000Installation {
    $checks = @{
        "Installation Path" = Test-Path "C:\Program Files\Temco Controls\T3000"
        "Executable" = Test-Path "C:\Program Files\Temco Controls\T3000\T3000.exe"
        "Database" = Test-Path "C:\ProgramData\Temco\T3000\t3000.db"
        "Modbus Port" = Test-NetConnection -Port 502 -ComputerName localhost
        "BACnet Port" = Test-NetConnection -Port 47808 -ComputerName localhost
        "API Port" = Test-NetConnection -Port 9103 -ComputerName localhost
    }

    $checks.GetEnumerator() | ForEach-Object {
        Write-Host "$($_.Key): $($_.Value)"
    }
}

Test-T3000Installation
```

Next Steps

- Configuration API - Programmatic configuration
- REST API - API integration

- WebSocket API - Real-time communication

System Requirements

Minimum Requirements

- **OS**: Windows 10 (64-bit)
- **Processor**: Intel Core i3 or equivalent
- **RAM**: 4 GB
- **Storage**: 500 MB available space
- **Network**: Ethernet or Wi-Fi adapter

Recommended Requirements

- **OS**: Windows 11 (64-bit)
- **Processor**: Intel Core i5 or better
- **RAM**: 8 GB or more
- **Storage**: 1 GB SSD space
- **Network**: Gigabit Ethernet adapter

Installation Steps

1. Download the Installer

Visit the official website to download the latest version:

```text
https://temcocontrols.com/downloads/t3000
```

Select the appropriate version for your system architecture (x64).

2. Run the Installer

1. Locate the downloaded file: `T3000_Setup_v9.0.exe`
2. Right-click and select **Run as Administrator**
3. If prompted by User Account Control, click **Yes**

3. Follow the Installation Wizard

Welcome Screen

- Click **Next** to continue

License Agreement

- Read the terms and conditions
- Select **I accept the agreement**
- Click **Next**

Choose Install Location

- Default: `C:\Program Files\Temco Controls\T3000`
- Click **Browse** to change location (optional)
- Click **Next**

Select Components

- ✓ T3000 Application (Required)
- ✓ USB Drivers (Recommended)
- ✓ Documentation (Optional)
- ✓ Sample Projects (Optional)
- Click **Next**

Start Menu Folder

- Default: `Temco Controls\T3000`
- Click **Next**

Additional Tasks

- ✓ Create desktop shortcut
- ✓ Create Quick Launch icon
- Click **Next**

Ready to Install

- Review your selections
- Click **Install**

4. Complete Installation

Wait for the installation to complete (typically 2-3 minutes).

When finished:

- Click **Finish** to exit the installer
- Optionally select **Launch T3000** to start the application

Post-Installation Configuration

First Launch

When you launch T3000 for the first time:

1. **License Activation**
   - Enter your license key (provided with purchase)
   - Or select **Trial Mode** for evaluation (30 days)
2. **Initial Configuration Wizard**
   - Select your network adapter
   - Choose default communication protocol
   - Set up database location
3. **User Account**
   - Create administrator account
   - Set password (minimum 8 characters)

Verify Installation

To verify T3000 is installed correctly:

1. Click **Help** → **About**
2. Check version number matches downloaded version
3. Verify all components are listed

Installing USB Drivers

If you need to connect via USB-RS485 adapters:

1. Connect the USB adapter
2. Windows will attempt to install drivers automatically
3. If manual installation is needed:
   - Open **Device Manager**
   - Locate the device under **Ports (COM & LPT)**
   - Right-click and select **Update Driver**
   - Browse to: `C:\Program Files\Temco Controls\T3000\Drivers\USB`

Firewall Configuration

Add firewall exceptions for T3000:

1. Open **Windows Defender Firewall**
2. Click **Allow an app through firewall**
3. Click **Change settings**
4. Find **T3000** in the list
5. Check both **Private** and **Public** networks
6. Click **OK**

Or add manually:

- Port **502** (Modbus TCP)
- Port **47808** (BACnet)

Updating T3000

To update to a newer version:

1. Download the latest installer
2. Run the installer (existing installation will be detected)
3. Select **Upgrade** when prompted
4. Your settings and data will be preserved

Uninstalling

To remove T3000:

1. Go to **Settings** → **Apps** → **Installed apps**
2. Find **T3000 Building Automation System**
3. Click **Uninstall**
4. Follow the uninstall wizard

Note: Your project files and database will be preserved unless you explicitly choose to delete them.

Troubleshooting Installation Issues

Installation Fails

- Ensure you have administrator privileges
- Temporarily disable antivirus software
- Check available disk space
- Download installer again (may be corrupted)

USB Drivers Not Working

- Try different USB port
- Reinstall USB drivers manually
- Check for Windows updates

Application Won't Start

- Right-click and **Run as Administrator**
- Check Windows Event Viewer for errors
- Verify .NET Framework is installed

Next Steps

- Configuration Guide - Set up your system
- Connecting Devices - Add your first device

Support

For installation support:

- Email: support@temcocontrols.com
- Phone: 1-800-TEMCO-00
- Online: https://support.temcocontrols.com

Configuration Guide

Learn how to configure T3000 for your building automation needs.

Initial Configuration

After installation, configure these essential settings.

Network Settings

Navigate to **Settings → Network**

IP Configuration

```
Network Adapter: [Select your adapter]
IP Address Range: 192.168.1.1 - 192.168.1.254
Subnet Mask: 255.255.255.0
Gateway: 192.168.1.1
```

Protocol Settings

- ✓ Enable Modbus TCP (Port 502)
- ✓ Enable BACnet (Port 47808)
- ○ Enable HTTP API (Port 8080)

Database Settings

Location

```
Default: C:\ProgramData\Temco\T3000\Database
```

Backup

- Enable automatic backups: ✓
- Backup frequency: Daily at 2:00 AM
- Keep backups for: 30 days
- Backup location: `C:\Backups\T3000`

User Management

Create user accounts with appropriate permissions.

Administrator Account

- Full access to all features
- Can create/modify users
- Can change system settings

Operator Account

- View and control data points
- Cannot modify device settings
- Cannot add/remove devices

Viewer Account

- Read-only access
- Can view dashboards and trends
- Cannot make changes

Communication Protocols

Modbus Configuration

Modbus TCP/IP

```yaml
Protocol: Modbus TCP
Port: 502
Timeout: 5000 ms
Retries: 3
Poll Rate: 1000 ms
```

Modbus RTU (Serial)

```yaml
Port: COM1
Baud Rate: 19200
Data Bits: 8
Parity: None
Stop Bits: 1
```

BACnet Configuration

```yaml
Protocol: BACnet/IP
Port: 47808
Device Instance: 389001
Network Number: 0
Max APDU: 1476
```

System Preferences

Display Settings

- **Temperature Units**: °F / °C
- **Time Format**: 12-hour / 24-hour
- **Date Format**: MM/DD/YYYY
- **Decimal Places**: 1

Alarm Settings

Email Notifications

```text
SMTP Server: smtp.gmail.com
Port: 587
Username: alerts@yourdomain.com
Use TLS: ✓
```

Alert Priorities

- Critical: Immediate notification
- High: Within 5 minutes
- Medium: Within 15 minutes
- Low: Daily summary

Data Logging

Trend Log Settings

- Sample interval: 1 minute (adjustable)
- Storage period: 1 year
- Auto-archive: Enabled
- Compression: Enabled

Security Settings

Password Policy

- Minimum length: 8 characters
- Require uppercase: ✓
- Require numbers: ✓
- Require symbols: ○
- Password expiry: 90 days

Session Settings

- Timeout: 30 minutes
- Concurrent logins: Allowed
- Remember last login: ✓

Performance Optimization

Polling Optimization

Balance between real-time updates and network load:

Device Count	Recommended Poll Rate
1-10	1000 ms
11-50	2000 ms
51-100	5000 ms
100+	10000 ms

Database Maintenance

Schedule regular maintenance:

- **Compact database**: Weekly
- **Rebuild indexes**: Monthly
- **Clear old logs**: Quarterly

Exporting/Importing Configuration

Export Settings

1. Go to **Settings** → **System**
2. Click **Export Configuration**
3. Save as `T3000_Config_Backup.json`

Import Settings

1. Go to **Settings** → **System**
2. Click **Import Configuration**
3. Select saved `.json` file
4. Restart application when prompted

Configuration Best Practices

✅ **Do:**

- Back up configuration regularly
- Use strong passwords
- Enable audit logging
- Test network connectivity before adding devices

❌ **Don't:**

- Use default passwords in production
- Disable security features without reason
- Set poll rates too aggressively
- Ignore backup alerts

Next Steps

- Connecting Devices
- Creating Graphics
- Setting Up Schedules

Configuration Guide

Programmatic Configuration

Configuration File Format

JSON

```
{
  "version": "9.0",
  "network": {
    "adapters": [
      {
        "name": "Ethernet",
        "ipRange": "192.168.1.0/24",
        "gateway": "192.168.1.1",
        "dns": ["8.8.8.8", "8.8.4.4"]
      }
    ],
    "protocols": {
      "modbusTcp": {
        "enabled": true,
        "port": 502,
        "timeout": 5000,
        "retries": 3,
        "slaveIds": [1, 2, 3, 4, 5]
      },
      "bacnet": {
        "enabled": true,
        "port": 47808,
        "deviceInstance": 389001,
        "networkNumber": 0,
        "maxApdu": 1476,
        "segmentation": "both",
        "apduTimeout": 3000
      },
      "http": {
        "enabled": true,
        "port": 9103,
        "cors": true,
        "apiKey": "your-api-key-here"
      }
    }
  },
  "database": {
    "path": "C:/ProgramData/Temco/T3000/t3000.db",
    "backup": {
      "enabled": true,
      "schedule": "0 2 * * *",
      "retention": 30,
      "location": "C:/Backups/T3000"
    },
    "performance": {
      "cacheSize": 10000,
      "walMode": true,
      "synchronous": "NORMAL"
    }
  },
  "security": {
    "passwordPolicy": {
      "minLength": 8,
```

```
      "requireUppercase": true,
      "requireNumbers": true,
      "requireSymbols": false,
      "expiryDays": 90
    },
    "session": {
      "timeout": 1800,
      "allowConcurrent": true,
      "rememberLastLogin": true
    },
    "audit": {
      "enabled": true,
      "logLevel": "INFO",
      "retention": 365
    }
  },
  "performance": {
    "pollRate": 1000,
    "maxConcurrentRequests": 10,
    "requestTimeout": 30000,
    "cacheEnabled": true
  }
}
```

Loading Configuration

TYPESCRIPT

```typescript
import { loadConfig, validateConfig } from '@temco/t3000-config';

// Load configuration from file
const config = await loadConfig('./config.json');

// Validate configuration
const validation = validateConfig(config);
if (!validation.valid) {
  console.error('Invalid configuration:', validation.errors);
  process.exit(1);
}

// Apply configuration
await system.applyConfig(config);
```

Environment-Based Configuration

TYPESCRIPT

```typescript
// config/development.ts
export const devConfig = {
  network: {
    protocols: {
      modbusTcp: { port: 5020 },  // Non-standard port for dev
      bacnet: { port: 47809 },
      http: { port: 3003 }
    }
  },
  database: {
    path: './dev-data/t3000.db'
  },
  logging: {
    level: 'DEBUG'
  }
};

// config/production.ts
export const prodConfig = {
  network: {
    protocols: {
      modbusTcp: { port: 502 },
      bacnet: { port: 47808 },
      http: { port: 9103 }
    }
  },
  database: {
    path: 'C:/ProgramData/Temco/T3000/t3000.db'
  },
  logging: {
    level: 'INFO'
  }
};

// Load based on environment
const config = process.env.NODE_ENV === 'production'
  ? prodConfig
  : devConfig;
```

API-Based Configuration

REST API Configuration

SHELL

```shell
## Get current configuration
curl http://localhost:9103/api/config

## Update network settings
curl -X PATCH http://localhost:9103/api/config/network \
  -H "Content-Type: application/json" \
  -d '{
    "protocols": {
      "modbusTcp": {
        "enabled": true,
        "port": 502,
        "timeout": 5000
      }
    }
  }'

## Update database settings
curl -X PATCH http://localhost:9103/api/config/database \
  -H "Content-Type: application/json" \
  -d '{
    "backup": {
      "enabled": true,
      "schedule": "0 2 * * *"
    }
  }'
```

Python SDK Configuration

PYTHON

```python
from t3000 import T3000Client, Config

## Connect to T3000 instance
client = T3000Client(host='localhost', port=9103)

## Get current configuration
config = client.get_config()

## Modify configuration
config.network.protocols.modbus_tcp.port = 502
config.network.protocols.bacnet.port = 47808

## Apply configuration
client.set_config(config)

## Or update specific section
client.update_config('network.protocols.modbusTcp', {
    'enabled': True,
    'port': 502,
    'timeout': 5000
})
```

Database Schema Configuration

SQLite Optimization

SQL

```sql
-- Performance pragmas
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = 10000;
PRAGMA temp_store = MEMORY;
PRAGMA mmap_size = 30000000000;

-- Create indexes for common queries
CREATE INDEX IF NOT EXISTS idx_devices_serial
  ON devices(serial_number);

CREATE INDEX IF NOT EXISTS idx_points_device_type
  ON data_points(device_id, point_type);

CREATE INDEX IF NOT EXISTS idx_trends_device_time
  ON trend_data(device_id, timestamp DESC);

-- Analyze for query optimization
ANALYZE;
```

Database Migrations

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

interface Migration {
  version: number;
  up: string;
  down: string;
}

const migrations: Migration[] = [
  {
    version: 1,
    up: `CREATE TABLE devices (...)`,
    down: `DROP TABLE devices`
  },
  {
    version: 2,
    up: `ALTER TABLE devices ADD COLUMN firmware_version TEXT`,
    down: `ALTER TABLE devices DROP COLUMN firmware_version`
  }
];

function runMigrations(db: Database.Database) {
  // Create migrations table
  db.exec(`
    CREATE TABLE IF NOT EXISTS migrations (
      version INTEGER PRIMARY KEY,
      applied_at INTEGER
    )
  `);

  // Get current version
  const current = db.prepare(
    'SELECT MAX(version) as v FROM migrations'
  ).get() as { v: number };

  const currentVersion = current?.v || 0;

  // Apply pending migrations
  for (const migration of migrations) {
    if (migration.version > currentVersion) {
      db.exec(migration.up);
      db.prepare(
        'INSERT INTO migrations (version, applied_at) VALUES (?, ?)'
      ).run(migration.version, Date.now());

      console.log(`Applied migration ${migration.version}`);
    }
  }
}
```

Protocol-Specific Configuration

BACnet Advanced Configuration

TYPESCRIPT

```typescript
interface BACnetConfig {
  deviceInstance: number;
  networkNumber: number;
  macAddress?: string;
  maxApdu: 1476 | 1024 | 480 | 206 | 128 | 50;
  segmentation: 'both' | 'transmit' | 'receive' | 'none';
  apduTimeout: number;
  numberOfApduRetries: number;
  deviceId: number;
  objectName: string;
  modelName: string;
  firmwareRevision: string;
  applicationSoftwareVersion: string;
  location: string;
  description: string;
  vendorIdentifier: number;
}

const bacnetConfig: BACnetConfig = {
  deviceInstance: 389001,
  networkNumber: 0,
  maxApdu: 1476,
  segmentation: 'both',
  apduTimeout: 3000,
  numberOfApduRetries: 3,
  deviceId: 389001,
  objectName: 'T3000-Web-Server',
  modelName: 'T3000',
  firmwareRevision: '9.0.0',
  applicationSoftwareVersion: '9.0.0',
  location: 'Building 1',
  description: 'T3000 BACnet Gateway',
  vendorIdentifier: 373
};
```

Modbus Register Mapping

TYPESCRIPT

```typescript
interface ModbusRegisterMap {
  startAddress: number;
  count: number;
  pointType: string;
  conversion: (raw: number) => number;
}

const registerMaps: Record<string, ModbusRegisterMap> = {
  inputs: {
    startAddress: 0,
    count: 64,
    pointType: 'AI',
    conversion: (raw) => raw / 10.0  // Convert to engineering units
  },
  outputs: {
    startAddress: 100,
    count: 32,
    pointType: 'AO',
    conversion: (raw) => raw / 10.0
  },
  variables: {
    startAddress: 200,
    count: 200,
    pointType: 'VAR',
    conversion: (raw) => raw / 10.0
  }
};
```

Configuration Validation

TYPESCRIPT

```typescript
import Ajv from 'ajv';

const configSchema = {
  type: 'object',
  required: ['network', 'database'],
  properties: {
    network: {
      type: 'object',
      required: ['protocols'],
      properties: {
        protocols: {
          type: 'object',
          properties: {
            modbusTcp: {
              type: 'object',
              properties: {
                port: { type: 'number', minimum: 1, maximum: 65535 },
                timeout: { type: 'number', minimum: 1000 },
                retries: { type: 'number', minimum: 0, maximum: 10 }
              }
            }
          }
        }
      }
    },
    database: {
      type: 'object',
      required: ['path'],
      properties: {
        path: { type: 'string', minLength: 1 }
      }
    }
  }
};

const ajv = new Ajv();
const validate = ajv.compile(configSchema);

function validateConfiguration(config: unknown): boolean {
  const valid = validate(config);
  if (!valid) {
    console.error('Configuration validation errors:', validate.errors);
  }
  return valid;
}
```

Next Steps

- REST API Reference
- Device API
- Database Schema

Initial Configuration

After installation, configure these essential settings.

Network Settings

Navigate to **Settings → Network**

IP Configuration

```
Network Adapter: [Select your adapter]
IP Address Range: 192.168.1.1 - 192.168.1.254
Subnet Mask: 255.255.255.0
Gateway: 192.168.1.1
```

Protocol Settings

- ✓ Enable Modbus TCP (Port 502)
- ✓ Enable BACnet (Port 47808)
- ○ Enable HTTP API (Port 8080)

Database Settings

Location

```
Default: C:\ProgramData\Temco\T3000\Database
```

Backup

- Enable automatic backups: ✓
- Backup frequency: Daily at 2:00 AM
- Keep backups for: 30 days
- Backup location: `C:\Backups\T3000`

User Management

Create user accounts with appropriate permissions.

Administrator Account

- Full access to all features
- Can create/modify users
- Can change system settings

Operator Account

- View and control data points
- Cannot modify device settings
- Cannot add/remove devices

Viewer Account

- Read-only access
- Can view dashboards and trends
- Cannot make changes

Communication Protocols

Modbus Configuration

Modbus TCP/IP

```yaml
Protocol: Modbus TCP
Port: 502
Timeout: 5000 ms
Retries: 3
Poll Rate: 1000 ms
```

Modbus RTU (Serial)

```yaml
Port: COM1
Baud Rate: 19200
Data Bits: 8
Parity: None
Stop Bits: 1
```

BACnet Configuration

```yaml
Protocol: BACnet/IP
Port: 47808
Device Instance: 389001
Network Number: 0
Max APDU: 1476
```

System Preferences

Display Settings

- **Temperature Units**: °F / °C
- **Time Format**: 12-hour / 24-hour
- **Date Format**: MM/DD/YYYY
- **Decimal Places**: 1

Alarm Settings

Email Notifications

```text
SMTP Server: smtp.gmail.com
Port: 587
Username: alerts@yourdomain.com
Use TLS: ✓
```

Alert Priorities

- Critical: Immediate notification
- High: Within 5 minutes
- Medium: Within 15 minutes
- Low: Daily summary

Data Logging

Trend Log Settings

- Sample interval: 1 minute (adjustable)
- Storage period: 1 year
- Auto-archive: Enabled
- Compression: Enabled

Security Settings

Password Policy

- Minimum length: 8 characters
- Require uppercase: ✓
- Require numbers: ✓
- Require symbols: ○
- Password expiry: 90 days

Session Settings

- Timeout: 30 minutes
- Concurrent logins: Allowed
- Remember last login: ✓

Performance Optimization

Polling Optimization

Balance between real-time updates and network load:

Device Count	Recommended Poll Rate
1-10	1000 ms
11-50	2000 ms
51-100	5000 ms
100+	10000 ms

Database Maintenance

Schedule regular maintenance:

- **Compact database**: Weekly
- **Rebuild indexes**: Monthly
- **Clear old logs**: Quarterly

Exporting/Importing Configuration

Export Settings

1. Go to **Settings** → **System**
2. Click **Export Configuration**
3. Save as `T3000_Config_Backup.json`

Import Settings

1. Go to **Settings** → **System**
2. Click **Import Configuration**
3. Select saved `.json` file
4. Restart application when prompted

Next Steps

- Connecting Devices
- Creating Graphics
- Setting Up Schedules

Configuration Best Practices

✅ **Do:**

- Back up configuration regularly
- Use strong passwords
- Enable audit logging
- Test network connectivity before adding devices

❌ **Don't:**

- Use default passwords in production
- Disable security features without reason
- Set poll rates too aggressively
- Ignore backup alerts

Quick Start Overview

This guide will help you get started with T3000 Building Automation System quickly and efficiently.

What is T3000?

T3000 is a powerful building automation software platform designed to:

- **Monitor** - Track all building systems in real-time
- **Control** - Adjust setpoints and schedules remotely
- **Analyze** - Review historical data and trends
- **Optimize** - Improve energy efficiency and comfort

Installation Steps

1. **Download** - Get the latest version from the downloads page
2. **Install** - Run the installer and follow the wizard
3. **Configure** - Set up network and communication settings
4. **Connect** - Add your first device to start monitoring

Basic Workflow

TEXT

```
1. Connect Devices
   ↓
2. Configure Data Points
   ↓
3. Create Graphics/Dashboards
   ↓
4. Set Up Schedules
   ↓
5. Configure Alarms
```

First Steps

1. Launch T3000

After installation, launch the application from your Start menu or desktop shortcut.

2. Network Setup

Configure your network settings to communicate with devices:

- IP Address range
- Subnet mask
- Communication protocol (Modbus TCP/RTU, BACnet)

3. Add Your First Device

Navigate to **Device Management** and click **Add Device** to scan for and connect to devices on your network.

4. Explore the Interface

Familiarize yourself with the main sections:

- **Dashboard** - Overview of system status
- **Devices** - Manage connected controllers
- **Data Points** - View and control inputs, outputs, variables
- **Graphics** - Visual representations of your systems
- **Trends** - Historical data charts

Common Use Cases

HVAC Control: Monitor temperature, humidity, and control setpoints across multiple zones.

Energy Management: Track energy consumption and identify optimization opportunities.

Alarm Monitoring: Get notified of critical events in real-time.

Scheduling: Automate building operations based on occupancy and time of day.

Next Steps

- Installation Guide - Detailed installation instructions
- Configuration - Configure your system settings
- Device Management - Connect your first device

Getting Help

If you encounter any issues:

- Check the FAQ
- Review Troubleshooting Guide
- Contact support@temcocontrols.com

Quick Start Overview

System Architecture

T3000 Web is a modern building automation platform built on:

Frontend Stack:

- React/TypeScript for UI components
- Vite for build tooling
- WebSocket for real-time communication

Backend Stack:

- Rust API server (Port 9103)
- SQLite database for persistence
- BACnet/Modbus protocol handlers

Communication Protocols:

- BACnet/IP (UDP 47808)
- Modbus TCP (TCP 502)
- WebSocket API (WS 9103)
- REST API (HTTP 9103)

Quick Start for Developers

1. Installation via Package Manager

```shell
## Download and extract
curl -O https://temcocontrols.com/downloads/t3000-web-v9.0.zip
unzip t3000-web-v9.0.zip -d /opt/t3000

## Run installer with silent mode
sudo ./T3000_Setup_v9.0.exe /S

## Or using Chocolatey (Windows)
choco install t3000-building-automation
```

2. Environment Configuration

Create configuration file `config.json`:

JSON

```
{
  "network": {
    "adapter": "Ethernet",
    "ipRange": "192.168.1.0/24",
    "protocols": {
      "modbusTcp": { "enabled": true, "port": 502 },
      "bacnet": { "enabled": true, "port": 47808 },
      "http": { "enabled": true, "port": 9103 }
    }
  },
  "database": {
    "path": "./data/t3000.db",
    "backup": {
      "enabled": true,
      "interval": "daily",
      "retention": 30
    }
  }
}
```

3. Programmatic Device Discovery

TYPESCRIPT

```
import { T3000Client } from '@temco/t3000-sdk';

const client = new T3000Client({
  host: 'localhost',
  port: 9103
});

// Discover devices
const devices = await client.discover({
  protocol: 'bacnet',
  timeout: 30000,
  subnet: '192.168.1.0/24'
});

console.log(`Found ${devices.length} devices`);

// Connect to device
const device = await client.connect({
  ip: '192.168.1.100',
  port: 47808,
  deviceId: 389001
});
```

4. API Integration Example

JAVASCRIPT

```javascript
// WebSocket connection for real-time data
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1']
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  console.log(`${data.point}: ${data.value}`);
};
```

5. System Initialization

PYTHON

```python
from t3000 import T3000System

## Initialize system
system = T3000System(config_path='config.json')

## Start services
system.start_api_server(port=9103)
system.start_bacnet_service()
system.start_modbus_service()

## Load devices from database
devices = system.load_devices()
print(f"Loaded {len(devices)} devices")

## Enable auto-discovery
system.enable_discovery(interval=300)  # 5 minutes
```

Integration Patterns

REST API Usage

SHELL

```shell
## Get device list
curl http://localhost:9103/api/devices

## Read data point
curl http://localhost:9103/api/devices/389001/points/IN1

## Write output value
curl -X PUT http://localhost:9103/api/devices/389001/points/OUT1 \
  -H "Content-Type: application/json" \
  -d '{"value": 75.0}'
```

Database Schema

```sql
-- Devices table
CREATE TABLE devices (
  id INTEGER PRIMARY KEY,
  serial_number INTEGER UNIQUE,
  ip_address TEXT,
  port INTEGER,
  protocol TEXT,
  status TEXT
);

-- Data points table
CREATE TABLE data_points (
  id INTEGER PRIMARY KEY,
  device_id INTEGER,
  point_type TEXT,
  label TEXT,
  value REAL,
  units TEXT,
  last_update INTEGER,
  FOREIGN KEY (device_id) REFERENCES devices(id)
);
```

Advanced Configuration

Custom Protocol Handler

```rust
use t3000_api::protocol::{ProtocolHandler, Device};

struct CustomProtocol;

impl ProtocolHandler for CustomProtocol {
    async fn discover(&self) -> Vec<Device> {
        // Custom discovery logic
        vec![]
    }

    async fn read_point(&self, device: &Device, point: &str) -> f64 {
        // Custom read logic
        0.0
    }
}

// Register handler
system.register_protocol("custom", CustomProtocol);
```

Next Steps

- REST API Reference
- WebSocket API
- Modbus Protocol

Inputs

Monitor and manage analog and digital input points for sensors and measurements.

Overview

Input points represent physical sensors connected to the BACnet device, including temperature sensors, pressure transducers, flow meters, and digital status inputs.

Input Types

Analog Inputs (AI)

Continuous measurement values:

Common Sensor Types:

- **Temperature**: Thermistors, RTDs, thermocouples
- **Pressure**: 4-20mA transducers, 0-10V sensors
- **Flow**: Differential pressure, ultrasonic
- **Humidity**: Relative humidity sensors
- **CO2**: Carbon dioxide sensors (PPM)
- **Voltage/Current**: Direct electrical measurements

Properties:

- **Present Value**: Current reading
- **Units**: Engineering units (°F, °C, PSI, CFM, etc.)
- **Range**: Min/max values (e.g., 0-100%)
- **Filter**: Averaging time constant
- **Calibration**: Offset and gain

Binary Inputs (BI)

On/off status readings:

Common Uses:

- **Status Switches**: Proof of operation
- **Door Contacts**: Open/closed sensors
- **Limit Switches**: End-of-travel indicators
- **Alarm Contacts**: Safety signals
- **Occupancy Sensors**: Motion detection

States:

- **Active** (1, On, Closed)
- **Inactive** (0, Off, Open)

Inputs Page Interface

Table View

#	Label	Description	Value	Units	Status	Auto/Manual	Last Update
1	Zone_Temp	Zone Temperature	72.5	°F	✅ Valid	Auto	2s ago
2	Supply_Press	Supply Pressure	45.2	PSI	✅ Valid	Auto	2s ago
3	Return_Temp	Return Air Temp	68.0	°F	✅ Valid	Auto	2s ago
4	CO2_Level	CO2 Concentration	650	PPM	✅ Valid	Auto	2s ago
5	Occupancy	Room Occupied	Active	-	✅ Valid	Auto	2s ago

Features

Real-Time Updates:

- Auto-refresh every 5 seconds (configurable)
- Manual refresh with 🔄 button
- Loading indicator during refresh

Batch Operations:

- Select multiple points
- Batch save changes
- Export selected points

Filtering & Search:

- Quick search by name/description
- Filter by type (AI/BI)
- Filter by status (valid/fault/override)
- Sort by any column

Input Configuration

Basic Settings

Click on an input row to configure:

Identification:

- **Label**: User-friendly name (e.g., "Zone Temperature")
- **Description**: Detailed description
- **Panel**: Associated control panel
- **Floor**: Physical location

Hardware:

- **Sensor Type**: Select sensor model
- **Range**: Measurement range (0-10V, 4-20mA, etc.)
- **Units**: Engineering units
- **Resolution**: Decimal places

Advanced Settings

Calibration:

```
TEXT
```

```
Raw Value:     5.25V
Offset:        -0.05V
Gain:          1.02
Calibrated:    5.26V
Scaled Value:  72.5°F
```

Filtering:

- **Filter Time**: Smoothing period (seconds)
- **Average Samples**: Number of readings to average
- **Deadband**: Minimum change to report

Limits:

- **High Limit**: Maximum acceptable value
- **Low Limit**: Minimum acceptable value
- **High Alarm**: Alarm threshold (upper)

- **Low Alarm**: Alarm threshold (lower)

Data Quality

Status Indicators

- ✅ **Valid**: Normal operation
- ⚠️ **Uncertain**: Questionable data quality
- ❌ **Fault**: Sensor error or out of range
- 🔧 **Override**: Manual override active

Fault Detection

Common Faults:

- **Open Circuit**: Sensor disconnected
- **Short Circuit**: Wiring fault
- **Out of Range**: Value exceeds limits
- **Communication Loss**: No recent updates

Fault Response:

- Display fault status
- Generate alarm
- Use default value (if configured)
- Notify operators

Input Trends

Real-Time Monitoring

View input trends in real-time:

1. Navigate to **Trend Logs** page
2. Select input points to chart
3. View live updating graphs
4. Zoom/pan to analyze patterns

Historical Data

Access stored input data:

- **Time Range**: Select date/time period
- **Export**: Download CSV/Excel
- **Statistics**: Min, max, avg, std dev
- **Comparison**: Plot multiple inputs

Alarms

Alarm Configuration

Set alarm thresholds for inputs:

High Alarm:

```
TEXT

Setpoint:     80°F
Deadband:     2°F
Priority:     Medium
Delay:        60 seconds
```

Low Alarm:

```
Setpoint:     65°F
Deadband:     2°F
Priority:     Low
Delay:        60 seconds
```

Alarm Actions

When alarm triggers:

- Visual indication in inputs table
- Entry in alarm log
- Email/SMS notification (if configured)
- Audible alert (if enabled)

Best Practices

Naming Conventions

Use consistent naming:

```
Format: [Location]_[Function]_[Measurement]

Examples:
- AHU1_Zone_Temp
- Boiler_Supply_Press
- Floor3_CO2_Level
```

Calibration

- Calibrate sensors annually
- Document calibration date
- Compare to known reference
- Record offset/gain values

Maintenance

1. **Regular Checks**
   - Verify readings match physical conditions
   - Check for drift over time
   - Test alarm functions
2. **Documentation**
   - Maintain sensor specifications
   - Record installation dates
   - Track maintenance history
3. **Spare Parts**
   - Stock common sensor types
   - Keep calibration tools available
   - Document sensor part numbers

Troubleshooting

Reading Errors

Symptom: Incorrect or unstable readings

Solutions:

1. Check sensor wiring
2. Verify sensor type configuration
3. Review calibration settings
4. Test with multimeter
5. Replace faulty sensor

No Updates

Symptom: Stale data, no refresh

Solutions:

1. Check device connection
2. Verify auto-refresh enabled
3. Check polling interval
4. Review communication logs

Alarm Nuisance

Symptom: False alarms

Solutions:

1. Adjust alarm thresholds
2. Increase deadband
3. Add time delay
4. Improve sensor filtering

Next Steps

- Outputs - Control output points
- Variables - Calculated variables
- Trend Logs - Historical analysis
- Alarms - Alarm management

Inputs

BACnet Input Objects

Reading Input Values via API

TYPESCRIPT

```typescript
// Read single input
const response = await fetch(
  `http://localhost:9103/api/devices/389001/points/IN1`
);
const data = await response.json();

console.log(`Value: ${data.value} ${data.units}`);
console.log(`Quality: ${data.quality}`);
console.log(`Timestamp: ${new Date(data.timestamp)}`);

// Read multiple inputs in batch
const points = ['IN1', 'IN2', 'IN3', 'IN4'];
const batchResponse = await fetch(
  `http://localhost:9103/api/devices/389001/points/batch`,
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ points })
  }
);
```

BACnet Property Reading

TYPESCRIPT

```typescript
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({ port: 47808 });

// Read present-value property
const value = await client.readProperty({
  deviceInstance: 389001,
  objectType: 'analog-input',
  objectInstance: 1,
  property: 'present-value'
});

// Read multiple properties
const properties = await client.readPropertyMultiple({
  deviceInstance: 389001,
  objects: [
    {
      objectId: { type: 'analog-input', instance: 1 },
      properties: [
        'present-value',
        'status-flags',
        'out-of-service',
        'units',
        'reliability'
      ]
    }
  ]
});
```

Input Signal Processing

Analog Input Scaling

TYPESCRIPT

```typescript
interface ScalingConfig {
  rawMin: number;
  rawMax: number;
  engMin: number;
  engMax: number;
  offset: number;
  gain: number;
}

function scaleAnalogInput(rawValue: number, config: ScalingConfig): number {
  // Apply offset and gain calibration
  const calibrated = (rawValue + config.offset) * config.gain;

  // Scale to engineering units
  const scaled = config.engMin +
    ((calibrated - config.rawMin) / (config.rawMax - config.rawMin)) *
    (config.engMax - config.engMin);

  return scaled;
}

// Example: 0-10V sensor scaled to 0-100°F with calibration
const config: ScalingConfig = {
  rawMin: 0,
  rawMax: 10,
  engMin: 0,
  engMax: 100,
  offset: -0.05,  // -0.05V offset
  gain: 1.02      // 2% gain correction
};

const temperature = scaleAnalogInput(5.25, config); // Returns ~52.6°F
```

Digital Filtering

TYPESCRIPT

```typescript
class ExponentialFilter {
  private filtered: number;
  private alpha: number;

  constructor(filterTime: number, sampleRate: number) {
    // Calculate alpha from filter time constant
    this.alpha = 1 - Math.exp(-sampleRate / filterTime);
    this.filtered = 0;
  }

  update(newValue: number): number {
    if (this.filtered === 0) {
      this.filtered = newValue;  // Initialize
    } else {
      this.filtered = this.alpha * newValue + (1 - this.alpha) * this.filtered;
    }
    return this.filtered;
  }
}

// Example: 10-second filter at 1Hz sample rate
const filter = new ExponentialFilter(10, 1);

// Apply to incoming sensor data
const rawValues = [72.1, 72.5, 71.8, 72.3, 72.0];
const filtered = rawValues.map(v => filter.update(v));
console.log(filtered); // Smoothed values
```

Moving Average Filter

TYPESCRIPT

```typescript
class MovingAverageFilter {
  private samples: number[] = [];
  private size: number;

  constructor(windowSize: number) {
    this.size = windowSize;
  }

  update(newValue: number): number {
    this.samples.push(newValue);

    if (this.samples.length > this.size) {
      this.samples.shift();
    }

    return this.samples.reduce((a, b) => a + b, 0) / this.samples.length;
  }
}
```

Fault Detection Algorithms

Range Checking

TYPESCRIPT

```typescript
interface InputLimits {
  sensorMin: number;
  sensorMax: number;
  physicalMin: number;
  physicalMax: number;
}

function detectRangeFault(value: number, limits: InputLimits): string | null {
  if (value < limits.sensorMin || value > limits.sensorMax) {
    return 'SENSOR_FAULT';  // Hardware fault
  }

  if (value < limits.physicalMin || value > limits.physicalMax) {
    return 'OUT_OF_RANGE';  // Physically impossible value
  }

  return null;  // No fault
}

// Example: Temperature sensor
const limits: InputLimits = {
  sensorMin: -40,    // Sensor spec
  sensorMax: 140,
  physicalMin: 50,   // Expected building range
  physicalMax: 90
};

const fault = detectRangeFault(150, limits); // Returns 'SENSOR_FAULT'
```

Rate of Change Detection

TYPESCRIPT

```typescript
class RateOfChangeDetector {
  private lastValue: number | null = null;
  private lastTime: number | null = null;

  check(value: number, maxRate: number): boolean {
    const now = Date.now();

    if (this.lastValue !== null && this.lastTime !== null) {
      const deltaValue = Math.abs(value - this.lastValue);
      const deltaTime = (now - this.lastTime) / 1000; // seconds
      const rate = deltaValue / deltaTime;

      if (rate > maxRate) {
        return true;  // Excessive rate of change
      }
    }

    this.lastValue = value;
    this.lastTime = now;
    return false;
  }
}

// Example: Detect temperature changing faster than 5°F/minute
const rateDetector = new RateOfChangeDetector();
const excessive = rateDetector.check(78, 5/60); // 5°F per minute = 0.083°F/s
```

Data Logging and Storage

High-Performance Trend Logging

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendLogger {
  private db: Database.Database;
  private buffer: Map<string, Array<{value: number, timestamp: number}>>;
  private batchSize = 100;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.buffer = new Map();

    // Create optimized table
    this.db.exec(`
      CREATE TABLE IF NOT EXISTS input_trends (
        device_id INTEGER NOT NULL,
        point_id TEXT NOT NULL,
        value REAL NOT NULL,
        timestamp INTEGER NOT NULL
      );

      CREATE INDEX IF NOT EXISTS idx_input_trends
        ON input_trends(device_id, point_id, timestamp DESC);
    `);
  }

  log(deviceId: number, pointId: string, value: number) {
    const key = `${deviceId}:${pointId}`;

    if (!this.buffer.has(key)) {
      this.buffer.set(key, []);
    }

    this.buffer.get(key)!.push({
      value,
      timestamp: Date.now()
    });

    // Flush when buffer is full
    if (this.buffer.get(key)!.length >= this.batchSize) {
      this.flush(deviceId, pointId);
    }
  }

  private flush(deviceId: number, pointId: string) {
    const key = `${deviceId}:${pointId}`;
    const data = this.buffer.get(key);

    if (!data || data.length === 0) return;

    const insert = this.db.prepare(`
      INSERT INTO input_trends (device_id, point_id, value, timestamp)
      VALUES (?, ?, ?, ?)
    `);

    const transaction = this.db.transaction((entries) => {
```

```
      for (const entry of entries) {
        insert.run(deviceId, pointId, entry.value, entry.timestamp);
      }
    });

    transaction(data);
    this.buffer.set(key, []);
  }
}
```

WebSocket Real-Time Streaming

TYPESCRIPT

```typescript
// Subscribe to input updates
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'IN3'],
    changeOfValue: 0.5,  // Only send if value changes by 0.5 or more
    interval: 5000       // Maximum update interval (ms)
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'point-update') {
    console.log(`${data.point}: ${data.value} (COV: ${data.cov})`);
    updateChart(data.point, data.value, data.timestamp);
  }
};
```

Next Steps

- REST API Reference
- WebSocket API
- Modbus Protocol

Outputs

Control analog and digital output points for actuators, dampers, valves, and equipment.

Overview

Output points represent physical control signals sent to actuators, variable frequency drives (VFDs), dampers, valves, and other controlled devices. Outputs can operate in automatic mode (controlled by programs) or manual override mode.

Output Types

Analog Outputs (AO)

Continuous control signals:

Common Applications:

- **Damper Control**: Modulating dampers (0-100%)
- **Valve Control**: Hot/cold water valves
- **VFD Speed Control**: Fan/pump speed (0-100%)
- **Setpoint Signals**: Temperature/pressure setpoints
- **4-20mA Signals**: Pneumatic transducers
- **0-10V Signals**: Electronic actuators

Properties:

- **Command Value**: Desired output (0-100%)
- **Feedback Value**: Actual position reading
- **Units**: % (percent), or engineering units
- **Range**: Min/max output limits
- **Override**: Manual control capability

Binary Outputs (BO)

On/off control signals:

Common Applications:

- **Equipment Start/Stop**: Fans, pumps, heaters
- **Relay Control**: Contactors, solenoids
- **Enable/Disable**: System enable signals
- **Alarm Outputs**: Strobe lights, horns
- **Safety Shutoffs**: Emergency stops

States:

- **Active** (1, On, Energized)
- **Inactive** (0, Off, De-energized)

Outputs Page Interface

Table View

#	Label	Description	Command	Feedback	Mode	Override	Status	Update
1	Damper_Pos	Supply Damper	45%	44%	Auto	No	✅ Normal	2s ago
2	Fan_Speed	Supply Fan VFD	75%	75%	Auto	No	✅ Normal	2s ago
3	HW_Valve	Hot Water Valve	0%	0%	Off	Yes	🔧 Override	2s ago
4	Pump_Run	Chilled Water Pump	On	On	Auto	No	✅ Normal	2s ago

Features

Real-Time Control:

- View current command values
- See actual feedback positions
- Monitor mode status (Auto/Manual/Off)
- Identify override conditions

Manual Override:

- Click output row to override
- Set manual value
- Return to automatic mode
- Emergency stop capability

Batch Operations:

- Select multiple outputs
- Batch mode changes
- Release all overrides
- Export configuration

Output Control

Operating Modes

Automatic Mode:

- Output controlled by programs
- Normal operating condition
- Responds to control algorithms
- Cannot be manually adjusted

Manual Mode:

- User sets output value directly
- Program control disabled
- Value held until released
- Indicated with 🔧 icon

Off Mode:

- Output disabled
- Fixed at 0% or Off state
- Safety/maintenance condition
- Must be manually enabled

Manual Override

To override an output:

1. Click on output row
2. Select **Manual Override** tab
3. Set desired value (0-100% or On/Off)
4. Click **Apply Override**
5. Output shows 🔧 override indicator

Release Override:

1. Click overridden output
2. Click **Release Override** button
3. Output returns to automatic mode

⚠ **Warning**: Manual overrides persist until explicitly released. Remember to return outputs to Auto mode after maintenance.

Output Configuration

Basic Settings

Identification:

- **Label**: Output name (e.g., "Supply_Damper")
- **Description**: Detailed description
- **Control Type**: Modulating/Two-Position
- **Panel**: Parent control panel

Control Parameters:

- **Range**: Output range (0-100%, 0-10V, 4-20mA)
- **Default**: Default value when disabled
- **Priority**: Control priority level
- **Relinquish**: Auto-release timeout

Advanced Settings

PID Tuning (if applicable):

```
TEXT
```

```
Proportional (P):  2.0
Integral (I):      0.5
Derivative (D):    0.1
Setpoint:          72°F
Deadband:          1°F
```

Limits:

- **High Limit**: Maximum allowed output
- **Low Limit**: Minimum allowed output
- **Rate Limit**: Maximum change per second
- **Deadband**: Minimum change threshold

Safety:

- **Failsafe Value**: Value on communication loss
- **Timeout**: Watchdog timer (seconds)
- **Lockout**: Disable manual override
- **Priority Array**: BACnet priority levels

Feedback Monitoring

Position Feedback

Many outputs have feedback sensors:

Feedback Types:

- **Position Sensor**: Actual damper/valve position
- **Speed Sensor**: VFD actual speed
- **Flow Switch**: Proof of flow
- **Status Contact**: Relay feedback

Feedback Comparison:

```
Command:    75%
Feedback:   73%
Error:      2%
Status:     ✅ Normal (within tolerance)
```

Feedback Errors:

- Command-feedback difference > tolerance
- Indicates mechanical issue
- Generates alarm
- May indicate:
  - Stuck actuator
  - Broken linkage
  - Sensor failure
  - Insufficient power

Output Sequences

Start/Stop Sequences

Equipment Start Sequence:

1. Pre-start checks
2. Enable output
3. Ramp to setpoint
4. Verify feedback
5. Confirm running status

Equipment Stop Sequence:

1. Ramp down gradually
2. Allow time delay
3. Disable output
4. Verify off status
5. Log stop event

Interlocks

Safety Interlocks:

- Prevent unsafe conditions
- Require prerequisite states
- Automatic shutdown on fault
- Override prevention

Example: VFD Interlock

```
IF (Filter_Status == Dirty) THEN
    VFD_Enable = OFF
    Alarm_Active = TRUE
END IF
```

Trending

Output Trends

Monitor output behavior:

Trend Charts:

- Command vs. Feedback comparison
- Mode changes over time
- Override history
- Performance analysis

Useful Metrics:

- Average position
- Cycling frequency
- Override duration
- Error magnitude

Alarms

Output Alarms

Common alarm conditions:

Feedback Error:

```
TEXT

Condition:   |Command - Feedback| > 10%
Duration:    > 60 seconds
Priority:    Medium
Action:      Notify, log event
```

Override Active:

```
TEXT

Condition:   Output in manual mode
Duration:    > 2 hours
Priority:    Low
Action:      Remind to release
```

Safety Limit:

```
TEXT

Condition:   Output at high/low limit
Duration:    > 5 minutes
Priority:    High
Action:      Alert maintenance
```

Best Practices

Naming Conventions

Use consistent output naming:

```
TEXT

Format: [Equipment]_[Function]_[Type]

Examples:
- AHU1_Supply_Damper
- Boiler1_HW_Valve
- ExhFan_Speed_VFD
- Pump2_Run_Command
```

Control Strategy

1. **Gradual Changes**
   - Use rate limiting
   - Avoid abrupt changes
   - Prevent equipment stress
2. **Feedback Verification**
   - Monitor feedback vs. command
   - Set appropriate tolerances
   - Investigate persistent errors
3. **Override Management**
   - Document override reason
   - Set time limit reminders
   - Review daily overrides
   - Release after maintenance

Safety Considerations

1. **Emergency Override**
   - Know how to quickly disable outputs
   - Test emergency stop procedures
   - Document shutdown sequences
2. **Failsafe Settings**
   - Configure safe default values
   - Set appropriate timeouts
   - Test communication loss behavior
3. **Authorization**
   - Limit override permissions
   - Log all manual interventions
   - Review override history

Troubleshooting

Output Not Responding

Symptoms: Command changes but no response

Solutions:

1. Check device connection
2. Verify not in manual override
3. Test output in manual mode
4. Check wiring and power
5. Verify output enable status

Feedback Mismatch

Symptoms: Command ≠ Feedback

Solutions:

1. Check feedback sensor wiring
2. Calibrate position feedback
3. Verify actuator is working
4. Check for mechanical binding

5. Test actuator manually

Unstable Control

Symptoms: Output hunting/oscillating

Solutions:

1. Tune PID parameters
2. Increase deadband
3. Add rate limiting
4. Check sensor noise
5. Verify loop stability

Next Steps

- Variables - Calculated variables
- Programs - Control programs
- PID Loops - PID control
- Schedules - Time-based control

Outputs

BACnet Output Control

Writing Output Values

TYPESCRIPT

```typescript
// Write single output via REST API
await fetch(`http://localhost:9103/api/devices/389001/points/OUT1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    value: 75.0,
    priority: 8  // Manual operator (BACnet priority)
  })
});

// BACnet write property
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({ port: 47808 });

await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-output',
  objectInstance: 1,
  property: 'present-value',
  value: 75.0,
  priority: 8
});
```

BACnet Priority Array

TYPESCRIPT

```typescript
interface PriorityLevel {
  priority: number;
  value: number | null;
  description: string;
}

const priorityArray: PriorityLevel[] = [
  { priority: 1, value: null, description: 'Manual Life Safety' },
  { priority: 2, value: null, description: 'Automatic Life Safety' },
  { priority: 3, value: null, description: 'Available' },
  { priority: 4, value: null, description: 'Available' },
  { priority: 5, value: null, description: 'Critical Equipment' },
  { priority: 6, value: null, description: 'Minimum On/Off' },
  { priority: 7, value: null, description: 'Available' },
  { priority: 8, value: 75.0, description: 'Manual Operator' },
  { priority: 9, value: null, description: 'Available' },
  { priority: 10, value: 72.0, description: 'Auto Control' },
  { priority: 11, value: null, description: 'Available' },
  { priority: 12, value: null, description: 'Available' },
  { priority: 13, value: null, description: 'Available' },
  { priority: 14, value: null, description: 'Available' },
  { priority: 15, value: null, description: 'Available' },
  { priority: 16, value: 50.0, description: 'Default/Relinquish' }
];

// Effective value is the highest priority non-null value
function getEffectiveValue(priorityArray: PriorityLevel[]): number {
  for (const level of priorityArray) {
    if (level.value !== null) {
      return level.value;
    }
  }
  return 0; // Failsafe if all null
}
```

Relinquish Override

TYPESCRIPT

```typescript
// Release manual override (write null at priority 8)
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-output',
  objectInstance: 1,
  property: 'present-value',
  value: null,  // null relinquishes this priority
  priority: 8
});
```

Output Control Algorithms

PID Output Calculation

TYPESCRIPT

```typescript
class PIDController {
  private integral = 0;
  private lastError = 0;
  private lastTime = Date.now();

  constructor(
    private kp: number,
    private ki: number,
    private kd: number,
    private outputMin: number = 0,
    private outputMax: number = 100
  ) {}

  calculate(setpoint: number, processValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000; // seconds

    const error = setpoint - processValue;

    // Proportional term
    const p = this.kp * error;

    // Integral term with anti-windup
    this.integral += error * dt;
    const i = this.ki * this.integral;

    // Derivative term
    const derivative = (error - this.lastError) / dt;
    const d = this.kd * derivative;

    // Calculate output
    let output = p + i + d;

    // Apply limits and anti-windup
    if (output > this.outputMax) {
      output = this.outputMax;
      this.integral -= error * dt; // Prevent windup
    } else if (output < this.outputMin) {
      output = this.outputMin;
      this.integral -= error * dt;
    }

    this.lastError = error;
    this.lastTime = now;

    return output;
  }

  reset() {
    this.integral = 0;
    this.lastError = 0;
  }
}
```

Rate Limiting

TYPESCRIPT

```typescript
class RateLimiter {
  private lastValue: number;
  private lastTime: number;

  constructor(
    initialValue: number,
    private maxRate: number  // Max change per second
  ) {
    this.lastValue = initialValue;
    this.lastTime = Date.now();
  }

  limit(targetValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000;

    const maxChange = this.maxRate * dt;
    const requestedChange = targetValue - this.lastValue;

    let actualChange = requestedChange;
    if (Math.abs(requestedChange) > maxChange) {
      actualChange = maxChange * Math.sign(requestedChange);
    }

    this.lastValue += actualChange;
    this.lastTime = now;

    return this.lastValue;
  }
}

// Example: Limit damper to 10% per second
const limiter = new RateLimiter(0, 10);

const command = 75;
const limited = limiter.limit(command);  // Gradually ramps to target
```

Deadband Control

TYPESCRIPT

```typescript
function applyDeadband(
  currentOutput: number,
  targetOutput: number,
  deadband: number
): number {
  const difference = Math.abs(targetOutput - currentOutput);

  if (difference < deadband) {
    return currentOutput;  // No change within deadband
  }

  return targetOutput;
}

// Example: 2% deadband on damper control
const current = 45.0;
const target = 46.5;
const output = applyDeadband(current, target, 2.0);  // Returns 46.5
```

Feedback Monitoring

Command-Feedback Comparison

TYPESCRIPT

```typescript
interface FeedbackStatus {
  command: number;
  feedback: number;
  error: number;
  errorPercent: number;
  status: 'normal' | 'warning' | 'fault';
}

function checkFeedback(
  command: number,
  feedback: number,
  tolerance: number = 5.0  // Percent
): FeedbackStatus {
  const error = Math.abs(command - feedback);
  const errorPercent = (error / command) * 100;

  let status: 'normal' | 'warning' | 'fault' = 'normal';
  if (errorPercent > tolerance * 2) {
    status = 'fault';
  } else if (errorPercent > tolerance) {
    status = 'warning';
  }

  return {
    command,
    feedback,
    error,
    errorPercent,
    status
  };
}

// Example: Check VFD feedback
const status = checkFeedback(75.0, 73.0, 5.0);
console.log(`Error: ${status.errorPercent.toFixed(1)}%, Status: ${status.status}`);
```

Stuck Output Detection

TYPESCRIPT

```typescript
class StuckOutputDetector {
  private history: number[] = [];
  private maxSamples = 20;

  update(feedback: number): boolean {
    this.history.push(feedback);

    if (this.history.length > this.maxSamples) {
      this.history.shift();
    }

    if (this.history.length < this.maxSamples) {
      return false;  // Not enough data
    }

    // Check if all values are identical
    const first = this.history[0];
    const allSame = this.history.every(v => Math.abs(v - first) < 0.1);

    return allSame;  // True if stuck
  }
}
```

Output Sequencing

Equipment Start Sequence

TYPESCRIPT

```typescript
async function startEquipment(outputId: string): Promise<boolean> {
  try {
    // 1. Pre-start checks
    const safeToStart = await performSafetyChecks(outputId);
    if (!safeToStart) {
      console.error('Safety checks failed');
      return false;
    }

    // 2. Enable output
    await writeOutput(outputId, 100, 8);

    // 3. Wait for proof of operation
    await sleep(5000);

    // 4. Verify feedback
    const feedback = await readFeedback(outputId);
    if (feedback < 90) {
      console.error('Failed to start - no feedback');
      await writeOutput(outputId, 0, 8);
      return false;
    }

    // 5. Log start event
    await logEvent('equipment-start', { outputId, timestamp: Date.now() });

    return true;
  } catch (error) {
    console.error('Start sequence failed:', error);
    return false;
  }
}
```

Interlock Logic

TYPESCRIPT

```typescript
class InterlockManager {
  private interlocks = new Map<string, () => boolean>();

  addInterlock(outputId: string, condition: () => boolean) {
    this.interlocks.set(outputId, condition);
  }

  checkInterlocks(outputId: string): boolean {
    const check = this.interlocks.get(outputId);
    if (!check) return true;  // No interlock defined

    return check();  // Returns true if safe to operate
  }

  async writeWithInterlock(outputId: string, value: number): Promise<boolean> {
    if (!this.checkInterlocks(outputId)) {
      console.warn(`Interlock prevents operation of ${outputId}`);
      return false;
    }

    await writeOutput(outputId, value, 10);
    return true;
  }
}

// Example: VFD interlock with filter status
const interlocks = new InterlockManager();

interlocks.addInterlock('VFD1', () => {
  const filterClean = readInput('Filter_Status') === 0;
  const smokeAlarm = readInput('Smoke_Alarm') === 0;
  return filterClean && !smokeAlarm;
});
```

Next Steps

- REST API Reference
- Modbus Register Mapping
- PID Loop Tuning

PID Loops

Proportional-Integral-Derivative control loops for precise automatic control.

Overview

PID loops provide precise automatic control by continuously calculating an error value as the difference between a desired setpoint and a measured process variable. The controller attempts to minimize the error by adjusting the control output.

PID Basics

Control Equation

```
TEXT

Output = P×Error + I×∫Error×dt + D×(dError/dt)
```

Where:

- **P** (Proportional): Immediate response to current error
- **I** (Integral): Eliminates steady-state error over time
- **D** (Derivative): Dampens oscillations and overshoot

PID Components

Proportional (P):

- Responds to current error magnitude
- Higher P = faster response, more aggressive
- Too high causes oscillation

Integral (I):

- Eliminates offset error
- Accumulates error over time
- Too high causes overshoot and instability

Derivative (D):

- Predicts future error based on rate of change
- Reduces overshoot and settling time
- Sensitive to noise

PID Loop Configuration

Basic Settings

Process:

- **Input**: Measured variable (e.g., Zone Temperature)
- **Output**: Control signal (e.g., Valve Position)
- **Setpoint**: Desired target value
- **Units**: Engineering units

Tuning Parameters:

```
TEXT

Proportional (P): 2.0
Integral (I):     0.5
Derivative (D):   0.1
```

Operating Range:

- **High Limit**: Maximum output (100%)

- **Low Limit**: Minimum output (0%)
- **Setpoint Range**: Allowed SP range

Advanced Settings

Deadband:

```
Deadband: 1.0°F
```

No control action if error < deadband

Rate Limiting:

```
Max Change: 10% per minute
```

Prevents rapid output changes

Anti-Windup:

```
Integral Limit: ±20%
```

Prevents integral term accumulation

Common PID Applications

Temperature Control

Heating Control:

```
Input:     Zone_Temperature
Setpoint:  72°F
Output:    Hot_Water_Valve
P: 2.0, I: 0.5, D: 0.1
```

Cooling Control:

```
Input:     Supply_Air_Temp
Setpoint:  55°F
Output:    Chilled_Water_Valve
P: 1.5, I: 0.3, D: 0.05
```

Pressure Control

Static Pressure:

```
Input:     Duct_Static_Press
Setpoint:  1.5 in.wc
Output:    Supply_Fan_VFD
P: 3.0, I: 1.0, D: 0.2
```

Flow Control

VAV Box Control:

```
Input:     Airflow_CFM
Setpoint:  500 CFM
Output:    Damper_Position
P: 2.5, I: 0.8, D: 0.15
```

PID Tuning

Manual Tuning

Ziegler-Nichols Method:

1. Set I and D to zero
2. Increase P until output oscillates
3. Note critical gain (Ku) and period (Tu)
4. Calculate:

```
P = 0.6 × Ku
I = 0.5 × Tu
D = 0.125 × Tu
```

Trial and Error:

1. Start with conservative values:

```
P = 1.0, I = 0.1, D = 0.01
```

2. Adjust P:
   - Increase if too slow
   - Decrease if oscillating
3. Add I to eliminate offset:
   - Start small (0.1)
   - Increase gradually
4. Add D if oscillating:
   - Start small (0.01)
   - Increase to dampen

Tuning Guidelines

Response Types:

- **Underdamped**: Overshoots, oscillates (reduce P or I)
- **Critically Damped**: Fast, no overshoot (ideal)
- **Overdamped**: Slow, sluggish (increase P)

Common Issues:

Problem	Solution
Slow response	Increase P
Oscillation	Decrease P, increase D
Steady-state error	Increase I
Overshoot	Decrease P or I, increase D
Noise amplification	Decrease D, add filter

Monitoring PID Performance

Real-Time Monitoring

View PID loop performance:

Loop Status:

```
Setpoint:     72.0°F
Input:        71.8°F
Error:        -0.2°F
Output:       45%
P Term:       -0.4%
I Term:       -0.1%
D Term:       0.0%
```

Performance Metrics:

- **Settling Time**: Time to reach setpoint
- **Overshoot**: Maximum deviation beyond setpoint
- **Steady-State Error**: Persistent offset
- **Oscillation Period**: Cycle time if oscillating

Trend Analysis

Plot PID variables over time:

- Setpoint (blue line)
- Input (green line)
- Output (orange line)
- Error (red line)

Look for:

- Quick response to setpoint changes
- Minimal overshoot
- No oscillation
- Zero steady-state error

PID Loop Table

Manage all PID loops:

#	Label	Input	Setpoint	Output	Status	Mode
1	Zone_Temp_Loop	71.8°F	72.0°F	45%	✅ Normal	Auto
2	SA_Temp_Loop	55.2°F	55.0°F	62%	✅ Normal	Auto
3	SP_Control	1.48"	1.50"	58%	✅ Normal	Auto

Features

- Real-time parameter updates
- Auto/Manual mode switching
- Setpoint adjustment
- Parameter tuning interface
- Performance charts

Best Practices

Loop Design

1. **Choose Appropriate Sensor**
   - Fast response time
   - Accurate measurement
   - Proper location
2. **Size Actuator Correctly**
   - Adequate authority (range)
   - Fast enough response
   - Linear characteristics
3. **Set Realistic Limits**
   - Prevent unsafe conditions
   - Allow adequate range
   - Consider seasonal changes

Tuning Strategy

1. **Start Conservative**
   - Begin with low gains
   - Increase gradually
   - Test incrementally
2. **One Parameter at a Time**
   - Adjust P first
   - Then I
   - Finally D
3. **Document Settings**
   - Record final parameters
   - Note tuning date
   - Keep tuning history

Maintenance

1. **Regular Review**
   - Check loop performance monthly
   - Retune if needed
   - Update for seasonal changes
2. **Calibration**
   - Calibrate sensors annually
   - Verify output devices
   - Check mechanical linkages
3. **Performance Analysis**

- Review trend data
- Calculate metrics
- Optimize settings

Troubleshooting

Poor Performance

Symptoms: Slow, oscillating, or unstable

Solutions:

1. Review tuning parameters
2. Check sensor calibration
3. Verify output device operation
4. Analyze trend data
5. Retune loop

Hunting

Symptoms: Continuous oscillation

Solutions:

1. Reduce P gain
2. Increase deadband
3. Add rate limiting
4. Check for mechanical issues
5. Increase D term (carefully)

Offset Error

Symptoms: Persistent error, doesn't reach setpoint

Solutions:

1. Increase I gain
2. Check for mechanical binding
3. Verify adequate output range
4. Ensure sensor accuracy

Next Steps

- Programs - Control programs
- Outputs - Output control
- Schedules - Setpoint scheduling

PID Loops

PID Controller Implementation

Full PID Class

TYPESCRIPT

```typescript
interface PIDConfig {
  kp: number;
  ki: number;
  kd: number;
  setpoint: number;
  outputMin: number;
  outputMax: number;
  deadband?: number;
  sampleTime?: number;  // milliseconds
}

class PIDController {
  private integral = 0;
  private lastError = 0;
  private lastTime = Date.now();
  private lastOutput = 0;

  constructor(private config: PIDConfig) {}

  update(processValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000; // seconds

    // Skip if sample time not elapsed
    if (this.config.sampleTime &&
        (now - this.lastTime) < this.config.sampleTime) {
      return this.lastOutput;
    }

    const error = this.config.setpoint - processValue;

    // Deadband
    if (this.config.deadband && Math.abs(error) < this.config.deadband) {
      return this.lastOutput;
    }

    // P term
    const pTerm = this.config.kp * error;

    // I term with anti-windup
    this.integral += error * dt;
    const iTerm = this.config.ki * this.integral;

    // D term on measurement (avoids derivative kick)
    const dInput = (processValue - (this.config.setpoint - this.lastError)) / dt;
    const dTerm = -this.config.kd * dInput;

    // Calculate output
    let output = pTerm + iTerm + dTerm;

    // Clamp output and handle anti-windup
    if (output > this.config.outputMax) {
      output = this.config.outputMax;
```

```
      this.integral -= error * dt;  // Anti-windup
    } else if (output < this.config.outputMin) {
      output = this.config.outputMin;
      this.integral -= error * dt;  // Anti-windup
    }

    // Update state
    this.lastError = error;
    this.lastTime = now;
    this.lastOutput = output;

    return output;
  }

  setSetpoint(setpoint: number) {
    this.config.setpoint = setpoint;
  }

  reset() {
    this.integral = 0;
    this.lastError = 0;
    this.lastOutput = 0;
  }

  getState() {
    return {
      integral: this.integral,
      lastError: this.lastError,
      output: this.lastOutput
    };
  }
}
```

Auto-Tuning Algorithm

TYPESCRIPT

```typescript
class PIDAutoTuner {
  async tune(input: string, output: string): Promise<PIDConfig> {
    console.log('Starting auto-tune...');

    // 1. Relay feedback test
    const relayAmplitude = 10;  // % output
    const measurements: number[] = [];
    const timestamps: number[] = [];

    // Run relay test for ~10 cycles
    for (let i = 0; i < 100; i++) {
      const pv = await this.readInput(input);
      measurements.push(pv);
      timestamps.push(Date.now());

      // Toggle output based on error
      const error = 72 - pv;  // Assume 72°F setpoint
      const outValue = error > 0 ? 50 + relayAmplitude : 50 - relayAmplitude;
      await this.writeOutput(output, outValue);

      await this.sleep(1000);  // 1 second samples
    }

    // 2. Analyze oscillations
    const analysis = this.analyzeOscillations(measurements, timestamps);

    // 3. Calculate PID parameters using Ziegler-Nichols
    const ku = (4 * relayAmplitude) / (Math.PI * analysis.amplitude);
    const tu = analysis.period / 1000;  // Convert to seconds

    return {
      kp: 0.6 * ku,
      ki: 1.2 * ku / tu,
      kd: 0.075 * ku * tu,
      setpoint: 72,
      outputMin: 0,
      outputMax: 100
    };
  }

  private analyzeOscillations(data: number[], times: number[]) {
    // Find peaks and calculate period
    const peaks: number[] = [];
    const peakTimes: number[] = [];

    for (let i = 1; i < data.length - 1; i++) {
      if (data[i] > data[i-1] && data[i] > data[i+1]) {
        peaks.push(data[i]);
        peakTimes.push(times[i]);
      }
    }

    const avgPeriod = peakTimes.length > 1 ?
      (peakTimes[peakTimes.length - 1] - peakTimes[0]) / (peakTimes.length - 1) :
      0;

    const avgAmplitude = peaks.reduce((a, b) => a + b, 0) / peaks.length;
```

```
    return {
      period: avgPeriod,
      amplitude: avgAmplitude
    };
  }
}
```

Cascade Control

TYPESCRIPT

```typescript
class CascadeController {
  private primary: PIDController;
  private secondary: PIDController;

  constructor(
    primaryConfig: PIDConfig,
    secondaryConfig: PIDConfig
  ) {
    this.primary = new PIDController(primaryConfig);
    this.secondary = new PIDController(secondaryConfig);
  }

  update(primaryPV: number, secondaryPV: number): number {
    // Primary loop output becomes secondary loop setpoint
    const secondarySetpoint = this.primary.update(primaryPV);
    this.secondary.setSetpoint(secondarySetpoint);

    // Secondary loop controls the actual output
    return this.secondary.update(secondaryPV);
  }
}

// Example: Zone temp controls supply temp setpoint,
// supply temp controls valve position
const cascade = new CascadeController(
  { kp: 2.0, ki: 0.5, kd: 0.1, setpoint: 72, outputMin: 50, outputMax: 65 },
  { kp: 3.0, ki: 1.0, kd: 0.2, setpoint: 55, outputMin: 0, outputMax: 100 }
);
```

PID Monitoring

TYPESCRIPT

```typescript
class PIDMonitor {
  logState(controller: PIDController, pv: number) {
    const state = controller.getState();

    console.log({
      timestamp: new Date().toISOString(),
      processValue: pv,
      setpoint: controller.config.setpoint,
      error: state.lastError,
      integral: state.integral,
      output: state.output,
      pTerm: controller.config.kp * state.lastError,
      iTerm: controller.config.ki * state.integral
    });
  }
}
```

Next Steps

- Outputs
- REST API
- Performance Tuning
- Performance Tuning - Optimization

Programs

Control programs for automation, sequences, and custom logic.

Overview

Programs contain custom control logic written in a simplified programming language. They execute continuously to automate building control sequences, implement control strategies, and respond to conditions.

Program Structure

Basic Program

```BASIC
10 REM Zone Temperature Control
20 IF ZONE_TEMP > ZONE_SP + 1 THEN COOLING = ON
30 IF ZONE_TEMP < ZONE_SP - 1 THEN HEATING = ON
40 IF ZONE_TEMP >= ZONE_SP - 0.5 AND ZONE_TEMP <= ZONE_SP + 0.5 THEN GOTO 60
50 GOTO 20
60 COOLING = OFF
70 HEATING = OFF
80 GOTO 20
```

Program Elements

Commands:

- `IF...THEN...ELSE` : Conditional logic
- `GOTO` : Jump to line number
- `GOSUB...RETURN` : Subroutine calls
- `FOR...NEXT` : Loops
- `REM` : Comments

Operators:

- Comparison: `>` , `<` , `>=` , `<=` , `=` , `<>`
- Logical: `AND` , `OR` , `NOT`
- Arithmetic: `+` , `-` , `*` , `/`

Common Applications

Occupancy Control

```BASIC
REM Office lighting and HVAC control
IF TIME > 0700 AND TIME < 1800 THEN
  OCC_MODE = TRUE
  LIGHTS = ON
  ZONE_SP = 72
ELSE
  OCC_MODE = FALSE
  LIGHTS = OFF
  ZONE_SP = 65
END IF
```

Equipment Sequencing

```basic
REM Chiller staging
IF LOAD > 80% THEN
  IF CH1_STATUS = OFF THEN START_CH1
  WAIT 300  ' 5 minute delay
  IF LOAD > 95% THEN START_CH2
END IF
```

Alarm Logic

```basic
REM High temperature alarm
IF ZONE_TEMP > HI_LIMIT THEN
  ALARM = ON
  SEND_EMAIL("High temp alert")
ELSE
  ALARM = OFF
END IF
```

Program Management

Program Table

View all programs in the Programs page:

#	Label	Status	Last Run	Exec Time	Errors
1	Zone_Control	✅ Running	1s ago	5ms	0
2	Schedule_Mgr	✅ Running	1s ago	2ms	0
3	Alarm_Logic	✅ Running	1s ago	3ms	0

Program Editor

Edit programs directly in the web interface:

1. Click on program row
2. Modify code in editor
3. Click **Validate** to check syntax
4. Click **Save** to store changes
5. Program automatically restarts

Debugging

Runtime Errors

Common errors:

- **Syntax Error**: Invalid command or syntax
- **Division by Zero**: Math error
- **Undefined Variable**: Reference to non-existent point
- **Timeout**: Program execution too long

Troubleshooting

- Use `REM` comments to document logic
- Test in manual mode first
- Monitor execution time
- Check variable values

- Review error logs

Best Practices

1. **Comment Your Code**

```basic
REM Author: John Doe
REM Date: 2026-01-06
REM Purpose: Main AHU control
```

2. **Use Meaningful Names**
   - `ZONE_TEMP` not `V1`
   - `OCC_MODE` not `FLAG1`
3. **Error Handling**

```basic
IF SENSOR_FAULT = TRUE THEN USE_DEFAULT
```

4. **Execution Time**
   - Keep programs short (<100 lines)
   - Avoid infinite loops
   - Use timers for delays
5. **Version Control**
   - Export programs regularly
   - Document changes
   - Test before deploying

Next Steps

- PID Loops - PID control
- Schedules - Time-based control
- Variables - Program variables

Programs

Program Execution Engine

Script Interpreter

TYPESCRIPT

```typescript
class ProgramInterpreter {
  private variables = new Map<string, number>();
  private labels = new Map<string, number>();
  private callStack: number[] = [];

  async execute(code: string): Promise<void> {
    const lines = code.split('\n');
    let currentLine = 0;

    // Parse labels
    this.parseLabels(lines);

    while (currentLine < lines.length) {
      const line = lines[currentLine].trim();

      if (line.startsWith('REM') || line === '') {
        currentLine++;
        continue;
      }

      const result = await this.executeLine(line);

      if (result.type === 'GOTO') {
        currentLine = result.target;
      } else if (result.type === 'GOSUB') {
        this.callStack.push(currentLine + 1);
        currentLine = result.target;
      } else if (result.type === 'RETURN') {
        currentLine = this.callStack.pop() || lines.length;
      } else {
        currentLine++;
      }
    }
  }

  private async executeLine(line: string): Promise<ExecutionResult> {
    // Parse and execute line
    if (line.startsWith('IF')) {
      return this.executeIf(line);
    } else if (line.startsWith('GOTO')) {
      return this.executeGoto(line);
    } else if (line.includes('=')) {
      return this.executeAssignment(line);
    }

    return { type: 'CONTINUE' };
  }
}
```

Control Flow

TYPESCRIPT

```typescript
interface ExecutionResult {
  type: 'CONTINUE' | 'GOTO' | 'GOSUB' | 'RETURN';
  target?: number;
}

class ControlFlow {
  executeIf(condition: string, thenClause: string): ExecutionResult {
    const conditionMet = this.evaluateCondition(condition);

    if (conditionMet) {
      if (thenClause.startsWith('GOTO')) {
        const target = parseInt(thenClause.split(' ')[1]);
        return { type: 'GOTO', target };
      } else {
        this.executeLine(thenClause);
      }
    }

    return { type: 'CONTINUE' };
  }

  private evaluateCondition(condition: string): boolean {
    // Parse condition like "TEMP > 72"
    const match = condition.match(/(.+?)\s*([><=!]+)\s*(.+)/);
    if (!match) return false;

    const [, left, op, right] = match;
    const leftVal = this.getValue(left.trim());
    const rightVal = this.getValue(right.trim());

    switch (op) {
      case '>': return leftVal > rightVal;
      case '<': return leftVal < rightVal;
      case '>=': return leftVal >= rightVal;
      case '<=': return leftVal <= rightVal;
      case '=': return leftVal === rightVal;
      case '<>': return leftVal !== rightVal;
      default: return false;
    }
  }
}
```

Program API

Load and Execute Programs

TYPESCRIPT

```typescript
// Get program list
const programs = await fetch(
  `http://localhost:9103/api/devices/389001/programs`
).then(r => r.json());

// Get program code
const program = await fetch(
  `http://localhost:9103/api/devices/389001/programs/1`
).then(r => r.json());

// Update program
await fetch(`http://localhost:9103/api/devices/389001/programs/1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    code: `REM Updated program\nIF TEMP > 75 THEN COOLING = ON`,
    enabled: true
  })
});
```

Program Validation

TYPESCRIPT

```typescript
class ProgramValidator {
  validate(code: string): ValidationResult {
    const errors: string[] = [];
    const warnings: string[] = [];
    const lines = code.split('\n');

    for (let i = 0; i < lines.length; i++) {
      const line = lines[i].trim();

      // Check syntax
      if (line.startsWith('IF') && !line.includes('THEN')) {
        errors.push(`Line ${i + 1}: IF without THEN`);
      }

      // Check for undefined variables
      const vars = this.extractVariables(line);
      for (const v of vars) {
        if (!this.isDefined(v)) {
          warnings.push(`Line ${i + 1}: Undefined variable ${v}`);
        }
      }

      // Check goto targets
      if (line.startsWith('GOTO')) {
        const target = parseInt(line.split(' ')[1]);
        if (!this.isValidLabel(target, lines)) {
          errors.push(`Line ${i + 1}: Invalid GOTO target ${target}`);
        }
      }
    }

    return {
      valid: errors.length === 0,
      errors,
      warnings
    };
  }
}
```

Scheduler Integration

Time-Based Execution

TYPESCRIPT

```typescript
class ProgramScheduler {
  private programs = new Map<number, NodeJS.Timeout>();

  schedule(programId: number, interval: number) {
    // Cancel existing schedule
    if (this.programs.has(programId)) {
      clearInterval(this.programs.get(programId)!);
    }

    // Schedule program execution
    const timer = setInterval(async () => {
      try {
        await this.executeProgram(programId);
      } catch (error) {
        console.error(`Program ${programId} error:`, error);
      }
    }, interval);

    this.programs.set(programId, timer);
  }

  async executeProgram(programId: number) {
    const program = await this.loadProgram(programId);
    const interpreter = new ProgramInterpreter();
    await interpreter.execute(program.code);
  }
}
```

Next Steps

- REST API Reference
- WebSocket API
- Variables

Variables

Calculated variables, setpoints, and derived values.

Overview

Variables store calculated values, setpoints, intermediate results, and system status flags. They're used for complex control logic, data manipulation, and system coordination.

Variable Types

Analog Variables (AV)

- Setpoints (temperature, pressure)
- Calculated values (averages, differentials)
- Timers and counters
- Mathematical results

Binary Variables (BV)

- Status flags
- Enable/disable switches
- Alarm conditions
- Mode selections

Common Uses

- **Temperature Setpoints**: Heating/cooling targets
- **Differential Calculations**: Supply - Return temperature
- **Runtime Accumulators**: Equipment operating hours
- **Status Flags**: System mode indicators
- **Energy Calculations**: kW, BTU/hr

Configuration

Variables can be:

- **Read-only**: Calculated by programs
- **Writable**: User-adjustable setpoints
- **Commandable**: Controlled by programs or operators

Best Practices

- Name clearly (e.g., Zone_Temp_SP)
- Document calculations
- Set appropriate limits
- Use consistent units

Next Steps

- Programs - Control logic
- PID Loops - PID control

Variables

Variable Management API

Reading and Writing Variables

TYPESCRIPT

```typescript
// Read variable value
const response = await fetch(
  `http://localhost:9103/api/devices/389001/points/VAR1`
);
const data = await response.json();

// Write variable value
await fetch(`http://localhost:9103/api/devices/389001/points/VAR1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ value: 72.5 })
});

// BACnet analog-value object
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-value',
  objectInstance: 1,
  property: 'present-value',
  value: 72.5
});
```

Variable Calculations

TYPESCRIPT

```typescript
class VariableCalculator {
  async updateCalculatedVariables(deviceId: number) {
    // Read source values
    const supplyTemp = await this.readPoint(deviceId, 'IN1');
    const returnTemp = await this.readPoint(deviceId, 'IN2');

    // Calculate differential
    const differential = supplyTemp - returnTemp;
    await this.writeVariable(deviceId, 'VAR_Diff', differential);

    // Calculate average
    const average = (supplyTemp + returnTemp) / 2;
    await this.writeVariable(deviceId, 'VAR_Avg', average);

    // Runtime accumulator
    const fanStatus = await this.readPoint(deviceId, 'BO1');
    if (fanStatus) {
      const runtime = await this.readVariable(deviceId, 'VAR_Runtime');
      await this.writeVariable(deviceId, 'VAR_Runtime', runtime + 1);
    }
  }
}
```

Setpoint Management

TYPESCRIPT

```typescript
interface SetpointSchedule {
  occupiedSP: number;
  unoccupiedSP: number;
  schedule: {
    start: string;  // "07:00"
    end: string;    // "18:00"
  };
}

class SetpointManager {
  async updateSetpoint(deviceId: number, config: SetpointSchedule) {
    const now = new Date();
    const hour = now.getHours();
    const minute = now.getMinutes();
    const currentTime = `${hour.toString().padStart(2, '0')}:${minute.toString().padStart(2, '0')}`;

    const isOccupied = currentTime >= config.schedule.start &&
                       currentTime <= config.schedule.end;

    const setpoint = isOccupied ? config.occupiedSP : config.unoccupiedSP;

    await this.writeVariable(deviceId, 'VAR_Setpoint', setpoint);
  }
}
```

Mathematical Operations

Formula Evaluation

TYPESCRIPT

```typescript
class FormulaEngine {
  evaluate(formula: string, variables: Record<string, number>): number {
    // Simple expression evaluator
    // Replace variable names with values
    let expression = formula;
    for (const [name, value] of Object.entries(variables)) {
      expression = expression.replace(new RegExp(name, 'g'), value.toString());
    }

    // Evaluate (Note: Use safe-eval in production)
    return eval(expression);
  }
}

// Example: Calculate enthalpy
const formula = '(0.24 * TEMP) + (HUM/100) * (1061 + 0.444 * TEMP)';
const engine = new FormulaEngine();

const enthalpy = engine.evaluate(formula, {
  TEMP: 75,   // °F
  HUM: 50     // %RH
});

console.log(`Enthalpy: ${enthalpy.toFixed(2)} BTU/lb`);
```

Statistical Functions

TYPESCRIPT

```typescript
class Statistics {
  static average(values: number[]): number {
    return values.reduce((a, b) => a + b, 0) / values.length;
  }

  static min(values: number[]): number {
    return Math.min(...values);
  }

  static max(values: number[]): number {
    return Math.max(...values);
  }

  static stdDev(values: number[]): number {
    const avg = this.average(values);
    const squareDiffs = values.map(v => Math.pow(v - avg, 2));
    const avgSquareDiff = this.average(squareDiffs);
    return Math.sqrt(avgSquareDiff);
  }
}

// Calculate zone temperature statistics
const temps = [72.1, 72.5, 71.8, 72.3, 72.0];

const stats = {
  avg: Statistics.average(temps),
  min: Statistics.min(temps),
  max: Statistics.max(temps),
  stdDev: Statistics.stdDev(temps)
};
```

Variable Persistence

Database Storage

SQL

```sql
CREATE TABLE variables (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  variable_id TEXT NOT NULL,
  value REAL NOT NULL,
  units TEXT,
  writable BOOLEAN DEFAULT 1,
  updated_at INTEGER NOT NULL,
  UNIQUE(device_id, variable_id),
  FOREIGN KEY (device_id) REFERENCES devices(id)
);

CREATE INDEX idx_variables_device
  ON variables(device_id, variable_id);
```

Variable Caching

```typescript
class VariableCache {
  private cache = new Map<string, {value: number, timestamp: number}>();
  private ttl = 5000; // 5 seconds

  async get(deviceId: number, varId: string): Promise<number> {
    const key = `${deviceId}:${varId}`;
    const cached = this.cache.get(key);

    if (cached && Date.now() - cached.timestamp < this.ttl) {
      return cached.value;
    }

    // Cache miss - fetch from device
    const value = await this.fetchFromDevice(deviceId, varId);
    this.cache.set(key, { value, timestamp: Date.now() });

    return value;
  }

  set(deviceId: number, varId: string, value: number) {
    const key = `${deviceId}:${varId}`;
    this.cache.set(key, { value, timestamp: Date.now() });
  }
}
```

Next Steps

- REST API Reference
- Programs
- Inputs

Connecting Devices

Learn how to discover and connect T3000 BACnet devices to your network.

Overview

The T3000 Web application provides multiple methods to discover and connect to BACnet devices on your network. This guide covers the different connection methods and best practices.

Discovery Methods

Auto Discovery

The application automatically scans your network for BACnet devices:

1. Navigate to the **Discover** page from the main menu
2. Click **Scan Network** to start the discovery process
3. Wait for the scan to complete (typically 10-30 seconds)
4. All discovered devices will appear in the device list

Supported Protocols:

- BACnet/IP (UDP port 47808)
- BACnet MSTP (serial communication)
- Modbus TCP/IP
- Modbus RTU (serial)

Manual Connection

To add a device manually:

1. Go to **Devices** > **Add Device**
2. Enter the device information:
   - **IP Address**: Device network address (e.g., 192.168.1.100)
   - **Port**: BACnet port (default: 47808)
   - **Device ID**: BACnet device instance number
   - **Protocol**: Select connection protocol
3. Click **Connect**

Connection Status

Device connection states:

- 🟢 **Online**: Device is connected and responding
- 🟡 **Connecting**: Attempting to establish connection
- 🔴 **Offline**: Device is not responding
- ⚠️ **Error**: Connection error occurred

Network Configuration

IP Address Setup

Ensure your device and computer are on the same network:

```
TEXT

Device IP:     192.168.1.100
Subnet Mask:   255.255.255.0
Gateway:       192.168.1.1
```

Firewall Settings

Allow the following ports through your firewall:

- **UDP 47808**: BACnet/IP communication
- **TCP 502**: Modbus TCP
- **TCP 9103**: T3000 API server
- **TCP 3003**: T3000 Web UI

Troubleshooting

Device Not Found

1. Verify the device is powered on
2. Check network connectivity (ping the device IP)
3. Ensure firewall allows BACnet traffic
4. Verify correct subnet configuration

Connection Timeout

- Increase timeout settings in **Settings** > **Communication**
- Check for network congestion
- Verify device is not overloaded with requests

Authentication Errors

- Verify device password (if authentication enabled)
- Check user permissions
- Ensure device firmware is up to date

Best Practices

1. **Use Static IP Addresses**: Assign fixed IPs to devices for consistent connectivity
2. **Document Device IDs**: Maintain a list of device IDs and locations
3. **Regular Health Checks**: Monitor connection status regularly
4. **Backup Configuration**: Export device settings periodically
5. **Network Segmentation**: Use VLANs to organize devices by zone/function

Next Steps

- Device Configuration - Configure device settings
- Device Monitoring - Monitor device status and data

Connecting Devices

Programmatic Device Discovery

BACnet Discovery via API

TYPESCRIPT

```typescript
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({
  port: 47808,
  broadcastAddress: '192.168.1.255',
  timeout: 30000
});

// Broadcast WhoIs request
const devices = await client.discover({
  lowLimit: 0,
  highLimit: 4194303,  // Max device instance
  networkNumber: null   // All networks
});

devices.forEach(device => {
  console.log(`Found device: ${device.deviceId} at ${device.address}`);
});
```

Modbus Discovery

TYPESCRIPT

```typescript
import { ModbusClient } from '@temco/t3000-sdk';

const client = new ModbusClient({
  host: '192.168.1.100',
  port: 502,
  timeout: 5000
});

// Test connection
const connected = await client.connect();
if (connected) {
  // Read device info
  const deviceId = await client.readHoldingRegisters(0, 1);
  console.log(`Modbus device ID: ${deviceId[0]}`);
}
```

REST API Discovery

```shell
## Trigger network scan
curl -X POST http://localhost:9103/api/discovery/scan \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "bacnet",
    "subnet": "192.168.1.0/24",
    "timeout": 30000
  }'

## Get discovered devices
curl http://localhost:9103/api/discovery/devices

## Response
[
  {
    "deviceId": 389001,
    "address": "192.168.1.100:47808",
    "protocol": "bacnet",
    "modelName": "T3-BB",
    "firmwareVersion": "8.5.0"
  }
]
```

Advanced Connection Configuration

BACnet Connection Parameters

```typescript
interface BACnetConnectionConfig {
  address: string;
  port: number;
  deviceInstance: number;
  networkNumber?: number;
  maxApdu: 1476 | 1024 | 480 | 206 | 128 | 50;
  segmentation: 'both' | 'transmit' | 'receive' | 'none';
  apduTimeout: number;
  numberOfApduRetries: number;
  maxSegments?: number;
}

const config: BACnetConnectionConfig = {
  address: '192.168.1.100',
  port: 47808,
  deviceInstance: 389001,
  networkNumber: 0,
  maxApdu: 1476,
  segmentation: 'both',
  apduTimeout: 3000,
  numberOfApduRetries: 3,
  maxSegments: 64
};

await client.connect(config);
```

Connection Pooling

TYPESCRIPT

```typescript
class DeviceConnectionPool {
  private connections = new Map<number, BACnetClient>();
  private maxConnections = 50;

  async getConnection(deviceId: number): Promise<BACnetClient> {
    if (this.connections.has(deviceId)) {
      return this.connections.get(deviceId)!;
    }

    if (this.connections.size >= this.maxConnections) {
      // Evict least recently used
      const lru = Array.from(this.connections.keys())[0];
      this.connections.delete(lru);
    }

    const client = await this.createConnection(deviceId);
    this.connections.set(deviceId, client);
    return client;
  }

  private async createConnection(deviceId: number): Promise<BACnetClient> {
    const device = await this.lookupDevice(deviceId);
    return new BACnetClient({
      address: device.address,
      port: device.port,
      deviceInstance: deviceId
    });
  }
}
```

Connection Monitoring

Health Checks

TYPESCRIPT

```typescript
class DeviceHealthMonitor {
  async checkDevice(deviceId: number): Promise<HealthStatus> {
    const start = Date.now();

    try {
      // Attempt read of system status object
      const status = await client.readProperty({
        objectType: 'device',
        objectInstance: deviceId,
        property: 'system-status'
      });

      const latency = Date.now() - start;

      return {
        online: true,
        latency,
        systemStatus: status.value,
        lastCheck: new Date()
      };
    } catch (error) {
      return {
        online: false,
        error: error.message,
        lastCheck: new Date()
      };
    }
  }

  async monitorDevices(deviceIds: number[], interval: number) {
    setInterval(async () => {
      for (const deviceId of deviceIds) {
        const health = await this.checkDevice(deviceId);
        await this.updateDatabase(deviceId, health);

        if (!health.online) {
          await this.sendAlert(deviceId, health);
        }
      }
    }, interval);
  }
}
```

Connection Retry Logic

TYPESCRIPT

```typescript
async function connectWithRetry(
  config: ConnectionConfig,
  maxRetries: number = 3,
  backoff: number = 1000
): Promise<BACnetClient> {
  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      const client = new BACnetClient(config);
      await client.connect();
      return client;
    } catch (error) {
      lastError = error;
      const delay = backoff * Math.pow(2, attempt);
      console.log(`Retry ${attempt + 1}/${maxRetries} after ${delay}ms`);
      await sleep(delay);
    }
  }

  throw new Error(`Failed after ${maxRetries} attempts: ${lastError}`);
}
```

Network Discovery Protocols

BACnet WhoIs/IAm

TYPESCRIPT

```typescript
class BACnetDiscovery {
  async sendWhoIs(lowLimit?: number, highLimit?: number) {
    const packet = encodeBACnetPacket({
      pduType: 'UnconfirmedRequest',
      service: 'who-is',
      lowLimit: lowLimit ?? 0,
      highLimit: highLimit ?? 4194303
    });

    await this.socket.send(packet, 47808, '255.255.255.255');
  }

  onIAm(callback: (device: DeviceInfo) => void) {
    this.socket.on('message', (msg, rinfo) => {
      const packet = decodeBACnetPacket(msg);

      if (packet.service === 'i-am') {
        callback({
          deviceId: packet.deviceInstance,
          address: rinfo.address,
          port: rinfo.port,
          maxApdu: packet.maxApduLength,
          segmentation: packet.segmentationSupported,
          vendorId: packet.vendorId
        });
      }
    });
  }
}
```

Modbus Device Scanning

TYPESCRIPT

```typescript
async function scanModbusDevices(
  baseIp: string,
  startSlaveId: number,
  endSlaveId: number
): Promise<ModbusDevice[]> {
  const devices: ModbusDevice[] = [];

  for (let slaveId = startSlaveId; slaveId <= endSlaveId; slaveId++) {
    try {
      const client = new ModbusClient({
        host: baseIp,
        port: 502,
        unitId: slaveId,
        timeout: 2000
      });

      await client.connect();

      // Read device identification
      const registers = await client.readHoldingRegisters(0, 10);

      devices.push({
        slaveId,
        address: baseIp,
        registers: registers
      });

      await client.disconnect();
    } catch (error) {
      // Device not responding at this slave ID
      continue;
    }
  }

  return devices;
}
```

Database Schema for Devices

SQL

```sql
CREATE TABLE devices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  serial_number INTEGER UNIQUE NOT NULL,
  ip_address TEXT,
  port INTEGER,
  protocol TEXT CHECK(protocol IN ('bacnet', 'modbus')),
  device_instance INTEGER,
  model_name TEXT,
  firmware_version TEXT,
  status TEXT CHECK(status IN ('online', 'offline', 'error')),
  last_seen INTEGER,
  created_at INTEGER DEFAULT (strftime('%s', 'now')),
  updated_at INTEGER DEFAULT (strftime('%s', 'now'))
);

CREATE INDEX idx_devices_serial ON devices(serial_number);
CREATE INDEX idx_devices_status ON devices(status);
CREATE INDEX idx_devices_protocol ON devices(protocol);

-- Connection history
CREATE TABLE connection_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER,
  event_type TEXT CHECK(event_type IN ('connect', 'disconnect', 'error')),
  message TEXT,
  timestamp INTEGER DEFAULT (strftime('%s', 'now')),
  FOREIGN KEY (device_id) REFERENCES devices(id)
);
```

WebSocket Real-Time Connection Status

TYPESCRIPT

```typescript
// Subscribe to connection events
const ws = new WebSocket('ws://localhost:9103/ws');

ws.send(JSON.stringify({
  type: 'subscribe',
  channel: 'device-status'
}));

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'device-status-change') {
    console.log(`Device ${data.deviceId}: ${data.status}`);
    updateUI(data.deviceId, data.status);
  }
};
```

Next Steps

- Device Configuration API
- REST API Reference
- WebSocket API
- Modbus Protocol

Overview

The T3000 Web application provides multiple methods to discover and connect to BACnet devices on your network. This guide covers the different connection methods and best practices.

Discovery Methods

Auto Discovery

The application automatically scans your network for BACnet devices:

1. Navigate to the **Discover** page from the main menu
2. Click **Scan Network** to start the discovery process
3. Wait for the scan to complete (typically 10-30 seconds)
4. All discovered devices will appear in the device list

Supported Protocols:

- BACnet/IP (UDP port 47808)
- BACnet MSTP (serial communication)
- Modbus TCP/IP
- Modbus RTU (serial)

Manual Connection

To add a device manually:

1. Go to **Devices** > **Add Device**
2. Enter the device information:
   - **IP Address**: Device network address (e.g., 192.168.1.100)
   - **Port**: BACnet port (default: 47808)
   - **Device ID**: BACnet device instance number
   - **Protocol**: Select connection protocol
3. Click **Connect**

Connection Status

Device connection states:

- 🟢 **Online**: Device is connected and responding
- 🟡 **Connecting**: Attempting to establish connection
- 🔴 **Offline**: Device is not responding
- ⚠️ **Error**: Connection error occurred

Network Configuration

IP Address Setup

Ensure your device and computer are on the same network:

```
TEXT

Device IP:     192.168.1.100
Subnet Mask:   255.255.255.0
Gateway:       192.168.1.1
```

Firewall Settings

Allow the following ports through your firewall:

- **UDP 47808**: BACnet/IP communication
- **TCP 502**: Modbus TCP
- **TCP 9103**: T3000 API server

- **TCP 3003**: T3000 Web UI

Troubleshooting

Device Not Found

1. Verify the device is powered on
2. Check network connectivity (ping the device IP)
3. Ensure firewall allows BACnet traffic
4. Verify correct subnet configuration

Connection Timeout

- Increase timeout settings in **Settings** > **Communication**
- Check for network congestion
- Verify device is not overloaded with requests

Authentication Errors

- Verify device password (if authentication enabled)
- Check user permissions
- Ensure device firmware is up to date

Best Practices

1. **Use Static IP Addresses**: Assign fixed IPs to devices for consistent connectivity
2. **Document Device IDs**: Maintain a list of device IDs and locations
3. **Regular Health Checks**: Monitor connection status regularly
4. **Backup Configuration**: Export device settings periodically
5. **Network Segmentation**: Use VLANs to organize devices by zone/function

Next Steps

- Device Configuration - Configure device settings
- Device Monitoring - Monitor device status and data
- Troubleshooting - Resolve common issues

Device Configuration

Configuring Your Device

Learn how to set up and customize your T3000 device settings.

Where to Find Settings

1. Open your device in T3000
2. Click on the **Settings** tab
3. You'll see all configuration options

Basic Setup

Give Your Device a Name:

- Find "Device Name" field
- Type a friendly name (like "Main Lobby Controller")
- This helps you identify it later

Set the Location:

- Enter where the device is installed
- Examples: "Building A - 2nd Floor", "Mechanical Room"

Network Connection

Setting IP Address:

Your device needs an IP address to communicate on the network.

- **IP Address**: Like a phone number for your device (e.g., 192.168.1.100)
- **Subnet Mask**: Usually 255.255.255.0 (ask your IT department if unsure)
- **Gateway**: The router address (usually ends in .1, like 192.168.1.1)

Tip: Write down these settings! You'll need them if you ever reset the device.

Time Settings

Keep Your Device's Clock Accurate:

1. **Time Zone**: Select your local timezone
2. **NTP Server** (automatic time): Enter `pool.ntp.org` or ask IT
3. **Sync Interval**: Set to 24 hours for daily updates

Why time matters:

- Schedules run at correct times
- Trend logs show accurate timestamps
- Alarms are properly timestamped

What Features to Enable

Check these boxes to turn features on:

- ☑ **Trend Logging** - Save historical data (recommended)
- ☑ **Alarms** - Get notified of problems (recommended)
- ☑ **Schedules** - Automate operations (if needed)
- ☑ **Remote Access** - Access from other locations (optional)

Saving Your Changes

Important Steps:

1. Make your changes
2. Click **Apply** - This tests your settings

3. Wait to see if device responds (usually 5-10 seconds)
4. If everything works, click **Save**

⚠ **Warning:** Some changes require the device to restart. You might lose connection for 1-2 minutes.

Backing Up Your Settings

Always backup before making big changes!

To Save Configuration:

1. Go to Settings > Backup
2. Click "Export Configuration"
3. Save the file somewhere safe (name it with today's date)

To Restore Configuration:

1. Go to Settings > Restore
2. Click "Choose File"
3. Select your backup file
4. Click "Import"

Quick Troubleshooting

Can't save settings?

- Check if device is online
- Make sure no one else is editing it
- Try refreshing the page

Device won't connect after changing IP?

- The IP address might be wrong
- Restore from backup
- Contact your network administrator

Getting Help

Need assistance with configuration? Switch to the **Technical** tab for advanced options and code examples.

Device Configuration API

Programmatically configure T3000 device settings via API.

Configuration Object Structure

TYPESCRIPT

```typescript
interface DeviceConfig {
  // Basic Information
  deviceName: string;
  description: string;
  location: string;
  firmwareVersion: string;  // read-only
  model: string;            // read-only

  // Network Configuration
  network: {
    ipAddress: string;
    subnetMask: string;
    gateway: string;
    dnsServer: string;
    dhcpEnabled: boolean;
  };

  // Protocol Settings
  bacnet: {
    deviceInstance: number;  // 0-4194303
    networkNumber: number;
    macAddress: string;
    apduTimeout: number;     // milliseconds
    apduRetries: number;
  };

  modbus: {
    slaveId: number;         // 1-247
    baudRate: number;        // 9600, 19200, 38400, 57600, 115200
    parity: 'none' | 'even' | 'odd';
    stopBits: 1 | 2;
  };

  // Time Synchronization
  time: {
    timezone: string;
    ntpServer: string;
    syncInterval: number;    // hours
  };

  // Communication Settings
  communication: {
    pollingInterval: number;      // seconds
    connectionTimeout: number;    // seconds
    maxConcurrentRequests: number;
    autoReconnect: boolean;
  };

  // Feature Flags
  features: {
    trendLogging: boolean;
    alarms: boolean;
    schedules: boolean;
    remoteAccess: boolean;
  };
}
```

Get Configuration

```typescript
async function getDeviceConfig(serial: number): Promise<DeviceConfig> {
  const response = await fetch(
    `http://localhost:9103/api/t3_device/devices/${serial}/config`
  );

  if (!response.ok) {
    throw new Error(`Failed to get config: ${response.statusText}`);
  }

  return await response.json();
}
```

Update Configuration

```typescript
async function updateDeviceConfig(
  serial: number,
  config: Partial<DeviceConfig>
): Promise<void> {
  const response = await fetch(
    `http://localhost:9103/api/t3_device/devices/${serial}/config`,
    {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(config)
    }
  );

  if (!response.ok) {
    throw new Error(`Failed to update config: ${response.statusText}`);
  }
}

// Example: Update device name and location
await updateDeviceConfig(237219, {
  deviceName: 'Main_Lobby_Controller',
  location: 'Building A - Lobby'
});
```

Batch Configuration

Update multiple devices with same settings:

TYPESCRIPT

```typescript
async function batchUpdateConfig(
  serials: number[],
  config: Partial<DeviceConfig>
): Promise<void> {
  const promises = serials.map(serial =>
    updateDeviceConfig(serial, config)
  );

  await Promise.all(promises);
}

// Apply NTP settings to all devices
await batchUpdateConfig(
  [237219, 237220, 237221],
  {
    time: {
      ntpServer: 'pool.ntp.org',
      syncInterval: 24,
      timezone: 'America/New_York'
    }
  }
);
```

Configuration Backup/Restore

Export Configuration:

TYPESCRIPT

```typescript
async function exportConfig(serial: number): Promise<string> {
  const config = await getDeviceConfig(serial);
  const timestamp = new Date().toISOString().split('T')[0];
  const filename = `device_${serial}_config_${timestamp}.json`;

  // Create blob and download
  const blob = new Blob([JSON.stringify(config, null, 2)], {
    type: 'application/json'
  });

  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);

  return filename;
}
```

Import Configuration:

TYPESCRIPT

```typescript
async function importConfig(
  serial: number,
  configFile: File
): Promise<void> {
  const text = await configFile.text();
  const config = JSON.parse(text) as DeviceConfig;

  // Remove read-only fields
  delete config.firmwareVersion;
  delete config.model;

  await updateDeviceConfig(serial, config);
}
```

Validation

Validate configuration before applying:

TYPESCRIPT

```typescript
function validateConfig(config: Partial<DeviceConfig>): string[] {
  const errors: string[] = [];

  // Validate BACnet device instance
  if (config.bacnet?.deviceInstance !== undefined) {
    if (config.bacnet.deviceInstance < 0 ||
        config.bacnet.deviceInstance > 4194303) {
      errors.push('BACnet device instance must be 0-4194303');
    }
  }

  // Validate Modbus slave ID
  if (config.modbus?.slaveId !== undefined) {
    if (config.modbus.slaveId < 1 || config.modbus.slaveId > 247) {
      errors.push('Modbus slave ID must be 1-247');
    }
  }

  // Validate IP address format
  if (config.network?.ipAddress) {
    const ipRegex = /^(\d{1,3}\.){3}\d{1,3}$/;
    if (!ipRegex.test(config.network.ipAddress)) {
      errors.push('Invalid IP address format');
    }
  }

  return errors;
}

// Usage
const config = { /* ... */ };
const errors = validateConfig(config);
if (errors.length > 0) {
  console.error('Configuration errors:', errors);
} else {
  await updateDeviceConfig(serial, config);
}
```

Configuration Templates

Predefined configuration templates:

```typescript
const configTemplates = {
  standard: {
    communication: {
      pollingInterval: 30,
      connectionTimeout: 5,
      maxConcurrentRequests: 10,
      autoReconnect: true
    },
    features: {
      trendLogging: true,
      alarms: true,
      schedules: true,
      remoteAccess: false
    }
  },

  highPerformance: {
    communication: {
      pollingInterval: 5,
      connectionTimeout: 2,
      maxConcurrentRequests: 20,
      autoReconnect: true
    }
  },

  lowBandwidth: {
    communication: {
      pollingInterval: 60,
      connectionTimeout: 10,
      maxConcurrentRequests: 5,
      autoReconnect: true
    }
  }
};

// Apply template
await updateDeviceConfig(serial, configTemplates.standard);
```

Configuration Change Events

Listen for configuration changes:

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'config_changed') {
    console.log('Configuration updated:', {
      serial: data.serial,
      field: data.field,
      oldValue: data.oldValue,
      newValue: data.newValue,
      timestamp: data.timestamp
    });
  }
};

// Subscribe to config changes
ws.send(JSON.stringify({
  type: 'subscribe_events',
  filters: { types: ['config_changed'] }
}));
```

Best Practices

1. **Validate Before Apply**
   - Always validate configuration objects
   - Check for conflicts (e.g., duplicate BACnet IDs)
   - Test in development environment first
2. **Atomic Updates**
   - Use transactions when updating related settings
   - Rollback on partial failure
   - Keep backup of previous configuration
3. **Change Management**
   - Log all configuration changes with timestamp
   - Include user/system that made change
   - Document reason for change
4. **Error Handling**

```typescript
try {
  await updateDeviceConfig(serial, config);
} catch (error) {
  // Rollback to previous config
  await importConfig(serial, backupFile);
  console.error('Config update failed, rolled back:', error);
}
```

Next Steps

- Device Monitoring - Monitor configured devices
- Troubleshooting - Resolve configuration issues
- REST API - Full API documentation

Device Monitoring

Monitor real-time device status, data points, and system health.

Overview

The T3000 monitoring system provides comprehensive visibility into device operation, data collection, and system performance.

Real-Time Dashboard

The main dashboard displays key metrics:

Device Status Panel

- **Connection State**: Online/Offline status
- **Last Communication**: Timestamp of last successful poll
- **CPU Usage**: Device processor utilization
- **Memory Usage**: RAM consumption
- **Uptime**: Time since last restart

Data Point Summary

Quick view of critical points:

- **Inputs**: Sensor readings (temperature, pressure, etc.)
- **Outputs**: Control signals and actuator states
- **Variables**: Calculated values
- **Alarms**: Active alarm count

Data Point Monitoring

Inputs Page

Monitor all input points:

Point	Description	Value	Units	Status	Last Update
IN1	Zone Temperature	72.5	°F	✅ Valid	2s ago
IN2	Supply Pressure	45.2	PSI	✅ Valid	2s ago
IN3	CO2 Level	650	PPM	✅ Valid	2s ago

Features:

- Real-time value updates (2-5 second refresh)
- Color-coded status indicators
- Trend sparklines
- Quick filters and search
- Export to CSV/Excel

Outputs Page

Monitor control outputs:

Point	Description	Command	Feedback	Mode	Override
OUT1	Damper Position	45%	44%	Auto	No
OUT2	Fan Speed	75%	75%	Auto	No
OUT3	Valve Position	0%	0%	Off	Yes

Variables Page

Monitor calculated variables:

- Setpoints
- Differential values
- Status flags
- Timers and counters

Trend Visualization

Real-Time Charts

View live data trends:

1. Navigate to **Trend Logs** page
2. Select points to monitor
3. Choose time range (1 hour, 4 hours, 24 hours)
4. View interactive charts with zoom/pan

Chart Types:

- Line charts (analog values)
- Step charts (digital states)
- Multi-axis plots (different units)
- Stacked area charts

Historical Data

Access stored trend data:

- **Time Range Selection**: Custom date/time picker
- **Data Export**: Download CSV, Excel, JSON
- **Statistics**: Min, max, average, standard deviation
- **Annotations**: Add notes to specific timestamps

Alarm Monitoring

Active Alarms

View current alarm conditions:

Priority	Point	Message	Time	Acknowledge
High	IN1	High Temperature	10:23 AM	☐
Medium	OUT2	Fan Feedback Error	10:15 AM	☑

Alarm Actions:

- Acknowledge individual alarms
- Acknowledge all alarms
- Filter by priority/type
- View alarm history

Notification Settings

Configure alarm notifications:

- **Email Alerts**: Send to specified addresses
- **SMS Notifications**: Text message alerts
- **Audio Alerts**: Browser notification sounds
- **Popup Windows**: Desktop notifications

System Health

Performance Metrics

Monitor system performance:

```
API Response Time:    45ms (avg)
Database Queries:     150/sec
Active Connections:   12
Memory Usage:         245MB / 512MB
CPU Load:            15%
```

Communication Stats

Track data collection:

- **Successful Polls**: Count of successful data requests
- **Failed Polls**: Error count and types
- **Average Response Time**: Communication latency
- **Retry Count**: Number of automatic retries

Auto-Refresh

Data automatically refreshes at configurable intervals:

- **Inputs/Outputs**: 5 seconds (default)
- **Variables**: 5 seconds (default)
- **Trends**: 10 seconds (real-time mode)
- **Alarms**: 3 seconds (high priority)

Manual Refresh: Click the **Refresh from Device** button to force immediate update.

Filters and Search

Quick Filters

- **Status**: Show only online/offline/error devices
- **Type**: Filter by point type (AI, AO, AV, BI, BO, BV)
- **Alarm**: Show only alarmed points
- **Override**: Show only overridden points

Search

Use the search box to find specific points:

- Search by name, description, or ID
- Case-insensitive matching
- Wildcard support (*, ?)

Data Export

Export monitored data:

Point List Export

1. Select points to export
2. Click **Export** button
3. Choose format (CSV, Excel, JSON)
4. Save file

Trend Data Export

1. Set time range
2. Select points
3. Choose interval (1s, 5s, 1m, etc.)
4. Export with timestamps

Best Practices

1. **Set Appropriate Refresh Rates**: Balance responsiveness vs. network load
2. **Use Filters**: Focus on critical points
3. **Monitor Trends**: Watch for unusual patterns
4. **Review Alarms Daily**: Don't ignore persistent alarms
5. **Export Regular Reports**: Keep historical records
6. **Configure Thresholds**: Set appropriate alarm limits

Performance Tips

- Limit number of simultaneous trend charts (max 10)
- Use longer refresh intervals for non-critical points
- Archive old trend data regularly
- Close unused browser tabs
- Use modern browsers (Chrome, Edge, Firefox)

Next Steps

- Trend Logs - Detailed trend analysis
- Alarms - Alarm configuration
- Troubleshooting - Resolve monitoring issues

Device Monitoring

Real-Time Data Streaming

WebSocket Subscriptions

TYPESCRIPT

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

// Subscribe to specific points
ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1'],
    interval: 5000  // Update interval in ms
  }));
};

// Handle real-time updates
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  switch (data.type) {
    case 'point-update':
      updatePointValue(data.point, data.value, data.timestamp);
      break;
    case 'alarm':
      handleAlarm(data.alarm);
      break;
    case 'device-status':
      updateDeviceStatus(data.deviceId, data.status);
      break;
  }
};
```

REST API Polling

TYPESCRIPT

```typescript
class DataPoller {
  private intervals = new Map<string, NodeJS.Timeout>();

  async pollPoints(deviceId: number, points: string[], interval: number) {
    const pollerId = `${deviceId}:${points.join(',')}`;

    // Clear existing poller
    if (this.intervals.has(pollerId)) {
      clearInterval(this.intervals.get(pollerId)!);
    }

    // Start new poller
    const timer = setInterval(async () => {
      try {
        const response = await fetch(
          `http://localhost:9103/api/devices/${deviceId}/points?ids=${points.join(',')}`
        );
        const data = await response.json();

        data.points.forEach(point => {
          this.emit('update', point);
        });
      } catch (error) {
        this.emit('error', error);
      }
    }, interval);

    this.intervals.set(pollerId, timer);
  }

  stopPolling(deviceId: number, points: string[]) {
    const pollerId = `${deviceId}:${points.join(',')}`;
    const timer = this.intervals.get(pollerId);
    if (timer) {
      clearInterval(timer);
      this.intervals.delete(pollerId);
    }
  }
}
```

Data Aggregation and Caching

In-Memory Cache

TYPESCRIPT

```typescript
interface CachedPoint {
  value: number;
  timestamp: number;
  quality: string;
  ttl: number;
}

class PointCache {
  private cache = new Map<string, CachedPoint>();
  private readonly defaultTTL = 10000; // 10 seconds

  set(pointId: string, value: number, quality: string = 'good') {
    this.cache.set(pointId, {
      value,
      timestamp: Date.now(),
      quality,
      ttl: this.defaultTTL
    });
  }

  get(pointId: string): CachedPoint | null {
    const cached = this.cache.get(pointId);
    if (!cached) return null;

    // Check if expired
    if (Date.now() - cached.timestamp > cached.ttl) {
      this.cache.delete(pointId);
      return null;
    }

    return cached;
  }

  invalidate(pointId: string) {
    this.cache.delete(pointId);
  }

  clear() {
    this.cache.clear();
  }
}
```

Batch Reading Optimization

```typescript
async function batchReadPoints(
  deviceId: number,
  points: string[],
  maxBatchSize: number = 50
): Promise<Map<string, number>> {
  const results = new Map<string, number>();

  // Split into batches
  for (let i = 0; i < points.length; i += maxBatchSize) {
    const batch = points.slice(i, i + maxBatchSize);

    const response = await fetch(
      `http://localhost:9103/api/devices/${deviceId}/points/batch`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ points: batch })
      }
    );

    const data = await response.json();
    data.values.forEach((value, index) => {
      results.set(batch[index], value);
    });
  }

  return results;
}
```

Trend Data Storage

SQLite Trend Table

```sql
CREATE TABLE trend_data (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id INTEGER NOT NULL,
  point_id TEXT NOT NULL,
  value REAL NOT NULL,
  quality TEXT DEFAULT 'good',
  timestamp INTEGER NOT NULL,
  FOREIGN KEY (device_id) REFERENCES devices(id)
);

CREATE INDEX idx_trends_device_point
  ON trend_data(device_id, point_id);
CREATE INDEX idx_trends_timestamp
  ON trend_data(timestamp DESC);

-- Partitioning by time for better performance
CREATE INDEX idx_trends_time_device
  ON trend_data(timestamp DESC, device_id, point_id);
```

Trend Data Insertion

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendLogger {
  private db: Database.Database;
  private insertStmt: Database.Statement;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);
    this.insertStmt = this.db.prepare(`
      INSERT INTO trend_data (device_id, point_id, value, quality, timestamp)
      VALUES (?, ?, ?, ?, ?)
    `);
  }

  log(deviceId: number, pointId: string, value: number, quality: string = 'good') {
    this.insertStmt.run(
      deviceId,
      pointId,
      value,
      quality,
      Date.now()
    );
  }

  logBatch(entries: Array<{deviceId: number, pointId: string, value: number}>) {
    const transaction = this.db.transaction((entries) => {
      for (const entry of entries) {
        this.insertStmt.run(
          entry.deviceId,
          entry.pointId,
          entry.value,
          'good',
          Date.now()
        );
      }
    });

    transaction(entries);
  }
}
```

Trend Data Query and Aggregation

TYPESCRIPT

```typescript
interface TrendQuery {
  deviceId: number;
  pointId: string;
  startTime: number;
  endTime: number;
  interval?: number;  // Aggregation interval in seconds
}

async function queryTrendData(query: TrendQuery) {
  const db = new Database('t3000.db');

  let sql: string;

  if (query.interval) {
    // Aggregated query
    sql = `
      SELECT
        (timestamp / ${query.interval * 1000}) * ${query.interval * 1000} as bucket,
        AVG(value) as avg_value,
        MIN(value) as min_value,
        MAX(value) as max_value,
        COUNT(*) as sample_count
      FROM trend_data
      WHERE device_id = ?
        AND point_id = ?
        AND timestamp BETWEEN ? AND ?
      GROUP BY bucket
      ORDER BY bucket
    `;
  } else {
    // Raw data query
    sql = `
      SELECT timestamp, value, quality
      FROM trend_data
      WHERE device_id = ?
        AND point_id = ?
        AND timestamp BETWEEN ? AND ?
      ORDER BY timestamp
    `;
  }

  return db.prepare(sql).all(
    query.deviceId,
    query.pointId,
    query.startTime,
    query.endTime
  );
}
```

Performance Monitoring

Metrics Collection

TYPESCRIPT

```typescript
class PerformanceMonitor {
  private metrics = {
    apiLatency: [] as number[],
    databaseQueries: 0,
    cacheHits: 0,
    cacheMisses: 0,
    activeConnections: 0
  };

  recordApiLatency(latency: number) {
    this.metrics.apiLatency.push(latency);
    if (this.metrics.apiLatency.length > 1000) {
      this.metrics.apiLatency.shift();
    }
  }

  getStats() {
    const latencies = this.metrics.apiLatency;
    return {
      avgLatency: latencies.reduce((a, b) => a + b, 0) / latencies.length,
      p95Latency: this.percentile(latencies, 0.95),
      p99Latency: this.percentile(latencies, 0.99),
      databaseQps: this.metrics.databaseQueries,
      cacheHitRate: this.metrics.cacheHits /
        (this.metrics.cacheHits + this.metrics.cacheMisses),
      activeConnections: this.metrics.activeConnections
    };
  }

  private percentile(arr: number[], p: number): number {
    const sorted = [...arr].sort((a, b) => a - b);
    const index = Math.ceil(sorted.length * p) - 1;
    return sorted[index];
  }
}
```

Automated Performance Optimization

TYPESCRIPT

```typescript
class AdaptivePolling {
  private currentInterval = 5000;
  private readonly minInterval = 1000;
  private readonly maxInterval = 30000;

  async adjustPollingRate(stats: PerformanceStats) {
    if (stats.avgLatency > 1000) {
      // High latency - reduce polling frequency
      this.currentInterval = Math.min(
        this.currentInterval * 1.5,
        this.maxInterval
      );
    } else if (stats.avgLatency < 200 && stats.cacheHitRate > 0.8) {
      // Good performance - can increase polling
      this.currentInterval = Math.max(
        this.currentInterval * 0.8,
        this.minInterval
      );
    }

    return this.currentInterval;
  }
}
```

Alarm Processing

Alarm Detection Engine

TYPESCRIPT

```typescript
interface AlarmRule {
  pointId: string;
  highLimit?: number;
  lowLimit?: number;
  deadband?: number;
  delay?: number;
  priority: 'high' | 'medium' | 'low';
}

class AlarmDetector {
  private rules = new Map<string, AlarmRule>();
  private states = new Map<string, {inAlarm: boolean, since: number}>();

  checkValue(pointId: string, value: number) {
    const rule = this.rules.get(pointId);
    if (!rule) return;

    let shouldAlarm = false;
    let message = '';

    if (rule.highLimit !== undefined && value > rule.highLimit) {
      shouldAlarm = true;
      message = `High limit exceeded: ${value} > ${rule.highLimit}`;
    } else if (rule.lowLimit !== undefined && value < rule.lowLimit) {
      shouldAlarm = true;
      message = `Low limit exceeded: ${value} < ${rule.lowLimit}`;
    }

    const state = this.states.get(pointId) || {inAlarm: false, since: 0};

    if (shouldAlarm && !state.inAlarm) {
      // New alarm
      const now = Date.now();
      if (rule.delay) {
        // Check if alarm persisted long enough
        if (!state.since) {
          state.since = now;
        } else if (now - state.since >= rule.delay * 1000) {
          this.triggerAlarm(pointId, message, rule.priority);
          state.inAlarm = true;
        }
      } else {
        this.triggerAlarm(pointId, message, rule.priority);
        state.inAlarm = true;
      }
    } else if (!shouldAlarm && state.inAlarm) {
      // Alarm cleared
      this.clearAlarm(pointId);
      state.inAlarm = false;
      state.since = 0;
    }

    this.states.set(pointId, state);
  }
```

```
  private triggerAlarm(pointId: string, message: string, priority: string) {
    // Store alarm in database
    // Send notifications
    // Emit event
  }

  private clearAlarm(pointId: string) {
    // Update alarm status
    // Send notification
  }
}
```

Next Steps

- REST API Reference
- WebSocket API
- Performance Tuning
- Troubleshooting - Resolve monitoring issues

Device Troubleshooting

Common issues and solutions for T3000 device connectivity and operation.

Connection Issues

Device Not Discovered

Symptoms:

- Device not appearing in discovery scan
- "No devices found" message

Solutions:

1. **Check Network Connectivity**

```shell
# Ping device IP
ping 192.168.1.100

# Check port availability
telnet 192.168.1.100 47808
```

2. **Verify Device Settings**
   - Confirm device IP address is correct
   - Check subnet mask matches network
   - Verify gateway configuration
   - Ensure BACnet port is 47808 (UDP)
3. **Check Firewall**
   - Allow UDP port 47808 (BACnet/IP)
   - Allow TCP port 502 (Modbus TCP)
   - Disable antivirus temporarily to test
   - Check Windows Firewall rules
4. **Network Broadcast Issues**
   - Some networks block broadcast packets
   - Try manual device addition instead
   - Contact IT to enable multicast

Connection Timeout

Symptoms:

- "Connection timeout" error
- Slow response times
- Intermittent disconnections

Solutions:

1. **Increase Timeout Values**
   - Go to Settings > Communication
   - Increase timeout from 5s to 10s
   - Increase retry count
2. **Reduce Network Load**
   - Decrease polling frequency
   - Limit concurrent connections

- Close other network applications

3. **Check Network Quality**
   - Test network latency: `ping -n 100 192.168.1.100`
   - Check for packet loss
   - Verify network switch/router health

Authentication Failed

Symptoms:

- "Authentication error" message
- "Access denied" errors

Solutions:

1. **Verify Credentials**
   - Check username/password
   - Verify case sensitivity
   - Try default credentials (if applicable)
2. **Check User Permissions**
   - Ensure user has adequate permissions
   - Verify user account is not locked
   - Contact administrator for access
3. **Device Configuration**
   - Verify authentication is enabled
   - Check allowed IP addresses
   - Review security settings

Data Reading Issues

Points Not Updating

Symptoms:

- Data values frozen/stale
- "Last update" timestamp old
- No new trend data

Solutions:

1. **Check Connection Status**
   - Verify device is online (green indicator)
   - Test communication with manual refresh
   - Review connection logs
2. **Verify Auto-Refresh**
   - Ensure auto-refresh is enabled
   - Check refresh interval setting
   - Try manual refresh (🔄 button)
3. **Clear Cache**

```javascript
// Browser console
localStorage.clear();
location.reload();
```

4. **Restart Services**
    - Restart T3000 API server
    - Refresh browser page
    - Reconnect device

Incorrect Values

Symptoms:

- Values don't match physical readings
- Negative values where impossible
- Out-of-range readings

Solutions:

1. **Check Calibration**
    - Review input calibration settings
    - Verify sensor type configuration
    - Check units conversion
2. **Verify Point Configuration**
    - Confirm correct input type (0-10V, 4-20mA)
    - Check scaling factors
    - Verify offset settings
3. **Test Hardware**
    - Check sensor wiring
    - Test with multimeter
    - Replace faulty sensors

Performance Issues

Slow Page Loading

Symptoms:

- Pages take long to load
- Browser freezes temporarily
- High CPU usage

Solutions:

1. **Reduce Data Load**
    - Decrease number of visible points
    - Increase refresh interval
    - Limit trend chart points
2. **Browser Optimization**
    - Use latest browser version
    - Clear browser cache
    - Close unused tabs
    - Disable browser extensions
3. **System Resources**
    - Close other applications
    - Check available RAM
    - Monitor CPU usage

Database Errors

Symptoms:

- "Database locked" errors
- "Failed after X retries" messages
- Slow save operations

Solutions:

1. **Wait and Retry**
   - Database automatically retries with exponential backoff
   - Wait up to 51 seconds for retry completion
   - Operation should succeed eventually
2. **Check Server Load**
   - Monitor API server CPU/memory
   - Reduce concurrent operations
   - Close other connections
3. **Database Maintenance**
   - Vacuum database (if SQLite)
   - Archive old trend data
   - Optimize database indexes

Trend Log Issues

No Trend Data

Symptoms:

- Empty trend charts
- "No data available" message
- Missing historical data

Solutions:

1. **Verify Trend Logging Enabled**
   - Check point configuration
   - Enable trend logging
   - Set appropriate log interval
2. **Check Time Range**
   - Verify selected time range has data
   - Try different date range
   - Check data retention settings
3. **Database Storage**
   - Verify sufficient disk space
   - Check database permissions
   - Review database logs

Gaps in Trend Data

Symptoms:

- Missing data segments
- Discontinuous charts
- Irregular timestamps

Causes:

- Network interruptions
- Device offline periods
- Database errors
- Polling failures

Solutions:

- Review connection logs for outages
- Check alarm history
- Verify continuous power supply

Alarm Issues

Alarms Not Triggering

Symptoms:

- No alarms despite out-of-range values
- Silent alarm conditions

Solutions:

1. **Verify Alarm Configuration**
   - Check alarm enable status
   - Verify threshold values
   - Review alarm priority
2. **Check Notification Settings**
   - Verify notification channels enabled
   - Test email/SMS configuration
   - Check browser notification permissions

False Alarms

Symptoms:

- Alarms for normal conditions
- Nuisance alarms

Solutions:

1. **Adjust Thresholds**
   - Review alarm setpoints
   - Add deadband/hysteresis
   - Use time delays
2. **Filter Noise**
   - Increase sensor filter time
   - Use averaging
   - Check for electrical interference

Browser Compatibility

UI Elements Not Working

Supported Browsers:

- ✅ Chrome 90+
- ✅ Edge 90+
- ✅ Firefox 88+

- ✅ Safari 14+

Not Supported:

- ❌ Internet Explorer
- ❌ Old browser versions

Solutions:

1. Update to latest browser version
2. Clear browser cache
3. Disable conflicting extensions
4. Try incognito/private mode

API Server Issues

Server Not Running

Check Server Status:

```powershell
## Windows
Get-Process | Where-Object { $_.Path -like "*T3000*api*" }

## If not running, start server
cd api
cargo run --release
```

Solutions:

1. Ensure API server is running on port 9103
2. Check server logs for errors
3. Verify no port conflicts
4. Restart server if needed

API Errors

Common Error Codes:

- **404**: Endpoint not found (check API path)
- **500**: Internal server error (check logs)
- **503**: Service unavailable (server overloaded)
- **timeout**: Server not responding

Getting Help

If issues persist:

1. **Check Logs**
   - Browser console (F12)
   - API server logs
   - Device logs
2. **Collect Information**
   - Error messages (exact text)
   - Steps to reproduce
   - Browser/OS version
   - Device firmware version
3. **Contact Support**
   - Include all collected information

- Attach relevant logs
- Provide screenshots

Next Steps

- Best Practices - Preventive measures
- Performance Tuning - Optimization tips
- FAQ - Frequently asked questions

Device Troubleshooting

Diagnostic Tools

Network Diagnostics

SHELL

```shell
## Test BACnet connectivity
nc -u -v 192.168.1.100 47808

## Send BACnet WhoIs packet
echo -ne '\x81\x0a\x00\x11\x01\x20\xff\xff\x00\xff\x10\x08' | nc -u 192.168.1.100 47808

## Test Modbus TCP
nc -v 192.168.1.100 502

## Capture BACnet traffic
tcpdump -i eth0 -n udp port 47808 -w bacnet.pcap
```

Connection Testing Script

TYPESCRIPT

```typescript
async function diagnost icsCheck(deviceIp: string): Promise<DiagnosticReport> {
  const report: DiagnosticReport = {
    timestamp: new Date(),
    tests: []
  };

  // 1. Ping test
  const pingResult = await testPing(deviceIp);
  report.tests.push({
    name: 'ICMP Ping',
    passed: pingResult.reachable,
    latency: pingResult.avgLatency,
    details: `${pingResult.packetLoss}% packet loss`
  });

  // 2. BACnet port test
  const bacnetResult = await testPort(deviceIp, 47808, 'udp');
  report.tests.push({
    name: 'BACnet Port (47808/UDP)',
    passed: bacnetResult.open,
    details: bacnetResult.message
  });

  // 3. Modbus port test
  const modbusResult = await testPort(deviceIp, 502, 'tcp');
  report.tests.push({
    name: 'Modbus Port (502/TCP)',
    passed: modbusResult.open,
    details: modbusResult.message
  });

  // 4. BACnet WhoIs test
  const whoIsResult = await sendBACnetWhoIs(deviceIp);
  report.tests.push({
    name: 'BACnet WhoIs Response',
    passed: whoIsResult.responded,
    details: `Device Instance: ${whoIsResult.deviceInstance || 'N/A'}`
  });

  return report;
}
```

Log Analysis

TYPESCRIPT

```typescript
class LogAnalyzer {
  analyzeConnectionLogs(logs: LogEntry[]): ConnectionAnalysis {
    const errors = logs.filter(l => l.level === 'error');
    const warnings = logs.filter(l => l.level === 'warn');

    // Detect patterns
    const timeoutErrors = errors.filter(e =>
      e.message.includes('timeout') || e.message.includes('ETIMEDOUT')
    );

    const authErrors = errors.filter(e =>
      e.message.includes('auth') || e.message.includes('permission')
    );

    const networkErrors = errors.filter(e =>
      e.message.includes('ECONNREFUSED') || e.message.includes('ENETUNREACH')
    );

    return {
      totalErrors: errors.length,
      totalWarnings: warnings.length,
      commonIssues: {
        timeouts: timeoutErrors.length,
        authentication: authErrors.length,
        network: networkErrors.length
      },
      recommendations: this.generateRecommendations({
        timeoutErrors,
        authErrors,
        networkErrors
      })
    };
  }

  private generateRecommendations(issues: any): string[] {
    const recs: string[] = [];

    if (issues.timeoutErrors.length > 5) {
      recs.push('Increase timeout settings or reduce polling frequency');
    }

    if (issues.authErrors.length > 0) {
      recs.push('Verify device credentials and user permissions');
    }

    if (issues.networkErrors.length > 3) {
      recs.push('Check network connectivity and firewall settings');
    }

    return recs;
  }
}
```

Performance Debugging

Database Lock Investigation

TYPESCRIPT

```typescript
// Enable WAL mode to reduce locking
import Database from 'better-sqlite3';

const db = new Database('t3000.db');

// Check current journal mode
const journalMode = db.pragma('journal_mode', { simple: true });
console.log('Journal mode:', journalMode);

// Enable WAL for concurrent access
db.pragma('journal_mode = WAL');

// Monitor lock waits
db.function('log_lock_wait', (duration: number) => {
  console.warn(`Database lock waited ${duration}ms`);
});

// Set busy timeout
db.pragma('busy_timeout = 5000');
```

Query Performance Analysis

SQL

```sql
-- Enable query planner
EXPLAIN QUERY PLAN
SELECT * FROM trend_data
WHERE device_id = 389001
  AND point_id = 'IN1'
  AND timestamp BETWEEN 1704067200 AND 1704153600
ORDER BY timestamp DESC;

-- Analyze table statistics
ANALYZE trend_data;

-- Check index usage
SELECT * FROM sqlite_stat1 WHERE tbl = 'trend_data';

-- Find missing indexes
SELECT * FROM (
  SELECT 'Missing index on trend_data(device_id)' as suggestion
  WHERE NOT EXISTS (
    SELECT 1 FROM sqlite_master
    WHERE type = 'index'
    AND tbl_name = 'trend_data'
    AND sql LIKE '%device_id%'
  )
);
```

Memory Profiling

TYPESCRIPT

```typescript
class MemoryProfiler {
  private baseline: number = 0;

  start() {
    if (global.gc) global.gc();
    this.baseline = process.memoryUsage().heapUsed;
  }

  checkpoint(label: string) {
    const current = process.memoryUsage();
    const delta = current.heapUsed - this.baseline;

    console.log(`[${label}] Memory delta: ${(delta / 1024 / 1024).toFixed(2)} MB`);
    console.log(`  Heap Used: ${(current.heapUsed / 1024 / 1024).toFixed(2)} MB`);
    console.log(`  External: ${(current.external / 1024 / 1024).toFixed(2)} MB`);
  }

  detectLeaks(iterations: number = 100) {
    const samples: number[] = [];

    for (let i = 0; i < iterations; i++) {
      if (global.gc) global.gc();
      samples.push(process.memoryUsage().heapUsed);
    }

    // Check for upward trend
    const slope = this.calculateSlope(samples);
    if (slope > 1024 * 100) { // 100 KB per iteration
      console.warn('Potential memory leak detected!');
      console.warn(`Growth rate: ${(slope / 1024).toFixed(2)} KB per iteration`);
    }
  }

  private calculateSlope(data: number[]): number {
    const n = data.length;
    const sumX = (n * (n - 1)) / 2;
    const sumY = data.reduce((a, b) => a + b, 0);
    const sumXY = data.reduce((sum, y, x) => sum + x * y, 0);
    const sumX2 = (n * (n - 1) * (2 * n - 1)) / 6;

    return (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
  }
}
```

Automated Error Recovery

Retry Mechanism with Exponential Backoff

TYPESCRIPT

```typescript
async function withRetry<T>(
  fn: () => Promise<T>,
  options: {
    maxRetries?: number;
    baseDelay?: number;
    maxDelay?: number;
    onRetry?: (attempt: number, error: Error) => void;
  } = {}
): Promise<T> {
  const {
    maxRetries = 3,
    baseDelay = 1000,
    maxDelay = 30000,
    onRetry = () => {}
  } = options;

  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      lastError = error as Error;

      if (attempt < maxRetries - 1) {
        const delay = Math.min(
          baseDelay * Math.pow(2, attempt),
          maxDelay
        );

        onRetry(attempt + 1, error as Error);
        await sleep(delay);
      }
    }
  }

  throw new Error(`Failed after ${maxRetries} attempts: ${lastError.message}`);
}
```

Circuit Breaker Pattern

TYPESCRIPT

```typescript
class CircuitBreaker {
  private failures = 0;
  private lastFailureTime = 0;
  private state: 'closed' | 'open' | 'half-open' = 'closed';

  constructor(
    private threshold: number = 5,
    private timeout: number = 60000,
    private resetTimeout: number = 30000
  ) {}

  async execute<T>(fn: () => Promise<T>): Promise<T> {
    if (this.state === 'open') {
      if (Date.now() - this.lastFailureTime > this.resetTimeout) {
        this.state = 'half-open';
      } else {
        throw new Error('Circuit breaker is OPEN');
      }
    }

    try {
      const result = await fn();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  private onSuccess() {
    this.failures = 0;
    this.state = 'closed';
  }

  private onFailure() {
    this.failures++;
    this.lastFailureTime = Date.now();

    if (this.failures >= this.threshold) {
      this.state = 'open';
      console.warn('Circuit breaker opened due to failures');
    }
  }
}
```

Self-Healing Connection Manager

TYPESCRIPT

```typescript
class SelfHealingConnectionManager {
  private connections = new Map<number, DeviceConnection>();
  private healthChecks = new Map<number, NodeJS.Timeout>();

  async ensureConnection(deviceId: number): Promise<DeviceConnection> {
    let conn = this.connections.get(deviceId);

    if (!conn || !conn.isHealthy()) {
      // Reconnect
      try {
        conn = await this.reconnect(deviceId);
        this.connections.set(deviceId, conn);
        this.startHealthCheck(deviceId);
      } catch (error) {
        console.error(`Failed to connect to device ${deviceId}:`, error);
        throw error;
      }
    }

    return conn;
  }

  private async reconnect(deviceId: number): Promise<DeviceConnection> {
    const device = await this.getDeviceConfig(deviceId);

    return await withRetry(
      () => this.createConnection(device),
      {
        maxRetries: 3,
        baseDelay: 2000,
        onRetry: (attempt, error) => {
          console.log(`Reconnecting to device ${deviceId}, attempt ${attempt}`);
        }
      }
    );
  }

  private startHealthCheck(deviceId: number) {
    const interval = setInterval(async () => {
      const conn = this.connections.get(deviceId);
      if (!conn) return;

      try {
        await conn.ping();
      } catch (error) {
        console.warn(`Health check failed for device ${deviceId}`);
        await this.reconnect(deviceId);
      }
    }, 30000); // Every 30 seconds

    this.healthChecks.set(deviceId, interval);
  }
}
```

Debugging Tools

Packet Capture Analysis

SHELL

```shell
## Capture BACnet traffic
tcpdump -i any -n udp port 47808 -w bacnet.pcap

## Analyze with tshark
tshark -r bacnet.pcap -Y bacnet -T fields \
  -e frame.time -e ip.src -e ip.dst -e bacnet.type

## Decode BACnet packet
tshark -r bacnet.pcap -Y bacnet -V | less
```

Debug Logging Configuration

TYPESCRIPT

```typescript
import winston from 'winston';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error'
    }),
    new winston.transports.File({
      filename: 'logs/combined.log'
    }),
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      )
    })
  ]
});

// Enable debug logging
logger.level = 'debug';

// Log with context
logger.debug('Reading point', {
  deviceId: 389001,
  pointId: 'IN1',
  protocol: 'bacnet'
});
```

Next Steps

- Performance Tuning
- REST API Debugging
- System Monitoring

- FAQ - Frequently asked questions

LAN Scanner — UDP Device Discovery

Pure Rust async port of the C++ `RefreshNetWorkDeviceListByUDPFunc()` from `T3000-Source/T3000/global_function.cpp:9796`.

Protocol Overview

T3000 devices listen on **UDP port 1234** for a broadcast query and respond with device metadata. The scanner broadcasts `[0x64, 0,0,0,0]` to `255.255.255.255:1234` and collects `0x65` responses from all devices on reachable subnets.

Constant	Value	Meaning
`UPD_BROADCAST_QRY_MSG`	`0x64` (100)	Query command byte
`RESPONSE_MSG`	`0x65` (101)	Device response command
`RESPONSE_TOTAL_SUB_INFO`	`0x2F` (47)	Sub-device list response
`BROADCAST_DEST_PORT`	1234	Devices listen here
`SCAN_PORT_RANGE`	57619–57623	Scanner binds here for replies

Architecture

TEXT

```
api/src/lan_scan/
├── mod.rs          # Module entry, re-exports
├── types.rs        # DiscoveredDevice (20 fields), ScanResult, product_map (130+ entries)
├── protocol.rs     # Constants, build_scan_query(), parse_scan_response(), parse_sub_device_info()
├── scanner.rs      # get_ip_mask_gateway(), scan_network(), scan_on_adapter(), bind_scan_port()
└── tests/lan_scan/ # 18 unit tests + 1 real-network test (ignored)
```

Response Packet Layout

Each response is interleaved: data byte, reserve byte, data byte, … Offsets below are indices into the raw buffer.

Offset	Size	Field	Notes
0	1	command	`0x65`
4,6,8,10	4×u8	serial_number	Little-endian u32
12	1	product_id	Maps to product_name via product_map
14	1	modbus_id	
16,18,20,22	4×u8	ip_address	
24–25	2	modbus_port	Big-endian u16
26–27	2	firmware_version	/10 for products 10,35,50,62,74,88
28–29	2	hardware_version	
30–33	4	parent_serial	Airlab bug: equal bytes → 0
34,35,57,58	4	object_instance	
36	1	panel_number	
37–56	20	panel_name	Null-terminated ASCII
59	1	isp_mode	0=app, 1=bootloader→skip, 2=corrupted→skip
60–61	2	bacnet_ip_port	Big-endian u16
62	1	hardware_info	
63	1	subnet_protocol	0→1 mapping, 12→10 for BACnet devices
64	1	command_version	Only if buf.len ≥ 67
65	1	subnet_port	Only if parent≠0
66	1	subnet_baudrate	Only if parent≠0
67	1	minitype	Always at 67

Sub-Device Info (0x2F)

Offset	Size	Field
0	1	command = 0x2F
1	1	device_count
2–5	4	parent_serial (u32 LE)
6–20	15	reserved
21+	pairs	(status:u8, modbus_id:u8) × device_count

Scanning Flow

```
TEXT
```

```
scan_network(timeout_secs)
  ├─ get_local_ips() → enumerates local adapter IPs
  │    └─ Windows: ipconfig /all parsing
  │    └─ Fallback: connect UDP to 8.8.8.8
  └─ For each adapter IP:
      ├─ bind_scan_port(ip) → binds to ip:57619..57623
      ├─ set_broadcast(true)
      ├─ send_to(255.255.255.255:1234, query)
      └─ recv_loop(timeout):
          ├─ 0x65 → parse_scan_response() → DiscoveredDevice
          └─ 0x2F → parse_sub_device_info() → SubDeviceInfo
  └─ Deduplicate by serial_number
```

Key Differences from C++

Aspect	C++	Rust
Adapter enum	`GetAdaptersInfo()` Win32 API	`ipconfig /all` string parsing
Broadcast target	`255.255.255.255:1234`	Same ✅
Bind port	`57619–57623`	Same ✅
Socket model	Blocking `select()` + `recvfrom()`	Async Tokio `UdpSocket`
Timeout	Fixed 8-second `select()`	Configurable via parameter
Dedup	Tracks seen IPs in send buffer	`HashSet<u32>` after scan

Bugs Caught During Porting

1. **Broadcast port**: Sent to 57619 instead of 1234 — devices never heard queries
2. **IP parsing**: `(Preferred)` suffix from `ipconfig /all` broke `Ipv4Addr::parse()`
3. **Variable shadowing**: `current_ip = Some(v)` used old variable with suffix
4. **Subnet protocol mapping**: Was `6→0`, should be `0→1` (MODBUS_RS485→MODBUS_TCPIP)
5. **Minitype offset**: Was conditional on `parent_serial`, should always be `buf[67]`
6. **Sub-device base offset**: Was 22, should be 21
7. **Airlab parent_serial bug**: Equal bytes → zero (matches C++ quirk)
8. **Firmware /10**: Only for specific product IDs (10, 35, 50, 62, 74, 88)

Real Hardware Test Results

Verified against live T3000 devices on the LAN — 5 devices found across two subnets, matching the C++ desktop application output:

```
TEXT
```

```
192.168.0.x: 4 devices (SN 271292, 271851, 212375, 240488)
192.168.1.x: 1 device  (SN 237219)
```

Usage

```shell
## Unit tests
cargo test --test lan_scan

## Real network scan (requires T3000 hardware on LAN)
cargo test --test lan_scan test_real_network_scan -- --ignored --nocapture

## From code
use t3_webview_api::lan_scan::scanner;
let result = scanner::scan_network(8).await;  // 8 second timeout
```

C++ Reference Files

- `T3000-Source/T3000/global_function.cpp:9796` — `RefreshNetWorkDeviceListByUDPFunc()`
- `T3000-Source/T3000/global_function.cpp:9037` — `AddNetDeviceForRefreshList()` parser
- `T3000-Source/T3000/global_function.cpp:8982` — `AddSubNetInfoIntoRefreshList()` sub-device parser
- `T3000-Source/T3000/global_function.cpp:9610` — `GetIPMaskGetWay()` adapter enumeration
- `T3000-Source/T3000/global_define.h:619` — `refresh_net_device` struct
- `T3000-Source/T3000/global_define.h:244` — `UDP_BROADCAST_PORT = 1234`

Alarms

Alarm monitoring, configuration, and notification system.

Overview

The alarm system monitors critical conditions and notifies operators when values exceed limits or equipment fails.

Alarm Types

Value Alarms:

- High limit exceeded
- Low limit exceeded
- Rate of change alarm
- Deviation alarm

Status Alarms:

- Equipment failure
- Communication loss
- Sensor fault
- Override active

System Alarms:

- Database errors
- Server issues
- License expiration

Alarm Priorities

- 🔴 **High**: Critical safety/equipment issues
- 🟠 **Medium**: Important but not critical
- 🟡 **Low**: Information/warnings
- ⚪ **Info**: Status changes

Alarm Page

View active and historical alarms:

Priority	Point	Message	Time	Status
High	Zone_Temp	High Temperature	10:23 AM	Active
Medium	Fan_Status	Feedback Error	10:15 AM	Acknowledged

Alarm Actions

- **Acknowledge**: Confirm awareness
- **Clear**: Resolve alarm condition
- **Silence**: Mute audio alerts
- **Filter**: View by priority/status

Notification Settings

Configure alarm notifications:

Email:

- Recipient addresses
- Subject line format
- Include alarm details

SMS:

- Phone numbers
- Message template
- Priority filtering

Browser:

- Desktop notifications
- Audio alerts
- Popup windows

Best Practices

- Set appropriate thresholds
- Use priority levels wisely
- Respond promptly
- Review alarm history
- Tune to reduce false alarms

Next Steps

- Inputs - Input alarms
- Monitoring - Real-time monitoring
- Troubleshooting - Resolve issues

Alarms

Alarm Management API

Alarm Data Structure

TYPESCRIPT

```typescript
interface Alarm {
  id: string;
  deviceId: number;
  pointId: string;
  type: 'high-limit' | 'low-limit' | 'fault' | 'override' | 'communication';
  priority: 'high' | 'medium' | 'low' | 'info';
  message: string;
  value: number;
  limit: number;
  timestamp: number;
  acknowledged: boolean;
  acknowledgedBy?: string;
  acknowledgedAt?: number;
}

// Get active alarms
const response = await fetch(`http://localhost:9103/api/alarms/active`);
const alarms: Alarm[] = await response.json();

// Acknowledge alarm
await fetch(`http://localhost:9103/api/alarms/${alarmId}/acknowledge`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ user: 'operator1' })
});
```

Alarm Detection Engine

TYPESCRIPT

```typescript
class AlarmEngine {
  private rules = new Map<string, AlarmRule>();
  private activeAlarms = new Map<string, Alarm>();

  checkPoint(pointId: string, value: number) {
    const rule = this.rules.get(pointId);
    if (!rule) return;

    const alarmId = `${pointId}:high-limit`;
    const isInAlarm = value > rule.highLimit;
    const existingAlarm = this.activeAlarms.get(alarmId);

    if (isInAlarm && !existingAlarm) {
      // New alarm
      if (this.checkDelay(pointId, rule.delay)) {
        this.triggerAlarm({
          id: alarmId,
          pointId,
          type: 'high-limit',
          priority: rule.priority,
          message: `${pointId} high limit exceeded`,
          value,
          limit: rule.highLimit
        });
      }
    } else if (!isInAlarm && existingAlarm) {
      // Alarm cleared
      this.clearAlarm(alarmId);
    }
  }

  private triggerAlarm(alarm: Alarm) {
    alarm.timestamp = Date.now();
    alarm.acknowledged = false;
    this.activeAlarms.set(alarm.id, alarm);

    // Send notifications
    this.notifySubscribers(alarm);
    this.sendEmail(alarm);
    this.logToDatabase(alarm);
  }
}
```

WebSocket Alarm Notifications

TYPESCRIPT

```typescript
// Subscribe to alarm events
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    channel: 'alarms',
    filters: {
      priority: ['high', 'medium'],
      deviceIds: [389001, 389002]
    }
  }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);

  if (data.type === 'alarm-triggered') {
    showNotification(data.alarm);
    playAlarmSound(data.alarm.priority);
  } else if (data.type === 'alarm-cleared') {
    clearNotification(data.alarmId);
  }
};
```

Email Notification

```typescript
import nodemailer from 'nodemailer';

class EmailNotifier {
  private transporter: nodemailer.Transporter;

  constructor(config: EmailConfig) {
    this.transporter = nodemailer.createTransporter({
      host: config.smtpHost,
      port: config.smtpPort,
      secure: config.useTLS,
      auth: {
        user: config.username,
        pass: config.password
      }
    });
  }

  async sendAlarmEmail(alarm: Alarm) {
    const html = `
      <h2>Alarm Notification</h2>
      <p><strong>Priority:</strong> ${alarm.priority.toUpperCase()}</p>
      <p><strong>Point:</strong> ${alarm.pointId}</p>
      <p><strong>Message:</strong> ${alarm.message}</p>
      <p><strong>Value:</strong> ${alarm.value}</p>
      <p><strong>Limit:</strong> ${alarm.limit}</p>
      <p><strong>Time:</strong> ${new Date(alarm.timestamp).toLocaleString()}</p>
    `;

    await this.transporter.sendMail({
      from: 'T3000 Alarms <alarms@t3000.local>',
      to: 'operator@example.com',
      subject: `[${alarm.priority.toUpperCase()}] ${alarm.pointId} ${alarm.type}`,
      html
    });
  }
}
```

Next Steps

- REST API
- WebSocket API
- Device Monitoring

Graphics

Visual interface for monitoring and controlling building systems.

Overview

Graphics provide intuitive visual representations of equipment, floor plans, and system diagrams with real-time data overlay.

Graphic Features

- **Floor Plans**: Building layouts with equipment locations
- **System Diagrams**: HVAC system schematics
- **Equipment Graphics**: Individual equipment views
- **Real-Time Data**: Live values displayed on graphics

Graphic Elements

- **Shapes**: Rectangles, circles, lines
- **Text**: Labels and values
- **Images**: Equipment photos, logos
- **Controls**: Buttons, sliders
- **Animations**: Moving/blinking elements based on status

Creating Graphics

1. Navigate to **Graphics** page
2. Click **New Graphic**
3. Add shapes and elements
4. Link data points to graphics
5. Save and publish

Best Practices

- Keep graphics simple and clear
- Use consistent colors and symbols
- Label all elements clearly
- Test on different screen sizes

Next Steps

- Dashboard - Main dashboard
- Monitoring - Data monitoring

Graphics

Graphics Rendering

SVG Graphics Engine

TYPESCRIPT

```typescript
interface GraphicElement {
  id: string;
  type: 'rect' | 'circle' | 'line' | 'text' | 'image';
  x: number;
  y: number;
  width?: number;
  height?: number;
  fill?: string;
  stroke?: string;
  dataBinding?: string;  // Point ID to bind
}

class GraphicsRenderer {
  private svg: SVGSVGElement;
  private elements = new Map<string, GraphicElement>();

  render(graphic: Graphic) {
    this.svg = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
    this.svg.setAttribute('viewBox', `0 0 ${graphic.width} ${graphic.height}`);

    for (const element of graphic.elements) {
      const svgEl = this.createElement(element);
      this.svg.appendChild(svgEl);

      if (element.dataBinding) {
        this.bindData(svgEl, element.dataBinding);
      }
    }

    return this.svg;
  }

  updateValue(pointId: string, value: number) {
    // Find elements bound to this point
    const elements = Array.from(this.elements.values()).filter(
      e => e.dataBinding === pointId
    );

    for (const element of elements) {
      this.updateElement(element, value);
    }
  }
}
```

Real-Time Data Binding

```typescript
class GraphicsDataBinder {
  private ws: WebSocket;
  private renderer: GraphicsRenderer;

  constructor(renderer: GraphicsRenderer) {
    this.renderer = renderer;
    this.ws = new WebSocket('ws://localhost:9103/ws');

    this.ws.onmessage = (event) => {
      const data = JSON.parse(event.data);

      if (data.type === 'point-update') {
        this.renderer.updateValue(data.point, data.value);
      }
    };
  }

  subscribeToPoints(pointIds: string[]) {
    this.ws.send(JSON.stringify({
      type: 'subscribe',
      points: pointIds
    }));
  }
}
```

Next Steps

- REST API
- WebSocket API

Holidays

Annual holiday and exception day scheduling.

Overview

Holiday schedules override normal weekly schedules for special events, holidays, and maintenance periods.

Holiday Calendar

Define holidays and special days:

- New Year's Day
- Independence Day
- Thanksgiving
- Christmas Day
- Maintenance days
- Company events

Creating Holidays

1. Go to **Holidays** page
2. Add new holiday
3. Set date (or recurring pattern)
4. Define exception schedule
5. Save

Schedule Priority

1. Holidays (highest priority)
2. Exception schedules
3. Weekly schedules (lowest priority)

Next Steps

- Schedules - Weekly schedules
- Programs - Control programs

Holidays

Holiday Management API

TYPESCRIPT

```typescript
interface Holiday {
  id: number;
  name: string;
  date: string;  // ISO date or recurrence pattern
  recurring: boolean;
  scheduleOverride: ScheduleEvent[];
}

// Get holidays
const holidays = await fetch(
  `http://localhost:9103/api/holidays`
).then(r => r.json());

// Create holiday
await fetch(`http://localhost:9103/api/holidays`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    name: 'New Year\'s Day',
    date: '2026-01-01',
    recurring: true,
    scheduleOverride: [
      { time: '00:00', value: 60 }  // Unoccupied all day
    ]
  })
});
```

Holiday Detection

TYPESCRIPT

```typescript
class HolidayChecker {
  private holidays: Holiday[];

  isHoliday(date: Date): Holiday | null {
    const dateStr = date.toISOString().split('T')[0];

    for (const holiday of this.holidays) {
      if (holiday.recurring) {
        // Check month-day match
        const holidayMD = holiday.date.substring(5);  // "MM-DD"
        const dateMD = dateStr.substring(5);

        if (holidayMD === dateMD) {
          return holiday;
        }
      } else {
        if (holiday.date === dateStr) {
          return holiday;
        }
      }
    }

    return null;
  }
}
```

Next Steps

- Schedules API
- REST API

Schedules

Time-based control schedules for automated equipment operation.

Overview

Schedules automate equipment operation based on time of day and day of week. Common uses include occupancy schedules, setpoint changes, and equipment start/stop times.

Schedule Types

- **Weekly Schedules**: Different times for each day of week
- **Exception Schedules**: Override for holidays/special events
- **Annual Schedules**: Yearly calendar events

Creating Schedules

1. Navigate to **Schedules** page
2. Click **Add Schedule**
3. Set times for each day
4. Assign to outputs/variables
5. Enable schedule

Example Schedule

Monday-Friday:

- 6:00 AM: Occupied mode, 72F
- 6:00 PM: Unoccupied mode, 65F

Saturday-Sunday:

- All day: Unoccupied mode, 60F

Next Steps

- Holidays - Holiday/exception schedules
- Programs - Schedule programs

Schedules

Schedule API

Schedule Data Model

TYPESCRIPT

```typescript
interface Schedule {
  id: number;
  name: string;
  enabled: boolean;
  weeklySchedule: DaySchedule[];
  exceptions: ExceptionSchedule[];
}

interface DaySchedule {
  dayOfWeek: 0 | 1 | 2 | 3 | 4 | 5 | 6;  // 0 = Sunday
  events: ScheduleEvent[];
}

interface ScheduleEvent {
  time: string;  // "HH:MM"
  value: number;
}

// Get all schedules
const schedules = await fetch(
  `http://localhost:9103/api/schedules`
).then(r => r.json());

// Create schedule
await fetch(`http://localhost:9103/api/schedules`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    name: 'Office Occupancy',
    enabled: true,
    weeklySchedule: [
      {
        dayOfWeek: 1,  // Monday
        events: [
          { time: '06:00', value: 72 },
          { time: '18:00', value: 65 }
        ]
      }
    ]
  })
});
```

Schedule Execution Engine

TYPESCRIPT

```typescript
class ScheduleExecutor {
  private schedules = new Map<number, Schedule>();
  private timers = new Map<number, NodeJS.Timeout>();

  start(schedule: Schedule) {
    // Calculate next event time
    const nextEvent = this.findNextEvent(schedule);
    if (!nextEvent) return;

    const delay = nextEvent.time - Date.now();

    const timer = setTimeout(() => {
      this.executeEvent(schedule, nextEvent);
      this.start(schedule);  // Schedule next event
    }, delay);

    this.timers.set(schedule.id, timer);
  }

  private findNextEvent(schedule: Schedule) {
    const now = new Date();
    const currentDay = now.getDay();
    const currentTime = `${now.getHours().toString().padStart(2, '0')}:${now.getMinutes().toString().padStart(2, '0')}`;

    // Check today's remaining events
    const todaySchedule = schedule.weeklySchedule.find(
      s => s.dayOfWeek === currentDay
    );

    if (todaySchedule) {
      const remainingEvents = todaySchedule.events.filter(
        e => e.time > currentTime
      );

      if (remainingEvents.length > 0) {
        return {
          time: this.parseTime(remainingEvents[0].time),
          value: remainingEvents[0].value
        };
      }
    }

    // Find next day's first event
    for (let i = 1; i <= 7; i++) {
      const nextDay = (currentDay + i) % 7;
      const daySchedule = schedule.weeklySchedule.find(
        s => s.dayOfWeek === nextDay
      );

      if (daySchedule && daySchedule.events.length > 0) {
        return {
          time: this.parseTime(daySchedule.events[0].time, i),
          value: daySchedule.events[0].value
        };
      }
    }
```

```
    return null;
  }
}
```

BACnet Schedule Object

TYPESCRIPT

```typescript
// Read BACnet schedule
const schedule = await client.readProperty({
  deviceInstance: 389001,
  objectType: 'schedule',
  objectInstance: 1,
  property: 'weekly-schedule'
});

// Write schedule
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'schedule',
  objectInstance: 1,
  property: 'weekly-schedule',
  value: {
    monday: [
      { time: { hour: 6, minute: 0 }, value: 72 },
      { time: { hour: 18, minute: 0 }, value: 65 }
    ]
  }
});
```

Next Steps

- REST API
- BACnet Objects
- Programs

Settings Field Mapping: C++ ↔ Frontend

Overview

This document maps the C++ `Str_Setting_Info` structure (400 bytes) to frontend React form fields.

C++ Source: `T3000-Source/T3000/CM5/ud_str.h` (lines 826-914) **Frontend Interface:** `src/t3-react/features/settings/services/settingsRefreshApi.ts`

Data Flow

```
Device (400 bytes)
    ↓ FFI Action 17 (GET_WEBVIEW_LIST)
Raw Byte Array [0-399]
    ↓ parseSettingsData()
DeviceSettings Object
    ↓ UI Binding
React Form Fields
    ↓ User Edit & Save
DeviceSettings Object
    ↓ serializeSettingsData()
Raw Byte Array [0-399]
    ↓ FFI Action 16 (UPDATE_WEBVIEW_LIST)
Device (400 bytes)
```

Field Mapping Table

Tab 1: Basic Information

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`ip_addr[4]`	0-3	uint8[]	4	`ip_addr`	Input (IP)	Format: "192.168.0.144"
`subnet[4]`	4-7	uint8[]	4	`subnet`	Input (IP)	Format: "255.255.255.0"
`gate_addr[4]`	8-11	uint8[]	4	`gate_addr`	Input (IP)	Format: "192.168.0.0"
`mac_addr[6]`	12-17	uint8[]	6	`mac_addr`	Input (MAC)	Format: "00:14:C6:F2:E0:C3"
`tcp_type`	18	uint8	1	`tcp_type`	Switch	0=DHCP, 1=STATIC
`mini_type`	19	uint8	1	`mini_type`	Dropdown	Device type
`panel_type`	51	uint8	1	`panel_type`	Dropdown	Panel type
`panel_name[20]`	52-71	char[]	20	`panel_name`	Input	Max 20 chars
`en_panel_name`	72	uint8	1	`en_panel_name`	Switch	0=disabled, 1=enabled
`panel_number`	73	uint8	1	`panel_number`	Input (number)	Panel ID
`n_serial_number`	151-154	uint32	4	`n_serial_number`	Input (readonly)	Device serial
`object_instance`	186-189	uint32	4	`object_instance`	Input (number)	BACnet instance

Tab 2: Communication

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
com0_config	38	uint8	1	com0_config	Dropdown	COM0 mode
com1_config	39	uint8	1	com1_config	Dropdown	COM1 mode
com2_config	40	uint8	1	com2_config	Dropdown	COM2 mode
com_baudrate0	44	uint8	1	com_baudrate0	Dropdown	Baud rate index
com_baudrate1	45	uint8	1	com_baudrate1	Dropdown	Baud rate index
com_baudrate2	46	uint8	1	com_baudrate2	Dropdown	Baud rate index
uart_parity[3]	235-237	uint8[]	3	uart_parity	Dropdown[]	0=None, 1=Odd, 2=Even
uart_stopbit[3]	238-240	uint8[]	3	uart_stopbit	Dropdown[]	0=1bit, 1=2bits
network_number	50	uint8	1	network_number	Input (number)	Low byte
network_number_hi	252	uint8	1	network_number_hi	Input (number)	High byte
mstp_network_number	179-180	uint16	2	mstp_network_number	Input (number)	MSTP network
mstp_id	230	uint8	1	mstp_id	Input (number)	MSTP ID
max_master	233	uint8	1	max_master	Input (number)	Max master
modbus_port	183-184	uint16	2	modbus_port	Input (number)	Modbus port
modbus_id	185	uint8	1	modbus_id	Input (number)	Modbus ID
BBMD_EN	181	uint8	1	BBMD_EN	Switch	BBMD enable
fix_com_config	257	uint8	1	fix_com_config	Input (number)	Fixed COM config

Tab 3: Time Settings

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
time_zone	149-150	int16	2	time_zone	Dropdown	Signed offset
time_update_since_1970	190-193	uint32	4	time_update_since_1970	Display	Unix timestamp
en_sntp	148	uint8	1	en_sntp	Switch	0=no, 1=disable, 2=enable
sntp_server[30]	195-224	char[]	30	sntp_server	Input	SNTP server address
time_zone_summer_daytime	194	uint8	1	time_zone_summer_daytime	Switch	DST enable
time_sync_auto_manual	228	uint8	1	time_sync_auto_manual	Switch	0=SNTP, 1=Manual
start_month	248	uint8	1	start_month	Dropdown	DST start month
start_day	249	uint8	1	start_day	Input (number)	DST start day
end_month	250	uint8	1	end_month	Dropdown	DST end month
end_day	251	uint8	1	end_day	Input (number)	DST end day

Tab 4: DynDNS Settings

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`en_dyndns`	144	uint8	1	`en_dyndns`	Switch	0=no, 1=disable, 2=enable
`dyndns_provider`	145	uint8	1	`dyndns_provider`	Dropdown	0=3322.org, 1=dyndns.com, 2=no-ip.com
`dyndns_user[20]`	74-93	char[]	20	`dyndns_user`	Input	Username
`dyndns_pass[20]`	94-113	char[]	20	`dyndns_pass`	Input (password)	Password
`dyndns_domain[30]`	114-143	char[]	30	`dyndns_domain`	Input	Domain name
`dyndns_update_time`	146-147	uint16	2	`dyndns_update_time`	Input (number)	Update interval (minutes)

Tab 5: Email Alarms

Email alarm settings are stored in a separate table/structure - not part of Str_Setting_Info

Tab 6: User Login

C++ Field	Offset	Type	Size	Frontend Field	Form Control	Notes
`user_name`	47	uint8	1	`user_name`	Switch	0=no, 1=disable, 2=enable

Tab 7: Expansion IO

Expansion IO settings are stored in a separate table - not part of Str_Setting_Info

---

Readonly/Metadata Fields

C++ Field	Offset	Type	Size	Frontend Field	Usage
`debug`	20	uint8	1	`debug`	Debug flag (internal)
`refresh_flash_timer`	41	uint8	1	`refresh_flash_timer`	Flash refresh (internal)
`en_plug_n_play`	42	uint8	1	`en_plug_n_play`	PnP enable (internal)
`reset_default`	43	uint8	1	`reset_default`	Reset flag (write 88)
`custmer_unite`	48	uint8	1	`custmer_unite`	Customer units
`usb_mode`	49	uint8	1	`usb_mode`	0=device, 1=host
`sd_exist`	182	uint8	1	`sd_exist`	SD card status
`zegbee_exsit`	225	uint8	1	`zegbee_exsit`	Zigbee exists
`LCD_Display`	226	uint8	1	`LCD_Display`	LCD on/off
`zigbee_panid`	231-232	uint16	2	`zigbee_panid`	Zigbee PAN ID
`special_flag`	234	uint8	1	`special_flag`	Special flags
`webview_json_flash`	253	uint8	1	`webview_json_flash`	JSON format flag
`max_var`	254	uint8	1	`max_var`	Max variables (ESP32)
`max_in`	255	uint8	1	`max_in`	Max inputs (ESP32)
`max_out`	256	uint8	1	`max_out`	Max outputs (ESP32)
`write_flash`	258	uint8	1	`write_flash`	Write flash flag

Data Type Conversions

IP Address (4 bytes → string)

TYPESCRIPT

```typescript
// C++: uint8_t ip_addr[4] = {192, 168, 0, 144}
// Frontend: "192.168.0.144"
const bytesToIP = (bytes: number[]): string => bytes.slice(0, 4).join('.');
const ipToBytes = (ip: string): number[] => ip.split('.').map(Number);
```

MAC Address (6 bytes → string)

TYPESCRIPT

```typescript
// C++: uint8_t mac_addr[6] = {0x00, 0x14, 0xC6, 0xF2, 0xE0, 0xC3}
// Frontend: "00:14:C6:F2:E0:C3"
const bytesToMAC = (bytes: number[]): string =>
  bytes.slice(0, 6).map(b => b.toString(16).padStart(2, '0').toUpperCase()).join(':');
const macToBytes = (mac: string): number[] =>
  mac.split(':').map(hex => parseInt(hex, 16));
```

String (char[] → string)

TYPESCRIPT

```typescript
// C++: char panel_name[20] = "Fandu144-BB"
// Frontend: "Fandu144-BB"
const bytesToString = (bytes: number[]): string =>
  String.fromCharCode(...bytes.filter(b => b !== 0));
const stringToBytes = (str: string, maxLen: number): number[] => {
  const bytes = new Array(maxLen).fill(0);
  for (let i = 0; i < Math.min(str.length, maxLen); i++) {
    bytes[i] = str.charCodeAt(i);
  }
  return bytes;
};
```

Multi-byte Numbers (Little Endian)

TYPESCRIPT

```typescript
// uint16: 2 bytes
const bytesToUint16 = (bytes: number[], offset: number): number =>
  bytes[offset] | (bytes[offset + 1] << 8);
const uint16ToBytes = (value: number): number[] =>
  [value & 0xFF, (value >> 8) & 0xFF];

// uint32: 4 bytes
const bytesToUint32 = (bytes: number[], offset: number): number =>
  bytes[offset] | (bytes[offset + 1] << 8) | (bytes[offset + 2] << 16) | (bytes[offset + 3] << 24);
const uint32ToBytes = (value: number): number[] =>
  [value & 0xFF, (value >> 8) & 0xFF, (value >> 16) & 0xFF, (value >> 24) & 0xFF];

// int16 (signed): 2 bytes
const bytesToInt16 = (bytes: number[], offset: number): number => {
  const value = bytes[offset] | (bytes[offset + 1] << 8);
  return value > 32767 ? value - 65536 : value; // Convert to signed
};
```

Dropdown Value Mappings

COM Config (com0_config, com1_config, com2_config)

TYPESCRIPT

```typescript
const COM_CONFIG_OPTIONS = [
  { value: 0, label: 'Not Used' },
  { value: 1, label: 'BACnet MSTP' },
  { value: 2, label: 'Main Modbus' },
  { value: 3, label: 'Main PTP' },
  { value: 4, label: 'Sub GSM' },
  { value: 5, label: 'Main Zigbee' },
  { value: 6, label: 'Sub Zigbee' },
  { value: 7, label: 'Sub Modbus' },
  { value: 8, label: 'RS232 Meter' },
  { value: 9, label: 'MSTP Master' },
];
```

Baudrate (com_baudrate0/1/2)

TYPESCRIPT

```typescript
const BAUDRATE_OPTIONS = [
  { value: 0, label: '9600' },
  { value: 1, label: '19200' },
  { value: 2, label: '38400' },
  { value: 3, label: '57600' },
  { value: 4, label: '115200' },
  { value: 5, label: '76800' },
];
```

UART Parity

TYPESCRIPT

```typescript
const PARITY_OPTIONS = [
  { value: 0, label: 'None' },
  { value: 1, label: 'Odd' },
  { value: 2, label: 'Even' },
];
```

UART Stop Bits

TYPESCRIPT

```typescript
const STOPBIT_OPTIONS = [
  { value: 0, label: '1 bit' },
  { value: 1, label: '2 bits' },
];
```

DynDNS Provider

TYPESCRIPT

```typescript
const DYNDNS_PROVIDER_OPTIONS = [
  { value: 0, label: 'www.3322.org' },
  { value: 1, label: 'www.dyndns.com' },
  { value: 2, label: 'www.no-ip.com' },
];
```

Example: Complete Field Parsing

TYPESCRIPT

```typescript
// From 400-byte array to DeviceSettings object
const all = response.data.device_data[0].All; // Array of 400 bytes

const settings: DeviceSettings = {
  // Network (bytes 0-18)
  ip_addr: `${all[0]}.${all[1]}.${all[2]}.${all[3]}`,
  subnet: `${all[4]}.${all[5]}.${all[6]}.${all[7]}`,
  gate_addr: `${all[8]}.${all[9]}.${all[10]}.${all[11]}`,
  mac_addr: all.slice(12, 18).map(b => b.toString(16).padStart(2, '0').toUpperCase()).join(':'),
  tcp_type: all[18],

  // Device info
  mini_type: all[19],
  panel_type: all[51],
  panel_name: String.fromCharCode(...all.slice(52, 72).filter(b => b !== 0)),
  en_panel_name: all[72],
  panel_number: all[73],

  // Serial number (4 bytes, little endian, offset 151)
  n_serial_number: all[151] | (all[152] << 8) | (all[153] << 16) | (all[154] << 24),

  // ... continue for all fields
};
```

Update Flow (Save Settings)

TYPESCRIPT

```
// From DeviceSettings object to 400-byte array
const all = new Array(400).fill(0);

// Network settings
const ipParts = settings.ip_addr.split('.').map(Number);
all[0] = ipParts[0];
all[1] = ipParts[1];
all[2] = ipParts[2];
all[3] = ipParts[3];

// MAC address
const macParts = settings.mac_addr.split(':').map(hex => parseInt(hex, 16));
all[12] = macParts[0];
all[13] = macParts[1];
// ... etc

// String fields
const nameBytes = stringToBytes(settings.panel_name, 20);
for (let i = 0; i < 20; i++) {
  all[52 + i] = nameBytes[i];
}

// Multi-byte numbers
const serialBytes = uint32ToBytes(settings.n_serial_number);
all[151] = serialBytes[0];
all[152] = serialBytes[1];
all[153] = serialBytes[2];
all[154] = serialBytes[3];

// Send to device via FFI
POST /api/t3000/ffi/call
{
  "action": 16,
  "panelId": panelId,
  "serialNumber": serialNumber,
  "entryType": 98,
  "entryIndex": 0,
  "All": all // 400-byte array
}
```

Trend Center Unified Design

Goal

Unify Trend Logs, Trend Policy, Trend Chart, and Verify Data into one clear user flow rooted at /t3000/trendlogs.

Current issue: users must jump across multiple pages with overlapping responsibilities.

Target result: one Trend Center page with tabs and deep links.

Scope

Primary entry:

- /t3000/trendlogs

Legacy pages retained for compatibility:

- /t3000/trend-policy
- /t3000/trends/chart
- Dashboard section Verify Data action

Compatibility rule:

- Keep old links working.
- Redirect or route-map old pages to the new tab model.

Information Architecture

Use /t3000/trendlogs as Trend Center, with internal tabs:

1. Overview

- 24h trend health summary
- quick actions (refresh, verify, open chart)
- monitor status highlights (active/stalled/no data)

2. Monitors

- monitor table (existing trendlogs list)
- monitor input list (existing right panel)
- open chart for selected monitor

3. Points and Tags

- global point picker for INPUT/OUTPUT/VARIABLE
- search and filter
- apply/remove Haystack tags
- save point/tag policy profile

4. Chart

- full chart workspace
- selected monitor and points sidebar
- time range presets and custom range
- compare multiple selected sensors

5. Verify Data

- existing verification UI as full tab view
- optional drawer launch for quick diagnostics

One-Page Wireframe

Text wireframe for desktop:

Trend Center (/t3000/trendlogs) [Device Selector] [Search] [Refresh] [Verify]

[Overview] [Monitors] [Points and Tags] [Chart] [Verify Data]

Tab: Overview

- KPI cards: monitors, active points, stale points, missing samples
- trend quality banner
- quick jumps: Open Monitor, Open Chart, Verify

Tab: Monitors

- Left: monitor table (80%)
- Right: monitor inputs list (20%)
- Actions: Open Chart, Verify Selected, Edit Policy

Tab: Points and Tags

- Left: device and point filters
- Center: selectable point list with type tabs
- Right: selected points and tag chips
- Actions: Add Tag, Remove Tag, Save Policy

Tab: Chart

- Left: selected sensors/points tree
- Right: chart canvas and legend
- Bottom: sampled data quality indicators

Tab: Verify Data

- Per-point status, expected vs actual, gap analysis, sparklines
- Actions: Export report, Jump to Chart

Mobile behavior:

- tabs become segmented control + drawer for filters
- monitor input panel collapses below monitor list
- chart legend becomes bottom sheet

User Flows

Flow A: Quick health check

1. Open /t3000/trendlogs
2. Land on Overview
3. Click Verify Data
4. Inspect stale points and jump to chart

Flow B: Configure global points and tags

1. Open Points and Tags tab
2. Select devices
3. Filter by INPUT/OUTPUT/VARIABLE
4. Select points
5. Apply Haystack tags

6. Save trend policy profile

Flow C: Analyze selected sensors

1. From Monitors tab select monitor
2. Open Chart tab with selected monitor context
3. Add/remove points
4. Change time range
5. Export data if needed

Haystack Tag Strategy

Treat tags as first-class filtering and governance metadata.

Minimum tag set per trended point:

- point type: input/output/variable
- semantic: temp/humidity/pressure/flow/co2/occupancy
- location or equip: zone/site/equip

Suggested fields:

- tags: string[]
- pointRef: serial:type:index
- source: manual | inferred
- updatedAt, updatedBy

Governance rules:

- warn if selected trending points are untagged
- warn if conflicting semantic tags exist on one point
- allow saved smart filters based on tag combinations

Route and Navigation Consolidation

Recommended route behavior

Keep:

- /t3000/trendlogs as main route

Map legacy routes to tab-based navigation:

- /t3000/trend-policy -> /t3000/trendlogs?tab=points-tags
- /t3000/trends/chart -> /t3000/trendlogs?tab=chart&serial_number=...&panel_id=...&trendlog_id=...&monitor_id=...

Dashboard Verify Data action:

- open /t3000/trendlogs?tab=verify&serial=...&panel=...

Query parameter contract

- tab: overview | monitors | points-tags | chart | verify
- serial: number
- panel: number
- trendlogId: string
- monitorId: string
- points: comma-separated pointRefs
- tags: comma-separated tag tokens

Implementation Mapping (Current Code)

Existing components to reuse:

- Trend center host page: src/t3-react/features/trendlogs/pages/TrendlogsPage.tsx

- Tag and global point selection logic: src/t3-react/features/trendlogs/pages/TrendPolicyPage.tsx
- Chart page/content: src/t3-react/features/trendlogs/pages/TrendChartPage.tsx and src/t3-react/features/trendlogs/components/TrendChartContent.tsx
- Verify data diagnostics: src/t3-react/features/trendlogs/components/TrendlogVerifyDrawer.tsx
- Routes: src/t3-react/app/router/routes.ts
- Breadcrumb labels: src/t3-react/layout/PageHeader.tsx
- Dashboard launch points: src/t3-react/features/dashboard/pages/DashboardPage.tsx

Phased Delivery Plan

Phase 1: Navigation unification (low risk)

- Add tab state to TrendlogsPage
- Allow deep-link via query parameter tab
- Add adapters from legacy routes to tab states
- Keep old routes active

Acceptance:

- Users can reach all trend functions from /t3000/trendlogs
- Old URLs still function

Phase 2: Embed policy and verify modules

- Mount TrendPolicy UI as Points and Tags tab content
- Mount verify component as Verify tab content
- Keep existing Verify drawer for quick access

Acceptance:

- No required context switch for point/tag configuration
- Verify diagnostics accessible from same page

Phase 3: Chart integration and context sharing

- Open chart tab in-page using shared state
- Pass selected monitor and selected points through shared trend store
- Keep /t3000/trends/chart for C++ compatibility

Acceptance:

- Chart loads with selected monitor context directly from Monitors tab
- C++ launch path unchanged

Phase 4: Haystack enhancements

- Persist tags with pointRef identities
- Add saved filters and validation warnings
- Add import/export for tag profiles

Acceptance:

- Users can filter and validate trends by semantic tags

Data Model Notes

If persistence is needed for tags and policy profile:

- Option A: store in existing backend table keyed by serial + pointRef
- Option B: client-side first using local storage, then migrate to backend

Recommended order:

1. Client state MVP

2. API for persistence
3. Multi-user conflict resolution

UX Design Rules

- One page for all trend actions.
- Keep verify and policy one click away.
- Minimize context switching.
- Keep chart context sticky when switching tabs.
- Always show selected device and selected monitor in top bar.

Success Metrics

- Reduced navigation hops from dashboard to chart/verify/policy
- Faster first chart render from monitor selection
- Lower percentage of untagged trended points
- Fewer support questions about where to configure trend policy

Proposed Next Build Tasks

1. Add Trend Center tabs and query-state handling in TrendlogsPage.
2. Add route adapters for /trend-policy and /trends/chart.
3. Move Verify UI into a full tab panel while keeping drawer action.
4. Integrate TrendPolicyPage logic into Points and Tags tab.
5. Add tag persistence API contract and implementation.

TrendLog Chart X-Axis Design

Overview

The TrendLog chart X-axis uses **Solution 3: Hybrid Right Edge Rounding**. The right edge is always rounded *forward* to the nearest clean boundary based on the active timebase, then the left edge is computed by subtracting the window duration from that rounded right edge.

Rounding Rules

Under 1 hour (5m, 10m, 30m)

Right edge rounds **up to the next minute ending in 0 or 5**.

Now	Right edge
07:22	07:25
07:25	07:25 *(exact boundary, no shift)*
07:25:01	07:30
07:17	07:20

1 hour and above (1h, 4h, 12h, 1d, 4d)

Right edge rounds **up to the next full :00 hour**.

Now	Right edge
07:22	08:00
08:00	08:00 *(exact boundary, no shift)*
08:00:01	09:00

Custom range

No rounding. The user supplies an explicit start and end time; those values are used as-is.

Window Calculation

```
TEXT

rightEdge  = roundRight(currentTime, timebase)   // applies rules above
leftEdge   = rightEdge - windowDuration(timebase)
```

Timebase	Window duration	Tick step
5m	5 min	1 min
10m	10 min	1 min
30m	30 min	5 min
1h	60 min	10 min
4h	4 h	30 min
12h	12 h	1 h
1d	24 h	2 h
4d	4 days	12 h

Examples (now = 07:22)

Timebase	Left edge	Right edge	Ticks (sample)
5m	07:20	07:25	07:20, 21, 22, 23, 24, 07:25
10m	07:15	07:25	07:15, 16 … 07:25
30m	06:55	07:25	06:55, 07:00, 05, 10, 15, 20, 07:25
1h	07:00	08:00	07:00, 10, 20, 30, 40, 50, 08:00
4h	04:00	08:00	04:00, 04:30 … 08:00
12h	Apr28 20:00	Apr29 08:00	20:00, 21:00 … 08:00
1d	Apr28 08:00	Apr29 08:00	08:00, 10:00 … 08:00
4d	Apr25 08:00	Apr29 08:00	Apr25 08:00, 12:00 …

Live Mode (Auto-Scroll)

When the chart is in **live mode** (`isRealTime = true` , `timeOffset = 0`):

- The right edge is recalculated every second from `Date.now()` .
- When `now` crosses a rounding boundary (e.g. 07:25:00 → 07:25:01) the right edge automatically advances to the next boundary (07:30), and the whole window shifts forward.
- The red dashed **"now" line** marks the exact current time within the window. The small area to the right of the "now" line is the forward tail (at most one step width).

Scroll Behaviour (Left / Right arrows)

Action	Effect
← **Left**	Exits live mode (if active), shifts window back by exactly one timebase period. Each additional click shifts another full period back.
→ **Right**	Shifts window forward by one timebase period. When `timeOffset` returns to 0, live mode is automatically restored.
Disabled for	Custom date range

Zoom Behaviour (Zoom In / Zoom Out)

Zoom changes the active timebase without affecting the live/historical state.

Progression (shortest → longest):

```
TEXT
5m → 10m → 30m → 1h → 4h → 12h → 1d → 4d
```

- **Zoom In (↑):** moves one step left in the progression (finer detail).
- **Zoom Out (↓):** moves one step right (wider view).
- Disabled at the ends of the progression and for Custom ranges.

Reset

Clicking **Reset** sets `timebase = 5m` , `timeOffset = 0` , and restores live mode.

Key Tick Label Rules

- Ticks are placed at clean multiples of the tick step within the window.
- On day-crossing timebases (12h, 1d, 4d), whenever the date changes the label shows `MonDD HH:MM` ; other labels show only `HH:MM` .

- The right-edge tick label is always shown in **green** to distinguish the rounded boundary from data ticks.
- The exact "now" position is marked with a **red dashed vertical line** when live mode is active.

Trend Logs

Historical data collection, analysis, and visualization.

Overview

Trend logs automatically collect and store historical data from inputs, outputs, and variables for analysis, reporting, and troubleshooting.

Trend Features

- **Real-Time Charts**: Live data visualization
- **Historical Playback**: Review past data
- **Multi-Point Comparison**: Plot multiple points
- **Custom Time Ranges**: Flexible date/time selection
- **Data Export**: CSV, Excel, JSON formats
- **Statistics**: Min, max, average, std deviation

Trend Configuration

Log Interval:

- Fast: Every 1 second
- Normal: Every 5 seconds
- Slow: Every 1 minute
- Hourly: Every hour

Data Retention:

- Real-time: 24 hours
- Historical: 1 year
- Archived: 5+ years

Using Trend Logs

1. Navigate to **Trend Logs** page
2. Select points to chart
3. Choose time range
4. View interactive charts
5. Zoom/pan to analyze
6. Export data if needed

Chart Types

- **Line Charts**: Continuous analog data
- **Step Charts**: Digital on/off states
- **Multi-Axis**: Different units on same chart
- **Comparison**: Multiple points overlaid

Analysis Features

- **Cursors**: Precise value reading
- **Statistics Panel**: Min/max/avg display
- **Annotations**: Add notes to charts
- **Alerts**: Highlight alarm conditions

Best Practices

- Log critical points continuously
- Use appropriate intervals
- Archive old data

- Review trends regularly
- Export data before archiving

Next Steps

- Trend Center Unified Design - Single-page trend workflow and phased migration plan
- Inputs - Input monitoring
- Alarms - Alarm analysis
- Monitoring - Real-time monitoring

Trend Logs

Trend Data API

Query Trend Data

TYPESCRIPT

```typescript
// Query raw trend data
const response = await fetch(
  `http://localhost:9103/api/trends?` +
  `deviceId=389001&` +
  `pointId=IN1&` +
  `start=${Date.now() - 86400000}&` +  // Last 24 hours
  `end=${Date.now()}`
);

const trends = await response.json();
// Returns: [{ timestamp, value, quality }, ...]

// Query aggregated data
const aggregated = await fetch(
  `http://localhost:9103/api/trends/aggregate?` +
  `deviceId=389001&` +
  `pointId=IN1&` +
  `start=${start}&` +
  `end=${end}&` +
  `interval=300`  // 5-minute intervals
).then(r => r.json());

// Returns: [{ timestamp, avg, min, max, count }, ...]
```

High-Performance Data Logging

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class TrendDatabase {
  private db: Database.Database;
  private insertStmt: Database.Statement;

  constructor(dbPath: string) {
    this.db = new Database(dbPath);

    // Optimized schema
    this.db.exec(`
      CREATE TABLE IF NOT EXISTS trends (
        timestamp INTEGER NOT NULL,
        device_id INTEGER NOT NULL,
        point_id TEXT NOT NULL,
        value REAL NOT NULL,
        quality TEXT DEFAULT 'good'
      );

      CREATE INDEX IF NOT EXISTS idx_trends_time
        ON trends(timestamp DESC);
      CREATE INDEX IF NOT EXISTS idx_trends_device_point
        ON trends(device_id, point_id, timestamp DESC);
    `);

    // WAL mode for concurrent access
    this.db.pragma('journal_mode = WAL');
    this.db.pragma('synchronous = NORMAL');

    this.insertStmt = this.db.prepare(`
      INSERT INTO trends (timestamp, device_id, point_id, value, quality)
      VALUES (?, ?, ?, ?, ?)
    `);
  }

  logBatch(entries: TrendEntry[]) {
    const transaction = this.db.transaction((entries) => {
      for (const e of entries) {
        this.insertStmt.run(
          e.timestamp,
          e.deviceId,
          e.pointId,
          e.value,
          e.quality || 'good'
        );
      }
    });

    transaction(entries);
  }
}
```

Data Aggregation

```typescript
function aggregateTrends(
  data: TrendPoint[],
  intervalMs: number
): AggregatedTrend[] {
  const buckets = new Map<number, number[]>();

  // Group by time bucket
  for (const point of data) {
    const bucket = Math.floor(point.timestamp / intervalMs) * intervalMs;

    if (!buckets.has(bucket)) {
      buckets.set(bucket, []);
    }

    buckets.get(bucket)!.push(point.value);
  }

  // Calculate statistics for each bucket
  return Array.from(buckets.entries()).map(([timestamp, values]) => ({
    timestamp,
    avg: values.reduce((a, b) => a + b, 0) / values.length,
    min: Math.min(...values),
    max: Math.max(...values),
    count: values.length,
    stdDev: calculateStdDev(values)
  }));
}
```

Next Steps

- REST API
- WebSocket API
- Performance Tuning

Best Practices

Recommended practices for optimal T3000 system operation.

System Design

Network Architecture

- Use dedicated VLAN for BACnet devices
- Assign static IP addresses
- Document network topology
- Implement firewall rules

Device Configuration

- Use consistent naming conventions
- Document all settings
- Export configurations regularly
- Maintain firmware versions

Data Management

- Set appropriate polling rates
- Archive old trend data
- Monitor database size
- Regular backups

Operations

Daily Tasks

- Check alarm status
- Review active overrides
- Monitor system health
- Verify auto-refresh working

Weekly Tasks

- Review trend data
- Check communication logs
- Update schedules as needed
- Test alarm notifications

Monthly Tasks

- Export configuration backups
- Review and optimize settings
- Update documentation
- Check disk space

Security

- Change default passwords
- Limit user access
- Use HTTPS in production
- Keep software updated
- Monitor access logs

Best Practices for T3000 Development

Code Architecture

Modular Design

TYPESCRIPT

```typescript
// ✅ Good: Separation of concerns
class DeviceManager {
  constructor(
    private connection: ConnectionPool,
    private cache: CacheManager,
    private logger: Logger
  ) {}

  async readPoint(deviceId: number, point: string): Promise<number> {
    // Check cache first
    const cached = this.cache.get(`${deviceId}:${point}`);
    if (cached) return cached;

    // Read from device
    const value = await this.connection.read(deviceId, point);

    // Update cache
    this.cache.set(`${deviceId}:${point}`, value, 10000);

    return value;
  }
}

// ❌ Bad: Tightly coupled
class DeviceManager {
  async readPoint(deviceId: number, point: string): Promise<number> {
    const cachedValue = localStorage.getItem(`${deviceId}:${point}`);
    if (cachedValue) return parseFloat(cachedValue);

    const response = await fetch(`http://localhost:9103/api/devices/${deviceId}/points/${point}`);
    const value = (await response.json()).value;

    localStorage.setItem(`${deviceId}:${point}`, value.toString());
    return value;
  }
}
```

Dependency Injection

TYPESCRIPT

```typescript
// Service container
class ServiceContainer {
  private services = new Map<string, any>();

  register<T>(name: string, factory: () => T): void {
    this.services.set(name, factory);
  }

  get<T>(name: string): T {
    const factory = this.services.get(name);
    if (!factory) throw new Error(`Service ${name} not registered`);
    return factory();
  }
}

const container = new ServiceContainer();

// Register services
container.register('logger', () => new ConsoleLogger());
container.register('cache', () => new MemoryCache());
container.register('connection', () =>
  new ConnectionPool(container.get('logger'))
);

// Use services
const deviceManager = new DeviceManager(
  container.get('connection'),
  container.get('cache'),
  container.get('logger')
);
```

Error Handling

Comprehensive Error Types

TYPESCRIPT

```typescript
enum ErrorCode {
  NETWORK_ERROR = 'NETWORK_ERROR',
  DEVICE_OFFLINE = 'DEVICE_OFFLINE',
  TIMEOUT = 'TIMEOUT',
  INVALID_DATA = 'INVALID_DATA',
  PERMISSION_DENIED = 'PERMISSION_DENIED'
}

class T3000Error extends Error {
  constructor(
    public code: ErrorCode,
    message: string,
    public details?: any
  ) {
    super(message);
    this.name = 'T3000Error';
  }
}

// Usage
async function readDevicePoint(deviceId: number, point: string): Promise<number> {
  try {
    const response = await fetch(`/api/devices/${deviceId}/points/${point}`);

    if (!response.ok) {
      if (response.status === 404) {
        throw new T3000Error(
          ErrorCode.DEVICE_OFFLINE,
          `Device ${deviceId} not found`,
          { deviceId }
        );
      }
      throw new T3000Error(
        ErrorCode.NETWORK_ERROR,
        `HTTP ${response.status}: ${response.statusText}`
      );
    }

    return (await response.json()).value;
  } catch (error) {
    if (error instanceof T3000Error) throw error;

    throw new T3000Error(
      ErrorCode.NETWORK_ERROR,
      'Failed to read device point',
      { originalError: error }
    );
  }
}
```

Retry Strategies

TYPESCRIPT

```typescript
interface RetryConfig {
  maxRetries: number;
  initialDelay: number;
  maxDelay: number;
  backoffFactor: number;
}

async function withRetry<T>(
  operation: () => Promise<T>,
  config: RetryConfig = {
    maxRetries: 3,
    initialDelay: 100,
    maxDelay: 5000,
    backoffFactor: 2
  }
): Promise<T> {
  let lastError: Error;
  let delay = config.initialDelay;

  for (let attempt = 0; attempt <= config.maxRetries; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error as Error;

      // Don't retry on certain errors
      if (error instanceof T3000Error &&
          error.code === ErrorCode.PERMISSION_DENIED) {
        throw error;
      }

      if (attempt < config.maxRetries) {
        await new Promise(resolve => setTimeout(resolve, delay));
        delay = Math.min(delay * config.backoffFactor, config.maxDelay);
      }
    }
  }

  throw lastError!;
}
```

Performance Best Practices

Efficient Data Fetching

TYPESCRIPT

```typescript
// ✅ Good: Batch requests
async function readMultiplePoints(
  deviceId: number,
  points: string[]
): Promise<Map<string, number>> {
  const response = await fetch(
    `/api/devices/${deviceId}/points?points=${points.join(',')}`
  );

  const data = await response.json();
  return new Map(Object.entries(data));
}

// ❌ Bad: Individual requests
async function readMultiplePoints(
  deviceId: number,
  points: string[]
): Promise<Map<string, number>> {
  const values = new Map<string, number>();

  for (const point of points) {
    const response = await fetch(
      `/api/devices/${deviceId}/points/${point}`
    );
    const data = await response.json();
    values.set(point, data.value);
  }

  return values;
}
```

Memoization

TYPESCRIPT

```typescript
function memoize<T extends (...args: any[]) => any>(
  fn: T,
  ttl: number = 5000
): T {
  const cache = new Map<string, { value: any; expires: number }>();

  return ((...args: any[]) => {
    const key = JSON.stringify(args);
    const cached = cache.get(key);

    if (cached && cached.expires > Date.now()) {
      return cached.value;
    }

    const value = fn(...args);
    cache.set(key, { value, expires: Date.now() + ttl });

    return value;
  }) as T;
}

// Usage
const getDeviceInfo = memoize(
  async (deviceId: number) => {
    const response = await fetch(`/api/devices/${deviceId}`);
    return response.json();
  },
  60000  // Cache for 1 minute
);
```

Testing

Unit Testing

TYPESCRIPT

```typescript
import { describe, it, expect, vi } from 'vitest';

describe('DeviceManager', () => {
  it('should read point from cache if available', async () => {
    const mockConnection = {
      read: vi.fn()
    };

    const mockCache = {
      get: vi.fn(() => 72.5),
      set: vi.fn()
    };

    const manager = new DeviceManager(
      mockConnection as any,
      mockCache as any,
      console
    );

    const value = await manager.readPoint(389001, 'IN1');

    expect(value).toBe(72.5);
    expect(mockCache.get).toHaveBeenCalledWith('389001:IN1');
    expect(mockConnection.read).not.toHaveBeenCalled();
  });

  it('should fetch from device and update cache on miss', async () => {
    const mockConnection = {
      read: vi.fn(() => Promise.resolve(75.0))
    };

    const mockCache = {
      get: vi.fn(() => null),
      set: vi.fn()
    };

    const manager = new DeviceManager(
      mockConnection as any,
      mockCache as any,
      console
    );

    const value = await manager.readPoint(389001, 'IN1');

    expect(value).toBe(75.0);
    expect(mockConnection.read).toHaveBeenCalledWith(389001, 'IN1');
    expect(mockCache.set).toHaveBeenCalledWith('389001:IN1', 75.0, 10000);
  });
});
```

Integration Testing

```typescript
import { test, expect } from 'vitest';

test('full device data flow', async () => {
  // Start test server
  const server = await startTestServer();

  // Add test device
  await server.addDevice({
    id: 389001,
    name: 'Test Device',
    ip: '127.0.0.1'
  });

  // Read point
  const client = new T3000Client('http://localhost:9103');
  const value = await client.readPoint(389001, 'IN1');

  expect(value).toBeGreaterThanOrEqual(0);

  await server.stop();
});
```

Security Best Practices

Input Validation

```typescript
import { z } from 'zod';

// Define schema
const WritePointSchema = z.object({
  deviceId: z.number().int().positive(),
  point: z.string().regex(/^(IN|OUT|VAR)\d{1,2}$/),
  value: z.number().min(-100).max(200),
  priority: z.number().int().min(1).max(16).optional()
});

// Validate input
app.post('/api/write-point', async (req, res) => {
  try {
    const data = WritePointSchema.parse(req.body);

    await writePoint(data.deviceId, data.point, data.value, data.priority);

    res.json({ success: true });
  } catch (error) {
    if (error instanceof z.ZodError) {
      res.status(400).json({ error: error.errors });
    } else {
      res.status(500).json({ error: 'Internal error' });
    }
  }
});
```

Authentication & Authorization

TYPESCRIPT

```typescript
import jwt from 'jsonwebtoken';

interface UserPayload {
  userId: string;
  role: 'admin' | 'operator' | 'viewer';
}

function authenticateToken(req: Request, res: Response, next: NextFunction) {
  const token = req.headers.authorization?.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!) as UserPayload;
    req.user = payload;
    next();
  } catch (error) {
    res.status(403).json({ error: 'Invalid token' });
  }
}

function requireRole(role: string) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (req.user?.role !== role && req.user?.role !== 'admin') {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }
    next();
  };
}

// Usage
app.post('/api/write-point',
  authenticateToken,
  requireRole('operator'),
  async (req, res) => {
    // Handle write
  }
);
```

Documentation

Code Comments

TYPESCRIPT

```typescript
/**
 * Reads a point value from a device with automatic retry and caching
 *
 * @param deviceId - The unique device identifier
 * @param point - Point name (e.g., 'IN1', 'OUT5', 'VAR10')
 * @param options - Optional configuration
 * @returns The current point value
 * @throws {T3000Error} If device is offline or read fails after retries
 *
 * @example
 * ```typescript
 * const temp = await readPoint(389001, 'IN1');
 * console.log(`Temperature: ${temp}°F`);
 * ```
 */
async function readPoint(
  deviceId: number,
  point: string,
  options?: ReadOptions
): Promise<number> {
  // Implementation
}
```

Next Steps

- FAQ
- Troubleshooting
- Performance Tuning

Frequently Asked Questions

General

Q: What browsers are supported? A: Chrome 90+, Edge 90+, Firefox 88+, Safari 14+. IE not supported.

Q: Do I need to install anything? A: Just a modern web browser. The API server runs on localhost.

Q: Can I access remotely? A: Yes, configure firewall to allow port 3003 (UI) and 9103 (API).

Connections

Q: Device not discovered? A: Check network connectivity, firewall settings, and device IP. Try manual connection.

Q: Connection keeps dropping? A: Increase timeout settings, check network quality, verify no IP conflicts.

Data

Q: Why is data not updating? A: Verify device online, check auto-refresh enabled, try manual refresh.

Q: How long is data stored? A: 24 hours real-time, 1 year historical. Archive for longer retention.

Q: Can I export data? A: Yes, from Trend Logs page - CSV, Excel, or JSON format.

Performance

Q: Page loading slow? A: Reduce displayed points, increase refresh interval, close unused tabs.

Q: Database locked errors? A: System automatically retries up to 51 seconds. Normal during high load.

Alarms

Q: Alarms not triggering? A: Check alarm enabled, verify thresholds, test notification settings.

Q: Too many false alarms? A: Adjust thresholds, add deadband, increase time delay, improve filtering.

Troubleshooting

Q: Where are log files? A: Browser console (F12) for UI errors, API logs in api/logs/ folder.

Q: How to reset system? A: Clear browser cache, restart API server, reload page.

Q: Who to contact for help? A: Check documentation first, then contact support with error details.

Developer FAQ

Architecture

Q: What's the technology stack? A:

- **Frontend**: Vue 3 + Quasar + TypeScript
- **Backend**: Rust + Axum + SQLite
- **Protocol**: BACnet, Modbus TCP/RTU
- **Communication**: REST API, WebSocket

Q: How is the project structured? A:

```text
T3000Webview9/
├── src/              # Vue 3 frontend
│   ├── t3-vue/       # Main application
│   └── shared/       # Shared utilities
├── api/              # Rust backend
│   ├── src/          # API implementation
│   └── migration/    # Database migrations
└── docs/             # Documentation
```

API

Q: How do I authenticate API requests? A: Use JWT tokens:

```typescript
const response = await fetch('/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: 'admin', password: 'pass' })
});
const { token } = await response.json();

// Use token in subsequent requests
fetch('/api/devices', {
  headers: { 'Authorization': `Bearer ${token}` }
});
```

Q: What's the rate limit for API calls? A: No hard rate limit, but recommended:

- Point reads: Max 10/second per device
- WebSocket subscriptions: Max 100 points
- Batch operations: Max 1000 items

Q: How do I handle API errors? A:

```typescript
try {
  const response = await fetch('/api/devices/389001');

  if (!response.ok) {
    const error = await response.json();
    console.error('API error:', error.message);

    switch (response.status) {
      case 404:
        // Device not found
        break;
      case 500:
        // Server error - retry
        break;
    }
  }

  const data = await response.json();
} catch (error) {
  // Network error
  console.error('Network error:', error);
}
```

Database

Q: What database is used? A: SQLite with WAL (Write-Ahead Logging) mode for concurrent access.

Q: How do I access the database directly? A:

```shell
sqlite3 api/t3000.db

## View devices
SELECT * FROM devices;

## Query trend data
SELECT * FROM trend_logs
WHERE device_id = 389001
  AND point = 'IN1'
  AND timestamp > strftime('%s', 'now', '-1 day');
```

Q: What's the database schema? A: Key tables:

- `devices` - Device information
- `points` - Point configurations
- `trend_logs` - Historical data
- `alarms` - Alarm definitions
- `schedules` - Schedule objects
- `users` - User accounts

BACnet/Modbus

Q: How do I add a custom BACnet object? A: Currently not supported via UI. Requires modifying Rust backend:

```rust
// In api/src/t3_device/bacnet.rs
impl BacnetDevice {
    pub fn read_custom_object(
        &self,
        object_type: ObjectType,
        instance: u32,
        property: PropertyIdentifier
    ) -> Result<Value> {
        // Implementation
    }
}
```

Q: Can I use Modbus RTU over USB? A: Yes, configure serial port in device settings:

```typescript
const device = await createDevice({
  protocol: 'modbus-rtu',
  port: '/dev/ttyUSB0',  // or COM3 on Windows
  baudrate: 19200,
  parity: 'none',
  stopbits: 1,
  slaveId: 1
});
```

Q: What BACnet services are supported? A:

- ReadProperty
- WriteProperty
- ReadPropertyMultiple
- SubscribeCOV
- I-Am/Who-Is
- DeviceCommunicationControl

WebSocket

Q: How do I subscribe to real-time updates? A:

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1'],
    interval: 5000  // Update every 5 seconds
  }));
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);

  if (message.type === 'point-update') {
    console.log(`${message.point}: ${message.value}`);
  }
};
```

Q: How many WebSocket connections can I have? A: No hard limit, but recommend:

- Max 10 connections per client
- Max 100 subscriptions per connection
- Use single connection with multiple subscriptions

Performance

Q: How do I optimize large data queries? A:

```typescript
// ✅ Good: Use pagination
const page1 = await fetch(
  '/api/trend-logs?device=389001&point=IN1&limit=1000&offset=0'
);
const page2 = await fetch(
  '/api/trend-logs?device=389001&point=IN1&limit=1000&offset=1000'
);

// ✅ Good: Use time-based filtering
const recent = await fetch(
  '/api/trend-logs?device=389001&start=' + (Date.now() - 86400000)
);

// ❌ Bad: Fetch everything
const all = await fetch('/api/trend-logs');
```

Q: Why is my React component re-rendering too often? A: Use memoization:

```typescript
import { useMemo } from 'react';

const DeviceChart = ({ deviceId, points }) => {
  const data = useMemo(
    () => processChartData(points),
    [points]  // Only recompute when points change
  );

  return <Chart data={data} />;
};
```

Debugging

Q: How do I enable debug logging? A:

```shell
## Frontend (browser console)
localStorage.setItem('debug', 't3000:*');

## Backend (environment variable)
RUST_LOG=debug cargo run
```

Q: How do I inspect WebSocket traffic? A: Use browser DevTools:

1. Open DevTools (F12)
2. Network tab
3. Filter by "WS"
4. Click connection to see messages

Q: Where are crash dumps stored? A:

- **Frontend**: Browser console errors
- **Backend**: `api/logs/error.log`
- **Panic dumps**: `api/crash-dumps/`

Deployment

Q: How do I build for production? A:

```shell
## Frontend
npm run build
## Output: dist/

## Backend
cd api
cargo build --release
## Output: target/release/t3000-api.exe
```

Q: Can I run on Linux? A: Yes, cross-compile:

```shell
rustup target add x86_64-unknown-linux-gnu
cargo build --release --target x86_64-unknown-linux-gnu
```

Q: How do I configure HTTPS? A: Use reverse proxy (nginx):

NGINX

```
server {
    listen 443 ssl;
    server_name t3000.example.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    location / {
        proxy_pass http://localhost:3003;
    }

    location /api {
        proxy_pass http://localhost:9103;
    }

    location /ws {
        proxy_pass http://localhost:9103;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Next Steps

- Best Practices
- Troubleshooting
- Performance Tuning

Performance Tuning

Optimize T3000 system performance and responsiveness.

Database Optimization

Retry Settings

Current configuration:

- Max retries: 10
- Base delay: 100ms
- Exponential backoff
- Total wait: ~51 seconds

Data Retention

- Real-time data: 24 hours
- Historical data: 1 year
- Archive older data to reduce database size

Network Tuning

Polling Rates

Recommended:

- Critical points: 5 seconds
- Normal points: 10 seconds
- Slow-changing: 60 seconds

Concurrent Requests

- Limit: 10 concurrent requests
- Queue additional requests
- Batch operations when possible

Browser Optimization

Chart Performance

- Limit charts to 10 maximum
- Use appropriate time ranges
- Reduce displayed points
- Close unused tabs

Memory Management

- Clear cache periodically
- Reload page if sluggish
- Use latest browser version

Server Optimization

API Server

- Monitor CPU/memory usage
- Restart if memory leak
- Use release build (not debug)
- Check logs for errors

Database

- Vacuum SQLite regularly
- Optimize indexes
- Archive old data
- Monitor file size

Advanced Performance Tuning

Database Optimization

SQLite Performance Tuning

SQL

```sql
-- Enable Write-Ahead Logging (WAL)
PRAGMA journal_mode = WAL;

-- Increase cache size (in pages, 1 page = 4KB)
PRAGMA cache_size = -64000;  -- 64MB cache

-- Use memory-mapped I/O
PRAGMA mmap_size = 268435456;  -- 256MB

-- Optimize synchronization
PRAGMA synchronous = NORMAL;  -- Faster than FULL, still safe

-- Set optimal page size
PRAGMA page_size = 4096;

-- Enable auto-vacuum
PRAGMA auto_vacuum = INCREMENTAL;
```

Query Optimization

TYPESCRIPT

```typescript
// ✅ Good: Use indexes
// Create index on frequently queried columns
await db.exec(`
  CREATE INDEX IF NOT EXISTS idx_trend_logs_device_point_time
  ON trend_logs(device_id, point, timestamp DESC)
`);

// Query will use index
const data = await db.all(`
  SELECT * FROM trend_logs
  WHERE device_id = ? AND point = ? AND timestamp > ?
  ORDER BY timestamp DESC
  LIMIT 1000
`, [deviceId, point, startTime]);

// ❌ Bad: Full table scan
const data = await db.all(`
  SELECT * FROM trend_logs
  WHERE CAST(device_id AS TEXT) = '${deviceId}'
`);
```

Connection Pooling

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

class DatabasePool {
  private pool: Database.Database[] = [];
  private available: Database.Database[] = [];
  private readonly maxConnections: number;

  constructor(dbPath: string, maxConnections = 5) {
    this.maxConnections = maxConnections;

    for (let i = 0; i < maxConnections; i++) {
      const db = new Database(dbPath);
      db.pragma('journal_mode = WAL');
      db.pragma('cache_size = -64000');
      this.pool.push(db);
      this.available.push(db);
    }
  }

  async acquire(): Promise<Database.Database> {
    while (this.available.length === 0) {
      await new Promise(resolve => setTimeout(resolve, 10));
    }
    return this.available.pop()!;
  }

  release(db: Database.Database): void {
    this.available.push(db);
  }

  async withConnection<T>(
    fn: (db: Database.Database) => Promise<T>
  ): Promise<T> {
    const db = await this.acquire();
    try {
      return await fn(db);
    } finally {
      this.release(db);
    }
  }
}
```

Batch Operations

TYPESCRIPT

```typescript
// ✅ Good: Batch insert
async function batchInsertTrendData(
  data: TrendData[]
): Promise<void> {
  const db = await dbPool.acquire();

  const insert = db.prepare(`
    INSERT INTO trend_logs (device_id, point, timestamp, value)
    VALUES (?, ?, ?, ?)
  `);

  const transaction = db.transaction((records: TrendData[]) => {
    for (const record of records) {
      insert.run(record.deviceId, record.point, record.timestamp, record.value);
    }
  });

  transaction(data);
  dbPool.release(db);
}

// Process 1000 records in one transaction
await batchInsertTrendData(records);  // ~10ms

// ❌ Bad: Individual inserts
for (const record of records) {
  await db.run(
    'INSERT INTO trend_logs VALUES (?, ?, ?, ?)',
    [record.deviceId, record.point, record.timestamp, record.value]
  );
}  // ~1000ms for 1000 records
```

Caching Strategies

Multi-Level Cache

TYPESCRIPT

```typescript
class MultiLevelCache {
  private l1Cache = new Map<string, CacheEntry>();  // Memory
  private l2Cache: LocalForage;  // IndexedDB
  private maxL1Size = 1000;

  async get(key: string): Promise<any | null> {
    // Check L1 (memory)
    const l1Entry = this.l1Cache.get(key);
    if (l1Entry && l1Entry.expires > Date.now()) {
      return l1Entry.value;
    }

    // Check L2 (IndexedDB)
    const l2Entry = await this.l2Cache.getItem<CacheEntry>(key);
    if (l2Entry && l2Entry.expires > Date.now()) {
      // Promote to L1
      this.l1Cache.set(key, l2Entry);
      return l2Entry.value;
    }

    return null;
  }

  async set(key: string, value: any, ttl: number): Promise<void> {
    const entry: CacheEntry = {
      value,
      expires: Date.now() + ttl
    };

    // Store in L1
    this.l1Cache.set(key, entry);

    // Evict if L1 too large
    if (this.l1Cache.size > this.maxL1Size) {
      const firstKey = this.l1Cache.keys().next().value;
      this.l1Cache.delete(firstKey);
    }

    // Store in L2
    await this.l2Cache.setItem(key, entry);
  }
}
```

Cache Warming

TYPESCRIPT

```typescript
class CacheWarmer {
  async warmDeviceCache(deviceIds: number[]): Promise<void> {
    const promises = deviceIds.map(async (deviceId) => {
      // Fetch and cache device info
      const device = await fetch(`/api/devices/${deviceId}`).then(r => r.json());
      cache.set(`device:${deviceId}`, device, 300000);  // 5 min

      // Fetch and cache points
      const points = await fetch(`/api/devices/${deviceId}/points`).then(r => r.json());
      cache.set(`points:${deviceId}`, points, 300000);
    });

    await Promise.all(promises);
  }
}
```

Network Optimization

Request Deduplication

TYPESCRIPT

```typescript
class RequestDeduplicator {
  private pending = new Map<string, Promise<any>>();

  async fetch<T>(key: string, fetcher: () => Promise<T>): Promise<T> {
    // Return existing request if pending
    if (this.pending.has(key)) {
      return this.pending.get(key)!;
    }

    // Create new request
    const promise = fetcher().finally(() => {
      this.pending.delete(key);
    });

    this.pending.set(key, promise);
    return promise;
  }
}

const deduper = new RequestDeduplicator();

// Multiple calls will only trigger one fetch
const [data1, data2, data3] = await Promise.all([
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json())),
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json())),
  deduper.fetch('device:389001', () => fetch('/api/devices/389001').then(r => r.json()))
]);  // Only 1 network request made
```

Request Batching

TYPESCRIPT

```typescript
class RequestBatcher {
  private queue: BatchRequest[] = [];
  private timer: NodeJS.Timeout | null = null;
  private batchSize = 10;
  private batchDelay = 50;  // ms

  async add(request: BatchRequest): Promise<any> {
    return new Promise((resolve, reject) => {
      this.queue.push({ ...request, resolve, reject });

      if (this.queue.length >= this.batchSize) {
        this.flush();
      } else if (!this.timer) {
        this.timer = setTimeout(() => this.flush(), this.batchDelay);
      }
    });
  }

  private async flush(): Promise<void> {
    if (this.timer) {
      clearTimeout(this.timer);
      this.timer = null;
    }

    if (this.queue.length === 0) return;

    const batch = this.queue.splice(0);

    try {
      const response = await fetch('/api/batch', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          requests: batch.map(r => ({ endpoint: r.endpoint, params: r.params }))
        })
      });

      const results = await response.json();

      batch.forEach((req, i) => {
        if (results[i].error) {
          req.reject(new Error(results[i].error));
        } else {
          req.resolve(results[i].data);
        }
      });
    } catch (error) {
      batch.forEach(req => req.reject(error));
    }
  }
}
```

HTTP/2 Server Push

TYPESCRIPT

```typescript
// Backend (Express + SPDY)
import spdy from 'spdy';

app.get('/devices/:id', (req, res) => {
  // Push related resources
  const stream = res.push('/api/devices/' + req.params.id + '/points', {
    request: { accept: 'application/json' },
    response: { 'content-type': 'application/json' }
  });

  stream.end(JSON.stringify(pointsData));

  res.json(deviceData);
});
```

Frontend Optimization

Virtual Scrolling

VUE

```vue
<template>
  <virtual-scroller
    :items="devices"
    :item-height="50"
    :buffer="200"
  >
    <template #default="{ item }">
      <DeviceRow :device="item" />
    </template>
  </virtual-scroller>
</template>

<script setup lang="ts">
// Only renders visible items + buffer
// Handles 10,000+ items smoothly
</script>
```

Lazy Loading Components

TYPESCRIPT

```typescript
import { defineAsyncComponent } from 'vue';

const HeavyChart = defineAsyncComponent(() =>
  import('./components/HeavyChart.vue')
);

const DeviceDetails = defineAsyncComponent({
  loader: () => import('./components/DeviceDetails.vue'),
  loadingComponent: LoadingSpinner,
  delay: 200,  // Show loading after 200ms
  timeout: 10000  // Error after 10s
});
```

Debouncing and Throttling

TYPESCRIPT

```typescript
import { debounce, throttle } from 'lodash-es';

// Debounce: Wait for user to stop typing
const searchDevices = debounce(async (query: string) => {
  const results = await fetch(`/api/devices/search?q=${query}`);
  // ...
}, 300);

// Throttle: Limit execution rate
const updateChart = throttle((data: number[]) => {
  chart.setData(data);
}, 1000);  // Max once per second
```

Memory Management

Object Pooling

TYPESCRIPT

```typescript
class ObjectPool<T> {
  private available: T[] = [];
  private factory: () => T;
  private reset: (obj: T) => void;

  constructor(factory: () => T, reset: (obj: T) => void, initialSize = 10) {
    this.factory = factory;
    this.reset = reset;

    for (let i = 0; i < initialSize; i++) {
      this.available.push(factory());
    }
  }

  acquire(): T {
    return this.available.pop() || this.factory();
  }

  release(obj: T): void {
    this.reset(obj);
    this.available.push(obj);
  }
}

// Usage: Reuse objects instead of creating new ones
const pointDataPool = new ObjectPool<PointData>(
  () => ({ deviceId: 0, point: '', value: 0, timestamp: 0 }),
  (obj) => { obj.deviceId = 0; obj.point = ''; obj.value = 0; obj.timestamp = 0; }
);
```

Garbage Collection Hints

```typescript
class LargeDataProcessor {
  private cache = new WeakMap<object, ProcessedData>();

  process(rawData: object): ProcessedData {
    // Check cache
    const cached = this.cache.get(rawData);
    if (cached) return cached;

    // Process and cache
    const processed = this.heavyProcessing(rawData);
    this.cache.set(rawData, processed);  // Auto-GC'd when rawData is GC'd

    return processed;
  }
}
```

Monitoring and Profiling

Performance Metrics

```typescript
class PerformanceMetrics {
  recordMetric(name: string, value: number, tags?: Record<string, string>) {
    // Send to monitoring service
    fetch('/api/metrics', {
      method: 'POST',
      body: JSON.stringify({ name, value, tags, timestamp: Date.now() })
    });
  }

  measureAsync<T>(name: string, fn: () => Promise<T>): Promise<T> {
    const start = performance.now();
    return fn().finally(() => {
      this.recordMetric(name, performance.now() - start);
    });
  }
}

// Usage
await metrics.measureAsync('fetch-device-data', () =>
  fetch('/api/devices/389001').then(r => r.json())
);
```

Next Steps

- Best Practices
- Troubleshooting
- FAQ

Troubleshooting Guide

Quick reference for common issues and solutions.

Quick Checklist

System Not Responding

1. Check API server running (port 9103)
2. Verify database accessible
3. Check browser console (F12)
4. Restart API server
5. Clear browser cache

Data Not Updating

1. Verify device connection (green status)
2. Check auto-refresh enabled
3. Click manual refresh
4. Review communication logs
5. Check network connectivity

Slow Performance

1. Reduce displayed points
2. Increase refresh interval
3. Close unused browser tabs
4. Check system resources
5. Archive old data

Error Messages

Database Locked

Solution: Wait for retry (up to 51s), reduce concurrent operations

Connection Timeout

Solution: Increase timeout, check network, verify device online

JSON Parse Error

Solution: API endpoint missing or returning HTML error page

Getting Help

See Device Troubleshooting for detailed solutions.

Advanced Troubleshooting

Diagnostic Tools

Health Check Endpoint

TYPESCRIPT

```typescript
interface HealthCheckResponse {
  status: 'healthy' | 'degraded' | 'unhealthy';
  uptime: number;
  database: {
    connected: boolean;
    responseTime: number;
  };
  devices: {
    total: number;
    online: number;
    offline: number;
  };
  memory: {
    used: number;
    total: number;
  };
}

// Check system health
const health = await fetch('/api/health');
const status: HealthCheckResponse = await health.json();

if (status.database.responseTime > 1000) {
  console.warn('Database slow, may need optimization');
}
```

Log Analysis

TYPESCRIPT

```typescript
// Fetch recent errors from API
const logs = await fetch('/api/logs?level=error&limit=100');
const errors = await logs.json();

// Analyze error patterns
const errorCounts = errors.reduce((acc, log) => {
  const key = log.message.split(':')[0];
  acc[key] = (acc[key] || 0) + 1;
  return acc;
}, {});

console.table(errorCounts);
```

Network Diagnostics

TYPESCRIPT

```typescript
class NetworkDiagnostics {
  async ping(host: string, port: number): Promise<boolean> {
    try {
      const response = await fetch(
        `http://${host}:${port}/api/ping`,
        { signal: AbortSignal.timeout(5000) }
      );
      return response.ok;
    } catch {
      return false;
    }
  }

  async traceroute(deviceId: number): Promise<DiagnosticResult> {
    const start = performance.now();

    try {
      const response = await fetch(`/api/devices/${deviceId}/ping`);
      const latency = performance.now() - start;

      return {
        success: response.ok,
        latency,
        hops: []  // Would require backend support
      };
    } catch (error) {
      return {
        success: false,
        error: error.message,
        latency: performance.now() - start
      };
    }
  }

  async checkBandwidth(): Promise<number> {
    const start = performance.now();
    const testSize = 1024 * 1024;  // 1 MB

    const response = await fetch('/api/bandwidth-test');
    await response.arrayBuffer();

    const duration = (performance.now() - start) / 1000;
    return (testSize / duration) / 1024;  // KB/s
  }
}
```

Common Issues and Fixes

Database Lock Debugging

TYPESCRIPT

```typescript
// Monitor database lock events
class DatabaseMonitor {
  private lockEvents: LockEvent[] = [];

  logLock(operation: string, duration: number) {
    this.lockEvents.push({
      operation,
      duration,
      timestamp: Date.now()
    });

    // Alert on long locks
    if (duration > 5000) {
      console.error(`Long database lock: ${operation} took ${duration}ms`);
    }
  }

  getStatistics() {
    const avg = this.lockEvents.reduce((sum, e) => sum + e.duration, 0)
                / this.lockEvents.length;
    const max = Math.max(...this.lockEvents.map(e => e.duration));

    return { avg, max, count: this.lockEvents.length };
  }
}

// Use wrapper to track locks
async function withLockMonitoring<T>(
  operation: string,
  fn: () => Promise<T>
): Promise<T> {
  const start = Date.now();
  try {
    return await fn();
  } finally {
    dbMonitor.logLock(operation, Date.now() - start);
  }
}
```

Memory Leak Detection

TYPESCRIPT

```typescript
class MemoryMonitor {
  private samples: MemorySample[] = [];

  startMonitoring(intervalMs = 10000) {
    setInterval(() => {
      if (performance.memory) {
        this.samples.push({
          timestamp: Date.now(),
          used: performance.memory.usedJSHeapSize,
          total: performance.memory.totalJSHeapSize,
          limit: performance.memory.jsHeapSizeLimit
        });

        // Keep last 100 samples
        if (this.samples.length > 100) {
          this.samples.shift();
        }

        this.checkForLeaks();
      }
    }, intervalMs);
  }

  private checkForLeaks() {
    if (this.samples.length < 10) return;

    const recent = this.samples.slice(-10);
    const older = this.samples.slice(-20, -10);

    const recentAvg = recent.reduce((sum, s) => sum + s.used, 0) / recent.length;
    const olderAvg = older.reduce((sum, s) => sum + s.used, 0) / older.length;

    const growthRate = (recentAvg - olderAvg) / olderAvg;

    if (growthRate > 0.1) {  // 10% growth
      console.warn(`Possible memory leak detected: ${(growthRate * 100).toFixed(1)}% growth`);
    }
  }
}
```

WebSocket Connection Issues

TYPESCRIPT

```typescript
class WebSocketDiagnostics {
  private reconnectAttempts = 0;
  private connectionErrors: Error[] = [];

  diagnose(ws: WebSocket): DiagnosticReport {
    const report: DiagnosticReport = {
      state: ws.readyState,
      reconnectAttempts: this.reconnectAttempts,
      recentErrors: this.connectionErrors.slice(-5),
      recommendations: []
    };

    if (this.reconnectAttempts > 5) {
      report.recommendations.push(
        'Excessive reconnection attempts. Check server availability.'
      );
    }

    const errorTypes = this.connectionErrors.map(e => e.message);
    if (errorTypes.filter(m => m.includes('timeout')).length > 3) {
      report.recommendations.push(
        'Frequent timeouts detected. Check network latency.'
      );
    }

    return report;
  }

  logError(error: Error) {
    this.connectionErrors.push(error);
    if (this.connectionErrors.length > 50) {
      this.connectionErrors.shift();
    }
  }
}
```

Performance Profiling

API Request Profiler

TYPESCRIPT

```typescript
class APIProfiler {
  private requests = new Map<string, RequestMetrics>();

  async profile<T>(
    endpoint: string,
    request: () => Promise<T>
  ): Promise<T> {
    const start = performance.now();

    try {
      const result = await request();
      this.recordSuccess(endpoint, performance.now() - start);
      return result;
    } catch (error) {
      this.recordFailure(endpoint, performance.now() - start);
      throw error;
    }
  }

  private recordSuccess(endpoint: string, duration: number) {
    const metrics = this.getMetrics(endpoint);
    metrics.totalRequests++;
    metrics.successfulRequests++;
    metrics.totalDuration += duration;
    metrics.avgDuration = metrics.totalDuration / metrics.successfulRequests;

    if (duration > metrics.maxDuration) {
      metrics.maxDuration = duration;
    }
    if (duration < metrics.minDuration) {
      metrics.minDuration = duration;
    }
  }

  private recordFailure(endpoint: string, duration: number) {
    const metrics = this.getMetrics(endpoint);
    metrics.totalRequests++;
    metrics.failedRequests++;
  }

  getReport(): ProfileReport {
    const report: ProfileReport = [];

    for (const [endpoint, metrics] of this.requests) {
      report.push({
        endpoint,
        ...metrics,
        successRate: metrics.successfulRequests / metrics.totalRequests
      });
    }

    return report.sort((a, b) => b.avgDuration - a.avgDuration);
  }
}
```

```
// Usage
const profiler = new APIProfiler();

const data = await profiler.profile(
  '/api/devices/389001',
  () => fetch('/api/devices/389001').then(r => r.json())
);

console.table(profiler.getReport());
```

Component Render Profiler

TYPESCRIPT

```
import { onMounted, onUpdated } from 'vue';

export function useRenderProfiler(componentName: string) {
  let renderCount = 0;
  let lastRender = Date.now();

  onMounted(() => {
    console.log(`[${componentName}] Mounted`);
  });

  onUpdated(() => {
    renderCount++;
    const now = Date.now();
    const timeSinceLastRender = now - lastRender;

    console.log(
      `[${componentName}] Render #${renderCount} ` +
      `(${timeSinceLastRender}ms since last)`
    );

    if (timeSinceLastRender < 16) {  // Less than 1 frame (60fps)
      console.warn(
        `[${componentName}] Rendering too frequently! ` +
        `Consider memoization or debouncing.`
      );
    }

    lastRender = now;
  });
}
```

Automated Recovery

Circuit Breaker Pattern

TYPESCRIPT

```typescript
enum CircuitState {
  CLOSED,
  OPEN,
  HALF_OPEN
}

class CircuitBreaker {
  private state = CircuitState.CLOSED;
  private failureCount = 0;
  private lastFailureTime = 0;
  private readonly threshold = 5;
  private readonly timeout = 60000;  // 1 minute

  async execute<T>(operation: () => Promise<T>): Promise<T> {
    if (this.state === CircuitState.OPEN) {
      if (Date.now() - this.lastFailureTime > this.timeout) {
        this.state = CircuitState.HALF_OPEN;
      } else {
        throw new Error('Circuit breaker is OPEN');
      }
    }

    try {
      const result = await operation();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  private onSuccess() {
    this.failureCount = 0;
    this.state = CircuitState.CLOSED;
  }

  private onFailure() {
    this.failureCount++;
    this.lastFailureTime = Date.now();

    if (this.failureCount >= this.threshold) {
      this.state = CircuitState.OPEN;
      console.error('Circuit breaker tripped to OPEN state');
    }
  }
}
```

Self-Healing

TYPESCRIPT

```typescript
class SelfHealingSystem {
  private healthChecks: HealthCheck[] = [];

  registerHealthCheck(check: HealthCheck) {
    this.healthChecks.push(check);
  }

  async monitorAndHeal(intervalMs = 30000) {
    setInterval(async () => {
      for (const check of this.healthChecks) {
        const healthy = await check.check();

        if (!healthy) {
          console.warn(`Health check failed: ${check.name}`);

          if (check.autoHeal) {
            try {
              await check.heal();
              console.log(`Auto-healed: ${check.name}`);
            } catch (error) {
              console.error(`Failed to heal ${check.name}:`, error);
            }
          }
        }
      }
    }, intervalMs);
  }
}

// Example health checks
const healingSystem = new SelfHealingSystem();

healingSystem.registerHealthCheck({
  name: 'Database Connection',
  async check() {
    try {
      await fetch('/api/health/db');
      return true;
    } catch {
      return false;
    }
  },
  async heal() {
    // Restart database connection
    await fetch('/api/admin/restart-db', { method: 'POST' });
  },
  autoHeal: true
});

healingSystem.registerHealthCheck({
  name: 'Memory Usage',
  async check() {
    if (performance.memory) {
      const usage = performance.memory.usedJSHeapSize /
                    performance.memory.jsHeapSizeLimit;
      return usage < 0.9;  // Less than 90%
    }
```

```
    return true;
  },
  async heal() {
    // Clear caches
    window.location.reload();
  },
  autoHeal: false  // Require manual intervention
});
```

Next Steps

- Performance Tuning
- Best Practices
- FAQ

Centralized Database Design

> Backend configuration, encryption, schema management, and API reference. See centralized-database-multipc.md for the multi-PC architecture. See multipc-data-flow-details.md for per-table data flows.

---

1. Overview

The centralized database feature allows T3000 to store device data in a shared database server (MSSQL, PostgreSQL, or MySQL) instead of — or in addition to — the default local SQLite. This enables multi-PC deployments where all PCs see the same data.

Supported Backends

Backend	Library	Status
SQLite	SeaORM + sqlx-sqlite	Default, always available
MSSQL	tiberius 0.12 + bb8 pool	Primary target for enterprise
PostgreSQL	SeaORM + sqlx-postgres	Supported
MySQL	SeaORM + sqlx-mysql	Supported

---

2. Password Encryption

Connection passwords are encrypted at rest using AES-256-GCM.

Implementation

```
File: api/src/database_management/db_backend_config.rs
Functions: encrypt_password(), decrypt_password()
```

- **Algorithm:** AES-256-GCM (authenticated encryption)
- **Key:** Derived from machine-specific or hardcoded key (see `ENCRYPTION_KEY`)
- **Nonce:** 12-byte random, prepended to ciphertext
- **Storage:** Base64-encoded `nonce || ciphertext || tag` in `password_encrypted` column

Flow

```
Save config:  plaintext → encrypt_password() → base64 → DB_BACKEND_CONFIG.password_encrypted
Load config:  DB_BACKEND_CONFIG.password_encrypted → base64 decode → decrypt_password() → plaintext
Connect:      plaintext password → database driver connection string
```

---

3. Schema Management

SQL Dialect Files

Each supported backend has its own CREATE TABLE schema:

File	Backend	Location
`webview_t3_device_schema.sql`	SQLite	`api/migration/sql/`
`webview_t3_device_mssql.sql`	MSSQL	`api/migration/sql/`
`webview_t3_device_postgres.sql`	PostgreSQL	`api/migration/sql/`
`webview_t3_device_mysql.sql`	MySQL	`api/migration/sql/`

Dialect Differences

Feature	SQLite	MSSQL	PostgreSQL	MySQL
Auto-increment PK	`INTEGER PRIMARY KEY`	`INT IDENTITY(1,1)`	`SERIAL`	`INT AUTO_INCREMENT`
Text type	`TEXT`	`NVARCHAR(MAX)`	`TEXT`	`TEXT`
Boolean	`INTEGER` (0/1)	`BIT`	`BOOLEAN`	`TINYINT(1)`
Blob	`BLOB`	`VARBINARY(MAX)`	`BYTEA`	`LONGBLOB`
IF NOT EXISTS	Supported	Not supported (use `IF NOT EXISTS` wrapper)	Supported	Supported

Migration Strategy

On first connection to a central DB:

1. Check if tables exist (query `information_schema.tables` or equivalent)
2. Run the appropriate dialect's CREATE TABLE statements
3. Store `database.version` in APPLICATION_CONFIG (local)
4. On subsequent connects, compare version and run ALTER TABLE if needed

---

4. MSSQL Raw SQL Adapter

For MSSQL, we use `tiberius` directly instead of SeaORM (which doesn't support MSSQL natively).

Connection Pool

RUST

```rust
// Type aliases in api/src/database_management/mssql_queries.rs
pub type MssqlPool = bb8::Pool<bb8_tiberius::ConnectionManager>;

// Pool created during startup with:
// - max_size: 10 (configurable)
// - connection_timeout: 30s
// - idle_timeout: 300s
```

Query Pattern

RUST

```rust
pub async fn mssql_query_all(
    pool: &MssqlPool,
    table: &str,
    device_instance: Option<i64>,
) -> Result<Vec<HashMap<String, serde_json::Value>>, String> {
    let mut conn = pool.get().await?;
    let query = format!("SELECT * FROM [{}] WHERE ...", table);
    let stream = conn.query(&query, &[]).await?;
    // Convert tiberius rows to HashMap<String, Value>
    // ...
}
```

MSSQL-Specific Functions

Function	Purpose
`mssql_query_all`	SELECT * from table with optional filters
`mssql_query_by_id`	SELECT by primary key
`mssql_insert`	INSERT single row
`mssql_update`	UPDATE by primary key
`mssql_delete`	DELETE by primary key
`mssql_upsert`	INSERT or UPDATE (MERGE)
`mssql_count`	COUNT(*) from table
`mssql_create_tables`	Run schema migration

---

5. DeviceDbConn Enum

Routes that access device data use `DeviceDbConn` to abstract over the active backend:

```rust
pub enum DeviceDbConn {
    SeaOrm(Arc<Mutex<DatabaseConnection>>),  // SQLite, Postgres, MySQL
    Mssql(MssqlPool),                        // MSSQL via tiberius
}
```

Resolution

```rust
pub fn resolve_device_db_conn(state: &T3AppState) -> DeviceDbConn {
    if let Some(pool) = &state.mssql_pool {
        DeviceDbConn::Mssql(pool.clone())
    } else if let Some(conn) = &state.t3_device_conn {
        DeviceDbConn::SeaOrm(conn.clone())
    } else {
        // Fallback to local
        DeviceDbConn::SeaOrm(state.local_config_conn.clone().unwrap())
    }
}
```

---

6. REST API Endpoints

Database Backend Configuration

Method	Path	Purpose
`GET`	`/api/database/backend/config`	Get active backend config (password masked)
`POST`	`/api/database/backend/config`	Save/update backend config
`POST`	`/api/database/backend/test`	Test connection to a backend
`POST`	`/api/database/backend/activate`	Switch active backend
`GET`	`/api/database/backend/status`	Current connection status
`GET`	`/api/database/backend/tables`	List tables in active backend

INI Configuration (to be added)

Method	Path	Purpose
`GET`	`/api/database/backend/ini`	Read current setting.ini [CentralDatabase]
`POST`	`/api/database/backend/ini`	Update setting.ini (local access only)

Request/Response Examples

POST /api/database/backend/config

JSON

```
{
  "backend_type": "mssql",
  "host": "192.168.1.100",
  "port": 1433,
  "database_name": "T3000_Central",
  "username": "t3000_user",
  "password": "secret123",
  "use_ssl": false,
  "role": "main",
  "store_logs": true
}
```

GET /api/database/backend/config (response)

JSON

```
{
  "id": 1,
  "backend_type": "mssql",
  "host": "192.168.1.100",
  "port": 1433,
  "database_name": "T3000_Central",
  "username": "t3000_user",
  "password": "********",
  "use_ssl": false,
  "is_active": true,
  "role": "main",
  "store_logs": true,
  "created_at": "2025-01-15T10:30:00Z",
  "updated_at": "2025-01-15T10:30:00Z"
}
```

7. DB_BACKEND_CONFIG Table Schema

```sql
CREATE TABLE IF NOT EXISTS DB_BACKEND_CONFIG (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    backend_type    TEXT NOT NULL DEFAULT 'sqlite',
    host            TEXT DEFAULT '',
    port            INTEGER DEFAULT 0,
    database_name   TEXT DEFAULT '',
    username        TEXT DEFAULT '',
    password_encrypted TEXT DEFAULT '',
    use_ssl         INTEGER DEFAULT 0,
    is_active       INTEGER DEFAULT 0,
    role            TEXT DEFAULT 'reader',
    store_logs      INTEGER DEFAULT 0,
    created_at      TEXT DEFAULT (datetime('now')),
    updated_at      TEXT DEFAULT (datetime('now'))
);
```

> This table lives in **local SQLite only** (Category G). It contains the encrypted credentials needed to connect to the central DB.

---

8. T3AppState Fields

```rust
pub struct T3AppState {
    // Primary webview database (users, auth)
    pub conn: Arc<Mutex<DatabaseConnection>>,

    // Device data connection — points to central DB when enabled, else local SQLite
    pub t3_device_conn: Option<Arc<Mutex<DatabaseConnection>>>,

    // Always points to local SQLite for Category G tables
    pub local_config_conn: Option<Arc<Mutex<DatabaseConnection>>>,

    // MSSQL connection pool (only set when active backend is MSSQL)
    pub mssql_pool: Option<MssqlPool>,

    // Multi-PC configuration (from setting.ini)
    pub central_db_enabled: bool,       // [CentralDatabase] enabled=1
    pub central_db_role: String,        // "main" or "reader"
    pub store_logs_to_central: bool,    // store_logs=1
}
```

---

9. Frontend: DatabaseConfigPage

Location: `src/t3-react/features/system/pages/DatabaseConfigPage.tsx`

Sections

1. **Status Card** — Shows current backend type, connection status, table count
2. **Backend Selector** — Radio buttons: SQLite, PostgreSQL, MySQL, MSSQL
3. **Connection Form** — Host, port, database, username, password, SSL toggle
4. **Role Selector** (new) — Main / Reader radio buttons
5. **Store Logs Toggle** (new) — Checkbox to enable SYSTEM_LOGS writing
6. **Actions** — Test Connection, Save, Scan Network (for auto-discovery)

State Management

Uses Zustand store with:

- `selectedType` — current backend selection
- `configs` — map of backend configs keyed by type
- `connectionStatus` — test result
- `role` — "main" | "reader"
- `storeLogs` — boolean

Centralized Database: Multi-PC Architecture (v4 Final)

> **Status:** Approved design — ready for implementation **Last updated:** 2025-01 **Related docs:**
>
> - centralized-database-design.md — backend config, encryption, schema
> - multipc-data-flow-details.md — per-table data flow reference

---

1. Problem Statement

T3000 manages building-automation panels (Temco BACnet controllers). A single site may have **multiple PCs** running T3000, each scanning different panels via FFI (T3000.exe DLL). Today each PC writes to its own local SQLite, so:

- No unified view of all panels across PCs.
- Trendlog detail data grows large, bloating local SQLite files.
- No way for a remote browser to see everything in one place.

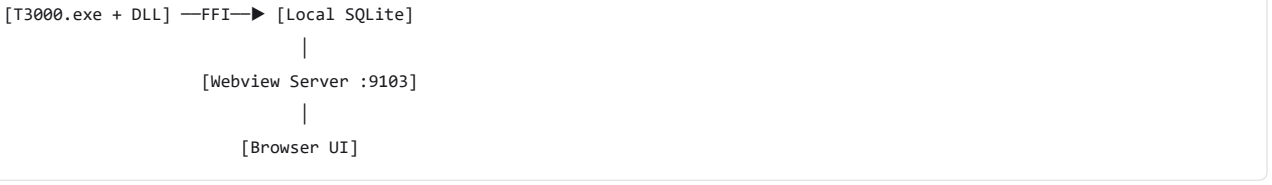
Goal: Let all PCs share one central database (MSSQL / PostgreSQL / MySQL) while keeping local SQLite as a safe fallback.

---

2. Scenarios

Scenario A — Single PC (Classic Mode)

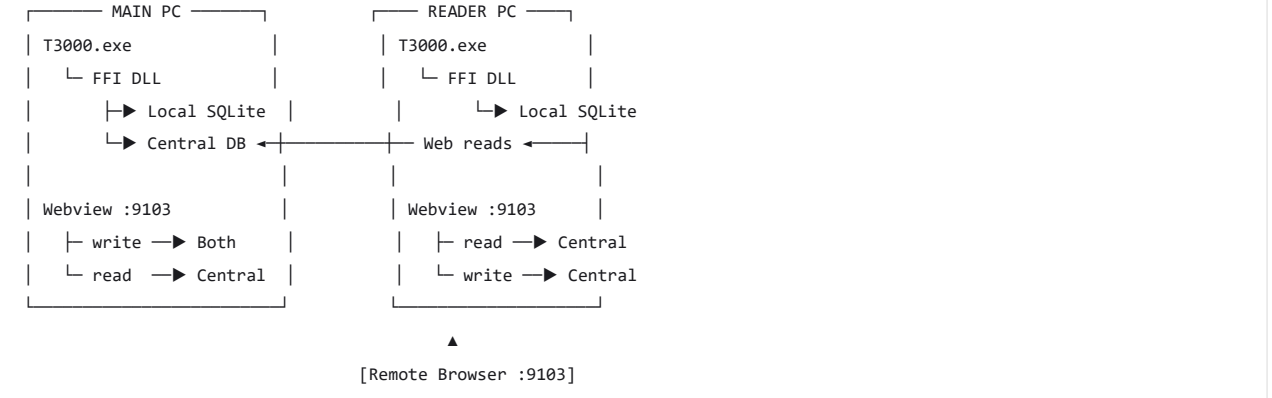
TEXT



- `enabled = 0` or no `[CentralDatabase]` section in `setting.ini`.
- Everything works exactly as today. Feature is invisible.

Scenario B — Multi-PC with Central DB

TEXT



Key rules:

1. **Main PC:** FFI writes basic data to **both** local + central. TRENDLOG_DATA_DETAIL goes to central **only**.
2. **Reader PC:** FFI writes basic data to **local only** (for fallback). Web server reads/writes from **central**.
3. **Any browser** connecting to any PC sees the same central data.

---

3. Configuration Design

3.1 setting.ini (per-PC, local file)

```
CONFIGURATION

[CentralDatabase]
enabled=1
role=main          ; "main" or "reader"
store_logs=1       ; write SYSTEM_LOGS to central DB
```

- **Location:** Same directory as T3000.exe (`C:\T3000\setting.ini` typically).
- **Why INI?** Each PC must independently know its role. This cannot be stored in a shared DB because the role determines *how* we talk to that DB.
- **Security:** Cannot be changed via remote web access — only local file edit.

3.2 DB_BACKEND_CONFIG table (local SQLite)

Stores the connection details for the central database:

Column	Purpose
`id`	Primary key
`backend_type`	`"sqlite"` , `"postgres"` , `"mysql"` , `"mssql"`
`host`	Server hostname/IP
`port`	Server port
`database_name`	Database name
`username`	Login username
`password_encrypted`	AES-256-GCM encrypted password
`use_ssl`	SSL flag
`is_active`	Which config is currently active
`role`	Mirrors INI role (for API convenience)
`store_logs`	Mirrors INI store_logs

3.3 APPLICATION_CONFIG table (local SQLite)

Runtime feature flags (key-value):

Key	Value	Purpose
`central_db.enabled`	`"1"` / `"0"`	Feature toggle
`central_db.role`	`"main"` / `"reader"`	PC role
`central_db.store_logs`	`"1"` / `"0"`	Log routing

These are populated at startup from `setting.ini` and `DB_BACKEND_CONFIG` .

---

4. Complete Table List — All 47 Tables

Category A: FFI-Synced Device Data (basic points)

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
1	DEVICES	Yes	Local + Central	Local only	Central
2	INPUTS	Yes	Local + Central	Local only	Central
3	OUTPUTS	Yes	Local + Central	Local only	Central
4	VARIABLES	Yes	Local + Central	Local only	Central

Category B: FFI-Synced Config/Programming

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
5	PROGRAMS	Yes	Local + Central	Local only	Central
6	SCHEDULES	Yes	Local + Central	Local only	Central
7	PID_TABLE	Yes	Local + Central	Local only	Central
8	HOLIDAYS	Yes	Local + Central	Local only	Central

Category C: Trendlog Metadata

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
9	TRENDLOGS	Yes	Local + Central	Local only	Central
10	TRENDLOG_INPUTS	Yes	Local + Central	Local only	Central
11	TRENDLOG_DATA	Yes	Local + Central	Local only	Central

Category D: Trendlog Detail Data (HIGH VOLUME — central only)

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
12	TRENDLOG_DATA_DETAIL	Yes	**Central ONLY**	Skip (not written)	Central

> **Why central only?** This table grows unbounded (millions of rows). Writing it to local SQLite defeats the purpose of centralization. On reader PCs it is not written at all because those panels are scanned by the main PC.

Category E: UI / Config Tables

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
13	GRAPHICS	No	Local + Central	—	Central
14	GRAPHIC_LABELS	No	Local + Central	—	Central
15	ALARMS	No*	Local + Central	—	Central
16	ALARM_SETTINGS	No	Local + Central	—	Central
17	MONITORDATA	No	Local + Central	—	Central
18	ARRAYS	No	Local + Central	—	Central
19	CONVERSION_TABLES	No	Local + Central	—	Central
20	CUSTOM_UNITS	No	Local + Central	—	Central
21	VARIABLE_UNITS	No	Local + Central	—	Central
22	USERS	No	Local + Central	—	Central
23	REMOTE_POINTS	No	Local + Central	—	Central
24	EMAIL_ALARMS	No	Local + Central	—	Central
25	EXTIO_DEVICES	No	Local + Central	—	Central
26	TSTAT_SCHEDULES	No	Local + Central	—	Central
27	MSV_DATA	No	Local + Central	—	Central

> *ALARMS may be populated by FFI in future; currently web-only. "—" under Reader PC Destination means these tables are managed via the web server, which writes to Central when enabled.

Category F: Device Settings

#	Table	FFI Writes?	Main PC Destination	Reader PC Destination	Web Server Source
28	NETWORK_SETTINGS	No	Local + Central	—	Central
29	COMMUNICATION_SETTINGS	No	Local + Central	—	Central
30	PROTOCOL_SETTINGS	No	Local + Central	—	Central
31	TIME_SETTINGS	No	Local + Central	—	Central
32	DYNDNS_SETTINGS	No	Local + Central	—	Central
33	HARDWARE_INFO	No	Local + Central	—	Central
34	FEATURE_FLAGS	No	Local + Central	—	Central
35	WIFI_SETTINGS	No	Local + Central	—	Central
36	MISC_SETTINGS	No	Local + Central	—	Central

Category G: Local-Only Infrastructure (NEVER go to central)

#	Table	Purpose	Why Local Only
37	APPLICATION_CONFIG	Feature flags, sync intervals	Per-PC settings
38	APPLICATION_CONFIG_HISTORY	Config change audit trail	Per-PC history
39	DB_BACKEND_CONFIG	Central DB connection details	Contains encrypted credentials
40	database_partition_config	Partition management config	Local DB management
41	database_files	Partition file tracking	Local DB management
42	database_partitions	Partition metadata	Local DB management
43	DATA_SYNC_METADATA	FFI sync cycle tracking	Per-PC sync state
44	TRENDLOG_DATA_SYNC_METADATA	Trendlog sync tracking	Per-PC sync state

Category H: Legacy / Deprecated (LOCAL only)

#	Table	Notes
45	TRENDLOG_DATA_OLD	Legacy format, kept for migration
46	TRENDLOG_VIEWS	UI view configuration
47	REMOTE_TSTAT_DB	Legacy remote thermostat cache

New Table: SYSTEM_LOGS (to be added)

#	Table	Main PC Destination	Reader PC Destination	Web Server Source
48	SYSTEM_LOGS	Central (if store_logs=1)	Central (if store_logs=1)	Central

> SYSTEM_LOGS captures application events, errors, and audit entries. Controlled by `store_logs` flag in setting.ini.

---

5. Startup Flow

```text
App starts
  │
  ├─ 1. Read setting.ini [CentralDatabase] section
  │     → enabled, role, store_logs
  │     → If section missing or enabled=0 → classic mode, done
  │
  ├─ 2. Open local SQLite (webview_t3_device.db)
  │     → Read DB_BACKEND_CONFIG where is_active=1
  │     → Decrypt password
  │
  ├─ 3. Connect to central DB
  │     → Run schema migration if needed (create tables)
  │     → Store connection in T3AppState
  │
  ├─ 4. Populate T3AppState fields:
  │     → central_db_enabled = true
  │     → central_db_role = "main" or "reader"
  │     → store_logs_to_central = true/false
  │     → t3_device_conn = central DB connection
  │     → local_config_conn = local SQLite (always kept)
  │
  └─ 5. Start services:
        → FFI sync service (behavior varies by role)
        → Web server (reads/writes from central)
```

---

6. FFI Sync Service Behavior

Main PC

```rust
// For DEVICES, INPUTS, OUTPUTS, VARIABLES, PROGRAMS, SCHEDULES,
// PID_TABLE, HOLIDAYS, TRENDLOGS, TRENDLOG_INPUTS, TRENDLOG_DATA:
fn sync_basic_data(panel_data) {
    write_to_local_sqlite(panel_data);    // always — fallback safety
    write_to_central_db(panel_data);      // dual write
}

// For TRENDLOG_DATA_DETAIL:
fn sync_trendlog_detail(detail_rows) {
    write_to_central_db(detail_rows);     // central ONLY — no local copy
    // Local SQLite is NOT written — saves gigabytes
}
```

Reader PC

```rust
// FFI still runs — this PC scans its own assigned panels
fn sync_basic_data(panel_data) {
    write_to_local_sqlite(panel_data);    // local only — for fallback
    // Does NOT write to central — main PC owns central writes
}

// TRENDLOG_DATA_DETAIL:
fn sync_trendlog_detail(detail_rows) {
    // Not applicable — reader panels' trendlog detail
    // should be handled by whichever PC is "main" for those panels
    // OR: write to central if this reader also collects trend data
    write_to_central_db(detail_rows);     // optional: central only
}
```

> **Important:** Reader FFI writing to local SQLite ensures that if the central DB goes down, the reader PC can switch back to classic mode and still have recent data.

---

7. Web Server Behavior

When central_db_enabled = true

Operation	Connection Used
Read any table (A–F, H)	Central DB
Write any table (A–F)	Central DB (+ local for main PC basic tables)
Read local-only tables (G)	Local SQLite via `local_config_conn`
Write local-only tables (G)	Local SQLite via `local_config_conn`

When central_db_enabled = false (classic mode)

Operation	Connection Used
All reads and writes	Local SQLite

---

8. Switch-Back Safety

If the central DB goes offline or the admin sets `enabled=0`:

1. **Main PC:** Has full basic data in local SQLite (dual-written). Only TRENDLOG_DATA_DETAIL is missing locally — acceptable because historical trend data is not critical for immediate operation.
2. **Reader PC:** Has its own panels' basic data locally. Can resume classic mode instantly.
3. **No data loss** for operational data — only historical trendlog detail is in central only.

---

9. Edge Cases

Scenario	Behavior
Central DB unreachable at startup	Fall back to classic mode, log warning
Central DB goes down mid-operation	Queue failed writes, retry with backoff, log errors
Two PCs claim "main" role	Both write to central — last-write-wins for basic data (safe for UPSERT operations). TRENDLOG_DATA_DETAIL uses INSERT so no conflict.
INI has role=main but no DB config	Treat as classic mode (enabled but not configured)
Reader PC has no network to central	Operates in classic mode with local SQLite
Schema version mismatch	Run migration on connect, compare `database.version` in APPLICATION_CONFIG

---

10. Implementation Phases

Phase 8: INI Configuration Reader

- Create `api/src/ini_config.rs` — parse `setting.ini` `[CentralDatabase]` section
- Wire into startup in `app_state.rs`

Phase 9: SYSTEM_LOGS Table

- Add `SYSTEM_LOGS` CREATE TABLE to all 4 SQL dialect files
- Create SeaORM entity and basic CRUD endpoint

Phase 10: Startup Wiring

- `create_t3_app_state()` reads INI → reads DB_BACKEND_CONFIG → sets T3AppState fields
- Connect to central DB, run migration
- Keep `local_config_conn` always pointing to local SQLite

Phase 11: FFI Dual-Write (Main PC)

- Modify `t3_ffi_sync_service.rs` : basic data → write to both connections
- TRENDLOG_DATA_DETAIL → write to central only
- Reader PC: FFI writes local only (no change from current behavior)

Phase 12: Web Server Routing

- All route handlers check `central_db_enabled`
- If enabled: read/write from `t3_device_conn` (central)
- Local-only tables always use `local_config_conn`
- Write operations on main PC also dual-write to local

Phase 13: INI API Endpoints

- `GET /api/database/backend/ini` — read current INI config
- `POST /api/database/backend/ini` — update INI config (local access only)

Phase 14: Frontend Updates

- Add role selector (main/reader) to DatabaseConfigPage
- Add store_logs toggle
- Show current mode status
- Disable role change for remote connections (security)

Phase 15: Testing & Verification

- Unit tests for INI parsing
- Integration tests for dual-write
- Manual test: two PCs, one main, one reader, verify data flows correctly

Centralized Logging Design

> Final architecture doc for the policy-driven logging rollout. See centralized-database-design.md and centralized-database-multipc.md for related backend architecture.

---

1. Final Decision

This is the canonical documentation for the centralized logging module.

- Keep this file as the single source of truth.
- Do not keep a duplicate copy in `docs/todo/`.

2. Current State (Truth)

Centralized logging is implemented, but rollout is still hybrid:

- Centralized path is active via `LoggingService`.
- Some legacy direct `ServiceLogger` calls still exist.
- Centralized sinks currently in use: DB and file (policy-driven).
- Console output still appears from legacy paths and is not a centralized policy sink.

3. Unified Target Flow

All emitters should end up using one path:

- emitter -> `LoggingService` -> policy evaluation -> sink routing -> DB/file

Operational rules:

1. Category/level are normalized in one place.
2. Runtime category policy controls enable/min-level/detail/sinks.
3. Errors always persist to DB (safety invariant).
4. High-volume categories prefer MSSQL when available; fallback to SQLite.
5. Existing file naming behavior stays compatible.

4. What Is Already Done

- Added centralized module under `api/src/logging/`.
- Routed core app-log path through centralized service.
- Updated frontend legacy LogUtil to shared implementation.
- Aligned file-log endpoint base path in React logs page.
- Documentation updated to reflect hybrid rollout instead of "fully complete".

5. What Must Still Be Done (Required)

To reach full centralization, remaining legacy direct file logging must be migrated.

Priority cleanup scope:

1. FFI sync paths
2. Partition monitor/query paths
3. Other remaining `ServiceLogger` direct call sites

Expected final behavior after cleanup:

- All logs (backend + frontend-originated) follow centralized policy.
- File output still exists, but only through centralized emission.
- No split behavior between legacy direct file writes and policy-based writes.

6. Acceptance Checklist

1. Single Rust logging facade handles all log events.

2. Logs page settings consistently affect backend and frontend-originated logs.
3. Existing file naming remains compatible.
4. Activity Log categories/levels remain correct.
5. Error events persist even if category policy is restrictive.
6. No duplicate writes from mixed old/new paths.
7. File Logs page route/path remains aligned with backend endpoint.

7. Validation Snapshot

- `cargo check` : pass
- `cargo test --no-run` : environment-dependent (heavy build)

This is sufficient for a final docs commit. Full rollout validation continues during code migration.

Device Panel Loading

Overview

When you select a device in T3000, the system loads all data from that device panel and displays it in your interface. This process involves network communication, database caching, and real-time updates.

What Happens When You Click a Device?

1. You Select a Device

Click on any device in the device tree (left sidebar). The device could be a T3-BB controller, T3-TB thermostat, or any BACnet device on your network.

2. Loading Indicator Appears

You'll see a loading spinner while the system retrieves data from the device.

3. Data Appears in Tabs

Once loaded, you'll see multiple tabs with device data:

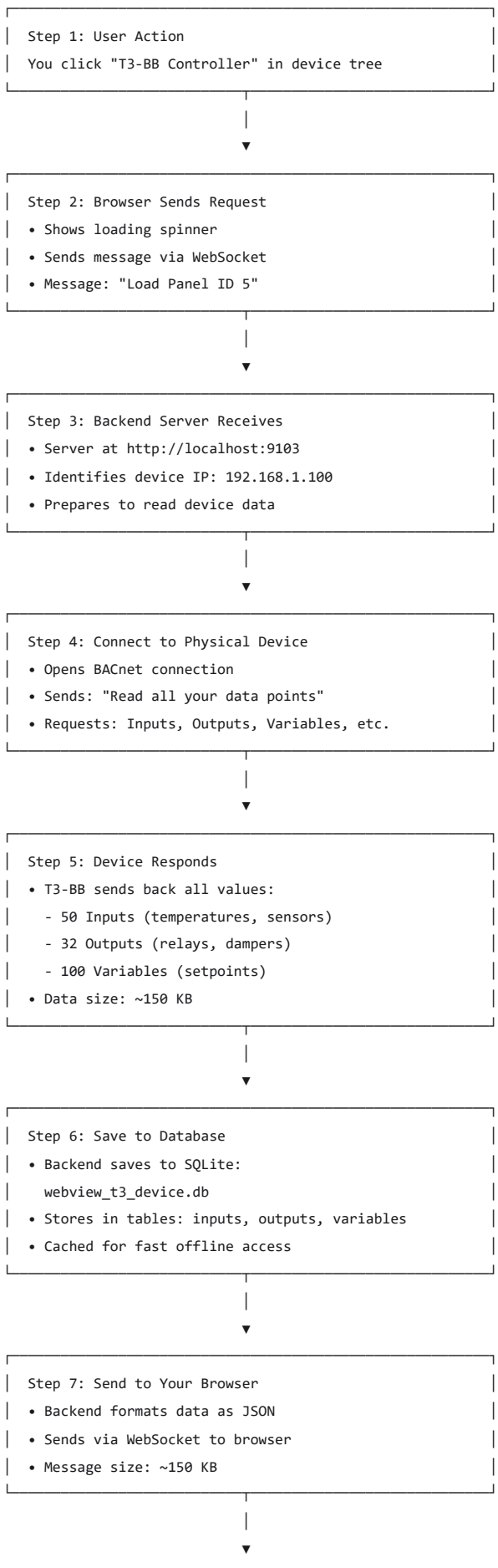
- **Inputs**: Sensor readings (temperature, humidity, pressure)
- **Outputs**: Control points (relays, dampers, valves)
- **Variables**: Setpoints and calculated values
- **Programs**: Control logic
- **Schedules**: Time-based automation
- **Graphics**: Visual layouts

4. Real-Time Updates

After initial load, any changes on the device automatically update in your interface without refreshing.

Data Flow Visualization

TEXT



```
┌──────────────────────────────────────────────────┐
│  Step 8: Display on Screen                       │
│  • Loading spinner disappears                    │
│  • Tables populate with data:                    │
│    - Inputs tab shows 50 rows                    │
│    - Outputs tab shows 32 rows                   │
│    - Variables tab shows 100 rows                │
│  • Ready for editing!                            │
└──────────────────────────────────────────────────┘


Total Time: 2-5 seconds for typical device
```

Why This Matters

Fast Offline Access: Once loaded, device data is cached locally. You can view it even if the device goes offline.

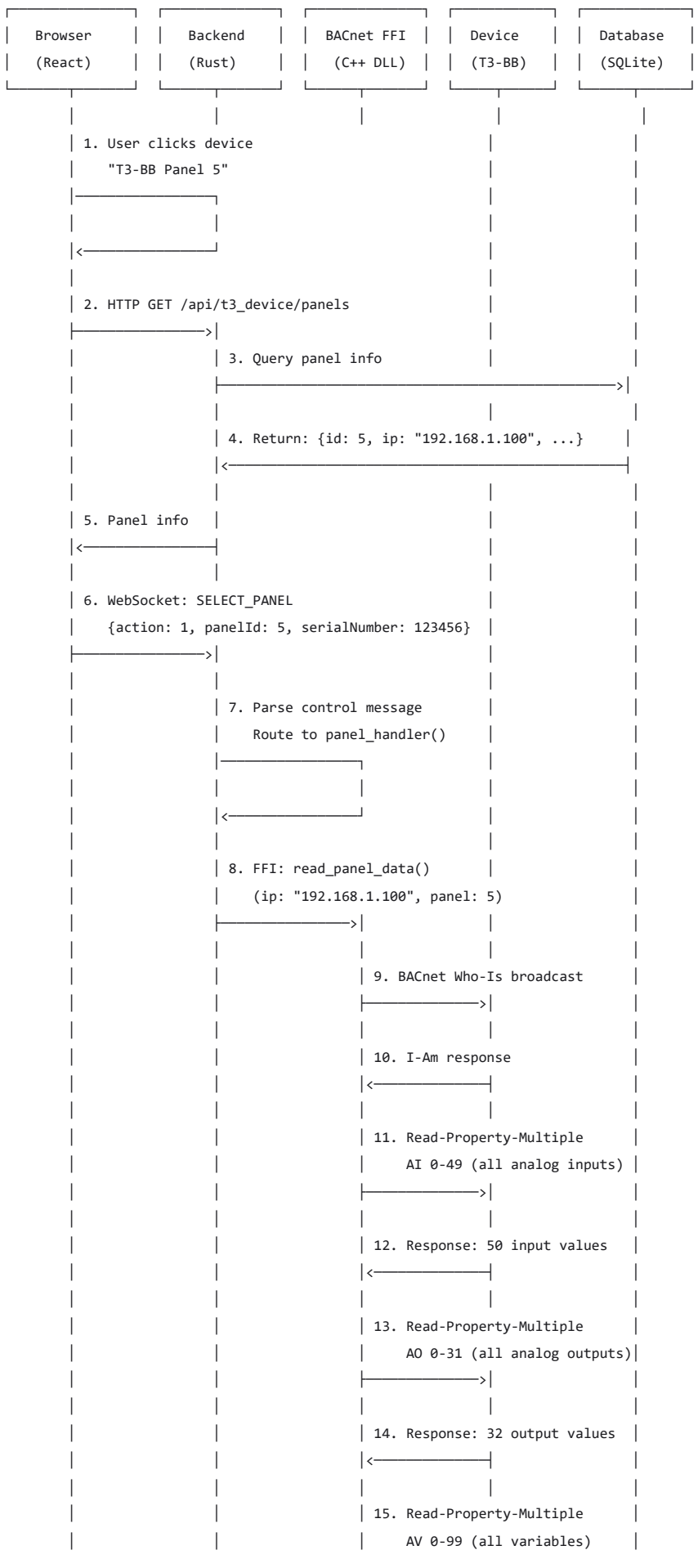
Real-Time Updates: Changes on the physical device appear in your interface automatically.

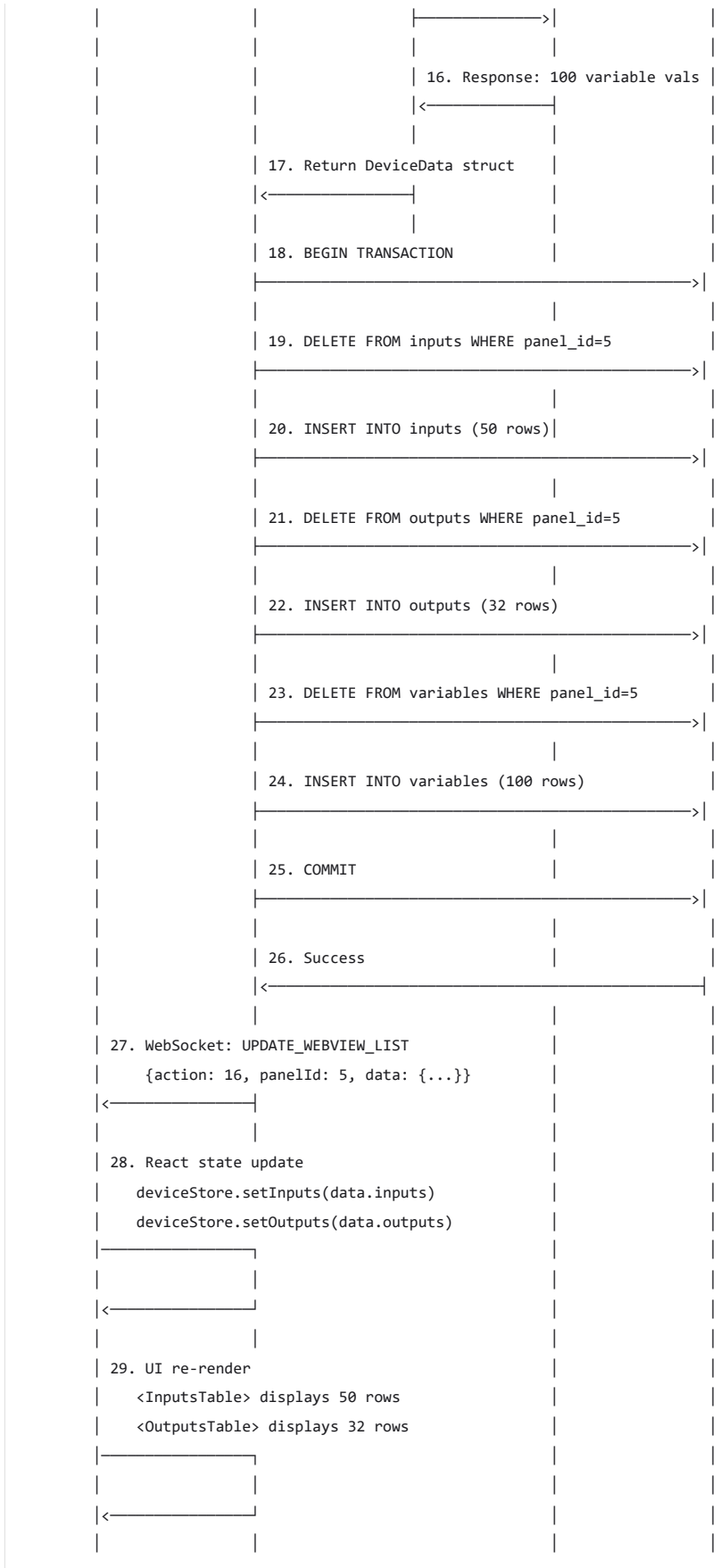
Efficient Network Usage: The system only loads data when you select a device, not all at once.

Complete Technical Data Flow

Full System Sequence Diagram

TEXT


Browser (React)
Backend (Rust)
BACnet FFI (C++ DLL)
Device (T3-BB)
Database (SQLite)
1. User clicks device "T3-BB Panel 5"
2. HTTP GET /api/t3_device/panels
3. Query panel info
4. Return: {id: 5, ip: "192.168.1.100", ...}
5. Panel info
6. WebSocket: SELECT_PANEL {action: 1, panelId: 5, serialNumber: 123456}
7. Parse control message Route to panel_handler()
8. FFI: read_panel_data() (ip: "192.168.1.100", panel: 5)
9. BACnet Who-Is broadcast
10. I-Am response
11. Read-Property-Multiple AI 0-49 (all analog inputs)
12. Response: 50 input values
13. Read-Property-Multiple AO 0-31 (all analog outputs)
14. Response: 32 output values
15. Read-Property-Multiple AV 0-99 (all variables)



API Endpoints Called

1. GET /api/t3_device/panels

HTTP

```
GET http://localhost:9103/api/t3_device/panels HTTP/1.1
Host: localhost:9103
Accept: application/json

Response 200 OK:
[
  {
    "id": 5,
    "serial_number": 123456,
    "product_name": "T3-BB",
    "ip_address": "192.168.1.100",
    "port": 47808,
    "location": "Building A - Floor 2",
    "modbus_id": 1,
    "panel_number": 5,
    "online": true
  }
]
```

2. WebSocket Control Message: SELECT_PANEL

JSON

```
{
  "action": 1,
  "source": 1,
  "panelId": 5,
  "serialNumber": 123456,
  "entryType": 0,
  "data": null
}
```

3. WebSocket Response: UPDATE_WEBVIEW_LIST

JSON

```
{
  "action": 16,
  "source": 0,
  "panelId": 5,
  "serialNumber": 123456,
  "entryType": 1,
  "data": {
    "inputs": [
      {
        "number": 0,
        "label": "Room Temperature",
        "value": 72.5,
        "units": "°F",
        "autoManual": "AUTO",
        "rangeLow": 32.0,
        "rangeHigh": 120.0,
        "objectinstance": 3000000
      },
      // ... 49 more inputs
    ],
    "outputs": [
      {
        "number": 0,
        "label": "Cooling Valve",
        "value": 65.0,
        "autoManual": "AUTO",
        "rangeLow": 0.0,
        "rangeHigh": 100.0,
        "objectinstance": 3000100
      },
      // ... 31 more outputs
    ],
    "variables": [
      {
        "number": 0,
        "label": "Cooling Setpoint",
        "value": 75.0,
        "units": "°F",
        "autoManual": "AUTO",
        "objectinstance": 3000200
      },
      // ... 99 more variables
    ]
  }
}
```

React Component Flow

Step 1: DeviceTree Component

TYPESCRIPT

```typescript
// File: src/t3-react/components/DeviceTree/DeviceTree.tsx
const DeviceTree: React.FC = () => {
  const [panels, setPanels] = useState<Panel[]>([]);

  useEffect(() => {
    // Load panels on mount
    fetch('http://localhost:9103/api/t3_device/panels')
      .then(res => res.json())
      .then(data => setPanels(data));
  }, []);

  const handleDeviceClick = (panel: Panel) => {
    // Dispatch action to load panel
    deviceStore.selectPanel(panel.id, panel.serial_number);
  };

  return (
    <Tree>
      {panels.map(panel => (
        <TreeItem
          key={panel.id}
          onClick={() => handleDeviceClick(panel)}
        >
          {panel.product_name} - {panel.location}
        </TreeItem>
      ))}
    </Tree>
  );
};
```

Step 2: Device Store (Zustand)

TYPESCRIPT

```typescript
// File: src/t3-react/stores/deviceStore.ts
interface DeviceState {
  currentPanelId: number | null;
  inputs: Input[];
  outputs: Output[];
  variables: Variable[];
  isLoading: boolean;
  selectPanel: (panelId: number, serialNumber: number) => void;
  updateData: (data: DeviceData) => void;
}

export const useDeviceStore = create<DeviceState>((set) => ({
  currentPanelId: null,
  inputs: [],
  outputs: [],
  variables: [],
  isLoading: false,

  selectPanel: (panelId, serialNumber) => {
    set({ isLoading: true, currentPanelId: panelId });

    // Send WebSocket message
    websocket.send(JSON.stringify({
      action: 1,  // SELECT_PANEL
      source: 1,  // WebUI
      panelId,
      serialNumber
    }));
  },

  updateData: (data) => {
    set({
      inputs: data.inputs,
      outputs: data.outputs,
      variables: data.variables,
      isLoading: false
    });
  }
}));
```

Step 3: WebSocket Hook

TYPESCRIPT

```typescript
// File: src/t3-react/hooks/useWebSocket.ts
export const useWebSocket = () => {
  const deviceStore = useDeviceStore();
  const [ws, setWs] = useState<WebSocket | null>(null);

  useEffect(() => {
    const websocket = new WebSocket('ws://localhost:9103/ws');

    websocket.onmessage = (event) => {
      const message = JSON.parse(event.data);

      if (message.action === 16) {  // UPDATE_WEBVIEW_LIST
        deviceStore.updateData(message.data);
      }
    };

    setWs(websocket);

    return () => websocket.close();
  }, []);

  return ws;
};
```

Step 4: Data Display Components

TYPESCRIPT

```tsx
// File: src/t3-react/components/InputsTable/InputsTable.tsx
const InputsTable: React.FC = () => {
  const { inputs, isLoading } = useDeviceStore();

  if (isLoading) {
    return <Spinner label="Loading inputs..." />;
  }

  return (
    <Table>
      <TableHeader>
        <TableRow>
          <TableHeaderCell>Number</TableHeaderCell>
          <TableHeaderCell>Label</TableHeaderCell>
          <TableHeaderCell>Value</TableHeaderCell>
          <TableHeaderCell>Units</TableHeaderCell>
          <TableHeaderCell>Mode</TableHeaderCell>
        </TableRow>
      </TableHeader>
      <TableBody>
        {inputs.map(input => (
          <TableRow key={input.objectinstance}>
            <TableCell>{input.number}</TableCell>
            <TableCell>{input.label}</TableCell>
            <TableCell>{input.value}</TableCell>
            <TableCell>{input.units}</TableCell>
            <TableCell>{input.autoManual}</TableCell>
          </TableRow>
        ))}
      </TableBody>
    </Table>
  );
};
```

Backend Handler Implementation

Rust Panel Handler

RUST

```rust
// File: api/src/t3_device/panel_handler.rs
use crate::ffi_interface::read_panel_data;
use crate::db_connection::Database;

pub async fn handle_select_panel(
    msg: ControlMessage,
    db: Arc<Mutex<Database>>,
    clients: Arc<Mutex<Vec<WebSocketClient>>>
) -> Result<(), Error> {
    // Step 1: Get panel info from database
    let panel = {
        let db_lock = db.lock().unwrap();
        db_lock.query_row(
            "SELECT ip_address, port, serial_number FROM panels WHERE id = ?",
            params![msg.panel_id],
            |row| {
                Ok(Panel {
                    ip: row.get(0)?,
                    port: row.get(1)?,
                    serial: row.get(2)?
                })
            }
        )?
    };

    // Step 2: Call FFI to read device via BACnet
    let device_data = unsafe {
        read_panel_data(
            CString::new(panel.ip)?.as_ptr(),
            panel.port,
            msg.panel_id
        )
    };

    if device_data.is_null() {
        return Err(Error::DeviceReadFailed);
    }

    // Step 3: Convert C++ data to Rust structs
    let data = unsafe {
        DeviceData::from_ffi(device_data)
    };

    // Step 4: Update database cache
    {
        let db_lock = db.lock().unwrap();
        let tx = db_lock.transaction()?;

        // Clear old data
        tx.execute("DELETE FROM inputs WHERE panel_id = ?", params![msg.panel_id])?;
        tx.execute("DELETE FROM outputs WHERE panel_id = ?", params![msg.panel_id])?;
        tx.execute("DELETE FROM variables WHERE panel_id = ?", params![msg.panel_id])?;

        // Insert new data
        for input in &data.inputs {
            tx.execute(
                "INSERT INTO inputs (panel_id, number, label, value, units, auto_manual, objectinstance)
                 VALUES (?, ?, ?, ?, ?, ?, ?)",
```

```
                params![
                    msg.panel_id, input.number, input.label,
                    input.value, input.units, input.auto_manual, input.objectinstance
                ]
            )?;
        }

        // Similar for outputs and variables...

        tx.commit()?;
    }

    // Step 5: Broadcast to all WebSocket clients
    let response = ControlMessage {
        action: 16,  // UPDATE_WEBVIEW_LIST
        source: 0,   // Server
        panel_id: msg.panel_id,
        serial_number: panel.serial,
        entry_type: 1,
        data: Some(serde_json::to_value(data)?),
    };

    let clients_lock = clients.lock().unwrap();
    for client in clients_lock.iter() {
        client.send(serde_json::to_string(&response)?).await?;
    }

    Ok(())
}
```

FFI Interface

RUST

```
// File: api/src/ffi_interface.rs
#[repr(C)]
pub struct CDeviceData {
    pub inputs: *mut CInput,
    pub inputs_count: usize,
    pub outputs: *mut COutput,
    pub outputs_count: usize,
    pub variables: *mut CVariable,
    pub variables_count: usize,
}

#[link(name = "bacnet_ffi")]
extern "C" {
    pub fn read_panel_data(
        ip: *const c_char,
        port: u16,
        panel_id: i32
    ) -> *mut CDeviceData;

    pub fn free_device_data(data: *mut CDeviceData);
}
```

Performance Metrics

Network Timeline

TEXT

```
T+0ms:    User clicks device
T+10ms:   GET /api/t3_device/panels completes
T+15ms:   WebSocket SELECT_PANEL sent
T+20ms:   Backend receives message
T+25ms:   FFI call initiated
T+100ms:  BACnet Who-Is broadcast
T+150ms:  Device I-Am received
T+200ms:  Read AI 0-49 started
T+800ms:  AI values received (600ms for 50 reads)
T+850ms:  Read AO 0-31 started
T+1300ms: AO values received (450ms for 32 reads)
T+1350ms: Read AV 0-99 started
T+2550ms: AV values received (1200ms for 100 reads)
T+2600ms: Database transaction started
T+2750ms: All inserts completed (150ms)
T+2800ms: WebSocket UPDATE_WEBVIEW_LIST sent
T+2850ms: React state updated
T+2900ms: UI re-rendered

Total: ~2.9 seconds
```

Data Sizes

- BACnet request per object: 48 bytes
- BACnet response per object: ~80 bytes
- Total network traffic: ~23 KB
- WebSocket JSON payload: ~150 KB
- Database transaction: ~182 inserts

Memory Usage

- C++ DeviceData struct: ~50 KB
- Rust deserialized data: ~80 KB
- React state: ~150 KB
- DOM rendering: ~200 KB
- Total: ~480 KB per device

Error Scenarios

Device Offline

TEXT

```
User clicks → GET /panels (✓) → WS message (✓) → FFI call (✗ timeout 5s)
                                                        ↓
                                Error message: "Device not responding"
                                                        ↓
                                UI shows: Red badge on device tree item
```

Network Timeout

TEXT

```
Reading AI 0-49 (✓) → Reading AO 0-31 (✗ timeout after 30/32)
                                        ↓
                        Partial data saved to database
                                        ↓
                UI shows: Inputs ✓, Outputs (30/32) ⚠, retry button
```

Database Lock

```
TEXT

User A loading Panel 5 → Transaction open → User B clicks Panel 6
                                                    ↓
                                    Wait for lock (max 2s) → Timeout
                                                    ↓
                                    Show "Please wait" → Retry automatically
```

Optimization Techniques

Batch Reading

```rust
RUST

// Instead of 50 separate BACnet calls:
for i in 0..50 {
    read_property(AI, i);  // 50 round-trips = slow
}

// Use Read-Property-Multiple:
read_property_multiple(AI, vec![0..50]);  // 1 round-trip = fast
```

Parallel Processing

```rust
RUST

// Read inputs, outputs, variables in parallel
let (inputs, outputs, variables) = tokio::join!(
    async { ffi::read_inputs(panel_id) },
    async { ffi::read_outputs(panel_id) },
    async { ffi::read_variables(panel_id) }
);
```

Progressive Loading

```typescript
TYPESCRIPT

// Load critical data first
deviceStore.selectPanel(panelId, { priority: 'inputs,outputs' });
// UI shows inputs/outputs immediately

// Load rest in background
setTimeout(() => {
  deviceStore.loadAdditionalData(panelId, { types: 'variables,programs' });
}, 1000);
```

Database Schema Details

SQL

```sql
-- Inputs table
CREATE TABLE inputs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  panel_id INTEGER NOT NULL,
  number INTEGER NOT NULL,
  label TEXT,
  value REAL,
  units TEXT,
  auto_manual TEXT DEFAULT 'AUTO',
  range_low REAL,
  range_high REAL,
  filter INTEGER DEFAULT 3,
  objectinstance INTEGER NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(panel_id, objectinstance),
  FOREIGN KEY (panel_id) REFERENCES panels(id)
);

CREATE INDEX idx_inputs_panel ON inputs(panel_id);
CREATE INDEX idx_inputs_objectinstance ON inputs(objectinstance);
```

File Locations Reference

- **Frontend**: `src/t3-react/components/DeviceTree/`
- **Store**: `src/t3-react/stores/deviceStore.ts`
- **WebSocket**: `src/t3-react/hooks/useWebSocket.ts`
- **Backend Handler**: `api/src/t3_device/panel_handler.rs`
- **FFI Interface**: `api/src/ffi_interface.rs`
- **Database**: `api/src/db_connection.rs`
- **C++ BACnet**: `api/build/bacnet_ffi.dll`

Flow Logging Architecture — Design Document

Status: Design only. No code changes made yet. **Date:** 2026-05-21 **Scope:** New flow-based tracing layer. Existing `T3_APP_LOG`, `ActivityLogTab`, and all current logging code are untouched.

---

0. Database placement and migration strategy

0a. Which database gets the flow tables?

Only `webview_t3_device.db` (local SQLite). Never the MSSQL center DB.

This is consistent with the existing design rule: flow logs are local-only diagnostic data, not operational records that need to be replicated to the center database. Every installation — both standalone and server mode — has a local `webview_t3_device.db`, so every installation always gets the tables.

The four SQL schema files in `migration/sql/` serve different purposes:

File	Used by	Flow tables needed?
`webview_t3_device_schema.sql`	SQLite fresh-install path	**YES**
`webview_t3_device_mssql.sql`	Reference / future MSSQL deploy	No
`webview_t3_device_mysql.sql`	Reference / future MySQL deploy	No
`webview_t3_device_postgres.sql`	Reference / future PostgreSQL deploy	No

0b. How the migration system works (existing infrastructure)

```
Startup
  └─ initialize_t3_device_database()
       ├─ [if no DB exists] create from embedded SQL schema OR copy ResourceFile/webview_t3_device.db
       └─ run_t3_device_migrations()
            └─ T3DeviceMigrator::up(&conn, None)   ← SeaORM
                 ├─ reads seaql_migrations table to find already-applied migrations
                 └─ applies only NEW migrations in order, skips already-applied ones
```

SeaORM maintains a `seaql_migrations` table inside the SQLite file itself. Every migration has a name derived from its file name (e.g. `m20260521_add_flow_log_tables`). On each startup, SeaORM compares the migration list in code against the table — migrations not yet in the table are run in order. Migrations already recorded are skipped. **Existing data is never touched.**

This means the same startup path handles both cases without any conditional logic:

Scenario	What happens
Fresh install, DB just created	`seaql_migrations` empty → all migrations run → flow tables created
Dev machine, up to date	Migration already in `seaql_migrations` → skipped
Production, real data, older release	Migration NOT in `seaql_migrations` → runs once safely → flow tables added
Production, re-deployed same version	Migration already in `seaql_migrations` → skipped

0c. Developer action required to ship the tables

Two files must be edited and one new file must be created:

1. **`api/migration/sql/webview_t3_device_schema.sql`** — add the DDL at the end. This file is embedded into the binary and used only when creating a database from scratch. Using `CREATE TABLE IF NOT EXISTS` here means it is idempotent — even if the migration also runs, there is no conflict.

2. **`api/migration/src/m20260521_add_flow_log_tables.rs`** — new migration file. This is what runs on all existing production databases. Pattern is identical to `m20260403_add_raw_calibration_fields.rs`: raw SQL via `db.execute_unprepared()`, `CREATE TABLE IF NOT EXISTS`, `down()` left as a no-op (SQLite `DROP TABLE` would lose data with no benefit).

3. **`api/migration/src/lib.rs`** — register the new migration inside `T3DeviceMigrator`.

That is all. No changes to `utils.rs`, `db_connection.rs`, or any startup code — the existing `run_t3_device_migrations()` call automatically picks it up.

0d. New migration file skeleton (for implementation)

RUST

```rust
// api/migration/src/m20260521_add_flow_log_tables.rs
use sea_orm_migration::{async_trait::async_trait, prelude::*};

#[derive(DeriveMigrationName)]
pub struct Migration;

#[async_trait]
impl MigrationTrait for Migration {
    async fn up(&self, manager: &SchemaManager) -> Result<(), DbErr> {
        let db = manager.get_connection();

        // T3_FLOW: one row per flow instance
        db.execute_unprepared(
            "CREATE TABLE IF NOT EXISTS T3_FLOW (
                id           INTEGER PRIMARY KEY AUTOINCREMENT,
                flow_id      TEXT    NOT NULL UNIQUE,
                flow_type    TEXT    NOT NULL,
                trigger_src  TEXT    NOT NULL,
                started_at   INTEGER NOT NULL,
                ended_at     INTEGER,
                status       TEXT    NOT NULL DEFAULT 'running',
                hostname     TEXT,
                total_steps  INTEGER NOT NULL DEFAULT 0,
                done_steps   INTEGER NOT NULL DEFAULT 0,
                error_count  INTEGER NOT NULL DEFAULT 0,
                meta         TEXT
            )"
        ).await?;

        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_type    ON T3_FLOW (flow_type)"
        ).await?;
        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_started ON T3_FLOW (started_at DESC)"
        ).await?;
        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_status  ON T3_FLOW (status)"
        ).await?;

        // T3_FLOW_STEP: one row per step inside a flow
        db.execute_unprepared(
            "CREATE TABLE IF NOT EXISTS T3_FLOW_STEP (
                id           INTEGER PRIMARY KEY AUTOINCREMENT,
                flow_id      TEXT    NOT NULL,
                seq          INTEGER NOT NULL,
                step_name    TEXT    NOT NULL,
                level        TEXT    NOT NULL DEFAULT 'info',
                source       TEXT,
                api_path     TEXT,
                action_type  INTEGER,
                status       TEXT    NOT NULL DEFAULT 'ok',
                duration_ms  INTEGER,
                payload_ref  TEXT,
                message      TEXT,
                details      TEXT,
                ts_unix      INTEGER NOT NULL,
                ts_fmt       TEXT    NOT NULL
```

```
            )"
        ).await?;

        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_step_flow ON T3_FLOW_STEP (flow_id)"
        ).await?;
        db.execute_unprepared(
            "CREATE INDEX IF NOT EXISTS idx_t3_flow_step_ts   ON T3_FLOW_STEP (ts_unix DESC)"
        ).await?;

        // T3_FLOW_PAYLOAD: tracks offloaded large-payload files
        db.execute_unprepared(
            "CREATE TABLE IF NOT EXISTS T3_FLOW_PAYLOAD (
                id           INTEGER PRIMARY KEY AUTOINCREMENT,
                flow_id      TEXT    NOT NULL,
                step_id      INTEGER NOT NULL,
                file_path    TEXT    NOT NULL,
                size_bytes   INTEGER NOT NULL,
                created_at   INTEGER NOT NULL,
                purged       INTEGER NOT NULL DEFAULT 0
            )"
        ).await?;

        Ok(())
    }

    async fn down(&self, _manager: &SchemaManager) -> Result<(), DbErr> {
        // Intentionally empty: dropping these tables would lose diagnostic history.
        // To remove manually: DROP TABLE T3_FLOW_PAYLOAD; DROP TABLE T3_FLOW_STEP; DROP TABLE T3_FLOW;
        Ok(())
    }
}
```

And register it in `lib.rs`:

RUST

```rust
pub struct T3DeviceMigrator;

impl MigratorTrait for T3DeviceMigrator {
    fn migrations() -> Vec<Box<dyn MigrationTrait>> {
        vec![
            Box::new(m20260403_add_raw_calibration_fields::Migration),
            Box::new(m20260521_add_flow_log_tables::Migration),   // ← new
        ]
    }
}
```

---

1. Why the tables are still needed

The `FlowHandle` being passed as a parameter is about **how you write** — not **whether you store**.

The tables are the persistent store. Without them there is nothing to query, page through, or drill into. The FlowHandle is just a lightweight in-memory context (a UUID + a db reference + a step counter) that knows which row to write under. Without the tables, the FlowHandle has nowhere to persist anything.

```
FlowHandle (memory)          SQLite tables (persistent)
──────────────────           ──────────────────────────
flow_id: "a3f1..."           T3_FLOW  row for "a3f1..."
step_counter: 3         ──►  T3_FLOW_STEP rows 1, 2, 3...
db: &DatabaseConnection      T3_FLOW_PAYLOAD rows if large
```

---

2. Confirmed system architecture (from code inspection)

2a. Two FFI entry points (both must be tracked)

From code inspection there are two separate services that call the C++ FFI function:

Entry point	File	Used by
`call_handle_webview_msg()` in `t3_ffi_sync_service.rs`	background sync scheduler	SYNC_CYCLE, GET_PANELS_LIST
`T3000FfiApiService::call_ffi()` in `t3_ffi_api_service.rs`	HTTP request handler	every user-triggered action from React

Both use the same global `BACNETWEBVIEW_HANDLE_WEBVIEW_MSG_FN` pointer and the same `ffi_call_lock()` mutex. Neither should be modified. Only their callers instrument flow steps.

2b. React call chain for user-triggered refreshes

```
User clicks Refresh (Inputs page)
  └─ InputRefreshApi.refreshAllFromDevice(serialNumber)
       └─ PanelDataRefreshService.refreshFromDevice({ type: "input", ... })
            └─ T3Transport.refreshDeviceRecords(serial, entryType)  ← Action 17
                 └─ POST /api/ffi  (HTTP to Rust)
                      └─ T3000FfiApiService.call_ffi(message)
                           └─ call_handle_webview_msg(17, buffer)  ← C++ FFI
```

This is the shared core path. ALL point types (inputs, outputs, variables, programs, schedules, PIDs, alarms, trendlogs, graphics) go through `PanelDataRefreshService` → `T3Transport` → `POST /api/ffi`.

2c. Background sync path

```
Tokio timer fires every N seconds
  └─ FfiSyncService::run_sync_cycle()
       ├─ get_panels_list_via_ffi()          ← Action 4
       └─ per device: sync_device(serial)
            ├─ get_logging_data()            ← Action 15 (background only)
            └─ write inputs/outputs/variables to SQLite/MSSQL
```

2d. Existing T3_APP_LOG schema (confirmed from migration SQL)

```SQL
CREATE TABLE IF NOT EXISTS T3_APP_LOG (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    ts_unix  INTEGER,
    ts_fmt   TEXT,
    level    TEXT,
    category TEXT,
    source   TEXT,
    hostname TEXT,
    device_serial TEXT,
    message  TEXT,
    details  TEXT
);
```

This table stays unchanged. New tables are additive.

---

3. New database tables

All stored in `webview_t3_device.db` (local SQLite) only. Never written to MSSQL.

Table: `T3_FLOW`

One row per flow instance. Created when a flow starts.

```SQL
CREATE TABLE IF NOT EXISTS T3_FLOW (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id      TEXT    NOT NULL UNIQUE,   -- UUID v4, generated at flow start
    flow_type    TEXT    NOT NULL,          -- see section 4 for valid values
    trigger_src  TEXT    NOT NULL,          -- "startup" | "scheduler" | "user" | "api" | "ws"
    started_at   INTEGER NOT NULL,          -- unix epoch ms
    ended_at     INTEGER,                   -- null until done() called
    status       TEXT    NOT NULL DEFAULT 'running', -- "running"|"ok"|"partial"|"failed"
    hostname     TEXT,
    total_steps  INTEGER NOT NULL DEFAULT 0, -- expected step count set at start
    done_steps   INTEGER NOT NULL DEFAULT 0,
    error_count  INTEGER NOT NULL DEFAULT 0,
    meta         TEXT                       -- optional JSON: e.g. {"device_serial":"12345"}
);

CREATE INDEX IF NOT EXISTS idx_t3_flow_type    ON T3_FLOW (flow_type);
CREATE INDEX IF NOT EXISTS idx_t3_flow_started ON T3_FLOW (started_at DESC);
CREATE INDEX IF NOT EXISTS idx_t3_flow_status  ON T3_FLOW (status);
```

Table: `T3_FLOW_STEP`

One row per step. Multiple rows per flow.

```sql
CREATE TABLE IF NOT EXISTS T3_FLOW_STEP (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id     TEXT    NOT NULL,   -- FK → T3_FLOW.flow_id
    seq         INTEGER NOT NULL,   -- step order: 1, 2, 3…
    step_name   TEXT    NOT NULL,   -- "dll_load" | "ffi_call" | "db_write" etc.
    level       TEXT    NOT NULL DEFAULT 'info', -- "info"|"warn"|"error"
    source      TEXT,               -- rust module name or react component
    api_path    TEXT,               -- optional: HTTP path e.g. "/api/ffi"
    action_type INTEGER,            -- optional: FFI action number e.g. 4, 15, 17
    status      TEXT    NOT NULL DEFAULT 'ok', -- "ok"|"skip"|"fail"
    duration_ms INTEGER,            -- elapsed time for this step
    payload_ref TEXT,               -- NULL or relative file path if payload offloaded
    message     TEXT,
    details     TEXT,               -- short summary; NULL if large payload offloaded to file
    ts_unix     INTEGER NOT NULL,
    ts_fmt      TEXT    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_t3_flow_step_flow ON T3_FLOW_STEP (flow_id);
CREATE INDEX IF NOT EXISTS idx_t3_flow_step_ts   ON T3_FLOW_STEP (ts_unix DESC);
```

Table: `T3_FLOW_PAYLOAD`

Tracks offloaded large payload files. Only created when `details` exceeds the threshold.

```sql
CREATE TABLE IF NOT EXISTS T3_FLOW_PAYLOAD (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    flow_id     TEXT    NOT NULL,
    step_id     INTEGER NOT NULL,   -- FK → T3_FLOW_STEP.id
    file_path   TEXT    NOT NULL,   -- relative: "T3WebLog/payloads/YYYY-MM/flowid_seq.json"
    size_bytes  INTEGER NOT NULL,
    created_at  INTEGER NOT NULL,
    purged      INTEGER NOT NULL DEFAULT 0  -- 0=exists, 1=file deleted
);
```

---

4. Flow types and their steps

DLL_INIT

Triggered once at service startup. Source: Rust.

seq	step_name	source	what it records
1	dll_search	ffi_init	paths tried for T3000.exe
2	dll_load	ffi_init	LoadLibraryA result
3	fn_bind_bacnet	ffi_init	BacnetWebView_HandleWebViewMsg bound ok/fail
4	fn_bind_settings	ffi_init	GetDeviceBasicSettings bound ok/fail
5	db_connect_sqlite	server	local SQLite opened
6	db_connect_mssql	server	MSSQL pool created or skipped (standalone)
7	services_start	server	HTTP 9103, 9104, WebSocket started
8	schema_migration	server	migration check result
9	init_done	server	total duration, any errors

SYNC_CYCLE

Triggered by background scheduler. Source: Rust. One flow per cycle.

seq	step_name	source	what it records
1	cycle_start	ffi_sync	interval_secs, timestamp
2	writer_selected	ffi_sync	target: center_mssql / local_sqlite
3	rediscover_check	ffi_sync	should GET_PANELS_LIST run? yes/no + reason
4	get_panels_list	ffi_sync	action=4, attempt N/M, result count, duration
5	device_sync_{serial}	ffi_sync	per device: ok/fail, points written, duration
6	trendlog_sync	ffi_sync	records written, duration
7	cycle_done	ffi_sync	N/M devices ok, total duration, error_count

Note: step 5 repeats once per device. `meta` on the flow carries `{"total_devices": N}`.

INPUTS_LOAD (and OUTPUTS_LOAD, VARIABLES_LOAD)

Triggered by user page load or refresh button. Spans React + Rust.

seq	step_name	source	what it records
1	page_open	React/InputsPage	device_serial, trigger: "mount"|"refresh"
2	api_dispatch	React/PanelDataRefreshService	action=17, entryType=1, index range
3	ffi_received	t3_ffi_api	message parsed, action confirmed
4	ffi_call	t3_ffi_api	call_handle_webview_msg(17), duration
5	ffi_response	t3_ffi_api	bytes received (payload offloaded if > 4096)
6	db_write	t3_ffi_api	records saved to SQLite, count, duration
7	ui_render	React/InputsPage	items rendered, duration

MESSAGE_ACTION (generic user-triggered FFI call)

Triggered by any React component sending a message to `/api/ffi`. Used when no dedicated flow type applies.

seq	step_name	source	what it records
1	action_received	t3_ffi_api	action number, device_serial
2	ffi_call	t3_ffi_api	call_handle_webview_msg(N), duration
3	ffi_response	t3_ffi_api	bytes, status (payload offloaded if large)
4	db_write	t3_ffi_api	optional: if response triggers a save

CONFIG_CHANGE

Triggered when user saves any setting.

seq	step_name	source	what it records
1	user_submit	React	form name, fields changed (no values — only keys)
2	api_received	config_api	endpoint, payload size
3	validate	config_api	pass/fail, reason
4	db_write	config_api	APPLICATION_CONFIG rows updated
5	history_write	config_api	APPLICATION_CONFIG_HISTORY row
6	service_notify	server	which runtime service acknowledged

USER_AUTH

Triggered on login/logout.

seq	step_name	source
1	login_attempt	auth
2	credential_check	auth
3	session_create	auth

---

5. Rust FlowHandle design

New file: `api/src/logging/flow.rs`

Struct

RUST

```rust
pub struct FlowHandle {
    pub flow_id: String,          // UUID
    pub flow_type: String,
    db: DatabaseConnection,
    step_counter: AtomicU32,      // auto-increments; caller does not manage seq manually
    payload_threshold: usize,     // default 4096 bytes
}
```

Public API

RUST

```rust
impl FlowHandle {
    /// Create a new flow. Inserts one T3_FLOW row immediately.
    pub async fn start(
        db: &DatabaseConnection,
        flow_type: &str,
        trigger_src: &str,
        total_steps: u32,
        meta: Option<&str>,        // optional JSON string
    ) -> FlowHandle;

    /// Record one step. Inserts T3_FLOW_STEP row.
    /// If `details` > payload_threshold bytes → offload to file, insert T3_FLOW_PAYLOAD.
    pub async fn step(
        &self,
        step_name: &str,
        level: &str,               // "info" | "warn" | "error"
        source: &str,
        status: &str,              // "ok" | "skip" | "fail"
        duration_ms: i64,
        message: &str,
        details: Option<&str>,     // large payloads auto-offloaded to file
        api_path: Option<&str>,
        action_type: Option<i32>,
    );

    /// Mark flow finished. Updates T3_FLOW row: status, ended_at, done_steps, error_count.
    pub async fn done(&self, status: &str);

    /// Attach to an existing flow created by another caller (used for cross-boundary flows).
    pub async fn resume(db: &DatabaseConnection, flow_id: &str) -> Option<FlowHandle>;
}
```

Payload offload logic (inside step())

TEXT

```
if details.len() > payload_threshold:
    dir  = T3000_RUNTIME/T3WebLog/payloads/YYYY-MM/
    file = {flow_id}_{seq}.json
    write file to disk
    insert T3_FLOW_PAYLOAD row
    store file path in T3_FLOW_STEP.payload_ref
    set  T3_FLOW_STEP.details = NULL
else:
    store details inline in T3_FLOW_STEP.details
    no file written
```

---

6. Cross-boundary flows (React + Rust sharing one flow_id)

For flows that span React and Rust (INPUTS_LOAD, CONFIG_CHANGE, MESSAGE_ACTION):

1. React generates a UUID at the moment the user action triggers.
2. React sends it with every HTTP request as a custom header: `X-Flow-Id: <uuid>`.
3. Rust API handler extracts it and calls `FlowHandle::resume(db, flow_id)`.
4. Rust records its steps under the same `flow_id`.
5. React records its own steps by calling `POST /api/flows/{flow_id}/client-step`.

React flow step payload:

```JSON
{
  "seq": 1,
  "step_name": "page_open",
  "source": "InputsPage",
  "status": "ok",
  "duration_ms": 0,
  "message": "User opened Inputs page",
  "details": null
}
```

For backend-only flows (DLL_INIT, SYNC_CYCLE) the `X-Flow-Id` header is never involved. Rust owns the full flow from start to done.

---

7. New API endpoints

New router: `api/src/logging/flow_api.rs`

```TEXT
GET  /api/flows
     ?type=DLL_INIT&status=failed&from=1716000000&to=1716086400&limit=50&page=0

GET  /api/flows/:flow_id
     → returns T3_FLOW row + all T3_FLOW_STEP rows for that flow

GET  /api/flows/:flow_id/payload/:step_id
     → streams the payload file if it exists and is not purged

POST /api/flows/:flow_id/client-step
     → React records a client-side step under an existing flow

GET  /api/flows/types
     → returns distinct flow_type values with counts

POST /api/flows/purge
     body: { "older_than_days": 30 }
     → deletes T3_FLOW + T3_FLOW_STEP rows and payload files older than N days
```

All read from local SQLite. No MSSQL interaction.

---

8. Payload cleanup

Cleanup runs on two triggers:

- Scheduled: once every 24 hours inside the existing background task loop.
- Manual: user clicks Purge on the Flow Log page.

Config keys in `APPLICATION_CONFIG`:

- `flow_log.retention_days` — default 30
- `flow_log.payload_size_threshold` — default 4096 (bytes)
- `flow_log.enabled` — default true (disable per flow_type: `flow_log.enabled.DLL_INIT` etc.)

Cleanup steps:

1. Query `T3_FLOW_PAYLOAD WHERE created_at < cutoff AND purged = 0`.
2. For each row: delete file from disk, set `purged = 1`.
3. Delete `T3_FLOW_STEP` and `T3_FLOW` rows older than cutoff (cascade).

4. Keep `T3_FLOW_PAYLOAD` rows with `purged = 1` for N more days as audit trail, then delete.

The `T3_FLOW` table is capped at 10,000 rows via a trim trigger (same pattern as T3_APP_LOG's 5,000 row cap) to prevent unbounded growth even if scheduled cleanup fails.

---

9. New UI page

Location and navigation

`src/t3-react/features/logs/pages/FlowLogPage.tsx`

Added as a new tab in the existing Logs page header:

```
TEXT

[ Activity Log ]  [ Flow Log ]  [ File Logs ]  [ Settings ]
```

Layout

```
TEXT

┌──────────────────────────────────────────────────────────────────┐
│  Flow Log          [Type ▼] [Status ▼] [Date range ▼]  [Refresh] [Purge]│
├──────────────────────────────────────────────────────────────────┤
│  Flow list (sortable table)                                      │
│  Short ID  | Type         | Trigger   | Started     | Duration | Status  │
│  ──────────┼──────────────┼───────────┼─────────────┼──────────┼──────── │
│  a3f1…     | DLL_INIT     | startup   | 02:58:00    | 1.2s     | ✅ ok   │
│  b2e8…     | SYNC_CYCLE   | scheduler | 02:58:31    | 8.4s     | ⚠ partial│
│  c9d2…     | INPUTS_LOAD  | user      | 03:29:49    | 0.3s     | ✅ ok   │
│  ──────────┼──────────────┼───────────┼─────────────┼──────────┼──────── │
│                                                                  │
│  ▼ b2e8… SYNC_CYCLE details  (expanded on row click)             │
│  ┌──────────────────────────────────────────────────────────┐    │
│  | # | Step name        | Source   | Status | Duration | Message      |    │
│  | 1 | cycle_start      | ffi_sync | ✅     | 0ms      | interval=30s |    │
│  | 2 | writer_selected  | ffi_sync | ✅     | 1ms      | center_mssql |    │
│  | 3 | get_panels_list  | ffi_sync | ⚠      | 32010ms  | attempt 1/2  |    │
│  | 4 | get_panels_list  | ffi_sync | ✅     | 1230ms   | 7 panels     |    │
│  | 5 | device_sync_1234 | ffi_sync | ✅     | 410ms    | 320 pts wr.  |    │
│  | 5 | device_sync_5678 | ffi_sync | ❌     | 45013ms  | timeout      |    │
│  | 6 | cycle_done       | ffi_sync | ⚠      | –        | 6/7 ok       |    │
│  |   |                  |          |        | [View payload ↗]        |    │
│  └──────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

Interactions

- Click any flow row → expand inline step table
- Click "View payload" on a step → open JSON viewer drawer (streams from `/api/flows/{id}/payload/{step_id}`)
- Purge button → modal with retention config, then calls `POST /api/flows/purge`
- Export button → download full flow as JSON (client-side serialise from already-loaded data)
- Type/Status/Date filters are applied via query params to the API

---

10. What is NOT changed

The following are explicitly left untouched by this design:

Item	Reason
`T3_APP_LOG` table	Still used by existing ActivityLog/sync health system
All `emit_app_log()` call sites	No changes to existing log emission
`ActivityLogTab` component	Still reads from `/api/sync/event-log`
`FileLogsTab`, `LogSettingsTab`	No changes
MSSQL mirror logic in `sinks.rs`	Unchanged for existing categories
`call_handle_webview_msg()` core FFI	Never modified — callers log, not the core
`T3000FfiApiService::call_ffi()`	Never modified — callers log, not the service

---

11. Implementation order

1. **Migration**: Add `T3_FLOW`, `T3_FLOW_STEP`, `T3_FLOW_PAYLOAD` to `webview_t3_device_schema.sql` + migration runner.
2. **FlowHandle**: `api/src/logging/flow.rs` — start / step / done / resume.
3. **Flow API router**: `api/src/logging/flow_api.rs` — GET flows, GET flow/{id}, POST client-step, POST purge.
4. **Wire into server**: add flow router to `create_t3_app()` in `server.rs`.
5. **Instrument DLL_INIT**: wrap startup in `t3_ffi_api_service.rs` and `main.rs`.
6. **Instrument SYNC_CYCLE**: wrap `run_sync_cycle()` in `t3_ffi_sync_service.rs`.
7. **X-Flow-Id middleware**: extract header in API handler, pass to FlowHandle::resume.
8. **Instrument INPUTS_LOAD** (and OUTPUTS/VARIABLES): wrap `t3_ffi_api_service::call_ffi` caller.
9. **React client-step hook**: small `useFlowLog(flowId)` hook that sends steps to `/api/flows/{id}/client-step`.
10. **FlowLogPage.tsx**: new React page with flow list + step drill-down.
11. **Cleanup scheduler**: wire 24h purge into existing background task in `server.rs`.
12. **Instrument remaining flows**: CONFIG_CHANGE, USER_AUTH, MESSAGE_ACTION.

T3000 Mobile Architecture Guide

Version: 1.0 **Date:** January 29, 2026 **Status:** ⏸️ Planning Phase - Not Started **Last Updated:** January 29, 2026

🚀 Quick Start (Resume After Break)

Haven't worked on this in a while? Start here:

1. **Check Current Status** → See Project Status below
2. **Review What's Done** → Check completed tasks in status section
3. **Find Next Task** → See unchecked items in status section
4. **Read Relevant Section** → Jump to that section number below
5. **Follow Implementation Steps** → Section 7 has detailed checklist
6. **Test** → Section 8 has testing checklist

Critical Rules (Don't Skip):

- ✅ ALWAYS test desktop after any change
- ✅ NEVER move files until mobile is working
- ✅ READ Common Pitfalls (Section 9) before coding

📊 Project Status

Current Phase: ⏸️ Phase 1 - Proof of Concept (NOT STARTED)

Completed Tasks:

- [x] Architecture planning
- [x] Documentation created
- [ ] Shared folder structure created
- [ ] Business logic extracted (Inputs)
- [ ] Mobile components created
- [ ] Device detection implemented
- [ ] View router created
- [ ] Testing completed

Next Immediate Task: → **Create folder structure** (See Section 7, Step 1)

Files Changed So Far: None (Desktop code unchanged)

Estimated Completion: Not started yet

📋 Table of Contents

1. Overview
2. Current Architecture
3. Target Architecture
4. Migration Strategy
5. Folder Structure
6. Code Patterns
7. Implementation Steps
8. Testing Checklist
9. Common Pitfalls

---

1. Overview

Goal

Add mobile-responsive UI to T3000 WebView while keeping desktop version working without changes.

Approach

Incremental Migration - Build mobile UI separately, gradually extract shared business logic, leave desktop unchanged.

Principles

- ✅ **Zero Risk to Desktop** - Desktop code stays working throughout
- ✅ **Shared Business Logic** - One source of truth for data/API calls
- ✅ **Separate UI Layers** - Desktop table vs Mobile card views
- ✅ **Gradual Migration** - One feature at a time, test thoroughly

---

2. Current Architecture

Project Structure (Before Migration)

TEXT

```
src/
├── shared/                              [Root-level shared - Vue ↔ React]
│   ├── api/client.ts                   (Axios client - UNUSED by React)
│   ├── auth/authService.ts             (Auth service - UNUSED by React)
│   ├── state/sharedState.ts            (Cross-framework state - UNUSED)
│   └── routes.ts                       (Route detection - USED)
│
├── t3-vue/                              [Vue 3 + Quasar app]
│   └── ... (Existing Vue app - UNCHANGED)
│
└── t3-react/                            [React + FluentUI Desktop App]
    ├── app/
    │   ├── App.tsx                     (Main app component)
    │   ├── main.tsx                    (React entry point)
    │   └── router/                     (React Router setup)
    │
    ├── config/
    │   └── constants.ts                ⚠️ API_BASE_URL (51+ files depend on this)
    │
    ├── services/                        ⚠️ Core services (should be shared)
    │   ├── deviceApi.ts
    │   ├── fileMenuService.ts
    │   ├── toolsMenuService.ts
    │   └── ... (9 total)
    │
    ├── store/                           ⚠️ Global stores (should be shared)
    │   ├── authStore.ts
    │   ├── statusBarStore.ts
    │   ├── treeStore.ts
    │   └── uiStore.ts
    │
    ├── shared/                          [React-specific shared code]
    │   ├── components/                 (12 UI components)
    │   ├── hooks/                      (24 hooks)
    │   ├── services/                   (panelDataRefreshService)
    │   ├── types/                      (TypeScript types)
    │   └── utils/                      (Utilities)
    │
    ├── features/                        [27 feature modules]
    │   ├── inputs/
    │   │   ├── components/             (Desktop UI)
    │   │   ├── data/                   (rangeData.ts)
    │   │   ├── pages/
    │   │   │   └── InputsPage.tsx      (Desktop page - 1411 lines)
    │   │   └── services/
    │   │       └── inputRefreshApi.ts
    │   ├── outputs/                    (Same structure)
    │   ├── variables/                  (Same structure)
    │   └── ... (24 more features)
    │
    └── layout/                          [Desktop-only layouts]
        ├── MainLayout.tsx              (TreePanel + Header + StatusBar)
        ├── Header.tsx
        ├── TreePanel.tsx
        └── StatusBar.tsx
```

Key Dependencies

`src/t3-react/config/constants.ts` imported by:

- 9 files in `services/*`
- 2 files in `shared/hooks/*`
- 1 file in `shared/components/*`
- 29 files in `features/**/pages/*`
- 8 files in `features/**/services/*`
- 1 file in `features/**/store/*`

Total: 51+ files depend on config/constants.ts

Current Import Patterns

TYPESCRIPT

```typescript
// Services import from config
import { API_BASE_URL } from '../config/constants';

// Features import from config
import { API_BASE_URL } from '../../../config/constants';

// Shared imports from config
import { API_BASE_URL } from '../../config/constants';

// Features import from shared
import { PanelDataRefreshService } from '../../../shared/services/panelDataRefreshService';
```

---

3. Target Architecture

Final Structure (After Complete Migration)

TEXT

```
src/
├── shared/                              [Platform-Agnostic Core]
│   ├── features/                       [Business Logic by Feature]
│   │   ├── inputs/
│   │   │   ├── hooks/
│   │   │   │   └── useInputsPage.ts   ← ALL page logic
│   │   │   ├── services/
│   │   │   │   ├── inputApi.ts        ← API calls
│   │   │   │   └── inputRefreshApi.ts ← Refresh logic
│   │   │   ├── stores/
│   │   │   │   └── inputStore.ts      ← State management (if needed)
│   │   │   ├── types/
│   │   │   │   └── input.types.ts     ← TypeScript interfaces
│   │   │   └── data/
│   │   │       └── rangeData.ts       ← Static data
│   │   │
│   │   ├── outputs/                   ← Same pattern
│   │   ├── variables/                 ← Same pattern
│   │   └── ... (27 features)
│   │
│   ├── core/                           [Core Infrastructure]
│   │   ├── api/
│   │   │   └── apiClient.ts           ← HTTP client
│   │   ├── config/
│   │   │   └── constants.ts           ← API_BASE_URL (from t3-react/config)
│   │   ├── services/                   ← Core services (from t3-react/services)
│   │   │   ├── deviceApi.ts
│   │   │   ├── fileMenuService.ts
│   │   │   ├── panelDataRefreshService.ts
│   │   │   └── ... (all services)
│   │   ├── stores/                     ← Global stores (from t3-react/store)
│   │   │   ├── authStore.ts
│   │   │   ├── statusBarStore.ts
│   │   │   ├── treeStore.ts
│   │   │   └── uiStore.ts
│   │   └── hooks/
│   │       ├── useDeviceType.ts       ← Detect device (desktop/mobile/tablet)
│   │       └── useBreakpoint.ts       ← Viewport detection
│   │
│   ├── components/                     [Shared UI Components]
│   │   └── ... (from t3-react/shared/components)
│   │
│   ├── hooks/                          [Shared Hooks]
│   │   └── ... (from t3-react/shared/hooks)
│   │
│   ├── types/                          [Shared Types]
│   │   └── ... (from t3-react/shared/types)
│   │
│   └── utils/                          [Utilities]
│       └── ... (from t3-react/shared/utils)
│
├── t3-react/                           [Desktop UI ONLY]
│   ├── features/
│   │   └── inputs/
│   │       ├── components/
```

```
│   │       │   ├── InputTable.tsx     ← Desktop table component
│   │       │   └── RangeSelectionDrawer.tsx
│   │       └── pages/
│   │           └── InputsPage.tsx     ← Uses shared/features/inputs/hooks
│   │
│   ├── layout/
│   │   ├── MainLayout.tsx             ← Desktop layout
│   │   ├── Header.tsx
│   │   ├── TreePanel.tsx
│   │   └── StatusBar.tsx
│   │
│   ├── components/                     ← Desktop-specific UI
│   │   └── ...
│   │
│   └── app/
│       ├── App.tsx
│       └── main.tsx
│
├── t3-mobile/                          [Mobile UI ONLY]
│   ├── features/
│   │   └── inputs/
│   │       ├── components/
│   │       │   └── InputCard.tsx      ← Mobile card component
│   │       └── pages/
│   │           └── InputsPageMobile.tsx ← Uses SAME shared hook
│   │
│   ├── layout/
│   │   ├── MobileLayout.tsx           ← Mobile layout
│   │   ├── BottomNavigation.tsx
│   │   └── MobileAppBar.tsx
│   │
│   ├── components/                     ← Mobile-specific UI
│   │   ├── MobileCard/
│   │   ├── BottomSheet/
│   │   ├── SwipeableList/
│   │   └── FloatingActionButton/
│   │
│   └── app/
│       ├── MobileApp.tsx
│       └── main.tsx
│
└── t3-vue/                             [Vue App - UNCHANGED]
    └── ...
```

New Import Patterns

TYPESCRIPT

```typescript
// Desktop page uses shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Mobile page uses SAME hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Both import from shared config
import { API_BASE_URL } from '../../../../shared/core/config/constants';

// Both use shared services
import { deviceApi } from '../../../../shared/core/services/deviceApi';
```

---

4. Migration Strategy

Phase 1: Proof of Concept (Week 1)

Goal: Create ONE mobile page (Inputs) without touching desktop code.

Step 1.1: Extract Business Logic

```shell
## Create shared/features/inputs/
src/shared/features/inputs/hooks/useInputsPage.ts

## Extract from InputsPage.tsx:
- All useState declarations
- All API calls
- All useEffect hooks
- All event handlers
- Return all state + methods
```

Step 1.2: Create Mobile UI

```shell
## Create mobile components
src/t3-mobile/components/MobileCard/MobileCard.tsx
src/t3-mobile/layout/MobileLayout.tsx

## Create mobile page
src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx
```

Step 1.3: Device Detection

```shell
## Create device detection hook
src/shared/core/hooks/useDeviceType.ts

## Returns: 'desktop' | 'mobile' | 'tablet'
```

Step 1.4: Routing

```shell
## Update router to detect device
src/shared/core/router/ViewRouter.tsx

## Route to desktop or mobile version based on viewport
```

Step 1.5: Test

- ✅ Desktop InputsPage still works (unchanged)
- ✅ Mobile InputsPageMobile renders on small viewport
- ✅ Both use same data (shared hook)
- ✅ Both can fetch/save data

Phase 2: Expand Mobile Pages (Week 2-3)

Repeat Phase 1 for:

1. Outputs
2. Variables

3. Devices/Discover
4. Trendlogs
5. Schedules

Phase 3: Refactor Desktop (Optional - Week 4+)

Once mobile is working, optionally refactor desktop to use shared hooks:

TYPESCRIPT

```typescript
// OLD Desktop Code (before)
const InputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... 1411 lines of logic
};

// NEW Desktop Code (after - optional)
const InputsPage = () => {
  const logic = useInputsPage(); // Use shared hook

  return <DesktopTable items={logic.inputs} />;
};
```

---

5. Folder Structure

Detailed File Layout

TEXT

```
src/shared/features/inputs/
├── hooks/
│   └── useInputsPage.ts              [ALL business logic]
├── services/
│   ├── inputApi.ts                   [CRUD operations]
│   └── inputRefreshApi.ts            [Refresh from device]
├── types/
│   └── input.types.ts                [TypeScript interfaces]
└── data/
    └── rangeData.ts                  [Static range definitions]
```

TEXT

```
src/t3-mobile/features/inputs/
├── components/
│   ├── InputCard.tsx                 [Mobile card view]
│   └── InputCard.module.css
└── pages/
    ├── InputsPageMobile.tsx          [Mobile page]
    └── InputsPageMobile.module.css
```

TEXT

```
src/t3-mobile/components/
├── MobileCard/
│   ├── MobileCard.tsx               [Generic card component]
│   └── MobileCard.module.css
├── BottomSheet/
│   ├── BottomSheet.tsx              [Modal for mobile]
│   └── BottomSheet.module.css
└── FloatingActionButton/
    ├── FloatingActionButton.tsx     [FAB for actions]
    └── FloatingActionButton.module.css
```

TEXT

```
src/shared/core/hooks/
├── useDeviceType.ts                 [Device detection]
└── useBreakpoint.ts                 [Viewport size detection]
```

---

6. Code Patterns

Pattern 1: Shared Business Logic Hook

File: `src/shared/features/inputs/hooks/useInputsPage.ts`

TYPESCRIPT

```typescript
import { useState, useEffect, useCallback, useRef } from 'react';
import { useDeviceTreeStore } from '../../../../t3-react/features/devices/store/deviceTreeStore';
import { API_BASE_URL } from '../../../core/config/constants';
import { InputPoint } from '../types/input.types';

/**
 * Shared business logic for Inputs page
 * Used by both Desktop and Mobile versions
 */
export const useInputsPage = () => {
  // ============================================
  // STATE - Extracted from InputsPage.tsx
  // ============================================
  const { selectedDevice } = useDeviceTreeStore();
  const [inputs, setInputs] = useState<InputPoint[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [refreshing, setRefreshing] = useState(false);
  const [editingCell, setEditingCell] = useState<{
    id: string;
    field: string;
  } | null>(null);
  const [editValue, setEditValue] = useState('');

  // ============================================
  // METHODS - Extracted from InputsPage.tsx
  // ============================================

  /**
   * Fetch inputs from backend
   */
  const fetchInputs = useCallback(async () => {
    if (!selectedDevice) {
      setInputs([]);
      return;
    }

    setLoading(true);
    setError(null);

    try {
      const url = `${API_BASE_URL}/api/t3_device/devices/${selectedDevice.serialNumber}/input-points`;
      const response = await fetch(url);

      if (!response.ok) {
        throw new Error(`Failed to fetch inputs: ${response.statusText}`);
      }

      const data = await response.json();
      setInputs(data.input_points || []);
    } catch (err) {
      const errorMessage = err instanceof Error ? err.message : 'Failed to load inputs';
      setError(errorMessage);
      console.error('Error fetching inputs:', err);
    } finally {
      setLoading(false);
    }
  }, [selectedDevice]);
```

```
  /**
   * Refresh single input from device
   */
  const refreshSingleInput = async (index: number) => {
    // ... refresh logic
  };

  /**
   * Save input value
   */
  const saveInput = async (id: string, field: string, value: any) => {
    // ... save logic
  };

  // ============================================
  // EFFECTS - Extracted from InputsPage.tsx
  // ============================================

  useEffect(() => {
    fetchInputs();
  }, [selectedDevice, fetchInputs]);

  // ============================================
  // RETURN - Everything the UI needs
  // ============================================

  return {
    // State
    inputs,
    loading,
    error,
    refreshing,
    editingCell,
    editValue,
    selectedDevice,

    // Methods
    fetchInputs,
    refreshSingleInput,
    saveInput,
    setEditingCell,
    setEditValue,
  };
};
```

Pattern 2: Desktop Page Using Shared Hook

File: `src/t3-react/features/inputs/pages/InputsPage.tsx`

TYPESCRIPT

```typescript
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { DataGrid } from '@fluentui/react-components';

/**
 * Desktop Inputs Page - Table View
 */
export const InputsPage = () => {
  const logic = useInputsPage(); // ← Use shared hook

  if (logic.loading) {
    return <Spinner />;
  }

  return (
    <div className={styles.container}>
      <Toolbar onRefresh={logic.fetchInputs} />

      <DataGrid>
        {logic.inputs.map((input) => (
          <DataGridRow key={input.inputId}>
            <DataGridCell>{input.label}</DataGridCell>
            <DataGridCell>{input.fValue}</DataGridCell>
            {/* ... more cells */}
          </DataGridRow>
        ))}
      </DataGrid>
    </div>
  );
};
```

Pattern 3: Mobile Page Using SAME Hook

File: `src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx`

TYPESCRIPT

```typescript
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { MobileCard } from '../../../components/MobileCard/MobileCard';

/**
 * Mobile Inputs Page - Card List View
 */
export const InputsPageMobile = () => {
  const logic = useInputsPage(); // ← SAME hook as desktop!

  if (logic.loading) {
    return <Spinner />;
  }

  return (
    <div className={styles.mobileContainer}>
      <MobileAppBar title="Inputs" onRefresh={logic.fetchInputs} />

      <div className={styles.cardList}>
        {logic.inputs.map((input) => (
          <MobileCard
            key={input.inputId}
            title={input.label}
            value={input.fValue}
            unit={input.units}
            status={input.status}
            onTap={() => handleCardTap(input)}
          />
        ))}
      </div>
    </div>
  );
};
```

Pattern 4: Device Detection Hook

File: `src/shared/core/hooks/useDeviceType.ts`

TYPESCRIPT

```typescript
import { useState, useEffect } from 'react';

export type DeviceType = 'desktop' | 'mobile' | 'tablet';

/**
 * Detect device type based on viewport width
 */
export const useDeviceType = (): DeviceType => {
  const [deviceType, setDeviceType] = useState<DeviceType>('desktop');

  useEffect(() => {
    const detectDevice = () => {
      const width = window.innerWidth;

      if (width < 768) {
        setDeviceType('mobile');
      } else if (width < 1024) {
        setDeviceType('tablet');
      } else {
        setDeviceType('desktop');
      }
    };

    // Detect on mount
    detectDevice();

    // Detect on resize
    window.addEventListener('resize', detectDevice);
    return () => window.removeEventListener('resize', detectDevice);
  }, []);

  return deviceType;
};
```

Pattern 5: View Router

File: `src/shared/core/router/ViewRouter.tsx`

TYPESCRIPT

```typescript
import { useDeviceType } from '../hooks/useDeviceType';
import { InputsPage } from '../../../t3-react/features/inputs/pages/InputsPage';
import { InputsPageMobile } from '../../../t3-mobile/features/inputs/pages/InputsPageMobile';

/**
 * Route to desktop or mobile version based on device
 */
export const InputsRoute = () => {
  const deviceType = useDeviceType();

  return deviceType === 'mobile'
    ? <InputsPageMobile />
    : <InputsPage />;
};
```

---

7. Implementation Steps

Step-by-Step Checklist

✅ Step 1: Create Shared Folder Structure

```shell
mkdir -p src/shared/features/inputs/hooks
mkdir -p src/shared/features/inputs/services
mkdir -p src/shared/features/inputs/types
mkdir -p src/shared/features/inputs/data
mkdir -p src/shared/core/hooks
```

✅ Step 2: Create Mobile Folder Structure

```shell
mkdir -p src/t3-mobile/features/inputs/components
mkdir -p src/t3-mobile/features/inputs/pages
mkdir -p src/t3-mobile/components/MobileCard
mkdir -p src/t3-mobile/layout
mkdir -p src/t3-mobile/app
```

✅ Step 3: Extract Types

```shell
## Create src/shared/features/inputs/types/input.types.ts
## Copy InputPoint interface from InputsPage.tsx
```

✅ Step 4: Extract Business Logic

```shell
## Create src/shared/features/inputs/hooks/useInputsPage.ts
## Extract from InputsPage.tsx:
1. All useState declarations
2. All API calls (fetchInputs, saveInput, etc.)
3. All useEffect hooks
4. All event handlers
5. Return object with state + methods
```

✅ Step 5: Create Device Detection

```shell
## Create src/shared/core/hooks/useDeviceType.ts
## Implement viewport-based detection
```

✅ Step 6: Create Mobile Components

```shell
## Create src/t3-mobile/components/MobileCard/MobileCard.tsx
## Create src/t3-mobile/layout/MobileLayout.tsx
## Create src/t3-mobile/layout/MobileAppBar.tsx
```

✅ Step 7: Create Mobile Page

```shell
## Create src/t3-mobile/features/inputs/pages/InputsPageMobile.tsx
## Import useInputsPage hook
## Render card list view
```

✅ Step 8: Create View Router

```shell
## Create src/shared/core/router/ViewRouter.tsx
## Detect device and route to desktop or mobile
```

✅ Step 9: Update App Router

```shell
## Update src/t3-react/app/router/routes.ts
## Use ViewRouter for /inputs route
```

✅ Step 10: Test

```shell
## Desktop: Resize browser to > 1024px → Should show desktop table
## Mobile: Resize browser to < 768px → Should show mobile cards
## Both should fetch same data
## Both should be able to save changes
```

---

8. Testing Checklist

Desktop Testing (After Migration)

- [ ] Desktop page loads without errors
- [ ] Data fetches correctly
- [ ] Table displays all columns
- [ ] Sorting works
- [ ] Filtering works
- [ ] Inline editing works
- [ ] Save button works
- [ ] Refresh button works
- [ ] Device selection works
- [ ] Auto-refresh works
- [ ] No console errors
- [ ] No TypeScript errors

Mobile Testing

- [ ] Mobile page loads on small viewport
- [ ] Data fetches correctly
- [ ] Cards display all info
- [ ] Tap to expand works
- [ ] Edit modal opens
- [ ] Save works
- [ ] Refresh works (pull-to-refresh)
- [ ] Bottom navigation works
- [ ] No console errors
- [ ] No TypeScript errors

Cross-Platform Testing

- [ ] Desktop → Mobile resize works
- [ ] Mobile → Desktop resize works
- [ ] Data is consistent between views
- [ ] Changes saved on desktop appear on mobile
- [ ] Changes saved on mobile appear on desktop
- [ ] Multiple devices sync correctly

---

9. Common Pitfalls

❌ Pitfall 1: Breaking Desktop Imports

Problem:

```typescript
// Moving files breaks existing imports
mv src/t3-react/config/constants.ts → src/shared/core/config/constants.ts
// Now 51+ files have broken imports!
```

Solution:

```shell
## DON'T move files until mobile is working
## Instead, create NEW files and gradually migrate
## Keep old files until all imports updated
```

❌ Pitfall 2: Duplicate Logic

Problem:

```typescript
// Desktop has logic
const InputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... business logic
};

// Mobile duplicates logic
const InputsPageMobile = () => {
  const [inputs, setInputs] = useState([]);
  // ... SAME business logic (BAD!)
};
```

Solution:

```typescript
// Extract to shared hook FIRST
export const useInputsPage = () => {
  const [inputs, setInputs] = useState([]);
  // ... business logic
  return { inputs, ... };
};

// Both use same hook
const InputsPage = () => {
  const logic = useInputsPage();
  return <DesktopTable items={logic.inputs} />;
};

const InputsPageMobile = () => {
  const logic = useInputsPage();
  return <MobileCards items={logic.inputs} />;
};
```

❌ Pitfall 3: Hardcoded Viewport Sizes

Problem:

```typescript
// Hardcoded breakpoints everywhere
const isMobile = window.innerWidth < 768;
```

Solution:

```typescript
// Centralized hook
const deviceType = useDeviceType();
```

❌ Pitfall 4: Forgetting TypeScript Types

Problem:

```typescript
// Shared hook but types are in desktop code
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { InputPoint } from '../../../../t3-react/features/inputs/pages/InputsPage'; // ❌ Wrong!
```

Solution:

```typescript
// Types also in shared
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';
import { InputPoint } from '../../../../shared/features/inputs/types/input.types'; // ✅ Correct
```

❌ Pitfall 5: Not Testing Desktop After Changes

Problem:

```shell
## Create mobile page, test mobile, forget to test desktop
## Desktop breaks but you don't notice until later
```

Solution:

```shell
## Test BOTH after every change:
1. Test desktop (resize to > 1024px)
2. Test mobile (resize to < 768px)
3. Test resize between them
```

---

10. Quick Reference

File Locations

What	Desktop	Mobile	Shared
Page Component	`t3-react/features/*/pages/*.tsx`	`t3-mobile/features/*/pages/*.tsx`	N/A
Business Logic	N/A	N/A	`shared/features/*/hooks/*.ts`
API Calls	N/A	N/A	`shared/features/*/services/*.ts`
Types	N/A	N/A	`shared/features/*/types/*.ts`
UI Components	`t3-react/features/*/components/*.tsx`	`t3-mobile/features/*/components/*.tsx`	N/A
Layout	`t3-react/layout/*.tsx`	`t3-mobile/layout/*.tsx`	N/A
Config	N/A	N/A	`shared/core/config/*.ts`
Global Services	N/A	N/A	`shared/core/services/*.ts`
Global Stores	N/A	N/A	`shared/core/stores/*.ts`

Import Patterns

```typescript
// Desktop page imports shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Mobile page imports shared hook
import { useInputsPage } from '../../../../shared/features/inputs/hooks/useInputsPage';

// Hook imports from shared services
import { API_BASE_URL } from '../../../core/config/constants';

// Hook imports from feature types
import { InputPoint } from '../types/input.types';
```

Breakpoints

Device	Width	Display
Mobile	< 768px	Card list, bottom nav, FAB
Tablet	768px - 1024px	TBD (use desktop or mobile?)
Desktop	> 1024px	Table, sidebar, toolbar

---

11. Next Steps

Phase 1 Tasks (This Week)

- [ ] Create folder structure (shared/ and t3-mobile/)
- [ ] Extract useInputsPage hook
- [ ] Create MobileCard component

- [ ] Create InputsPageMobile
- [ ] Create useDeviceType hook
- [ ] Create ViewRouter
- [ ] Update app router
- [ ] Test desktop (unchanged)
- [ ] Test mobile (new)
- [ ] Document any issues

Phase 2 Tasks (Next 2-3 Weeks)

- [ ] Repeat for Outputs
- [ ] Repeat for Variables
- [ ] Repeat for Devices
- [ ] Repeat for Trendlogs
- [ ] Create mobile navigation
- [ ] Add touch gestures
- [ ] Optimize mobile performance

Phase 3 Tasks (Optional - Later)

- [ ] Refactor desktop to use shared hooks
- [ ] Move services to shared/
- [ ] Move stores to shared/
- [ ] Consolidate duplicate code
- [ ] Add tablet-specific UI

---

12. Contact & Support

Questions? Issues?

- Check this document first
- Review code examples above
- Test on both desktop and mobile before asking

Update this document when:

- Adding new patterns
- Finding new pitfalls
- Discovering better approaches
- Completing major milestones

---

13. How to Resume After a Break

If You Haven't Worked on This in Weeks/Months:

Step 1: Check Project Status

```shell
## Open this file
docs/MOBILE_ARCHITECTURE_GUIDE.md

## Look at "Project Status" section at top
## See what's completed vs what's left
```

Step 2: Verify Desktop Still Works

```shell
## Start dev server
npm run dev

## Open http://localhost:3003/#/t3000/inputs
## Test: Can you see inputs? Can you edit? Can you save?
## If YES → Desktop is fine, continue
## If NO → Something broke, check git history
```

Step 3: Check What Exists

```shell
## Check if shared folders exist
ls src/shared/features/
ls src/t3-mobile/

## If folders exist → Continue from where you left off
## If folders don't exist → Start from Step 1 in Section 7
```

Step 4: Run Tests

```shell
## Check for TypeScript errors
npm run type-check

## Check for build errors
npm run build

## If errors → Fix them before continuing
## If no errors → Good to continue
```

Step 5: Find Your Next Task

```shell
## Look at "Project Status" section
## Find first unchecked [ ] item
## Go to that section in this document
## Follow the instructions
```

If Implementation Was Interrupted:

Desktop broke?

```shell
## Rollback changes
git status
git diff
git restore <file>

## Or reset to last working commit
git log --oneline
git reset --hard <commit-hash>
```

Mobile not working?

```shell
## Mobile is separate, won't affect desktop
## Just fix the mobile code
## Desktop should still work
```

Lost track of what you were doing?

```shell
## Check git log
git log --oneline --graph --all --decorate

## Check uncommitted changes
git status
git diff

## Read commit messages to understand progress
```

Common "Resume After Break" Scenarios:

Scenario 1: Forgot what the architecture is → Re-read Sections 2, 3, 4

Scenario 2: Forgot how to extract shared hooks → Re-read Section 6, Pattern 1

Scenario 3: Desktop is broken → Check Section 9, Pitfall 1

Scenario 4: Don't remember where files should go → Check Section 10, Quick Reference table

Scenario 5: Not sure if mobile is working → Follow Section 8, Mobile Testing checklist

---

14. Maintenance & Updates

When to Update This Document:

1. **After completing a major milestone**
   - Update "Project Status" section
   - Check off completed tasks
   - Update "Last Updated" date
2. **When you discover a new pitfall**
   - Add to Section 9 (Common Pitfalls)
   - Include solution
3. **When you find a better pattern**
   - Add to Section 6 (Code Patterns)
   - Note why it's better
4. **When folder structure changes**
   - Update Section 5 (Folder Structure)
   - Update Section 10 (Quick Reference)

Version History:

Version	Date	Changes	Author
1.0	Jan 29, 2026	Initial creation, planning phase	AI + Team

Last Updated: January 29, 2026 **Version:** 1.0 **Status:** ✅ Ready to Start Phase 1

Remember: Update the "Project Status" section at the top when you complete tasks!

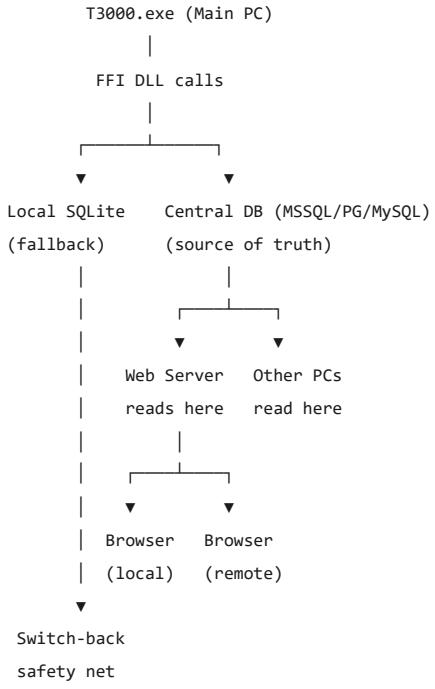
Multi-PC Data Flow Details

> Per-table data flow reference for the centralized database multi-PC architecture. See centralized-database-multipc.md for the overall design.

---

1. Data Flow Diagram — Main PC

TEXT



FFI Write Flow (Main PC)

TEXT

```
Panel scan complete → FFI callback fires
  │
  ├─ Category A-C tables (DEVICES, INPUTS, OUTPUTS, VARIABLES,
  │   PROGRAMS, SCHEDULES, PID_TABLE, HOLIDAYS,
  │   TRENDLOGS, TRENDLOG_INPUTS, TRENDLOG_DATA):
  │
  │   ├─ 1. UPSERT to local SQLite      ← always first (fast, reliable)
  │   └─ 2. UPSERT to central DB        ← dual write
  │
  └─ Category D table (TRENDLOG_DATA_DETAIL):
      │
      └─ 1. INSERT to central DB ONLY   ← no local copy (saves GB)
```

Web Server Flow (Main PC)

```
Browser request arrives
  │
  ├─ READ request for data table (Cat A-F):
  │    └─ Query central DB → return results
  │
  ├─ WRITE request for data table (Cat A-F):
  │    ├─ 1. Write to central DB
  │    └─ 2. Write to local SQLite (dual-write for fallback)
  │
  ├─ READ/WRITE for local-only table (Cat G):
  │    └─ Use local_config_conn (always local SQLite)
  │
  └─ READ TRENDLOG_DATA_DETAIL:
       └─ Query central DB (only copy exists there)
```

---

2. Data Flow Diagram — Reader PC

```
                    T3000.exe (Reader PC)
                          │
                     FFI DLL calls
                     (own panels only)
                          │
                          ▼
                   Local SQLite ONLY
                   (fallback data)

                   Central DB ◄──── Web Server reads/writes
                        │
                   ┌────┴────┐
                   ▼         ▼
                Browser   Browser
                (local)   (remote)
```

FFI Write Flow (Reader PC)

```
Panel scan complete → FFI callback fires
  │
  ├─ Category A-C tables:
  │    └─ UPSERT to local SQLite ONLY
  │         (does NOT write to central — main PC owns that)
  │
  └─ Category D (TRENDLOG_DATA_DETAIL):
       └─ INSERT to central DB ONLY
          (reader's own panels' trend data still goes to central)
```

Web Server Flow (Reader PC)

```
Browser request arrives
  │
  ├─ READ request for data table (Cat A-F):
  │   └─ Query central DB → return results
  │       (sees ALL data from all PCs)
  │
  ├─ WRITE request for data table (Cat A-F):
  │   └─ Write to central DB ONLY
  │       (reader doesn't dual-write via web — only main does)
  │
  ├─ READ/WRITE for local-only table (Cat G):
  │   └─ Use local_config_conn (always local SQLite)
  │
  └─ READ TRENDLOG_DATA_DETAIL:
      └─ Query central DB
```

---

3. Per-Table Detailed Flows

3.1 DEVICES (Category A)

Source: FFI `GET_PANELS_LIST` → DLL scans network for Temco controllers.

PC Role	FFI Action	Web Read	Web Write
Main	UPSERT local + UPSERT central	Central	Central + local
Reader	UPSERT local only	Central	Central only
Classic	UPSERT local	Local	Local

Conflict handling: UPSERT by `(serial_number)` or `(number, product_model)` — last write wins. Since each PC scans different panels, conflicts are rare.

3.2 INPUTS / OUTPUTS / VARIABLES (Category A)

Source: FFI `GET_PANEL_DATA` → reads point values from each discovered device.

PC Role	FFI Action	Web Read	Web Write
Main	UPSERT local + UPSERT central	Central	Central + local
Reader	UPSERT local only	Central	Central only
Classic	UPSERT local	Local	Local

Key: `(pid, panel_number, point_index)` — unique per panel, no cross-PC conflict.

3.3 PROGRAMS / SCHEDULES / PID_TABLE / HOLIDAYS (Category B)

Source: FFI reads these from panels along with point data.

Same flow as Category A. These are configuration data that changes rarely.

3.4 TRENDLOGS / TRENDLOG_INPUTS / TRENDLOG_DATA (Category C)

Source: FFI `LOGGING_DATA` cycle — discovery of trendlog configs and summary data.

Same flow as Category A. These are metadata/summary tables, relatively small.

3.5 TRENDLOG_DATA_DETAIL (Category D) ⭐ Special

Source: FFI `LOGGING_DATA` cycle — actual timestamped trend values.

PC Role	FFI Action	Web Read	Web Write
Main	INSERT central ONLY	Central	Central
Reader	INSERT central ONLY	Central	Central
Classic	INSERT local	Local	Local

Why special:

- Grows to millions of rows (one row per sample per trendlog per timestamp).
- Append-only (INSERT, never UPDATE).
- Keeping in local SQLite defeats the purpose of centralization.
- On switch-back, historical trendlogs are not critical for immediate operation.

3.6 UI/Config Tables (Category E) — GRAPHICS, ALARMS, etc.

Source: Web server only (user creates graphics, configures alarms via browser).

PC Role	FFI Action	Web Read	Web Write
Main	—	Central	Central + local
Reader	—	Central	Central only
Classic	—	Local	Local

3.7 Device Settings (Category F)

Source: Web server (user configures network/communication settings via browser).

Same flow as Category E.

3.8 Local-Only Tables (Category G)

Source: Internal application state.

Table	Always Uses
APPLICATION_CONFIG	Local SQLite
APPLICATION_CONFIG_HISTORY	Local SQLite
DB_BACKEND_CONFIG	Local SQLite
database_partition_config	Local SQLite
database_files	Local SQLite
database_partitions	Local SQLite
DATA_SYNC_METADATA	Local SQLite
TRENDLOG_DATA_SYNC_METADATA	Local SQLite

These tables are **never** written to or read from the central DB. They contain per-PC state that must remain local.

3.9 SYSTEM_LOGS (New Table)

Source: Application events, errors, audit trail.

PC Role	Write Destination	Read Source
Main (store_logs=1)	Central DB	Central DB
Reader (store_logs=1)	Central DB	Central DB
Any (store_logs=0)	Not written	—
Classic	Not applicable	—

4. Connection Resolution Logic

RUST

```rust
/// Determines which DB connection to use for a given operation.
fn resolve_connection(
    state: &T3AppState,
    table_category: TableCategory,
    operation: Operation,
) -> &DatabaseConnection {
    if !state.central_db_enabled {
        // Classic mode — always local
        return &state.local_config_conn;
    }

    match table_category {
        // Category G: Always local
        LocalOnly => &state.local_config_conn,

        // Category D: Always central when enabled
        TrendlogDetail => &state.t3_device_conn, // points to central

        // Categories A-F: Central for reads, dual for writes on main
        _ => match operation {
            Read => &state.t3_device_conn, // central
            Write => &state.t3_device_conn, // central (caller also writes local if main)
        },
    }
}
```

---

5. Failure Scenarios

Central DB goes offline during operation

TEXT

```
Write to central fails
  │
  ├─ FFI sync (Main):
  │   ├─ Local write already succeeded (done first)
  │   ├─ Log error, increment retry counter
  │   └─ Next sync cycle retries automatically
  │
  ├─ FFI sync (Reader):
  │   └─ Only writing local — unaffected
  │
  └─ Web server:
      ├─ Return 503 to browser with error message
      ├─ Log failure
      └─ UI shows "Central DB unavailable" banner
```

Switch from central to classic mode

TEXT

```
Admin sets enabled=0 in setting.ini, restarts app
  │
  ├─ T3AppState.central_db_enabled = false
  ├─ t3_device_conn points to local SQLite
  ├─ All reads/writes go to local
  └─ Warning: TRENDLOG_DATA_DETAIL from central period is not in local
```

Network partition between PCs

TEXT

```
Main PC and Reader PC can't reach each other
  │
  ├─ Both can still reach central DB → no impact
  ├─ Only Main can reach central → Reader falls back to local reads
  └─ Neither can reach central → both operate in degraded/classic mode
```

T3000 WebView System Architecture

Overview

The T3000 WebView is a modern web-based building automation system that provides real-time monitoring and control of HVAC devices through a hybrid desktop/web architecture.

Architecture Type: Hybrid Desktop Application with Web Frontend **Technology Stack:** Rust Backend + Vue.js/React Frontend + SQLite Database **Communication:** BACnet Protocol + WebSocket + REST API

System Components

Frontend Layer (Client)

The web interface you interact with runs in your browser or embedded WebView.

- **Vue.js Application**: Main user interface for device control
- **React Modules**: Database viewer and developer tools
- **Quasar Framework**: Responsive UI components

Backend Layer (Rust API Server)

The backend server runs on port 9103 and handles all communication.

- **REST API**: Device management, configuration, and database queries
- **WebSocket**: Real-time updates and control messages
- **BACnet Bridge**: Connects to physical devices via FFI layer

Data Layer (SQLite Databases)

Three databases store all system data:

- **T3000.db**: Main configuration, devices, users, settings
- **webview_t3_device.db**: Device data cache (inputs, outputs, variables)
- **T3000_Trendlog.db**: Historical trend data and time-series records

Device Layer (BACnet Network)

Physical building controllers communicate via BACnet/IP protocol:

- T3-BB (BACnet Building Controller)
- T3-TB (Thermostat Controller)
- T3-LB (Lighting Controller)

How Device Loading Works

Step 1: Network Scan

When you open the app, it scans the network for BACnet devices by broadcasting a "Who-Is" message.

Step 2: Device Discovery

Devices respond with "I-Am" messages containing their IP address, device ID, and model information.

Step 3: Data Retrieval

When you select a device:

1. Frontend sends a SELECT_PANEL message via WebSocket
2. Backend connects to the device's IP address
3. BACnet layer reads all data points (inputs, outputs, variables)
4. Data is cached in the database
5. Frontend receives the data and displays it in tables/grids

Step 4: Real-Time Updates

Changes are monitored and updated automatically:

- Device value changes trigger WebSocket broadcasts
- Frontend updates the UI without page refresh
- Database cache stays synchronized

User Interface

Device Tree

Browse all discovered devices in a hierarchical tree view organized by location or type.

Data Points

View and edit device data in organized tabs:

- **Inputs**: Temperature sensors, humidity, pressure
- **Outputs**: Relays, dampers, valves
- **Variables**: Setpoints, calculations, custom values
- **Programs**: Control logic and automation
- **Schedules**: Time-based control
- **Graphics**: Visual building layouts

Trend Logs

Historical data visualization with customizable time ranges and multiple data series.

Configuration

Network Settings

Configure IP addresses, subnets, and protocol ports in Settings → Network.

User Management

Create operator and administrator accounts with role-based permissions.

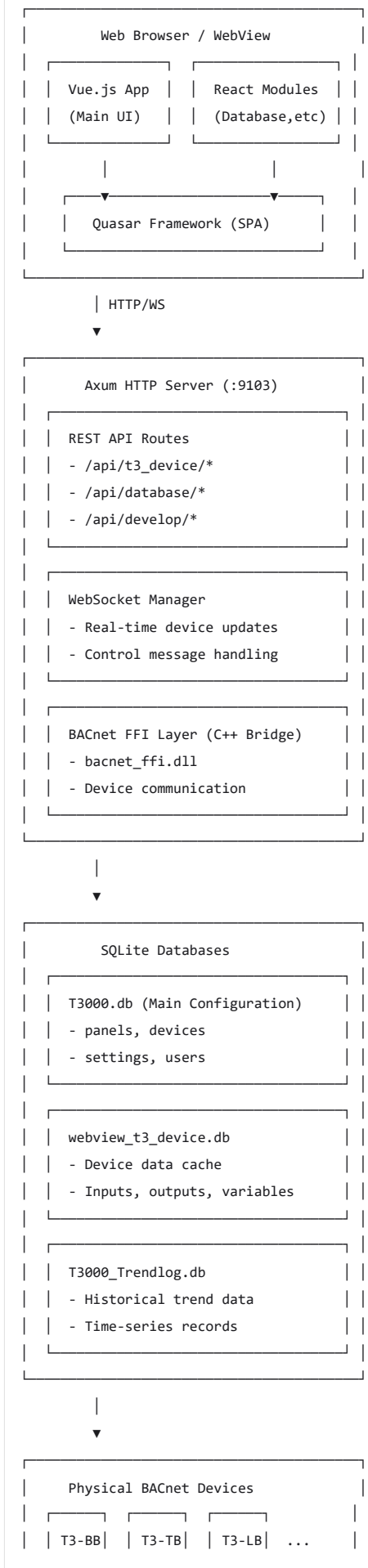
Backups

Enable automatic daily backups to protect your configuration and historical data.

Detailed System Architecture

Component Architecture Diagram

TEXT


Web Browser / WebView
Vue.js App (Main UI)
React Modules (Database,etc)
Quasar Framework (SPA)
HTTP/WS
Axum HTTP Server (:9103)
REST API Routes
- /api/t3_device/*
- /api/database/*
- /api/develop/*
WebSocket Manager
- Real-time device updates
- Control message handling
BACnet FFI Layer (C++ Bridge)
- bacnet_ffi.dll
- Device communication
SQLite Databases
T3000.db (Main Configuration)
- panels, devices
- settings, users
webview_t3_device.db
- Device data cache
- Inputs, outputs, variables
T3000_Trendlog.db
- Historical trend data
- Time-series records
Physical BACnet Devices
T3-BB
T3-TB
T3-LB
...



TEXT

```
### Device Data Loading Flow
```

TEXT

```
---

## Communication Protocols

### 1. Frontend ↔ Backend

**REST API (HTTP)**
- Device scanning
- Configuration management
- Database queries
- File operations

**WebSocket (Real-time)**
- Control messages (action-based)
- Live data updates
- Device status changes
- Trend data streaming

### 2. Backend ↔ Devices

**BACnet/IP Protocol**
- Who-Is / I-Am (device discovery)
- Read-Property (data retrieval)
- Write-Property (control commands)
- Subscribe-COV (change notifications)

---

## Control Message System

### Message Structure
```json
{
  "action": <number>,
  "source": 0|1,          // 0=T3000, 1=WebUI
  "panelId": <number>,
  "serialNumber": <number>,
  "entryType": <number>,
  "data": { ... }
}
```

Common Actions

Action	Name	Purpose
1	SELECT_PANEL	Load device data
2	SAVE_ENTRY	Write single value
16	UPDATE_WEBVIEW_LIST	Bulk write
17	READ_SINGLE_ENTRY	Read one entry
```

Database Schema

T3000.db

```SQL
-- Device registration
panels (
  id, serial_number, product_name,
  ip_address, port, location
)

-- User management
users (
  id, username, password_hash, role
)

-- System settings
application_settings (
  key, value, category
)
```

webview_t3_device.db

```SQL
-- Device data cache
inputs (
  id, panel_id, number, label,
  value, units, auto_manual
)

outputs (
  id, panel_id, number, label,
  value, auto_manual, range
)

variables (
  id, panel_id, number, label,
  value, units, auto_manual
)
```

Deployment Architecture

Development Mode

```TEXT
Frontend (Quasar Dev Server)
  ↓ Port 3004
  ↓ Proxy to backend
Backend (Rust cargo run)
  ↓ Port 9103
  ↓ SQLite files in ./Database
Devices (Local network)
```

Production Mode

```
TEXT
```

```
Electron/Tauri Desktop App
  ├── Bundled Frontend (SPA)
  ├── Embedded Rust Backend
  └── SQLite in %AppData%/T3000
       ↓
  BACnet Network
```

Key Design Patterns

1. Event-Driven Architecture

- WebSocket for real-time updates
- Publisher/subscriber pattern
- Async/await for non-blocking I/O

2. Caching Strategy

- Database cache for offline access
- In-memory state for active devices
- Background sync with physical devices

3. State Management

- Frontend: Vue Pinia stores
- Backend: Rust Arc<Mutex> for shared state
- Database: SQLite transactions

4. Error Handling

- Frontend: Try-catch with user notifications
- Backend: Result<T, E> with proper propagation
- FFI: Safe error conversion across boundaries

Performance Considerations

Optimization Points

1. **Device Scanning**: Parallel network scans
2. **Data Loading**: Batch reads via bulk messages
3. **UI Updates**: Debounced re-renders
4. **Database**: Indexed queries on objectinstance
5. **WebSocket**: Message compression for large datasets

Scalability Limits

- Max devices per network: ~250 (BACnet limitation)
- Max concurrent connections: ~100 WebSocket clients
- Database size: Tested up to 10GB trendlog data

Security Model

Authentication

- User login with bcrypt password hashing
- Session tokens for API access

- Role-based permissions (admin/user)

Network Security

- BACnet authentication (optional)
- HTTPS for production deployment
- CORS configuration for web access

Future Enhancements

Planned Features

1. **Cloud Sync**: Multi-site management
2. **Mobile App**: React Native companion
3. **AI Analytics**: Predictive maintenance
4. **RESTful BACnet**: Bridge to cloud services

Technical Debt

1. Migrate from mixed Vue/React to single framework
2. Implement GraphQL for flexible data queries
3. Add comprehensive integration tests
4. Improve FFI error handling with detailed codes

Getting Started for Developers

Prerequisites

```shell
## Rust toolchain
rustup install stable

## Node.js & npm
nvm install 18

## Quasar CLI
npm install -g @quasar/cli
```

Development Setup

```shell
## 1. Start backend
cd api
cargo run

## 2. Start frontend
npm run client-dev

## 3. Access UI
open http://localhost:3004
```

Project Structure

```
T3000Webview9/
├── api/              # Rust backend
│   ├── src/
│   │   ├── t3_device/     # Device management
│   │   ├── t3_develop/    # Developer tools
│   │   └── database_management/
│   └── build/        # FFI build outputs
├── src/              # Frontend source
│   ├── t3-vue/       # Vue components
│   └── t3-react/     # React modules
├── docs/             # Documentation
└── Database/         # SQLite files
```

Troubleshooting Guide

Common Issues

Device not detected

- Check BACnet router configuration
- Verify IP subnet matches
- Ensure Who-Is broadcasts are allowed

WebSocket disconnects

- Increase timeout in backend config
- Check firewall settings on port 9103
- Verify network stability

Database locked errors

- Only one writer at a time (SQLite limitation)
- Use WAL mode for concurrent reads
- Close connections properly in code

References

- BACnet Protocol Standard
- Axum Web Framework
- Quasar Framework
- SQLite Documentation

T3 Firmware — LVGL Display Architecture

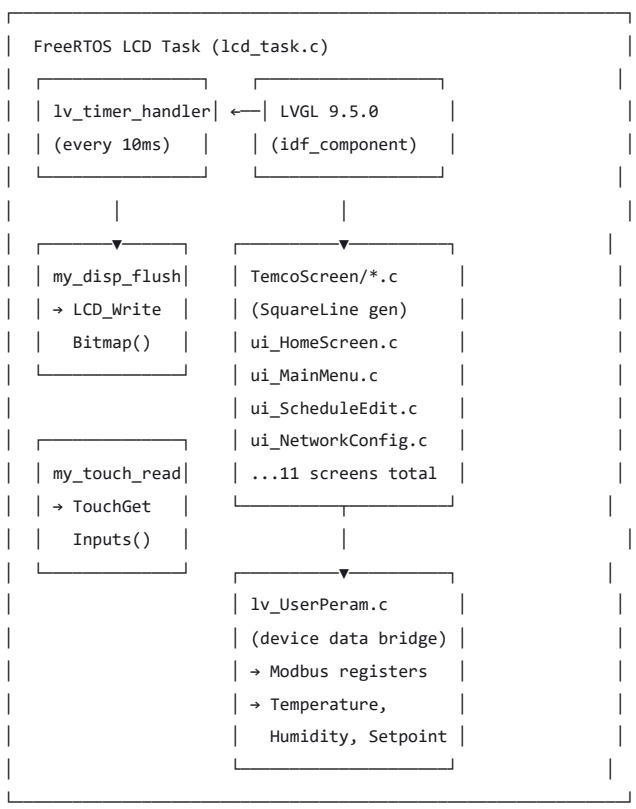
Part 1 — Current Firmware (SquareLine Studio)

Branch: `Bhavik-Tstate-11` **Tool:** SquareLine Studio 1.6.0 **LVGL:** 9.5.0 (via ESP-IDF component manager) **Display:** ILI9341 320×240 via hardware SPI + external touch

---

Architecture Overview

TEXT



---

1. How Screens Are Created (SquareLine Studio)

SquareLine Studio generates static C code. Each screen is a function that calls `lv_obj_create()` etc.:

C

```c
// ui_HomeScreen.c — generated by SquareLine Studio 1.6.0
void ui_HomeScreen_screen_init(void) {
    ui_HomeScreen = lv_obj_create(NULL);
    lv_obj_set_size(ui_HomeScreen, 320, 240);

    // Create temperature label
    ui_TemperatureVal = lv_label_create(ui_HomeScreen);
    lv_label_set_text(ui_TemperatureVal, "22.5");
    lv_obj_set_pos(ui_TemperatureVal, 80, 20);
    lv_obj_set_style_text_font(ui_TemperatureVal, &ui_font_Arial80, LV_PART_MAIN);

    // Create fan mode button
    ui_AutoBtn = lv_btn_create(ui_HomeScreen);
    lv_obj_set_pos(ui_AutoBtn, 200, 150);
    // ...add label, image, event handler
}
```

All screens are created at boot in `ui_init()`:

```c
void ui_init(void) {
    ui_StartUpScreen_screen_init();   // Boot/splash
    ui_HomeScreen_screen_init();      // Main thermostat
    ui_MainMenu_screen_init();        // Settings menu
    ui_WifiConfig_screen_init();      // WiFi settings
    ui_NetworkConfig_screen_init();   // Network/IP settings
    ui_Protocols_screen_init();       // BACnet/Modbus settings
    ui_Parameters_screen_init();      // Parameter viewer
    ui_Time_screen_init();            // Time/clock
    ui_ScheduleScreen_screen_init();  // Weekly schedule
    ui_ScheduleEditScreen_screen_init(); // Schedule editor
    ui_HolidayCalenderScreen_screen_init(); // Holidays
    lv_scr_load(ui_StartUpScreen);    // Show boot screen first
}
```

---

2. Screen Navigation

Screen switching is hardcoded, typically from event callbacks in `ui_events.c`:

```c
// ui_events.c — event handler that switches screens
void FanSetAutoMode(lv_event_t * e) {
    Event_Cb_FanSetAutoMode(e);    // device logic
    _ui_screen_change(&ui_MainMenu, LV_SCR_LOAD_ANIM_FADE_ON, 200, 0, &ui_MainMenu_screen_init);
}
```

Or directly via `lv_scr_load()`:

```c
lv_scr_load(ui_HomeScreen);
```

No navigation graph or flow engine — each screen knows its own navigation targets.

---

3. Device Data Binding (lv_UserPeram.c)

The bridge between LVGL UI and Modbus device data:

```c
// lv_UserPeram.c
void UpdateTemperatureDisplay(void) {
    // Read from Modbus register
    int32_t temp = Temperature_InVal / 10;
    char buf[16];
    sprintf(buf, "%d.%d°C", temp, abs(Temperature_InVal % 10));
    lv_label_set_text(ui_TemperatureVal, buf);
}

// Polled periodically in the LCD task's timer callback
void lv_UserPeram_Poll(void) {
    if (Temperature_InVal != last_temp) {
        UpdateTemperatureDisplay();
        last_temp = Temperature_InVal;
    }
    if (Humidity_InVal != last_hum) {
        UpdateHumidityDisplay();
        last_hum = Humidity_InVal;
    }
    // ... fan mode, system mode, time, WiFi status, etc.
}
```

Data flow: `Modbus RTU → deviceparams → lv_UserPeram → LVGL objects`

---

4. Display Driver (lcd_task.c + lcd_drv.c)

- **SPI**: Hardware SPI via ESP-IDF `spi_master` driver (pins: MOSI, MISO, CLK, CS, DC)
- **Buffer**: Full-frame buffer in PSRAM (`MALLOC_CAP_SPIRAM`)
- **Flush**: `my_disp_flush()` → `LCD_WriteBitmap()` → SPI DMA transfer
- **Touch**: I2C touch controller via `TouchGetInputs()` → LVGL input device
- **Tick**: `esp_timer_get_time() / 1000` for `lv_tick_set_cb()`
- **Task**: FreeRTOS task at 10ms tick — calls `lv_timer_handler()` + `lv_UserPeram_Poll()`

```c
void lcd_task(void *arg) {
    LCD_Init();                // ILI9341 init + backlight
    TouchInit();               // I2C touch init
    LCD_RegesterlvglDriver();  // lv_display_create + flush/read callbacks
    ui_init();                 // Create all SquareLine screens

    while (1) {
        lv_timer_handler();        // LVGL rendering
        lv_UserPeram_Poll();       // Update device data
        T3_Lv_Scr_Manager();       // Screen auto-rotate/timer
        vTaskDelay(pdMS_TO_TICKS(LCD_TASK_DELAY_MS));  // 10ms
    }
}
```

---

5. Build System

YAML

```yaml
## main/idf_component.yml
dependencies:
  idf:
    version: '>=5.5.3'
  espressif/led_strip: '*'
  lvgl/lvgl:
    version: "==9.5.0"      # Pulled by ESP-IDF component manager
```

CMAKE

```cmake
## main/CMakeLists.txt
idf_component_register(SRCS
    "LCD_Driver/lcd_drv.c"
    "LCD_Driver/lcd_task.c"
    "LCD_Driver/touch_drv.c"
    "LCD_Driver/lv_UserPeram.c"
    ...
)
## Auto-include all SquareLine generated .c files
file(GLOB TEMCO_SCREEN_SOURCES "TemcoScreen/*.c")
target_sources(${COMPONENT_LIB} PRIVATE ${TEMCO_SCREEN_SOURCES})
```

---

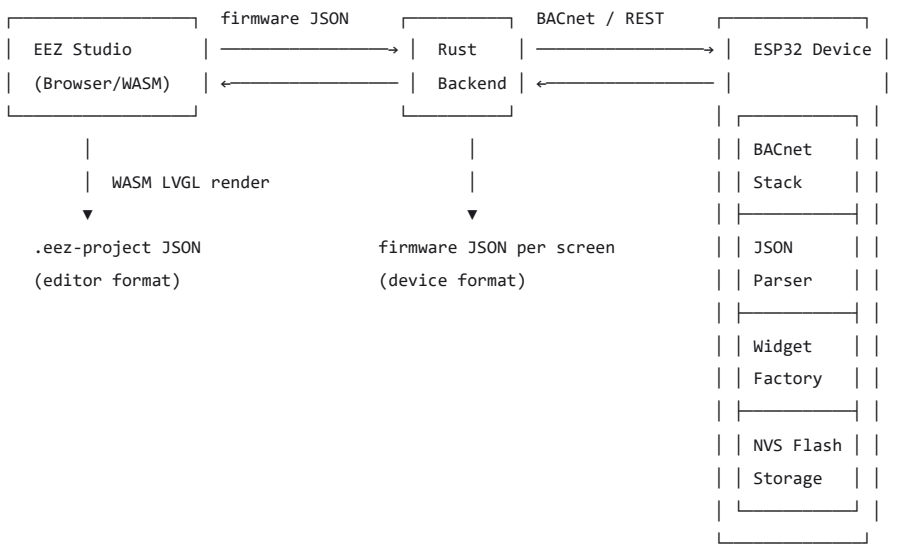
6. Screens List

File	Screen	Lines	Purpose
`ui_StartUpScreen.c`	Boot splash	~50	Logo + initialization
`ui_HomeScreen.c`	Main thermostat	868	Temperature, humidity, fan, mode
`ui_MainMenu.c`	Settings menu	349	Navigation to sub-screens
`ui_WifiConfig.c`	WiFi settings	326	SSID, password, scan
`ui_NetworkConfig.c`	Network settings	769	IP, subnet, BACnet port
`ui_Protocols.c`	Protocol config	371	BACnet, Modbus settings
`ui_Parameters.c`	Parameter viewer	151	Device parameters
`ui_Time.c`	Clock settings	557	Date/time/schedule
`ui_ScheduleScreen.c`	Schedule overview	666	Weekly program list
`ui_ScheduleEditScreen.c`	Schedule editor	2,608	Day/time/temp editing
`ui_HolidayCalenderScreen.c`	Holiday calendar	107	Holiday date picker

Part 2 — EEZ Studio Project Flow (Web ↔ Device)

Architecture

TEXT



2.1 Two JSON Formats

Format	Producer	Consumer	Content
`.eez-project`	EEZ Studio save	Editor, build system	Full project: settings, pages, styles, fonts, bitmaps (base64), actions, connectionLines
Firmware JSON	`firmware-export.ts`	Device firmware, mock store	Per-screen: widgets flattened, actions resolved, bitmaps as names only

2.2 Export: .eez-project → Firmware JSON

`firmware-export.ts` transforms the editor format to per-screen device JSON:

TEXT

```
.eez-project                          Firmware JSON (per screen)
————————                              ——————————————————————

userPages[]                              screens[]
  └ components[]                          └ widgets[]
      ├ LVGLScreenWidget (unwrapped)            ├ { sub_type: "label", ... }
      ├ LVGLLabelWidget                         ├ { sub_type: "button", ... }
      ├ LVGLButtonWidget                        └ { sub_type: "action", ... }
      ├ LVGLActionComponent
      └ connectionLines (resolved)       events attached directly:
                                             CLICKED → actions: [changeScreen(...)]
```

Field mapping (export):

`.eez-project` field	Firmware JSON field
`left` / `top`	`x_pos` / `y_pos`
`width` / `height`	`width` / `height`
`text` / `textType`	`obj_text` / `text_type`
`localStyles.definition`	`style`
`eventHandlers[]`	`events: { TRIGGER: { actions: [...] } }`
`actions[].action: "changeScreen"`	`actions: [{ action: "changeScreen", screen: "..." }]`
`children[]`	`children: { ... }` (recursive)

Action resolution: `connectionLines` linking widget events → `LVGLActionComponent` are resolved at export time. Actions are attached directly to widget events in the firmware JSON. Standalone action components are stripped.

2.3 Transfer: Browser ↔ Device

Two paths (auto-detected by `DeviceRestClient`):

Path	Direction	Endpoint	Transport
REST (primary)	Push	`PUT /api/v1/screens`	HTTP to ESP32 port 8000
	Pull	`GET /api/v1/screens`	HTTP from ESP32 port 8000
BACnet (fallback)	Push	`POST /api/eez-device/screens/push/:panelId`	HTTP → Rust → BACnet → Device
	Pull	`POST /api/eez-device/screens/pull/:panelId`	HTTP → Rust → BACnet → Device

API surface (`/api/eez-device/*`):

Method	Endpoint	Purpose
`GET`	`/screens`	Load all screens
`PUT`	`/screens`	Deploy all screens
`GET`	`/screens/:name`	Load single screen
`PUT`	`/screens/:name`	Deploy single screen
`PATCH`	`/screens/:name`	Delta update
`PATCH`	`/screens/:name/widgets/:id`	Widget delta
`POST`	`/screens/push/:panelId`	BACnet push
`POST`	`/screens/pull/:panelId`	BACnet pull
`POST`	`/images/push/:panelId`	Upload bitmap
`GET`	`/images/pull/:panelId/:name`	Download bitmap
`DELETE`	`/images/:panelId/:name`	Remove bitmap
`GET`	`/devices`	List devices

2.4 Import: Firmware JSON → .eez-project

`firmware-loader.ts` reverses the export transform:

```
Firmware JSON (per screen)          .eez-project
──────────────────────────          ────────────

widgets[]                             userPages[]
  ├ { sub_type: "label", ... }          └ components[]
  ├ { sub_type: "button", ... }               ├ LVGLLabelWidget
  └ events: { CLICKED: { actions } }          ├ LVGLButtonWidget
       ↓                                       ├ LVGLActionComponent (rebuilt)
  LVGLActionComponent created                  └ connectionLines (rebuilt)
  connectionLine created
```

Field mapping (import):

Firmware JSON field	`.eez-project` field
`x_pos` / `y_pos`	`left` / `top`
`obj_text` / `text_type`	`text` / `textType`
`style`	`localStyles.definition`
`events[].actions[]`	`LVGLActionComponent` + `connectionLine`
`children`	`children` / `components` (recursive)
`min` / `max` / `value`	`min` / `max` / `value` / `valueType`
`src` (image)	`image`
`checked` / `checked_type`	`checkedState` / `checkedStateType`
`long_mode` / `recolor`	`longMode` / `recolor`
`inner_align`	`innerAlign`
`options` / `selected`	`options` / `selected`
`widget` (user_widget)	`userWidgetPageName`

2.5 Frontend Rendering (Browser)

LVGL WASM runtime renders `.eez-project` pages in the browser:

```
.eez-project JSON
      ↓
ProjectStore.create() → loadProject() → setProject()
      ↓
LVGL WASM runtime (Emscripten-compiled)
  ├ v8.4.0, v9.2.2, v9.3.0, v9.4.0, v9.5.0
  └ Served from /eez-studio-wasm/wasm/lvgl/{version}/
      ↓
Canvas-based LVGL display buffer → browser-rendered preview
```

Runtime loading (from `lvgl-versions.ts`):

1. Detects browser environment
2. Loads JS glue via `<script>` tag (sets `globalThis.LVGLWasmRuntime`)
3. Instantiates Emscripten module with LVGL compiled to WASM
4. Creates virtual display in an HTML canvas

Supported LVGL versions: v8.4.0, v9.2.2, v9.3.0, v9.4.0, v9.5.0 (default for new projects).

2.6 Full Round-Trip Flow

```
TEXT
```

```
DEPLOY:                                IMPORT:
EEZ Studio                             Open Page
  │ "Deploy to Device"                 │ "Import from Device"
  ▼                                    ▼
firmware-export.ts                     DeviceRestClient.connect()
  │ .eez-project → DeviceScreen[]      │ isReachable() → REST or BACnet
  ▼                                    ▼
DeviceRestClient.deployAllScreens()    GET /api/eez-device/screens
  │ PUT /api/eez-device/screens        │
  ▼                                    ▼
BACnet → ESP32 NVS                     firmware-loader.ts
  │ device stores + parses             │ Firmware JSON → .eez-project
  ▼                                    ▼
Device creates LVGL widgets            WASM LVGL render in browser
```

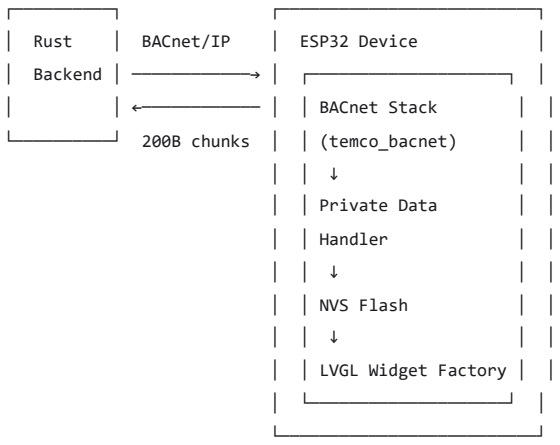
2.7 Device BACnet API

When the real device supports EEZ Studio screens, it exposes a BACnet-based API for screen push/pull.

2.7.1 BACnet Transfer Protocol

Screens are transferred in **200-byte compressed chunks** via BACnet Private Data objects on the ESP32:

```
TEXT
```



BACnet Private Data Commands — implemented on the ESP32:

Command	Direction	Purpose
`WRITE_JSON_SCREEN`	Rust → Device	Screen metadata (name, zip size)
`WRITE_JSON_ITEM`	Rust → Device	200-byte chunk of compressed JSON
`READ_JSON_SCREEN`	Rust → Device	Request screen metadata
`READ_JSON_ITEM`	Rust → Device	Request 200-byte chunk
`WRITE_SETTING_COMMAND`	Rust → Device	Set `webview_json_flash = 2`

Deploy sequence:

1. Rust sends `WRITE_SETTING_COMMAND` → device sets `webview_json_flash = 2`
2. Rust sends `WRITE_JSON_SCREEN` → device allocates NVS space, prepares for chunks
3. Rust sends `WRITE_JSON_ITEM` × N (200 bytes each, 2 per BACnet write) → device assembles in NVS buffer
4. Device verifies CRC, decompresses with zlib, parses JSON

5. Device creates/destroys LVGL widgets from JSON definitions

Read sequence:

1. Rust sends `READ_JSON_SCREEN` → device returns screen metadata (names, sizes)
2. Rust sends `READ_JSON_ITEM` × N → device returns 200-byte chunks from NVS
3. Rust assembles chunks, decompresses, returns JSON to browser

2.7.2 NVS Flash Storage

Each screen uses **10 slots** of 200 bytes in NVS flash:

```
TEXT

Slot layout:  screenIndex × 10 + chunkIndex  (0..9)
Per screen:   10 slots × 200 bytes = 2 KB max compressed
Per device:   up to N screens in NVS partition
```

The `webview_json_flash` flag (Modbus register offset 253) indicates the device mode:

- `0` = legacy hardcoded UI
- `2` = EEZ Studio JSON mode

2.7.3 REST API (ESP32)

As an alternative to BACnet, the ESP32 hosts a lightweight HTTP server:

Method	Endpoint	Purpose
`GET`	`/api/v1/screens`	Return all stored screens
`PUT`	`/api/v1/screens`	Store screen definitions
`GET`	`/api/v1/screens/:name`	Single screen
`PUT`	`/api/v1/screens/:name`	Update single screen
`PATCH`	`/api/v1/screens/:name`	Delta update

`DeviceRestClient` probes REST first (port 8000, 2s timeout), falls back to BACnet if unreachable.

2.7.4 Device Firmware Modules

To support EEZ Studio JSON on the ESP32:

1. **BACnet Private Data handlers** for `WRITE_JSON_SCREEN`, `WRITE_JSON_ITEM`, `READ_JSON_SCREEN`, `READ_JSON_ITEM`
2. **Zlib decompression** — screens arrive compressed in 200-byte chunks
3. **JSON parser** (cJSON) — parse firmware JSON to extract widget definitions
4. **NVS storage** — persist screens in flash, survive reboots
5. **Widget factory** — map `sub_type` → `lv_obj_create()` at runtime
6. **Expression engine** — resolve `variables.temp` to Modbus register values
7. **Action engine** — handle `changeScreen`, `setVariable` actions
8. **Image loader** — convert base64 bitmaps to LVGL image descriptors
9. **REST server** — lightweight HTTP on port 8000 for direct access

---

3.1 Building BACnet/REST API on the ESP32

The device needs two API surfaces to receive and serve screen definitions.

3.1.1 BACnet Private Data API

The ESP32's existing BACnet stack (`temco_bacnet`) is extended with Private Data object handlers:

Commands implemented on the device:

Command	Direction	Handler	Purpose
`WRITE_JSON_SCREEN`	Client → Device	`bacnet_json_screen_write()`	Receive screen metadata (name, compressed size)
`WRITE_JSON_ITEM`	Client → Device	`bacnet_json_item_write()`	Receive 200-byte compressed chunk
`READ_JSON_SCREEN`	Client → Device	`bacnet_json_screen_read()`	Return screen metadata
`READ_JSON_ITEM`	Client → Device	`bacnet_json_item_read()`	Return 200-byte chunk
`WRITE_SETTING_COMMAND`	Client → Device	(existing handler)	Set `webview_json_flash = 2`

Deploy sequence (per screen):

1. `WRITE_SETTING_COMMAND` → set `webview_json_flash = 2`
2. `WRITE_JSON_SCREEN` → device allocates buffer, stores `{ name, total_chunks, zip_size }`
3. `WRITE_JSON_ITEM` × N → device appends 200-byte chunks to buffer
4. Device verifies CRC, decompresses with zlib, parses JSON with cJSON
5. Device calls widget factory to create/destroy LVGL objects
6. Device calls `lv_scr_load()` if this is the active screen

Read sequence:

1. `READ_JSON_SCREEN` → device returns screen names + metadata from NVS
2. `READ_JSON_ITEM` × N → device reads 200-byte chunks from NVS, returns them

Files to create: `main/bacnet_json.c/h`, `main/screen_store.c/h`

3.1.2 REST API (HTTP on ESP32)

The ESP32 runs a lightweight HTTP server on port 8000 using ESP-IDF's `esp_http_server`:

```
TEXT
```

```
ESP32 Boot → rest_api_init()
  ├─ GET    /api/v1/screens                    → load all screens
  ├─ PUT    /api/v1/screens                    → deploy all screens
  ├─ GET    /api/v1/screens/:name              → load single screen
  ├─ PUT    /api/v1/screens/:name              → deploy single screen
  ├─ PATCH  /api/v1/screens/:name              → delta update
  ├─ PATCH  /api/v1/screens/:name/widgets/:id  → widget-level delta
  ├─ POST   /api/v1/images                     → upload bitmap image
  ├─ GET    /api/v1/images/:name               → download bitmap image
  └─ DELETE /api/v1/images/:name               → remove bitmap image
```

Handler details:

Handler	Action
`rest_get_screens()`	Read all screens from NVS, return `{ screens: [...], meta: {...} }`
`rest_put_screens()`	Parse body with cJSON, iterate screens, store each in NVS, return `{ deployed, failed, status }`
`rest_get_screen()`	Read single screen by name from NVS, return `{ name, json }`
`rest_put_screen()`	Store single screen to NVS, return `{ name, status }`
`rest_patch_screen()`	Load screen from NVS, apply dot-path changes (`widgets.btn.text`), save back
`rest_patch_widget()`	Load screen, navigate to `widgets.<id>.<path>`, apply change
`rest_push_image()`	Receive `{ name, data_base64 }`, decode base64, convert to LVGL format with `lv_img_conv`, store in NVS
`rest_pull_image()`	Read image from NVS, encode as base64, return
`rest_delete_image()`	Remove image from NVS

Key ESP-IDF APIs used:

- `httpd_start()` / `httpd_register_uri_handler()` — HTTP server
- `nvs_open()` / `nvs_set_str()` / `nvs_get_str()` / `nvs_commit()` — NVS storage
- `cJSON_Parse()` / `cJSON_Print()` / `cJSON_GetObjectItem()` — JSON parsing

Files to create: `main/rest_api.c/h`, `components/cJSON/cJSON.c/h`

3.1.3 NVS Storage Layout

```
NVS Namespace: "screens"
  Key: "index"      → JSON array of screen names ["Home", "Settings", ...]
  Key: "Home"       → full firmware JSON string for this screen
  Key: "Settings"   → full firmware JSON string for this screen
  ...

NVS Namespace: "images"
  Key: "bg_home"    → base64-encoded image data
  Key: "btn_heat"   → base64-encoded image data
  ...

NVS Namespace: "storage"
  Key: "WEBWIEW_JASON" → uint8 flag (0 = legacy, 2 = EEZ Studio)
```

Maximum NVS string size: ~4000 bytes per entry (ESP32 NVS limit). For larger screens, the BACnet path uses chunked transfer with zlib compression.

Flash budget (ESP32 32MB, partition `storage` at 0x760000):

Item	Size
`storage` partition	128 KB (0x20000)
Existing NVS usage (settings, WiFi, SNTP, etc.)	~5 KB
Free for screen JSON	~120 KB
Per-screen JSON (uncompressed)	~30 KB
Per-screen JSON (zlib compressed, typical)	5–10 KB
11 screens compressed	55–110 KB
Fits?	✅ Yes (tight for all 11; ~10 KB overhead remaining)

For projects larger than ~120 KB compressed, chunk the screens across multiple BACnet deploys or increase the `storage` partition size.

NVS entry size limit: ESP32 NVS has a ~4000-byte-per-entry limit. Screens larger than 4 KB compressed must be split across multiple NVS keys (e.g., `"Home_0"`, `"Home_1"`, ...) and reassembled on read. The BACnet chunked path (WRITE_JSON_ITEM) already handles this — the REST API should follow the same pattern for large screens.

Storage recommendations:

Concern	Suggestion
Including images (36 bitmaps × 3KB = ~108KB)	Bump `storage` partition to 384KB (`0x60000`). ESP32 has 32MB flash — plenty of room.
NVS 4KB-per-entry limit on images	Use SPIFFS or LittleFS partition for images instead of NVS. No per-entry size limit, better for binary blobs.
Partition layout (proposed)	Add a `spiffs` partition for images at ~0x7C0000, 256KB. Keep JSON screens in NVS `storage` .

3.1.4 Device Lifecycle Flow

First boot (factory state):

TEXT

```
Device powers on
  ├─ NVS "WEBWIEW_JASON" = 0 (legacy mode, or not set)
  ├─ No JSON in NVS namespace "screens"
  ├─ Load legacy SquareLine screens (compiled C code)
  └─ Display legacy UI
```

After first deploy (via BACnet or REST):

TEXT

```
Browser → Deploy screens
  ├─ WRITE_SETTING_COMMAND → webview_json_flash = 2
  ├─ WRITE_JSON_SCREEN + WRITE_JSON_ITEM × N → compressed JSON
  ├─ Device decompresses, parses JSON
  ├─ Device stores JSON in NVS "screens" namespace (survives reboot)
  ├─ Device sets webview_json_flash = 2 in NVS
  ├─ screen_manager_load_all() → creates LVGL screens from JSON
  └─ Display EEZ Studio UI
```

Normal boot (JSON already stored):

TEXT

```
Device powers on
  ├─ NVS "WEBWIEW_JASON" = 2
  ├─ Read screen index from NVS "screens"/"index"
  ├─ For each screen:
  │    ├─ Read JSON string from NVS
  │    ├─ Parse with cJSON
  │    ├─ Create LVGL screen (lv_obj_create)
  │    └─ widget_factory_create() → populate with widgets
  ├─ Load first screen: lv_scr_load(g_screens[0].screen)
  └─ Display EEZ Studio UI
```

Screen update (deploy after boot):

TEXT

```
Browser → Deploy updated screens
  ├─ Device receives JSON via BACnet/REST
  ├─ For each updated screen:
  │    ├─ Destroy existing LVGL objects for that screen
  │    ├─ Parse new JSON
  │    ├─ Store new JSON in NVS (overwrite)
  │    └─ widget_factory_create() → rebuild LVGL objects
  ├─ If updated screen is the active screen:
  │    └─ lv_scr_load() to refresh display
  └─ Response: { deployed: N, failed: 0, status: "ok" }
```

Fallback to legacy (if JSON is corrupted or missing):

TEXT

```
Device powers on
  ├─ NVS "WEBWIEW_JASON" = 2
  ├─ Failed to read/parse JSON from NVS
  ├─ Set webview_json_flash = 0 (revert to legacy)
  ├─ Load legacy SquareLine screens
  └─ Display legacy UI
```

Key design decisions:

Decision	Rationale
JSON stored in NVS, not RAM	Survives power loss and reboots
`webview_json_flash` flag	Fast check at boot: skip JSON loading if in legacy mode
Widgets recreated on every boot	No need to serialize LVGL object state — JSON is the source of truth
Fallback to legacy on parse error	Device always shows something, even if JSON is corrupt
Screen update destroys + recreates	Simplifies code — no incremental widget state tracking needed

---

3.2 Dynamic LVGL Rendering from Firmware JSON

The device parses firmware JSON at runtime and creates LVGL widgets dynamically — no recompilation needed.

3.2.1 Widget Factory

Mapping from firmware `sub_type` to LVGL create functions:

C

```c
// main/widget_factory.c
void widget_factory_create(lv_obj_t *parent, cJSON *widget_json) {
    const char *sub_type = cJSON_GetObjectItem(widget_json, "sub_type")->valuestring;
    lv_obj_t *obj = NULL;

    if      (strcmp(sub_type, "label") == 0)    { obj = lv_label_create(parent); }
    else if (strcmp(sub_type, "button") == 0)   { obj = lv_btn_create(parent); }
    else if (strcmp(sub_type, "arc") == 0)      { obj = lv_arc_create(parent); }
    else if (strcmp(sub_type, "bar") == 0)      { obj = lv_bar_create(parent); }
    else if (strcmp(sub_type, "image") == 0)    { obj = lv_image_create(parent); }
    else if (strcmp(sub_type, "switch") == 0)   { obj = lv_switch_create(parent); }
    else if (strcmp(sub_type, "slider") == 0)   { obj = lv_slider_create(parent); }
    else if (strcmp(sub_type, "dropdown") == 0) { obj = lv_dropdown_create(parent); }
    else if (strcmp(sub_type, "panel") == 0)    { obj = lv_obj_create(parent); }

    if (obj) {
        widget_factory_apply_position(obj, widget_json);    // x_pos, y_pos, width, height
        widget_factory_apply_style(obj, widget_json);       // style definition
        widget_factory_apply_events(obj, widget_json);      // CLICKED → LV_EVENT_CLICKED
        widget_factory_apply_state_flags(obj, widget_json); // hidden, disabled
        widget_factory_apply_properties(obj, widget_json);  // type-specific props
        widget_factory_apply_children(obj, widget_json);    // recursive
    }
}
```

Sub-type properties:

sub_type	LVGL create	Properties applied
label	lv_label_create()	obj_text, long_mode, recolor
button	lv_btn_create()	obj_text (inner label), inner_align, checked
arc	lv_arc_create()	min, max, value, mode
bar	lv_bar_create()	min, max, value, mode
image	lv_image_create()	src, rotation, pivot_x/y
switch	lv_switch_create()	checked
slider	lv_slider_create()	min, max, value, mode
dropdown	lv_dropdown_create()	options[], selected
panel	lv_obj_create()	children (container only)

3.2.2 Expression Engine

Resolves dynamic expressions to Modbus register values:

```c
// main/expression.c
float expression_eval(const char *expr) {
    if (strncmp(expr, "variables.", 10) == 0)
        return modbus_read_variable(expr + 10);
    if (strncmp(expr, "zones[", 6) == 0)
        return modbus_read_zone(expr);
    return (float)atof(expr);
}
```

Pattern	Example	Resolution
variables.<name>	variables.temp	Modbus register lookup
zones[N].<field>	zones[0].setpoint	Array + field access

3.2.3 Screen Manager

```c
// main/screen_manager.c
typedef struct { char name[32]; lv_obj_t *screen; cJSON *json; } device_screen_t;
device_screen_t g_screens[MAX_SCREENS];
int g_screen_count = 0;

void screen_manager_load_all(void) {
    // Read screen index from NVS, load each screen JSON, create LVGL objects
}
void screen_manager_switch(const char *name) {
    // lv_scr_load_anim(g_screens[i].screen, LV_SCR_LOAD_ANIM_FADE_ON, 200, 0, false);
}
void screen_manager_reload(const char *name) {
    // After BACnet deploy: destroy old screen, re-parse JSON, recreate widgets
}
```

3.2.4 Action Engine

C

```c
// main/action_engine.c
void action_engine_execute(cJSON *actions) {
    cJSON *action; cJSON_ArrayForEach(action, actions) {
        const char *type = cJSON_GetObjectItem(action, "action")->valuestring;
        if (strcmp(type, "changeScreen") == 0)
            screen_manager_switch(cJSON_GetObjectItem(action, "screen")->valuestring);
        else if (strcmp(type, "setVariable") == 0)
            modbus_write_variable(cJSON_GetObjectItem(action, "variable")->valuestring,
                                  cJSON_GetObjectItem(action, "value")->valuedouble);
    }
}
```

Files to create: `main/widget_factory.c/h`, `main/expression.c/h`, `main/screen_manager.c/h`, `main/action_engine.c/h`

3.3 Export Current Device Interface to JSON

The current firmware has 11 SquareLine Studio screens compiled as C code. They need to be exported to firmware JSON format so the frontend can load and render them.

3.3.1 Export Approach

SquareLine screens are C code — recreate each screen in EEZ Studio manually, then deploy:

TEXT

```
SquareLine .sls project → .c/.h files (in firmware)
       ↓ manually recreate in EEZ Studio
EEZ Studio .eez-project
       ↓ Deploy to Device (firmware-export.ts → firmware JSON)
firmware JSON per screen → NVS storage → Frontend can Import
```

3.3.2 Screen Mapping

SquareLine Screen	Firmware JSON	Widgets
`ui_HomeScreen`	`home`	Temperature, humidity, fan/mode buttons, WiFi icon
`ui_MainMenu`	`main_menu`	Navigation buttons to sub-screens
`ui_WifiConfig`	`wifi_config`	SSID, password, scan, IP fields
`ui_NetworkConfig`	`network_config`	IP, subnet, BACnet port
`ui_Protocols`	`protocols`	Protocol toggles
`ui_Parameters`	`parameters`	Read-only labels
`ui_Time`	`time`	Clock, timezone
`ui_ScheduleScreen`	`schedule`	Program list
`ui_ScheduleEditScreen`	`schedule_edit`	Day/time/temp editor
`ui_HolidayCalenderScreen`	`holidays`	Holiday picker
`ui_StartUpScreen`	`startup`	Logo

3.3.3 Widget Conversion Example

C

```c
// SquareLine C code:
ui_TemperatureVal = lv_label_create(ui_HomeScreen);
lv_label_set_text(ui_TemperatureVal, "22.5");
lv_obj_set_pos(ui_TemperatureVal, 80, 20);
lv_obj_set_style_text_font(ui_TemperatureVal, &ui_font_Arial80, LV_PART_MAIN);
```

JSON

```json
{
  "sub_type": "label", "x_pos": 80, "y_pos": 20,
  "obj_text": "22.5", "text_type": "literal",
  "style": { "DEFAULT": { "text_font": "Arial80" } }
}
```

3.3.4 Image Assets

SquareLine embeds images as C arrays (`ui_img_fan_auto_new_png.c`). The `lv_img_conv_v9` tool converts them to LVGL format. Image data is sent separately via `/api/v1/images` (REST) or `/api/eez-device/images/push/:panelId` (BACnet) and referenced by name in screen JSON. Bitmap names in the firmware JSON match the SquareLine asset names.

Device Interface Deployment via BACnet — Design

Send firmware screens from the EEZ Editor to T3000 hardware devices, and load them back for display in EEZ Studio.

1. **Push firmware to device** — Transfer the screen definitions from the editor to the hardware controller.
2. **Pull firmware from device** — Load the stored screen definitions back from the hardware for rendering in EEZ Studio.

---

1. Current State

`firmware-export.ts` transforms `.eez-project` → per-screen firmware JSON files (one per screen, ~30KB each). "Deploy" button writes files to disk — **does not reach hardware**. See EEZ Project JSON Format.

The T3000 already pushes graphic data to devices over BACnet. Firmware deploy can reuse the same path:

Existing pipe: `Browser → HTTP → Rust → FFI → C++ → BACnet → Device`

- `write_webview_data()` in `BacnetScreen.cpp:164` reads a zip file, chunks it into 200-byte blocks, and pushes it to the device via `WritePrivateData_Blocking`.
- `WEBVIEW_MESSAGE_TYPE` in `BacnetWebView.cpp:61` defines actions 0–17 — Rust has the same enum in `t3_ffi_sync_service.rs:53`.
- `call_handle_webview_msg()` in `t3_ffi_sync_service.rs:246` is the Rust → C++ bridge, calling `BacnetWebView_HandleWebViewMsg()` in `BacnetWebView_Exports.cpp:97` which dispatches to the handler.

---

2. Recommended Approach

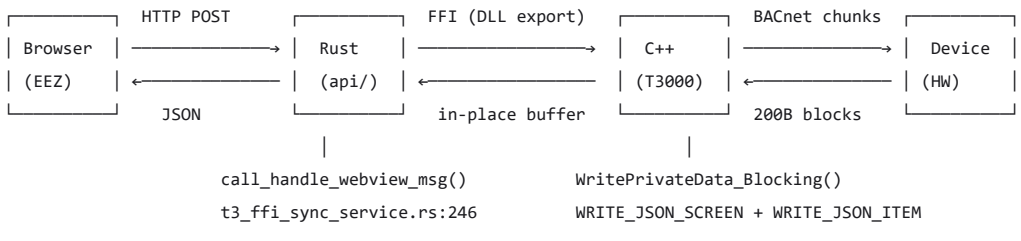
Extend the existing `WEBVIEW_MESSAGE_TYPE` enum with a new action (18) and reuse the same 200-byte chunked `WritePrivateData_Blocking` pattern from `SAVE_GRAPHIC_DATA` (action 2).

2.1. Send Firmware to Device

Layer	Change
Rust + C++ enum	Add `DEPLOY_FIRMWARE = 18` to `WebViewMessageType` in `t3_ffi_sync_service.rs:53` and `BacnetWebView.cpp:61`
Rust route	New `POST /api/devices/:id/deploy-firmware` — wraps firmware JSON with `action=18`, calls `call_handle_webview_msg()`
C++ handler	New `case DEPLOY_FIRMWARE` in `HandleWebViewMsg()` — compresses JSON, chunks 200B blocks, pushes via `WritePrivateData_Blocking(WRITE_JSON_ITEM)`
Browser	Wire "Deploy" button in `Toolbar.tsx` to the new endpoint

2.1.1. Architecture

TEXT



Why this path? The T3000 already has `SAVE_GRAPHIC_DATA` (action=2) doing exactly this — receiving graphic JSON from browser, writing to disk, zipping, chunking into 200-byte blocks, and pushing to device via `WritePrivateData_Blocking`. Firmware deploy is the same pattern: we skip the disk step and send firmware JSON directly through the Rust→C++ bridge.

2.1.2. New `WEBVIEW_MESSAGE_TYPE` Enum Extension

Both enums must stay in sync. Add to:

Rust — `api/src/t3_device/t3_ffi_sync_service.rs:53`:

```rust
pub enum WebViewMessageType {
    // ... existing 0-17 ...
    DEPLOY_FIRMWARE = 18,  // ← NEW: deploy firmware screen JSON to device
}
```

C++ — `T3000/BacnetWebView.cpp:61`:

```cpp
enum WEBVIEW_MESSAGE_TYPE {
    // ... existing 0-17 ...
    DEPLOY_FIRMWARE = 18,  // ← NEW
};
```

2.1.3. JSON Message Format (Browser → Rust → C++)

Browser sends to Rust endpoint; Rust wraps with action code and forwards to C++:

```json
{
    "action": 18,
    "panelId": 5,
    "serialNumber": 12345,
    "screens": [
        {
            "name": "Home",
            "json": "{\"fonts\":[{\"name\":\"regular_16\",\"size\":16}],\"bitmaps\":[\"bg_home\"],\"widgets\":{\"label_temp\":{\
        }
    ]
}
```

C++ responds:

```json
{
    "action": "DEPLOY_FIRMWARE_RES",
    "deployed": 3,
    "failed": 0,
    "status": true
}
```

2.1.4. C++ Handler (New in `HandleWebViewMsg`)

C++

```cpp
// T3000/BacnetWebView.cpp — inside HandleWebViewMsg()

case WEBVIEW_MESSAGE_TYPE::DEPLOY_FIRMWARE:
{
    int panelId = json["panelId"].asInt();
    UINT serialNumber = json["serialNumber"].asInt();
    const Json::Value& screens = json["screens"];

    // Find BACnet device instance from panel info
    int deviceInstance = 0;
    for (auto& info : g_bacnet_panel_info) {
        if (info.nseiral_number == serialNumber && info.panel_number == panelId) {
            deviceInstance = info.object_instance; break;
        }
    }
    if (!deviceInstance) {
        WrapErrorMessage(builder, tempjson, outmsg, _T("Device not found"));
        break;
    }

    int deployed = 0, failed = 0;
    for (int i = 0; i < screens.size(); i++) {
        std::string jsonStr = screens[i]["json"].asString();
        int ret = write_firmware_raw(deviceInstance, panelId, serialNumber, i, jsonStr);
        (ret >= 0) ? deployed++ : failed++;
    }

    tempjson["action"] = "DEPLOY_FIRMWARE_RES";
    tempjson["deployed"] = deployed;
    tempjson["failed"]   = failed;
    tempjson["status"]   = (failed == 0);
    outmsg = CString(Json::writeString(builder, tempjson).c_str());
    break;
}
```

2.1.5. BACnet Transfer — New `write_firmware_raw()` (Reuses Existing Pattern)

C++

```
// T3000/BacnetScreen.cpp — new function, variant of write_webview_data() line 164
// Same 200-byte chunked pattern, but accepts raw JSON string (no disk I/O)

int write_firmware_raw(int deviceInstance, int panelId,
                       UINT serialNumber, int screenIndex,
                       const std::string& jsonStr)
{
    // 1. Compress JSON in memory
    std::vector<char> compressed = ZipInMemory(jsonStr.data(), jsonStr.size());

    // 2. Verify device firmware supports the feature
    if (GetPrivateDataSaveSPBlocking(deviceInstance, READ_SETTING_COMMAND, 0, 0,
          sizeof(Str_Setting_Info), 1) <= 0) return -1;
    if (g_Device_Basic_Setting[panelId].reg.pro_info.firmware0_rev_main * 10
        + g_Device_Basic_Setting[panelId].reg.pro_info.firmware0_rev_sub < WEBVIEW_JSON_FEATURE)
        return -1;  // firmware too old (pre-v643)

    // 3. Set JSON flash mode flag
    g_Device_Basic_Setting[panelId].reg.webview_json_flash = 2;
    WritePrivateData_Blocking(deviceInstance, WRITE_SETTING_COMMAND, 0, 0, 5,
                              (char*)&g_Device_Basic_Setting[panelId].reg);

    // 4. Write screen metadata (zip size)
    g_json_screen_data[panelId][screenIndex].file_data.zip_size = compressed.size();
    WritePrivateData_Blocking(deviceInstance, WRITE_JSON_SCREEN,
                              screenIndex, screenIndex, 5,
                              (char*)&g_json_screen_data[panelId][screenIndex]);

    // 5. Chunk and write — 200 bytes per block, 2 blocks per BACnet write
    int totalChunks = compressed.size() / 200 + 1;
    for (int i = 0; i < totalChunks; i++) {
        int copySize = (i < totalChunks - 1) ? 200 : (compressed.size() % 200);
        memcpy(&g_json_item_data[panelId][10 * screenIndex + i],
               &compressed[i * 200], copySize);
    }

    for (int g = 0; g < totalChunks; g += BAC_READ_JSON_ITEM_GROUP_NUMBER) {
        int start = screenIndex * 10 + g;
        int end   = min(start + BAC_READ_JSON_ITEM_GROUP_NUMBER - 1, screenIndex * 10 + 9);
        WritePrivateData_Blocking(deviceInstance, WRITE_JSON_ITEM, start, end, 5,
                                  (char*)&g_json_item_data[panelId][start]);
        if (!offline_mode) Sleep(SEND_COMMAND_DELAY_TIME);
    }
    return 0;
}
```

Constants from `T3000/global_define.h`:

- `WEBVIEW_JSON_FEATURE = 643` — minimum firmware version
- `BAC_READ_JSON_ITEM_GROUP_NUMBER = 2` — chunks per BACnet write
- `SEND_COMMAND_DELAY_TIME` — ~50ms backoff between writes
- Each screen uses 10 slots (screen × 10 to screen × 10 + 9)

2.1.6. Rust Route (New)

RUST

```rust
// NEW FILE: api/src/t3_device/firmware_deploy_routes.rs

use axum::{extract::{Path, State}, http::StatusCode, response::Json, routing::post, Router};
use serde::Deserialize;
use serde_json::{json, Value};
use crate::app_state::T3AppState;
use crate::t3_device::t3_ffi_sync_service::{call_handle_webview_msg, WebViewMessageType};

pub fn create_firmware_deploy_routes() -> Router {
    Router::new()
        .route("/api/devices/:id/deploy-firmware", post(deploy_firmware))
}

#[derive(Deserialize)]
struct DeployFirmwareRequest {
    serial_number: i32,
    screens: Vec<FirmwareScreen>,
}

#[derive(Deserialize)]
struct FirmwareScreen {
    name: String,
    json: Value,  // the full screen JSON from firmware-export.ts
}

async fn deploy_firmware(
    State(_state): State<T3AppState>,
    Path(device_id): Path<i32>,
    Json(payload): Json<DeployFirmwareRequest>,
) -> Result<Json<Value>, StatusCode> {
    // Wrap with action code, matching C++ WEBVIEW_MESSAGE_TYPE::DEPLOY_FIRMWARE = 18
    let json_str = serde_json::to_string(&json!({
        "action": WebViewMessageType::DEPLOY_FIRMWARE as i32,
        "panelId": device_id,
        "serialNumber": payload.serial_number,
        "screens": payload.screens.iter().map(|s| json!({
            "name": s.name,
            "json": s.json.to_string()
        })).collect::<Vec<_>>(),
    })).unwrap();

    // Reuse the same 65536-byte buffer pattern as all other FFI calls
    let mut buffer = vec![0u8; 65536];
    buffer[..json_str.len()].copy_from_slice(json_str.as_bytes());

    match call_handle_webview_msg(WebViewMessageType::DEPLOY_FIRMWARE as i32, &mut buffer) {
        Ok(0) => {
            let response: Value = serde_json::from_slice(&buffer)
                .unwrap_or(json!({"status": "ok"}));
            Ok(Json(response))
        }
        _ => Err(StatusCode::INTERNAL_SERVER_ERROR),
    }
}
```

Register in `api/src/t3_device/routes.rs`:

```rust
use crate::t3_device::firmware_deploy_routes::create_firmware_deploy_routes;
// inside the Router builder:
.merge(create_firmware_deploy_routes())
```

2.1.7. Browser Side (Wire Deploy Button)

```typescript
// In Toolbar.tsx — the "Deploy to Device" button handler
// Firmware JSON already generated by firmware-export.ts → transformToDeviceJson()

async function handleDeploy(deviceId: number, serialNumber: number) {
    const project = parseProject();  // parse .eez-project
    const deviceScreens = transformToDeviceJson(project);  // existing function
    const screens = Object.entries(deviceScreens).map(([name, screen]) => ({
        name,
        json: screen  // full DeviceScreen object
    }));

    const response = await fetch(`/api/devices/${deviceId}/deploy-firmware`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ serial_number: serialNumber, screens })
    });

    const result = await response.json();
    showNotification(`Deployed: ${result.deployed}, Failed: ${result.failed}`);
}
```

2.2. Read Firmware from Device

Gap: `GET_WEBVIEW_LIST` (action 17) reads the graphic editor format, not firmware JSON. To read firmware screens back, we need either:

- A new action (e.g., `READ_FIRMWARE = 19`) in C++ that reads firmware JSON from the device, or
- Extend the existing `Read_Webview_Data_Special()` to return firmware JSON instead of graphic format

The device firmware, on boot, parses the stored JSON and creates LVGL widgets from the definitions.

---

3. Why This Approach (Not Others)

Option	Pros	Cons	Verdict
BACnet/WS (REST)	Standard, JSON-native	Requires HTTP server on device, complex TLS	❌ Overkill for embedded
BACnet ObjectFile	Standard BACnet object	File size limits, complex state machine	❌ Poor fit for ~30KB JSON
BACnet/SC (WebSocket)	Modern transport	Requires hub, new stack, certificate mgmt	❌ Major refactor
Extend existing private transfer	Already works, proven, devs know it	Non-standard, needs new message type (18)	✅ Fastest, lowest risk

---

4. Implementation Plan

```
TEXT

Phase 1: C++ Layer (T3000_Building_Automation_System/T3000/)
  ├── BacnetWebView.cpp    — add DEPLOY_FIRMWARE = 18 to enum + case handler
  ├── BacnetScreen.cpp     — add write_firmware_raw() function
  └── BacnetScreen.h       — declare write_firmware_raw()

Phase 2: Rust Layer (api/src/t3_device/)
  ├── t3_ffi_sync_service.rs      — add DEPLOY_FIRMWARE = 18 to enum
  ├── firmware_deploy_routes.rs   — NEW: POST /api/devices/:id/deploy-firmware
  └── routes.rs                   — register create_firmware_deploy_routes()

Phase 3: Browser Layer (src/lib/t3-eez-studio/)
  ├── firmware-export.ts   — already done (transformToDeviceJson exists)
  └── Toolbar.tsx          — wire "Deploy" button to POST /api/devices/:id/deploy-firmware

Phase 4: Device Firmware (T3000 hardware)
  ├── Receive chunks → reassemble → decompress → store to flash
  └── Parse JSON → create LVGL widgets → render UI on boot
```

---

5. Data Flow Summary

```
TEXT

Step 1: User clicks "Deploy" in EEZ Editor
   ↓  firmware-export.ts transforms .eez-project → per-screen JSON

Step 2: Browser POSTs to Rust
   ↓  POST /api/devices/5/deploy-firmware
   ↓  Body: { serial_number: 12345, screens: [{name:"Home", json:{...}}, ...] }

Step 3: Rust wraps + calls FFI
   ↓  Adds "action": 18 (DEPLOY_FIRMWARE)
   ↓  call_handle_webview_msg(18, json_buffer) → BacnetWebView_HandleWebViewMsg(18, msg, 65536)

Step 4: C++ dispatches
   ↓  HandleWebViewMsg → case DEPLOY_FIRMWARE
   ↓  Validates panelId/serialNumber → finds deviceInstance

Step 5: C++ transfers to device via BACnet
   ↓  write_firmware_raw(deviceInstance, panelId, serial#, screenIdx, jsonStr)
   ↓  Compress → chunk 200B blocks → WritePrivateData_Blocking(WRITE_JSON_ITEM)

Step 6: Device receives and stores
   ↓  Reassemble chunks → decompress → flash: /screens/Home.json
   ↓  On reload: parse JSON → create LVGL widgets → render UI
```

---

6. Device-Hosted REST API (Direct Communication)

6.1. Motivation

The BACnet path routes everything through T3000 (`Browser → Rust → C++ → BACnet → Device`). The new requirement is:

- **ESP32 hosts an embedded HTTP REST API server** (e.g., ESP-IDF HTTP Server or Mongoose)
- **Clients communicate directly** with the device — no mandatory T3000 hop
- **T3000 can still read/write** device data, but as a peer HTTP client, not a mandatory gateway
- **Delta updates**: after the initial full sync, send only changed keys to reduce payload

6.2. Architecture Comparison

TEXT

```
BEFORE (BACnet-only):                          AFTER (REST API primary):
┌──────────┐  HTTP  ┌──────────┐  FFI  ┌──────┐  BACnet  ┌──────────┐
│ Browser  │ ─────→ │  Rust    │ ────→ │ C++  │ ───────→ │  Device  │
│ (EEZ)    │ ←───── │  (api/)  │ ←──── │(T3000)│ ←─────── │  (ESP32) │
└──────────┘  JSON  └──────────┘       └──────┘  chunks  └──────────┘

┌──────────┐  HTTP (REST)                          ┌──────────┐
│ Browser  │ ──────────────────────────────────────→ │  Device  │
│ (EEZ)    │ ←────────────────────────────────────── │  (ESP32) │
└──────────┘  JSON (direct)                         └──────────┘
     │                                                    │
     └── GET/PUT /api/v1/screens ─────────────────────────┘

┌──────────┐  HTTP (REST)                          ┌──────────┐
│ T3000    │ ──────────────────────────────────────→ │  Device  │
│ (Rust)   │ ←────────────────────────────────────── │  (ESP32) │
└──────────┘  peer client                           └──────────┘
```

6.3. Default Flow: Full Sync

The primary design is full sync — send all, load all. Delta is an optimization layered on top.

Import flow (two-step):

Step	Endpoint	Purpose
Device summary	`GET /api/v1/device/info`	Lightweight metadata: panel name, screen size, screen list, image count
Load all screens	`GET /api/v1/screens`	Pull ALL screens with full widget JSON
Deploy to device	`PUT /api/v1/screens`	Push ALL screens to device in one request

TEXT

```
First connect / Import:
  Browser ── GET /api/v1/device/info ──→ Device
  Browser ←── {panel_name, screen_size:{width,height}, screen_count,
               screens:["Home","Settings",...], image_count, ...} ── Device

  Browser ── GET /api/v1/screens ──→ Device
  Browser ←── {screens: [{name:"Home", json:{widgets:{...},fonts:[...],...}}, ...]} ── Device

First deploy:
  Browser ── PUT /api/v1/screens {screens: [...]} ──→ Device
  Browser ←── {deployed: 3, status: "ok"} ── Device
```

6.4. Full API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/device/info`	**Device summary** — panel name, screen size, screen list, image/font counts (no widget JSON)
`GET`	`/api/v1/screens`	**Load all** — full JSON for every screen
`PUT`	`/api/v1/screens`	**Deploy all** — full JSON for every screen (body: `{screens: [...]}`)
`GET`	`/api/v1/screens/:name`	Load single screen
`PUT`	`/api/v1/screens/:name`	Deploy/replace single screen
`PATCH`	`/api/v1/screens/:name`	Delta update — only changed keys (optimization, see §6.5)
`PATCH`	`/api/v1/screens/:name/widgets/:id`	Delta single widget (optimization)
`POST`	`/api/v1/screens/:name/actions`	Execute action (button press, setpoint change, etc.)

6.4.1. Device Info Response (`GET /api/v1/device/info`)

JSON

```
{
    "panel_name": "T3-BB",
    "serial_number": 0,
    "screen_size": { "width": 480, "height": 320 },
    "screen_count": 11,
    "screens": [
        "start_up_screen",
        "home_screen",
        "main_menu",
        "schedule_screen",
        "schedule_edit_screen",
        "holiday_calender_screen",
        "network_config",
        "protocols",
        "parameters",
        "time",
        "wifi_config"
    ],
    "image_count": 16,
    "font_count": 1,
    "firmware_version": "5.1.0"
}
```

6.5. Delta Update Protocol (`PATCH` — Optimization)

After the initial full sync is established, subsequent updates can send only modified key paths:

TEXT

```
PATCH /api/v1/screens/Home
{
  "changes": [
    { "path": "widgets.temp_label.obj_text",           "value": "zones[2].temperature" },
    { "path": "widgets.fan_switch.state",               "value": 1 },
    { "path": "widgets.mode_btn.style.checked.bg_color", "value": 4278190080 }
  ]
}
```

The device merges these changes into its stored JSON and hot-reloads only the affected widgets — avoiding a full screen rebuild.

Response:

JSON

```
{
  "applied": 3,
  "rejected": 0,
  "status": "ok"
}
```

6.6. Mixed-Mode: REST Primary + BACnet Fallback

- **Primary path**: Browser → HTTP → ESP32 REST API (direct, no T3000)
- **Fallback path**: Browser → T3000 → BACnet → Device (when device is not on the same local network, or behind a BACnet router)

The EEZ Editor detects reachability at connect time:

TEXT

```
1. Try GET http://<device-ip>/api/v1/screens
   ├─ 200 OK → Use REST API (direct mode)
   └─ Timeout / Unreachable → Fall back to BACnet path through T3000
```

6.7. Device-Side Implementation Notes

ESP32 HTTP Server options:

- ESP-IDF HTTP Server (`esp_http_server.h`) — lightweight, built-in
- Mongoose — single-file embedded networking library
- ESPAsyncWebServer — popular Arduino-compatible option

Storage:

- Screens stored as individual JSON files in SPIFFS/LittleFS: `/screens/<name>.json`
- On `PUT /api/v1/screens` , each screen is written to flash and validated
- On boot, device reads all screen JSONs and builds LVGL widgets

Concurrency:

- The HTTP server runs on a dedicated FreeRTOS task
- Screen updates are queued; the LVGL task applies them on the next render cycle
- Mutex guards the shared screen JSON state

6.8. Data Flow Summary (REST API Path)

TEXT

```
Step 1: User clicks "Deploy" in EEZ Editor
    → firmware-export.ts transforms .eez-project → per-screen JSON

Step 2: Browser sends directly to device
    → PUT http://<device-ip>/api/v1/screens
    → Body: { screens: [{name:"Home", json:{...}}, ...] }

Step 3: ESP32 HTTP server receives
    → Validates JSON structure
    → Writes each screen to flash: /screens/<name>.json

Step 4: ESP32 applies changes
    → Queues screen update for LVGL task
    → LVGL task rebuilds affected widgets on next render cycle

Step 5: Device responds
    → { deployed: 3, failed: 0, status: "ok" }
```

6.9. Image/Icon API — Binary Assets

Images and icons (bitmaps, PNGs, compressed LVGL image formats) are binary assets, not JSON. They are transferred as complete files — never partial or delta updates. The display JSON references them by `imageId` rather than embedding raw bytes.

6.9.1. Design Rationale

Concern	Decision
Embed vs. reference	Reference by `imageId` — keeps display JSON lean (~30KB), avoids base64 bloat
Transfer unit	Always full file — images are opaque binary blobs; partial updates are meaningless
Caching	`ETag` + `If-None-Match` — device returns 304 if the image hasn't changed
Format	LVGL-compatible binary (`.bin` header + RGBA/INDEXED pixel data), or raw PNG for editor convenience
Storage	Per-image file in SPIFFS/LittleFS: `/images/<id>.bin`

6.9.2. API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/images`	**List all** — returns `{ images: [{id, name, width, height, format, size, checksum}] }`
`GET`	`/api/v1/images/:id`	**Download** — binary image blob (Content-Type: `application/octet-stream`)
`GET`	`/api/v1/images/:id/info`	**Metadata** — `{id, name, width, height, format, size, checksum}` without the binary body
`PUT`	`/api/v1/images/:id`	**Upload/replace** — binary body, creates or overwrites
`POST`	`/api/v1/images`	**Upload new** — multipart form with `id` , `name` , and binary `file` field; returns `{id, status}`
`DELETE`	`/api/v1/images/:id`	**Remove** — delete image from device flash
`GET`	`/api/v1/images/:id?format=png`	**Convert & download** — device converts `.bin` → PNG on-the-fly for editor previews

6.9.3. How Display JSON References Images

Screen JSON uses an `imageId` field instead of embedding pixel data:

JSON

```
{
  "name": "Home",
  "bitmaps": [
    { "id": "bg_home",      "name": "bg_home.bin",      "type": "background" },
    { "id": "icon_fan_on",  "name": "icon_fan_on.bin",  "type": "icon" },
    { "id": "icon_fan_off", "name": "icon_fan_off.bin", "type": "icon" }
  ],
  "widgets": {
    "bg": {
      "type": "Image",
      "imageId": "bg_home",
      "x_pos": 0, "y_pos": 0,
      "width": 480, "height": 320
    },
    "fan_icon": {
      "type": "Image",
      "imageId": "icon_fan_off",
      "action": "setImageId",
      "bindings": [
        { "property": "imageId", "expression": "fans[0].running ? 'icon_fan_on' : 'icon_fan_off'" }
      ]
    }
  }
}
```

Key rules:

- `bitmaps` declares all images the screen needs — the editor uses this for bundle calculation.
- `imageId` in a widget is a string key, resolved to a file via the bitmaps table.
- Dynamic image switching (e.g., fan on/off) uses `imageId` binding expressions — the device maps the string to the cached binary at render time.
- The `imageId` namespace is **per-device** (not per-screen). Screens can share images to avoid duplicate transfers.

6.9.4. Upload Flow

TEXT

```
Step 1: Editor collects all imageIds referenced by the project
    → scans .eez-project bitmaps → resolves to local file paths

Step 2: Check what the device already has
    → GET /api/v1/images
    → Response: { images: [{id:"bg_home", checksum:"a1b2c3"}, ...] }

Step 3: Compute diff — skip images with matching checksums
    → only upload new or changed images

Step 4: Upload each image (parallel for throughput, sequential for embedded devices)
    → PUT /api/v1/images/bg_home
    → Header: Content-Type: application/octet-stream
    → Header: X-Image-Name: bg_home.bin
    → Header: X-Image-Checksum: sha256:a1b2c3d4...
    → Body: <binary pixel data>

Step 5: Device stores
    → Validates binary header (magic bytes, dimensions, format)
    → Writes to flash: /images/bg_home.bin
    → Updates checksum registry for fast diff on next connect
    → Responds: { id: "bg_home", status: "stored", checksum: "sha256:a1b2c3d4..." }
```

6.9.5. Download Flow (Editor: Import from Device)

TEXT

```
Step 1: Editor requests image list
    → GET /api/v1/images
    → Knows which images exist, their sizes, and checksums

Step 2: For each image needed by the editor:
    → GET /api/v1/images/bg_home
    → Header: If-None-Match: "a1b2c3d4"   ← client's cached checksum

Step 3: Device responds:
    → 200 OK + binary body           (image is new or changed)
    → 304 Not Modified               (client already has this version — skip)
```

6.9.6. Caching & ETag

The device computes an ETag from the image's content hash (SHA-256 or CRC32 for constrained devices):

TEXT

```
GET /api/v1/images/bg_home
→ 200 OK
  ETag: "a1b2c3d4e5f6..."
  Content-Type: application/octet-stream
  Content-Length: 153600
  X-Image-Format: rgba8888
  X-Image-Width: 480
  X-Image-Height: 320

GET /api/v1/images/bg_home
  If-None-Match: "a1b2c3d4e5f6..."
→ 304 Not Modified
```

This avoids retransferring large images that haven't changed between editing sessions.

6.9.7. Format Negotiation

The device stores images in its native LVGL binary format. The editor can request conversion:

Query	Response Content-Type	Use Case
(none)	`application/octet-stream`	Raw LVGL `.bin` for display rendering
`?format=png`	`image/png`	Editor preview, thumbnail generation
`?format=raw`	`application/octet-stream` + headers	Machine-to-machine transfer

If the device cannot convert (`?format=png` on a memory-constrained MCU), it returns `501 Not Implemented`. The editor falls back to downloading the raw `.bin` and converting client-side (via `lv_img_conv` in the browser).

6.9.8. Device-Side Implementation Notes

Storage layout:

```
TEXT

/spiffs/images/
├── _registry.json          ← { "bg_home": {"file":"bg_home.bin","checksum":"a1b2",...}, ... }
├── bg_home.bin
├── icon_fan_on.bin
└── icon_fan_off.bin
```

ESP32 handler pseudocode:

C

```c
// GET /api/v1/images/:id
esp_err_t image_get_handler(httpd_req_t *req) {
    char id[64];
    get_path_param(req, "id", id);

    // Check ETag for 304
    char etag[65];
    registry_get_checksum(id, etag);
    if (check_if_none_match(req, etag)) {
        httpd_resp_set_status(req, "304 Not Modified");
        return httpd_resp_send(req, NULL, 0);
    }

    // Read from flash
    char path[128];
    snprintf(path, sizeof(path), "/spiffs/images/%s.bin", id);
    FILE *f = fopen(path, "rb");
    if (!f) { httpd_resp_send_404(req); return ESP_FAIL; }

    // Stream binary response
    httpd_resp_set_type(req, "application/octet-stream");
    httpd_resp_set_hdr(req, "ETag", etag);
    httpd_resp_set_hdr(req, "X-Image-Format", "rgba8888");

    char buf[1024];
    size_t n;
    while ((n = fread(buf, 1, sizeof(buf), f)) > 0) {
        httpd_resp_send_chunk(req, buf, n);
    }
    httpd_resp_send_chunk(req, NULL, 0);  // end chunked
    fclose(f);
    return ESP_OK;
}

// PUT /api/v1/images/:id
esp_err_t image_put_handler(httpd_req_t *req) {
    char id[64];
    get_path_param(req, "id", id);

    // Read body in chunks, write to flash
    char path[128];
    snprintf(path, sizeof(path), "/spiffs/images/%s.bin", id);
    FILE *f = fopen(path, "wb");

    char buf[1024];
    int received, total = 0;
    while ((received = httpd_req_recv(req, buf, sizeof(buf))) > 0) {
        fwrite(buf, 1, received, f);
        total += received;
    }
    fclose(f);

    // Update registry
    char checksum[65];
    compute_sha256(path, checksum);
    registry_set(id, path, total, checksum);

    // Respond
```

```
    char resp[256];
    snprintf(resp, sizeof(resp), "{\"id\":\"%s\",\"status\":\"stored\",\"size\":%d,\"checksum\":\"%s\"}", id, total, checksum);
    httpd_resp_set_type(req, "application/json");
    return httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
}
```

Memory budget (ESP32):

- 1024-byte streaming buffer (reused for both GET and PUT)
- Registry in RAM: ~100 bytes per image × typical 20 images = 2KB
- Flash budget: depends on SPIFFS partition size — typically 1–4MB for images
- No image decoding in the HTTP handler — LVGL decodes from flash on render demand

6.9.9. Updated Full API Surface

Method	Endpoint	Purpose
`GET`	`/api/v1/screens`	Load all screens
`PUT`	`/api/v1/screens`	Deploy all screens
`GET`	`/api/v1/screens/:name`	Load single screen
`PUT`	`/api/v1/screens/:name`	Deploy/replace single screen
`PATCH`	`/api/v1/screens/:name`	Delta update — only changed keys
`PATCH`	`/api/v1/screens/:name/widgets/:id`	Delta single widget
`POST`	`/api/v1/screens/:name/actions`	Execute action
`GET`	`/api/v1/images`	**List all images**
`GET`	`/api/v1/images/:id`	**Download image binary**
`GET`	`/api/v1/images/:id/info`	**Image metadata**
`PUT`	`/api/v1/images/:id`	**Upload/replace image**
`POST`	`/api/v1/images`	**Upload new image (multipart)**
`DELETE`	`/api/v1/images/:id`	**Remove image**

6.9.10. Full Deploy Flow (Screens + Images)

```
User clicks "Deploy":

Phase 1 — Images first (larger, fewer round-trips)
  ├── GET  /api/v1/images                  → list + checksums
  ├── Diff: skip matching checksums
  └── PUT  /api/v1/images/bg_home          → for each new/changed image
      PUT  /api/v1/images/icon_fan_on
      PUT  /api/v1/images/icon_fan_off
      PUT  /api/v1/images/icon_mode_heat

Phase 2 — Screens (now safe: all imageIds resolve)
  └── PUT  /api/v1/screens                 → full JSON deployment

Result: Device has all images on flash before any screen references them.
```

Import from Device — Design

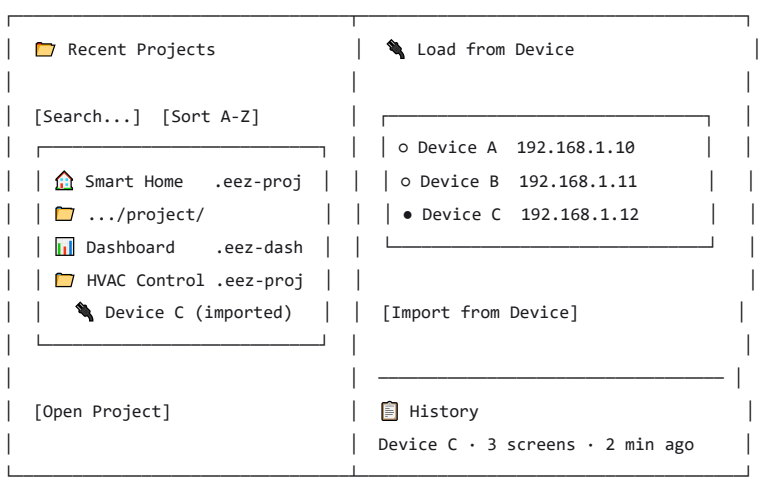
Import firmware screens from a T3000 hardware controller, convert to `.eez-project`, and open in EEZ Studio.

---

1. UI Layout

Two-column layout on the Open tab:

TEXT



---

2. Step-by-Step Implementation

Step 1 — List Devices

File: `src/lib/t3-eez-studio/home/open-projects.tsx`

Add a 2-column layout. Right column fetches device list from `/api/devices`:

TYPESCRIPT

```typescript
// New state for right column
const [devices, setDevices] = useState<DeviceInfo[]>([]);
const [selectedDevice, setSelectedDevice] = useState<DeviceInfo | null>(null);
const [importLog, setImportLog] = useState<string[]>([]);
const [importing, setImporting] = useState(false);

// Fetch devices on mount
useEffect(() => {
    fetch('/api/devices')
        .then(r => r.json())
        .then(setDevices)
        .catch(() => setDevices([]));
}, []);
```

Step 2 — Select Device & Click Import

Right column shows radio list of devices. User selects one, clicks [Import from Device]:

TYPESCRIPT

```
<button
    disabled={!selectedDevice || importing}
    onClick={() => startImport(selectedDevice)}
>
    Import from Device
</button>
```

Step 3 — Create Project Skeleton

Create the project folder and `device-import/` staging directory before any network I/O. This ensures the project container exists on disk even if the device never responds.

TYPESCRIPT

```
// Step 3 — Create project skeleton
const projectDir = `project/${device.panel_name}`;
const stagingDir = `${projectDir}/device-import`;
await fetch(`/api/files/mkdir?path=${encodeURIComponent(stagingDir)}`, { method: "POST" });
```

Step 4 — Fetch Screens via REST API (Primary) with BACnet Fallback

Uses `DeviceRestClient` (`src/lib/t3-eez-studio/project-editor/build/device-rest-client.ts`) which probes the device for REST API availability and falls back to BACnet through T3000 if unreachable.

Primary path (REST): `GET http://<device-ip>/api/v1/screens` — single request, all screens returned at once. **Fallback path (BACnet):** `POST /api/devices/:id/read-firmware` through T3000 — 200-byte chunks, per-screen.

The screen JSONs are written to `device-import/<name>.json` immediately — if the connection drops mid-transfer, already-fetched screens are safe on disk.

The `device-import/` folder is **kept after import** — it's a local cache. Re-import reads from disk, skipping already-fetched screens (no re-fetch needed).

TYPESCRIPT

```
// Step 4 — Probe device, fetch all screens, save each to device-import/
import { DeviceRestClient } from "project-editor/build/device-rest-client";

const client = new DeviceRestClient();

// Probe: tries REST first, falls back to BACnet if unreachable
const conn = await client.connect(device.ip, device.panel_id, device.serial_number);
addLog(`Connected via ${conn.mode.toUpperCase()}`);

// Fetch all screens in one call (REST) or progressive loop (BACnet)
const result = await client.loadAllScreens();
addLog(`Loaded ${result.screens.length} screens`);

// Save each screen to device-import/ immediately
const stagingScreens: FirmwareScreen[] = [];
for (const screen of result.screens) {
    const screenPath = `${stagingDir}/${screen.name}.json`;
    await fetch(`/api/files/write?path=${encodeURIComponent(screenPath)}`,
        { method: "PUT", body: JSON.stringify(screen.json) });
    stagingScreens.push(screen);
}
```

Step 5 — Build .eez-project from device-import/

Read the raw device JSONs from `device-import/`, convert to `.eez-project` format, save to project root.

```typescript
// Step 5 — Build .eez-project from staging
const { firmwareToProject } = await import("project-editor/build/firmware-loader");
const project = firmwareToProject(stagingScreens, {
    panel_name: device.panel_name,
    serial_number: device.panel_serial_number,
});

const projectPath = `${projectDir}/${device.panel_name}.eez-project`;
await fetch(`/api/files/write?path=${encodeURIComponent(projectPath)}`,
    { method: "PUT", body: JSON.stringify(project, null, 2) });
```

Step 6 — Open in Editor

```typescript
// Step 6 — Open project in EEZ editor
settingsController.addItemToMRU(projectPath, { projectType: "LVGL", hasFlowSupport: true });
const readResp = await fetch(`/api/files/read?path=${encodeURIComponent(projectPath)}`);
const projectJson = await readResp.json();
await initProjectEditor(tabs, ProjectEditorTab);
```

Step 7 — Transport Layer (Implemented: DeviceRestClient)

The transport is handled entirely by `DeviceRestClient` at `src/lib/t3-eez-studio/project-editor/build/device-rest-client.ts`. It encapsulates both paths:

Path	Load (Import)	Deploy
REST (primary)	`GET http://<ip>/api/v1/screens`	`PUT http://<ip>/api/v1/screens`
BACnet (fallback)	`POST /api/devices/:id/read-firmware` (through T3000)	`POST /api/devices/:id/deploy-firmware` (through T3000)

Connection probe (`client.connect(ip, panelId, serialNumber)`):

1. Tries `HEAD http://<ip>/api/v1/screens` with 2s timeout
2. If reachable → REST mode
3. If unreachable → BACnet fallback (requires panelId + serialNumber)
4. If neither works → error

No new Rust endpoints needed — the REST path goes directly to the ESP32 device. The BACnet fallback path reuses the existing T3000 bridge infrastructure (actions 18/19).

Step 8 — Import History

After import, save a history entry so the user can reopen the drawer:

```typescript
interface ImportHistoryEntry {
    deviceName: string;
    serialNumber: number;
    screenCount: number;
    timestamp: Date;
    log: string[];
}

// Save to localStorage after import
const history = JSON.parse(localStorage.getItem('importHistory') || '[]');
history.unshift({
    deviceName: device.panel_name,
    serialNumber: device.serial_number,
    screenCount: screens.length,
    timestamp: new Date(),
    log: importLog,
});
localStorage.setItem('importHistory', JSON.stringify(history.slice(0, 20)));
```

Step 9 — Badge on Imported Projects

Show `🔌 Device C (imported)` badge in the left column for projects with `importedFrom` metadata:

```typescript
// In open-projects.tsx renderNode
const hasImportMeta = mruItem.filePath && /* check project JSON for importedFrom */;
{hasImportMeta && (
    <span className="import-badge" title={`From ${meta.device} · ${meta.importedAt}`}>
        🔌 {meta.device} (imported)
    </span>
)}
```

---

3. Project Template

Setting	Value
Project type	LVGL with flow support
LVGL version	`9.5.0` (fixed, matches hardware)
File extension	`.eez-project`
Project root	`<data_root>/project/<DeviceName>/`
Project file	`<data_root>/project/<DeviceName>/<DeviceName>.eez-project`
Staging cache	`<data_root>/project/<DeviceName>/device-import/*.json` (kept, never deleted)
Metadata	`importedFrom: { device, serialNumber, importedAt }`

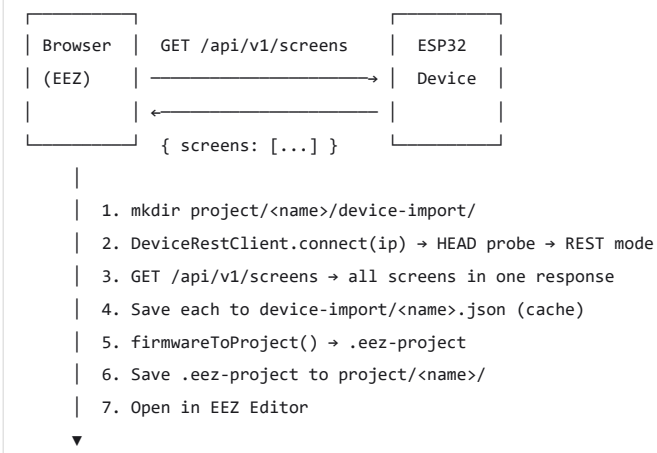
```text
project/<DeviceName>/
  ├── <DeviceName>.eez-project     ← built from device-import/
  └── device-import/               ← raw device JSONs (cache, kept)
        ├── Screen1.json
        ├── Screen2.json
        └── Screen3.json
```

---

4. Flow Summary

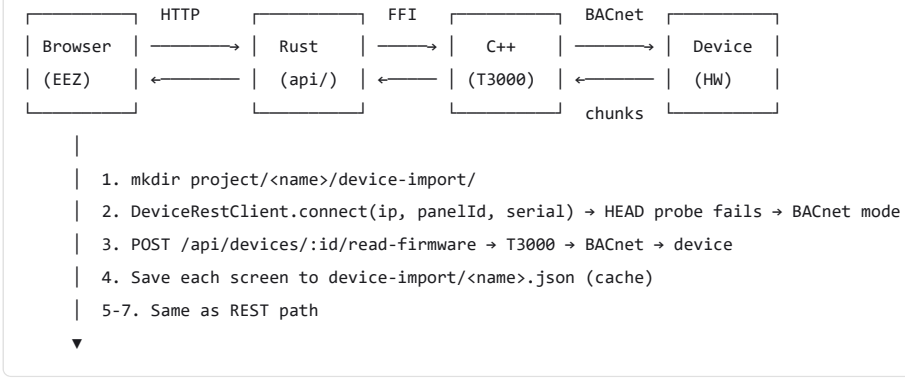
Primary: REST API (direct to ESP32 device)

TEXT



Fallback: BACnet through T3000 (device unreachable via REST)

TEXT



Key behaviors:

- `DeviceRestClient` probes REST first, falls back to BACnet automatically
- `device-import/` folder is created before any network I/O (safe if device offline)
- Each screen saved to `device-import/` immediately on arrival (survives connection drop)
- `device-import/` is never deleted — re-import rebuilds `.eez-project` from cache
- REST path: single `GET` returns all screens at once
- BACnet path: 200-byte chunked transfer through T3000

.EEZ-Project JSON Format — Design Document

Overview

The `.eez-project` file is a JSON document that describes a complete LVGL UI project created in EEZ Studio. This document covers the full structure of the JSON format.

Context: The T3000 Webview platform uses EEZ Studio (browser-based) as its UI designer. Projects designed in the browser export to this JSON format. The format is consumed by:

- The browser-based simulator (renders the UI in a web canvas using LVGL WASM)
- The Rust backend (provides font extraction & file management)
- The embedded firmware (creates LVGL widgets dynamically on the device)

Two Output Formats

EEZ Studio produces **two different JSON formats** — one for the editor, one for the device:

```mermaid
flowchart TD
    A[EEZ Studio Browser]
    A -->|"Save" button| B[.eez-project JSON]
    A -->|"Deploy to Device" button| C[Firmware JSON]
    B --> D[Disk / Editor reload]
    C --> E[BACnet Transfer]
    E --> F[Hardware Firmware]
    F -->|Parse JSON| G[EEZ Embedded Runtime]
    G -->|Create widgets| H[LVGL Display]

    H -.->|"Import from Device"| I[Reverse transform]
    I -.-> A
```

Button	Output	Consumer	Size (Smart Home example)
Save	`.eez-project` JSON	Editor (reopen), build system	~1.79 MB (full embedded assets)
Deploy to Device	Per-screen firmware JSON	Hardware firmware	~30 KB per screen (stripped)
Import from Device	`.eez-project` JSON (reconstructed)	Editor (open existing device UI)	Reverse of Deploy

---

1. Top-Level Structure

```json
{
  "settings":     { ... },   // Project-wide settings
  "variables":    { ... },   // Global variables & structures
  "actions":      [ ... ],   // Reusable action definitions
  "userPages":    [ ... ],   // All pages (screens)
  "userWidgets":  [ ... ],   // Reusable custom widgets
  "lvglStyles":   [ ... ],   // LVGL style definitions
  "lvglGroups":   [ ... ],   // Widget groups
  "fonts":        [ ... ],   // Font assets
  "bitmaps":      [ ... ],   // Image/bitmap assets
  "colors":       [ ... ],   // Named colors
  "themes":       [ ... ]    // Theme definitions
}
```

---

2. Settings

JSON

```
{
  "settings": {
    "general": {
      "projectVersion": "v3",          // project format version
      "projectType": "lvgl",          // "lvgl" | "dashboard" | "lvgl+flow"
      "lvglVersion": "9.5.0",             // "8.4.0" | "9.0" | "9.2.2" | "9.3.0" | "9.4.0" | "9.5.0"
      "flowSupport": true,            // enables flow engine for widget bindings
      "displayWidth": 800,            // target display width in px
      "displayHeight": 480,           // target display height in px
      "colorFormat": "RGB",           // "RGB" | "BGR" | "RGB565" | "ARGB8888" | "XRGB8888"
      "darkTheme": false,             // dark theme enabled
      "cacheFonts": true,             // cache rasterized fonts to disk
      "extensions": [],               // enabled extension IDs
      "imports": []                   // imported project paths
    },
    "build": {
      "configurations": [
        {
          "name": "Default"
          // configuration-specific overrides
        }
      ],
      "files": [
        {
          "fileName": "screens.c",
          "template": "// C template for code generation"
        }
      ]
    }
  }
}
```

Key fields for embedded runtime:

Field	Purpose
`projectType`	Determines if flow engine is needed
`lvglVersion`	Select correct LVGL API version (8.x vs 9.x)
`displayWidth/Height`	Set LVGL display buffer size
`colorFormat`	Set `LV_COLOR_DEPTH` and byte order
`flowSupport`	If true, must run flow engine for widget bindings

3. Variables (Data Bindings)

JSON

```
{
  "variables": {
    "globalVariables": [
      {
        "name": "temperature",            // variable name (used in expressions)
        "type": "float",                  // "float" | "integer" | "boolean" | "string" | enum/struct name
        "defaultValue": "0.0",             // initial value as string
        "persist": true                    // save value across reboots
      },
      {
        "name": "buttonPressed",
        "type": "boolean",
        "defaultValue": "false",
        "persist": false
      }
    ],
    "structures": [
      {
        "name": "Point",
        "fields": [
          { "name": "x", "type": "float" },
          { "name": "y", "type": "float" }
        ]
      }
    ],
    "enums": [
      {
        "name": "FanSpeed",
        "members": ["OFF", "LOW", "MEDIUM", "HIGH"]
      }
    ]
  }
}
```

Variables in expressions use dot notation:

TEXT

```
variables.temperature        → simple variable
variables.sensor.value      → struct field
variables.speed              → enum value
```

---

4. Pages (Screens)

Each page is a full-screen container with widgets:

JSON

```
{
  "userPages": [
    {
      "name": "Main",                      // page name (used by changeScreen action)
      "left": 0,                            // page X offset
      "top": 0,                             // page Y offset
      "width": 800,                         // page width (usually matches display)
      "height": 480,                        // page height (usually matches display)
      "createAtStart": true,                // load this page on boot
      "deleteOnScreenUnload": false,        // keep page in memory after navigating away

      "components": [
        // Page background (always an LVGLScreenWidget)
        {
          "type": "LVGLScreenWidget",
          "left": 0,
          "top": 0,
          "width": 800,
          "height": 480,
          "style": {
            "useStyle": "main_screen_bg"
          },
          "localStyles": {
            "bg_color": "#1A1A2E",
            "bg_opa": "COVER"
          },
          "children": [
            // ... all widgets on this page
          ]
        }
      ],

      "connectionLines": [],
      "localVariables": []
    }
  ]
}
```

Page Background

The first `component` is always an `LVGLScreenWidget`. Its:

- `style.useStyle` — references a named style from `lvglStyles`
- `localStyles` — inline style overrides (background color, opacity, etc.)
- `children` — all visible widgets on the page

To set a page background color:

JSON

```
"localStyles": {
  "bg_color": "#1A1A2E",
  "bg_opa": "COVER"
}
```

To set a background image:

```JSON
"localStyles": {
  "bg_img_src": "my_background_image"
}
```

---

5. All Widget Types

5.1 Widget Common Properties (every widget has these)

Property	Type	Example	Description
`type`	string	`"LVGLLabelWidget"`	Widget type identifier
`left`	number	`100`	X position (pixels)
`top`	number	`50`	Y position (pixels)
`width`	number/string	`200` or `"content"`	Width (pixels or auto)
`height`	number/string	`30` or `"content"`	Height (pixels or auto)
`leftUnit`	string	`"px"`	Unit for left
`topUnit`	string	`"px"`	Unit for top
`widthUnit`	string	`"px"` or `"content"`	Unit for width
`heightUnit`	string	`"px"` or `"content"`	Unit for height
`style`	object	`{ "useStyle": "default" }`	Reference to named style
`localStyles`	object	`{ "bg_color": "#FF0000" }`	Inline style overrides
`eventHandlers`	array	`[{ "trigger": "CLICKED", ... }]`	Event bindings
`children`	array	`[ {Widget}, ... ]`	Child widgets
`hiddenFlagType`	string	`"literal"` or `"expression"`	Visibility control
`hiddenFlag`	string	`""` or `"variables.show"`	Hidden condition
`clickableFlag`	bool	`true`	Whether clickable
`clickableFlagType`	string	`"literal"`	Clickable control
`disabledStateType`	string	`"literal"` or `"expression"`	Disabled control
`widgetFlags`	string	`"CLICKABLE	PRESS_LOCK"`	
`groupIndex`	number	`0`	Input group index
`customInputs`	array	`[]`	Custom flow inputs
`customOutputs`	array	`[]`	Custom flow outputs
`timeline`	array	`[]`	Animation timeline
`states`	string	`""`	Widget state overrides

5.2 Widget-Specific Properties

LVGLLabelWidget (Text Label)

JSON

```
{
  "type": "LVGLLabelWidget",
  "left": 10, "top": 20,
  "width": 200, "height": 30,
  "text": "Temperature: 25.3 °C",
  "textType": "expression",           // "literal" | "expression"
  "longMode": "WRAP",                 // "WRAP" | "DOT" | "SCROLL" | "SCROLL_CIRCULAR" | "CLIP"
  "recolor": false,                   // Enable recolor with #RRGGBB tags
  "style": { "useStyle": "label_style" },
  "eventHandlers": [
    {
      "eventName": "CLICKED",
      "handlerType": "flow",
      "userData": 0
    }
  ]
}
```

textType values:

- `"literal"` → static text, `text` is the actual string
- `"expression"` → dynamic, `text` contains an expression like `"Temperature: " + string(variables.temp) + " °C"`

LVGLButtonWidget (Button)

JSON

```
{
  "type": "LVGLButtonWidget",
  "left": 100, "top": 200,
  "width": 120, "height": 50,
  "eventHandlers": [
    {
      "trigger": "CLICKED",
      "action": "setVariable",
      "variable": "variables.ledOn",
      "value": "true"
    }
  ],
  "children": [
    {
      "type": "LVGLLabelWidget",
      "width": "content", "height": "content",
      "text": "Turn ON",
      "textType": "literal"
    }
  ]
}
```

Button has a child Label — the button's text is a nested `LVGLLabelWidget`.

LVGLSwitchWidget (Toggle Switch)

JSON

```
{
  "type": "LVGLSwitchWidget",
  "left": 50, "top": 100,
  "width": 60, "height": 30,
  "checkedStateType": "expression",     // "literal" | "expression"
  "checkedState": "variables.power",    // boolean or expression
  "eventHandlers": [
    {
      "trigger": "VALUE_CHANGED",
      "action": "setVariable",
      "variable": "variables.power",
      "value": "event.value"
    }
  ]
}
```

LVGLSliderWidget (Slider)

JSON

```
{
  "type": "LVGLSliderWidget",
  "left": 20, "top": 300,
  "width": 300, "height": 20,
  "minType": "literal", "min": "0",
  "maxType": "literal", "max": "100",
  "valueType": "expression", "value": "variables.brightness",
  "eventHandlers": [
    {
      "trigger": "VALUE_CHANGED",
      "action": "setVariable",
      "variable": "variables.brightness",
      "value": "event.value"
    }
  ]
}
```

LVGLArcWidget (Arc Gauge)

JSON

```
{
  "type": "LVGLArcWidget",
  "left": 400, "top": 50,
  "width": 150, "height": 150,
  "minType": "literal", "min": "0",     // "literal" | "expression"
  "maxType": "literal", "max": "100",
  "valueType": "expression", "value": "variables.humidity",
  "rotation": 135,                      // arc rotation in degrees
  "bgStartAngle": 0, "bgEndAngle": 270,  // background arc angles
  "startAngleType": "literal", "startAngle": "135",
  "endAngleType": "literal", "endAngle": "45",
  "mode": "NORMAL",               // "NORMAL" | "REVERSE" | "SYMMETRICAL"
  "localStyles": {
    "arc_color": "#00FF00",
    "arc_width": 5,
    "bg_arc_color": "#333333"
  }
}
```

LVGLBarWidget (Progress Bar)

JSON

```
{
  "type": "LVGLBarWidget",
  "left": 20, "top": 400,
  "width": 300, "height": 20,
  "minType": "literal", "min": "0",
  "maxType": "literal", "max": "100",
  "valueType": "expression", "value": "variables.progress",
  "mode": "NORMAL",               // "NORMAL" | "SYMMETRICAL" | "RANGE"
  "orientation": "HORIZONTAL",    // "HORIZONTAL" | "VERTICAL"
  "localStyles": {
    "bg_color": "#00FF00",
    "bg_opa": "COVER"
  }
}
```

LVGLImageWidget (Image)

JSON

```
{
  "type": "LVGLImageWidget",
  "left": 10, "top": 10,
  "width": 64, "height": 64,
  "src": "logo",                  // bitmap name from "bitmaps" array
  "pivotX": 32, "pivotY": 32,    // rotation pivot point
  "rotationType": "literal", "rotation": "0",   // "literal" | "expression", 0.1 deg units
  "zoomType": "literal", "zoom": "256"          // "literal" | "expression", 256 = 1x
}
```

LVGLMeterWidget (Meter/Gauge)

JSON

```
{
  "type": "LVGLMeterWidget",
  "left": 500, "top": 20,
  "width": 200, "height": 200,
  "scales": [
    {
      "minType": "literal", "min": "0",
      "maxType": "literal", "max": "100",
      "angleRange": 270,
      "rotation": 135,
      "majorTicks": [
        { "valueType": "literal", "value": "0" },
        { "valueType": "literal", "value": "25" },
        { "valueType": "literal", "value": "50" },
        { "valueType": "literal", "value": "75" },
        { "valueType": "literal", "value": "100" }
      ],
      "indicators": [
        {
          "type": "needle_line",
          "valueType": "expression", "value": "variables.speed",
          "localStyles": { "line_color": "#FF0000", "line_width": 2 }
        }
      ]
    }
  ]
}
```

LVGLChartWidget (Chart)

JSON

```
{
  "type": "LVGLChartWidget",
  "left": 10, "top": 50,
  "width": 400, "height": 200,
  "chartType": "LINE",          // "LINE" | "BAR" | "SCATTER"
  "series": [
    {
      "color": "#00FF00",
      "pointCount": 50
    }
  ],
  "xAxis": {
    "minType": "literal", "min": "0",
    "maxType": "literal", "max": "100",
    "majorTicks": [{ "value": "0" }, { "value": "50" }, { "value": "100" }]
  },
  "yAxis": {
    "minType": "literal", "min": "0",
    "maxType": "literal", "max": "100"
  }
}
```

LVGLContainerWidget (Container)

JSON

```
{
  "type": "LVGLContainerWidget",
  "left": 10, "top": 50,
  "width": 300, "height": 200,
  "layoutType": "FLEX",          // "NONE" | "FLEX" | "GRID"
  "flexFlow": "ROW_WRAP",        // "ROW" | "COLUMN" | "ROW_WRAP" | "COLUMN_WRAP" | "ROW_REVERSE" | "COLUMN_REVERSE"
  "children": [
    // Widgets positioned by the layout engine
  ]
}
```

Full Widget Type Reference

Type	LVGL Object	Key Properties
`LVGLScreenWidget`	`lv_obj_create(NULL)`	`bg_color`, `bg_img_src`
`LVGLLabelWidget`	`lv_label_create()`	`text`, `textType`, `longMode`, `recolor`
`LVGLButtonWidget`	`lv_btn_create()`	`children` (contains label)
`LVGLSwitchWidget`	`lv_switch_create()`	`checkedState`, `checkedStateType`
`LVGLSliderWidget`	`lv_slider_create()`	`min`, `max`, `value`
`LVGLArcWidget`	`lv_arc_create()`	`min`, `max`, `value`, `rotation`, `mode`
`LVGLBarWidget`	`lv_bar_create()`	`min`, `max`, `value`, `mode`, `orientation`
`LVGLImageWidget`	`lv_img_create()`	`src`, `rotation`, `zoom`
`LVGLMeterWidget`	`lv_meter_create()`	`scales[].indicators[]`
`LVGLChartWidget`	`lv_chart_create()`	`chartType`, `series[]`, `xAxis`, `yAxis`
`LVGLContainerWidget`	`lv_obj_create()`	`layoutType`, `flexFlow`
`LVGLTextAreaWidget`	`lv_textarea_create()`	`text`, `placeholder`, `maxLength`
`LVGLCheckboxWidget`	`lv_checkbox_create()`	`text`, `checkedState`
`LVGLDropdownWidget`	`lv_dropdown_create()`	`options`, `selectedIndex`
`LVGLRollerWidget`	`lv_roller_create()`	`options`, `selectedIndex`, `visibleRows`
`LVGLLedWidget`	`lv_led_create()`	`color`, `brightness`
`LVGLSpinnerWidget`	`lv_spinner_create()`	—
`LVGLKeyboardWidget`	`lv_keyboard_create()`	`mode`
`LVGLCalendarWidget`	`lv_calendar_create()`	—
`LVGLTabWidget`	`lv_tabview_create()`	`tabs[]`, `tabPosition`
`LVGLCanvasWidget`	`lv_canvas_create()`	`canvasWidth`, `canvasHeight`
`LVGLQRCodeWidget`	`lv_qrcode_create()`	`text`, `size`
`LVGLLineWidget`	`lv_line_create()`	`points`
`LVGLWindowWidget`	`lv_win_create()`	`title`
`LVGLTableWidget`	`lv_table_create()`	`rows`, `columns`, `cellValues`
`LVGLSpinboxWidget`	`lv_spinbox_create()`	`range`, `value`, `step`
`LVGLImgButtonWidget`	`lv_imgbtn_create()`	`srcReleased`, `srcPressed`
`LVGLButtonMatrixWidget`	`lv_btnmatrix_create()`	`map[]`
`LVGLColorwheelWidget`	`lv_colorwheel_create()`	`mode`
`LVGLMenuWidget`	`lv_menu_create()`	`items[]`
`LVGLMsgBoxWidget`	`lv_msgbox_create()`	`title`, `text`, `buttons[]`
`LVGLLottieWidget`	`lv_lottie_create()`	`src`
`LVGLAnimationImageWidget`	`lv_animimg_create()`	`src`, `duration`
`LVGLScaleWidget`	`lv_scale_create()`	`mode`, `majorTicks[]`
`LVGLPanelWidget`	`lv_obj_create()`	(styled container)
`LVGLTileViewWidget`	`lv_tileview_create()`	`tiles[]`
`LVGLListWidget`	`lv_list_create()`	`items[]`

Type	LVGL Object	Key Properties
`LVGLSpanWidget`	`lv_spangroup_create()`	`spans[]`
`LVGLUserWidgetWidget`	References `userWidgets[]`	`userWidgetName`

---

6. Style System

Styles define the visual appearance (colors, fonts, borders, shadows, etc.).

Named Style (global, referenced by `useStyle`)

JSON

```
{
  "lvglStyles": [
    {
      "name": "main_screen_bg",
      "properties": {
        "bg_color": "#1A1A2E",
        "bg_opa": "COVER",
        "border_width": 0,
        "pad_all": 0
      },
      "stateStyles": {
        "PRESSED": {
          "bg_color": "#2A2A3E"
        },
        "FOCUSED": {
          "border_color": "#00FF00",
          "border_width": 2
        }
      }
    },
    {
      "name": "label_style",
      "properties": {
        "text_color": "#FFFFFF",
        "text_font": "roboto_16",
        "text_align": "CENTER"
      }
    }
  ]
}
```

Local Style Inline

Each widget can have inline style overrides:

JSON

```
"localStyles": {
  "bg_color": "#FF0000",
  "bg_opa": "COVER",
  "border_width": 2,
  "border_color": "#00FF00",
  "radius": 8,
  "pad_all": 5,
  "pad_top": 10,
  "shadow_width": 3,
  "shadow_color": "#000000",
  "shadow_ofs_x": 2,
  "shadow_ofs_y": 2,
  "text_color": "#FFFFFF",
  "text_font": "roboto_16",
  "text_align": "CENTER",
  "text_letter_space": 1,
  "text_line_space": 2,
  "arc_color": "#00FF00",
  "arc_width": 5,
  "arc_rounded": true,
  "bg_img_src": "my_image",
  "bg_img_opa": "COVER",
  "bg_img_recolor": "#FFFFFF",
  "bg_img_tiled": true,
  "line_color": "#FF0000",
  "line_width": 2,
  "line_dash_width": 5,
  "line_dash_gap": 3,
  "line_rounded": true,
  "transform_width": 200,
  "transform_height": 200,
  "transform_angle": 450,        // 0.1 degree units
  "transform_zoom": 256,         // 256 = 1x
  "opa": "COVER"
}
```

LVGL Style Property Reference

JSON Property	LVGL Style	Type
`bg_color`	`LV_STYLE_BG_COLOR`	hex color
`bg_opa`	`LV_STYLE_BG_OPA`	0-255 or "COVER"
`bg_img_src`	`LV_STYLE_BG_IMG_SRC`	image name
`bg_img_opa`	`LV_STYLE_BG_IMG_OPA`	0-255
`border_color`	`LV_STYLE_BORDER_COLOR`	hex color
`border_width`	`LV_STYLE_BORDER_WIDTH`	px
`border_opa`	`LV_STYLE_BORDER_OPA`	0-255
`radius`	`LV_STYLE_RADIUS`	px
`pad_top/bottom/left/right`	`LV_STYLE_PAD_*`	px
`pad_all`	all pads	px
`shadow_width`	`LV_STYLE_SHADOW_WIDTH`	px
`shadow_color`	`LV_STYLE_SHADOW_COLOR`	hex
`shadow_ofs_x/y`	`LV_STYLE_SHADOW_OFS_*`	px
`text_color`	`LV_STYLE_TEXT_COLOR`	hex color
`text_font`	`LV_STYLE_TEXT_FONT`	font name
`text_align`	`LV_STYLE_TEXT_ALIGN`	"LEFT"/"CENTER"/"RIGHT"
`text_letter_space`	`LV_STYLE_TEXT_LETTER_SPACE`	px
`text_line_space`	`LV_STYLE_TEXT_LINE_SPACE`	px
`arc_color`	`LV_STYLE_ARC_COLOR`	hex color
`arc_width`	`LV_STYLE_ARC_WIDTH`	px
`arc_rounded`	`LV_STYLE_ARC_ROUNDED`	bool
`line_color`	`LV_STYLE_LINE_COLOR`	hex color
`line_width`	`LV_STYLE_LINE_WIDTH`	px
`line_dash_width`	`LV_STYLE_LINE_DASH_WIDTH`	px
`line_dash_gap`	`LV_STYLE_LINE_DASH_GAP`	px
`line_rounded`	`LV_STYLE_LINE_ROUNDED`	bool
`transform_angle`	`LV_STYLE_TRANSFORM_ANGLE`	0.1 deg
`transform_zoom`	`LV_STYLE_TRANSFORM_ZOOM`	256=1x
`opa`	`LV_STYLE_OPA`	0-255

---

7. Event Handlers & Flow Actions

In `.eez-project` (editor format)

Flow actions are stored as separate `LVGLActionComponent` components, connected to widgets via `connectionLines`:

JSON

```
// Widget with flow event
{
  "type": "LVGLButtonWidget",
  "objID": "f6315498-...",
  "eventHandlers": [{
    "eventName": "CLICKED",
    "handlerType": "flow",
    "userData": 0
  }]
}

// Action component (separate, in page components)
{
  "type": "LVGLActionComponent",
  "objID": "850d8d9f-...",
  "actions": [{
    "action": "changeScreen",
    "screen": "security_screen",
    "fadeMode": "FADE_IN",
    "speed": 200,
    "useStack": true
  }]
}

// Connection line (in page.connectionLines)
{
  "source": "f6315498-...",   // button's objID
  "output": "CLICKED",
  "target": "850d8d9f-...",   // action's objID
  "input": "@seqin"
}
```

In firmware JSON (device format)

During export, `connectionLines` are resolved and actions are attached directly to widget events. Action components are stripped from the output:

JSON

```
{
  "events": {
    "CLICKED": {
      "actions": [
        {
          "action": "changeScreen",
          "screen": "security_screen",
          "fadeMode": "FADE_IN",
          "speed": 200,
          "useStack": true
        }
      ]
    }
  }
}
```

Supported Action Properties

Property	Type	Description
`action`	string	`"changeScreen"`
`screen`	string	Target screen name
`fadeMode`	string	`"FADE_IN"`, `"FADE_OUT"`, `"FADE_ON"`, `"NONE"`
`speed`	number	Animation speed in ms
`useStack`	bool	Push to navigation stack

Trigger Types

Trigger	LVGL Event	When
`CLICKED`	`LV_EVENT_CLICKED`	Press+release on widget
`PRESSED`	`LV_EVENT_PRESSED`	Widget pressed
`RELEASED`	`LV_EVENT_RELEASED`	Widget released
`VALUE_CHANGED`	`LV_EVENT_VALUE_CHANGED`	Slider/arc/bar value changed
`LONG_PRESSED`	`LV_EVENT_LONG_PRESSED`	Long press
`SCROLL`	`LV_EVENT_SCROLL`	Widget scrolled
`FOCUSED`	`LV_EVENT_FOCUSED`	Widget focused
`DEFOCUSED`	`LV_EVENT_DEFOCUSED`	Widget lost focus
`CHECKED`	`LV_EVENT_VALUE_CHANGED`	Switch/checkbox checked
`UNCHECKED`	`LV_EVENT_VALUE_CHANGED`	Switch/checkbox unchecked
`KEY_DOWN`	`LV_EVENT_KEY`	Key pressed
`SCREEN_LOAD_START`	custom	When page loads
`SCREEN_UNLOAD_START`	custom	When page unloads

Action Types (Flow Actions)

Action	Description	Parameters
`changeScreen`	Navigate to another page	`screen: "PageName"`, `fadeMode`, `speed`, `useStack`
`setVariable`	Set a variable value	`variable, value`
`toggleVariable`	Toggle a boolean variable	`variable`
`incrementVariable`	Increment a numeric variable	`variable, value`
`executeActions`	Run a list of sub-actions	`actions: [...]`
`ifCondition`	Conditional execution	`condition, thenActions, elseActions`
`delay`	Wait before next action	`milliseconds`
`loop`	Repeat actions	`count, actions`
`callAction`	Call a named action from `actions[]`	`actionName`
`showWidget`	Show a widget	`widget`
`hideWidget`	Hide a widget	`widget`
`startAnimation`	Start a timeline animation	`timeline`
`setPageVariable`	Set a page-local variable	`variable, value`

Expression Syntax for Dynamic Values

When `*Type` is `"expression"`, the value is evaluated dynamically. Expressions use JavaScript-like syntax:

```text
// Simple variable reference
variables.temperature

// Math
(variables.temp * 9/5) + 32

// String formatting
"Value: " + string(variables.temp) + " °C"

// Conditional (ternary)
variables.temp > 30 ? "#FF0000" : "#00FF00"

// Function calls
min(variables.a, variables.b)
max(variables.a, variables.b)
round(variables.value, 2)
```

---

8. Colors

References used in styles:

```json
{
  "colors": [
    {
      "name": "primary",
      "color": "#FF6B35"
    },
    {
      "name": "background_dark",
      "color": "#1A1A2E"
    },
    {
      "name": "accent_green",
      "color": "#00FF00"
    }
  ]
}
```

---

9. Fonts & Bitmaps

Fonts

Font files are embedded as raw base64 strings (no data URI prefix). Each font entry specifies:

Field	Type	Description
`name`	string	Font name referenced by `text_font` in styles
`renderingEngine`	string	`"LVGL"` (LVGL rasterizer)
`source`	object	`{ objID, filePath, size }` — original source font file metadata
`embeddedFontFile`	string	**Raw base64** of the OTF/TTF font file (no `data:` prefix)
`lvglBinFile`	string	**Raw base64** of the LVGL pre-rasterized binary font blob
`bpp`	number	Bits per pixel: `1`, `2`, `4`, or `8`
`threshold`	number	Binarization threshold (typically `128`)
`height`	number	Font height in px
`ascent`	number	Ascent in px
`descent`	number	Descent in px
`glyphs`	array	Custom glyph overrides (usually empty)
`lvglRanges`	string	Unicode ranges, e.g. `"32 - 127,176"`
`lvglSymbols`	string	Extra symbol characters

JSON


```
{
  "fonts": [
    {
      "name": "regular_16",
      "renderingEngine": "LVGL",
      "source": {
        "objID": "7132b131-8d39-...",
        "filePath": "MyriadPro-Regular.otf",
        "size": 16
      },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXGYAAAA...",
      "lvglBinFile": "MAAAAGhlYWQBAAAABAAQAAwA/P8MAPz/AwD8/wwA...",
      "bpp": 4,
      "threshold": 128,
      "height": 16,
      "ascent": 12,
      "descent": 4,
      "glyphs": [],
      "lvglRanges": "32 - 127,176",
      "lvglSymbols": ""
    }
  ]
}
```


Bitmaps (Images)

Images are stored as **data URIs** with PNG base64 encoding:

Field	Type	Description
`name`	string	Image name referenced by `src` in widgets or `bg_img_src` in styles
`image`	string	**Data URI**: `"<base64_string>"`
`bpp`	number	Bits per pixel of the source image (usually `32` for RGBA PNG)
`alwaysBuild`	boolean	Force rebuild on every export
`lvglBinaryOutputFormat`	number	Target LVGL color format enum (see table below)
`lvglDither`	boolean	Apply dithering when converting to lower BPP

`lvglBinaryOutputFormat` values:

Value	LVGL Constant	Description
`1`	`LVGL_COLOR_FORMAT_I1`	1-bit indexed
`2`	`LVGL_COLOR_FORMAT_I2`	2-bit indexed
`3`	`LVGL_COLOR_FORMAT_ARGB8888`	32-bit ARGB
`4`	`LVGL_COLOR_FORMAT_I4`	4-bit indexed
`5`	`LVGL_COLOR_FORMAT_I8`	8-bit indexed
`6`	`LVGL_COLOR_FORMAT_RGB565`	16-bit RGB565
`7`	`LVGL_COLOR_FORMAT_RGB888`	24-bit RGB
`8`	`LVGL_COLOR_FORMAT_XRGB8888`	32-bit XRGB (no alpha)
`9`	`LVGL_COLOR_FORMAT_A8`	8-bit alpha only

JSON

```
{
  "bitmaps": [
    {
      "name": "background_1",
      "image": "...",
      "bpp": 32,
      "alwaysBuild": false,
      "lvglBinaryOutputFormat": 3,
      "lvglDither": false
    }
  ]
}
```

10. Themes

JSON

```
{
  "themes": [
    {
      "name": "Dark",
      "colors": {
        "bg": "#1A1A2E",
        "text": "#FFFFFF",
        "primary": "#FF6B35",
        "secondary": "#16213E"
      }
    },
    {
      "name": "Light",
      "colors": {
        "bg": "#FFFFFF",
        "text": "#000000",
        "primary": "#FF6B35",
        "secondary": "#F0F0F0"
      }
    }
  ]
}
```

---

11. Embedded Runtime Algorithm (Pseudocode)

```c
// ====== INITIALIZATION ======
void eez_runtime_init(const char *json_string) {
    project = cJSON_Parse(json_string);

    // Set LVGL display size from settings
    int w = cJSON_GetObjectItem(project, "settings")...["displayWidth"];
    int h = cJSON_GetObjectItem(project, "settings")...["displayHeight"];
    lv_disp_drv_init_with_size(w, h);

    // Load fonts
    cJSON *fonts = cJSON_GetObjectItem(project, "fonts");
    for (font in fonts) {
        lv_font_t *f = load_lvgl_font(font->name, font->size, font->bpp, font->data);
        register_font(font->name, f);
    }

    // Load bitmaps
    cJSON *bitmaps = cJSON_GetObjectItem(project, "bitmaps");
    for (bmp in bitmaps) {
        lv_img_dsc_t *img = load_lvgl_image(bmp->name, bmp->data, bmp->format);
        register_image(bmp->name, img);
    }

    // Build style cache
    cJSON *styles = cJSON_GetObjectItem(project, "lvglStyles");
    for (style in styles) {
        lv_style_t *s = build_lvgl_style(style->properties);
        register_style(style->name, s);
    }
}

// ====== PAGE NAVIGATION ======
void eez_load_page(const char *page_name) {
    cJSON *page = find_page(project, page_name);
    lv_obj_t *screen = lv_obj_create(NULL);
    render_widget(screen, page->components[0]);
    lv_scr_load(screen);
    current_page = page;
}

// ====== WIDGET RENDERING ======
void render_widget(lv_obj_t *parent, cJSON *w) {
    const char *type = cJSON_GetObjectItem(w, "type")->valuestring;
    lv_obj_t *obj = NULL;

    // === Widget Factory ===
    if (strcmp(type, "LVGLScreenWidget") == 0)
        obj = lv_obj_create(parent);
    else if (strcmp(type, "LVGLLabelWidget") == 0) {
        obj = lv_label_create(parent);
        const char *text = cJSON_GetObjectItem(w, "text")->valuestring;
        lv_label_set_text(obj, text);
    }
    else if (strcmp(type, "LVGLButtonWidget") == 0)
        obj = lv_btn_create(parent);
    else if (strcmp(type, "LVGLSwitchWidget") == 0)
        obj = lv_switch_create(parent);
```

```
    else if (strcmp(type, "LVGLSliderWidget") == 0) {
        obj = lv_slider_create(parent);
        lv_slider_set_range(obj, get_number(w, "min"), get_number(w, "max"));
    }
    // ... etc for all 43 widget types

    // === Position & Size ===
    lv_obj_set_pos(obj, get_number(w, "left"), get_number(w, "top"));
    if (isContentSize(w, "width"))
        lv_obj_set_width(obj, LV_SIZE_CONTENT);
    else
        lv_obj_set_width(obj, get_number(w, "width"));
    // Same for height

    // === Apply Styles ===
    apply_style(obj, w);  // Combine useStyle + localStyles

    // === Event Handlers ===
    cJSON *handlers = cJSON_GetObjectItem(w, "eventHandlers");
    for (int i = 0; i < cJSON_GetArraySize(handlers); i++) {
        cJSON *h = cJSON_GetArrayItem(handlers, i);
        register_event_handler(obj, h, w);
    }

    // === Render Children ===
    cJSON *children = cJSON_GetObjectItem(w, "children");
    for (int i = 0; i < cJSON_GetArraySize(children); i++) {
        render_widget(obj, cJSON_GetArrayItem(children, i));
    }
}

// ====== DATA BINDING LOOP ======
void eez_update_bindings() {
    cJSON *page = current_page;
    update_widget_recursive(page->components[0]);
}

void update_widget_recursive(cJSON *w) {
    // Check if any property has expression binding
    if (isExpression(w, "text"))      update_label_text(w);
    if (isExpression(w, "value"))     update_slider_value(w);
    if (isExpression(w, "checkedState")) update_switch_state(w);
    if (isExpression(w, "hiddenFlag"))   update_visibility(w);
    // ... etc

    // Recurse
    for (child in w->children)
        update_widget_recursive(child);
}
```

---

12. Real-World Example: Smart Home (LVGL 9.0)

File: `project/examples/Smart Home (LVGL 9.x)/Smart Home (LVGL 9.x).eez-project` **Size:** 1.8 MB JSON **Target:** LVGL 9.0, 800×480 display **Contents:** 3 pages, 6 fonts (2 typefaces × multiple sizes), 36 bitmaps (all ARGB8888), 2 named styles, 2 widget groups, 1 theme, 4 global variables, 0 actions

12.1 Project Settings (Actual)

JSON


```
{
  "settings": {
    "general": {
      "projectVersion": "v3",
      "projectType": "lvgl",
      "lvglVersion": "9.0",
      "flowSupport": true,
      "displayWidth": 800,
      "displayHeight": 480,
      "colorFormat": "BGR",
      "description": "Smart home example for the LVGL version 9.x",
      "keywords": "smart-home lvgl9"
    }
  }
}
```


12.2 Page Structure

The project has 3 pages defined in `userPages`:

- `heating_screen` — main thermostat panel (createAtStart: `true`)
- `security_screen` — security/lock control (createAtStart: `true`)
- `lighting_screen` — lighting control panel (createAtStart: `true`)

Widget tree for `heating_screen` (simplified):

TEXT

```
heating_screen (Page)
└── LVGLScreenWidget (800×480, dark background)
    ├── LVGLContainerWidget (top bar)
    │   ├── LVGLLabelWidget "Smart Home Control"
    │   └── LVGLLabelWidget (clock display, expression-bound)
    ├── LVGLContainerWidget (main content area)
    │   ├── LVGLArcWidget (temperature gauge, 0-40°C)
    │   ├── LVGLLabelWidget "25.3°C" (expression: variables.currentTemp)
    │   ├── LVGLButtonWidget (mode: HEAT)
    │   │   └── LVGLLabelWidget "HEAT"
    │   ├── LVGLButtonWidget (mode: COOL)
    │   │   └── LVGLLabelWidget "COOL"
    │   └── LVGLSliderWidget (target temp 16-30°C)
    └── LVGLContainerWidget (bottom status bar)
        ├── LVGLImageWidget (WiFi icon)
        ├── LVGLLabelWidget "Connected"
        └── LVGLImageWidget (battery icon)
```

12.3 Fonts

Fonts are embedded as raw base64 OTF files in `embeddedFontFile` and pre-rasterized LVGL binary fonts in `lvglBinFile`. Each font entry specifies its source metadata in a nested `source` object.

Actual font entries from the project:

JSON


```
{
  "fonts": [
    { "name": "regular_16", "bpp": 4, "height": 16, "ascent": 12, "descent": 4, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Regular.otf", "size": 16 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXGYAAAA..." },
    { "name": "regular_21", "bpp": 4, "height": 22, "ascent": 16, "descent": 6, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Regular.otf", "size": 21 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXGYAAAA..." },
    { "name": "regular_36", "bpp": 4, "height": 37, "ascent": 28, "descent": 9, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Regular.otf", "size": 36 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXGYAAAA..." },
    { "name": "bold_17",    "bpp": 4, "height": 18, "ascent": 14, "descent": 4, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Bold.otf", "size": 17 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXhkAAAA..." },
    { "name": "bold_21",    "bpp": 4, "height": 22, "ascent": 17, "descent": 5, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Bold.otf", "size": 21 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXhkAAAA..." },
    { "name": "bold_23",    "bpp": 4, "height": 24, "ascent": 18, "descent": 6, "renderingEngine": "LVGL",
      "source": { "filePath": "MyriadPro-Bold.otf", "size": 23 },
      "embeddedFontFile": "T1RUTwANAIAAAwBQQkFTRWUlXb0AAXhkAAAA..." }
  ]
}
```


> **Note:** The same OTF file (e.g., `MyriadPro-Regular.otf`) is embedded in multiple font entries at different sizes. EEZ Studio rasterizes each size independently, producing unique `lvglBinFile` blobs.

12.4 Bitmaps / Images

36 images are embedded in the project. Each bitmap uses a **data URI** (`...`) with an `lvglBinaryOutputFormat` of `3` (ARGB8888). All source images are 32 BPP RGBA PNGs.

JSON


```
{
  "bitmaps": [
    { "name": "background_1",   "bpp": 32, "lvglBinaryOutputFormat": 3, "lvglDither": false,
      "image": "..." },
    { "name": "background_2",   "bpp": 32, "lvglBinaryOutputFormat": 3, "lvglDither": false,
      "image": "..." },
    { "name": "heating_button", "bpp": 32, "lvglBinaryOutputFormat": 3, "lvglDither": false,
      "image": "..." },
    { "name": "switch_on",      "bpp": 32, "lvglBinaryOutputFormat": 3, "lvglDither": false,
      "image": "..." }
  ]
}
```


12.5 Variables

The project defines 4 global variables with struct-backed array types:

JSON

```
{
  "variables": {
    "globalVariables": [
      { "name": "users",         "type": "array:struct:User" },
      { "name": "zones",         "type": "array:struct:Zone" },
      { "name": "selected_user", "type": "integer", "defaultValue": "0" },
      { "name": "selected_zone", "type": "integer", "defaultValue": "0" }
    ],
    "structures": [
      {
        "name": "User",
        "fields": [{ "name": "name", "type": "string" }]
      },
      {
        "name": "Zone",
        "fields": [
          { "name": "name", "type": "string" },
          { "name": "temperature", "type": "float" },
          { "name": "power", "type": "float" },
          { "name": "locked", "type": "boolean" },
          { "name": "lighting_percent", "type": "float" },
          { "name": "heating_saved", "type": "boolean" },
          { "name": "lighting_saved", "type": "boolean" }
        ]
      }
    ]
  }
}
```

12.6 Navigation

Page switching uses `LVGLActionComponent` + `connectionLines`. Example from a button on `heating_screen`:

JSON

```
// Button on heating screen
{
  "type": "LVGLButtonWidget",
  "objID": "btn_heat_1",
  "eventHandlers": [
    {
      "eventName": "CLICKED",
      "handlerType": "flow",
      "userData": 0
    }
  ]
}

// Action component (separate, linked by connectionLine)
{
  "type": "LVGLActionComponent",
  "objID": "act_nav_sec",
  "actions": [
    {
      "action": "changeScreen",
      "screen": "security_screen",
      "fadeMode": "FADE_IN",
      "speed": 200,
      "useStack": true
    }
  ]
}

// Connection line (in page.connectionLines)
{
  "source": "btn_heat_1",
  "output": "CLICKED",
  "target": "act_nav_sec",
  "input": "@seqin"
}
```

Firmware JSON (exported): Actions are resolved and attached directly to widget events:

JSON

```
{
  "events": {
    "CLICKED": {
      "actions": [
        { "action": "changeScreen", "screen": "security_screen" }
      ]
    }
  }
}
```

Appendix A: Base64-Encoded Data Types

This appendix documents every base64-encoded field found in real `.eez-project` files, based on analysis of the **Smart Home (LVGL 9.x)** project (1.8 MB, 36 bitmaps, 6 fonts, 3 pages).

A.1 Summary of Base64 Types

There are **three distinct base64 encoding patterns** used in the JSON:

#	Field	Location	Encoding	What It Contains
1	`image`	`bitmaps[].image`	Data URI	PNG image → `...`
2	`image`	`settings.general.image`	Data URI	PNG preview icon → `...`
3	`embeddedFontFile`	`fonts[].embeddedFontFile`	Raw base64	OTF/TTF font binary (no `data:` prefix)
4	`lvglBinFile`	`fonts[].lvglBinFile`	Raw base64	LVGL pre-rasterized binary font blob

A.2 Type 1 & 2: PNG Data URIs (`image`)

Used in: `bitmaps[].image` and `settings.general.image`

Format: `"<base64_encoded_png>"`

This is a standard RFC 2397 Data URI. The base64 payload decodes to a valid PNG file. The MIME type is always `image/png` (even if the source was JPEG or BMP — EEZ Studio converts all images to PNG before embedding).

Decoding: Strip the `` prefix, base64-decode the remainder to get the raw PNG bytes.

Example (truncated):

```
TEXT
...
```

Real project statistics (36 bitmaps):

Name	Image Length	BPP	LVGL Output Format
background_1	102,598	32	ARGB8888 (3)
background_2	138,406	32	ARGB8888 (3)
background_3	109,454	32	ARGB8888 (3)
heating_button	7,390	32	ARGB8888 (3)
heating_button_hoover	12,122	32	ARGB8888 (3)
security_button	6,718	32	ARGB8888 (3)
security_button_hoover	11,290	32	ARGB8888 (3)
lighting_button	7,246	32	ARGB8888 (3)
lighting_button_hoover	11,774	32	ARGB8888 (3)
face_0	11,490	32	ARGB8888 (3)
face_1	11,182	32	ARGB8888 (3)
face_2	10,446	32	ARGB8888 (3)
header_menu	7,498	32	ARGB8888 (3)
button_main	7,658	32	ARGB8888 (3)
save	918	32	ARGB8888 (3)
saved	1,398	32	ARGB8888 (3)
temperature_background	15,786	32	ARGB8888 (3)
power_background	16,090	32	ARGB8888 (3)
watch	4,374	32	ARGB8888 (3)
slider_indicator	8,638	32	ARGB8888 (3)
slider_knob	598	32	ARGB8888 (3)
garage_arrows	730	32	ARGB8888 (3)
garage_arrows_hoover	770	32	ARGB8888 (3)
account_box	5,734	32	ARGB8888 (3)
arrow_account_hoover	538	32	ARGB8888 (3)
arrow_account	462	32	ARGB8888 (3)
checkmark	4,438	32	ARGB8888 (3)
big_checkmark	7,422	32	ARGB8888 (3)
switch_off	1,582	32	ARGB8888 (3)
switch_on	1,978	32	ARGB8888 (3)
light_bulb	12,030	32	ARGB8888 (3)
slider_lighting	1,234	32	ARGB8888 (3)
arrow_next_hover	594	32	ARGB8888 (3)
arrow_prev_hoover	618	32	ARGB8888 (3)
arrow_next	578	32	ARGB8888 (3)
arrow_prev	602	32	ARGB8888 (3)

A.3 Type 3: Raw Base64 Font File (`embeddedFontFile`)

Used in: `fonts[].embeddedFontFile`

Format: Raw base64 string (no `data:` prefix, no MIME type). Decodes to a complete OTF or TTF binary font file.

Magic bytes: The first 5 bytes of the base64 string `"T1RUTwA..."` decode to `OTTO\x00` (0x4F54544F00), the standard OpenType magic number.

Decoding: Base64-decode the entire string to get the raw OTF/TTF bytes.

Real project statistics (6 fonts):

Name	Source File	Source Size	Font File Length	LVGL Bin Length	BPP
`regular_16`	MyriadPro-Regular.otf	16 px	135,644	7,584	4
`regular_21`	MyriadPro-Regular.otf	21 px	135,644	11,080	4
`regular_36`	MyriadPro-Regular.otf	36 px	135,644	24,800	4
`bold_17`	MyriadPro-Bold.otf	17 px	137,964	8,928	4
`bold_21`	MyriadPro-Bold.otf	21 px	137,964	11,816	4
`bold_23`	MyriadPro-Bold.otf	23 px	137,964	13,436	4

> **Note:** The same OTF file is reused across multiple font sizes (e.g., `regular_16`, `regular_21`, and `regular_36` all embed the same 135,644-byte `MyriadPro-Regular.otf`). EEZ Studio rasterizes at the target `bpp` and generates the `lvglBinFile`.

A.4 Type 4: Raw Base64 LVGL Binary Font (`lvglBinFile`)

Used in: `fonts[].lvglBinFile`

Format: Raw base64 string (no `data:` prefix). Decodes to the LVGL internal binary font format — a pre-rasterized font blob that LVGL can load directly via `lv_binfont_create()`.

Magic bytes: The first 4 bytes (`"MAAA..."` → `0x30000000`) identify it as an LVGL binary font with glyph bitmaps and kerning data already baked in.

Decoding: Base64-decode the entire string → pass the resulting byte array to `lv_binfont_create()`.

A.5 How the Embedded Runtime Handles Base64

```c
// Image loading (data URI → LVGL image descriptor)
const char *img_field = cJSON_GetObjectItem(bmp, "image")->valuestring;

// 1. Strip the "" prefix
const char *prefix = "";
const char *b64 = strstr(img_field, prefix);
if (b64) b64 += strlen(prefix);
else b64 = img_field;  // fallback: raw base64

// 2. Base64 decode → PNG bytes
size_t png_len;
uint8_t *png_data = base64_decode(b64, &png_len);

// 3. PNG decode → raw pixels (RGBA8888)
lv_image_dsc_t *img = png_to_lvgl_image(png_data, png_len);

// 4. Convert to target format if needed (ARGB8888, RGB565, etc.)
if (bmp->lvglBinaryOutputFormat == 6)  // RGB565
    img = convert_to_rgb565(img);

// 5. Register image for use in styles/widgets
register_image(bmp->name, img);


// Font loading (raw base64 → LVGL binary font)
const char *font_b64 = cJSON_GetObjectItem(font, "lvglBinFile")->valuestring;
size_t bin_len;
uint8_t *bin_data = base64_decode(font_b64, &bin_len);
lv_font_t *lvgl_font = lv_binfont_create_from_buffer(bin_data, bin_len);
register_font(font->name, lvgl_font);
```

A.6 Key Differences from Earlier EEZ Versions

Aspect	EEZ Studio v2 / LVGL 8.x	EEZ Studio v3 / LVGL 9.x
Font storage	`data` field containing raw base64 TTF	`embeddedFontFile` (OTF) + `lvglBinFile` (rasterized)
Font source metadata	`size`, `bpp` at top level	`source: { filePath, size }` nested object
Image fields	`width`, `height`, `format`, `data`	`image` (data URI), `bpp`, `lvglBinaryOutputFormat`, `lvglDither`
Image encoding	Raw base64, format specified via `format` string	`...` data URI, always PNG
LVGL version	`"8.4.0"`	`"9.0"`, `"9.2.2"`, `"9.3.0"`, `"9.4.0"`, `"9.5.0"`
Rendering engine field	Not present	`renderingEngine: "LVGL"`

Appendix B: Firmware JSON Format

B.1 Why

The `.eez-project` JSON is the **editor format** — it contains everything EEZ Studio needs to reopen and edit a project: embedded font binaries (base64 TTF), embedded image data (base64 PNG), editor-internal identifiers, build settings, and animation timelines.

The hardware controller does not need any of this. It has fonts and images pre-loaded in flash, already knows its display resolution and LVGL version, and renders widgets from properties — not from editor metadata.

The **Firmware JSON** strips all editor-only data, keeping only what the controller needs: widget positions, style overrides, event bindings, and font/image **names** (not binaries). Each screen is a separate JSON file — transferred individually so memory usage stays low and screens can be

updated independently.

B.2 What Changed

.eez-project field	Firmware JSON	Action
`type: "LVGLLabelWidget"`	`type: "Widget"` + `sub_type: "label"`	Fixed `type` + type in `sub_type`
`type: "LVGLButtonWidget"`	`type: "Widget"` + `sub_type: "button"`	Same rule for all widget types
`type: "LVGLArcWidget"`	`type: "Widget"` + `sub_type: "arc"`	
`type: "LVGLBarWidget"`	`type: "Widget"` + `sub_type: "bar"`	
`type: "LVGLImageWidget"`	`type: "Widget"` + `sub_type: "image"`	
`type: "LVGLSwitchWidget"`	`type: "Widget"` + `sub_type: "switch"`	
`type: "LVGLPanelWidget"`	`type: "Widget"` + `sub_type: "panel"`	
`type: "LVGLDropdownWidget"`	`type: "Widget"` + `sub_type: "dropdown"`	
`type: "LVGLScreenWidget"`	*(unwrapped)*	Children promoted to `widgets{}`
`type: "LVGLActionComponent"`	*(removed)*	Flow logic, not UI
`type: "SetVariableActionComponent"`	*(removed)*	Flow logic, not UI
`left`, `top`	`x_pos`, `y_pos`	Renamed (present on every widget)
`width`, `height`	`width`, `height`	Unchanged (present on every widget)
`identifier`	*(becomes key name)*	`"heating_button_1": {...}`
`text`, `textType`	`obj_text`, `text_type`	Renamed (present on every widget)
`eventHandlers[]`	`events: { EVENT: {...} }`	Array → flat object
`localStyles.definition`	`style`	Flattened one level
`children[]`	`children: {...}`	Named keys, same structure recursively
`embeddedFontFile` (base64 TTF)	`{name, size}` in `fonts[]`	Stripped, names only
`image` (base64 PNG)	`"name"` in `bitmaps[]`	Stripped, names only
`settings`, `themes`, `lvglStyles`, `lvglGroups`	*(removed)*	Hardware-known or editor-only
`objID`, `*Unit`, `*FlagType`, `states`, `timeline`, `groupIndex`	*(removed)*	Editor-only
`customInputs`, `customOutputs`, `connectionLines`	*(removed)*	Flow wiring, not UI
(project root)	`{ "screen_name": { ... } }`	Screen name becomes root key, contains `fonts` + `bitmaps` + `widgets`

B.3 Consistent Widget Structure

Every widget has these 8 fields always present:

Field	Type	Always	Arc example	Label example
`type`	string	`"Widget"`	`"Widget"`	`"Widget"`
`sub_type`	string	✅	`"arc"`	`"label"`
`x_pos`	number	✅	`480`	`208`
`y_pos`	number	✅	`120`	`160`
`width`	number	✅	`80`	`200`
`height`	number	✅	`80`	`23`
`obj_text`	string	✅	`""`	`"zones[...].temperature"`
`text_type`	string	✅	`"literal"`	`"expression"`

Optional on any widget:

- `style` — per-state overrides (`DEFAULT` , `PRESSED` , `CHECKED` , `DISABLED`)
- `events` — handlers (`CLICKED` , `VALUE_CHANGED` , `PRESSED` , etc.)
- `children` — nested widgets (same structure recursively)

Type-specific — appended only when needed:

`sub_type`	Extra fields
`label`	`long_mode` , `recolor`
`button`	*(uses `style` + `events`)*
`arc` , `bar` , `slider`	`min` , `max` , `value` , `value_type` , `value_left` , `value_left_type` , `mode`
`image`	`src` , `rotation` , `pivot_x` , `pivot_y`
`switch`	`checked` , `checked_type`
`panel`	*(uses `children`)*
`dropdown`	`options` , `selected`
`user_widget`	`widget`

B.4 Final Format

Based on `Smart Home (LVGL 9.x).eez-project` — `heating_screen` transformed:

JSON

```
{
  "heating_screen": {
    "fonts": [
      { "name": "regular_16", "size": 16 },
      { "name": "regular_21", "size": 21 },
      { "name": "regular_36", "size": 36 },
      { "name": "bold_17",    "size": 17 },
      { "name": "bold_21",    "size": 21 },
      { "name": "bold_23",    "size": 23 }
    ],
    "bitmaps": [
      "background_1", "background_2", "background_3",
      "heating_button", "heating_button_hoover",
      "security_button", "security_button_hoover",
      "lighting_button", "lighting_button_hoover",
      "face_0", "face_1", "face_2",
      "arrow_next", "arrow_prev", "arrow_next_hover", "arrow_prev_hoover",
      "switch_on", "switch_off", "light_bulb", "slider_lighting",
      "save", "saved", "checkmark", "big_checkmark",
      "header_menu", "button_main", "account_box",
      "temperature_background", "power_background",
      "watch", "slider_indicator", "slider_knob",
      "garage_arrows", "garage_arrows_hoover",
      "arrow_account", "arrow_account_hoover"
    ],
    "widgets": {
      "background": {
        "type": "Widget",
        "sub_type": "image",
        "x_pos": 0, "y_pos": 0, "width": 800, "height": 480,
        "obj_text": "",
        "text_type": "literal",
        "src": "background_1"
      },
      "heating_button_1": {
        "type": "Widget",
        "sub_type": "button",
        "x_pos": 31, "y_pos": 90, "width": 111, "height": 114,
        "obj_text": "",
        "text_type": "literal",
        "style": {
          "DEFAULT":  { "bg_img_src": "heating_button", "bg_opa": 0, "border_opa": 0, "shadow_opa": 0 },
          "PRESSED":  { "bg_img_src": "heating_button_hoover", "bg_opa": 255 },
          "CHECKED":  { "bg_img_src": "heating_button_hoover" }
        }
      },
      "security_button_1": {
        "type": "Widget",
        "sub_type": "button",
        "x_pos": 31, "y_pos": 217, "width": 111, "height": 114,
        "obj_text": "",
        "text_type": "literal",
        "style": {
          "DEFAULT":  { "bg_img_src": "security_button", "bg_opa": 0, "border_opa": 0, "shadow_opa": 0 },
          "PRESSED":  { "bg_img_src": "security_button_hoover", "bg_opa": 0 },
          "CHECKED":  { "bg_img_src": "security_button_hoover" }
        },
        "events": {
```

```
      "CLICKED": { "action": "flow", "user_data": 0 }
    }
  },
  "temperature_label": {
    "type": "Widget",
    "sub_type": "label",
    "x_pos": 208, "y_pos": 160, "width": 200, "height": 23,
    "obj_text": "zones[selected_zone].temperature",
    "text_type": "expression",
    "style": {
      "DEFAULT": { "text_font": "regular_36", "text_color": "#FF6B35", "text_align": "CENTER" }
    }
  },
  "power_arc": {
    "type": "Widget",
    "sub_type": "arc",
    "x_pos": 480, "y_pos": 120, "width": 80, "height": 80,
    "obj_text": "",
    "text_type": "literal",
    "min": 0, "max": 100,
    "value": "zones[selected_zone].power",
    "value_type": "expression",
    "style": {
      "DEFAULT": { "arc_color": "#00AAFF", "arc_width": 4, "bg_opa": 30 }
    }
  },
  "lock_switch": {
    "type": "Widget",
    "sub_type": "switch",
    "x_pos": 520, "y_pos": 300, "width": 60, "height": 30,
    "obj_text": "",
    "text_type": "literal",
    "checked": "zones[selected_zone].locked",
    "checked_type": "expression",
    "style": {
      "DEFAULT": { "bg_img_src": "switch_off" },
      "CHECKED": { "bg_img_src": "switch_on" }
    }
  },
  "header_1": {
    "type": "Widget",
    "sub_type": "user_widget",
    "x_pos": 0, "y_pos": 0, "width": 800, "height": 75,
    "obj_text": "",
    "text_type": "literal",
    "widget": "header"
  },
  "lighting_slider": {
    "type": "Widget",
    "sub_type": "slider",
    "x_pos": 176, "y_pos": 392, "width": 192, "height": 38,
    "obj_text": "",
    "text_type": "literal",
    "min": 0, "max": 100,
    "value": "zones[selected_zone].lighting_percent",
    "value_type": "expression",
    "style": {
      "DEFAULT": { "bg_opa": 0, "border_opa": 0 }
    }
  },
```

```
      "heating_temperature_panel": {
        "type": "Widget",
        "sub_type": "panel",
        "x_pos": 164, "y_pos": 90, "width": 303, "height": 298,
        "obj_text": "",
        "text_type": "literal",
        "children": {
          "temp_background": {
            "type": "Widget",
            "sub_type": "image",
            "x_pos": 2, "y_pos": 0, "width": 303, "height": 158,
            "obj_text": "",
            "text_type": "literal",
            "src": "temperature_background"
          },
          "temperature_value": {
            "type": "Widget",
            "sub_type": "label",
            "x_pos": 10, "y_pos": 30, "width": 100, "height": 30,
            "obj_text": "zones[selected_zone].temperature",
            "text_type": "expression",
            "style": {
              "DEFAULT": { "text_font": "regular_36", "text_color": "#FF6B35", "text_align": "CENTER" }
            }
          }
        }
      }
    }
  }
}
```

B.5 Widget Type Mapping

.eez-project type	Firmware `sub_type`
`LVGLLabelWidget`	`"label"`
`LVGLButtonWidget`	`"button"`
`LVGLArcWidget`	`"arc"`
`LVGLBarWidget`	`"bar"`
`LVGLImageWidget`	`"image"`
`LVGLSwitchWidget`	`"switch"`
`LVGLPanelWidget`	`"panel"`
`LVGLDropdownWidget`	`"dropdown"`
`LVGLSliderWidget`	`"slider"`
`LVGLUserWidgetWidget`	`"user_widget"` + `"widget": "<name>"`
`LVGLScreenWidget`	*(unwrapped — children promoted to `widgets{}`)*
`LVGLActionComponent`	*(removed)*
`SetVariableActionComponent`	*(removed)*
`WatchVariableActionComponent`	*(removed)*

B.6 Firmware Parser Pseudocode

```c
// ── Entry point: parse one screen JSON ──
void parse_screen(cJSON *root) {
    cJSON *screen = root->child;  // first key = screen name
    if (!screen) return;

    // Load fonts (register name → lv_font_t lookup)
    cJSON *fonts = cJSON_GetObjectItem(screen, "fonts");
    if (fonts) register_fonts(fonts);

    // Load bitmaps (register name → lv_img_dsc_t lookup)
    cJSON *bitmaps = cJSON_GetObjectItem(screen, "bitmaps");
    if (bitmaps) register_bitmaps(bitmaps);

    // Create all widgets
    cJSON *widgets = cJSON_GetObjectItem(screen, "widgets");
    if (widgets) parse_widgets(widgets, lv_scr_act());
}

// ── Recursively parse widgets (named keys) ──
void parse_widgets(cJSON *widgets, lv_obj_t *parent) {
    cJSON *w = widgets->child;  // iterate named children
    for (; w; w = w->next) {
        // ═══ Base fields — EVERY widget has these 8 ═══
        char *sub_type = cJSON_GetObjectItem(w, "sub_type")->valuestring;
        int x  = cJSON_GetObjectItem(w, "x_pos")->valueint;
        int y  = cJSON_GetObjectItem(w, "y_pos")->valueint;
        int wd = cJSON_GetObjectItem(w, "width")->valueint;
        int ht = cJSON_GetObjectItem(w, "height")->valueint;
        char *obj_text = cJSON_GetObjectItem(w, "obj_text")->valuestring;
        char *text_type = cJSON_GetObjectItem(w, "text_type")->valuestring;

        // ── Create base object ──
        lv_obj_t *obj = lv_obj_create(parent);
        lv_obj_set_pos(obj, x, y);
        lv_obj_set_size(obj, wd, ht);

        // ── Optional: style (same for ALL types) ──
        cJSON *style = cJSON_GetObjectItem(w, "style");
        if (style) apply_style(obj, style);

        // ── Type-specific create + props ──
        if (strcmp(sub_type, "label") == 0) {
            obj = lv_label_create(parent);
            lv_label_set_text(obj, obj_text);
            if (strcmp(text_type, "expression") == 0) {
                bind_label_expression(obj, obj_text);
            }
            cJSON *lm = cJSON_GetObjectItem(w, "long_mode");
            if (lm) lv_label_set_long_mode(obj, parse_long_mode(lm->valuestring));
            if (cJSON_GetObjectItem(w, "recolor")) lv_label_set_recolor(obj, true);

        } else if (strcmp(sub_type, "button") == 0) {
            obj = lv_btn_create(parent);
            // style + events already handled above

        } else if (strcmp(sub_type, "arc") == 0 || strcmp(sub_type, "bar") == 0) {
            if (sub_type[0] == 'a') obj = lv_arc_create(parent);
```

```
            else obj = lv_bar_create(parent);
            int min = cJSON_GetObjectItem(w, "min")->valueint;
            int max = cJSON_GetObjectItem(w, "max")->valueint;
            lv_arc_set_range(obj, min, max);   // same API for bar
            cJSON *val = cJSON_GetObjectItem(w, "value");
            if (val && cJSON_GetObjectItem(w, "value_type")) {
                bind_value_expression(obj, val->valuestring);
            }

        } else if (strcmp(sub_type, "image") == 0) {
            obj = lv_img_create(parent);
            cJSON *src = cJSON_GetObjectItem(w, "src");
            if (src) lv_img_set_src(obj, lookup_image(src->valuestring));

        } else if (strcmp(sub_type, "switch") == 0) {
            obj = lv_switch_create(parent);
            cJSON *checked = cJSON_GetObjectItem(w, "checked");
            if (checked) bind_switch_state(obj, checked->valuestring);

        } else if (strcmp(sub_type, "dropdown") == 0) {
            obj = lv_dropdown_create(parent);
            cJSON *opts = cJSON_GetObjectItem(w, "options");
            if (opts) {
                lv_dropdown_clear_options(obj);
                cJSON *opt = opts->child;
                for (; opt; opt = opt->next)
                    lv_dropdown_add_option(obj, opt->valuestring, LV_DROPDOWN_POS_LAST);
            }
            cJSON *sel = cJSON_GetObjectItem(w, "selected");
            if (sel) lv_dropdown_set_selected(obj, sel->valueint);

        } else if (strcmp(sub_type, "panel") == 0) {
            // panel = plain container, position/size already set
        }

        // ═══ After setting size/pos on type-specific object ═══
        lv_obj_set_pos(obj, x, y);
        lv_obj_set_size(obj, wd, ht);

        // ── Optional: events (same for ALL types) ──
        cJSON *events = cJSON_GetObjectItem(w, "events");
        if (events) register_events(obj, events);

        // ── Optional: children (same for ALL types) ──
        cJSON *children = cJSON_GetObjectItem(w, "children");
        if (children) parse_widgets(children, obj);
    }
}
```

Static Resource Copy — Build-Time Asset Pipeline

Date: 2026-07-10 **Project:** T3000 Webview — EEZ Studio Migration (Electron → Browser) **Topic:** Automatic resource copy from origin repos into Rust build output for release packaging

---

1. Why

The Rust backend serves three categories of static assets at runtime. None of them live in this repo — they must be copied from sibling repos into the Cargo output tree at build time.

Category	Served at	Content	Origin
Framework sources	`/api/eez-studio/read-text-file?path=...`	C source files for flow runtime compilation	`studio-wasm-libs/release/`
WASM runtimes	`/eez-studio-wasm/wasm/...`	Emscripten-compiled LVGL + dashboard runtimes	`studio-wasm-libs/release/wasm/`
Font assets	`/eez-studio-assets/...`	WOFF/WOFF2/TTF fonts + CSS	`eez-studio/packages/eez-studio-ui/_stylesheets/`

Browser runtime path

The browser has no filesystem access. EEZ Studio code (ported from Electron) uses a browser-stub `fs.promises.readFile()` that resolves to HTTP:

```
TEXT

browser JS → browser-stub fs.promises.readFile()
  → GET /api/eez-studio/read-text-file?path=eez-framework-amalgamation/eez-flow.h
    → Rust server: resolve_path(data_root(), path)
      → T3Web/t3-eez/resources/eez-framework-amalgamation/eez-flow.h
```

`data_root()` (in `api/src/t3_eez_studio/mod.rs`) → `{current_dir}/T3Web/t3-eez/`.

---

2. Build Script — `api/build.rs`

Runs automatically on every `cargo build` and `cargo build --release`. All destinations resolve to `{profile_root}/T3Web/t3-eez/resources/`.

2.1. Three Copy Helpers

Helper	What it copies	When to use
`copy_resource_dir(manifest_dir, root, relative_src)`	**All files** in a directory → `resources/{dirname}/`	Whole directories (framework sources, docker-build)
`copy_selected_files(manifest_dir, root, relative_src_dir, files, dest_rel)`	**Named files** with optional renaming → `resources/{dest_rel}/`	Picking specific files from a large directory
`copy_wasm_and_fonts(manifest_dir, root)`	All WASM runtimes + font assets (uses `copy_selected_files` internally)	One call for the full WASM + fonts pipeline

2.2. Framework Source Directories (whole-directory copy)

```rust
// eez-framework-amalgamation — flow runtime C sources
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../studio-wasm-libs/release/eez-framework-amalgamation",
);
// → resources/eez-framework-amalgamation/
//     eez-flow.h, eez-flow.cpp, eez-flow-lz4.c, eez-flow-lz4.h, ...

// docker-build — build scripts for LVGL project compilation
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../eez-studio/resources/docker-build",
);
// → resources/docker-build/
```

2.3. WASM Runtimes (selected-file copy)

`copy_wasm_and_fonts()` copies three groups:

Dashboard + GUI Lite runtime → `resources/eez-studio-wasm/wasm/`:

```text
eez_runtime.js, eez_runtime.wasm, eez_gui_lite_runtime.js,
eez_gui_lite_runtime.wasm, lz4.js, lz4.wasm
```

LVGL per-version runtimes → `resources/eez-studio-wasm/wasm/lvgl/{version}/`:

```text
8.4.0/  lvgl_runtime_v8.4.0.js, lvgl_runtime_v8.4.0.wasm
9.2.2/  lvgl_runtime_v9.2.2.js, lvgl_runtime_v9.2.2.wasm
9.3.0/  lvgl_runtime_v9.3.0.js, lvgl_runtime_v9.3.0.wasm
9.4.0/  lvgl_runtime_v9.4.0.js, lvgl_runtime_v9.4.0.wasm
9.5.0/  lvgl_runtime_v9.5.0.js, lvgl_runtime_v9.5.0.wasm
```

Font assets → `resources/eez-studio-assets/`:

```text
FontAwesome5-Solid+Brands+Regular.woff, material-icons.css,
MaterialIcons-Regular.woff2, Roboto-Regular.ttf, RobotoMono-Regular.ttf
```

2.4. Complete Output Tree

```
target/<profile>/T3Web/t3-eez/resources/
├── eez-framework-amalgamation/      ← copy_resource_dir (studio-wasm-libs)
│   ├── eez-flow.h
│   ├── eez-flow.cpp
│   ├── eez-flow-lz4.c
│   ├── eez-flow-lz4.h
│   ├── eez-flow-sha256.c
│   └── eez-flow-sha256.h
├── docker-build/                    ← copy_resource_dir (eez-studio)
│   ├── Dockerfile
│   └── ...
├── eez-studio-wasm/                 ← copy_wasm_and_fonts (selected files)
│   └── wasm/
│       ├── eez_runtime.js
│       ├── eez_runtime.wasm
│       ├── eez_gui_lite_runtime.js
│       ├── eez_gui_lite_runtime.wasm
│       ├── lz4.js
│       ├── lz4.wasm
│       └── lvgl/
│           ├── 8.4.0/
│           ├── 9.2.2/
│           ├── 9.3.0/
│           ├── 9.4.0/
│           └── 9.5.0/
└── eez-studio-assets/               ← copy_wasm_and_fonts (fonts)
    ├── FontAwesome5-Solid+Brands+Regular.woff
    ├── material-icons.css
    ├── MaterialIcons-Regular.woff2
    ├── Roboto-Regular.ttf
    └── RobotoMono-Regular.ttf
```

---

3. Server Routing

The Rust server (`api/src/server.rs`) mounts each resource directory at a URL prefix:

URL prefix	Serves from	Route function
`/eez-studio-wasm`	`resources/eez-studio-wasm/`	`routes_wasm()`
`/eez-studio-assets`	`resources/eez-studio-assets/`	`routes_assets()`
`/api/eez-studio`	`data_root()` (via `read_text_file` handler)	`bridge_routes()`
`/` (fallback)	SPA dist directory	`routes_static()`

RUST

```rust
// server.rs — create_t3_app()
.nest("/eez-studio-wasm", routes_wasm())
.nest("/eez-studio-assets", routes_assets())
.fallback_service(routes_static())

fn routes_wasm() -> Router {
    Router::new().nest_service("/", get_service(ServeDir::new(
        data_root().join("resources").join("eez-studio-wasm")
    )))
}
```

---

4. Vite Dev Server Proxy

In development, Vite runs on port 3003 and proxies WASM/asset requests to the Rust backend on port 9103:

JAVASCRIPT

```javascript
// quasar.config.js — extendViteConf
viteConf.server.proxy = {
  "/api/eez-studio": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
  "/eez-studio-wasm": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
  "/eez-studio-assets": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
};
```

> **Note:** The proxy prefix was renamed from `/t3-eez-studio` to `/eez-studio-wasm` to match the Rust-side naming. Both `build.rs` destination and `server.rs` route use `eez-studio-wasm`.

---

5. Release Packaging Flow

```
TEXT
```

```
┌─ Dev machine ──────────────────────────────────────────┐
│                                                        │
│  1. vite build                                         │
│     → dist/spa/                                        │
│                                                        │
│  2. cargo build --release                              │
│     → target/release/*.dll                             │
│     → target/release/T3Web/t3-eez/resources/           │
│         eez-framework-amalgamation/    (framework C sources)  │
│         docker-build/                  (Docker build scripts)  │
│         eez-studio-wasm/wasm/          (WASM runtimes)         │
│         eez-studio-assets/             (fonts)                 │
│                                                        │
│  3. Manual packaging:                                  │
│     copy target/release/*.dll          → package/      │
│     copy target/release/T3Web/         → package/T3Web/ │
│     copy dist/spa/                     → package/www/  │
└────────────────────────────────────────────────────────┘
```

Both the DLL and its runtime resources originate from a single `target/release/` tree.

---

6. Relevant Files

File	Role
`api/build.rs`	Three copy helpers; copies all resources into `target/<profile>/T3Web/t3-eez/resources/`
`api/src/server.rs`	Mounts `/eez-studio-wasm`, `/eez-studio-assets`, SPA fallback
`api/src/t3_eez_studio/mod.rs`	`data_root()` → `{cwd}/T3Web/t3-eez/`; `read_text_file` handler
`api/src/t3_eez_studio/bridge_routes.rs`	`/api/eez-studio/*` routes for file read/write
`quasar.config.js`	Vite dev proxy for `/eez-studio-wasm` and `/eez-studio-assets`
`../../studio-wasm-libs/release/`	Origin: framework amalgamation + WASM runtimes
`../../eez-studio/resources/docker-build/`	Origin: Docker build scripts
`../../eez-studio/packages/eez-studio-ui/_stylesheets/`	Origin: font assets

---

7. Adding More Static Resources

Whole directory (all files):

```
RUST
```

```rust
// In build.rs → main()
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../some-repo/path/to/dir",
);
// → resources/dir/
```

Specific files (with optional renaming):

```rust
copy_selected_files(
    &manifest_dir,
    profile_root,
    "../../some-repo/path/to/source",
    &[
        ("source_name.ext", "sub/dest_name.ext"),
    ],
    "my-dest-dir",  // → resources/my-dest-dir/
);
```

New server route:

```rust
// In server.rs → create_t3_app()
.nest("/my-prefix", routes_my_prefix())

fn routes_my_prefix() -> Router {
    Router::new().nest_service("/", get_service(ServeDir::new(
        data_root().join("resources").join("my-dest-dir")
    )))
}
```

New Vite proxy:

```javascript
// In quasar.config.js → extendViteConf → viteConf.server.proxy
"/my-prefix": {
    target: "http://localhost:9103",
    changeOrigin: true,
},
```

LVGL WASM Browser Build Migration

Date: 2025-06-25 (updated 2026-07-10) **Project:** T3000 Webview — EEZ Studio Migration (Electron → Browser) **Branch:** WASM runtime for LVGL project editor

---

Overview

Migrated the EEZ Studio LVGL flow runtime WASM from Electron/Node.js target to browser (Vite/Quasar). The original `studio-wasm-libs` produces WASM modules targeting Node.js APIs (`__dirname` , `NODEJS_CATCH_EXIT`). These were rebuilt with Emscripten targeting `ENVIRONMENT=web` and deployed as static assets served by the Rust backend.

Two WASM runtimes are now served:

Runtime	Served at	Used by
LVGL per-version	`/eez-studio-wasm/wasm/lvgl/{version}/`	LVGL project page rendering (lvgl-versions.ts)
Dashboard (eez_runtime)	`/eez-studio-wasm/wasm/`	Flow runtime for dashboards + GUI lite (wasm-worker.ts)

Architecture

TEXT

```
┌──────────────────────────────────────────────┐
│ Browser (Vite/Quasar)                        │
│                                              │
│  lvgl-versions.ts                            │
│    │ detect window !== undefined             │
│    │ construct URL:                          │
│    │   /eez-studio-wasm/wasm/lvgl/{v}/...    │
│    ▼                                         │
│  <script src=...> → WebAssembly.instantiate()  │
│    │                                         │
│  LVGL Runtime (Module._lvglInit, _lvglTick...)  │
│                                              │
│  wasm-worker.ts (Web Worker)                 │
│    │ eez_runtime.js, eez_runtime.wasm        │
│    │   /eez-studio-wasm/wasm/                │
│    ▼                                         │
│  Flow Runtime (dashboard + GUI lite)         │
└──────────────────────────────────────────────┘
        │
        │ HTTP GET (dev: Vite proxy → Rust; prod: Rust serve)
        ▼
┌──────────────────────────────────────────────┐
│ Rust Backend (port 9103)                     │
│                                              │
│  routes_wasm() — ServeDir at:                │
│    T3Web/t3-eez/resources/eez-studio-wasm/   │
│                                              │
│  Contents (copied by build.rs):              │
│   wasm/                                      │
│   ├── eez_runtime.{js,wasm}                  │
│   ├── eez_gui_lite_runtime.{js,wasm}         │
│   ├── lz4.{js,wasm}                          │
│   └── lvgl/                                  │
│       ├── 8.4.0/lvgl_runtime_v8.4.0.{js,wasm}  │
│       ├── 9.2.2/lvgl_runtime_v9.2.2.{js,wasm}  │
│       ├── 9.3.0/lvgl_runtime_v9.3.0.{js,wasm}  │
│       ├── 9.4.0/lvgl_runtime_v9.4.0.{js,wasm}  │
│       └── 9.5.0/lvgl_runtime_v9.5.0.{js,wasm}  │
└──────────────────────────────────────────────┘
```

Source Repository

`https://github.com/eez-open/studio-wasm-libs`

Local clone: `C:\QN\temcocontrols\studio-wasm-libs`

Submodules

Submodule	Path	Version
`eez-framework`	`eez-framework/`	master
`lvgl`	`lvgl-runtime/v8.4.0/lvgl`	v8.4.0
`lvgl`	`lvgl-runtime/v9.2.2/lvgl`	v9.2.2
`lvgl`	`lvgl-runtime/v9.3.0/lvgl`	v9.3.0
`lvgl`	`lvgl-runtime/v9.4.0/lvgl`	v9.4.0
`lvgl`	`lvgl-runtime/v9.5.0/lvgl`	v9.5.0

Patches Applied

1. `lvgl-runtime/common/pre.js` — Browser file resolution

DIFF

```
- return __dirname + "/" + path;
+ return new URL(path, import.meta.url).href;
```

`__dirname` is a Node.js global. Browsers use `import.meta.url`.

2. All 5 `lv_conf.h` files — Disable FreeType

DIFF

```
- #define LV_USE_FREETYPE 1
+ #define LV_USE_FREETYPE 0
```

FreeType requires system font loading (`/home/mvladic/freetype-2.14.1`) not available in browser WASM.

Files: `lvgl-runtime/{v8.4.0,v9.2.2,v9.3.0,v9.4.0,v9.5.0}/lv_conf.h`

3. All 5 `CMakeLists.txt` — Remove hardcoded FreeType paths

Removed:

- `-L/home/mvladic/freetype-2.14.1/build -lfreetype` from linker flags
- `include_directories(/home/mvladic/freetype-2.14.1/include)` line

Files: `lvgl-runtime/{v8.4.0,v9.2.2,v9.3.0,v9.4.0,v9.5.0}/CMakeLists.txt`

4. `lvgl-runtime/common/src/studio_api.cpp` — Guard FreeType code

Wrapped `lvglCreateFreeTypeFont()` with `#if LV_USE_FREETYPE` / `#endif` preprocessor guards to prevent compilation errors from `lv_ft_info_t` usage in v8.x code path.

5. `lvgl-runtime/v9.5.0/exported-functions.txt` — Remove FreeType exports

Removed 17 `_lv_freetype_*` symbols from the Emscripten export list. These functions don't exist when FreeType is disabled, causing `wasm-ld` linker errors.

6. `lvgl-runtime/common/src/flow.cpp` — Remove `setNotStopped()`

Removed `eez::flow::setNotStopped()` call. Upstream `eez-framework` removed this function; `g_isStopped` defaults to `true` and is set to `false` by `start()` (called during `_init()`), making the call redundant.

Build Environment

Tool	Version	Path
Emscripten SDK	6.0.1	`C:\QN\temcocontrols\emsdk`
CMake	4.3.3	`C:\Program Files\CMake\bin`
Ninja	1.13.2	`C:\QN\temcocontrols\ninja`

Build Command

```cmd
call C:\QN\temcocontrols\emsdk\emsdk_env.bat
set PATH=C:\Program Files\CMake\bin;C:\QN\temcocontrols\ninja;%PATH%
cd lvgl-runtime\<version>
mkdir build && cd build
emcmake cmake ..
emmake ninja -j4
```

Build Results

Version	JS	WASM	Files
v8.4.0	608 KB	1.96 MB	238 source files
v9.2.2	763 KB	1.97 MB	476 source files
v9.3.0	815 KB	2.10 MB	584 source files
v9.4.0	856 KB	2.14 MB	691 source files
v9.5.0	889 KB	2.18 MB	694 source files

Build & Deploy Pipeline

```text
1. Edit C++ source in studio-wasm-libs/
       |
2. Build all: cd studio-wasm-libs && .\build-all.bat
       |  Output → studio-wasm-libs/release/wasm/
       |    eez_runtime.{js,wasm}
       |    eez_gui_lite_runtime.{js,wasm}
       |    lz4.{js,wasm}
       |    lvgl_runtime_v{8.4.0,9.2.2,9.3.0,9.4.0,9.5.0}.{js,wasm}
       |
3. cargo build (or cargo build --release)
       |  build.rs auto-copies from studio-wasm-libs/release/wasm/
       |  → target/<profile>/T3Web/t3-eez/resources/eez-studio-wasm/wasm/
       |
4. Rust server serves at runtime
       |  /eez-studio-wasm/wasm/... → T3Web/t3-eez/resources/eez-studio-wasm/
```

> **Never manually copy files into `api/target/`**. The pipeline is: edit in `studio-wasm-libs` → `build-all.bat` → `cargo build` (auto-copy) → serve.

Frontend Code Changes

`lvgl-versions.ts` — LVGL runtime URL

DIFF

```
- const jsUrl = `/t3-eez-studio/wasm/lvgl/${version}/${fileName}`;
+ const jsUrl = `/eez-studio-wasm/wasm/lvgl/${version}/${fileName}`;
```

Also the v8.4.0 hardcoded fallback path:

DIFF

```
- wasmFlowRuntime: "project-editor/flow/runtime/wasm/lvgl_runtime_v8.4.0.js",
+ wasmFlowRuntime: "/eez-studio-wasm/wasm/lvgl/8.4.0/lvgl_runtime_v8.4.0.js",
```

`wasm-worker.ts` — Dashboard flow runtime (Web Worker)

TYPESCRIPT

```
const wasmPath = "/eez-studio-wasm/wasm/eez_runtime.wasm";
(g as any).__dirname = "/eez-studio-wasm/wasm";
s.src = `/eez-studio-wasm/wasm/eez_runtime.js?v=${EEZ_RUNTIME_CACHE_BUSTER}`;
```

Vite Configuration

Dev proxy (`quasar.config.js`)

In development, Vite (port 3003) proxies WASM requests to the Rust backend (port 9103):

JAVASCRIPT

```
viteConf.server.proxy = {
  "/eez-studio-wasm": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
  "/eez-studio-assets": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
};
```

WASM MIME type

JAVASCRIPT

```
viteConf.server.headers = { "*.wasm": { "Content-Type": "application/wasm" } };
```

Ensures Vite dev server serves `.wasm` files with correct MIME type for `WebAssembly.instantiateStreaming()` .

Runtime Loading Flow

LVGL Runtime (lvgl-versions.ts)

1. User opens an LVGL project page in the EEZ Studio editor
2. `page-runtime.ts` creates a runtime instance
3. `lvgl-versions.ts:getLvglWasmFlowRuntimeConstructor()` is called
4. Detects `typeof window !== "undefined"` (browser environment)
5. Extracts version from `wasmFlowRuntime` property (e.g., `"9.5.0"`)
6. Constructs URL: `/eez-studio-wasm/wasm/lvgl/<version>/lvgl_runtime_v<version>.js`

7. Loads Emscripten JS glue via `<script>` tag → sets `globalThis.LVGLWasmRuntime`
8. WASM module exports LVGL functions (`_lvglInit` , `_lvglTick` , etc.)
9. Fallback: Proxy-based no-op if WASM fails to load (prevents crashes)

Dashboard Flow Runtime (wasm-worker.ts)

1. Web Worker loads `eez_runtime.js` via `importScripts()`
2. `__dirname` set to `/eez-studio-wasm/wasm` for Emscripten's `locateFile`
3. `WebAssembly.instantiateStreaming(fetch(wasmPath))` loads `eez_runtime.wasm`
4. Worker communicates with main thread via `postMessage` (action-based protocol)
5. Flow runtime executes dashboard logic in the worker, sends rendering commands to main thread

---

Path Rename: `t3-eez-studio` → `eez-studio-wasm`

All WASM-related URLs were renamed to match the Rust-side naming:

File	Change
`lvgl-versions.ts` (line 199)	Hardcoded v8.4.0 path
`lvgl-versions.ts` (line 613)	Dynamic URL construction
`wasm-worker.ts` (lines 51, 82, 85)	`wasmPath` , `__dirname` , script `src`
`quasar.config.js` (line 231)	Vite proxy entry
`api/build.rs` (3 calls)	`copy_selected_files` dest_rel → `"eez-studio-wasm"`
`api/src/server.rs` (lines 131, 254)	`ServeDir` path + `.nest("/eez-studio-wasm", ...)`

Known Limitations

1. **FreeType disabled** — Custom TTF/OTF fonts not supported. Only built-in LVGL bitmap fonts (Montserrat).
2. **No file system** — Emscripten FS not initialized in browser. Image loading via memory buffers only.

Future Rebuilds

When `studio-wasm-libs` upstream updates:

1. `git pull` + `git submodule update --init --recursive`
2. Re-apply patches 1-6 above
3. Run `build-all.bat` (builds all 5 LVGL versions + eez_runtime + eez_gui_lite_runtime + lz4)
4. `cargo build` — `build.rs` auto-copies WASM files to target
5. Restart Rust backend to serve updated files

Status Bar Component

Overview

The Status Bar sits at the bottom of the T3000 application window and shows real-time feedback about the system state. It reports RX/TX packet counts and displays color-coded status messages to keep you informed about what the system is doing.

What You See

The Status Bar has two visible sections:

RX/TX Statistics

Shows how many data packets have been received (**RX**) and transmitted (**TX**) between your browser and the backend server. These counters increment as the system communicates.

Status Message

Displays a text message describing the current operation or result. The message bar changes color depending on the type of information:

Color	Meaning	Example
Gray	Informational	`Refreshing variables from device...`
Green	Success	`✓ Synced 42 variables from Panel-1`
Yellow	Warning	`No devices in database`
Red	Error	`Failed to sync: Network error`

If a message is longer than 100 characters, an **info icon** () appears next to the message. Hover over it to see the full text in a tooltip.

Where Messages Come From

Messages appear automatically when the system performs actions. Here are common scenarios:

- **Loading a device** → `Loading variables from Panel-1 (Action 17)...`
- **Refresh complete** → `✓ Synced 50 inputs from Panel-1`
- **Database operations** → `Loaded 15 devices`
- **Errors** → `Failed to sync: Connection refused`

You do not need to interact with the Status Bar — it updates automatically as you use the application.

Component Architecture

File Locations

File	Purpose
`src/t3-react/layout/StatusBar.tsx`	Presentational component (Fluent UI styled)
`src/t3-react/store/statusBarStore.ts`	Zustand state management
`src/t3-react/layout/MainLayout.tsx`	Wires store → component via props

Props Interface

TYPESCRIPT

```typescript
export interface StatusBarProps {
  rxCount?: number;        // Received packet count (default: 0)
  txCount?: number;        // Transmitted packet count (default: 0)
  buildingName?: string;   // Building/panel name (default: '')
  deviceName?: string;     // Device name (default: '')
  protocol?: string;       // Protocol name (default: '')
  connectionType?: string; // Connection type (default: '')
  message?: string;        // Status message text (default: 'Ready')
  messageType?: MessageType; // 'info' | 'success' | 'error' | 'warning' (default: 'info')
}
```

> **Note:** Panes for Building/Device and Protocol info are currently **commented out** in the JSX. Only RX/TX stats and the message pane are active.

Zustand Store API

TYPESCRIPT

```typescript
type MessageType = 'info' | 'success' | 'error' | 'warning';

interface StatusBarState {
  // State fields
  rxCount: number;
  txCount: number;
  buildingName: string;
  deviceName: string;
  protocol: string;
  connectionType: string;
  message: string;
  messageType: MessageType;

  // Actions
  incrementRx: () => void;
  incrementTx: () => void;
  setRxTx: (rx: number, tx: number) => void;
  setConnection: (building: string, device: string) => void;
  setProtocol: (protocol: string, connectionType: string) => void;
  setMessage: (message: string, type?: MessageType) => void;
  reset: () => void;
}
```

How to Dispatch Messages

From inside a React component (reactive hook):

TYPESCRIPT

```typescript
import { useStatusBarStore } from '@t3-react/store/statusBarStore';

const setMessage = useStatusBarStore((state) => state.setMessage);

// Informational
setMessage('Refreshing variables from device...', 'info');

// Success
setMessage('✓ Synced 42 variables from Panel-1', 'success');

// Warning
setMessage('No devices found in T3000', 'warning');

// Error
setMessage('Failed to sync: Network error', 'error');

// type defaults to 'info' if omitted
setMessage('Loading data...');
```

From outside React (e.g., stores, services, callbacks):

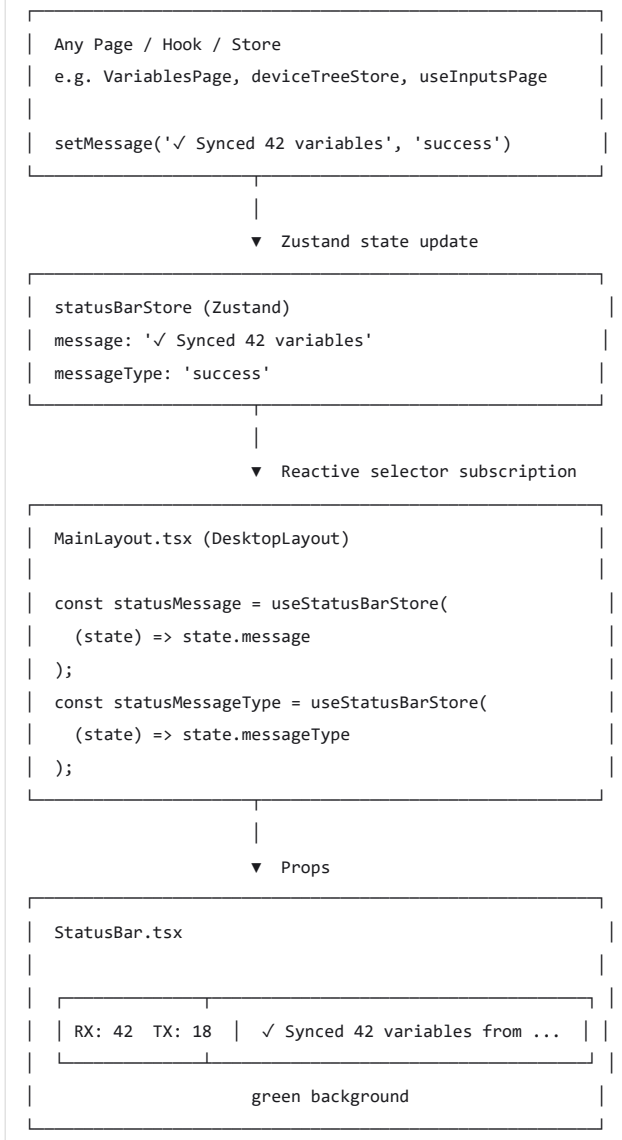
TYPESCRIPT

```typescript
import { useStatusBarStore } from '@t3-react/store/statusBarStore';

useStatusBarStore.getState().setMessage('Loaded 15 devices', 'success');
useStatusBarStore.getState().setMessage('Error: Connection refused', 'error');
```

Data Flow Diagram

TEXT



Message Type Styling

The message type controls both the background color of the entire status bar and the text color of the message:

`messageType`	Bar Background	Text Color	CSS Class
`'info'`	`#f0f0f0` (gray)	`#323130` (default)	—
`'success'`	`#dff6dd` (green)	`#0e700e`	`statusBarSuccess` / `messagePaneSuccess`
`'error'`	`#fde7e9` (red)	`#a4262c`	`statusBarError` / `messagePaneError`
`'warning'`	`#fff4ce` (yellow)	`#8a5d00`	`statusBarWarning` / `messagePaneWarning`

Long Message Tooltip

When `message.length > 100`, a Fluent UI `Tooltip` wraps an `Info16Regular` icon next to the message text. Hovering shows the full message positioned `above-start`.

Key Consumers

The status bar receives messages from across the application:

- **Device tree** (`deviceTreeStore.ts`) — device load, sync, delete operations

- **Variables page** (`useVariablesPage.ts` , `VariablesPage.tsx`) — fetch, refresh, sync
- **Inputs page** (`useInputsPage.ts` , `InputsPage.tsx`) — fetch, refresh, sync
- **Outputs page** (`useOutputsPage.ts` , `OutputsPage.tsx`) — fetch, refresh, sync
- **Graphics page** (`GraphicsPage.tsx`) — graphic load, sync
- **Tree toolbar** (`TreeToolbar.tsx`) — device refresh errors

Design Notes

- **Global singleton** — One Zustand store; any code path can write messages without prop drilling.
- **No auto-dismiss** — Messages persist until overwritten by another call or `reset()` is invoked.
- **Reactive rendering** — Zustand selectors ensure StatusBar re-renders only when the subscribed fields change.
- **Thread-safe** — `useStatusBarStore.getState()` allows non-React code (stores, callbacks, services) to dispatch messages safely.

Document generated for T3000 Webview v9.0 — Status Bar Component

Shared Center DB — Overview

Use this page to understand what Shared Center DB is and whether you need it. If you manage multiple T3000 PCs and want them to share trend logs and device data across the network — this is the feature for you.

- Full SQL Server install steps: SQL Server Express Setup
- Full T3000 UI walkthrough: T3000 Center DB Configuration

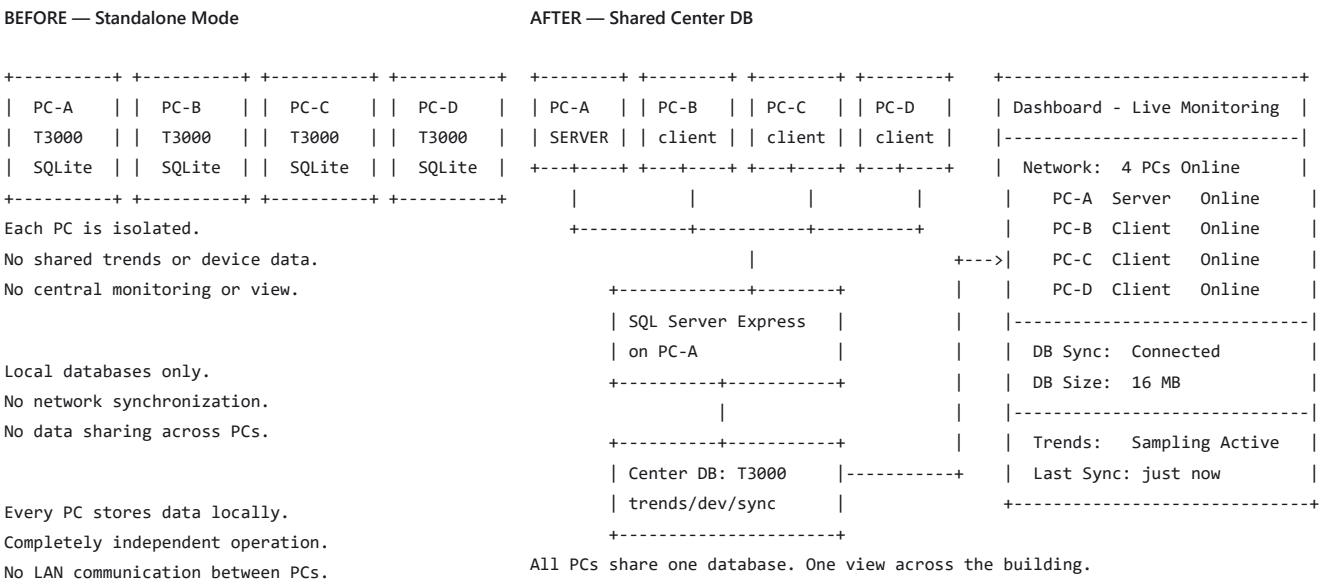
---

1. What Is It?

Summary: Shared Center DB changes T3000 from isolated PCs to one shared system. Instead of each PC keeping separate data, all PCs connect to one central SQL Server database so users see the same building view.

How to read this figure:

1. **Before (left):** each PC is standalone with its own local SQLite data.
2. **After (right):** one host PC runs SQL Server and all other PCs connect as clients.
3. **Dashboard panel:** users monitor shared network status, DB sync health, and trend activity from one place.



---

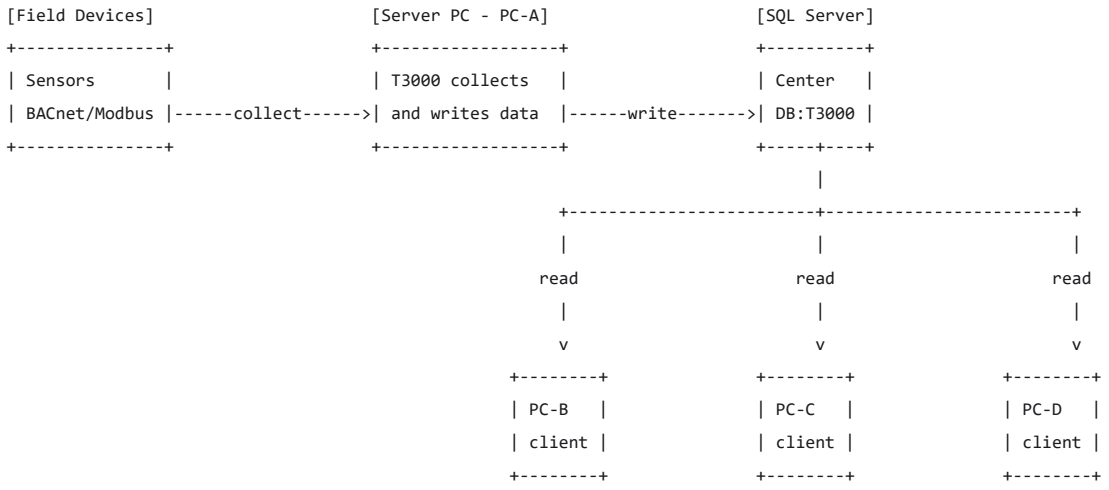
2. How It Works

Summary: Field data is collected by the Server PC, written to the Center DB, then read by all Client PCs. This keeps dashboards and trends consistent across the team.

Runtime flow (simple):

1. Field devices publish values and events.
2. Server PC collects data and writes shared/trend data to the center SQL DB.
3. Client PCs read shared/trend data from the center SQL DB and show the same status and trends.
4. Local SQLite keeps core local runtime info (not trendlog storage).

TEXT



- **Server PC** — writes shared/trend data to the center SQL DB; local SQLite keeps core local runtime info only
- **Client PCs** — read shared/trend data from the center SQL DB; local SQLite keeps core local runtime info for offline continuity

---

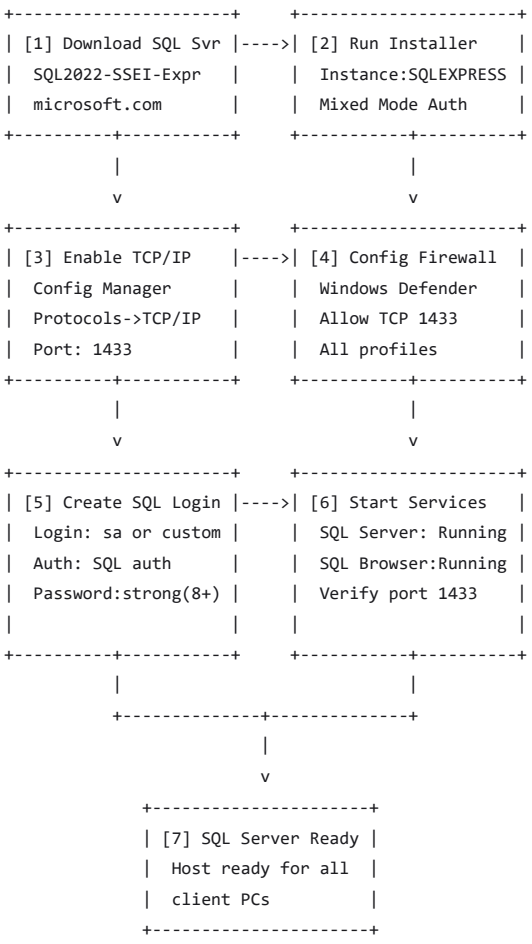
3. Setup — Two Phases

Summary: This setup gives your team one shared view of the building. Users on any T3000 PC can see the same trends, device data, and connection status in real time.

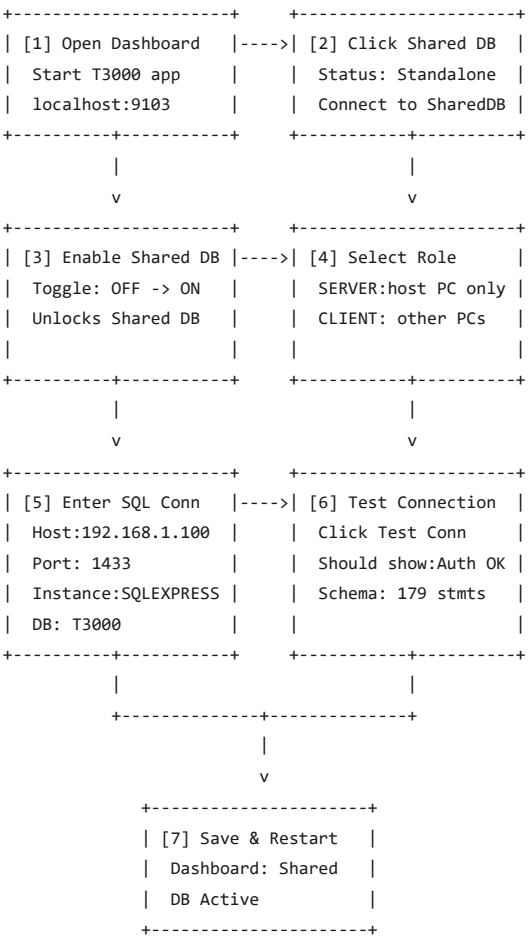
How to use this section:

1. Do **Phase 1** once on the main PC that will host SQL Server.
2. Do **Phase 2** on every T3000 PC:
   - Host PC = **Server** role
   - All other PCs = **Client** role

Phase 1 — Host PC only (~30 min)



Phase 2 — Each T3000 PC (~5 min)



---

4. Server vs Client Role

Role	Choose when	What it does
Server	This PC hosts SQL Server	Writes shared/trend data to center DB; local SQLite stores core local runtime info only
Client	Any other PC on the LAN	Reads shared/trend data from center DB; local SQLite stores core local runtime info only

One Server. All other PCs are Clients.

---

5. What Success Looks Like

After saving the configuration and restarting T3000, open the Dashboard at `localhost:9103`. You should see:

TEXT

```
+-------------------------------------------------+
|  Shared DB                              ACTIVE  |
|  SQL Server . Server                 Connected  |
|                                                 |
|  Network Overview                               |
|    Server (This PC)   192.168.1.100     Online  |
|    Client - PC-B      192.168.1.101     Online  |
|    Client - PC-C      192.168.1.102     Online  |
|                                                 |
|  Sync & DB Health     Connected                 |
|  Center DB Target     SQL Server / T3000        |
|  Trend Logs (24h)     Sampling Active           |
+-------------------------------------------------+
```

---

6. Quick Troubleshooting

Symptom	Likely Cause	First Check
Connection timeout	SQL service stopped or firewall blocking	Start SQL service; allow TCP 1433 inbound
Login failed	Wrong credentials or Windows-only auth mode	Check username/password; enable Mixed Mode
Instance not found by Scan LAN	SQL Browser service not running	Start SQL Server Browser, or enter instance manually
Still shows Standalone after save	T3000 not restarted	Close and reopen T3000

---

7. Next Steps

- **Set up SQL Server on the host PC:** SQL Server Express Setup
- **Configure T3000 UI on each PC:** T3000 Center DB Configuration

SQL Server Express 2022 Setup for T3000 (Detailed Step-by-Step)

Estimated time: 25 to 40 minutes.

> **Official documentation:** Microsoft SQL Server installation documentation: https://learn.microsoft.com/en-us/sql/database-engine/install-windows/install-sql-server This guide may be used as an alternative step-by-step reference for the T3000 setup workflow.

> **Database size limit (SQL Server Express):** SQL Server Express has a maximum relational database size of **10 GB per database**. This limit is defined by Microsoft in SQL Server 2022 Scale limits. If a single database requires more than 10 GB, use SQL Server Standard or higher. Source: https://learn.microsoft.com/en-us/sql/sql-server/editions-and-components-of-sql-server-2022#scale-limits

What You Need Before Starting

1. Windows 10 or Windows 11 PC (64-bit) with administrator rights.
2. Internet access.
3. T3000 application installed.
4. At least 1.5 GB free disk space.

> **Tip: Separate install folder and data folder** Keep SQL Server program files in the default install location, and place database data files on another drive. During setup, open the **Data Directories** tab and set **Data Root Directory** to a path like **D:\SQLData**. This keeps the C: drive from filling up and leaves more room for Windows and applications.

Step 1: Download SQL Server 2022 Express

1. Open your web browser.
2. Go to this URL:

```
TEXT

https://www.microsoft.com/en-us/download/details.aspx?id=104781&msockid=01e188d1ef006e6936d29f94ee4a6f7a
```

3. Scroll down to **Express** section.
4. Click **Download now** under SQL Server 2022 Express.
5. Save file to your Desktop.

Expected file name:

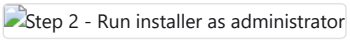
- `SQL2022-SSEI-Expr.exe` (small web installer, ~5 MB, downloads installation files during setup)



Step 2: Run SQL Server Express Installer

1. Right-click the downloaded file.
2. Click **Run as administrator**.
3. Click **Yes** on User Account Control prompt.

> If Windows shows a security warning ("publisher can't be verified"), click **Run** to continue. This is expected for the Microsoft installer.

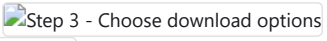
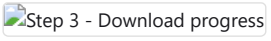
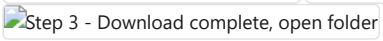


Step 3: Prepare SQL Server Installer Media (Pre-Install)

1. In the SQL Server Express launcher, select **Download Media**.
2. On the download options screen, keep **Express Core** selected.
3. Select a download folder (for example: `C:\Users\<your-user>\Downloads`).
4. Click **Download** and wait for the media download to finish.
5. When download is complete, click **Open folder**.

Expected result:

- The full installer file is available in the selected folder.
- Continue by running that installer in the next step.

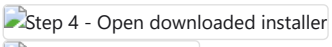
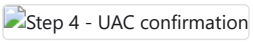
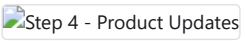
  


Step 4: Start the Full SQL Server Installation

1. In the folder opened from Step 3, find and run **SQLEXPR_x64_ENU.exe**.
2. If User Account Control appears, click **Yes**.
3. In **SQL Server Installation Center**, click:
   - **New SQL Server standalone installation or add features to an existing installation**
4. On **Product Updates**, click **Next**.
5. On **Install Rules**, review the results and click **Next**.

Notes:

- A Windows Firewall warning on Install Rules can appear and does not block installation.
- Network and firewall settings will be configured in later steps.

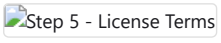
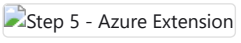
   


Step 5: Select Installation Type and Accept Terms

1. On **Installation Type**, select **Perform a new installation of SQL Server 2022**.
2. Click **Next**.
3. On **License Terms**, check **I accept the license terms and Privacy Statement**.
4. Click **Next**.
5. On **Azure Extension for SQL Server**, leave **Use Microsoft Update to check for updates** unchecked.
6. Click **Next**.

Notes:

- If an existing SQL instance is listed, keep **Perform a new installation** selected for SQL Express.
- The Azure Extension option is not required for this setup.

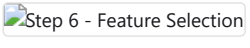
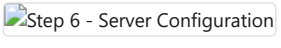
  

Step 6: Select Features, Instance, and Server Configuration

1. On **Feature Selection**, keep the default selections shown in the installer.
2. Click **Next**.
3. On **Instance Configuration**, set:
   - **Named instance**: `SQLExpress`
   - **Instance ID**: `SQLEXPRESS`
4. Click **Next**.
5. On **Server Configuration**, review the service settings and click **Next**.

Notes:

- Use the same instance naming shown in the screenshots to match later connection steps.
- SQL Server Browser may appear as **Disabled** at this stage and can be configured later.

Step 7: Configure Database Engine and Data Directories

This step has two important tabs: **Server Configuration** and **Data Directories**.

1. In **Database Engine Configuration** on the **Server Configuration** tab:
   - Select **Mixed Mode (SQL Server authentication and Windows authentication)**.

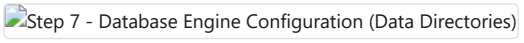
- Enter and confirm a strong password for the `sa` account.
- Click **Add Current User** to add your Windows account as SQL administrator.

2. Verify that your Windows account appears in **Specify SQL Server administrators**.
3. Click the **Data Directories** tab.
4. Review the default paths for data, logs, backup, and temp files.
5. If the C: drive is limited, set **Data root directory** to another drive (for example `D:\SQLData`).
6. Keep paths consistent under the selected root so future maintenance is easier.
7. After confirming both tabs, click **Next**.

Recommended practice:

- Keep SQL Server program files and database data files on separate drives when possible.
- This reduces C: drive growth and improves long-term system stability.



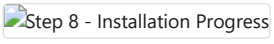


Step 8: Run Installation and Confirm Completion

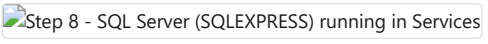
1. On the **Installation Progress** page, wait until setup finishes.
2. Do not close the installer while the progress bar is active.
3. When the **Complete** page appears, verify each selected feature shows **Succeeded**.
4. Confirm the details area shows **Install successful**.
5. Click **Close**.

Verification checklist:

- **Database Engine Services** status is **Succeeded**.
- **SQL Server Replication** status is **Succeeded** (if selected in Feature Selection).
- Setup summary shows installation completed successfully.
- In **Windows Services**, **SQL Server (SQLEXPRESS)** shows **Running**.







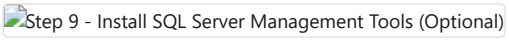
Step 9 (Optional): Open SQL Server Management Tools Download Page

Use this step only if you want a graphical management tool (SSMS) for manual database tasks.

1. In **SQL Server Installation Center**, stay on the **Installation** tab.
2. Click **Install SQL Server Management Tools**.
3. Your browser opens the Microsoft download page for SSMS.
4. Continue with SSMS installation only if your workflow requires manual administration.

Notes:

- This step is optional for core SQL Server Engine installation.
- It is recommended for troubleshooting, query testing, and manual database setup.



Step 10: Configure TCP/IP in SQL Server Configuration Manager

Official Microsoft documentation (reference first):

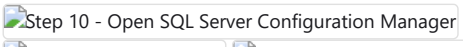
- https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/enable-or-disable-a-server-network-protocol

1. Open **SQL Server 2022 Configuration Manager** from the Windows Start menu.
2. Expand **SQL Server Network Configuration**.
3. Click **Protocols for SQLEXPRESS**.
4. In the right panel, open **TCP/IP** properties and set **Enabled** = **Yes**.
5. Click **OK** to save.
6. If your environment also requires Named Pipes, open **Named Pipes** properties and set **Enabled** = **Yes**.

7. Open **Services**, right-click **SQL Server (SQLEXPRESS)**, and click **Restart**.
8. Wait until service status returns to **Running**.

Verification checklist:

- **TCP/IP** shows **Enabled** under Protocols for SQLEXPRESS.
- **SQL Server (SQLEXPRESS)** is **Running** after restart.



Step 11: Enable Remote Access for TCP 1433 (Windows Firewall)

Official Microsoft documentation (reference first):

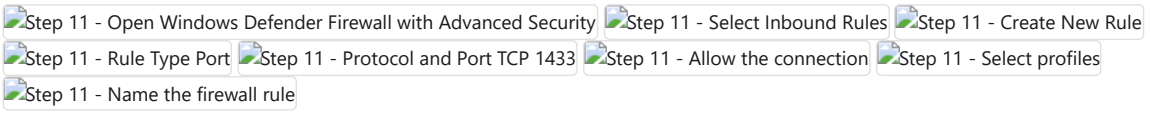
- https://learn.microsoft.com/en-us/sql/sql-server/install/configure-the-windows-firewall-to-allow-sql-server-access

Use this step when clients connect from another PC to this SQL Server instance.

1. Open **Windows Defender Firewall with Advanced Security**.
2. In the left panel, click **Inbound Rules**.
3. In the right panel, click **New Rule...**.
4. In Rule Type, select **Port** and click **Next**.
5. Select **TCP**, choose **Specific local ports**, enter `1433`, then click **Next**.
6. Select **Allow the connection** and click **Next**.
7. Select the network profiles required by your environment (Domain/Private/Public), then click **Next**.
8. Enter a rule name (for example: `SQL SERVER:1433`) and click **Finish**.

Verification checklist:

- New inbound rule for TCP 1433 appears in Firewall Inbound Rules.
- Remote clients can reach the SQL host on port 1433.



Step 12 (Optional): Verify Connection via mssql Tool

Use the **mssql** extension in VS Code to confirm SQL Server is reachable before continuing.

1. Open **VS Code** and install the **mssql** extension if not already installed (search `mssql` in the Extensions panel).
2. Press `Ctrl+Shift+P` -> type **MS SQL: Add Connection** -> press Enter.
3. Enter the server name: `localhost\SQLEXPRESS`.
4. Choose **SQL Login**, enter username `sa` and the password you set in Step 7.
5. Click **Connect**.
6. A green status bar item showing the server name confirms the connection is working.

> If the connection fails, verify TCP/IP is enabled (Step 10), mixed mode is set (Step 7), and the SQL Server service is running (Step 8).



Summary

This guide covered the full installation and configuration of **SQL Server 2022 Express** on a Windows PC for use with T3000.

After completing all steps, you should have:

- SQL Server 2022 Express installed with instance name SQLEXPRESS
- SQL Server Management Studio (SSMS) installed for database administration
- TCP/IP protocol enabled on port **1433**
- Windows Firewall rule allowing TCP 1433 (required for remote connections)

- Mixed mode authentication (SQL + Windows) enabled and active
- SQL Server restarted to apply configuration changes
- (Optional) Connection verified via the mssql tool in VS Code

Next step: Configure T3000 to connect to this SQL Server instance. See the T3000 Database Configuration guide for how to enter connection details, test the connection, initialize the schema, and set the shared database role.

T3000 Center DB Configuration

Use this guide to configure a shared central SQL Server database in T3000. This allows multiple T3000 PCs on the same network to share trend logs and device data through one central Microsoft SQL Server instance.

Before starting, make sure SQL Server Express is installed and TCP/IP is enabled. See the SQL Server Express Setup guide if needed.

Step 1: Open T3000 and Update to the Latest Version

Open the T3000 Building Automation System desktop application.

Before configuring the shared database, make sure T3000 is up to date. Click **Help — Check for Updates** in the menu bar and install any available update. The Shared DB feature requires the latest version to work correctly.


Step 1 — T3000 desktop application

Step 2: Go to the Dashboard

In the T3000 webview, click **Dashboard** in the top navigation bar. You can also open the Dashboard directly in a browser:

- **On this PC:** http://localhost:9103/#/t3000/dashboard
- **From another PC on the same network:** `http://<this-PC-IP>:9103/#/t3000/dashboard` (e.g. `http://192.168.1.22:9103/#/t3000/dashboard`)

You will see two mode cards:

- **Standalone** — stores data locally in SQLite on this PC (currently Active)
- **Shared DB** — connects to a central SQL Server database shared across multiple T3000 PCs

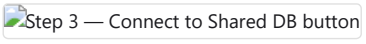
The **Shared DB** card shows *Not configured. Click to connect to a shared SQL Server database.* Click anywhere on the Shared DB card to begin setup.


Step 2 — Dashboard showing Standalone active and Shared DB not configured

---

Step 3: Click "Connect to Shared DB"

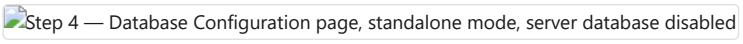
After clicking the Shared DB card, a **Connect to Shared DB** button appears. Click it to open the Database Configuration page.


Step 3 — Connect to Shared DB button

Step 4: Database Configuration Page

The **Database Configuration** page opens, showing:

- **Centralized Trendlog & Data Storage** — description of the feature
- **Database Status** — currently shows *Standalone · Online*
- **Server / Client Configuration** — toggle is currently **Server Database: Disabled**


Step 4 — Database Configuration page, standalone mode, server database disabled

---

Step 5: Enable Server Database and Select Role

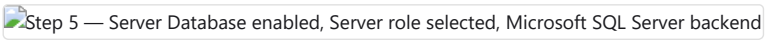
Toggle **Server Database** to **Enabled**.

Two role cards appear:

- **Server** — this PC writes trend logs and device data to the shared database. Typically the main PC that hosts the SQL Server instance. Data is written to both local SQLite and the server database for redundancy.
- **Client** — this PC reads trend logs and device data from the shared database. It continues to write its own local data to SQLite for offline use.

Select **Server** for the PC hosting the SQL Server instance. Select **Client** for all other PCs.

Under **Backend Type**, select **Microsoft SQL Server**.



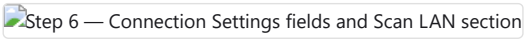
Step 6: Enter Connection Settings

Fill in the **Connection Settings — Microsoft SQL Server** fields:

Field	Description	Example
Host / IP	IP address or hostname of the SQL Server machine	`192.168.1.100`
Port	TCP port SQL Server is listening on	`1433`
Instance Name	SQL Server instance name	`SQLEXPRESS`
Database Name	Database to use (will be created by Init Schema)	`T3000`
Username	SQL login username	`sa`
Password	SQL login password	*(your password)*

Optionally use the **Connection URL** field to override all fields above with a single URL in the format `mssql://user:pass@host:port/dbname`.

To discover SQL Server instances automatically, click **Scan LAN**. The scanner probes ports 1432, 1433, and 1434 on your local subnet and returns all instances found. Click **Use** next to any result to fill in the connection fields automatically.



Step 7: Use Scan LAN Results and Save

After scanning, the results show all discovered SQL Server instances. Click **Use** next to the instance you want to connect to. The connection fields are filled in automatically.

Once the fields are set, click **Save Configuration**. A success banner confirms:

> **Success** Server Database server mode + Microsoft SQL Server settings saved. Restart T3000 to apply.

> **Note:** If SQL Server Browser service is not running on the target machine, instances may appear as *TCP port check only* without an instance name. In this case, enter the instance name manually.



Step 8: Test Connection

Click **Test Connection** to verify the credentials and network connectivity.

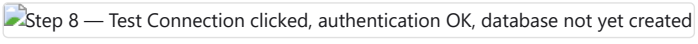
If the database does not exist yet, a notice appears:

> **Notice** Connected to SQL Server — authentication OK. Database 'T3000' does not exist yet. Click 'Init Schema' to create it.

This is expected for a first-time setup. Authentication succeeded — the database just needs to be initialized.

If connection fails, check:

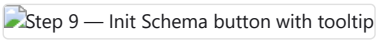
- SQL Server service is running
- TCP/IP is enabled in SQL Server Configuration Manager
- Windows Firewall allows the configured port (1433 or 1432)
- Username and password are correct



Step 9: Initialize the Schema

Click **Init Schema** to create all required T3000 tables (inputs, outputs, variables, trend logs, etc.) on the server database.

> Init Schema is safe to run multiple times. It will not delete existing data.

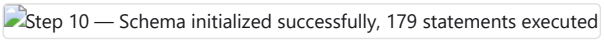


Step 10: Schema Initialization Complete

A success banner confirms:

> **Success** Schema initialized — 179 statements executed.

All T3000 tables have been created on the SQL Server database. The server is now ready to receive data.

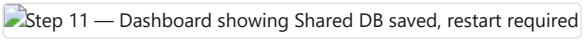


Step 11: Restart T3000 to Activate

Return to the Dashboard. The **Shared DB** card now shows:

> *Shared DB (Server) saved — restart T3000 to activate.* **Database Configuration**

Close and reopen T3000 to apply the new configuration.



Step 12: Verify — Shared DB Active After Restart

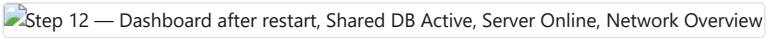
After restarting T3000, the Dashboard shows **Shared DB** as **Active** with badges:

- **Server** — this PC is the server role
- **SQL Server** — connected to Microsoft SQL Server
- **Connected** — connection is live

The **Network Overview** section shows:

- **Server (This PC)** — Online, with the machine hostname and IP address
- *No client PCs connected yet. Clients appear once they send heartbeats.* — this will populate as Client PCs connect

T3000 Field Devices shows all devices in the building.



Step 13: Confirm Sync & Database Health

Scroll down on the Dashboard to see:

System Overview shows:

- **Shared DB** — Connected
- **SQL Server · Server** — active backend and role
- **DB Size** — the current size of the SQL Server database (e.g. 16.0 MB at `D:\Program Files\Microsoft\...`)

Sync & Database Health shows:

- **Connected** badge, SQL Server, Direct write mode
- **Center DB Target** — `SQL Server · 192.168.x.x / T3000`
- Last Sync, Devices Today, Records Today — populate as sync cycles run

Trend Logs — Last 24 Hours shows trend data from all tracked points once devices begin polling:

- **Tracked** — number of tracked data points
- **Sampling Active** — points actively recording in the last 2 hours
- **Records (24h)** — total records written in the last 24 hours
- **Last Sample** — timestamp of the most recent data point



Troubleshooting

If you run into issues, first check that SQL Server Express is correctly installed and configured — most connection problems start there. See the **SQL Server Express Setup guide** for the full installation and configuration walkthrough, including how to:

- Enable TCP/IP in SQL Server Configuration Manager
- Set a static port (1433 or 1432)
- Create a SQL login with the right permissions
- Open the correct port in Windows Firewall
- Start the SQL Server Browser service

Symptom	Likely Cause	Fix
Connection timeout	SQL Server service stopped or unreachable	Start the SQL Server service; verify network connectivity and firewall rules
Login failed	Wrong username or password	Check SQL login credentials; confirm **SQL Server Authentication** is enabled (not Windows-only)
Cannot connect remotely	Windows Firewall blocking the port	Allow TCP 1432 or 1433 inbound — see SQL Server Express Setup
Database does not exist	First-time setup, database not yet created	Run **Init Schema** after Test Connection shows "authentication OK"
Schema init error	Insufficient permissions on the SQL login	Ensure the SQL login has `CREATE DATABASE` and `CREATE TABLE` rights
Scan LAN finds no instances	SQL Server Browser service not running	Start the **SQL Server Browser** service on the target machine
Instance shows "TCP port check only"	SQL Browser not running, instance name unknown	Start SQL Server Browser, or enter the instance name manually in the connection fields
Dashboard still shows Standalone after save	T3000 not restarted after saving	Close and reopen T3000 to apply the new configuration
T3000 feature missing or not working	Running an older version of T3000	Click **Help — Check for Updates** and install the latest version

---

Summary

The following table shows the complete setup flow at a glance:

Step	Action	Result
1	Open T3000, click **Help — Check for Updates**	T3000 is on the latest version
2	Open Dashboard in browser (`localhost:9103` or `<this-PC-IP>:9103`)	Dashboard visible
3	Click **Shared DB** card — **Connect to Shared DB**	Opens Database Configuration page
4	Review Database Configuration page	Currently in Standalone mode
5	Enable **Server Database**, select **Server** role, select **Microsoft SQL Server**	Server DB enabled
6	Fill in Host / IP, Port, Instance Name, Database, Username, Password	Connection fields ready
7	Click **Scan LAN** — click **Use** next to target instance — click **Save Configuration**	Configuration saved
8	Click **Test Connection**	Authentication OK (database may not exist yet — that's normal)
9	Click **Init Schema**	Schema creation starts
10	Wait for success banner: *179 statements executed*	All T3000 tables created on SQL Server
11	Return to Dashboard, see restart prompt — close and reopen T3000	T3000 restarts with new config
12	Dashboard shows **Shared DB · Active · Connected**	Server is live and receiving data
13	Check **Sync & Database Health** and **Trend Logs** sections	Data flowing into SQL Server

> For Client PC setup, repeat Steps 1-13 on each client PC, selecting **Client** role in Step 5. No Init Schema is needed on client PCs.

Tstat LCD System — Documentation Index

This folder contains the complete technical documentation for the Tstat10/Tstat11 LCD UI system: a visual drag-and-drop screen designer that outputs JSON, rendered identically on browser, device, and phone.

Design Approach

Start with a blank 320×480 canvas. Drag widgets from a toolbox (label, text, input, dropdown, icon, divider, etc.) onto the canvas. Snap to the 17×10 character grid. Adjust position, font, alignment, and data bindings. Export as JSON. Sync to device.

Documents

Document	Description
System Overview	End-to-end architecture, data flow, and integration points
LCD Screen Designer	**Core document** — blank canvas, widget toolbox, grid snap, properties panel, drag-and-drop design flow
Widget Types	LVGL-style widget type reference (label, text, input, dropdown, icon, etc.)
JSON Schema	The `screenDefinition.json` format specification
Implementation Status	What's built, what's pending, build order

Key Specs

- **LCD Resolution:** 320 × 480 px (fixed)
- **Character Grid:** 17 columns × 10 rows
- **Font:** Bold monospace (Fira Mono / Consolas)
- **Background:** Temco Blue (#003366)
- **Devices:** Tstat10 (white unit), Tstat11 (32M dev board)
- **Transport:** JSON → Modbus register writes (no firmware flash)

Related Code

- Simulator: `src/t3-react/features/tstat10-simulator/`
- HVAC Draw Tool: `src/lib/t3-hvac/`
- Original prototype: `D:\1025\github\temcocontrols\Tstat10_Simulator\`

LCD Screen Designer

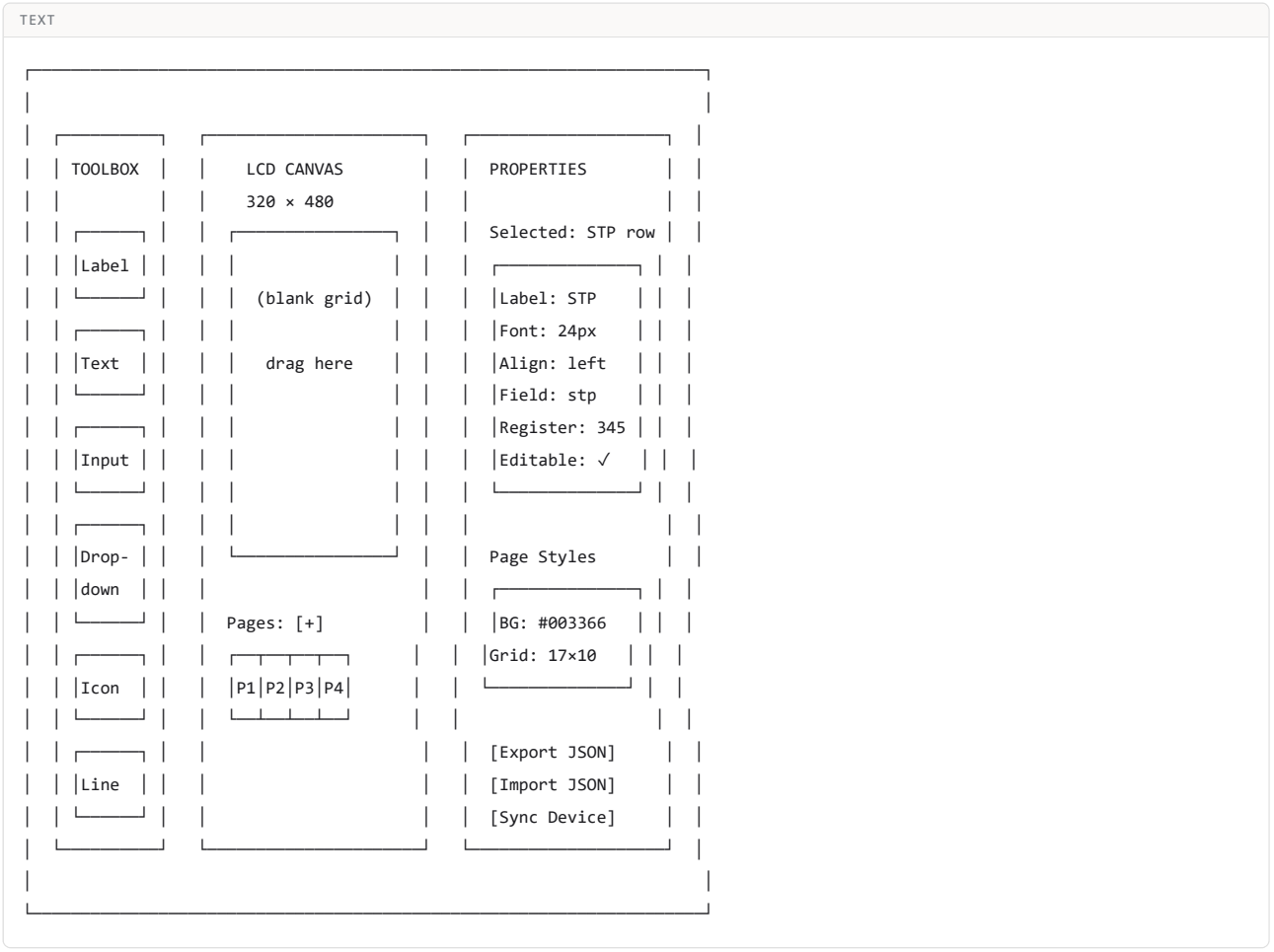
Visual drag-and-drop editor for creating LCD screen layouts from a blank canvas.

Core Concept

The designer starts with an **empty 320×480 canvas** — a blank LCD screen. A **Widget Toolbox** panel provides categorized widget types that the designer drags onto the canvas. Widgets snap to the 17×10 character grid. The designer adjusts each widget's position, text, font size, alignment, and data bindings. Multiple pages can be created. The final output is exported as `screenDefinition.json` and synced to the device.

This is the primary design tool — not a secondary "edit" mode on top of existing pages. The hardware screens shown in photos (Main, Setpoint, Inside, Comm Settings) are simply **example layouts** that were created using this designer.

TEXT



3-Panel Layout

Panel	Position	Purpose
Widget Toolbox	Left	Categorized list of draggable widget types
LCD Canvas	Center	The 320×480 design surface with grid overlay
Properties Panel	Right	Edit properties of selected widget + page styles

Widget Toolbox

The toolbox provides all available widget types organized by category:

Display Widgets

Widget	Icon	Description	Drag Behavior
Label	`Aa`	Static text label	Drops at grid position, default "Label"
Text	`T`	Data-bound text (shows register value)	Drops at grid position, pick data field
Large Text	`T+`	Big number display (temperature, etc.)	Spans multiple rows, centered
Header	`H`	Page title text	Drops at top, centered, larger font

Input Widgets

Widget	Icon	Description	Drag Behavior
Input Row	`[=]`	Label + editable value pair	Full-width row, label left + value right
Dropdown Row	`[▼]`	Label + value with dropdown options	Full-width row, configure options list
Edit Value	`[#]`	Standalone editable number in a box	Centered, large font, with +/-

Decorative Widgets

Widget	Icon	Description	Drag Behavior
Icon	`☆`	Single icon (house, wifi, fan, etc.)	Pick from icon palette
Icon Bar	`☆☆☆`	Row of navigation icons	Full-width, pick icons
Divider	`–`	Horizontal separator line	Full-width at grid row
WiFi Icon	`≈`	WiFi signal indicator	Fixed top-right corner

Footer Widgets

Widget	Icon	Description	Drag Behavior
Hint Text	`...`	Button hint text ("+ Edit -")	Bottom area
Nav Text	`◄►`	Navigation hint ("< Back Next >")	Bottom area

Grid Snapping & Alignment

Character Grid

The canvas is divided into a **17 column × 10 row** character grid:

TEXT

```
   Col: 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
Row 0: ┌──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┐
Row 1: ├──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┤
Row 2: ├──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┼──┤
  ...
Row 9: └──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┘
```

- **Cell size:** ~18.8px wide × 48px tall
- All widgets snap to grid cell boundaries when placed
- Widgets can span multiple cells (colSpan, rowSpan)

Snap Behavior

Action	Behavior
Drag from toolbox	Widget follows cursor, shows ghost position snapped to nearest grid cell
Drop on canvas	Widget snaps to the grid cell under the cursor
Drag to reposition	Widget moves between grid cells, snapping on release
Resize	Widgets resize in grid-cell increments (colSpan/rowSpan)

Alignment Aids

Feature	Description
Grid overlay	Visible grid lines showing all 17×10 cells
Snap indicators	Blue highlight showing which cell the widget will snap to
Alignment guides	Lines appear when widget aligns with edges of other widgets
Center markers	Guides for horizontal/vertical centering
Overlap warning	Red highlight if dropping on an occupied cell

Properties Panel

When a widget is selected on the canvas, the Properties Panel shows its editable properties:

Common Properties (all widgets)

Property	Control	Description
Position	Row / Col inputs	Grid row and column (with nudge arrows)
Span	ColSpan / RowSpan	How many cells the widget covers
Font Size	Dropdown	12px, 14px, 18px, 24px, 36px, 48px, 72px
Font Weight	Toggle	Normal / Bold
Text Align	Buttons	Left / Center / Right
Color	Color picker	Text color
Delete	Button	Remove this widget

Label / Text Properties

Property	Control	Description
Text	Text input	The static text to display
Field	Dropdown	Data field to bind (for dynamic text)
Suffix	Text input	Unit text after value ("°C", "%")

Input Row / Dropdown Properties

Property	Control	Description
Label	Text input	Left-side label text
Field	Dropdown	Data field to bind
Register	Number input	Modbus register number
Editable	Checkbox	Can the user change this value at runtime
Options	Tag input	List of allowed values (for dropdown)
Min / Max	Number inputs	Range for numeric values
Step	Number input	Increment step (e.g., 0.5)

Icon Properties

Property	Control	Description
Icon	Icon picker palette	Choose from available icons
Size	Dropdown	24px, 28px, 32px, 48px
Opacity	Slider	0.0 to 1.0

Page-Level Properties (shown when no widget selected)

Property	Control	Description
Page ID	Text input	Unique identifier
Page Label	Text input	Display name
Background	Color picker	Page background color
Highlight	Color picker	Focused widget color
Default Font	Dropdown	Base font for all widgets

Design Flow

Creating a New Screen from Scratch

```text
1. Click [+ New Page] → blank 320×480 canvas appears
2. Grid overlay visible (17×10 cells)
3. Drag "Header" from toolbox → drop at row 1 → type "Communication\nSettings"
4. Drag "Input Row" → drop at row 4 → set label="Modbus", field="modbus", register=6
5. Drag "Input Row" → drop at row 5 → set label="Baud", field="baud", options=[9600,115200,...]
6. Drag "Input Row" → drop at row 6 → set label="Mode", field="mode", options=["MODBUS","BACNET"]
7. Drag "Hint Text" → drop at row 8 → type "+  Edit  -"
8. Drag "Nav Text" → drop at row 9 → type "< Back   Next >"
9. Adjust fonts, alignment, colors in Properties Panel
10. Click [Export JSON] → downloads screenDefinition.json
11. Click [Sync Device] → pushes to hardware
```

Recreating the Main Display from Photos

```
1. [+ New Page] → blank canvas
2. Drag "WiFi Icon" → auto-places at row 0, col 16
3. Drag "Large Text" → row 1, bind field="temp", suffix="°C", fontSize=72px
4. Drag "Divider" → row 3
5. Drag "Input Row" → row 3, label="STP", field="stp", editable=true
6. Drag "Input Row" → row 4, label="FAN", field="fan", options=["AUTO",-2,-1,0,1,2,3]
7. Drag "Input Row" → row 5, label="SYS", field="sys", options=["AUTO",0,1,2]
8. Drag "Text" → row 6, bind field="ip", fontSize=18px
9. Drag "Icon Bar" → row 9, pick icons: moon, house, snowflake, fan
10. Done — preview matches the hardware photo
```

Editing an Existing Layout

```
1. Click [Import JSON] → load existing screenDefinition.json
2. All pages appear with widgets rendered on canvas
3. Click any widget → selected (blue outline, handles appear)
4. Drag to reposition → snaps to new grid cell
5. Edit properties in right panel
6. Add new widgets from toolbox
7. Delete unwanted widgets (select → Delete or ×)
8. Export updated JSON
```

Page Management

Page Tabs

Below the canvas, page tabs show all pages:

```
┌──────┐ ┌──────────┐ ┌────────┐ ┌──────────────┐ ┌───┐
│ Main │ │ Setpoint │ │ Inside │ │ Comm Settings │ │ + │
└──────┘ └──────────┘ └────────┘ └──────────────┘ └───┘
  ↑ active
```

Action	Behavior
Click tab	Switch to that page
[+] button	Add new blank page
Right-click tab	Rename, duplicate, delete, reorder
Drag tab	Reorder pages (changes navigation order)

Templates (Optional)

For convenience, the designer can start from templates instead of a blank canvas:

Template	Pre-built Widgets
Blank	Empty canvas (default)
Main Display	Large temp + rows + icon bar
Settings Page	Header + input rows + footer
Info Display	Icons + read-only text

Templates are just pre-made JSON that gets loaded — the designer can modify everything.

Keyboard Shortcuts

Key	Action
Delete / Backspace	Delete selected widget
Ctrl+C	Copy selected widget
Ctrl+V	Paste widget
Ctrl+Z	Undo
Ctrl+Shift+Z	Redo
Ctrl+S	Export JSON
Arrow keys	Nudge selected widget by 1 grid cell
Ctrl+A	Select all widgets
Escape	Deselect

Data Model

TYPESCRIPT

```typescript
interface DesignerState {
  definition: ScreenDefinition;     // The full multi-page JSON
  activePageId: string;             // Currently viewed page
  selectedWidgetId: string | null;  // Currently selected widget for editing
  isDirty: boolean;                 // Unsaved changes
  clipboard: Widget | null;         // Copy/paste buffer
  history: ScreenDefinition[];      // Undo stack
  historyIndex: number;             // Current position in undo stack
  gridVisible: boolean;             // Show/hide grid overlay
  snapEnabled: boolean;             // Grid snap on/off
}
```

View Mode vs Design Mode

Aspect	Design Mode	View Mode
Purpose	Create & edit layouts	Test & preview the end-user experience
Canvas	Shows grid, snap guides, selection handles	Clean LCD rendering (no guides)
Click	Selects widget for editing	Simulates button press
Arrow keys	Nudge selected widget	Navigate pages / edit values
Toolbox	Visible (left panel)	Hidden (debug panel shows instead)
Properties	Visible (right panel)	Hidden (or shows register browser)
Who uses it	Engineer / Designer	Tester / End user

JSON Schema — screenDefinition.json

The complete specification for the multi-page LCD screen definition file.

Top-Level Structure

JSON

```
{
  "device": "tstat11",
  "version": "1.0.0",
  "lcdSize": { "width": 320, "height": 480 },
  "grid": { "cols": 17, "rows": 10 },
  "globalStyles": {
    "bg": "#003366",
    "fontFamily": "'Fira Mono', 'Consolas', 'Menlo', monospace",
    "fontSize": "24px",
    "fontWeight": "700",
    "textColor": "#ffffff"
  },
  "data": {
    "temp": 23.3,
    "stp": 15.5,
    "hum": 45,
    "fan": "AUTO",
    "sys": "AUTO",
    "modbus": 133,
    "baud": 115200,
    "mode": "MODBUS",
    "ip": "168.0.151"
  },
  "pages": [ ... ],
  "navigation": {
    "type": "linear",
    "order": ["main", "setpoint", "inside", "comm_settings"],
    "wrapAround": true
  }
}
```

Field Descriptions

Root Object

Field	Type	Required	Description
`device`	string	Yes	Device model identifier ("tstat10", "tstat11")
`version`	string	Yes	Schema version
`lcdSize`	object	Yes	LCD pixel dimensions
`lcdSize.width`	number	Yes	Always 320
`lcdSize.height`	number	Yes	Always 480
`grid`	object	Yes	Character grid dimensions
`grid.cols`	number	Yes	Number of columns (17)
`grid.rows`	number	Yes	Number of rows (10)
`globalStyles`	object	No	Default styles for all pages
`data`	object	Yes	Current device data values
`pages`	array	Yes	Array of page definitions
`navigation`	object	Yes	How pages are connected

`data` Object

Runtime values from the device. Each field can be referenced by widgets via the `field` property.

Field	Type	Description	Register
`temp`	number	Current temperature (°C)	Read-only sensor
`stp`	number	Setpoint temperature (°C)	Read/write
`hum`	number	Humidity (%)	Read-only sensor
`fan`	string/number	Fan mode	Read/write
`sys`	string/number	System mode	Read/write
`modbus`	number	Modbus address	Read/write
`baud`	number	Baud rate	Read/write
`mode`	string	Protocol (MODBUS/BACNET)	Read/write
`ip`	string	IP address	Read-only

`pages` Array

Each page object:

JSON

```
{
  "id": "main",
  "label": "Home",
  "icon": "house",
  "styles": {
    "bg": "#003366",
    "highlight": "#008080",
    "fontFamily": "'Fira Mono', monospace",
    "fontSize": "24px",
    "fontWeight": "700",
    "textWidthChars": 6,
    "valueBoxWidthChars": 8
  },
  "widgets": [ ... ]
}
```

Field	Type	Required	Description
`id`	string	Yes	Unique page identifier
`label`	string	Yes	Human-readable page name
`icon`	string	No	Icon for page in icon bar navigation
`styles`	object	No	Page-level style overrides (merges with globalStyles)
`styles.bg`	string	No	Background color
`styles.highlight`	string	No	Focused row highlight color
`styles.fontFamily`	string	No	Font family
`styles.fontSize`	string	No	Base font size
`styles.fontWeight`	string	No	Font weight
`styles.textWidthChars`	number	No	Label column width in characters
`styles.valueBoxWidthChars`	number	No	Value box width in characters
`widgets`	array	Yes	Array of widget objects on this page

`navigation` Object

JSON

```
{
  "type": "linear",
  "order": ["main", "setpoint", "inside", "comm_settings"],
  "wrapAround": true
}
```

Field	Type	Required	Description
`type`	string	Yes	"linear" (cycle through) or "tree" (hierarchical, future)
`order`	array	Yes	Page IDs in navigation order
`wrapAround`	boolean	No	Wrap from last to first page? Default true

Complete Example

JSON


```
{
  "device": "tstat11",
  "version": "1.0.0",
  "lcdSize": { "width": 320, "height": 480 },
  "grid": { "cols": 17, "rows": 10 },
  "globalStyles": {
    "bg": "#003366",
    "fontFamily": "'Fira Mono', 'Consolas', 'Menlo', monospace",
    "fontSize": "24px",
    "fontWeight": "700",
    "textColor": "#ffffff"
  },
  "data": {
    "temp": 23.3,
    "stp": 15.5,
    "hum": 45,
    "fan": "AUTO",
    "sys": "AUTO",
    "modbus": 133,
    "baud": 115200,
    "mode": "MODBUS",
    "ip": "168.0.151"
  },
  "pages": [
    {
      "id": "main",
      "label": "Home",
      "icon": "house",
      "widgets": [
        { "type": "wifi_icon", "row": 0, "col": 16 },
        { "type": "large_text", "field": "temp", "row": 1, "colSpan": 17, "rowSpan": 2, "fontSize": "72px", "suffix": "°C" },
        { "type": "divider", "row": 3 },
        { "type": "label_value", "id": "stp", "label": "STP", "field": "stp", "row": 3, "editable": true, "suffix": "°C" },
        { "type": "label_value", "id": "fan", "label": "FAN", "field": "fan", "row": 4, "editable": true, "options": ["AUTO", -2
        { "type": "label_value", "id": "sys", "label": "SYS", "field": "sys", "row": 5, "editable": true, "options": ["AUTO", 0,
        { "type": "text", "field": "ip", "row": 6, "fontSize": "18px" },
        { "type": "icon_bar", "row": 9, "icons": [
          { "icon": "moon" },
          { "icon": "house", "pageId": "main" },
          { "icon": "snowflake", "pageId": "inside" },
          { "icon": "fan" }
        ]}
      ]
    },
    {
      "id": "setpoint",
      "label": "Setpoint",
      "widgets": [
        { "type": "wifi_icon", "row": 0, "col": 16 },
        { "type": "header", "text": "Setpoint", "row": 1 },
        { "type": "edit_value", "field": "stp", "row": 2, "fontSize": "36px", "editable": true, "minValue": 5.0, "maxValue": 35.
        { "type": "icon", "icon": "house", "row": 4, "col": 1 },
        { "type": "text", "text": "Inside", "row": 4, "col": 8 },
        { "type": "large_text", "field": "temp", "row": 5, "rowSpan": 2, "fontSize": "48px", "suffix": "°C" },
        { "type": "text", "text": "HUM", "row": 8, "col": 0 },
        { "type": "text", "field": "hum", "row": 8, "col": 5, "suffix": "%" }
      ]
```

```
    },
    {
      "id": "inside",
      "label": "Inside",
      "icon": "house",
      "widgets": [
        { "type": "wifi_icon", "row": 0, "col": 16 },
        { "type": "icon", "icon": "house", "row": 1, "col": 1 },
        { "type": "text", "text": "Inside", "row": 1, "col": 8 },
        { "type": "large_text", "field": "temp", "row": 3, "colSpan": 17, "rowSpan": 2, "fontSize": "72px", "suffix": "°C" },
        { "type": "text", "text": "HUM", "row": 7, "col": 0 },
        { "type": "text", "field": "hum", "row": 7, "col": 5, "suffix": "%" },
        { "type": "icon", "icon": "snowflake", "row": 7, "col": 14, "size": "28px" },
        { "type": "icon", "icon": "fan", "row": 8, "col": 14, "size": "28px" }
      ]
    },
    {
      "id": "comm_settings",
      "label": "Comm Settings",
      "styles": {
        "highlight": "#008080",
        "textWidthChars": 6,
        "valueBoxWidthChars": 8
      },
      "widgets": [
        { "type": "header", "text": "Communication\nSettings", "row": 1 },
        { "type": "label_value", "id": "modbus", "label": "Modbus", "field": "modbus", "register": 6, "row": 4, "editable": true
        { "type": "label_value", "id": "baud", "label": "Baud", "field": "baud", "register": 7, "row": 5, "editable": true, "opt
        { "type": "label_value", "id": "mode", "label": "Mode", "field": "mode", "register": 8, "row": 6, "editable": true, "opt
        { "type": "footer_hint", "text": "+  Edit  -", "row": 8 },
        { "type": "footer_nav", "text": "< Back   Next >", "row": 9 }
      ]
    }
  ],
  "navigation": {
    "type": "linear",
    "order": ["main", "setpoint", "inside", "comm_settings"],
    "wrapAround": true
  }
}
```

Validation Rules

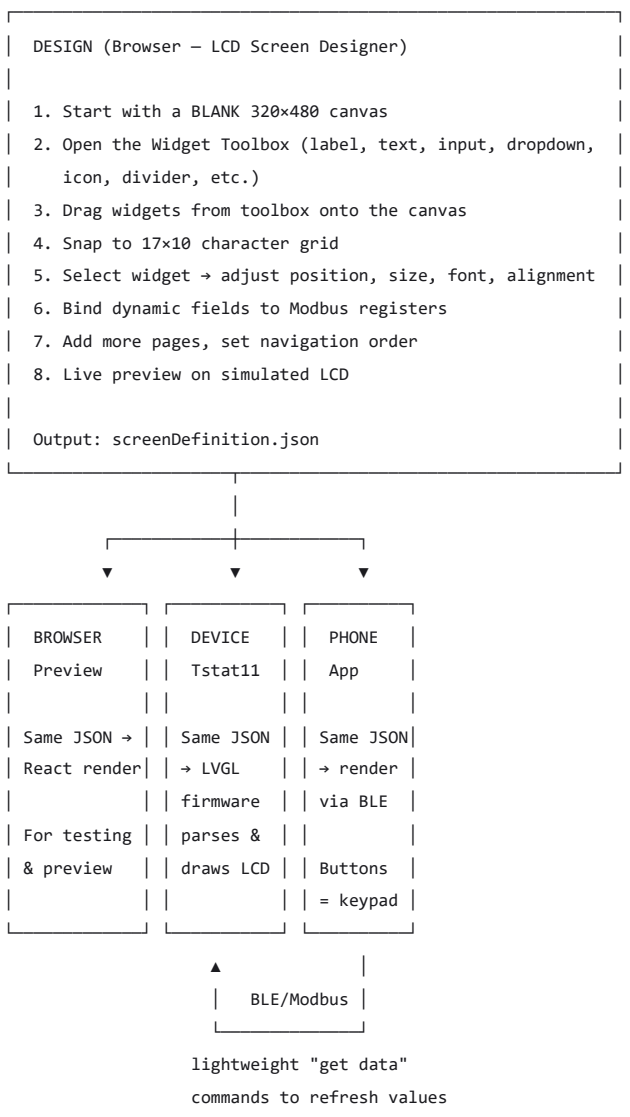
1. Every page `id` must be unique
2. Every page `id` referenced in `navigation.order` must exist in `pages`
3. Widget `field` values must reference keys in the `data` object
4. Widget `register` values must be valid Modbus register numbers
5. `options` arrays with all numbers should be sorted ascending
6. `row` values must be 0 to `grid.rows - 1`
7. `col` values must be 0 to `grid.cols - 1`

System Overview

End-to-End Architecture

The Tstat LCD system uses a **single JSON definition** to drive LCD rendering across three platforms: browser simulator, physical device, and phone app.

TEXT



Core Principles

1. Blank Canvas + Toolbox Design

The LCD designer starts with an **empty 320×480 canvas** (not pre-built templates). The designer drags widgets from a toolbox onto the canvas, snaps them to the character grid, and adjusts their properties. This is similar to how form designers or LVGL GUI builders work.

From the original design spec (lcd-menu.md):

> "The only HTML should be the empty #tstat-lcd container. Everything else must be generated by JavaScript from the JSON description."

The existing hardware screens (Main, Setpoint, Inside, Comm Settings) are just **example layouts** that were designed this way. Any custom layout can be created from scratch.

2. No Firmware Flash for UI Changes

> "User-defined menu systems will be converted into JSON strings (or a compressed byte-version). These strings will be chunked and sent to the Tstat10 via Modbus Register Writes. The Tstat10 firmware will receive the text, parse the LVGL instructions, and rebuild the local screen menu without a firmware flash."

3. One Definition, Three Renderers

The same `screenDefinition.json` is consumed by:

- **Browser simulator** — React components render the JSON
- **Device firmware** — ESP32 parses JSON, draws via LVGL on 320×480 LCD
- **Phone app** — Renders from local resources, sends lightweight commands via BLE

4. Phone App Knows All Temco Devices

The phone app ships with built-in knowledge of all Temco device UIs. It only needs to send lightweight "get data" commands to refresh values. The full UI structure is rendered locally from the JSON definition.

Integration Points

A. T3000 Desktop (Windows)

- T3000 hosts the simulator via **WebView2 (Chromium)**
- Data flow: T3000 polls device over BACnet/Modbus → pushes updates via `ExecuteScriptAsync()` → calls `window.updateUI(json)`

B. T3000 Web / T3000Webview

- Simulator runs as a native web component or iframe
- Compatible with the broader T3000Webview dashboard

C. Device Firmware (ESP32)

- Fixed coordinate rendering at 320×480
- No LVGL library used — firmware draws directly to frame buffer
- JSON parsed at runtime to position text/values

D. Phone App (BLE)

- Auto-connects to strongest RSSI Temco device
- If out of range, shows nearby top 5 devices
- Returns to original device when back in range
- Renders LCD UI locally from JSON definition
- Button presses sent as keypad commands via BLE

Hardware Specs

Spec	Value
LCD Resolution	320 × 480 px
Character Grid	17 columns × 10 rows
Font	Bold monospace
Background	#003366 (Temco Blue)
Highlight	#008080 (Teal)
Buttons	4 physical: ◄ ▼ ▲ ►
BLE Support	32M devices only (high ROM/RAM usage)

Data Flow Summary

TEXT

```
Design Time:
  Blank Canvas → Drag widgets from Toolbox → Snap to grid →
  Adjust properties → Preview → Export screenDefinition.json

Runtime (Device):
  screenDefinition.json → Modbus register writes → ESP32 firmware → LCD

Runtime (Phone):
  BLE auto-connect → "get data" command → device returns values
  Phone renders UI from local JSON + fresh values

Runtime (Browser/T3000):
  T3000 polls device → updateUI(json) → WebView2 renders
```

Widget Types

All widget types available for LCD page definitions. Each maps to an LVGL equivalent on the device firmware.

Widget Type Reference

`large_text` — Big Number Display

Used for prominent temperature readings. Spans multiple grid rows.

JSON

```
{
  "type": "large_text",
  "field": "temp",
  "row": 1,
  "col": 0,
  "colSpan": 17,
  "rowSpan": 3,
  "fontSize": "72px",
  "suffix": "°C",
  "align": "center"
}
```

Property	Type	Required	Description
`field`	string	Yes	Data field to display (e.g., `temp`, `stp`)
`row`	number	Yes	Grid row (0-based)
`col`	number	No	Grid column (0-based), default 0
`colSpan`	number	No	Number of columns to span, default 17 (full width)
`rowSpan`	number	No	Number of rows to span, default 2
`fontSize`	string	No	CSS font size, default "72px"
`suffix`	string	No	Text after value (e.g., "°C", "%")
`align`	string	No	"center", "left", "right"

LVGL equivalent: `lv_label` with large font

`label_value` — Label + Value Row

The most common widget. Shows a label on the left and a value in a rounded box on the right. Used for editable settings rows.

JSON

```
{
  "type": "label_value",
  "id": "stp",
  "label": "STP",
  "field": "stp",
  "register": 345,
  "row": 3,
  "editable": true,
  "options": [10, 15, 20, 25, 30],
  "maxValue": 35,
  "minValue": 5,
  "suffix": "°C"
}
```

Property	Type	Required	Description
`id`	string	Yes	Unique widget identifier
`label`	string	Yes	Left-side text label
`field`	string	Yes	Data field for the value
`register`	number	No	Modbus register number for sync
`row`	number	Yes	Grid row
`editable`	boolean	No	Can user change this value? Default false
`options`	array	No	Allowed values (cycle through with ▲/▼)
`maxValue`	number	No	Max for numeric increment
`minValue`	number	No	Min for numeric increment
`suffix`	string	No	Unit suffix
`navigateTo`	string	No	Page ID to navigate to when selected

LVGL equivalent: `lv_obj` container with `lv_label` (label) + `lv_label` (value in box)

`edit_value` — Standalone Editable Value

A large editable number in a rounded box. Used on dedicated edit pages (e.g., Setpoint page).

JSON

```
{
  "type": "edit_value",
  "field": "stp",
  "register": 345,
  "row": 2,
  "fontSize": "36px",
  "editable": true,
  "minValue": 5.0,
  "maxValue": 35.0,
  "step": 0.5
}
```

Property	Type	Required	Description
`field`	string	Yes	Data field
`register`	number	No	Modbus register
`row`	number	Yes	Grid row
`fontSize`	string	No	Default "36px"
`editable`	boolean	No	Default true
`minValue`	number	No	Minimum allowed
`maxValue`	number	No	Maximum allowed
`step`	number	No	Increment step (e.g., 0.5 for temperature)

LVGL equivalent: `lv_label` with `lv_obj` border/background

`header` — Page Title

Large centered text at the top of a page.

JSON

```
{
  "type": "header",
  "text": "Communication\nSettings",
  "row": 1,
  "fontSize": "24px"
}
```

Property	Type	Required	Description
`text`	string	Yes	Title text (supports `\n` for line breaks)
`row`	number	No	Starting grid row, default 1
`fontSize`	string	No	Default "24px"

LVGL equivalent: `lv_label` centered

`text` — Static or Data-Bound Text

General-purpose text element. Can be static ("Inside") or data-bound (IP address).

JSON

```
{
  "type": "text",
  "text": "Inside",
  "row": 4,
  "col": 8,
  "fontSize": "18px"
}
```

Or data-bound:

JSON

```
{
  "type": "text",
  "field": "ip",
  "row": 6,
  "col": 0,
  "fontSize": "18px"
}
```

Property	Type	Required	Description
`text`	string	No	Static text (use this OR `field`, not both)
`field`	string	No	Data field for dynamic text
`row`	number	Yes	Grid row
`col`	number	No	Grid column, default 0
`fontSize`	string	No	Default matches page style
`suffix`	string	No	Appended to value (e.g., "%")
`color`	string	No	Text color override

LVGL equivalent: `lv_label`

`icon` — Single Icon

Displays a named icon at a grid position.

JSON

```
{
  "type": "icon",
  "icon": "house",
  "row": 4,
  "col": 1,
  "size": "32px"
}
```

Property	Type	Required	Description
`icon`	string	Yes	Icon name (see Icon Library below)
`row`	number	Yes	Grid row
`col`	number	Yes	Grid column
`size`	string	No	Icon size, default "32px"
`opacity`	number	No	0.0 to 1.0, for dimmed/active states

Available icons: `house`, `moon`, `snowflake`, `fan`, `wifi`, `sun`, `leaf`, `thermometer`

LVGL equivalent: `lv_img` or custom bitmap

`icon_bar` — Row of Navigation Icons

A horizontal row of icons at the bottom of the page, used for page navigation.

JSON

```
{
  "type": "icon_bar",
  "row": 9,
  "icons": [
    { "icon": "moon", "pageId": "schedule" },
    { "icon": "house", "pageId": "main" },
    { "icon": "snowflake", "pageId": "inside" },
    { "icon": "fan", "pageId": "fan_control" }
  ]
}
```

Property	Type	Required	Description
`row`	number	Yes	Grid row
`icons`	array	Yes	Array of icon objects
`icons[].icon`	string	Yes	Icon name
`icons[].pageId`	string	No	Page to navigate to when selected

LVGL equivalent: Row of `lv_img` buttons

`wifi_icon` — WiFi Signal Indicator

Special widget for WiFi signal strength, always top-right.

JSON

```
{
  "type": "wifi_icon",
  "row": 0,
  "col": 16
}
```

Property	Type	Required	Description
`row`	number	No	Default 0
`col`	number	No	Default 16 (far right)

LVGL equivalent: `lv_img` with signal strength variants

`divider` — Horizontal Line

A separator line spanning the full width.

JSON

```
{
  "type": "divider",
  "row": 3,
  "color": "#ffffff",
  "opacity": 0.3
}
```

Property	Type	Required	Description
`row`	number	Yes	Grid row to place the line
`color`	string	No	Line color, default "#ffffff"
`opacity`	number	No	Default 0.3

LVGL equivalent: `lv_line`

`footer_hint` — Hint Text

Small text at the bottom showing button hints.

JSON

```
{
  "type": "footer_hint",
  "text": "+  Edit  -",
  "row": 8
}
```

LVGL equivalent: `lv_label` at bottom area

`footer_nav` — Navigation Text

Navigation hints at the very bottom.

JSON

```
{
  "type": "footer_nav",
  "text": "< Back   Next >",
  "row": 9
}
```

LVGL equivalent: `lv_label` at bottom area

Widget Type Summary

Type	Purpose	Editable	LVGL
`large_text`	Big temperature/number display	No	`lv_label`
`label_value`	Settings row (label + value)	Yes	`lv_obj` + `lv_label` × 2
`edit_value`	Standalone editable number	Yes	`lv_label` + `lv_obj`
`header`	Page title	No	`lv_label`
`text`	Static or dynamic text	No	`lv_label`
`icon`	Single icon	No	`lv_img`
`icon_bar`	Navigation icon row	No	`lv_img` × N
`wifi_icon`	WiFi signal	No	`lv_img`
`divider`	Horizontal line	No	`lv_line`
`footer_hint`	Button hints	No	`lv_label`
`footer_nav`	Navigation hints	No	`lv_label`